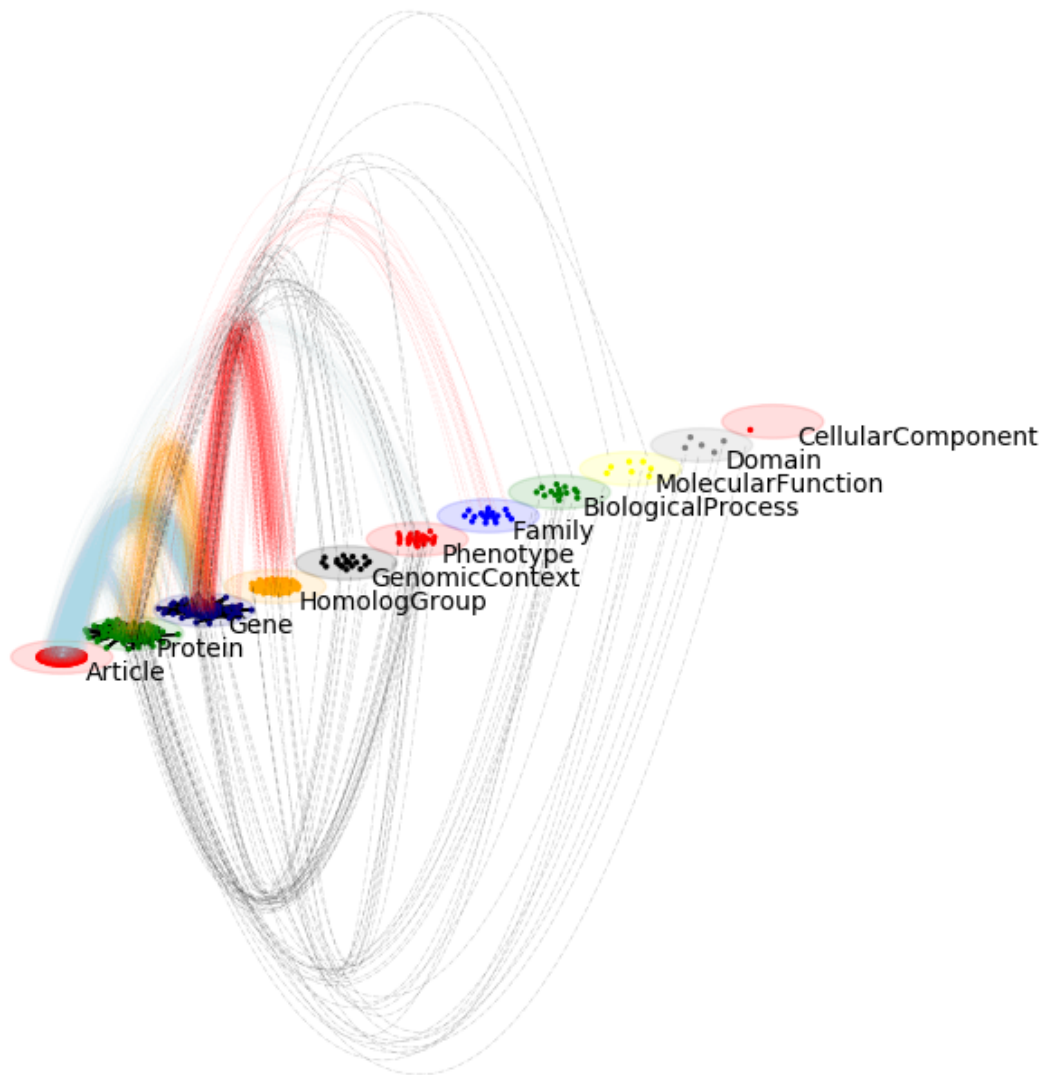




Py3plex: Multilayer Network Analysis in Python

Release 0.95a

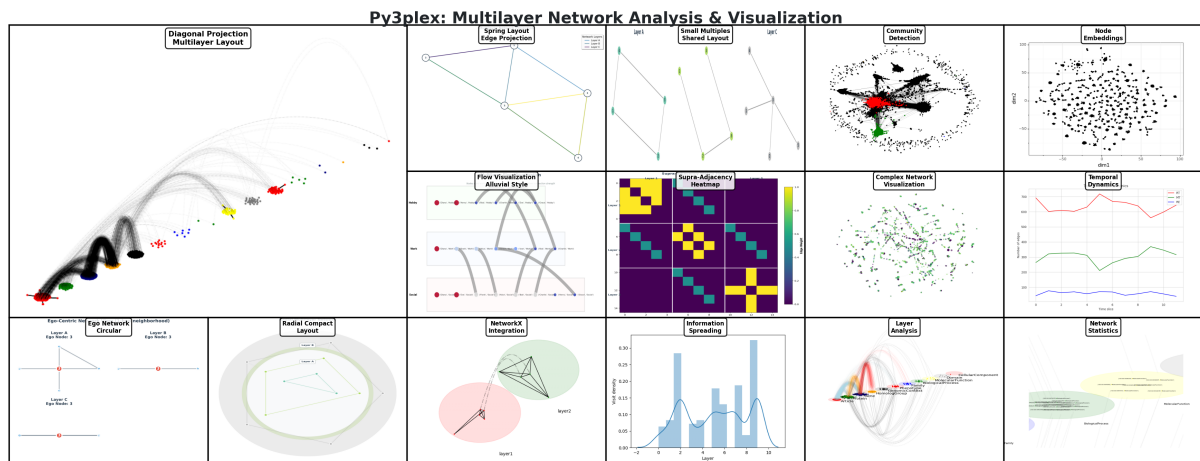
Blaž Škrlj

Dec 03, 2025

GETTING STARTED

1	Welcome to py3plex	3
2	Overview	5
2.1	What is py3plex Actually For?	5
2.2	Real-World Impact	5
2.3	What Can I Do in the First Hour, First Day, and First Week?	6
2.4	When Should I Use py3plex Instead of Just NetworkX?	6
2.5	Typical Workflows	7
2.6	Design at a Glance	7
3	Installation	9
3.1	Install from GitHub	9
3.2	Docker Installation (Alternative)	9
3.3	Install from source for development	9
3.4	Optional Dependencies	9
3.5	Requirements	10
4	Quick Start	11
5	Why py3plex?	13
5.1	vs. NetworkX	13
5.2	vs. Other Multilayer Tools	13
6	Use Cases	15
7	What's in this Documentation?	17
8	Documentation Contents	19
8.1	Part I: Getting Started	19
8.2	Part II: Concepts & Architecture	48
8.3	Part III: User Guide	95
8.4	Part IV: Environments & Deployment	240
8.5	Part V: Py3plex GUI	264
8.6	Part VI: Developer & Contributor Docs	316
8.7	Part VII: Examples	344
8.8	Part VIII: Reference & Citation	350
9	Examples	551
10	Citation	553
11	Indices and Tables	555
	Python Module Index	557
	Index	559

“Complexity is the enemy of execution.” — Tony Robbins



A practical, hands-on guide to analyzing and visualizing multilayer networks in Python.

Welcome to py3plex. This documentation is organized as a progressive learning journey—from your first five-minute example through conceptual foundations to advanced production workflows. Whether you’re a researcher exploring social network dynamics, a bioinformatician analyzing protein interactions across multiple evidence types, or a data scientist building features for machine learning, this guide will show you how to work effectively with multilayer network structures.

py3plex enables scalable analysis and visualization of multilayer and multiplex networks in Python, supporting complex network modeling across diverse scientific and applied domains.

WELCOME TO PY3PLEX

The world is connected. Social relationships span platforms and contexts. Transportation networks combine trains, buses, and flights. Biological systems involve protein interactions, gene regulation, and metabolic pathways. Yet most network analysis tools treat connections as if they were all the same.

py3plex is different. It's built from the ground up for networks where the *type* of connection matters—where a friendship is different from a professional contact, where an experimental protein interaction is different from a computationally predicted one, where a subway connection is different from a bus route.

This documentation will take you from zero to competent multilayer network analyst. Whether you're a researcher looking to apply network science to your domain, a data scientist exploring graph-based features for machine learning, or an engineer building tools that leverage network structure, you'll find what you need here.

OVERVIEW

2.1 What is py3plex Actually For?

py3plex is a lightweight Python library designed specifically for analyzing and visualizing **heterogeneous and multilayer networks**—network structures where relationships are richer and more complex than simple graphs. Unlike traditional network analysis tools that focus on homogeneous networks (single node and edge type), py3plex provides specialized capabilities for networks with:

- **Multiple node types** (e.g., authors, papers, venues in an academic network)
- **Multiple edge types** (e.g., friendship, mentorship, collaboration)
- **Multiple layers of interaction** (e.g., different social platforms, modes of transportation)
- **Temporal dynamics** (e.g., evolving relationships over time)
- **Heterogeneous attributes** (e.g., edge weights varying by relationship type)

Real-world systems are rarely simple: a social network involves friendships *and* professional relationships; a transportation system includes buses *and* trains *and* flights; a biological system has protein-protein interactions *and* gene regulation *and* metabolic pathways. When you model these systems as multilayer networks rather than flattening them into a single graph, you preserve information that matters for analysis—and py3plex makes this kind of analysis practical.

Target Users: Researchers in network science, computational biology, complex systems, social network analysis, infrastructure/transportation modeling, and applied ML on graphs.

2.2 Real-World Impact

Multilayer network analysis isn't just an academic exercise. Here are some ways it has been used in practice:

- **Drug discovery:** Researchers model protein interactions, genetic associations, and drug-target relationships as a multilayer network. Proteins that are central across multiple evidence layers are prioritized as drug targets.
- **Pandemic modeling:** Epidemiologists model human contact networks with layers for household, workplace, and community interactions. Each layer has different transmission dynamics, enabling more accurate predictions.
- **Infrastructure resilience:** Urban planners model transportation systems with layers for different modes (subway, bus, bike-share). Analyzing inter-layer dependencies reveals vulnerabilities—what happens when a major subway hub fails?
- **Social media analysis:** Researchers studying influence model users across platforms (Twitter, Instagram, YouTube). Cross-platform influencers behave differently from single-platform celebrities.
- **Recommendation systems:** E-commerce platforms model user-item-category relationships as heterogeneous networks. Meta-path-based features improve recommendation quality.

py3plex provides the tools to conduct these kinds of analyses—from simple exploration to publication-ready workflows.

2.3 What Can I Do in the First Hour, First Day, and First Week?

First Hour:

- Install py3plex and create your first multilayer network (see *Chapter 1: Your First Five Minutes*)
- Load a dataset, compute basic statistics, and visualize your network with one of the built-in layouts
- Query your network with the SQL-like DSL (`SELECT nodes WHERE layer="friends" AND degree > 5`)

First Day:

- Work through the 10-minute tutorial for a complete workflow (see *Chapter 2: The Complete Workflow in Ten Minutes*)
- Compute multilayer statistics: layer density, node activity, edge overlap between layers
- Detect communities using multilayer Louvain and visualize them with community colors
- Generate random walks and create basic node embeddings for ML tasks

First Week:

- Understand the core model: node-layer pairs, supra-adjacency matrices, and how py3plex extends NetworkX
- Implement complete analysis pipelines: load → compute statistics → detect communities → visualize → export
- Tune algorithm parameters (resolution, inter-layer coupling) for your specific domain
- Build production-ready workflows with Arrow/Parquet I/O for scalability

2.4 When Should I Use py3plex Instead of Just NetworkX?

NetworkX is excellent for single-layer, homogeneous networks. Use py3plex when:

1. **Your data has natural layers.** If you're tempted to add a `layer` attribute to NetworkX edges or to maintain multiple separate graphs, py3plex gives you a principled representation and layer-aware algorithms.
2. **You need multilayer-specific metrics.** Statistics like node activity (how many layers a node participates in), edge overlap (how many edges appear in multiple layers), or versatility centrality (node importance across layers) require multilayer-aware implementations.
3. **You want consistent cross-layer analysis.** Multilayer community detection finds groups that are consistent across layers. Multilayer centrality identifies nodes that are important in their context, not just locally.
4. **You need specialized visualization.** Diagonal projection and layered layouts show multilayer structure clearly. Hairball plots with layer-colored nodes reveal cross-layer patterns.
5. **You want a SQL-like query interface.** The py3plex DSL lets you query and filter networks intuitively (`SELECT edges WHERE source_layer="l1" AND weight > 0.5`).

If your network is genuinely a single layer with one node type and one edge type, plain NetworkX is sufficient. py3plex adds value when that simplifying assumption doesn't hold.

Key Features:

- Native support for multiplex and multilayer network structures
- **SQL-like DSL** for intuitive network queries and analysis
- Diagonal projection visualization for large multilayer networks
- Comprehensive multilayer centrality measures (17+ statistics)
- Community detection across network layers (Louvain, Infomap, multilayer modularity)
- Network decomposition and feature extraction for ML
- Random walk and node embedding algorithms (Node2Vec, DeepWalk)

- High-performance I/O with Arrow/Parquet for large-scale analysis
- Integration with NetworkX, igraph, and other graph libraries

2.5 Typical Workflows

py3plex supports three main workflow patterns:

1. Exploratory Multilayer Network Analysis (Medium-sized Datasets)

For networks with a few thousand nodes across 2–10 layers, this is the most common workflow:

- **Load:** Read your multilayer edgelist, GraphML, or CSV
- **Explore:** Run `basic_stats()`, examine layer densities, node activity distributions
- **Analyze:** Compute multilayer statistics, detect communities, compute centralities
- **Visualize:** Create hairball plots with community colors, diagonal projections
- **Iterate:** Refine layer selection, tune parameters, re-analyze

Best for: Social network analysis, biological networks (PPI, gene regulation), citation networks

2. Feature Extraction + Embeddings for ML Tasks

When your goal is to use network structure as features for machine learning:

- **Load:** Import your network
- **Generate walks:** Use `generate_walks()` with Node2Vec-style biased sampling
- **Train embeddings:** Feed walks to Word2Vec or use py3plex's embedding wrappers
- **Export features:** Create node-level feature vectors for classification or link prediction
- **Integrate:** Use embeddings in scikit-learn pipelines or deep learning models

Best for: Node classification, link prediction, graph-level classification, similarity search

3. Large-Scale Multilayer Analysis (Performance Constraints)

For networks with 100k+ nodes or when running many experiments:

- **Use Arrow/Parquet I/O:** 2–3x faster read/write than JSON, better compression
- **Work layer-by-layer:** Extract single layers for independent analysis when possible
- **Use sparse matrices:** Always use `sparse=True` for supra-adjacency matrices
- **Sample for exploration:** Use subnetworks and random sampling for initial exploration
- **Batch processing:** Use CLI/Docker for reproducible, scriptable analysis

Best for: Large social graphs, infrastructure networks, repeated experiments, production pipelines

2.6 Design at a Glance

py3plex is organized into four main components:

Core (`py3plex.core`):

The `multi_layer_network` class wraps NetworkX `MultiGraph`/`MultiDiGraph` with layer-aware semantics. Nodes are represented as `(node_id, layer_id)` tuples, enabling the same entity to appear in multiple layers. This representation is compatible with all NetworkX algorithms while providing multilayer-specific operations.

Algorithms (`py3plex.algorithms`):

- **Statistics:** Layer density, node activity, edge overlap, versatility centrality, and more
- **Community detection:** Louvain, Infomap, multilayer modularity (Mucha et al. 2010)

- **Centrality:** Multilayer degree, betweenness, closeness, PageRank
- **Random walks:** Basic walks, Node2Vec-style biased walks, layer-constrained walks

Visualization (``py3plex.visualization``):

- **Hairball plots:** Force-directed layouts with layer coloring
- **Diagonal projection:** 3D-like representation showing layer structure
- **Community visualization:** Nodes colored by detected community

I/O (``py3plex.io``):

- **Traditional formats:** Edgelist, multiedgelist, GraphML, GML, pickle
- **Modern formats:** Apache Arrow, Parquet for high-performance serialization
- **Schema-based API:** `read()` and `write()` with automatic format detection

INSTALLATION

3.1 Install from GitHub

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

3.2 Docker Installation (Alternative)

py3plex is also available as a Docker container with all dependencies pre-installed:

```
# Clone and build
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
docker build -t py3plex:latest .

# Run commands
docker run --rm py3plex:latest --version
docker run --rm py3plex:latest selftest
```

See *Docker Usage Guide* for complete Docker documentation.

3.3 Install from source for development

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e .
```

3.4 Optional Dependencies

```
# Advanced community detection with Infomap
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]

# Additional algorithms (Louvain, cdlib)
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[algos]

# Advanced visualization (plotly, igraph)
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]

# For development (includes testing and linting tools)
pip install -e ".[dev]"
```

3.5 Requirements

- Python 3.8 or higher
- NetworkX, NumPy, SciPy (automatically installed)
- Optional: Matplotlib, Plotly for visualization

QUICK START

Create a simple multilayer network:

```
from py3plex.core import multinet

# Create a new multilayer network
network = multinet.multi_layer_network()

# Add edges within layers
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1],
    ['B', 'layer1', 'C', 'layer1', 1],
    ['A', 'layer2', 'B', 'layer2', 1],
], input_type="list")

# Display basic statistics
network.basic_stats()

# Visualize
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default([network], display=True)
```

Load from file and analyze:

```
# Load from edge list
network = multinet.multi_layer_network().load_network(
    "data.edgelist", input_type="edgelist", directed=False)

# Compute multilayer statistics
from py3plex.algorithms.statistics import multilayer_statistics as mls
density = mls.layer_density(network, 'layer1')
versatility = mls.versatility_centrality(network, centrality_type='degree')

# Community detection
from py3plex.algorithms.community_detection import community_wrapper
communities = community_wrapper.best_partition(network.core_network)
```

Query with DSL (SQL-like syntax):

```
from py3plex.dsl import execute_query

# Query nodes by layer and degree
result = execute_query(network, 'SELECT nodes WHERE layer="layer1" AND degree > 1')

# Compute centrality for filtered nodes
```

(continues on next page)

(continued from previous page)

```
result = execute_query(network, 'SELECT nodes WHERE layer="layer1" COMPUTE_↵
↵betweenness_centrality')
```

See *SQL-like DSL for Multilayer Networks* for comprehensive DSL documentation.

Pythonic interface - use networks naturally:

```
from py3plex.core.multinet import multi_layer_network

# Create from edges directly with factory method
network = multi_layer_network.from_edges([
    {'source': 'A', 'target': 'B', 'source_type': 'l1', 'target_type': 'l1'},
    {'source': 'B', 'target': 'C', 'source_type': 'l1', 'target_type': 'l1'}
])

# Use len(), bool, and 'in' naturally
if network: # True if network has nodes
    print(f"{len(network)} nodes across {network.layer_count} layers")

if ('A', 'l1') in network: # Check membership
    print("Node A exists in layer l1")

# Iterate directly
for node in network:
    print(node)

# Method chaining
network.add_nodes([{'source': 'D', 'type': 'l1'}]).add_edges([...])
```

See *Working with Networks* for complete Pythonic interface documentation.

WHY PY3PLEX?

5.1 vs. NetworkX

NetworkX excels at single-layer homogeneous networks but lacks native multilayer support. py3plex builds on NetworkX while adding:

- Native multilayer data structures
- Layer-aware algorithms
- Specialized multilayer visualizations
- Heterogeneous network decomposition

5.2 vs. Other Multilayer Tools

py3plex provides:

- Lightweight and easy to integrate
- Comprehensive statistical measures (17+ multilayer metrics)
- Publication-ready visualizations
- Active development and research backing

USE CASES

py3plex is particularly well-suited for:

- **Biological networks:** Protein-protein interactions with multiple evidence types, gene regulatory networks
- **Social networks:** Multi-platform analysis (Twitter + Facebook), relationship type analysis
- **Citation networks:** Author-paper-venue multilayer structures, knowledge graphs
- **Transportation:** Multi-modal networks (bus, train, air), urban mobility
- **Temporal networks:** Evolving social networks, dynamic community evolution

WHAT'S IN THIS DOCUMENTATION?

This documentation is organized as a comprehensive guide that takes you from beginner to advanced user. It's structured in eight parts that you can read sequentially like a book, or jump to specific sections based on your needs:

Part I - Getting Started: New to py3plex? Begin with the 5-minute quickstart (*Chapter 1: Your First Five Minutes*) for a hands-on introduction, then continue to the 10-minute tutorial (*Chapter 2: The Complete Workflow in Ten Minutes*) for a comprehensive walkthrough.

Part II - Concepts & Architecture: Want to understand the theory? Check out *Chapter 3: Multilayer Networks 101* for foundational concepts and *Chapter 4: The py3plex Core Model* for py3plex's design philosophy.

Part III - User Guide: Ready to dive deep? The user guide chapters cover every major feature from network creation to visualization, each building on the previous.

Part IV - Deployment: Moving to production? See the *Docker Usage Guide* and *Performance and Scalability Best Practices* guides for deployment strategies.

Part V - GUI: Prefer a visual interface? The GUI section covers the web-based interface for interactive network exploration.

Part VI - Developer Docs: Want to contribute? Read the *Development Guide* and *Contributing to py3plex* pages.

Part VII - Examples: Learning by doing? Browse the curated *Examples* for working code.

Part VIII - Reference: Need API details? The reference section provides comprehensive algorithm and API documentation.

DOCUMENTATION CONTENTS

8.1 Part I: Getting Started

Chapters 1–2: From zero to productive in 15 minutes.

Whether you're new to py3plex or multilayer network analysis, this section provides everything you need to get up and running quickly. We begin with a 5-minute quickstart that demonstrates the core concepts through hands-on examples, then expand into a comprehensive 10-minute tutorial covering the most common workflows. The installation guide ensures you have all dependencies configured correctly, while the troubleshooting section addresses common issues you might encounter.

8.1.1 Getting Started: Your Journey Begins

"The beginning is the most important part of the work." — Plato

Why Start Here?

You've decided to explore multilayer networks. Perhaps you're a biologist trying to understand how proteins interact through multiple mechanisms. Perhaps you're a social scientist studying how people connect across different platforms. Perhaps you're an engineer modeling transportation systems that span multiple modes. Whatever brought you here, you've recognized that real-world systems are rarely simple—they have structure, layers, and complexity that matter.

This section is designed to get you from zero to productive as quickly as possible. We believe that the best way to learn is by doing, so we've structured these guides as hands-on experiences rather than passive reading.

What You'll Find Here

The 5-Minute Quickstart is exactly what it sounds like: a rapid introduction that gets you to your first visualization and analysis in minutes. You'll create a network, compute some statistics, and see results. This is ideal if you want to understand what py3plex can do before committing more time.

The 10-Minute Tutorial is a deeper dive. It tells a story—you're a data scientist analyzing a multi-platform social network—and walks you through a complete analysis workflow. By the end, you'll understand the core mental model of multilayer networks and have working code for common tasks.

The Installation Guide covers the practical details: how to install py3plex, which optional dependencies you might want, and how to verify your installation is working.

Common Issues addresses the problems you might encounter. We've collected the most frequent questions and provided clear solutions.

A Note on Learning

Network analysis is a skill that develops with practice. The concepts in this section will make more sense after you've used them a few times. Don't worry if something seems abstract at first—return to these guides after working through some examples, and you'll find new layers of understanding.

If you're completely new to network analysis, we recommend reading through the conceptual material in Part II after completing these quickstart guides. Understanding *why* multilayer networks matter will help you make better modeling decisions in your own work.

Let's Begin

Ready? Start with *Chapter 1: Your First Five Minutes* for a rapid introduction, or dive into *Chapter 2: The Complete Workflow in Ten Minutes* for a complete workflow.

Note

Learning Path Recommendation:

- New to py3plex? → Start with the 5-Minute Quickstart
- New to multilayer networks? → Read the 10-Minute Tutorial, then Part II (Concepts)
- Experienced with networks, new to py3plex? → Skim the 5-Minute Quickstart, then jump to Part III (User Guide)

8.1.2 Chapter 1: Your First Five Minutes

In which we create a multilayer network from scratch, visualize it, and discover what makes multilayer analysis different from single-layer approaches.

Every journey into multilayer network analysis begins with a simple question: *What happens when relationships aren't all the same?*

Traditional network analysis treats every connection equally—a friend is a friend, a colleague is a colleague, and the tool you use doesn't care which is which. But real-world systems are messier. The same people might be connected in fundamentally different ways: as friends *and* as colleagues, as family members *and* as business partners, as collaborators on one project *and* competitors on another.

py3plex exists because these distinctions matter. This quickstart will show you how, in just five minutes.

The Scenario

Imagine you have data about a small group of people and their relationships across two contexts: **friends** (personal relationships) and **colleagues** (professional relationships). You want to understand:

- Who is well-connected in each context?
- Who appears in both contexts (and might bridge personal and professional networks)?
- Are the friendship and colleague networks similarly structured?

This scenario appears everywhere in the real world:

- A researcher studying work-life balance might ask: do people who mix their friend and colleague networks experience different outcomes?
- A consultant analyzing company culture might investigate: are “bridge” employees who connect different social circles more central to information flow?
- A social scientist might explore: does social support from friends vs. colleagues serve different psychological functions?

These questions require understanding the *multilayer* structure of relationships—not just who knows whom, but *how* they know each other. Let's solve this with py3plex.

Installation

Install py3plex from GitHub:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

If you prefer Docker, see *Installation and Setup* for container-based setup.

Hello World: Your First Multilayer Network

Create a simple multilayer network with just a few lines of code:

```
from py3plex.core import multinet

# Create a new multilayer network
network = multinet.multi_layer_network()

# Add edges (nodes are created automatically)
# Format: [source_node, source_layer, target_node, target_layer, weight]
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Bob', 'friends', 'Carol', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
    ['Bob', 'colleagues', 'Dave', 'colleagues', 1]
], input_type="list")

# Display basic statistics
network.basic_stats()
```

Output:

```
Number of nodes: 6
Number of edges: 4
Number of unique nodes (as node-layer tuples): 6
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'friends': 3 nodes
  Layer 'colleagues': 3 nodes
```

What just happened? We created a multilayer network with 2 layers. Notice that `basic_stats()` reports both **6 nodes** (counting each person in each layer separately) and **4 unique node IDs** (counting each person once regardless of how many layers they appear in). This distinction is fundamental: Alice appears in both layers, but Alice-in-friends and Alice-in-colleagues are different **node-layer pairs**.

Sanity check: The “Nodes per layer” section confirms that each layer has 3 nodes. If you see layers with 0 or 1 nodes, you likely have a data loading issue. If the unique node count seems wrong, check your layer naming for consistency.

Visualize Your Network

Create a simple visualization:

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Simple visualization
draw_multilayer_default([network], display=True)
```

This creates a visual plot showing your multilayer network with nodes colored by layer.

What you should see: Nodes from the “friends” layer will be one color, nodes from the “colleagues” layer another. Edges connect nodes within and across layers. This immediate visual feedback helps you verify that your data was

loaded correctly—if layers are missing or edges seem wrong, you’ll often spot it in the visualization before you spot it in statistics.

Basic Analysis

Compute some basic network statistics:

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# How dense is each layer?
friends_density = mls.layer_density(network, 'friends')
colleagues_density = mls.layer_density(network, 'colleagues')

print(f"Friends layer density: {friends_density:.3f}")
print(f"Colleagues layer density: {colleagues_density:.3f}")

# How active is Bob across layers?
bob_activity = mls.node_activity(network, 'Bob')
print(f"Bob's activity: {bob_activity:.3f}")
```

Output:

```
Friends layer density: 0.667
Colleagues layer density: 0.667
Bob's activity: 1.000
```

Interpreting the results:

- **Density 0.667** means 2 out of 3 possible edges exist in each layer (with 3 nodes, a complete graph has 3 edges; we have 2). This is a fairly dense small network—real-world networks are typically much sparser (density < 0.1).
- **Bob’s activity 1.000** means Bob appears in 100% of layers (both of them). He’s a “bridge” person who connects both contexts. Compare this to Carol (activity 0.5, only in friends) or Dave (activity 0.5, only in colleagues). High-activity nodes are often interesting for understanding cross-layer dynamics.

Query with DSL (SQL-like)

Use SQL-like queries for intuitive network analysis:

```
from py3plex.dsl import execute_query

# Find nodes with high degree
result = execute_query(network, 'SELECT nodes WHERE degree > 1')
print(f"Found {result['count']} high-degree nodes")

# Get nodes in a specific layer
result = execute_query(network, 'SELECT nodes WHERE layer="friends"')
print(f"Friends layer has {result['count']} nodes")

# Compute centrality
result = execute_query(network,
    'SELECT nodes WHERE layer="friends" COMPUTE betweenness centrality')
```

What this gives you: The DSL is particularly useful when you’re exploring data interactively. Instead of chaining Python method calls, you can express queries in a readable, declarative style. The `COMPUTE` clause adds computed metrics to the result, making it easy to filter and analyze in one step.

See *SQL-like DSL for Multilayer Networks* for comprehensive DSL documentation.

What You Learned

Let's pause and take stock. In five minutes, you've learned the core mental model that distinguishes py3plex from traditional network tools:

1. **Node-layer pairs are fundamental.** In a multilayer network, a node like “Bob” isn't just a single entity—it's represented as ('Bob', 'friends') and ('Bob', 'colleagues') separately. This allows the same person to have different attributes and connections in different contexts. This representation isn't a limitation; it's what makes multilayer analysis possible.
2. **Layers preserve context.** Each layer represents a different type of relationship or interaction mode. Unlike tools that flatten everything into one graph, py3plex keeps layers distinct throughout your analysis. When you compute statistics or detect communities, the layer structure is always available.
3. **py3plex extends NetworkX.** Under the hood, py3plex uses NetworkX graphs. This means you can use any NetworkX algorithm on your multilayer network—computing PageRank, finding shortest paths, or any of hundreds of other algorithms—while py3plex adds multilayer-specific capabilities on top.
4. **Multilayer statistics reveal hidden patterns.** Metrics like node activity (who appears in multiple contexts?), layer density (how connected is each context?), and edge overlap (do the same relationships appear in both contexts?) help you understand how layers relate to each other. These insights are invisible to single-layer tools.

What's Next?

Congratulations! You've created your first multilayer network and taken the first steps toward understanding its structure.

In *Chapter 2: The Complete Workflow in Ten Minutes*, we'll expand on these foundations with a more comprehensive walkthrough: loading real data from files, computing centrality measures, detecting communities that span layers, generating random walks for embeddings, and creating publication-ready visualizations.

Ready to continue? Move on to *Chapter 2: The Complete Workflow in Ten Minutes* for the full 10-minute tutorial.

Need to install first? See *Installation and Setup* for complete setup instructions including optional dependencies.

Something not working? Check *Common Issues and Troubleshooting* for solutions to common problems.

Prefer to explore? Browse the `examples/` directory for 50+ working examples covering every major feature.

Next chapter: :doc:`tutorial\_10min` — where we build complete analysis pipelines from loading to visualization.

8.1.3 Chapter 2: The Complete Workflow in Ten Minutes

In which we build a complete multilayer network analysis pipeline—from loading data through community detection to visualization—and learn to interpret results for research.

In the previous chapter, you created a tiny multilayer network by hand and saw the basic building blocks. Now we'll tackle something more realistic: a complete analysis workflow that you could actually use in research.

By the end of this chapter, you'll have a template for any multilayer network analysis project. Whether you're studying social networks, biological interactions, or transportation systems, the workflow stays the same: load → explore → analyze → visualize → interpret.

The Scenario: A Multi-Platform Social Network

Imagine you're a data scientist at a research institute studying how people interact across different online platforms. You've been given a dataset of users interacting on two social platforms over time: a messaging app (layer 1) and a professional network (layer 2). Some users are active on both platforms, others on only one. Your goals are:

1. **Understand the structure** of each platform's network
2. **Identify who is central** in each context—and who bridges both

3. **Find communities** that span platforms
4. **Generate features** for a downstream classification task

This scenario captures the essence of multilayer network analysis: you have multiple “views” of the same entities (users), and you want to understand both the individual views and how they relate. Let’s work through this with py3plex.

What You Will Learn

In 10 minutes, you will learn how to:

1. Create and load multilayer networks from various sources
2. Navigate network structure with iteration and extraction
3. Compute multilayer statistics (and interpret what they mean)
4. Detect communities that span multiple layers
5. Perform random walks for generating node embeddings
6. Create visualizations for exploration and publication
7. Narrate your findings for a research paper

Prerequisites

Make sure py3plex is installed:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

Part 1: Building and Loading Networks

Time estimate: 3 minutes

1.1 Creating Networks from Scratch

Goal: Build a multilayer network programmatically to understand the data model.

Let’s start by creating a small network by hand. This helps you understand exactly what’s happening before we load larger data from files.

```
from py3plex.core import multinet

# Create a new multilayer network
network = multinet.multi_layer_network()

# Add edges within layers (this automatically creates nodes)
# Format: [source_node, source_layer, target_node, target_layer, weight]
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1],
    ['B', 'layer1', 'C', 'layer1', 1],
    ['A', 'layer2', 'B', 'layer2', 1],
    ['B', 'layer2', 'D', 'layer2', 1]
], input_type="list")

# Display basic statistics
network.basic_stats()
```

Output:

```

Number of nodes: 5
Number of edges: 4
Number of unique nodes (as node-layer tuples): 5
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'layer1': 3 nodes
  Layer 'layer2': 3 nodes

```

What to notice: We have 4 unique people (A, B, C, D) but 5 node-layer pairs because A and B appear in both layers. C only appears in layer1; D only in layer2. Node B is the most “active” (appears in both layers) and is connected in both contexts—this makes B a potential bridge between the two platforms.

1.2 Loading Networks from Files

Goal: Load real data from a file and verify it loaded correctly.

py3plex supports multiple input formats. Here is how to load from an edge list:

```

from py3plex.core import multinet

# Load from a multiedgelist file
# Format: source target layer
network = multinet.multi_layer_network().load_network(
    "datasets/multiedgelist.txt",
    input_type="multiedgelist",
    directed=False
)

# Check what we loaded
network.basic_stats()

```

Output:

```

Number of nodes: 5
Number of edges: 4
Number of unique nodes (as node-layer tuples): 5
Number of unique node IDs (across all layers): 5
Nodes per layer:
  Layer '1': 3 nodes
  Layer '2': 2 nodes

```

Supported formats:

- multiedgelist - source, target, layer
- edgelist - simple source, target pairs
- gpickle - NetworkX pickle format
- gml, graphml - standard graph formats

3. Exploring Network Structure (2 minutes)

Goal: Navigate the network to understand its components and extract relevant parts.

Iterate Through Nodes and Edges

```

# Loop through all nodes
print("Nodes:")
for node in network.get_nodes(data=True):

```

(continues on next page)

(continued from previous page)

```

print(node)

# Loop through all edges
print("\nEdges:")
for edge in network.get_edges(data=True):
    print(edge)

# Get neighbors of a specific node in a layer
node_of_interest = "1"
layer_id = "1"
neighbors = list(network.get_neighbors(node_of_interest, layer_id=layer_id))
print(f"\nNeighbors of {node_of_interest} in layer {layer_id}:", neighbors)

```

Output (first few items):

```

Nodes (first 5):
(('1', '1'), {'type': '1'})
(('2', '1'), {'type': '1'})
(('6', '2'), {'type': '2'})
(('3', '1'), {'type': '1'})
(('5', '2'), {'type': '2'})

Edges (first 5):
(('1', '1'), ('2', '1'), {'weight': '1'})
(('1', '1'), ('3', '1'), {'weight': '1'})
(('2', '1'), ('6', '2'), {'weight': '1'})
(('3', '1'), ('5', '2'), {'weight': '1'})

Neighbors of 1 in layer 1: [('2', '1'), ('3', '1')]

```

What to notice: Nodes are tuples like ('1', '1') meaning “node 1 in layer 1”. The `type` attribute records the layer. Edges between layers (like ('2', '1') to ('6', '2')) are **inter-layer edges**—these represent relationships that cross contexts, such as a professional connection forming between users who met on a personal platform.

Extract Subnetworks

```

# Extract a single layer
layer_1 = network.subnetwork(['1'], subset_by="layers")
print("Layer 1 nodes:", list(layer_1.get_nodes()))

# Extract specific nodes
node_subset = network.subnetwork(['1', '2'], subset_by="node_names")
print("Node subset:", list(node_subset.get_nodes()))

# Extract specific node-layer pairs
specific_pairs = network.subnetwork(
    [('1', '1'), ('2', '1')],
    subset_by="node_layer_names"
)
print("Specific pairs:", list(specific_pairs.get_nodes()))

```

Output:

```

Layer 1 nodes: 3 nodes
Node subset: 2 node-layer pairs
Specific pairs: [('2', '1'), ('1', '1')]

```

When to use each: Use `subset_by="layers"` when you want to analyze one platform independently. Use `subset_by="node_names"` to trace a specific user across all their platforms. Use `subset_by="node_layer_names"` for precise control over exactly which node-layer pairs to include.

4. Computing Network Metrics (2 minutes)

Goal: Measure node importance and understand who is central in each layer.

Basic Centrality Measures

```
# Get a single layer as NetworkX graph
layer_1 = network.subnetwork(['1'], subset_by="layers")

# Compute degree centrality using NetworkX wrapper
degree_centrality = layer_1.monoplex_nx_wrapper("degree_centrality")
print("Degree centrality:", degree_centrality)

# Compute betweenness centrality
betweenness = layer_1.monoplex_nx_wrapper("betweenness_centrality")
print("Betweenness centrality:", betweenness)
```

Output (top nodes shown):

```
Degree centrality (first 5):
('1', '1'): 1.0000
('2', '1'): 0.5000
('3', '1'): 0.5000

Betweenness centrality (first 5):
('1', '1'): 1.0000
('2', '1'): 0.0000
('3', '1'): 0.0000
```

Multilayer Centrality

For multilayer-specific centrality measures:

```
from py3plex.algorithms.multilayer_algorithms.centrality import MultilayerCentrality

# Initialize centrality calculator
calc = MultilayerCentrality(network)

# Compute multilayer degree centrality (considers node participation across layers)
ml_degree = calc.overlapping_degree_centrality(weighted=False)
print("Multilayer degree centrality:", ml_degree)

# Compute multilayer betweenness centrality
ml_betweenness = calc.multilayer_betweenness_centrality()
print("Multilayer betweenness centrality:", ml_betweenness)
```

Output (top nodes shown):

```
Multilayer degree centrality (first 5):
1: 2.0000
2: 1.0000
3: 1.0000
5: 0.0000
6: 0.0000
```

(continues on next page)

(continued from previous page)

Multilayer betweenness centrality (first 5):

```
( '1', '1'): 0.6667
( '2', '1'): 0.5000
( '3', '1'): 0.5000
( '6', '2'): 0.0000
( '5', '2'): 0.0000
```

Interpreting centrality: Node 1 has the highest multilayer degree (2.0), meaning it's the most connected when we sum across layers. The betweenness values tell us node 1 in layer 1 lies on 66.7% of shortest paths—it's a broker connecting others. Nodes with high centrality in *multiple* measures are often the most important to study further.

5. Multilayer Network Statistics (2 minutes)

Goal: Understand layer structure and how layers relate to each other.

py3plex provides many specialized statistics for analyzing multilayer networks. Here are the most commonly used:

Basic Layer Statistics

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Measure how densely connected each layer is
density_layer1 = mls.layer_density(network, 'layer1')
density_layer2 = mls.layer_density(network, 'layer2')
print(f"Layer 1 density: {density_layer1:.3f}")
print(f"Layer 2 density: {density_layer2:.3f}")

# Measure diversity across layers
entropy = mls.entropy_of_multiplexity(network)
print(f"Layer diversity (entropy): {entropy:.3f} bits")
```

Output:

```
Layer 1 density: 0.667
Layer 2 density: 0.000
Layer diversity (entropy): 0.000 bits
```

Interpreting density:

- **High density (> 0.3):** The layer is tightly connected. In social terms, “everyone knows everyone.”
- **Low density (< 0.1):** Sparse connections, typical of large real-world networks.
- **Density = 0:** No edges in this layer (check your data loading!).

Interpreting entropy: Low entropy means activity is concentrated in few layers; high entropy means it's spread evenly across many layers.

Node-Level Statistics

```
# Compute node activity (how many layers a node participates in)
activity = mls.node_activity(network, node='1')
print(f"Node activity for node 1: {activity:.3f}")
```

Output:

```
Node activity for node 1: 0.500
```


Interpreting activity: Node 1 appears in 50% of layers (1 out of 2). In our multi-platform scenario, this user is active on one platform. A user with activity 1.0 would be a “super-connector” present everywhere—often these are the most interesting nodes for understanding cross-platform dynamics.

Network-Level Statistics

```
# Measure how much layers differ in structure (edge overlap)
edge_overlap_value = mls.edge_overlap(network, 'layer1', 'layer2')
print(f"Edge overlap between layers: {edge_overlap_value:.3f}")

# Measure layer similarity using Jaccard index
jaccard = mls.layer_similarity(network, 'layer1', 'layer2', method='jaccard')
print(f"Jaccard similarity between layers: {jaccard:.3f}")
```

Output:

```
Edge overlap between layers: 0.000
Jaccard similarity between layers: 0.000
```

Interpreting overlap and similarity:

- **High overlap (> 0.5):** The same relationships exist in both layers—maybe the layers are redundant.
- **Low overlap (< 0.1):** Layers capture different relationships—this is where multilayer analysis adds value.
- **Jaccard = 0:** No common edges. The layers are completely complementary.

Available statistics:

See the `py3plex.algorithms.statistics.multilayer_statistics` module for all available statistics including:

- `layer_density` - Density of edges within a layer
- `entropy_of_multiplexity` - Diversity across layers
- `node_activity` - Number of layers a node participates in
- `edge_overlap` - Edge overlap between layers
- `layer_similarity` - Similarity between layers (Jaccard, Pearson, etc.)
- `algebraic_connectivity` - Connectivity measure from Laplacian
- `community_participation_coefficient` - How communities span layers
- `interdependence` - Layer interdependence measure
- `multilayer_clustering_coefficient` - Clustering in multilayer context
- `multiplex_betweenness centrality` - Betweenness in multiplex networks
- `multiplex_closeness centrality` - Closeness in multiplex networks
- And more...

6. Community Detection (2 minutes)

py3plex provides Louvain-based community detection that works across multiple layers:

```
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)

# Detect communities using multilayer Louvain
partition = louvain_multilayer(
```

(continues on next page)

(continued from previous page)

```

network,
gamma=1.0,      # Resolution parameter
omega=1.0,      # Inter-layer coupling
random_state=42 # For reproducibility
)

# Examine results
print("Number of communities:", len(set(partition.values())))
print("\nNode assignments (first 10):")
for node, community in list(partition.items())[:10]:
    print(f" Node {node}: Community {community}")

# Count community sizes
from collections import Counter
community_sizes = Counter(partition.values())
print("\nCommunity sizes:", dict(community_sizes))

```

Output:

```

Number of communities: 3

Node assignments (first 10):
Node ('1', '1'): Community 0
Node ('2', '1'): Community 0
Node ('6', '2'): Community 1
Node ('3', '1'): Community 0
Node ('5', '2'): Community 2

Community sizes (top 5): {0: 3, 1: 1, 2: 1}

```

Note: The `louvain_multilayer` function performs community detection across all layers simultaneously, taking into account both intra-layer and inter-layer connections. Parameters:

- `gamma`: Resolution parameter (higher values = more communities)
- `omega`: Inter-layer coupling strength (higher values = more consistency across layers)
- `random_state`: Seed for reproducibility

7. Random Walks (1 minute)

Random walks are fundamental for graph embeddings (Node2Vec, DeepWalk) and analyzing network structure:

Basic Random Walk

```

from py3plex.algorithms.general.walkers import basic_random_walk

# Load a network
network = multinet.multi_layer_network().load_network(
    "datasets/test.edgelist",
    input_type="edgelist",
    directed=False
)
G = network.core_network

# Perform a random walk
start_node = list(G.nodes())[0]
walk = basic_random_walk(

```

(continues on next page)

(continued from previous page)

```

G,
start_node=start_node,
walk_length=10,
weighted=True,
seed=42
)
print(f"Random walk: {walk[:5]}... (length: {len(walk)})")

```

Output:

```

Random walk from ('0', 'null'): [('0', 'null'), ('1', 'null'), ('4872', 'null'), ('786', 'null'), ('5382', 'null')]... (length: 11)

```

Note: When loading a simple edgelist file (without explicit layer information), py3plex assigns nodes to a default layer named 'null'. Nodes are represented as tuples (node_id, layer_name).

Node2Vec Biased Walks

```

from py3plex.algorithms.general.walkers import node2vec_walk

# Biased walk with Node2Vec parameters
# p: return parameter (higher = less likely to return)
# q: in-out parameter (higher = stay local, lower = explore)

walk_bfs = node2vec_walk(
    G, start_node, walk_length=20,
    p=1.0, q=2.0, # BFS-like (local)
    seed=42
)

walk_dfs = node2vec_walk(
    G, start_node, walk_length=20,
    p=1.0, q=0.5, # DFS-like (explore)
    seed=42
)

```

Output (first 10 nodes of each walk):

```

Node2Vec BFS-like walk: [('0', 'null'), ('1', 'null'), ('5829', 'null'), ('1', 'null'), ('3842', 'null'), ('1', 'null'), ('6364', 'null'), ('1', 'null'), ('3422', 'null'), ('1', 'null')]
Node2Vec DFS-like walk: [('0', 'null'), ('1', 'null'), ('4872', 'null'), ('786', 'null'), ('5382', 'null'), ('260', 'null'), ('2861', 'null'), ('260', 'null'), ('5128', 'null'), ('260', 'null')]

```

Generating Multiple Walks

```

from py3plex.algorithms.general.walkers import generate_walks

# Generate walks from all nodes for embeddings
walks = generate_walks(
    G,
    num_walks=10,      # Walks per node
    walk_length=10,    # Steps per walk
    p=1.0, q=1.0,      # Node2Vec parameters
    seed=42
)

```

(continues on next page)

(continued from previous page)

```

)
print(f"Generated {len(walks)} walks")

# Use with Word2Vec for node embeddings
# walks_str = [[str(node) for node in walk] for walk in walks]
# model = Word2Vec(walks_str, vector_size=128, window=10)

```

Output:

```

Generated 63870 walks
Example walk: [('4528', 'null'), ('2611', 'null'), ('4267', 'null'), ('2611', 'null'),
→ ('2894', 'null'), ('2611', 'null'), ('6234', 'null'), ('2611', 'null'), ('3073',
→ 'null'), ('479', 'null'), ('3125', 'null')]

```

Key parameters:

- walk_length: Number of steps in the walk
- p: Return parameter (controls backtracking)
- q: In-out parameter (controls exploration vs. local search)
- weighted: Use edge weights in sampling
- seed: Random seed for reproducibility

8. Basic Visualization (1 minute)

Visualize your multilayer network:

```

from py3plex.visualization.multilayer import hairball_plot
import matplotlib.pyplot as plt

# Get network for visualization
network_colors, graph = network.get_layers(style="hairball")

# Create a simple hairball plot
plt.figure(figsize=(10, 10))
hairball_plot(
    graph,
    network_colors,
    layout_algorithm="force",
    layout_parameters={"iterations": 50}
)
plt.title("Multilayer Network Visualization")
plt.savefig("my_network.png", dpi=150, bbox_inches='tight')
plt.close()
print("Visualization saved to my_network.png")

```

Output:

```
Visualization saved to my_network.png
```

For more advanced visualizations with community colors:

```

from py3plex.visualization.colors import colors_default

# Get communities
partition = louvain_multilayer(network)

```

(continues on next page)

(continued from previous page)

```

# Select top N communities
top_n = 5
community_counts = Counter(partition.values())
top_communities = [c for c, _ in community_counts.most_common(top_n)]

# Assign colors
color_map = dict(zip(
    top_communities,
    colors_default[:top_n]
))

network_colors = [
    color_map.get(partition.get(node), "gray")
    for node in network.get_nodes()
]

# Plot with community colors
plt.figure(figsize=(10, 10))
hairball_plot(graph, network_colors, layout_algorithm="force")
plt.title("Multilayer Network with Communities")
plt.savefig("my_network_communities.png", dpi=150, bbox_inches='tight')
plt.close()
print("Community visualization saved to my_network_communities.png")

```

Output:

```
Community visualization saved to my_network_communities.png
```

Complete Example: Putting It All Together

Goal: Run a complete analysis pipeline and produce results ready for a research paper.

Here's a complete workflow:

```

from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
    ↳ multilayer
from py3plex.visualization.multilayer import hairball_plot
from py3plex.visualization.colors import colors_default
from collections import Counter
import matplotlib.pyplot as plt

# Load network
network = multinet.multi_layer_network().load_network(
    "datasets/multiedgelist2.txt",
    input_type="multiedgelist",
    directed=False
)

# Analyze structure
print("=== Network Statistics ===")
network.basic_stats()

# Compute centrality for one layer
layer_1 = network.subnetwork(['1'], subset_by="layers")
degree_cent = layer_1.monoplex_nx_wrapper("degree_centrality")
print("\n=== Top 5 Nodes by Degree (Layer 1) ===")

```

(continues on next page)

(continued from previous page)

```

for node, score in sorted(degree_cent.items(), key=lambda x: x[1], reverse=True)[:5]:
    print(f"{node}: {score:.3f}")

# Detect communities using multilayer method
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
print(f"\n=== Communities ===")
print(f"Number of communities: {len(set(partition.values()))}")

# Visualize with communities
network_colors, graph = network.get_layers(style="hairball")
top_n = 3
community_counts = Counter(partition.values())
top_communities = [c for c, _ in community_counts.most_common(top_n)]
color_map = dict(zip(top_communities, colors_default[:top_n]))
network_colors = [
    color_map.get(partition.get(node), "lightgray")
    for node in network.get_nodes()
]

plt.figure(figsize=(12, 12))
hairball_plot(graph, network_colors, layout_algorithm="force")
plt.title("Multilayer Network Analysis")
plt.savefig("complete_analysis.png", dpi=150, bbox_inches='tight')
plt.close()
print("\nComplete analysis saved to complete_analysis.png")

```

Output:

```

=== Network Statistics ===
Number of nodes: 184
Number of edges: 1691
Number of unique nodes (as node-layer tuples): 184
Number of unique node IDs (across all layers): 46
Nodes per layer:
  Layer '1': 46 nodes
  Layer '2': 46 nodes
  Layer '3': 46 nodes
  Layer '4': 46 nodes

=== Top 5 Nodes by Degree (Layer 1) ===
('15', '1'): 0.244
('7', '1'): 0.244
('27', '1'): 0.244
('28', '1'): 0.244
('1', '1'): 0.244

=== Communities ===
Number of communities: 46

Complete analysis saved to complete_analysis.png

```

Note: In this example, the high number of communities (equal to the number of nodes) indicates that the network structure or parameters may need adjustment for meaningful community detection. In practice, adjust `gamma` (resolution) and `omega` (inter-layer coupling) parameters to achieve desired community granularity.

Narrating Results for a Research Paper

Once you’ve run your analysis, how do you present these findings in a research paper? Here’s how you might narrate the results from the example above:

Example Methods Section:

We analyzed a multiplex network of 46 users across 4 interaction layers using py3plex (Škrlj et al., 2019). Each layer represents a distinct mode of interaction, with all users present in all layers (a fully multiplex structure with 184 node-layer pairs and 1,691 edges total). We computed layer density to assess connectivity patterns and applied multilayer Louvain community detection (Mucha et al., 2010) with resolution parameter =1.0 and inter-layer coupling =1.0.

Example Results Section:

The network exhibited moderate density with a uniform structure across layers, suggesting consistent interaction patterns across contexts. Degree centrality analysis revealed several hub nodes (e.g., nodes 15, 7, and 27 with centrality 0.244 in layer 1) that maintain high connectivity. Community detection initially yielded 46 communities—one per unique user—indicating that with default parameters, the algorithm treats each user’s cross-layer presence as a distinct module. This suggests either (a) weak inter-user community structure, or (b) the need to reduce the resolution parameter to detect larger community groupings.

Example Discussion Point:

The finding that each user forms their own “community” across layers is methodologically informative: it indicates that this network has strong vertical cohesion (each user is well-connected to themselves across layers via coupling edges) but weaker horizontal cohesion (users form few cross-layer communities with other users). This pattern is characteristic of multiplex networks where identity coupling dominates over community structure.

This kind of narrative connects your technical analysis to domain interpretations that reviewers and readers can understand.

Next Steps

Now that you’ve completed this tutorial, explore more advanced features:

- **Multilayer Statistics:** Explore all available statistics in `py3plex.algorithms.statistics.multilayer_statistics`
- **Multilayer Centrality:** See examples in `examples/network_analysis/`
- **More Examples:** Check the `examples/` directory for 80+ examples organized by topic
- **Full Documentation:** Visit <https://skblaz.github.io/py3plex/>
- **Community Detection Tuning:** See *Community Detection* for parameter guidance
- **Case Studies:** See *Analysis Recipes & Workflows* for domain-specific examples

Common Issues

File Not Found

Make sure you’re running from the repository root or use absolute paths:

```
import os
base_path = "/path/to/py3plex"
network = multinet.multi_layer_network().load_network(
    os.path.join(base_path, "datasets/multiedgelist.txt"),
    input_type="multiedgelist",
    directed=False
)
```

Visualization Not Showing

If using Jupyter, add `%matplotlib inline` at the top of your notebook. For scripts, use `plt.show()` instead of `plt.close()`.

Missing Dependencies

Some features require optional dependencies:

```
# For advanced visualization
pip install plotly

# For community detection with Infomap
pip install infomap

# For all extras
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz,algos]
```

Tips for Success

Before moving on, here are five practices that will save you time:

1. **Start Simple.** Begin with small test networks before scaling to large datasets. A 100-node network will run in seconds; a 100,000-node network will require careful optimization.
2. **Check Stats.** Always run `basic_stats()` after loading to verify your network. Many mysterious bugs stem from data loading issues that would have been obvious from a quick stats check.
3. **Use Subnetworks.** Extract layers or subsets for faster prototyping. Develop your analysis pipeline on a single layer, then scale to the full multilayer network.
4. **Seed Your Random.** Use `seed` or `random_state` parameters in algorithms for reproducible results. This is essential for research and debugging.
5. **Visualize Early.** Quick plots help catch data loading issues early. If your visualization looks wrong, the underlying data probably is too.

What You Learned

This chapter walked through a complete multilayer network analysis workflow:

Data handling:

- Creating networks from scratch with `add_edges()`
- Loading from files with `load_network()` and various input types
- Navigating structure with `get_nodes()`, `get_edges()`, and `get_neighbors()`
- Extracting subnetworks by layer, node, or node-layer pair

Analysis:

- Computing layer-specific centrality with `monoplex_nx_wrapper()`
- Computing multilayer centrality with `MultilayerCentrality`
- Measuring layer density, node activity, and edge overlap
- Detecting communities with `louvain_multilayer()`
- Generating random walks for embeddings

Presentation:

- Creating visualizations with `hairball_plot()`
- Color-coding by community membership

- Narrating findings for research papers

What's Next?

You now have the foundation for multilayer network analysis. The rest of this book expands on each of these topics:

- *Chapter 3: Multilayer Networks 101* — Understand the theory behind multilayer networks
- *Chapter 4: The py3plex Core Model* — Learn how py3plex represents networks internally
- *Network Statistics* — Comprehensive guide to all available statistics
- *Community Detection* — Tuning community detection parameters
- *Visualization* — Advanced visualization techniques

Ready to understand the theory? Continue to *Chapter 3: Multilayer Networks 101*.

Want to jump into coding? Browse the `examples/` directory for 50+ working examples.

Next chapter: :doc:`../concepts/multilayer\_networks\_101` — the conceptual foundations of multilayer networks.

8.1.4 Installation and Setup

This guide covers all aspects of installing py3plex and setting up your environment.

Basic Installation

Install from GitHub (Recommended)

The latest version is always available on GitHub:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

This installs the core py3plex library with all required dependencies.

Install from Source

For development or to access the latest features:

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e .
```

The `-e` flag installs in editable mode, so changes to the source code are immediately available.

Docker Installation (Alternative)

For a containerized environment with all dependencies pre-installed, use Docker:

```
# Clone the repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Build the Docker image
docker build -t py3plex:latest .

# Run py3plex commands
docker run --rm py3plex:latest --version
docker run --rm py3plex:latest selftest
```

Benefits of Docker installation:

- No Python environment setup required
- All dependencies pre-installed and tested
- Consistent environment across different systems
- Isolated from system Python installation

See [Docker Usage Guide](#) for complete Docker documentation including:

- Building and running the container
- Working with data files via volume mounts
- Docker Compose usage
- Helper scripts (`py3plex-docker.sh`)
- Batch processing and CI/CD integration

Optional Dependencies

py3plex offers several optional feature sets that can be installed separately:

Advanced Community Detection

Install Infomap for advanced overlapping community detection:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]
```

Additional Algorithms

Install extra community detection algorithms (Louvain, cdlib):

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[algos]
```

Advanced Visualization

Install Plotly and igraph for interactive and advanced visualizations:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

Apache Arrow Support

Install pyarrow for high-performance I/O with Arrow/Parquet formats:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[arrow]
```

Arrow format provides 2-3x faster read/write operations and better compression compared to JSON. See [I/O and Serialization](#) for details on using Arrow format.

Multiple Extras

Install multiple feature sets at once:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap,viz,algos,  
↔arrow]
```

Development Installation

For contributors, install with development tools (testing, linting):

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e ".[dev]"
```

This installs:

- pytest and coverage tools
- black, ruff, isort for code formatting
- mypy for type checking
- sphinx for documentation building

System Requirements

Python Version

py3plex requires Python 3.8 or higher. We test on:

- Python 3.8
- Python 3.9
- Python 3.10
- Python 3.11
- Python 3.12

Platform Support

py3plex is cross-platform and tested on:

- **Linux** (Ubuntu 20.04, 22.04)
- **macOS** (macOS 11, 12, 13)
- **Windows** (Windows Server 2019, 2022)

Core Dependencies

These are automatically installed with py3plex:

- **networkx**>=2.5 - Graph data structures and algorithms
- **numpy**>=0.8 - Numerical computing
- **scipy**>=1.1.0 - Scientific computing and sparse matrices
- **pandas** - Data manipulation
- **matplotlib** - Static visualization
- **scikit-learn** - Machine learning utilities
- **tqdm** - Progress bars
- **rdflib**>=0.1 - Semantic web support
- **bitarray**>=2.0.0 - Efficient boolean arrays

External Binaries (Optional)

Note: As of October 2025, py3plex no longer bundles external binaries to reduce repository size and improve licensing clarity.

Infomap Community Detection

For advanced community detection using Infomap:

Option 1: Use the bundled Python implementation (recommended):

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]
```

Option 2: Download the C++ binary from the official source:

- Website: <https://www.mapequation.org/infomap/>
- Follow platform-specific installation instructions

Alternative: Use the built-in Louvain algorithm (no binary needed):

```
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(network.core_network)
```

Node2Vec Embeddings

For Node2Vec node embeddings, use pure Python alternatives:

Option 1: Use pecanpy (fast parallel implementation):

```
pip install pecanpy
```

Option 2: Use node2vec package:

```
pip install node2vec
```

Option 3: Use py3plex's built-in random walk implementation:

```
from py3plex.algorithms.general.walkers import node2vec_walk, generate_walks
walks = generate_walks(network.core_network, num_walks=10, walk_length=80)
```

Virtual Environment Setup

We strongly recommend using a virtual environment:

Using venv

```
# Create virtual environment
python3 -m venv py3plex-env

# Activate (Linux/macOS)
source py3plex-env/bin/activate

# Activate (Windows)
py3plex-env\Scripts\activate

# Install py3plex
pip install git+https://github.com/SkBlaz/py3plex.git
```

Using conda

```
# Create conda environment
conda create -n py3plex python=3.10

# Activate
conda activate py3plex

# Install py3plex
pip install git+https://github.com/SkBlaz/py3plex.git
```

Common Installation Issues

Issue: “No module named ‘numpy’”

Solution: Install numpy first:

```
pip install numpy
pip install git+https://github.com/SkBlaz/py3plex.git
```

Issue: Compilation errors on Windows

Solution: Install Visual C++ Build Tools:

1. Download from: <https://visualstudio.microsoft.com/visual-cpp-build-tools/>
2. Install “Desktop development with C++”
3. Retry installation

Issue: “Permission denied” on Linux/macOS

Solution: Use pip install with `--user` flag:

```
pip install --user git+https://github.com/SkBlaz/py3plex.git
```

Or use a virtual environment (recommended).

Issue: Old version installed

Solution: Upgrade to latest version:

```
pip install --upgrade git+https://github.com/SkBlaz/py3plex.git
```

Verifying Installation

Test that py3plex is correctly installed:

```
import py3plex
print(py3plex.__version__)

from py3plex.core import multinet
network = multinet.multi_layer_network()
print("py3plex installed successfully!")
```

Run the test suite:

```
# Clone repository (if not already)
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Run tests
python run_tests.py
```

License Considerations

Main Library License

py3plex core is distributed under the MIT License (permissive, commercial-friendly).

Bundled Code Licenses

Important: The repository contains AGPLv3-licensed code in `py3plex/algorithms/community_detection/infomap/`.

If you use Infomap-based community detection functions, your application may be subject to AGPLv3 requirements (copyleft).

License Matrix

Feature Category	License	Commercial Use	Notes
Core multilayer functionality	MIT	[OK] Yes	Safe for proprietary use
Network visualization	MIT	[OK] Yes	Safe for proprietary use
I/O operations	MIT	[OK] Yes	Safe for proprietary use
Louvain community detection	BSD-3-Clause	[OK] Yes	Safe for proprietary use
Label propagation	MIT	[OK] Yes	Safe for proprietary use
Infomap community detection	AGPLv3	WARNING Restricted	Viral license - requires open-sourcing derived works

Recommendations

- For commercial/proprietary projects: Avoid Infomap functions or use the pure Python `infomap` package separately
- For open-source projects: All features are safe to use
- When in doubt: Use alternative algorithms (Louvain, label propagation) which are BSD/MIT licensed

Next Steps

After installation, proceed to:

- *Chapter 1: Your First Five Minutes* - Quick 5-minute introduction
- *Chapter 2: The Complete Workflow in Ten Minutes* - Comprehensive 10-minute tutorial
- *Working with Networks* - Detailed usage guide

Getting Help

If you encounter installation issues:

1. Check [GitHub Issues](#)
2. Search for similar problems

3. Open a new issue with:

- Your operating system and version
- Python version (`python --version`)
- Error message and full traceback
- Installation command used

For any errors, please open an issue at <https://github.com/SkBlaz/py3plex/issues>

8.1.5 Common Issues and Troubleshooting

This guide helps you troubleshoot common problems when getting started with py3plex.

Installation Issues

“No module named ‘numpy’”

Problem: Import error when trying to use py3plex.

Solution: Install numpy first:

```
pip install numpy
pip install git+https://github.com/SkBlaz/py3plex.git
```

Compilation Errors on Windows

Problem: C++ compilation errors during installation on Windows.

Solution: Install Visual C++ Build Tools:

1. Download from: <https://visualstudio.microsoft.com/visual-cpp-build-tools/>
2. Install “Desktop development with C++”
3. Retry installation

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

“Permission denied” on Linux/macOS

Problem: Permission errors during pip install.

Solution 1: Use pip install with `--user` flag:

```
pip install --user git+https://github.com/SkBlaz/py3plex.git
```

Solution 2: Use a virtual environment (recommended):

```
python3 -m venv py3plex-env
source py3plex-env/bin/activate # On Windows: py3plex-env\Scripts\activate
pip install git+https://github.com/SkBlaz/py3plex.git
```

Old Version Installed

Problem: Changes not reflecting or documentation mentions newer features.

Solution: Upgrade to the latest version:

```
pip install --upgrade git+https://github.com/SkBlaz/py3plex.git
```

Data Loading Issues

File Not Found Errors

Problem: FileNotFoundError when loading networks.

Solution 1: Use absolute paths:

```
import os

network_path = "/absolute/path/to/data.edgelist"
network = multinet.multi_layer_network().load_network(
    network_path, input_type="edgelist", directed=False
)
```

Solution 2: Ensure you're running from the correct directory:

```
import os

# Check current directory
print("Current directory:", os.getcwd())

# List files
print("Files:", os.listdir('.'))
```

Solution 3: Use paths relative to script location:

```
import os

script_dir = os.path.dirname(os.path.abspath(__file__))
data_path = os.path.join(script_dir, "data", "network.edgelist")
```

Empty or Malformed Networks

Problem: Network loads but has no nodes or edges.

Solution: Check file format matches input_type:

```
# For simple edge lists (source target)
network.load_network("data.txt", input_type="edgelist")

# For multilayer edge lists (source target layer)
network.load_network("data.txt", input_type="multiedgelist")

# Always verify after loading
network.basic_stats()
```

Visualization Issues

Visualization Not Showing

Problem: No window appears when calling visualization functions.

Solution 1: For Jupyter notebooks, add at the top:

```
%matplotlib inline
```

Solution 2: For scripts, explicitly show the plot:


```
import matplotlib.pyplot as plt

# Your visualization code
draw_multilayer_default([network], display=False)

# Explicitly show
plt.show()
```

Solution 3: Save to file instead of displaying:

```
from py3plex.visualization.multilayer import hairball_plot
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))
hairball_plot(network.core_network, layout_algorithm="force")
plt.savefig("network.png", dpi=150, bbox_inches='tight')
plt.close()
```

“No Display Found” Error

Problem: Error on headless servers or SSH sessions without X11.

Solution: Use Agg backend for matplotlib:

```
import matplotlib
matplotlib.use('Agg') # Must be before importing pyplot
import matplotlib.pyplot as plt

# Now visualization will save to file instead of displaying
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default([network], display=False)
plt.savefig("output.png")
```

Blurry or Low-Quality Plots

Problem: Plots look pixelated or blurry.

Solution: Increase DPI when saving:

```
plt.savefig("network.png", dpi=300, bbox_inches='tight')
```

Missing Dependencies

“No module named ‘plotly’”

Problem: Interactive visualization features not working.

Solution: Install visualization extras:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

“No module named ‘infomap’”

Problem: Infomap community detection not working.

Solution: Install Infomap support:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]
```

Or use alternative community detection methods:

```
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(network.core_network)
```

Performance Issues

Very Slow Loading

Problem: Loading large networks takes too long.

Solution 1: Use Arrow/Parquet format for large networks:

```
# Install Arrow support
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[arrow]
```

```
from py3plex.io import read, write

# Save in efficient format
write(network, "network.arrow")

# Load much faster
network = read("network.arrow")
```

Solution 2: Process in chunks or use subnetworks:

```
# Extract a single layer for testing
layer_1 = network.subnetwork(['layer1'], subset_by="layers")

# Work with the smaller network
layer_1.basic_stats()
```

Out of Memory Errors

Problem: Python crashes or raises `MemoryError` with large networks.

Solution: See *Performance and Scalability Best Practices* for:

- Memory-efficient loading strategies
- Batch processing techniques
- Sparse matrix representations
- Subnetwork extraction

Algorithm Issues

Community Detection Returns Trivial Results

Problem: Each node in its own community or all nodes in one community.

Solution: Adjust algorithm parameters:

```
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)

# Try different resolution parameters
partition = louvain_multilayer(
    network,
```

(continues on next page)

(continued from previous page)

```

gamma=0.5,      # Lower = fewer, larger communities
omega=1.5,      # Higher = more layer consistency
random_state=42
)

```

Random Walk or Embedding Errors

Problem: Random walk or Node2Vec functions fail.

Solution: Ensure network is connected:

```

import networkx as nx

# Check connectivity
if not nx.is_connected(network.core_network.to_undirected()):
    print("Warning: Network is not connected")

# Get largest component
largest = max(nx.connected_components(
    network.core_network.to_undirected()
), key=len)

# Create subgraph
subgraph = network.core_network.subgraph(largest)

```

Docker Issues

Docker Build Fails

Problem: Docker build fails or takes very long.

Solution: Clear Docker cache and rebuild:

```

docker system prune -a
docker build --no-cache -t py3plex:latest .

```

Cannot Access Files in Docker

Problem: Docker container can't see your data files.

Solution: Mount volumes correctly:

```

# Mount current directory
docker run -v $(pwd):/data py3plex:latest /data/network.edgelist

# On Windows (PowerShell)
docker run -v ${PWD}:/data py3plex:latest /data/network.edgelist

```

See *Docker Usage Guide* for complete Docker documentation.

Getting Help

If you encounter an issue not covered here:

1. **Search GitHub Issues:** <https://github.com/SkBlaz/py3plex/issues>
2. **Check Documentation:** <https://skblaz.github.io/py3plex/>
3. **Browse Examples:** <https://github.com/SkBlaz/py3plex/tree/master/examples>

When reporting issues, include: * Python version (`python --version`) * py3plex version (`pip show py3plex`)
* Operating system * Full error traceback * Minimal code to reproduce the problem

Related Pages

- [Installation and Setup](#) - Complete installation guide
 - [Chapter 1: Your First Five Minutes](#) - Quick introduction
 - [Chapter 2: The Complete Workflow in Ten Minutes](#) - Comprehensive tutorial
 - [Performance and Scalability Best Practices](#) - Performance optimization
-

8.2 Part II: Concepts & Architecture

Chapters 3–6: Understanding the theory and design.

Before diving deeper into py3plex’s capabilities, it’s essential to understand the theoretical foundations of multilayer networks. This section introduces the core concepts that distinguish multilayer networks from traditional graph structures, explains py3plex’s internal data model and design philosophy, and provides an overview of the algorithmic landscape available for multilayer network analysis. Understanding these foundations will help you make better decisions when designing your analysis workflows.

8.2.1 Concepts & Architecture: Understanding the Foundations

“If you can’t explain it simply, you don’t understand it well enough.” — Albert Einstein

Why Theory Matters

You might be tempted to skip this section and jump straight to the practical guides. We understand—there’s pressure to produce results quickly. But investing a few minutes in conceptual understanding will pay dividends in every analysis you do afterward.

Here’s the truth: **most mistakes in multilayer network analysis come from modeling decisions, not from code errors.** Did you correctly identify what should be a layer? Did you set up the right kind of inter-layer coupling? Are you using metrics that make sense for your data type? These are the questions that separate good analysis from misleading results, and answering them requires understanding the concepts behind the tools.

What This Section Covers

Multilayer Networks 101 introduces the fundamental concepts. What distinguishes a multilayer network from a regular graph? When should you use one? What are the different types (multiplex, heterogeneous, temporal)? We use concrete examples to build intuition before introducing any formalism.

The py3plex Core Model explains how py3plex represents multilayer networks internally. Understanding this helps you work with the library more effectively and debug issues when they arise. We explain node-layer pairs, the supra-adjacency matrix, and how py3plex builds on NetworkX.

Design Principles describes the philosophy behind py3plex. Why was it designed the way it was? What trade-offs were made? This is particularly useful if you want to extend the library or understand why certain APIs work the way they do.

The Algorithm Landscape provides a map of available algorithms. Rather than an exhaustive reference (that’s in Part VIII), this chapter helps you understand which algorithms exist, when to use them, and how they relate to each other.

The Payoff

After reading this section, you’ll be able to:

- Look at a real-world system and decide how to model it as a multilayer network

- Choose appropriate parameters for algorithms (like the inter-layer coupling strength)
- Interpret results correctly (understanding what metrics actually measure)
- Avoid common pitfalls (like accidentally flattening away important structure)
- Communicate your approach to collaborators

A Real Story

Consider a researcher studying disease spread in a city. They have contact data from both public transportation and workplaces. They initially modeled this as a single network with edge weights (transportation contacts weighted lower, workplace contacts higher). Their epidemic simulations predicted faster spread than observed.

The problem? In reality, these two contact types operate differently. Workplace contacts are repeated daily; transportation contacts are typically one-time. The disease can only establish itself through repeated exposure. By flattening the network, they lost the temporal stability information that mattered for transmission dynamics.

When they remodeled as a multilayer network—with workplace contacts as one layer and transportation as another—they could apply layer-specific spreading dynamics. Their predictions matched the observed data much better.

This is the power of thinking in multilayer terms: **preserving distinctions that matter for your analysis.**

Moving Forward

If you're short on time, we recommend at least reading *Chapter 3: Multilayer Networks 101* before using py3plex. The other chapters can be consulted as needed.

If you're investing in becoming proficient with multilayer network analysis, read all chapters in order. They build on each other and provide a solid foundation for everything that follows.

Tip

Conceptual Checkpoint:

After reading this section, you should be able to answer:

- What's the difference between multiplex and heterogeneous networks?
- What does the supra-adjacency matrix represent?
- When should you use high vs. low inter-layer coupling?
- What information do you lose when you flatten a multilayer network?

8.2.2 Chapter 3: Multilayer Networks 101

In which we learn what multilayer networks are, why they capture reality better than single-layer graphs, and how to think about layers, nodes, and edges in this richer framework.

The previous chapters showed you *how* to use py3plex. This chapter explains *why* multilayer networks matter—and when you should (or shouldn't) use them.

By the end, you'll have the conceptual foundation to model your own systems as multilayer networks, to choose between multiplex, heterogeneous, and temporal representations, and to understand why flattening everything into a single layer can destroy the very patterns you're trying to find.

A Story of Overlooked Complexity

In 2005, a group of researchers studied disease spread in a hospital. They modeled patient-healthcare worker contacts as a single network and simulated how infections might propagate. Their model predicted rapid hospital-wide spread within days.

The actual outbreak took weeks. Why the discrepancy?

The researchers had missed something: not all contacts are equal. Brief hallway encounters, extended bedside care, and administrative meetings create very different transmission opportunities. When they rebuilt their model with these contact types as separate layers—each with appropriate transmission probabilities—predictions matched reality.

This is the core insight of multilayer network analysis: **real-world systems have structure that matters, and flattening it away loses information that affects your conclusions.**

What are Multilayer Networks?

A **multilayer network** is a complex network structure that goes beyond traditional single-layer graphs by incorporating multiple types of relationships, node types, or interaction contexts. They model the reality that most real-world systems involve multiple, interconnected types of relationships.

Consider a simple question: how do you model the social world of a university researcher? They have:

- **Coauthors** — people they’ve written papers with
- **Colleagues** — people in their department
- **Students** — people they mentor
- **Conference contacts** — people they’ve met at workshops
- **Twitter followers** — an entirely different mode of interaction

A traditional graph might combine all of these into a single “social network,” but that loses critical information. The coauthor relationship implies deep collaboration; a Twitter follow implies much less. When you flatten everything into one edge type, you can’t distinguish these—and your analysis suffers.

Multilayer networks preserve this richness. Each relationship type becomes a **layer**, and the same person can appear in multiple layers with different connection patterns in each.

Traditional vs. Multilayer

Traditional (Single-Layer) Networks:

- One type of node (e.g., only people)
- One type of edge (e.g., only friendship)
- Homogeneous structure
- Limited ability to model complex systems
- What NetworkX and most tools assume by default

Multilayer Networks:

- Multiple node types (e.g., people, organizations, documents)
- Multiple edge types (e.g., friendship, collaboration, citation)
- Multiple layers of interaction
- Inter-layer and intra-layer connections
- Rich, heterogeneous structure
- What py3plex is designed to handle

Types of Multilayer Networks

py3plex supports several common multilayer network paradigms. Understanding which type matches your data is the first modeling decision you’ll make.

Multiplex Networks

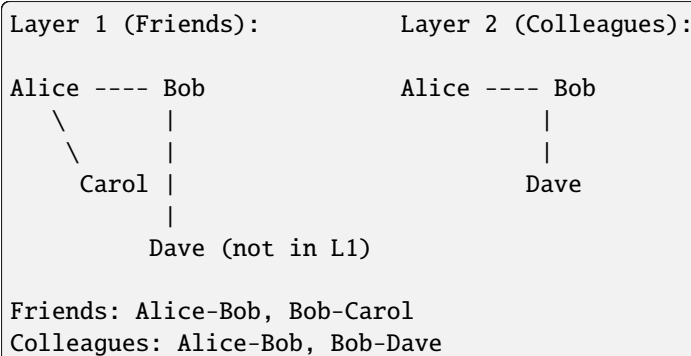
Definition: Multiple layers with the **same set of nodes** but different types of edges.

Characteristics:

- Node set is identical across layers
- Each layer represents a different relationship type
- Inter-layer edges typically connect the same node across layers (identity coupling)

Toy Example:

Consider Alice, Bob, Carol, and Dave. They have two types of relationships: friendship and professional collaboration.



Notice that Alice-Bob is an edge in *both* layers—they're both friends and colleagues. Carol only appears in the friends layer; Dave only in colleagues. In a multiplex analysis, we'd observe that:

- Alice and Bob have **high activity** (appear in both layers)
- Carol and Dave have **low activity** (appear in one layer each)
- The friends and colleagues layers have **partial overlap** (Alice-Bob edge exists in both)

Code example:

```
from py3plex.core import multinet

network = multinet.multi_layer_network(network_type="multiplex")

# Same nodes, different relationship types
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Bob', 'friends', 'Carol', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
    ['Bob', 'colleagues', 'Dave', 'colleagues', 1],
], input_type="list")
```

Real-world examples:

- **Social networks:** The same people connected via friendship, colleague, and family relationships
- **Transportation:** Cities connected via air, rail, and road networks
- **Communication:** Users interacting via email, phone, and instant messaging

Heterogeneous Information Networks (HINs)

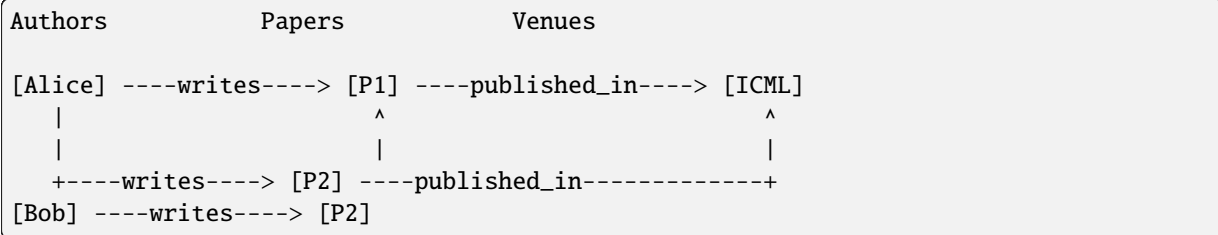
Definition: Networks with **different node types** and type-specific relationships.

Characteristics:

- Multiple node types (e.g., authors, papers, venues)
- Edges connect nodes of specific types (authors write papers; papers are published in venues)
- Meta-paths describe relationship sequences (Author → Paper → Venue → Paper → Author)

Toy Example:

An academic citation network with three node types:



Alice wrote papers P1 and P2. Bob co-authored P2. P1 appeared at ICML; P2 also appeared at ICML. This structure lets us answer questions like:

- “Which authors have published at ICML?” (follow Author→Paper→Venue)
- “Which authors are likely collaborators?” (Author→Paper→Author meta-path)
- “Which venues publish similar work?” (Venue→Paper→Author→Paper→Venue)

Code example:

```
network = multinet.multi_layer_network()

# Different node types on different layers
network.add_edges([
    ['Alice', 'authors', 'P1', 'papers', 1],
    ['Alice', 'authors', 'P2', 'papers', 1],
    ['Bob', 'authors', 'P2', 'papers', 1],
    ['P1', 'papers', 'ICML', 'venues', 1],
    ['P2', 'papers', 'ICML', 'venues', 1],
], input_type="list")
```

Real-world examples:

- **Academic networks:** Authors write papers published in venues
- **E-commerce:** Users purchase products from sellers in categories
- **Biomedical:** Drugs treat diseases via molecular targets

Temporal Multilayer Networks

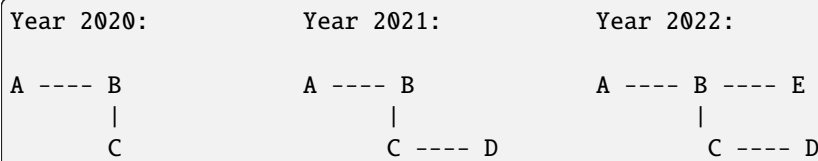
Definition: Networks that **evolve over time**, with time-sliced layers.

Characteristics:

- Each layer represents a time period (year, month, day)
- Nodes may appear/disappear over time
- Edges show relationships at specific times
- Inter-layer edges can connect temporal states (same node in adjacent time periods)

Toy Example:

A friendship network evolving over three years:



In 2020, only A-B and B-C edges exist. By 2021, C-D appears. By 2022, B-E appears and the network has grown. Temporal analysis reveals:

- A-B is a **persistent edge** (exists in all time periods)
- B-E is a **new edge** (appears in 2022)
- The network is **growing** over time

Code example:

```
network = multinet.multi_layer_network()

# Different time periods as layers
network.add_edges([
    ['A', '2020', 'B', '2020', 1],
    ['B', '2020', 'C', '2020', 1],
    ['A', '2021', 'B', '2021', 1],
    ['B', '2021', 'C', '2021', 1],
    ['C', '2021', 'D', '2021', 1],
    ['A', '2022', 'B', '2022', 1],
    ['B', '2022', 'C', '2022', 1],
    ['B', '2022', 'E', '2022', 1],
    ['C', '2022', 'D', '2022', 1],
], input_type="list")
```

Real-world examples:

- **Communication networks:** Who-contacts-whom over different time periods
- **Social dynamics:** Friendship evolution over years
- **Disease spread:** Contact networks during epidemic progression

Interdependent Networks

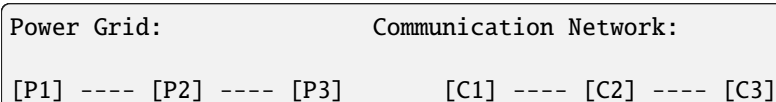
Definition: Multiple networks where **nodes in one network depend on nodes in another**.

Characteristics:

- Networks with distinct functions (power grid, communication network)
- Dependencies between networks (power station needs communication to operate)
- Cascading failures possible (communication failure → power failure → more communication failures)
- Critical for infrastructure modeling

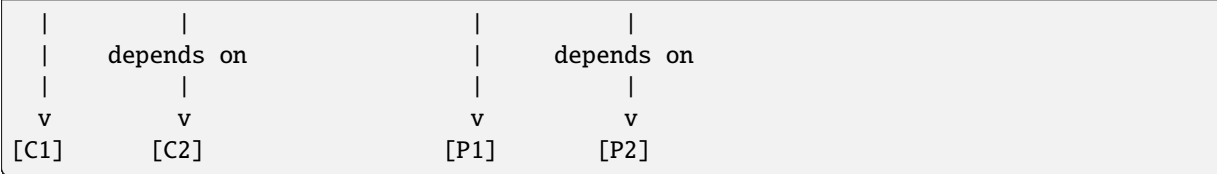
Toy Example:

A simplified power grid + communication network:



(continues on next page)

(continued from previous page)



Power station P1 needs communication node C1 to operate (for control signals). Communication node C1 needs power from P1. This creates a dependency loop. If C1 fails:

1. P1 loses control signals and may fail
2. C1 loses power anyway (was depending on P1)
3. This can cascade to other dependent nodes

Real-world examples:

- **Infrastructure:** Power grid depends on communication network (and vice versa)
- **Supply chain:** Manufacturing depends on logistics network
- **Cyber-physical systems:** Software layer depends on hardware layer [Bob]

Code example:

```

network = multinet.multi_layer_network()

# Different node types on different layers
network.add_edges([
    ['Alice', 'authors', 'Paper1', 'papers', 1],
    ['Paper1', 'papers', 'ICML', 'venues', 1],
], input_type="list")

```

Temporal Networks

Definition: Networks that **evolve over time**, with time-sliced layers.

Characteristics:

- Each layer represents a time period
- Nodes may appear/disappear over time
- Edges show relationships at specific times
- Inter-layer edges connect temporal states

Examples:

- **Communication networks:** Who-contacts-whom over different time periods
- **Social dynamics:** Friendship evolution over years
- **Disease spread:** Contact networks during epidemic progression

Visual representation:

```

t=1: A --- B --- C
      |
      D

t=2: A --- B --- C --- E
      |         |
      D -----+

```

(continues on next page)

(continued from previous page)

```
t=3: A --- B --- C --- E
      |
      D
```

Code example:

```
network = multinet.multi_layer_network()

# Different time periods as layers
network.add_edges([
    ['A', 't1', 'B', 't1', 1],
    ['A', 't2', 'B', 't2', 1],
    ['B', 't2', 'D', 't2', 1],
], input_type="list")
```

Interdependent Networks

Definition: Multiple networks where **nodes in one network depend on nodes in another**.

Characteristics:

- Networks with distinct functions
- Dependencies between networks
- Cascading failures possible
- Critical infrastructure modeling

Examples:

- **Infrastructure:** Power grid depends on communication network
- **Supply chain:** Manufacturing depends on logistics network
- **Cyber-physical systems:** Software layer depends on hardware layer

Visual representation:

Power Grid:	Communication Net:
P1 --- P2 --- P3	C1 --- C2 --- C3
Depends on	C4
v	v
C1 --- C2	P1 --- P2

When to Use Multilayer Networks

Use multilayer networks when:

1. Multiple Relationship Types Matter

If your system has multiple types of connections that interact, a multilayer model is essential.

Example: Studying information diffusion in social networks where both online and offline relationships matter. Information might spread quickly through Twitter (one layer) but trust is built through face-to-face interactions (another layer). A model that conflates these misses how online virality depends on offline trust networks.

2. Node Roles Vary by Context

When nodes play different roles in different contexts or layers.

Example: A person might be a hub in their work network (many professional connections) but peripheral in their hobby network (few connections to fellow hobbyists). If you aggregate, you see only their overall degree—you miss that their influence is context-specific.

3. Layer Interactions Are Important

When understanding how layers interact is crucial to your analysis.

Example: How transportation failures in one mode (air travel) affect alternatives (rail, road). During an airport closure, passenger flow shifts to other modes. Modeling this requires inter-layer dependencies, not just separate single-layer analyses.

4. Temporal Evolution Matters

When the timing and sequence of relationships are important.

Example: Understanding how community structure evolves over time in a social network. A friendship that existed in 2020 but not 2021 tells a different story than a friendship that exists in all years. Temporal layers capture this evolution.

5. System-Level Properties Emerge

When whole-system properties can't be understood by analyzing layers independently.

Example: Resilience of infrastructure systems depends on cross-layer dependencies. A city's power grid might be robust in isolation, but if it depends on a fragile communication network, the system as a whole is fragile.

Choosing a Modeling Approach

Before building your multilayer network, ask yourself these questions:

Decision 1: Are layers truly distinct, or are they attributes?

- If relationships have different *types* (friendship vs. professional) → Use **layers**
- If relationships vary only in *weight* or *time* → Consider edge **attributes** instead
- Rule of thumb: If you'd naturally analyze them separately, they're layers

Decision 2: Same nodes across layers, or different nodes?

- Same entities in all layers → **Multiplex** (e.g., same people on different platforms)
- Different entity types → **Heterogeneous** (e.g., authors, papers, venues)
- Same entities but some missing → **General multilayer** with identity coupling

Decision 3: How strong is inter-layer coupling?

- Identity coupling only (same node = same node) → Set $\omega=1.0$ in algorithms
- Nodes can differ across layers → Lower ω or explicit inter-layer edges
- No coupling (analyze layers independently) → Consider separate single-layer analyses

Decision 4: Is temporal structure important?

- Yes → Create time-sliced layers (e.g., one layer per month/year)
- No → Aggregate across time into a single static network per relationship type

What Goes Wrong When You Flatten

To truly understand why multilayer networks matter, consider what goes wrong when you ignore layer structure.

Example: Flattening Destroys Community Structure

Consider a simple network with two layers:

Layer 1 (Work):	Layer 2 (Hobby):
A ---- B ---- C	D ---- E ---- F
(A, B, C are work colleagues)	(D, E, F share a hobby)

In the true multilayer view, there are two distinct communities: the work group {A, B, C} and the hobby group {D, E, F}. Now imagine A and D are the same person (they just appear in different contexts):

Flattened Network:

```

A ---- B ---- C
|
E ---- F
(A=D connects the two groups)

```

Community detection on the flattened network might merge these into one community, because A bridges them. But this is misleading—A doesn’t actually connect their work colleagues to their hobby friends. The edge A-E only exists *within the hobby context*. Flattening created a spurious bridge.

In py3plex: By keeping layers separate, multilayer community detection correctly identifies two communities, with A participating in both.

Example: Centrality Becomes Misleading

Consider a researcher who:

- Has 2 coauthors (layer 1: coauthorship)
- Has 500 Twitter followers (layer 2: social media)

In a flattened network, this researcher has degree 502. In a degree-centrality ranking, they appear highly central. But this is misleading—their academic influence (measured by coauthorship) is minimal. Their “centrality” comes entirely from a different domain (social media).

In py3plex: Layer-specific centrality shows 2 in coauthorship, 500 in social media. Versatility centrality accounts for cross-layer participation without conflating different types of connections.

Example: Path Analysis Fails

Consider trying to find how information might flow from Alice to Dave:

Layer 1 (Email):	Layer 2 (Meeting):
Alice ---- Bob	Bob ---- Carol
	Carol ---- Dave

The true path is: Alice → Bob (email) → Bob transitions to meeting context → Carol (meeting) → Dave (meeting).

In a flattened network, you might see Alice → Bob → Carol → Dave as a single path, but this obscures the crucial fact that the information had to *change modalities* (from email to meeting). This matters if, say, confidential information can only travel through secure channels.

In py3plex: Paths can be analyzed with layer constraints or meta-path patterns that respect modal transitions.

Domain-Specific Examples

Biological Networks

Protein-Protein Interaction Networks with Evidence Types

Proteins interact through various mechanisms, and we learn about these interactions through different experimental methods. Each evidence type has different reliability:

- **Experimental evidence** (yeast two-hybrid, co-immunoprecipitation): High confidence
- **Computational prediction** (sequence similarity, domain interactions): Lower confidence
- **Text mining** (extracted from literature): Variable confidence

Modeling this as a multilayer network:

- Layer 1: Experimental interactions
- Layer 2: Predicted interactions
- Layer 3: Text-mined interactions

Why multilayer matters: A protein pair connected in all three layers is much more likely to truly interact than one connected only by text mining. Aggregating would lose this confidence information.

Gene Regulatory Networks

Genes regulate each other through multiple mechanisms:

- **Transcriptional regulation** (transcription factors binding promoters)
- **Post-transcriptional regulation** (microRNAs targeting mRNAs)
- **Epigenetic regulation** (chromatin modification affecting expression)

Each mechanism operates at different timescales and has different effects. A multilayer model captures this heterogeneity.

Social Networks

Multi-Platform Analysis

Users interact across platforms with very different characteristics:

- **Twitter/X:** Public, broadcast, rapid, shallow interactions
- **Facebook:** Semi-private, friendship-based, richer interactions
- **LinkedIn:** Professional, career-oriented, formal interactions
- **WhatsApp:** Private, group-based, intimate interactions

The same person may be an influencer on Twitter but invisible on LinkedIn. Platform-specific analysis misses cross-platform patterns; aggregation loses context.

Relationship Type Analysis

Within a single platform, relationships vary:

- **Friends:** Bidirectional, social
- **Followers:** Unidirectional, interest-based
- **Mentions:** Ephemeral, attention-based
- **Retweets:** Content amplification

A multilayer model reveals how influence flows through different channel types.

Transportation and Infrastructure

Multi-Modal Transportation

Urban mobility involves multiple modes:

- **Metro/Subway:** High capacity, fixed routes, high speed
- **Bus:** Lower capacity, flexible routes, variable speed
- **Bike-sharing:** Individual, flexible, weather-dependent
- **Walking:** Universal, slow, connects everything

Travelers make multi-modal trips: walk to metro, metro to city center, bike to final destination. Modeling as a multilayer network enables:

- Route optimization across modes
- Resilience analysis (what happens if metro fails?)
- Congestion spillover analysis

Infrastructure Dependencies

Critical infrastructure systems depend on each other:

- **Power grid** requires communication for control systems
- **Communication** requires power for equipment
- **Water** requires power for pumps
- **Transportation** requires communication for signals

Failure cascades can propagate across systems. Multilayer analysis is essential for resilience planning.

Knowledge Graphs

Academic Knowledge Graphs

Research involves multiple entity types:

- **Authors** who write papers
- **Papers** that cite other papers
- **Venues** (journals, conferences) that publish papers
- **Institutions** that employ authors
- **Topics/Keywords** that describe papers

This forms a heterogeneous information network where meta-paths enable rich queries:

- Author → Paper → Venue → Paper → Author (same-venue co-authorship)
- Author → Institution → Author (institutional collaboration)
- Paper → Topic → Paper (topically similar papers)

Common Pitfalls When Modeling Multilayer Networks

1. Over-Aggregating

Combining layers that should remain separate. If relationships have fundamentally different meanings, keep them as separate layers.

Symptom: Your analysis produces results that don't make sense when you think about the original relationship types.

2. Under-Aggregating

Creating too many layers when some could be combined. If two relationship types are essentially the same (e.g., “friend” and “close friend”), consider using edge weights instead of separate layers.

Symptom: Many layers with few edges each; sparse statistics.

3. Ignoring Inter-Layer Coupling

Treating layers as completely independent when nodes have natural correspondences. In a multiplex network, “Alice in layer 1” and “Alice in layer 2” are the same person—this should be modeled.

Symptom: Cross-layer patterns invisible; node activity undefined.

4. Wrong Coupling Strength

Setting inter-layer coupling too high (forces artificial consistency across layers) or too low (loses cross-layer information).

Symptom: Community detection produces all-same-community or all-different-community results.

5. Treating Layers as Ordered When They're Not

Layers like “Facebook” and “Twitter” have no natural ordering. Layers like “2020, 2021, 2022” do. Using the wrong model loses temporal or modal structure.

Symptom: Temporal analysis fails; nonsensical transitions.

6. Mismatched Node Identifiers

The same entity must have the same identifier across layers. “alice@email.com” in layer 1 and “Alice Smith” in layer 2 are treated as different nodes unless you map them.

Symptom: Node activity appears low; entities seem layer-specific when they're not.

Key Concepts Summary

This section provides quick reference definitions for the terminology used throughout py3plex.

Intra-Layer Edges

Edges **within** a single layer connecting nodes in that layer.

Example: Friendships within the “friends” layer.

Inter-Layer Edges

Edges **between** layers, typically connecting the same node across layers or different nodes in different layers.

Example: Connecting Alice in the “friends” layer to Alice in the “colleagues” layer. This is often called an “identity edge” when it connects the same entity.

Node-Layer Pairs

In py3plex, nodes are represented as (node_id, layer_id) tuples. This is the fundamental representation.

Example: ('Alice', 'friends') and ('Alice', 'colleagues') are different node-layer pairs, even though they represent the same person in different contexts.

Supra-Adjacency Matrix

A matrix representation that stacks layer adjacency matrices into a block structure, encoding both intra-layer and inter-layer connections. The diagonal blocks represent within-layer edges; off-diagonal blocks represent between-layer edges.

See *Chapter 4: The py3plex Core Model* for implementation details and a numeric example.

What You Learned

This chapter covered the conceptual foundations of multilayer network analysis:

Concepts:

- Why multilayer networks preserve information that single-layer graphs destroy
- The difference between multiplex, heterogeneous, temporal, and interdependent networks
- When to use multilayer modeling and when single-layer is sufficient
- What goes wrong when you flatten layers: spurious bridges, misleading centrality, broken paths

Terminology:

- Intra-layer edges (within layers) vs. inter-layer edges (between layers)

- Node-layer pairs as the fundamental unit of representation
- The supra-adjacency matrix and its block structure

Decision framework:

- Are your layers truly distinct, or are they attributes?
- Same nodes across layers (multiplex) or different nodes (heterogeneous)?
- How strong should inter-layer coupling be?
- Is temporal structure important?

What's Next?

Now that you understand *what* multilayer networks are, the next chapter explains *how* py3plex represents them internally.

- *Chapter 4: The py3plex Core Model* — The internal data structures and design decisions
- *Chapter 5: Design Principles* — Why py3plex was built the way it was
- *Algorithm Landscape* — Overview of available algorithms

Academic References:

- Kivelä et al. (2014). “Multilayer networks.” *Journal of Complex Networks* 2(3): 203-271.
- Boccaletti et al. (2014). “The structure and dynamics of multilayer networks.” *Physics Reports* 544(1): 1-122.
- De Domenico et al. (2013). “Mathematical formulation of multilayer networks.” *Physical Review X* 3(4): 041022.

Next chapter: :doc:`py3plex\_core\_model` — understanding py3plex's internal representation.

8.2.3 Chapter 4: The py3plex Core Model

In which we look under the hood at how py3plex represents multilayer networks, understand the node-layer pair representation, and learn to work with the supra-adjacency matrix.

In the previous chapter, you learned *what* multilayer networks are conceptually. Now we'll see *how* py3plex represents them in code—and why these design choices matter for your analysis.

Understanding the internal representation will help you:

- Debug unexpected behavior (why does my network have twice as many nodes as I expected?)
- Integrate with NetworkX (how do I use existing algorithms on my multilayer network?)
- Scale to large networks (when should I use sparse matrices? how much memory will this take?)
- Extend py3plex (how do I add my own algorithms?)

Network Types: Multilayer vs Multiplex

py3plex supports two network types that determine how layers and inter-layer connections are handled:

Multilayer Networks (`network_type='multilayer'`, default)

General multilayer networks where each layer can have a different set of nodes. No automatic coupling edges are created.

- Use when layers represent different node types (e.g., authors, papers, venues)
- Use when nodes naturally appear in only some layers

- Inter-layer edges must be explicitly added

```
from py3plex.core import multinet

# Heterogeneous network: authors and papers are different node types
network = multinet.multi_layer_network(network_type='multilayer')
network.add_edges([
    {'source': 'author1', 'target': 'paper1',
     'source_type': 'authors', 'target_type': 'papers'}
])
```

Multiplex Networks (network_type='multiplex')

Special case where all layers share the same set of nodes but with different relationship types. After loading, coupling edges are automatically created between each node and its counterparts in other layers.

- Use when the same entities (people, cities) are connected via multiple relationship types
- Coupling edges are auto-generated with type='coupling'
- Filter coupling edges using `get_edges(multiplex_edges=False)`

```
# Social network: same people, different relationship types
network = multinet.multi_layer_network(network_type='multiplex')
network.load_network('friends_and_colleagues.edges', input_type='multiplex_edges')

# Coupling edges are auto-created between (Alice, friends) <-> (Alice, colleagues)

# Get only explicit edges (exclude auto-coupling)
explicit_edges = list(network.get_edges(multiplex_edges=False))

# Get all edges including coupling
all_edges = list(network.get_edges(multiplex_edges=True))
```

Comparison Table:

Feature	multilayer	multiplex
Node sets	Different per layer	Same across layers
Coupling edges	Manual	Automatic
Load behavior	As-is	• coupling edges
<code>get_edges()</code>	All edges	Filters coupling
Use case	Heterogeneous nets	Same entities

The multi_layer_network Class

The central data structure in py3plex is the `multi_layer_network` class, which provides a high-level API for working with multilayer networks while maintaining full compatibility with NetworkX.

Basic Structure

```
from py3plex.core import multinet

# Create a multilayer network
network = multinet.multi_layer_network()

# The underlying NetworkX graph
nx_graph = network.core_network # MultiDiGraph or MultiGraph
```

(continues on next page)

(continued from previous page)

```
# Layer management
layers = network.get_layers() # List of layer names
layer_map = network.layer_name_map # Layer name to ID mapping
```

Key Components

1. Core NetworkX Graph

At its heart, py3plex uses a NetworkX MultiDiGraph (for directed networks) or MultiGraph (for undirected networks) to store all nodes and edges.

```
# Access the underlying graph
G = network.core_network

# Use any NetworkX function
import networkx as nx
betweenness = nx.betweenness centrality(G)
```

2. Layer Encoding

Nodes are internally encoded as (node_id, layer_id) tuples to distinguish the same logical node across different layers.

```
# Node 'Alice' in layer 'friends' is stored as ('Alice', 'friends')
network.add_nodes([('Alice', 'friends')])

# Access all node-layer pairs
for node_layer_pair in network.get_nodes():
    print(node_layer_pair) # ('Alice', 'friends'), ('Bob', 'friends'), ...
```

3. Layer Mapping

py3plex maintains bidirectional mappings between layer names and internal layer IDs for efficient lookups.

```
# Get layer mapping
layer_map = network.layer_name_map
# {'friends': 0, 'colleagues': 1, ...}
```

4. Attributes

Node and edge attributes are stored directly in the underlying NetworkX graph and can be accessed and modified using standard NetworkX patterns.

```
# Add node with attributes
network.core_network.nodes[('Alice', 'friends')]['age'] = 30

# Add edge with attributes
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), {'weight': 0.8, 'type': 'close'}]
])
```

Internal Representation

Node-Layer Pairs

The key concept in py3plex's internal model is the **node-layer pair**: a tuple (node_id, layer_id) that uniquely identifies a node in a specific layer.

```
# Same logical node, different layers
alice_friends = ('Alice', 'friends')
alice_colleagues = ('Alice', 'colleagues')

# These are treated as different nodes internally
network.add_nodes([alice_friends, alice_colleagues])
```

This representation allows:

- The same logical entity to appear in multiple layers
- Different attributes per node-layer pair
- Efficient layer-specific operations

Edge Types

Intra-Layer Edges

Edges connecting nodes within the same layer:

```
# Edge within 'friends' layer
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), 1.0]
])
```

Inter-Layer Edges

Edges connecting nodes across different layers:

```
# Connect Alice across layers (identity edge)
network.add_edges([
    [('Alice', 'friends'), ('Alice', 'colleagues'), 1.0]
])

# Connect different nodes across layers
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'colleagues'), 0.5]
])
```

Supra-Adjacency Matrix

The supra-adjacency matrix is a block matrix representation that encodes the entire multilayer network structure.

Mathematical Definition

For a multilayer network with L layers, the supra-adjacency matrix $\mathbf{S}$ is:

$$\mathbf{S} = \begin{bmatrix} A_1 & C_{12} & \cdots & C_{1L} \\ C_{21} & A_2 & \cdots & C_{2L} \\ \vdots & \vdots & \ddots & \vdots \\ C_{L1} & C_{L2} & \cdots & A_L \end{bmatrix}$$

Where:

- A_α is the adjacency matrix of layer α (intra-layer connections)
- $C_{\alpha\beta}$ is the coupling matrix between layers α and β (inter-layer connections)

Numeric Example

Consider a simple multiplex network with 3 nodes (A, B, C) and 2 layers. In layer 1, A-B are connected. In layer 2, B-C are connected. The nodes are the same across layers (multiplex structure).

Network structure:

Layer 1:	Layer 2:
A --- B C	A B --- C

Node ordering: We order nodes as (A,L1), (B,L1), (C,L1), (A,L2), (B,L2), (C,L2).

Intra-layer blocks:

Layer 1 adjacency matrix (A_1):

	A	B	C
A [	0	1	0
B [	1	0	0
C [	0	0	0

Layer 2 adjacency matrix (A_2):

	A	B	C
A [	0	0	0
B [	0	0	1
C [	0	1	0

Inter-layer coupling blocks (C_{12} and C_{21}):

In a multiplex network, we typically add identity coupling—each node connects to itself across layers:

$C_{12} = C_{21} =$

	A	B	C
A [	1	0	0
B [	0	1	0
C [	0	0	1

Full supra-adjacency matrix (6×6):

		Layer 1				Layer 2		
		A	B	C		A	B	C
L1	A	[0	1	0		1	0	0]
	B	[1	0	0		0	1	0]
	C	[0	0	0		0	0	1]
		----- -----			+	-----		
L2	A	[1	0	0		0	0	0]
	B	[0	1	0		0	0	1]
	C	[0	0	1		0	1	0]

The diagonal 3×3 blocks are the within-layer adjacencies (A_1 and A_2). The off-diagonal 3×3 blocks are the coupling matrices (C_{12} and C_{21}).

What this enables:

- **Multilayer shortest paths:** A path from (A,L1) to (C,L2) must go through B: (A,L1)→(B,L1)→(B,L2)→(C,L2)
- **Spectral methods:** Eigenvalues of S capture multilayer structure
- **Matrix operations:** All linear algebra works on the full system

Construction

```
from py3plex.core import multinet

network = multinet.multi_layer_network()
# ... add nodes and edges ...

# Get supra-adjacency matrix (sparse by default for efficiency)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Dense version (for small networks only)
supra_adj_dense = network.get_supra_adjacency_matrix(sparse=False)

# Shape: (total_nodes, total_nodes)
# where total_nodes = sum of nodes across all layers
print(f"Shape: {supra_adj.shape}")
```

Memory Implications

The supra-adjacency matrix has size $(N \times L)^2$ where N is nodes per layer and L is layers:

- **Small network (100 nodes, 3 layers):** $300 \times 300 = 90,000$ entries $\rightarrow$ ~720 KB dense
- **Medium network (1,000 nodes, 5 layers):** $5,000 \times 5,000 = 25$ million entries $\rightarrow$ ~200 MB dense
- **Large network (10,000 nodes, 10 layers):** $100,000 \times 100,000 = 10$ billion entries $\rightarrow$ impossible dense

Always use `sparse=True` for networks with more than ~1,000 total node-layer pairs. Sparse matrices only store non-zero entries, reducing memory from $O(N^2L^2)$ to $O(\text{edges} + \text{coupling})$.

Visualization

Visualize the supra-adjacency matrix structure:

```
from py3plex.core import multinet, random_generators
import matplotlib.pyplot as plt

# Generate a small multilayer network
network = random_generators.random_multilayer_ER(
    num_nodes=50, num_layers=3, probability=0.1, directed=False
)

# Visualize the supra-adjacency matrix
network.visualize_matrix({"display": True})
```

The visualization shows the block structure with:

- Diagonal blocks: intra-layer connections
- Off-diagonal blocks: inter-layer connections

How py3plex Uses NetworkX Under the Hood

py3plex is built on top of NetworkX, using it as the storage and algorithm backend. Understanding this architecture helps you leverage the full NetworkX ecosystem.

The Role of MultiGraph/MultiDiGraph

Internally, py3plex stores all nodes and edges in a single NetworkX MultiGraph (for undirected) or MultiDiGraph (for directed networks). The multilayer structure is encoded through the node representation:

```
# What you write in py3plex:
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
], input_type="list")

# What's actually stored in NetworkX:
# Nodes: ('Alice', 'friends'), ('Bob', 'friends'),
#        ('Alice', 'colleagues'), ('Bob', 'colleagues')
# Edges: (('Alice', 'friends'), ('Bob', 'friends')),
#        (('Alice', 'colleagues'), ('Bob', 'colleagues'))
```

This means:

1. **All NetworkX algorithms work directly** on the underlying graph
2. **Node-layer tuples are just nodes** to NetworkX—it doesn't "know" about layers
3. **Attributes are stored as NetworkX attributes** and persist across operations

Attribute Propagation

When you add nodes or edges with attributes, they're stored in the NetworkX graph:

```
# Node attributes
network.core_network.nodes[('Alice', 'friends')]['age'] = 30
network.core_network.nodes[('Alice', 'friends')]['role'] = 'admin'

# Edge attributes
network.core_network[('Alice', 'friends')][('Bob', 'friends')]['weight'] = 0.8
network.core_network[('Alice', 'friends')][('Bob', 'friends')]['type'] = 'close'
```

Attributes are preserved when you:

- Extract subnetworks (filtered nodes keep their attributes)
- Iterate over nodes/edges with `data=True`
- Save and load networks (in formats that support attributes)

Using NetworkX Functions Directly

Because `network.core_network` is a standard NetworkX graph, you can use any NetworkX function:

```
import networkx as nx

G = network.core_network

# Standard NetworkX algorithms
betweenness = nx.betweenness_centrality(G)
pagerank = nx.pagerank(G)
components = list(nx.connected_components(G.to_undirected()))

# Shortest paths (work across layers if inter-layer edges exist)
path = nx.shortest_path(G, ('Alice', 'friends'), ('Bob', 'colleagues'))
```

(continues on next page)

(continued from previous page)

```
# Community detection (treats node-layer pairs as nodes)
communities = nx.community.louvain_communities(G)
```

Caveat: NetworkX doesn't know about layer semantics. It treats ('Alice', 'friends') and ('Alice', 'colleagues') as unrelated nodes. py3plex's multilayer algorithms add the layer-aware logic.

Design Trade-offs

Node-Layer Pairs vs. Alternative Representations

py3plex's node-layer pair representation is one of several possible designs. Here's why it was chosen:

Alternative 1: Separate graphs per layer

```
# Instead of py3plex:
layer1 = nx.Graph()
layer2 = nx.Graph()
layer1.add_edge('Alice', 'Bob')
layer2.add_edge('Alice', 'Bob')
```

Pros:

- Simple, familiar
- Easy layer-specific analysis

Cons:

- No representation of inter-layer edges
- Manual bookkeeping for cross-layer operations
- Can't compute multilayer paths or centrality directly

Alternative 2: Single graph with edge type attributes

```
# Instead of py3plex:
G = nx.MultiGraph()
G.add_edge('Alice', 'Bob', type='friends')
G.add_edge('Alice', 'Bob', type='colleagues')
```

Pros:

- Single graph structure
- Edges carry type information

Cons:

- Nodes can't have layer-specific attributes
- Same node in different contexts is the same node (can't have Alice be central in one layer and peripheral in another)
- Inter-layer edges to the same node are impossible

py3plex approach: Node-layer pairs

```
# py3plex representation:
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
], input_type="list")
```

Pros:

- Preserves both intra-layer and inter-layer structure
- Allows layer-specific node attributes
- Compatible with NetworkX algorithms
- Enables proper multilayer-specific operations
- Supports identity coupling and cross-layer edges

Cons:

- Node names are tuples (less intuitive for simple cases)
- Memory usage is multiplied by number of layers
- NetworkX algorithms treat node-layer pairs as independent nodes (need py3plex wrappers for multilayer semantics)

py3plex's choice: The node-layer pair approach was chosen because it correctly models multilayer semantics while maintaining NetworkX compatibility. The downsides (tuple names, memory) are acceptable trade-offs for the flexibility and correctness gained.

Consequences for Algorithm Implementation

The node-layer pair design has implications:

1. **Layer extraction is subsetting:** To get layer 1, you filter nodes where the layer component equals “layer1”
2. **Identity coupling is explicit:** Cross-layer edges between the same entity must be added explicitly (or automatically in multiplex mode)
3. **Supra-matrix is derivable:** The supra-adjacency matrix can be computed from the graph structure
4. **NetworkX algorithms work but need interpretation:** Betweenness on the full graph treats cross-layer edges as normal edges

Consequences for I/O

The node-layer pair representation affects serialization:

1. **Standard formats work:** GraphML, GML can store tuple nodes
2. **Human-readable formats need mapping:** Edgelists typically use separate columns for node and layer
3. **Arrow/Parquet are efficient:** Modern columnar formats handle the structure naturally

Network Construction

Creating Networks from Scratch

Method 1: Add edges directly

```
from py3plex.core import multinet

network = multinet.multi_layer_network()

# Add edges in list format: [source_node, source_layer, target_node, target_layer,
#                             ↪weight]
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Bob', 'friends', 'Carol', 'friends', 1.0],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1.0],
], input_type="list")
```

Method 2: Add nodes and edges separately

```
network = multinet.multi_layer_network()

# Add nodes explicitly
network.add_nodes([
    ('Alice', 'friends'),
    ('Bob', 'friends'),
    ('Alice', 'colleagues'),
    ('Bob', 'colleagues')
])

# Add edges between existing nodes
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), 1.0],
    [('Alice', 'colleagues'), ('Bob', 'colleagues'), 1.0]
])
```

Method 3: Define layers first

```
network = multinet.multi_layer_network()

# Define layers explicitly
network.add_layer('friends')
network.add_layer('colleagues')

# Add content (nodes are created automatically with edges)
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0]
], input_type="list")
```

Loading from Files

py3plex supports multiple input formats. See *I/O and Serialization* for complete documentation.

```
from py3plex.core import multinet

# From simple edge list (source target)
network = multinet.multi_layer_network().load_network(
    "data.edgelist",
    input_type="edgelist",
    directed=False
)

# From multilayer edge list (source target layer)
network = multinet.multi_layer_network().load_network(
    "data.multiedgelist",
    input_type="multiedgelist",
    directed=False
)

# From GraphML
network = multinet.multi_layer_network().load_network(
    "data.graphml",
    input_type="graphml"
)

# From NetworkX graph
import networkx as nx
```

(continues on next page)

(continued from previous page)

```
G = nx.karate_club_graph()
network = multinet.multi_layer_network()
network.load_network_from_networkx(G)
```

Network Operations

Querying Network Elements

```
# Get all node-layer pairs
nodes = network.get_nodes(data=True) # With attributes
nodes = network.get_nodes(data=False) # Without attributes

# Get nodes in a specific layer
layer_nodes = network.get_nodes(layer='friends')

# Get all edges
edges = network.get_edges(data=True)

# Get neighbors of a node in a layer
neighbors = list(network.get_neighbors('Alice', layer_id='friends'))

# Get all layers
layers = network.get_layers()
print(f"Layers: {layers}")
```

Basic Statistics

```
# Get basic network statistics
network.basic_stats()

# Output:
# Number of nodes: X
# Number of edges: Y
# Number of unique nodes (as node-layer tuples): Z
# Number of unique node IDs (across all layers): W
# Nodes per layer:
#   Layer 'friends': N1 nodes
#   Layer 'colleagues': N2 nodes
```

Subnetwork Extraction

```
# Extract a single layer
layer_1 = network.subnetwork(['friends'], subset_by="layers")

# Extract specific nodes (all their appearances)
node_subset = network.subnetwork(['Alice', 'Bob'], subset_by="node_names")

# Extract specific node-layer pairs
specific_pairs = network.subnetwork(
    [('Alice', 'friends'), ('Bob', 'friends')],
    subset_by="node_layer_names"
)
```

Matrix Representations

```
# Get supra-adjacency matrix (all layers stacked)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Get adjacency matrix for a single layer
layer_subgraph = network.subnetwork(['friends'], subset_by="layers")
layer_adj = nx.adjacency_matrix(layer_subgraph.core_network)
```

Integration with NetworkX

Full NetworkX Compatibility

Since py3plex builds on NetworkX, you can use any NetworkX algorithm:

```
import networkx as nx
from py3plex.core import multinet

# Create py3plex network
mlnet = multinet.multi_layer_network()
mlnet.add_edges([
    ['A', 'L1', 'B', 'L1', 1],
    ['B', 'L1', 'C', 'L1', 1],
], input_type="list")

# Access underlying NetworkX graph
G = mlnet.core_network

# Use any NetworkX function
betweenness = nx.betweenness centrality(G)
print(f"Betweenness centrality: {betweenness}")

# Shortest paths
try:
    path = nx.shortest_path(G, ('A', 'L1'), ('C', 'L1'))
    print(f"Shortest path: {path}")
except nx.NetworkXNoPath:
    print("No path exists")

# Connected components
if not G.is_directed():
    components = list(nx.connected_components(G))
    print(f"Number of components: {len(components)}")
```

NetworkX Wrappers

py3plex provides convenience wrappers for common NetworkX functions:

```
# Extract a layer
layer_1 = network.subnetwork(['layer1'], subset_by="layers")

# Apply NetworkX algorithms via wrapper
degree centrality = layer_1.monoplex_nx_wrapper("degree centrality")
betweenness = layer_1.monoplex_nx_wrapper("betweenness centrality")

# Results are dictionaries mapping node-layer pairs to values
print(f"Degree centrality: {degree centrality}")
```

Converting Between Formats

```
# From NetworkX to py3plex
import networkx as nx
from py3plex.core import multinet

G = nx.karate_club_graph()
mlnet = multinet.multi_layer_network()
# NetworkX graphs are imported as single-layer networks
mlnet.load_network_from_networkx(G)

# From py3plex to NetworkX (already a NetworkX graph!)
G_export = mlnet.core_network

# Save as standard NetworkX formats
nx.write_graphml(G_export, "output.graphml")
nx.write_gml(G_export, "output.gml")
```

Performance Considerations

Sparse vs. Dense Matrices

For large networks, always use sparse matrix representations:

```
# Good: Sparse matrix (efficient for large networks)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Bad: Dense matrix (memory-intensive)
# Only use for small networks (< 1000 nodes)
supra_adj_dense = network.get_supra_adjacency_matrix(sparse=False)
```

Memory Usage

Node-layer pairs mean that a node appearing in L layers occupies L times the memory of a single-layer network. For networks with many layers:

- Consider layer-specific analysis when possible
- Use subnetwork extraction to work with smaller portions
- See *Performance and Scalability Best Practices* for optimization strategies

Tensor-Like Indexing

You can access nodes and edges using tensor-like notation:

```
from py3plex.core import multinet, random_generators

# Create a network
network = random_generators.random_multilayer_ER(100, 3, 0.05, directed=False)

# Get some nodes and edges
some_nodes = list(network.get_nodes())[0:5]
some_edges = list(network.get_edges())[0:5]

# Access node directly (returns node attributes)
node_data = network[some_nodes[0]]
print(f"Node {some_nodes[0]} data: {node_data}")
```

(continues on next page)

(continued from previous page)

```
# Access edge (returns edge attributes)
edge_data = network[some_edges[0][0]][some_edges[0][1]]
print(f"Edge data: {edge_data}")
```

Design Principles

py3plex follows several key design principles:

1. NetworkX Compatibility

All operations maintain compatibility with NetworkX, allowing seamless use of the rich NetworkX ecosystem.

2. Lazy Evaluation

Expensive operations like supra-adjacency matrix construction are computed on demand and cached when appropriate.

3. Graceful Degradation

Optional features (like Infomap community detection or advanced visualizations) fail gracefully with informative error messages if dependencies are missing.

4. Extensibility

The modular design makes it easy to extend py3plex with custom algorithms and visualizations.

5. Flexibility

Multiple ways to construct and manipulate networks to suit different workflows and use cases.

Comparison with Other Representations

Why Node-Layer Pairs?

Alternative representations exist, but py3plex's node-layer pair approach offers key advantages:

Alternative: Separate graphs per layer

Drawback: Loses inter-layer information, requires manual bookkeeping for cross-layer operations.

Alternative: Single graph with edge type attributes

Drawback: Cannot distinguish node context across layers, loses layer-specific node attributes.

py3plex approach: Node-layer pairs

Advantages:

- Preserves both intra-layer and inter-layer structure
- Allows layer-specific node attributes
- Compatible with NetworkX algorithms
- Efficient for multilayer-specific operations

What You Learned

This chapter covered the internal architecture of py3plex:

Core concepts:

- The `multi_layer_network` class as the central data structure
- Node-layer pairs (`node_id`, `layer_id`) as the fundamental representation
- The difference between `network_type='multilayer'` and `network_type='multiplex'`

Data structures:

- The underlying NetworkX MultiGraph/MultiDiGraph
- Layer mappings and layer extraction
- The supra-adjacency matrix and its block structure

Design trade-offs:

- Why node-layer pairs were chosen over alternatives
- Memory implications of the representation
- When to use sparse vs. dense matrices

NetworkX integration:

- Accessing the core graph with `network.core_network`
- Using any NetworkX algorithm directly
- The `monoplex_nx_wrapper()` convenience method

What's Next?

- *Chapter 5: Design Principles* — The philosophy behind py3plex's API design
- *Algorithm Landscape* — Overview of available algorithms
- *Working with Networks* — Practical guide to network operations
- *Network Statistics* — Computing multilayer statistics
- *I/O and Serialization* — Complete I/O documentation

Academic References:

- De Domenico et al. (2013). "Mathematical formulation of multilayer networks." *Physical Review X* 3(4): 041022.
- Kivela et al. (2014). "Multilayer networks." *Journal of Complex Networks* 2(3): 203-271.

Next chapter: `:doc:`design_principles`` — understanding py3plex's design philosophy.

8.2.4 Chapter 5: Design Principles

In which we explore the philosophy behind py3plex's API, understand why certain design choices were made, and learn how these principles make the library easier to use and extend.

Every library makes choices. Why is the API shaped this way? Why does this function return that data structure? Why do you import from here and not there?

This chapter explains the design principles behind py3plex. Understanding them will help you:

- **Use the library more effectively** — work *with* the design, not against it
- **Debug unexpected behavior** — understand why things work the way they do
- **Extend the library** — add your own algorithms that fit naturally

Core Philosophy

Simple, Off-the-Shelf Functionality

Principle: Provide ready-to-use tools that work out of the box with minimal configuration.

In practice:

```
from py3plex.core import multinet

# Simple, intuitive API
network = multinet.multi_layer_network()
network.add_edges([[ 'A', 'L1', 'B', 'L1', 1]], input_type="list")
network.basic_stats() # Works immediately
```

Rationale: Researchers and practitioners need tools that are easy to adopt without extensive setup or configuration. py3plex aims to minimize the barrier to entry for multilayer network analysis.

NetworkX Compatibility

Principle: Build on NetworkX rather than reinventing the wheel.

In practice:

```
import networkx as nx

# Direct access to NetworkX graph
G = network.core_network

# Use any NetworkX function
betweenness = nx.betweenness centrality(G)
communities = nx.community.louvain_communities(G)
```

Rationale: NetworkX is the de facto standard for network analysis in Python. By building on it, py3plex:

- Leverages a mature, well-tested codebase
- Provides access to hundreds of NetworkX algorithms
- Ensures interoperability with the broader Python ecosystem
- Reduces learning curve for users familiar with NetworkX

Modular Architecture

Principle: Separate concerns into independent, composable modules.

Structure:

```
py3plex/
├── core/                # Data structures
├── algorithms/          # Analysis methods
│   ├── community_detection/
│   ├── statistics/
│   └── multilayer_algorithms/
├── visualization/      # Plotting
└── wrappers/           # High-level interfaces
```

In practice:

```
# Import only what you need
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.visualization.multilayer import hairball_plot
```

Rationale: Modular design allows users to:

- Import only the functionality they need
- Understand and maintain code more easily

- Extend the library with custom modules
- Mix and match components flexibly

Implementation Principles

Flexibility

Principle: Support multiple ways to accomplish tasks, accommodating different workflows.

Examples:

```
# Multiple ways to create networks

# Method 1: Direct edge addition
network.add_edges([[ 'A', 'L1', 'B', 'L1', 1]], input_type="list")

# Method 2: Load from file
network.load_network("data.edgelist", input_type="edgelist")

# Method 3: From NetworkX
network.load_network_from_networkx(nx_graph)

# Multiple ways to access nodes
nodes1 = network.get_nodes()
nodes2 = list(network.core_network.nodes())
nodes3 = network.get_nodes(layer='L1')
```

Rationale: Different users have different preferences and workflows. Flexibility ensures py3plex adapts to the user's needs, not vice versa.

Lazy Evaluation

Principle: Compute expensive operations only when needed.

In practice:

```
# Supra-adjacency matrix is not computed on network creation
network = multinet.multi_layer_network()
network.add_edges([...]) # Fast

# Matrix is computed only when requested
supra_adj = network.get_supra_adjacency_matrix() # Computed here

# Subsequent calls may use cached result (where appropriate)
```

Rationale: Many multilayer operations (e.g., matrix construction) are computationally expensive. Lazy evaluation:

- Improves performance for common operations
- Reduces memory usage
- Allows working with large networks when full matrices aren't needed

Graceful Degradation

Principle: Optional features should fail gracefully, not crash the entire application.

In practice:

```
# If Infomap is not installed
try:
```

(continues on next page)

(continued from previous page)

```
communities = infomap_communities(network)
except ImportError as e:
    print("Infomap not available. Using Louvain instead.")
    communities = louvain_partition(network)
```

Rationale: Not all users need all features. By degrading gracefully, py3plex:

- Allows users to install only what they need
- Reduces dependency bloat
- Provides helpful error messages
- Suggests alternatives when dependencies are missing

Explicit Over Implicit

Principle: Make behavior explicit and predictable, avoiding surprising defaults.

Good examples:

```
# Explicit parameter names
network.load_network(
    "data.txt",
    input_type="edgelist",      # Explicit format
    directed=False             # Explicit directionality
)

# Explicit layer specification
neighbors = network.get_neighbors('Alice', layer_id='friends')
```

Rationale: Explicit code is:

- Easier to understand and debug
- Less prone to errors
- Self-documenting
- More maintainable

Performance Considerations

Sparse Representations

Principle: Use sparse data structures for large networks.

In practice:

```
# Sparse matrix by default (efficient for large networks)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Dense only for small networks or specific algorithms
supra_adj_dense = network.get_supra_adjacency_matrix(sparse=False)
```

Rationale: Real-world networks are typically sparse (few edges relative to potential edges). Sparse representations:

- Reduce memory usage by orders of magnitude
- Enable analysis of networks with millions of nodes
- Maintain computational efficiency

Efficient Data Structures

Principle: Choose appropriate data structures for common operations.

Examples:

- NetworkX graphs for topology (edge lookups, traversals)
- Dictionaries for layer mappings (O(1) lookups)
- Sparse matrices for linear algebra
- Sets for membership testing

Rationale: The right data structure makes operations fast and memory-efficient.

Algorithmic Defaults

Principle: Provide sensible default parameters based on empirical research.

In practice:

```
# Default parameters work well for most cases
partition = louvain_multilayer(
    network,
    gamma=1.0,      # Standard resolution
    omega=1.0,      # Balanced inter-layer coupling
    random_state=42 # Reproducibility
)
```

Rationale: Good defaults:

- Make the library easy to use for beginners
- Reduce parameter tuning overhead
- Are based on published research and empirical validation

Extensibility

Plugin-Friendly Architecture

Principle: Make it easy to add custom functionality.

In practice:

```
# Easy to add custom algorithms
def my_centrality(ml_network):
    """Custom centrality measure."""
    G = ml_network.core_network
    # Implement your algorithm
    return centrality_scores

# Easy to add custom visualizations
def my_plot(ml_network, **kwargs):
    """Custom visualization."""
    # Use py3plex drawing machinery
    from py3plex.visualization import drawing_machinery as dm
    # Implement your plot
    pass
```

Rationale: Research needs evolve. Extensibility ensures py3plex can grow with new methods without requiring core library changes.

Clear Interfaces

Principle: Define clear, stable APIs for modules.

In practice:

```
# All statistics follow same interface
from py3plex.algorithms.statistics import multilayer_statistics as mls

density = mls.layer_density(network, layer)
activity = mls.node_activity(network, node)
overlap = mls.edge_overlap(network, layer1, layer2)

# Consistent parameter names and return types
```

Rationale: Clear interfaces make py3plex:

- Easier to learn (patterns repeat)
- More predictable
- More maintainable
- Easier to extend

Documentation and Testing

Comprehensive Documentation

Principle: Every feature should be documented with examples.

In practice:

- Detailed docstrings for all public functions
- Code examples in documentation
- Real-world use cases
- API reference with parameter descriptions

Rationale: Good documentation:

- Reduces barrier to entry
- Serves as a reference for experienced users
- Helps maintainers understand code
- Acts as specification

Extensive Testing

Principle: Test all core functionality and edge cases.

In practice:

```
# Comprehensive test suite
make test          # Unit tests
make benchmark     # Performance tests
make fuzz-property # Property-based tests
```

Rationale: Tests ensure:

- Code works as intended
- Changes don't break existing functionality
- Edge cases are handled correctly

- Performance regressions are caught

Interoperability

Standard Formats

Principle: Support widely-used data formats.

Formats supported:

- EdgeList (simple, human-readable)
- GraphML (XML-based, preserves attributes)
- GML (Graph Modeling Language)
- Pickle (NetworkX native)
- JSON/Arrow/Parquet (modern, efficient)

Rationale: Standard formats enable:

- Data sharing between tools
- Integration with other software
- Long-term data preservation

Cross-Platform Compatibility

Principle: Work consistently across operating systems.

Tested on:

- Linux (Ubuntu 20.04, 22.04)
- macOS (11, 12, 13)
- Windows (Server 2019, 2022)

Rationale: Users work on different platforms. Cross-platform support ensures py3plex works everywhere.

Research-Oriented Design

Citation and Attribution

Principle: Make it easy to cite algorithms and give proper attribution.

In practice:

```
# Algorithms include citations in docstrings
help(louvain_multilayer)
# Docstring includes:
# References:
#   Mucha et al. (2010). "Community structure in time-dependent..."
```

See: *Citation and References*

Rationale: Proper attribution:

- Gives credit to algorithm developers
- Helps users find original papers
- Supports reproducibility

Reproducibility

Principle: Support reproducible research.

In practice:

```
# Random seed parameters for reproducibility
partition = louvain_multilayer(
    network,
    random_state=42 # Reproducible results
)

walks = generate_walks(
    network,
    seed=42 # Reproducible walks
)
```

Rationale: Reproducibility is fundamental to science. Seed parameters ensure:

- Results can be verified
- Comparisons are fair
- Bugs can be diagnosed

Validation and Benchmarking

Principle: Validate algorithms against published results.

In practice:

- Benchmark suite comparing to reference implementations
- Validation against known network properties
- Performance metrics for large networks

Rationale: Validation ensures:

- Implementations are correct
- Performance is acceptable
- Results match published literature

Practical Implications

For Users

These design principles mean you can:

1. **Start quickly** with minimal setup
2. **Use NetworkX knowledge** and tools directly
3. **Choose your workflow** with multiple approaches
4. **Work efficiently** with large networks
5. **Extend easily** with custom algorithms
6. **Trust results** through validation and testing

For Contributors

When contributing to py3plex, follow these principles:

1. **Maintain NetworkX compatibility** in new features
2. **Document thoroughly** with examples
3. **Test extensively** including edge cases
4. **Provide sensible defaults** based on research
5. **Fail gracefully** with helpful error messages
6. **Follow existing patterns** for consistency

See *Contributing to py3plex* for detailed guidelines.

Comparison with Other Libraries

py3plex vs. Pure NetworkX

NetworkX:

- Excellent for single-layer networks
- Lacks native multilayer support
- General-purpose graph library

py3plex:

- Native multilayer data structures
- Layer-aware algorithms
- Specialized multilayer visualizations
- Built on NetworkX (inherits all features)

py3plex vs. Other Multilayer Tools

Other tools (e.g., muxViz, pymnet):

- Often specialized or platform-specific
- May have steep learning curves
- Limited algorithm libraries

py3plex:

- Pure Python (easy installation)
- Comprehensive algorithm library
- NetworkX integration
- Active development
- Research-backed

What You Learned

This chapter covered the design philosophy behind py3plex:

Core principles:

- **Simple, off-the-shelf functionality** — work out of the box with minimal configuration
- **NetworkX compatibility** — leverage the entire NetworkX ecosystem
- **Modular architecture** — import only what you need

Implementation principles:

- **Flexibility** — multiple ways to accomplish tasks
- **Lazy evaluation** — expensive operations computed on demand
- **Graceful degradation** — optional features fail with helpful messages
- **Explicit over implicit** — clear, predictable behavior

Performance principles:

- **Sparse representations** — efficient for large networks
- **Sensible defaults** — good starting parameters based on research

Quality principles:

- **Comprehensive documentation** — every feature has examples
- **Extensive testing** — validated against published results
- **Reproducibility** — seed parameters for deterministic results

What's Next?

- *Algorithm Landscape* — Overview of available algorithms
- *Working with Networks* — Practical guide to network operations
- *Development Guide* — Contributing to py3plex

Academic References:

- Škrlj et al. (2019). “Py3plex toolkit for visualization and analysis of multilayer networks.” *Applied Network Science* 4(1): 94.

Next chapter: `:doc:`algorithm_landscape`` — a tour of py3plex's algorithmic capabilities.

8.2.5 Algorithm Landscape

This guide provides a narrative overview of the algorithm families available in py3plex, helping you understand when to use each type of algorithm for multilayer network analysis.

Overview

py3plex provides algorithms in seven main categories:

1. **Community Detection** - Finding groups of densely connected nodes
2. **Centrality & Importance** - Measuring node and edge importance
3. **Network Statistics** - Computing network-level and layer-specific metrics
4. **Random Walks & Embeddings** - Node representations and sampling
5. **Node Ranking & Classification** - Ordering and classifying nodes
6. **Visualization & Layout** - Rendering and spatial arrangement
7. **Benchmarking & Generation** - Testing and synthetic network creation

For detailed parameter lists and API reference, see *Algorithm Roadmap*.

Community Detection

Goal: Identify groups of nodes that are more densely connected to each other than to the rest of the network.

When to Use

Use community detection when you want to:

- Discover functional modules in biological networks
- Find social groups in multilayer social networks
- Identify topical clusters in citation networks
- Understand organizational structure
- Detect anomalies (nodes that don't fit any community)

Algorithm Families

Modularity-Based Methods

Best for: Fast community detection on large networks

Examples:

- **Louvain** - Fast, greedy modularity optimization
- **Leiden** - Improved version of Louvain with better stability
- **Multilayer Louvain** - Extends Louvain to multiple layers with inter-layer coupling

Trade-offs:

- ✓ Fast (scales to millions of nodes)
- ✓ Easy to use (minimal parameters)
- Resolution limit (may miss small communities)
- Stochastic (different runs give different results)

```
from py3plex.algorithms.community_detection import community_louvain

# Single-layer Louvain
partition = community_louvain.best_partition(network.core_network)

# Multilayer Louvain
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)
partition = louvain_multilayer(network, gamma=1.0, omega=1.0)
```

Flow-Based Methods

Best for: Finding communities based on information flow

Examples:

- **Infomap** - Minimizes description length of random walks
- **Walktrap** - Based on random walk distances

Trade-offs:

- ✓ Theoretically grounded (information theory)
- ✓ Finds hierarchical structure
- Slower than modularity methods
- May require external binaries

```
from py3plex.algorithms.community_detection import community_wrapper

# Infomap (requires binary)
partition = community_wrapper.infomap_communities(
    network.core_network,
    binary_path="/path/to/infomap"
)
```

Overlapping Community Detection

Best for: Networks where nodes belong to multiple communities

Examples:

- **NoRC** - Network of Ranked Communities
- **Clique Percolation** - Based on overlapping cliques

Trade-offs:

- ✓ More realistic for many real networks
- ✓ Captures multi-role nodes
- More complex to interpret
- Computationally expensive

```
from py3plex.algorithms.community_detection import community_wrapper

# NoRC overlapping communities
partition = community_wrapper.NoRC_communities(network, alpha=0.5)
```

Choosing a Community Detection Method

Use this decision tree:

```
Do you need overlapping communities?
├─ Yes → NoRC or Clique Percolation
└─ No
    └─ Is your network multilayer?
        ├── Yes → Multilayer Louvain or Leiden
        └─ No
            └─ What's most important?
                ├── Speed → Louvain
                ├── Stability → Leiden
                └─ Theory → Infomap
```

Centrality & Importance

Goal: Quantify the importance of nodes or edges in the network.

When to Use

Use centrality measures when you want to:

- Identify influential nodes (e.g., key proteins, opinion leaders)
- Find bottlenecks in flow networks
- Prioritize nodes for intervention or study
- Understand network vulnerability

- Analyze node roles across layers

Algorithm Families

Degree-Based Centrality

Measures: Local connectivity

Examples:

- **Degree Centrality** - Number of connections
- **Multilayer Degree** - Degree summed across layers
- **Versatility** - Participation across layers

When to use:

- Quick assessment of connectivity
- Identifying hubs
- Local importance measures

```
import networkx as nx

# Single-layer degree centrality
degree_cent = nx.degree_centrality(network.core_network)

# Multilayer versatility
from py3plex.algorithms.statistics import multilayer_statistics as mls
versatility = mls.versatility_centrality(network, centrality_type='degree')
```

Distance-Based Centrality

Measures: Global position in network

Examples:

- **Closeness Centrality** - Average distance to all other nodes
- **Betweenness Centrality** - Fraction of shortest paths through node
- **Harmonic Centrality** - Harmonic mean of distances

When to use:

- Identifying central nodes
- Finding information brokers
- Understanding information flow

```
# Betweenness centrality
betweenness = nx.betweenness_centrality(network.core_network)

# Closeness centrality
closeness = nx.closeness_centrality(network.core_network)
```

Eigenvector-Based Centrality

Measures: Importance based on connections to important nodes

Examples:

- **Eigenvector Centrality** - First eigenvector of adjacency matrix
- **PageRank** - Google's ranking algorithm
- **HITS** - Hubs and authorities

When to use:

- Ranking web pages or citations
- Finding influential nodes in social networks
- Authority/hub analysis

```
# PageRank
pagerank = nx.pagerank(network.core_network)

# Eigenvector centrality
eigenvector = nx.eigenvector_centrality(network.core_network)
```

Choosing a Centrality Measure

Consider these questions:

1. **Local vs. Global?** - Local → Degree centrality - Global → Closeness or betweenness
2. **Direct connections or influence?** - Direct → Degree - Influence → PageRank or eigenvector
3. **Network type?** - Directed → PageRank or HITS - Undirected → Any method - Weighted → Use weight-aware versions
4. **Computational cost?** - Large network → Degree or PageRank (fast) - Small network → Betweenness (expensive but informative)

Network Statistics

Goal: Characterize network structure at the global, layer, or node level.

When to Use

Use network statistics when you want to:

- Compare networks quantitatively
- Understand structural properties
- Validate network models
- Monitor network evolution
- Detect anomalies

Statistic Families

Global Statistics

Examples:

- Number of nodes, edges, layers
- Density, clustering coefficient
- Average path length
- Degree distribution

When to use: Basic network characterization

```
# Basic stats
network.basic_stats()

# Clustering
import networkx as nx
```

(continues on next page)

(continued from previous page)

```
clustering = nx.average_clustering(network.core_network)

# Density
from py3plex.algorithms.statistics import multilayer_statistics as mls
density = mls.layer_density(network, 'layer1')
```

Layer-Specific Statistics

Examples:

- Layer density
- Layer similarity (Jaccard, correlation)
- Edge overlap between layers
- Inter-layer degree correlation

When to use: Comparing layers, understanding layer relationships

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Layer density
density_l1 = mls.layer_density(network, 'layer1')

# Layer similarity
similarity = mls.layer_similarity(network, 'layer1', 'layer2', method='jaccard')

# Edge overlap
overlap = mls.edge_overlap(network, 'layer1', 'layer2')
```

Node-Level Statistics

Examples:

- Node activity (presence across layers)
- Participation coefficient
- Cross-layer degree correlation

When to use: Characterizing node behavior across layers

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Node activity
activity = mls.node_activity(network, 'Alice')

# Participation coefficient
participation = mls.community_participation_coefficient(network, partition)
```

Random Walks & Embeddings

Goal: Generate node representations or sample the network through traversal.

When to Use

Use random walks and embeddings when you want to:

- Create low-dimensional node representations for ML
- Sample large networks efficiently
- Measure node similarity

- Generate features for classification/clustering
- Understand structural equivalence

Algorithm Families

Random Walk Algorithms

Examples:

- **Basic Random Walk** - Uniform random traversal
- **Node2Vec** - Biased walks (BFS/DFS-like)
- **DeepWalk** - Uniform random walks for embeddings

Parameters:

- `walk_length` - Steps per walk
- `num_walks` - Walks per node
- `p` (Node2Vec) - Return parameter
- `q` (Node2Vec) - In-out parameter

```
from py3plex.algorithms.general.walkers import generate_walks, node2vec_walk

# Basic walks
walks = generate_walks(
    network.core_network,
    num_walks=10,
    walk_length=80
)

# Node2Vec walks (BFS-like: stay local)
walks_bfs = generate_walks(
    network.core_network,
    num_walks=10,
    walk_length=80,
    p=1.0, q=2.0 # BFS-like
)

# Node2Vec walks (DFS-like: explore)
walks_dfs = generate_walks(
    network.core_network,
    num_walks=10,
    walk_length=80,
    p=1.0, q=0.5 # DFS-like
)
```

Embedding Methods

Examples:

- **Node2Vec** - Skip-gram on random walks
- **DeepWalk** - Word2Vec on uniform walks
- **LINE** - First/second-order proximity preservation

When to use:

- Node classification
- Link prediction

- Visualization in 2D/3D
- Clustering
- Similarity computation

```
from py3plex.wrappers import node2vec_embedding

# Generate embeddings
embeddings = node2vec_embedding.generate_embeddings(
    network.core_network,
    dimensions=128,
    walk_length=80,
    num_walks=10,
    p=1.0,
    q=1.0
)

# Use embeddings for ML
# embeddings is a numpy array: (num_nodes, dimensions)
```

Choosing Walk Parameters

walk_length:

- Short (10-20): Fast, local neighborhood
- Medium (40-80): Balanced, standard choice
- Long (100+): Global structure, slower

p (return parameter):

- $p < 1$: More likely to return to previous node
- $p = 1$: Balanced
- $p > 1$: Less likely to return

q (in-out parameter):

- $q < 1$: DFS-like (explore distant nodes)
- $q = 1$: Balanced
- $q > 1$: BFS-like (stay in local neighborhood)

Visualization & Layout

Goal: Create visual representations of multilayer networks.

When to Use

Use visualization when you want to:

- Explore network structure
- Present findings
- Communicate patterns
- Identify visual anomalies
- Create publication-ready figures

Layout Families

Force-Directed Layouts

Examples:

- Spring layout
- Fruchterman-Reingold
- Force Atlas

Best for: General-purpose visualization, showing community structure

```
from py3plex.visualization.multilayer import hairball_plot

hairball_plot(
    network.core_network,
    layout_algorithm="force",
    layout_parameters={"iterations": 50}
)
```

Specialized Multilayer Layouts

Examples:

- Diagonal projection
- Layered stacking
- Circular layer arrangement

Best for: Emphasizing multilayer structure

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Diagonal layout for multilayer networks
draw_multilayer_default(
    [network],
    display=True,
    layout_style='diagonal'
)
```

Hierarchical Layouts

Examples:

- Reingold-Tilford tree
- Radial tree
- Sugiyama layered

Best for: Trees and DAGs

For complete visualization documentation, see [Visualization](#).

Benchmarking & Generation

Goal: Create synthetic networks for testing and validation.

When to Use

Use synthetic network generation when you want to:

- Test algorithm performance
- Validate implementations

- Create controlled experiments
- Generate training data
- Benchmark scalability

Generator Families

Random Network Models

Examples:

- **Erdős-Rényi** - Random edges with fixed probability
- **Barabási-Albert** - Preferential attachment (scale-free)
- **Watts-Strogatz** - Small-world networks

```
from py3plex.core import random_generators

# Random multilayer ER network
network = random_generators.random_multilayer_ER(
    num_nodes=100,
    num_layers=3,
    probability=0.05,
    directed=False
)
```

Benchmark Networks

Examples:

- LFR benchmark (with ground-truth communities)
- Configuration model (preserving degree distribution)
- Block model (with planted partition)

Algorithm Combinations

Common Workflows

Community Detection + Visualization:

```
# Detect communities
partition = louvain_multilayer(network)

# Visualize with community colors
from py3plex.visualization.multilayer import hairball_plot
from py3plex.visualization.colors import colors_default
from collections import Counter

top_communities = [c for c, _ in Counter(partition.values()).most_common(5)]
color_map = dict(zip(top_communities, colors_default[:5]))
node_colors = [color_map.get(partition.get(node), 'gray')
                for node in network.get_nodes()]

hairball_plot(network.core_network, node_colors, layout_algorithm="force")
```

Centrality + Ranking:

```
# Compute centrality
centrality = nx.betweenness_centrality(network.core_network)
```

(continues on next page)

(continued from previous page)

```
# Rank nodes
ranked = sorted(centrality.items(), key=lambda x: x[1], reverse=True)

# Top 10
print("Top 10 nodes:", ranked[:10])
```

Embeddings + Classification:

```
# Generate embeddings
embeddings = node2vec_embedding.generate_embeddings(network.core_network)

# Use for classification (with sklearn)
from sklearn.ensemble import RandomForestClassifier
clf = RandomForestClassifier()
# clf.fit(embeddings, labels)
# predictions = clf.predict(embeddings_test)
```

Performance Considerations**Algorithm Complexity**

Algorithm Family	Typical Complexity	Notes
Degree Centrality	$O(m)$	Fast, scales well
Betweenness	$O(nm)$ or $O(nm + n^2 \log n)$	Expensive for large networks
Louvain	$O(n \log n)$	Fast community detection
Node2Vec	$O(k \cdot l \cdot n)$	k =walks, l =length, n =nodes
Force Layout	$O(n^2)$	Slow for >1000 nodes

When working with large networks:

- Use approximate algorithms (e.g., sampling for betweenness)
- Consider layer-by-layer analysis
- Use sparse representations
- See *Performance and Scalability Best Practices*

Next Steps

- *Community Detection* - Detailed community detection guide
- *Network Statistics* - Complete statistics reference
- *Random Walk Algorithms* - Embeddings and walks
- *Visualization* - Visualization techniques
- *Algorithm Roadmap* - Detailed algorithm parameters and API

Academic References:

- Fortunato & Hric (2016). "Community detection in networks: A user guide." *Physics Reports* 659: 1-44.
- Grover & Leskovec (2016). "node2vec: Scalable feature learning for networks." *KDD*.
- Newman (2018). "Networks" (2nd ed.). Oxford University Press.

8.3 Part III: User Guide

Chapters 7–16: Mastering every feature.

With the fundamentals in place, this section provides comprehensive how-to guides for every major feature of py3plex. You'll learn how to create and manipulate multilayer networks, compute network statistics and centrality measures, detect communities across layers, perform random walks for embeddings, handle various input/output formats, create publication-ready visualizations, use the SQL-like DSL for intuitive queries, use the dplyr-style chainable API for data transformations, and combine these techniques into complete analysis workflows. Each chapter builds on the previous ones, creating a progressive learning path.

8.3.1 User Guide: From Understanding to Mastery

"Knowing is not enough; we must apply. Willing is not enough; we must do." — Johann Wolfgang von Goethe

The Heart of the Documentation

This section is the practical core of the py3plex documentation. While Part I got you started and Part II built your conceptual foundation, Part III is where you develop real competence.

Each chapter in this section covers a major capability of py3plex. They're designed to be read in order the first time through, building on each other. After that, they serve as reference material you can return to for specific tasks.

Chapter Overview

Working with Networks covers the fundamental operations: creating networks, loading data, querying structure, extracting subnetworks. This is the foundation for everything else.

Statistics teaches you how to measure what matters. Layer density, node activity, edge overlap, versatility—these metrics quantify the multilayer structure of your data. Understanding what they measure (and don't measure) is essential for valid analysis.

Community Detection addresses one of the most common analysis tasks: finding groups of nodes that naturally cluster together. In multilayer networks, this is more complex than single-layer community detection, and the algorithms offer parameters you need to understand.

Random Walks & Embeddings covers techniques for turning network structure into features for machine learning. Random walks are also fundamental to many other algorithms, so understanding them opens doors to advanced methods.

I/O and Formats deals with the practical matter of getting data in and out of py3plex. Real-world data comes in many formats, and you often need to export results for other tools or collaborators.

Visualization teaches you to create visual representations of your networks. Good visualizations reveal patterns that statistics miss and are essential for communicating findings.

DSL (Domain-Specific Language) introduces py3plex's SQL-like query language. For interactive exploration and quick analysis, the DSL is often faster than writing Python code.

Graph Operations covers the dplyr-style chainable API for data transformations. This modern interface makes complex operations readable and composable.

Recipes & Workflows provides ready-to-use solutions for common tasks. When you need to accomplish something specific, check here first—we've probably already solved it.

Case Studies demonstrates complete analyses from start to finish. These are the most valuable chapters for understanding how to apply py3plex to real problems.

How to Use This Section

If you're learning py3plex for the first time:

Read the chapters in order. Each builds on concepts from previous chapters, and the examples often reference earlier material. Plan to work through the code examples—reading code is not the same as running it.

If you're looking for a specific solution:

Check the Recipes & Workflows chapter first. If your task isn't there, look in the relevant topical chapter (e.g., Community Detection for clustering problems).

If you're deepening your expertise:

The Case Studies chapter shows how experts approach real problems. Pay attention not just to what they do, but why they make specific choices.

A Philosophy of Practice

We believe that proficiency comes from deliberate practice with feedback. This means:

1. **Run the code.** Don't just read it. Type it in (or copy-paste), run it, and look at the output.
2. **Modify and experiment.** After running an example, change something. What happens if you use a different parameter? A different algorithm? A different dataset?
3. **Apply to your own data.** The examples use sample data, but the goal is to apply these techniques to your own problems. Start with small subsets of your data and scale up.
4. **Reflect on what you learn.** When something surprises you, stop and understand why. These surprises are often where the deepest learning happens.

The Real-World Connection

Every technique in this section exists because it solves real problems. Community detection has helped researchers identify functional modules in protein interaction networks. Random walk embeddings have improved drug-drug interaction prediction. Visualization has revealed unexpected patterns in transportation networks.

As you work through these chapters, keep your own applications in mind. Ask yourself: How would I use this for my data? What questions could this help me answer?

Let's Go Deeper

Ready to develop real competence with py3plex? Start with *Working with Networks* to learn the fundamental operations.

Note

Progress Tracking:

As you work through this section, you might find it helpful to keep notes on:

- Techniques you've tried and how they worked
- Parameters that seem to work well for your data
- Questions that arise (these are great topics for the Community Detection or Statistics chapters)
- Ideas for your own analysis that you want to try later

8.3.2 Working with Networks

This guide covers everything you need to know about creating, loading, and manipulating multilayer networks in py3plex.

Creating Networks from Scratch

Method 1: Add Edges Directly

The simplest way to create a network is to add edges directly. Nodes are created automatically.

```

from py3plex.core import multinet

# Create empty network
network = multinet.multi_layer_network()

# Add edges in list format
# Format: [source_node, source_layer, target_node, target_layer, weight]
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Bob', 'friends', 'Carol', 'friends', 1.0],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1.0],
    ['Bob', 'colleagues', 'Dave', 'colleagues', 1.0]
], input_type="list")

# Verify
network.basic_stats()

```

Output:

```

Number of nodes: 6
Number of edges: 4
Number of unique nodes (as node-layer tuples): 6
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'friends': 3 nodes
  Layer 'colleagues': 3 nodes

```

Method 2: Add Nodes and Edges Separately

For more control, add nodes explicitly before adding edges:

```

from py3plex.core import multinet

network = multinet.multi_layer_network()

# Add nodes explicitly (as node-layer tuples)
network.add_nodes([
    ('Alice', 'friends'),
    ('Bob', 'friends'),
    ('Carol', 'friends'),
    ('Alice', 'colleagues'),
    ('Bob', 'colleagues'),
    ('Dave', 'colleagues')
])

# Add edges between existing nodes
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), 1.0],
    [('Bob', 'friends'), ('Carol', 'friends'), 1.0],
    [('Alice', 'colleagues'), ('Bob', 'colleagues'), 1.0],
    [('Bob', 'colleagues'), ('Dave', 'colleagues'), 1.0]
])

network.basic_stats()

```

Method 3: Define Layers First

You can define layers explicitly before adding content:

```
network = multinet.multi_layer_network()

# Define layers
network.add_layer('friends')
network.add_layer('colleagues')
network.add_layer('family')

# Check layers
print("Layers:", network.get_layers())

# Add edges (nodes created automatically)
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Alice', 'family', 'Carol', 'family', 1.0]
], input_type="list")
```

Output:

```
Layers: ['friends', 'colleagues', 'family']
```

Method 4: Build from NetworkX Graphs

Start with existing NetworkX graphs for each layer:

```
import networkx as nx
from py3plex.core import multinet

# Create layer graphs
friends = nx.Graph()
friends.add_edges_from([('Alice', 'Bob'), ('Bob', 'Carol')])

colleagues = nx.Graph()
colleagues.add_edges_from([('Alice', 'Bob'), ('Bob', 'Dave')])

# Combine into multilayer network
network = multinet.multi_layer_network()

# Add edges from each graph with layer label
for u, v in friends.edges():
    network.add_edges([[u, 'friends', v, 'friends', 1.0]], input_type="list")

for u, v in colleagues.edges():
    network.add_edges([[u, 'colleagues', v, 'colleagues', 1.0]], input_type="list")
```

Loading Networks from Files

py3plex supports multiple file formats for loading networks. See *I/O and Serialization* for complete I/O documentation.

From Simple Edge Lists

Format: Simple source target pairs (one per line)

File example (data.edgelist):

```
Alice Bob
Bob Carol
Carol Dave
```

Code:

```
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "data.edgelist",
    input_type="edgelist",
    directed=False
)

network.basic_stats()
```

Output:

```
Number of nodes: 4
Number of edges: 3
Number of unique nodes (as node-layer tuples): 4
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'null': 4 nodes
```

Note: Simple edge lists create a single layer named 'null' by default.

From Multilayer Edge Lists

Format: source target layer (space or tab-separated)

File example (data.multiedgelist):

```
Alice Bob friends
Bob Carol friends
Alice Bob colleagues
Bob Dave colleagues
```

Code:

```
network = multinet.multi_layer_network().load_network(
    "data.multiedgelist",
    input_type="multiedgelist",
    directed=False
)

network.basic_stats()
```

Output:

```
Number of nodes: 6
Number of edges: 4
Number of unique nodes (as node-layer tuples): 6
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'friends': 3 nodes
  Layer 'colleagues': 3 nodes
```

From GraphML

GraphML is an XML-based format that preserves node and edge attributes:

```
network = multinet.multi_layer_network().load_network(  
    "data.graphml",  
    input_type="graphml"  
)
```

From GML

Graph Modeling Language is another standard format:

```
network = multinet.multi_layer_network().load_network(  
    "data.gml",  
    input_type="gml"  
)
```

From NetworkX Pickle

NetworkX's native pickle format:

```
network = multinet.multi_layer_network().load_network(  
    "data.gpickle",  
    input_type="gpickle"  
)
```

From NetworkX Graphs

Load directly from a NetworkX graph object:

```
import networkx as nx  
from py3plex.core import multinet  
  
# Create or load a NetworkX graph  
G = nx.karate_club_graph()  
  
# Convert to py3plex  
network = multinet.multi_layer_network()  
network.load_network_from_networkx(G)  
  
# Result is a single-layer network  
network.basic_stats()
```

Modern I/O with Arrow/Parquet

For high-performance I/O, use the modern schema-based API. See *I/O and Serialization* for details.

```
from py3plex.io import read, write  
  
# Read (auto-detects format from extension)  
network = read('network.arrow') # or .parquet, .json  
  
# Write  
write(network, 'output.arrow')
```


Network Operations

Querying Network Elements

Get all nodes:

```
# Without attributes
nodes = network.get_nodes(data=False)
print("Nodes:", list(nodes)[:5])

# With attributes
nodes_with_data = network.get_nodes(data=True)
for node, attrs in list(nodes_with_data)[:3]:
    print(f"Node: {node}, Attributes: {attrs}")
```

Get nodes in a specific layer:

```
# Filter by layer
friends_nodes = network.get_nodes(layer='friends')
print("Friends layer nodes:", list(friends_nodes))
```

Get all edges:

```
# Without attributes
edges = network.get_edges(data=False)
print("Edges:", list(edges)[:5])

# With attributes
edges_with_data = network.get_edges(data=True)
for u, v, attrs in list(edges_with_data)[:3]:
    print(f"Edge: {u} -> {v}, Attributes: {attrs}")
```

Get neighbors:

```
# Get neighbors of a node in a specific layer
neighbors = list(network.get_neighbors('Alice', layer_id='friends'))
print(f"Alice's friends: {neighbors}")
```

Get layers:

```
layers = network.get_layers()
print(f"Layers: {layers}")
print(f"Number of layers: {len(layers)}")
```

Extracting Subnetworks

Extract a single layer:

```
# Get all nodes and edges in one layer
friends_subnet = network.subnetwork(['friends'], subset_by="layers")
friends_subnet.basic_stats()
```

Extract multiple layers:

```
# Combine multiple layers
social_subnet = network.subnetwork(['friends', 'family'], subset_by="layers")
```

Extract specific nodes (all layers):

```
# Get all appearances of specific nodes
alice_bob_subnet = network.subnetwork(['Alice', 'Bob'], subset_by="node_names")
alice_bob_subnet.basic_stats()
```

Extract specific node-layer pairs:

```
# Get specific node-layer combinations
specific_subnet = network.subnetwork(
    [('Alice', 'friends'), ('Bob', 'friends')],
    subset_by="node_layer_names"
)
specific_subnet.basic_stats()
```

Iterating Over Network Elements

Iterate over nodes:

```
# Simple iteration
for node in network.get_nodes():
    print(node) # Prints (node_id, layer_id) tuples

# With attributes
for node, attrs in network.get_nodes(data=True):
    print(f"{node}: {attrs}")
```

Iterate over edges:

```
# Simple iteration
for u, v in network.get_edges():
    print(f"{u} -> {v}")

# With attributes
for u, v, attrs in network.get_edges(data=True):
    print(f"{u} -> {v}: weight={attrs.get('weight', 1.0)}")
```

Iterate over layers:

```
for layer_name in network.get_layers():
    layer_subnet = network.subnetwork([layer_name], subset_by="layers")
    num_nodes = len(list(layer_subnet.get_nodes()))
    num_edges = len(list(layer_subnet.get_edges()))
    print(f"Layer {layer_name}: {num_nodes} nodes, {num_edges} edges")
```

Modifying Networks

Adding Elements

Add new nodes:

```
# Add single node
network.add_nodes([('Eve', 'friends')])

# Add multiple nodes
network.add_nodes([
    ('Eve', 'colleagues'),
    ('Frank', 'colleagues')
])
```

Add new edges:

```
# Add single edge
network.add_edges([
    ['Alice', 'friends', 'Eve', 'friends', 1.0]
], input_type="list")

# Add multiple edges
network.add_edges([
    ['Eve', 'colleagues', 'Frank', 'colleagues', 1.0],
    ['Bob', 'colleagues', 'Frank', 'colleagues', 1.0]
], input_type="list")
```

Add new layer:

```
network.add_layer('family')
```

Removing Elements

To remove nodes or edges, work with the underlying NetworkX graph:

```
# Access NetworkX graph
G = network.core_network

# Remove node (removes all edges)
G.remove_node(('Alice', 'friends'))

# Remove edge
G.remove_edge(('Bob', 'friends'), ('Carol', 'friends'))

# Remove multiple nodes
G.remove_nodes_from([('Alice', 'colleagues'), ('Bob', 'colleagues')])
```

Modifying Attributes**Node attributes:**

```
# Access NetworkX graph
G = network.core_network

# Set node attribute
G.nodes[('Alice', 'friends')]['age'] = 30
G.nodes[('Alice', 'friends')]['role'] = 'admin'

# Get node attribute
age = G.nodes[('Alice', 'friends')].get('age')
print(f"Alice's age: {age}")
```

Edge attributes:

```
# Set edge attribute
G[('Alice', 'friends')][('Bob', 'friends')]['weight'] = 0.8
G[('Alice', 'friends')][('Bob', 'friends')]['type'] = 'close'

# Get edge attribute
weight = G[('Alice', 'friends')][('Bob', 'friends')].get('weight')
print(f"Edge weight: {weight}")
```

Network Properties

Basic Statistics

```
# Get comprehensive stats
network.basic_stats()

# Output shows:
# - Total nodes
# - Total edges
# - Unique node-layer pairs
# - Unique node IDs
# - Nodes per layer
```

Manual counting:

```
# Count elements manually
num_nodes = len(list(network.get_nodes()))
num_edges = len(list(network.get_edges()))
num_layers = len(network.get_layers())

print(f"Nodes: {num_nodes}, Edges: {num_edges}, Layers: {num_layers}")
```

Network Type

```
# Check if directed
G = network.core_network
is_directed = G.is_directed()
print(f"Directed: {is_directed}")

# Check if multigraph
is_multi = G.is_multigraph()
print(f"Multigraph: {is_multi}")
```

Connectivity

```
import networkx as nx

G = network.core_network

# For undirected networks
if not G.is_directed():
    is_connected = nx.is_connected(G)
    num_components = nx.number_connected_components(G)
    print(f"Connected: {is_connected}")
    print(f"Number of components: {num_components}")

# For directed networks
else:
    is_weakly_connected = nx.is_weakly_connected(G)
    is_strongly_connected = nx.is_strongly_connected(G)
    print(f"Weakly connected: {is_weakly_connected}")
    print(f"Strongly connected: {is_strongly_connected}")
```

Matrix Representations

```
# Get supra-adjacency matrix
supra_adj = network.get_supra_adjacency_matrix(sparse=True)
print(f"Supra-adjacency shape: {supra_adj.shape}")

# For small networks, get dense version
if num_nodes < 1000:
    supra_adj_dense = network.get_supra_adjacency_matrix(sparse=False)
```

See *Chapter 4: The py3plex Core Model* for details on supra-adjacency matrices.

Pythonic Interface

py3plex networks support Python's standard protocols, making them feel natural to use.

Using len(), bool, and in

```
from py3plex.core import multinet

# Create a network
network = multinet.multi_layer_network()
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Bob', 'friends', 'Carol', 'friends', 1.0]
], input_type="list")

# Get number of nodes with len()
print(f"Number of nodes: {len(network)}") # Output: 3

# Use in boolean context
if network:
    print("Network has nodes")

# Check if empty
empty_net = multinet.multi_layer_network()
if not empty_net:
    print("Network is empty")

# Check node membership with 'in'
if ('Alice', 'friends') in network:
    print("Alice exists in friends layer")

# Check edge membership
if (('Alice', 'friends'), ('Bob', 'friends')) in network:
    print("Edge between Alice and Bob exists")
```

Output:

```
Number of nodes: 3
Network has nodes
Network is empty
Alice exists in friends layer
Edge between Alice and Bob exists
```

Property Accessors

Access common network metrics directly as properties:

```
from py3plex.core import multinet

network = multinet.multi_layer_network()
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1],
    ['B', 'layer2', 'C', 'layer2', 1]
], input_type="list")

# Property accessors - cleaner than method calls
print(f"Nodes: {network.node_count}")      # 4
print(f"Edges: {network.edge_count}")      # 2
print(f"Layers: {network.layer_count}")    # 2
print(f"Layer names: {network.layers}")    # ['layer1', 'layer2']
print(f"Is empty: {network.is_empty}")    # False
```

Output:

```
Nodes: 4
Edges: 2
Layers: 2
Layer names: ['layer1', 'layer2']
Is empty: False
```

Iterating Directly

Iterate over nodes directly using `for ... in`:

```
from py3plex.core import multinet

network = multinet.multi_layer_network()
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1]
], input_type="list")

# Iterate directly over nodes
for node in network:
    print(node)
```

Output:

```
('A', 'layer1')
('B', 'layer1')
```

Method Chaining

Build networks fluently with method chaining:

```
from py3plex.core import multinet

# Chain add_nodes and add_edges calls
network = (multinet.multi_layer_network()
    .add_nodes([{'source': 'A', 'type': 'layer1'}])
    .add_nodes([{'source': 'B', 'type': 'layer1'}])
    .add_edges([
```

(continues on next page)

(continued from previous page)

```

        'source': 'A', 'target': 'B',
        'source_type': 'layer1', 'target_type': 'layer1'
    ]])

print(f"Created network with {len(network)} nodes and {network.edge_count} edge")

```

Output:

```
Created network with 2 nodes and 1 edge
```

Factory Methods

Create networks in one line using factory methods:

```

from py3plex.core.multinet import multi_layer_network

# Create from edges directly
network = multi_layer_network.from_edges([
    {'source': 'A', 'target': 'B', 'source_type': 'l1', 'target_type': 'l1'},
    {'source': 'B', 'target': 'C', 'source_type': 'l1', 'target_type': 'l1'},
    {'source': 'A', 'target': 'C', 'source_type': 'l2', 'target_type': 'l2'}
])

print(f"Network: {network.node_count} nodes, {network.layer_count} layers")

# Create from list format
network2 = multi_layer_network.from_edges([
    ['X', 'layer1', 'Y', 'layer1', 1],
    ['Y', 'layer1', 'Z', 'layer1', 1]
], input_type='list', directed=False)

```

Output:

```
Network: 4 nodes, 2 layers
```

Create from NetworkX

Convert existing NetworkX graphs:

```

import networkx as nx
from py3plex.core.multinet import multi_layer_network

# Create a NetworkX graph with multilayer nodes
G = nx.Graph()
G.add_edge(('A', 'layer1'), ('B', 'layer1'))
G.add_edge(('B', 'layer1'), ('C', 'layer1'))

# Convert to py3plex
network = multi_layer_network.from_networkx(G)
print(f"Converted: {len(network)} nodes")

```

Output:

```
Converted: 3 nodes
```

Complete Example

Here's a complete example combining all the Pythonic features:

```
from py3plex.core.multinet import multi_layer_network

# Create network using factory method
network = multi_layer_network.from_edges([
    {'source': 'Alice', 'target': 'Bob',
     'source_type': 'friends', 'target_type': 'friends'},
    {'source': 'Bob', 'target': 'Carol',
     'source_type': 'friends', 'target_type': 'friends'},
    {'source': 'Alice', 'target': 'Bob',
     'source_type': 'work', 'target_type': 'work'}
])

# Check network state
if network:
    print(f"Network has {len(network)} nodes across {network.layer_count} layers")
    print(f"Layers: {network.layers}")

# Check membership
if ('Alice', 'friends') in network:
    print("Alice is in the friends layer")

# Iterate over nodes
print("\nAll nodes:")
for node in network:
    print(f"  {node}")

# Add more data with chaining
network.add_edges([
    {'source': 'Carol', 'target': 'Dave',
     'source_type': 'friends', 'target_type': 'friends'}
]).add_nodes([{'source': 'Eve', 'type': 'work'}])

print(f"\nAfter additions: {network.node_count} nodes, {network.edge_count} edges")
```

Output:

```
Network has 4 nodes across 2 layers
Layers: ['friends', 'work']
Alice is in the friends layer

All nodes:
('Alice', 'friends')
('Bob', 'friends')
('Carol', 'friends')
('Alice', 'work')
('Bob', 'work')

After additions: 6 nodes, 4 edges
```

NetworkX Integration

Direct Access

The `core_network` attribute gives you direct access to the NetworkX graph:


```
import networkx as nx

# Get NetworkX graph
G = network.core_network

# Use any NetworkX function
degree = dict(G.degree())
betweenness = nx.betweenness centrality(G)
clustering = nx.clustering(G)

print(f"Average clustering: {nx.average_clustering(G):.3f}")
```

NetworkX Wrappers

py3plex provides convenience wrappers for common NetworkX functions:

```
# Extract a layer first
friends_layer = network.subnetwork(['friends'], subset_by="layers")

# Apply NetworkX algorithms via wrapper
degree Centrality = friends_layer.monoplex_nx_wrapper("degree Centrality")
betweenness = friends_layer.monoplex_nx_wrapper("betweenness Centrality")
pagerank = friends_layer.monoplex_nx_wrapper("pagerank")

# Results are dictionaries
top_degree = sorted(degree Centrality.items(), key=lambda x: x[1], reverse=True)[:5]
print("Top 5 by degree:", top_degree)
```

Converting to NetworkX

Since py3plex networks ARE NetworkX graphs, conversion is trivial:

```
# Export to standard NetworkX formats
import networkx as nx

G = network.core_network

# Save as GraphML
nx.write_graphml(G, "output.graphml")

# Save as GML
nx.write_gml(G, "output.gml")

# Save as pickle
nx.write_gpickle(G, "output.gpickle")
```

See *I/O and Serialization* for more export options.

Best Practices

File Organization

```
# Use absolute paths or path relative to script
import os

# Get directory of current script
```

(continues on next page)

(continued from previous page)

```

script_dir = os.path.dirname(os.path.abspath(__file__))
data_dir = os.path.join(script_dir, "data")
network_path = os.path.join(data_dir, "network.edgelist")

# Load
network = multinet.multi_layer_network().load_network(
    network_path,
    input_type="edgelist"
)

```

Always Verify After Loading

```

# Load network
network = multinet.multi_layer_network().load_network(...)

# ALWAYS check stats
network.basic_stats()

# Verify it matches expectations
assert len(network.get_layers()) == 3, "Expected 3 layers"
assert len(list(network.get_nodes())) > 0, "Network is empty!"

```

Use Appropriate Data Structures

```

# For large networks, use sparse matrices
supra_adj = network.get_supra_adjacency_matrix(sparse=True) # Good

# Avoid dense matrices for large networks
# supra_adj = network.get_supra_adjacency_matrix(sparse=False) # Bad for n > 1000

```

Common Patterns

Pattern 1: Load, Analyze, Visualize

```

from py3plex.core import multinet
from py3plex.visualization.multilayer import draw_multilayer_default

# Load
network = multinet.multi_layer_network().load_network(
    "data.multiedgelist", input_type="multiedgelist"
)

# Analyze
network.basic_stats()

# Visualize
draw_multilayer_default([network], display=True)

```

Pattern 2: Create from Multiple Sources

```

# Combine data from multiple files
network = multinet.multi_layer_network()

# Load friends from file

```

(continues on next page)

(continued from previous page)

```
import pandas as pd
friends_df = pd.read_csv("friends.csv")
for _, row in friends_df.iterrows():
    network.add_edges([[row['source'], 'friends', row['target'], 'friends', 1]],
                      input_type="list")

# Load colleagues from database (pseudocode)
# colleagues = query_database("SELECT * FROM colleagues")
# for source, target in colleagues:
#     network.add_edges([[source, 'colleagues', target, 'colleagues', 1]],
#                       input_type="list")
```

Pattern 3: Layer-by-Layer Analysis

```
# Analyze each layer separately
for layer_name in network.get_layers():
    print(f"\n=== Layer: {layer_name} ===")

    # Extract layer
    layer_subnet = network.subnetwork([layer_name], subset_by="layers")

    # Compute metrics
    layer_subnet.basic_stats()

    # Optional: visualize
    # draw_multilayer_default([layer_subnet], display=True)
```

Next Steps

- *Network Statistics* - Computing network metrics
- *Community Detection* - Finding communities
- *I/O and Serialization* - Complete I/O documentation
- *Visualization* - Visualization techniques
- *Chapter 4: The py3plex Core Model* - Understanding the internal representation

Related Examples:

- `example_multilayer_functionality.py` - Core operations
- `example_IO.py` - Loading and saving
- `example_manipulation.py` - Network manipulation
- `example_networkx_wrapper.py` - NetworkX integration

Repository: <https://github.com/SkBlaz/py3plex/tree/master/examples>

8.3.3 Network Statistics

How to measure and compare the structure of your multilayer network.

Once you've loaded a multilayer network, the next question is: *what does it look like?* This chapter shows you how to compute statistics that answer fundamental questions about your network's structure:

- **How dense is each layer?** Sparse layers may indicate missing data or genuine sparsity.
- **Who participates in multiple contexts?** High-activity nodes may be bridges or hubs.

- **How similar are your layers?** Similar layers might be redundant; dissimilar layers reveal complementary information.
- **What patterns emerge across layers?** Metrics like edge overlap and inter-layer degree correlation reveal cross-layer structure.

py3plex provides three levels of network statistics:

1. **Global Statistics** — Whole network properties (density, clustering, etc.)
2. **Layer-Specific Statistics** — Per-layer measures and comparisons
3. **Node-Level Statistics** — Node activity and participation across layers

For centrality measures (degree, betweenness, PageRank, etc.), see [Algorithm Landscape](#).

Basic Network Statistics

Quick Stats

The fastest way to get basic information:

```
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "data.multiedgelist", input_type="multiedgelist"
)

# Display comprehensive stats
network.basic_stats()
```

Output:

```
Number of nodes: 184
Number of edges: 1691
Number of unique nodes (as node-layer tuples): 184
Number of unique node IDs (across all layers): 46
Nodes per layer:
  Layer '1': 46 nodes
  Layer '2': 46 nodes
  Layer '3': 46 nodes
  Layer '4': 46 nodes
```

Manual Counting

```
# Count elements
num_nodes = len(list(network.get_nodes()))
num_edges = len(list(network.get_edges()))
num_layers = len(network.get_layers())

# Unique node IDs (across all layers)
unique_nodes = set()
for node, layer in network.get_nodes():
    unique_nodes.add(node)
num_unique_nodes = len(unique_nodes)

print(f"Nodes (node-layer pairs): {num_nodes}")
print(f"Edges: {num_edges}")
```

(continues on next page)

(continued from previous page)

```
print(f"Layers: {num_layers}")
print(f"Unique node IDs: {num_unique_nodes}")
```

Layer-Specific Statistics

The `multilayer_statistics` module provides comprehensive statistics:

```
from py3plex.algorithms.statistics import multilayer_statistics as mls
```

Layer Density

Definition: Fraction of possible edges that exist in a layer.

Formula: $density = \frac{2m}{n(n-1)}$ for undirected graphs

Use case: Measure how connected a layer is.

```
# Density of individual layers
density_layer1 = mls.layer_density(network, 'layer1')
density_layer2 = mls.layer_density(network, 'layer2')

print(f"Layer 1 density: {density_layer1:.4f}")
print(f"Layer 2 density: {density_layer2:.4f}")
```

Interpretation:

- 0.0 = No edges (empty layer)
- 1.0 = Complete graph (all possible edges exist)
- Typical real-world networks: 0.001 - 0.1

Layer Similarity

Definition: How similar two layers are in structure.

Methods: Jaccard index, Pearson correlation, cosine similarity

Use case: Identify redundant or complementary layers.

```
# Jaccard similarity (based on edges)
jaccard = mls.layer_similarity(
    network, 'layer1', 'layer2', method='jaccard'
)

# Pearson correlation
pearson = mls.layer_similarity(
    network, 'layer1', 'layer2', method='pearson'
)

print(f"Jaccard similarity: {jaccard:.4f}")
print(f"Pearson correlation: {pearson:.4f}")
```

Interpretation:

- 1.0 = Identical layers
- 0.0 = Completely different
- Negative values (Pearson) = Anti-correlated

Edge Overlap

Definition: Fraction of edges that appear in both layers.

Use case: Measure redundancy between layers.

```
overlap = mls.edge_overlap(network, 'layer1', 'layer2')
print(f"Edge overlap: {overlap:.4f}")
```

Interpretation:

- 1.0 = All edges in common
- 0.0 = No edges in common

Inter-Layer Degree Correlation

Definition: Correlation between node degrees in different layers.

Use case: Determine if “hubs” in one layer are hubs in another.

```
correlation = mls.inter_layer_degree_correlation(
    network, 'layer1', 'layer2'
)
print(f"Degree correlation: {correlation:.4f}")
```

Interpretation:

- 1.0 = Perfect positive correlation (hubs in layer1 are hubs in layer2)
- 0.0 = No correlation
- -1.0 = Perfect negative correlation

Node-Level Statistics

Node Activity

Definition: Fraction of layers in which a node appears.

Use case: Identify nodes that participate in multiple contexts.

```
# Activity for a single node
activity_alice = mls.node_activity(network, 'Alice')
print(f"Alice's activity: {activity_alice:.4f}")

# Compute for all nodes
all_activities = {}
unique_nodes = set(node for node, layer in network.get_nodes())
for node in unique_nodes:
    all_activities[node] = mls.node_activity(network, node)

# Top 5 most active nodes
top_active = sorted(all_activities.items(), key=lambda x: x[1], reverse=True)[:5]
print("Most active nodes:", top_active)
```

Interpretation:

- 1.0 = Node appears in all layers
- 0.5 = Node appears in half of layers
- Close to 0.0 = Node appears in few layers

Versatility Centrality

Definition: Node importance considering activity across layers.

Use case: Find nodes that are important across multiple layers.

```
# Versatility based on degree
versatility_degree = mls.versatility_centrality(
    network, centrality_type='degree'
)

# Versatility based on betweenness
versatility_betweenness = mls.versatility_centrality(
    network, centrality_type='betweenness'
)

# Top versatile nodes
top_versatile = sorted(
    versatility_degree.items(),
    key=lambda x: x[1],
    reverse=True
)[:10]
print("Top 10 versatile nodes:", top_versatile)
```

Interpretation:

- Higher values = More important across multiple layers
- Combines centrality within layers with cross-layer participation

Participation Coefficient

Definition: Measures how evenly a node's connections are distributed across layers.

Use case: Identify nodes that bridge different layers.

```
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)

# Detect communities first
partition = louvain_multilayer(network)

# Compute participation coefficient
participation = mls.community_participation_coefficient(network, partition)

# Top bridging nodes
top_bridging = sorted(
    participation.items(),
    key=lambda x: x[1],
    reverse=True
)[:10]
print("Top bridging nodes:", top_bridging)
```

Interpretation:

- 1.0 = Connections evenly distributed across layers
- 0.0 = All connections in one layer

Network-Level Statistics

Entropy of Multiplexity

Definition: Measures layer diversity (how evenly nodes/edges are distributed across layers).

Use case: Quantify structural diversity of the multilayer network.

```
entropy = mls.entropy_of_multiplexity(network)
print(f"Entropy of multiplexity: {entropy:.4f} bits")
```

Interpretation:

- 0.0 = All activity in one layer (no diversity)
- Higher values = More evenly distributed across layers
- Maximum = $\log_2(\text{number of layers})$

Algebraic Connectivity

Definition: Second smallest eigenvalue of the Laplacian matrix.

Use case: Measure network robustness (higher = more robust).

```
algebraic_conn = mls.algebraic_connectivity(network, 'layer1')
print(f"Algebraic connectivity: {algebraic_conn:.4f}")
```

Interpretation:

- 0.0 = Disconnected network
- Higher values = Better connectivity and robustness

Multilayer Clustering Coefficient

Definition: Extension of clustering coefficient to multilayer networks.

Use case: Measure local cohesion across layers.

```
clustering = mls.multilayer_clustering_coefficient(network)
print(f"Multilayer clustering: {clustering:.4f}")
```

Interpretation:

- 1.0 = Every node's neighbors form a clique
- 0.0 = No clustering (tree-like structure)

Using NetworkX Statistics

Since py3plex networks are NetworkX graphs, you can use any NetworkX statistic:

Basic NetworkX Metrics

```
import networkx as nx

G = network.core_network

# Clustering coefficient
clustering = nx.average_clustering(G)
print(f"Average clustering: {clustering:.4f}")

# Transitivity
```

(continues on next page)

(continued from previous page)

```

transitivity = nx.transitivity(G)
print(f"Transitivity: {transitivity:.4f}")

# Density
density = nx.density(G)
print(f"Density: {density:.4f}")

```

Degree Distribution

```

import networkx as nx
from collections import Counter

G = network.core_network

# Get degree sequence
degrees = [d for n, d in G.degree()]

# Degree distribution
degree_dist = Counter(degrees)
print("Degree distribution:", dict(sorted(degree_dist.items())))

# Average degree
avg_degree = sum(degrees) / len(degrees)
print(f"Average degree: {avg_degree:.2f}")

```

Path-Based Statistics

```

import networkx as nx

G = network.core_network

# Check if connected first
if nx.is_connected(G.to_undirected()):
    # Average shortest path length
    avg_path = nx.average_shortest_path_length(G)
    print(f"Average shortest path: {avg_path:.2f}")

    # Diameter
    diameter = nx.diameter(G)
    print(f"Diameter: {diameter}")
else:
    print("Network is not connected")

# Largest component
largest_cc = max(nx.connected_components(G.to_undirected()), key=len)
largest_size = len(largest_cc)
print(f"Largest component: {largest_size} nodes")

```

Statistical Comparison

Comparing Networks

Compare two multilayer networks:

```

from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Load two networks
network1 = multinet.multi_layer_network().load_network("data1.multiedgelist")
network2 = multinet.multi_layer_network().load_network("data2.multiedgelist")

# Compare basic stats
print("Network 1:")
network1.basic_stats()

print("\nNetwork 2:")
network2.basic_stats()

# Compare layer densities (if same layers)
for layer in network1.get_layers():
    if layer in network2.get_layers():
        density1 = mls.layer_density(network1, layer)
        density2 = mls.layer_density(network2, layer)
        print(f"{layer}: {density1:.4f} vs {density2:.4f}")

```

Layer-by-Layer Analysis

Systematic comparison of all layers:

```

import pandas as pd
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Collect stats for each layer
layer_stats = []
for layer in network.get_layers():
    # Extract layer
    layer_subnet = network.subnetwork([layer], subset_by="layers")
    G_layer = layer_subnet.core_network

    # Compute stats
    stats = {
        'layer': layer,
        'nodes': G_layer.number_of_nodes(),
        'edges': G_layer.number_of_edges(),
        'density': mls.layer_density(network, layer),
        'clustering': nx.average_clustering(G_layer)
    }
    layer_stats.append(stats)

# Display as table
df = pd.DataFrame(layer_stats)
print(df.to_string(index=False))

```

Output:

layer	nodes	edges	density	clustering
1	46	143	0.1384	0.4521
2	46	139	0.1346	0.4123
3	46	136	0.1317	0.3892
4	46	134	0.1298	0.3756

Exporting Statistics

Save to CSV

```
import pandas as pd

# Collect node statistics
node_stats = []
unique_nodes = set(node for node, layer in network.get_nodes())
for node in unique_nodes:
    stats = {
        'node': node,
        'activity': mls.node_activity(network, node),
        # Add more stats as needed
    }
    node_stats.append(stats)

# Save
df = pd.DataFrame(node_stats)
df.to_csv("node_statistics.csv", index=False)
```

Save to JSON

```
import json

# Collect statistics
stats = {
    'num_nodes': len(list(network.get_nodes())),
    'num_edges': len(list(network.get_edges())),
    'num_layers': len(network.get_layers()),
    'layers': {}
}

# Layer stats
for layer in network.get_layers():
    stats['layers'][layer] = {
        'density': float(mls.layer_density(network, layer)),
        # Add more stats
    }

# Save
with open("network_stats.json", 'w') as f:
    json.dump(stats, f, indent=2)
```

Best Practices

1. Always check basic stats first

```
network.basic_stats() # Before doing any analysis
```

2. Extract layers for layer-specific analysis

```
layer1 = network.subnetwork(['layer1'], subset_by="layers")
# Now apply NetworkX functions
```

3. Cache expensive computations

```
# Compute once
versatility = mls.versatility_centrality(network, centrality_type='degree')

# Reuse
top_nodes = sorted(versatility.items(), key=lambda x: x[1], reverse=True)
```

4. Handle edge cases

```
# Check for empty layers
layer_subnet = network.subnetwork(['layer1'], subset_by="layers")
if len(list(layer_subnet.get_edges())) == 0:
    print("Layer is empty, skipping...")
else:
    density = mls.layer_density(network, 'layer1')
```

Complete Example

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
import networkx as nx

# Load network
network = multinet.multi_layer_network().load_network(
    "data.multiedgelist", input_type="multiedgelist"
)

print("=== Basic Statistics ===")
network.basic_stats()

print("\n=== Layer Statistics ===")
for layer in network.get_layers():
    density = mls.layer_density(network, layer)
    print(f"{layer}: density = {density:.4f}")

print("\n=== Node Activity ===")
unique_nodes = set(node for node, layer in network.get_nodes())
activities = {node: mls.node_activity(network, node) for node in unique_nodes}
top_active = sorted(activities.items(), key=lambda x: x[1], reverse=True)[:5]
for node, activity in top_active:
    print(f"{node}: {activity:.4f}")

print("\n=== Layer Similarity ===")
layers = network.get_layers()
if len(layers) >= 2:
    similarity = mls.layer_similarity(
        network, layers[0], layers[1], method='jaccard'
    )
    print(f"{layers[0]} vs {layers[1]}: {similarity:.4f}")

print("\n=== Global Metrics ===")
entropy = mls.entropy_of_multiplexity(network)
print(f"Entropy of multiplexity: {entropy:.4f} bits")
```

What You Learned

This chapter covered the statistical measures available for multilayer networks:

Basic network statistics:

- `basic_stats()` for quick overview of nodes, edges, and layers
- Manual counting with `get_nodes()`, `get_edges()`, `get_layers()`

Layer-specific statistics:

- Layer density with `layer_density()`
- Layer similarity with `layer_similarity()` (Jaccard, Pearson)
- Edge overlap with `edge_overlap()`
- Inter-layer degree correlation with `inter_layer_degree_correlation()`

Node-level statistics:

- Node activity with `node_activity()` — what fraction of layers does a node appear in?
- Versatility centrality with `versatility_centrality()` — importance across layers
- Participation coefficient with `community_participation_coefficient()`

Network-level statistics:

- Entropy of multiplexity — how evenly distributed is the structure?
- Algebraic connectivity — how robust is the network?
- Multilayer clustering coefficient — local cohesion across layers

Integration:

- All NetworkX statistics work directly on `network.core_network`
- Extract layers with `subnetwork()` for layer-specific analysis

What's Next?

- *Community Detection* — Finding communities that span layers
- *Visualization* — Visualizing statistics and network structure
- *Algorithm Landscape* — Overview of all algorithms
- *Algorithm Roadmap* — Complete API reference

Related Examples:

- `example_multilayer_statistics.py` — Statistical analysis examples
- `example_layer_comparison.py` — Comparing layers
- `example_node_metrics.py` — Node-level metrics

Repository: <https://github.com/SkBlaz/py3plex/tree/master/examples>

8.3.4 Community Detection

Finding groups that span multiple layers of interaction.

Networks are rarely homogeneous. People cluster into social groups. Proteins form functional modules. Cities organize into regional hubs. **Community detection** finds these natural groupings—but in multilayer networks, the question is more subtle: *do communities exist within layers, across layers, or both?*

This chapter shows you how to detect communities in multilayer networks, how to tune algorithm parameters for your specific domain, and how to interpret results that may differ from single-layer community detection.

Overview

Community detection identifies **groups of nodes** that are more **densely connected** to each other than to the rest of the network. For **multilayer networks**, communities can **span multiple layers**, accounting for both **intra-layer** and **inter-layer** structure.

Key insight: A person who is moderately connected across multiple social platforms (layers) may be more central to a cross-platform community than someone who is highly connected on just one platform. Multilayer community detection captures this.

Supported Algorithms

py3plex provides **several community detection algorithms**:

- **Louvain** — **Fast modularity optimization** (recommended for most use cases)
 - **Infomap** — **Flow-based** community detection (requires external binary)
 - **Label Propagation** — **Semi-supervised** approach with known seed communities
 - **Multilayer Modularity** — **True multilayer** community detection (Mucha et al. 2010)
-

Louvain Algorithm

Basic Usage

Fastest algorithm for large networks:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection import community_louvain

# Create or load network
network = multinet.multi_layer_network()
network.load_network("data.graphml", input_type="graphml")

# Detect communities using Louvain
communities = community_louvain.best_partition(network.core_network)

# Print results
for node, community_id in communities.items():
    print(f"Node {node} -> Community {community_id}")
```

Parameters

```
# With custom resolution parameter
communities = community_louvain.best_partition(
    network.core_network,
    resolution=1.0 # Higher = more communities, Lower = fewer communities
)
```

Resolution parameter:

- **resolution=1.0** - **Standard modularity**
- **resolution>1.0** - **More, smaller communities**
- **resolution<1.0** - **Fewer, larger communities**

Advantages

- **Very fast:** $O(n \log n)$
- **Scales to millions** of nodes
- **BSD license** (commercial-friendly)
- **Well-established** and widely used

Disadvantages

- **Non-deterministic** (random initialization)
- **Cannot find overlapping** communities
- **Resolution limit** issues

Infomap Algorithm

Basic Usage

Flow-based approach for detecting communities:

```
from py3plex.algorithms.community_detection import community_wrapper

# Detect communities using Infomap
communities = community_wrapper.infomap_communities(
    network.core_network,
    binary_path="/path/to/infomap" # Path to Infomap binary
)
```

With Hierarchical Structure

```
# Get hierarchical community structure
hierarchical_communities = community_wrapper.infomap_communities(
    network.core_network,
    binary_path="/path/to/infomap",
    hierarchical=True
)
```

Advantages

- Can detect overlapping communities
- Flow-based (natural for many applications)
- Hierarchical structure
- Information-theoretic foundation

Disadvantages

- Requires external binary
- AGPLv3 license (viral copyleft - problematic for commercial use)
- Slower than Louvain

Label Propagation

Semi-Supervised Detection

Use when you have some known community memberships:

```
from py3plex.algorithms.community_detection import label_propagation

# Provide seed labels for some nodes
seed_labels = {
    'node1': 0,
    'node2': 0,
    'node3': 1,
    'node4': 1
}

# Propagate labels to unlabeled nodes
communities = label_propagation.propagate(
    network.core_network,
    seed_labels=seed_labels,
    max_iter=100
)
```

Fully Unsupervised

```
# Without seed labels (random initialization)
communities = label_propagation.propagate(
    network.core_network,
    max_iter=100
)
```

Advantages

- Very fast: $O(m)$ linear in edges
- Can incorporate prior knowledge
- MIT license
- Simple and interpretable

Disadvantages

- Non-deterministic
- Sensitive to initialization
- May not converge
- Lower quality than Louvain/Infomap

Multilayer Modularity

True Multilayer Detection

Accounts for multilayer structure following Mucha et al. (2010):

```
from py3plex.algorithms.community_detection import multilayer_modularity as mlm

# Get supra-adjacency matrix
supra_adj = network.get_supra_adjacency_matrix(sparse=True)
```

(continues on next page)

(continued from previous page)

```
# Detect communities with multilayer modularity
communities = mlm.multilayer_louvain(
    supra_adj,
    gamma=1.0,      # Resolution parameter
    omega=1.0       # Inter-layer coupling strength
)
```

Parameter Tuning

```
# Emphasize layer-specific structure
communities = mlm.multilayer_louvain(
    supra_adj,
    gamma=1.0,
    omega=0.1 # Low coupling = layer-specific communities
)

# Emphasize cross-layer structure
communities = mlm.multilayer_louvain(
    supra_adj,
    gamma=1.0,
    omega=10.0 # High coupling = cross-layer communities
)
```

Mathematical Formulation

Multilayer modularity is defined as:

$$Q^{ML} = \frac{1}{2\mu} \sum_{ij\alpha\beta} \left[(A_{ij}^{[\alpha]} - \gamma_{\alpha} P_{ij}^{[\alpha]}) \delta_{\alpha\beta} + \omega_{\alpha\beta} \delta_{ij} \right] \delta(g_i^{[\alpha]}, g_j^{[\beta]})$$

Where:

- $A_{ij}^{[\alpha]}$ is the adjacency matrix of layer α
- $P_{ij}^{[\alpha]}$ is the null model (e.g., configuration model)
- γ_{α} is the resolution parameter for layer α
- $\omega_{\alpha\beta}$ is the coupling strength between layers
- $\delta(g_i^{[\alpha]}, g_j^{[\beta]})$ is 1 if nodes are in the same community, 0 otherwise

Advantages

- Accounts for multilayer structure
- Implements state-of-the-art algorithm
- Configurable inter-layer coupling
- Published in Science (Mucha et al. 2010)

Disadvantages

- More computationally expensive
- Requires parameter tuning
- May not scale to very large networks (>100k nodes)

Evaluating Community Quality

Modularity Score

```
import networkx as nx

# Compute modularity
modularity = nx.community.modularity(network.core_network, communities)
print(f"Modularity: {modularity:.3f}")
```

Interpretation:

- $Q > 0.3$: Good community structure
- $Q > 0.5$: Strong community structure
- $Q < 0.3$: Weak or no community structure

Coverage and Performance

```
# Coverage: fraction of edges within communities
coverage = nx.community.coverage(network.core_network, communities)

# Performance: fraction of correctly classified node pairs
performance = nx.community.performance(network.core_network, communities)

print(f"Coverage: {coverage:.3f}")
print(f"Performance: {performance:.3f}")
```

Visualizing Communities

Color by Community

```
from py3plex.visualization.multilayer import hairball_plot
import matplotlib.pyplot as plt

# Map communities to colors
node_colors = [communities.get(node, 0) for node in network.core_network.nodes()]

# Visualize with community colors
hairball_plot(
    network.core_network,
    node_color=node_colors,
    layout_algorithm='force',
    cmap='tab10'
)
plt.show()
```

Community Size Distribution

```
from collections import Counter
import matplotlib.pyplot as plt

# Count community sizes
community_sizes = Counter(communities.values())
sizes = list(community_sizes.values())

# Plot distribution
```

(continues on next page)

(continued from previous page)

```
plt.hist(sizes, bins=20)
plt.xlabel('Community Size')
plt.ylabel('Frequency')
plt.title('Community Size Distribution')
plt.show()
```

Understanding Single-Layer vs. Multilayer Community Detection

Before diving into algorithm specifics, it's important to understand the conceptual differences between approaches.

Single-Layer Community Detection

Traditional community detection finds groups in a single graph. Applied to a multilayer network, you have two options:

1. **Flatten and detect:** Aggregate all layers into one graph, then find communities. This loses layer information.
2. **Detect per layer:** Find communities independently in each layer. This ignores cross-layer structure.

Neither captures the full multilayer picture.

Multilayer Community Detection

Multilayer algorithms find communities that are consistent across layers while respecting layer-specific structure. They ask: “Which nodes cluster together across multiple contexts?”

Key insight: A node that is moderately connected in many layers may be more “community-central” than a node highly connected in just one layer.

Overlapping vs. Non-Overlapping Communities

Non-overlapping: Each node belongs to exactly one community. Algorithms like Louvain and Leiden produce non-overlapping partitions.

Overlapping: Nodes can belong to multiple communities. Algorithms like NoRC and clique percolation find overlapping structure.

When to use overlapping: When nodes naturally belong to multiple groups (e.g., a person in both a work community and a hobby community).

Flow-Based vs. Modularity-Based Views

Modularity-based (Louvain, Leiden): Optimize a quality function that compares edge density within communities to expected density. Fast, widely used, but has resolution limit issues.

Flow-based (Infomap): Model random walks on the network and find community structure that minimizes description length of those walks. Theoretically grounded, finds hierarchical structure, but slower.

When to use which:

- Use **modularity-based** for speed and when you don't need hierarchical structure
- Use **flow-based** when you care about information flow or want to find nested communities

Parameter Tuning Cookbook

Tuning Resolution (Gamma)

The resolution parameter controls community size:

```

from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)

# Experiment with different resolution values
for gamma in [0.5, 1.0, 1.5, 2.0]:
    partition = louvain_multilayer(network, gamma=gamma, omega=1.0, random_state=42)
    num_comms = len(set(partition.values()))
    print(f"gamma={gamma}: {num_comms} communities")

```

Interpretation guide:

- **Very few communities (2-5) when you expect more:** is too low → increase
- **Many singleton communities:** is too high → decrease
- **One giant community + many tiny ones:** Resolution limit problem → try > 1 or use Leiden

Recommended starting procedure:

1. Start with $=1.0$ (standard modularity)
2. Look at community size distribution
3. If too coarse, try $=1.5, 2.0$
4. If too fine, try $=0.5, 0.25$

Tuning Inter-Layer Coupling (Omega)

The coupling parameter controls how much layers influence each other:

```

# Experiment with different coupling values
for omega in [0.1, 0.5, 1.0, 2.0, 5.0]:
    partition = louvain_multilayer(network, gamma=1.0, omega=omega, random_state=42)
    num_comms = len(set(partition.values()))

    # Check cross-layer consistency
    # (how often does the same node get same community across layers?)
    # ... (compute consistency metric)
    print(f"omega={omega}: {num_comms} communities")

```

Interpretation guide:

- $= 0$: Layers are independent (equivalent to detecting per-layer, then combining)
- $= 1$: Balanced coupling (default, usually good)
- > 1 : Strong coupling (forces cross-layer consistency)
- $\rightarrow \infty$: All layers must have identical community structure

Domain-specific guidance:

- **Multiplex social networks:** Start with $=1.0$ (people are the same across platforms)
- **Temporal networks:** $=0.5$ to 1.0 (communities can evolve but not too fast)
- **Heterogeneous networks:** $=0.1$ to 0.5 (different node types may have different community structure)

Diagnosing Bad Partitions

Problem: All nodes in one community

```

if len(set(partition.values())) == 1:
    print("All nodes in single community - try increasing gamma")

```

Causes: Network is too dense, too low, or network genuinely has no community structure.

Problem: Each node is its own community

```
if len(set(partition.values())) == len(partition):
    print("All singletons - try decreasing gamma or increasing omega")
```

Causes: Network is too sparse, too high, too low.

Problem: Communities don't match domain expectations

Actions:

1. Visualize communities and examine specific nodes
2. Check if high-degree nodes are correctly assigned
3. Verify that known groups (e.g., departments) are recovered
4. Consider using ground-truth labels for NMI comparison

Mini Case Studies

Case Study 1: Biological Network Communities

Scenario: A protein-protein interaction network with 3 layers representing different experimental evidence types (yeast two-hybrid, co-immunoprecipitation, affinity purification).

Goal: Find functional modules (groups of proteins with shared biological function).

Approach:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)

# Load network
network = multinet.multi_layer_network().load_network(
    "ppi_multilayer.txt", input_type="multiedgelist", directed=False
)

# Use moderate coupling - different evidence types should
# contribute to same modules, but we don't require perfect consistency
partition = louvain_multilayer(
    network,
    gamma=1.0,      # Standard resolution
    omega=0.5,      # Moderate coupling
    random_state=42
)

# Validate: Do communities correspond to GO biological process terms?
# Compare community assignments to known functional annotations
```

Expected outcome: Communities should correspond to functional modules like “cell cycle,” “DNA repair,” “metabolic pathways.” Proteins appearing in multiple layers with high connectivity should be community hubs.

Case Study 2: Transportation Network Communities

Scenario: A multi-modal transportation network with layers for metro, bus, and bike-share in a city.

Goal: Find “travel basins”—regions where people travel together within a mode and switch between modes at hubs.

Approach:

```

# Load network
network = multinet.multi_layer_network().load_network(
    "transport_network.txt", input_type="multiedgelist", directed=False
)

# Higher coupling - the same station serves multiple modes
partition = louvain_multilayer(
    network,
    gamma=1.2,      # Slightly higher to find smaller regions
    omega=1.5,      # Strong coupling at multimodal hubs
    random_state=42
)

# Validate: Do communities correspond to geographic regions?
# Are major transfer stations correctly identified as community boundaries?

```

Expected outcome: Communities should correspond to neighborhoods or districts. Multimodal hubs (stations serving metro + bus + bike) should appear at community boundaries or as bridges between communities.

Comparing Algorithms

Run Multiple Algorithms

```

# Run different algorithms
louvain_comms = community_louvain.best_partition(network.core_network)
label_prop_comms = label_propagation.propagate(network.core_network)

# Compare number of communities
print(f"Louvain: {len(set(louvain_comms.values()))} communities")
print(f"Label Prop: {len(set(label_prop_comms.values()))} communities")

# Compare modularity
louvain_mod = nx.community.modularity(network.core_network,
                                     [set(n for n, c in louvain_comms.items() if c_
↪ == i)
                                     for i in set(louvain_comms.values())])
label_mod = nx.community.modularity(network.core_network,
                                    [set(n for n, c in label_prop_comms.items() if c_
↪ == i)
                                    for i in set(label_prop_comms.values())])

print(f"Louvain modularity: {louvain_mod:.3f}")
print(f"Label Prop modularity: {label_mod:.3f}")

```

Normalized Mutual Information

Compare similarity between community structures:

```

from sklearn.metrics import normalized_mutual_info_score

# Convert to lists
louvain_list = [louvain_comms[node] for node in network.core_network.nodes()]
label_list = [label_prop_comms[node] for node in network.core_network.nodes()]

# Compute NMI
nmi = normalized_mutual_info_score(louvain_list, label_list)
print(f"NMI between Louvain and Label Prop: {nmi:.3f}")

```

Interpretation:

- NMI = 1.0: Identical community structures
- NMI = 0.0: Completely different structures
- NMI > 0.5: Similar structures

Best Practices**Algorithm Selection**

Network Size		Speed Priority	Quality Priority	Recommendation
Small (<1K)		Any	Any	Try all algorithms
Medium (1K-10K)		Louvain	Louvain/Infomap	Louvain (good balance)
Large (10K-100K)		Louvain/Label Prop	Louvain	Louvain
Very Large (>100K)		Label Prop	Louvain	Label Prop or sample

Parameter Guidelines**Louvain resolution:**

- Start with `resolution=1.0`
- Increase if communities are too large
- Decrease if communities are too fragmented

Multilayer coupling (omega):

- `omega=1.0` - Default, balanced
- `omega<1.0` - Emphasize layer-specific structure
- `omega>1.0` - Emphasize cross-layer structure

Validation

Always validate community detection results:

1. **Visual inspection** — Plot and examine communities
2. **Modularity** — Check modularity score (>0.3 is good)
3. **Size distribution** — Check for giant communities or singletons
4. **Domain knowledge** — Do communities make sense for your application?
5. **Ground truth comparison** — If you have labels, compute NMI or Adjusted Rand Index

Common Failure Modes

- **Trivial partitions:** All-in-one or all-singletons → tune `resolution` and `omega`
- **Unstable results:** Different runs give very different partitions → use `random_state` and run multiple times
- **Over-fragmentation:** Too many small communities → decrease `resolution` or try Leiden
- **Resolution limit:** Can't find small communities in large networks → increase `resolution` or use hierarchical methods

What You Learned

This chapter covered community detection in multilayer networks:

Algorithms:

- **Louvain** — Fast, $O(n \log n)$, BSD license, good for most use cases
- **Infomap** — Flow-based, finds hierarchical structure, AGPLv3 license
- **Label Propagation** — Very fast, linear in edges, supports semi-supervised detection
- **Multilayer Modularity** — True multilayer detection with inter-layer coupling

Parameter tuning:

- **Resolution** — Higher = more, smaller communities; lower = fewer, larger
- **Coupling** — Higher = cross-layer consistency; lower = layer-specific structure
- Start with $\gamma=1.0$, $\beta=1.0$ and adjust based on results

Interpretation:

- Trivial partitions (all-in-one or all-singletons) indicate parameter tuning needed
- High modularity (>0.3) suggests good community structure
- Validate with visualization, domain knowledge, and ground truth if available

Conceptual differences:

- Single-layer detection treats node-layer pairs independently
- Multilayer detection finds communities consistent across layers
- Overlapping vs. non-overlapping communities serve different use cases

References

Louvain:

Blondel, V. D., et al. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*.

Infomap:

Rosvall, M., & Bergstrom, C. T. (2008). Maps of random walks on complex networks reveal community structure. *PNAS*, 105(4), 1118-1123.

Multilayer Modularity:

Mucha, P. J., et al. (2010). Community structure in time-dependent, multiscale, and multiplex networks. *Science*, 328(5980), 876-878.

See [Citation and References](#) for complete citations with DOIs.

What's Next?

- [Random Walk Algorithms](#) — Generate embeddings for ML tasks
- [Visualization](#) — Visualize communities with color-coding
- [Algorithm Landscape](#) — Overview of all algorithms

Related Examples:

- `examples/communities/example_community_detection.py` — Complete workflow
- `examples/communities/example_multilayer_louvain.py` — Parameter tuning

8.3.5 Random Walk Algorithms

Py3plex provides comprehensive random walk primitives for graph-based algorithms, including Node2Vec, DeepWalk, and diffusion processes. These implementations support weighted, directed, and multilayer networks with deterministic reproducibility.

Overview

Random walks are fundamental building blocks for many graph algorithms:

- **Node2Vec:** Learn node embeddings using biased random walks
- **DeepWalk:** Generate node embeddings through uniform random walks
- **Personalized PageRank:** Compute node importance via random walks with restart
- **Diffusion processes:** Model information or disease propagation
- **Community detection:** Identify network structure through walk patterns

Key Features

- [OK] Basic random walks with proper edge weight handling
- [OK] Second-order (biased) random walks following Node2Vec logic
- [OK] Multiple simultaneous walks with deterministic seeding
- [OK] Support for directed, weighted, and multigraphs
- [OK] Multilayer network-aware walks with layer constraints
- [OK] Efficient sparse adjacency matrix handling
- [OK] Comprehensive test suite validating correctness properties

Basic Random Walk

The basic random walk samples the next node proportionally to normalized edge weights.

Simple Example

```
from py3plex.algorithms.general.walkers import basic_random_walk
import networkx as nx

# Create a simple graph
G = nx.karate_club_graph()

# Perform a random walk
walk = basic_random_walk(
    G,
    start_node=0,
    walk_length=10,
    seed=42 # for reproducibility
)

print(f"Walk: {walk}")
print(f"Length: {len(walk)}") # 11 nodes (includes start)
```

Weighted Graphs

Edge weights are automatically used when `weighted=True` (default):

```
from py3plex.algorithms.general.walkers import basic_random_walk
import networkx as nx

# Create weighted graph
G = nx.Graph()
G.add_weighted_edges_from([
    (0, 1, 10.0), # high weight -> more likely
    (0, 2, 1.0),  # low weight -> less likely
    (1, 2, 5.0),
])

# Weighted walk (respects edge weights)
walk = basic_random_walk(G, 0, walk_length=20, weighted=True, seed=42)

# Unweighted walk (uniform transitions)
walk_uniform = basic_random_walk(G, 0, walk_length=20, weighted=False, seed=42)
```

Directed Graphs

Directed edges are handled correctly:

```
import networkx as nx
from py3plex.algorithms.general.walkers import basic_random_walk

# Create directed graph
G = nx.DiGraph()
G.add_edges_from([(0, 1), (1, 2), (2, 0), (1, 3)])

# Walk respects edge direction
walk = basic_random_walk(G, 0, walk_length=10, seed=42)

# Verify all transitions follow directed edges
for i in range(len(walk) - 1):
    assert G.has_edge(walk[i], walk[i+1])
```

Node2Vec Biased Random Walk

Second-order random walks with return parameter p and in-out parameter q following the Node2Vec algorithm (Grover & Leskovec, 2016).

Parameters

When transitioning from node $t \rightarrow v \rightarrow x$, the probability of choosing x is biased:

- If $x == t$ (return to previous): weight / p
- If x is neighbor of t (stay close): $\text{weight} / 1$
- If x is not neighbor of t (explore): weight / q

Parameter Effects:

- $p > 1$: Less likely to return to previous node (explore forward)
- $p < 1$: More likely to return (backtrack)
- $q > 1$: Less likely to explore distant nodes (stay local, BFS-like)
- $q < 1$: More likely to explore distant nodes (venture far, DFS-like)
- $p = q = 1$: Equivalent to basic random walk

Basic Usage

```
from py3plex.algorithms.general.walkers import node2vec_walk
import networkx as nx

G = nx.karate_club_graph()

# Balanced walk (p=1, q=1)
walk_balanced = node2vec_walk(G, 0, walk_length=20, p=1.0, q=1.0, seed=42)

# BFS-like walk (stay local)
walk_bfs = node2vec_walk(G, 0, walk_length=20, p=1.0, q=2.0, seed=42)

# DFS-like walk (explore outward)
walk_dfs = node2vec_walk(G, 0, walk_length=20, p=1.0, q=0.5, seed=42)
```

Exploring vs Backtracking

```
from py3plex.algorithms.general.walkers import node2vec_walk
import networkx as nx

# Triangle graph
G = nx.Graph()
G.add_edges_from([(0, 1), (1, 2), (2, 0)])

# Low p, high q: tends to backtrack
walk_backtrack = node2vec_walk(G, 0, walk_length=20, p=0.1, q=10.0, seed=42)

# High p, low q: tends to explore outward
walk_explore = node2vec_walk(G, 0, walk_length=20, p=10.0, q=0.1, seed=42)

# Count backtracking patterns
backtracks = sum(1 for i in range(2, len(walk_backtrack))
                 if walk_backtrack[i] == walk_backtrack[i-2])
print(f"Backtracks: {backtracks}")
```

Multiple Walk Generation

Generate multiple walks efficiently with deterministic seeding.

Generate Walks from All Nodes

```
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate 10 walks from each node
walks = generate_walks(
    G,
    num_walks=10,
    walk_length=10,
    seed=42
)

print(f"Total walks: {len(walks)}") # 340 (34 nodes × 10 walks)
```

(continues on next page)

(continued from previous page)

```
print(f"First walk: {walks[0]}")
```

Generate from Specific Nodes

```
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate walks only from nodes 0, 1, 2
walks = generate_walks(
    G,
    num_walks=5,
    walk_length=15,
    start_nodes=[0, 1, 2],
    seed=42
)

print(f"Total walks: {len(walks)}") # 15 (3 nodes × 5 walks)
```

Node2Vec-Style Walks

```
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate biased walks with p=0.5, q=2.0
walks = generate_walks(
    G,
    num_walks=10,
    walk_length=20,
    p=0.5,
    q=2.0,
    seed=42
)
```

Edge Sequences

Return walks as edge sequences instead of node sequences:

```
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate edge sequences
edge_walks = generate_walks(
    G,
    num_walks=5,
    walk_length=10,
    return_edges=True,
    seed=42
)
```

(continues on next page)

(continued from previous page)

```
# First walk is a list of edges
print(edge_walks[0]) # [(0, 3), (3, 2), (2, 1), ...]
```

Multilayer Network Walks

Perform random walks on multilayer networks with layer constraints.

Layer-Constrained Walks

```
from py3plex.algorithms.general.walkers import layer_specific_random_walk
from py3plex.core import multinet

# Create multilayer network
network = multinet.multi_layer_network()
network.add_layer("social")
network.add_layer("biological")

network.add_nodes([
    ("A", "social"), ("B", "social"), ("C", "social"),
    ("A", "biological"), ("B", "biological")
])

network.add_edges([
    (("A", "social"), ("B", "social")),
    (("B", "social"), ("C", "social")),
    (("A", "biological"), ("B", "biological")),
])

# Walk constrained to social layer
walk = layer_specific_random_walk(
    network.core_network,
    start_node="A---social",
    walk_length=10,
    layer="social",
    cross_layer_prob=0.0, # never cross layers
    seed=42
)

# All nodes in walk are in social layer
print(walk)
```

Cross-Layer Transitions

Allow occasional transitions between layers:

```
from py3plex.algorithms.general.walkers import layer_specific_random_walk

# Same network as above

# Walk with 10% probability of crossing layers
walk = layer_specific_random_walk(
    network.core_network,
    start_node="A---social",
    walk_length=20,
```

(continues on next page)

(continued from previous page)

```
    layer="social",
    cross_layer_prob=0.1, # 10% chance to cross
    seed=42
)

# May contain nodes from both layers
print(walk)
```

Reproducibility

All walk functions support deterministic seeding for reproducibility.

Fixed Seed

```
from py3plex.algorithms.general.walkers import basic_random_walk
import networkx as nx

G = nx.karate_club_graph()

# Same seed produces identical walks
walk1 = basic_random_walk(G, 0, 50, seed=42)
walk2 = basic_random_walk(G, 0, 50, seed=42)

assert walk1 == walk2 # Identical
```

Different Seeds

```
# Different seeds produce different walks
walk1 = basic_random_walk(G, 0, 50, seed=42)
walk2 = basic_random_walk(G, 0, 50, seed=123)

assert walk1 != walk2 # Different
```

Batch Reproducibility

```
from py3plex.algorithms.general.walkers import generate_walks

G = nx.karate_club_graph()

# Same seed produces identical walk sets
walks1 = generate_walks(G, num_walks=100, walk_length=10, seed=42)
walks2 = generate_walks(G, num_walks=100, walk_length=10, seed=42)

assert walks1 == walks2
```

Performance Tips

Large Graphs

For large sparse graphs (>100k nodes), use basic walks for better performance:

```
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx
```

(continues on next page)

(continued from previous page)

```

# Large sparse graph
G = nx.erdos_renyi_graph(100000, 0.0001, seed=42)

# Use basic walk (p=1, q=1) for speed
walks = generate_walks(
    G,
    num_walks=10,
    walk_length=100,
    p=1.0, # No bias = faster
    q=1.0,
    seed=42
)

```

Parallel Processing

For generating many walks, consider parallel processing:

```

from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx
from multiprocessing import Pool

G = nx.karate_club_graph()

def generate_batch(seed):
    return generate_walks(G, num_walks=10, walk_length=20, seed=seed)

# Generate walks in parallel
with Pool(4) as pool:
    batch_walks = pool.map(generate_batch, range(10))

# Flatten results
all_walks = [walk for batch in batch_walks for walk in batch]

```

Correctness Validation

The implementation has been validated against the following properties:

Uniformity Test

On unweighted regular graphs, transition probabilities are uniform:

```

import networkx as nx
from py3plex.algorithms.general.walkers import basic_random_walk
from collections import Counter

# Complete graph (regular)
G = nx.complete_graph(10)

# Count visits to neighbors from node 0
visits = Counter()
for i in range(10000):
    walk = basic_random_walk(G, 0, 1, weighted=False, seed=i)
    if len(walk) > 1:
        visits[walk[1]] += 1

# All neighbors should be visited roughly equally

```

(continues on next page)

(continued from previous page)

```
print(visits)
# Expected: ~1111 visits to each of 9 neighbors
```

Conservation Test

Sum of transition probabilities equals 1.0 (within machine epsilon):

```
import networkx as nx
import numpy as np

G = nx.karate_club_graph()

# For each node, verify probability conservation
for node in G.nodes():
    neighbors = list(G.neighbors(node))
    if len(neighbors) > 0:
        weights = np.array([G[node][n].get('weight', 1.0) for n in neighbors])
        probs = weights / weights.sum()
        assert abs(probs.sum() - 1.0) < 1e-10 # Machine epsilon
```

Bias Consistency Test

Node2Vec parameters p and q produce expected biases:

```
import networkx as nx
from py3plex.algorithms.general.walkers import node2vec_walk

# Triangle graph
G = nx.Graph()
G.add_edges_from([(0, 1), (1, 2), (2, 0)])

# Low p should encourage backtracking
backtracks_low_p = 0
for i in range(1000):
    walk = node2vec_walk(G, 0, 10, p=0.1, q=1.0, seed=i)
    for j in range(2, len(walk)):
        if walk[j] == walk[j-2]:
            backtracks_low_p += 1

# High p should discourage backtracking
backtracks_high_p = 0
for i in range(1000):
    walk = node2vec_walk(G, 0, 10, p=10.0, q=1.0, seed=i)
    for j in range(2, len(walk)):
        if walk[j] == walk[j-2]:
            backtracks_high_p += 1

assert backtracks_low_p > backtracks_high_p * 2 # Significantly more
```

Edge Weight Test

Visit frequency matches edge weight ratios:

```
import networkx as nx
from py3plex.algorithms.general.walkers import basic_random_walk
```

(continues on next page)

(continued from previous page)

```

from collections import Counter

G = nx.Graph()
G.add_weighted_edges_from([
    (0, 1, 1.0),
    (0, 2, 2.0),
    (0, 3, 3.0),
])

visits = Counter()
for i in range(10000):
    walk = basic_random_walk(G, 0, 1, weighted=True, seed=i)
    if len(walk) > 1:
        visits[walk[1]] += 1

# Expected ratio: 1:2:3
total = sum(visits.values())
ratios = [visits[1]/total, visits[2]/total, visits[3]/total]
expected = [1/6, 2/6, 3/6]

# Within 5% tolerance
for obs, exp in zip(ratios, expected):
    assert abs(obs - exp) < 0.05

```

Complete Embedding Workflow

This section provides a complete, end-to-end workflow for generating node embeddings from a multilayer network and using them for downstream machine learning tasks.

Step 1: Prepare the Network

```

from py3plex.core import multinet
import numpy as np

# Set random seed for reproducibility
np.random.seed(42)

# Load your multilayer network
network = multinet.multi_layer_network().load_network(
    "your_network.multiedgelist",
    input_type="multiedgelist",
    directed=False
)

# Verify the network
network.basic_stats()

# Get the core NetworkX graph
G = network.core_network

```

Step 2: Generate Random Walks

```

from py3plex.algorithms.general.walkers import generate_walks

# Generate walks with Node2Vec-style biased sampling

```

(continues on next page)

(continued from previous page)

```
# Recommended starting parameters
walks = generate_walks(
    G,
    num_walks=10,      # 10 walks per node (more = better embeddings, slower)
    walk_length=80,    # 80 steps per walk (adjust based on graph diameter)
    p=1.0,             # Return parameter (1.0 = balanced)
    q=1.0,             # In-out parameter (1.0 = balanced)
    seed=42            # For reproducibility
)

print(f"Generated {len(walks)} walks")
print(f"Sample walk: {walks[0][:10]}...") # First 10 nodes of first walk
```

Step 3: Train Embedding Model

```
# Requires gensim: pip install gensim
from gensim.models import Word2Vec

# Convert walks to strings (Word2Vec expects string tokens)
walks_str = [[str(node) for node in walk] for walk in walks]

# Train the embedding model
model = Word2Vec(
    walks_str,
    vector_size=128,  # Embedding dimension (64-256 typical)
    window=5,        # Context window size
    min_count=1,      # Include nodes that appear at least once
    sg=1,             # Use skip-gram (1) vs CBOW (0)
    workers=4,        # Parallel training threads
    epochs=5,         # Training epochs
    seed=42           # Reproducibility
)

print(f"Trained embeddings for {len(model.wv)} nodes")
print(f"Embedding dimension: {model.wv.vector_size}")
```

Step 4: Extract and Use Embeddings

```
import numpy as np

# Get all node IDs
node_ids = list(model.wv.index_to_key)

# Create embedding matrix
embeddings = np.array([model.wv[node_id] for node_id in node_ids])
print(f"Embedding matrix shape: {embeddings.shape}")

# Get embedding for a specific node
sample_node = node_ids[0]
node_embedding = model.wv[sample_node]
print(f"Embedding for {sample_node}: {node_embedding[:5]}... (dim={len(node_
↪ embedding)})")

# Find similar nodes
similar_nodes = model.wv.most_similar(sample_node, topn=5)
```

(continues on next page)

(continued from previous page)

```
print(f"\nMost similar to {sample_node}:")
for node, similarity in similar_nodes:
    print(f"    {node}: {similarity:.4f}")
```

Step 5: Downstream Tasks

Node Clustering:

```
from sklearn.cluster import KMeans
import numpy as np

# Cluster nodes based on embeddings
n_clusters = 5
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
clusters = kmeans.fit_predict(embeddings)

# Assign clusters to nodes
node_clusters = dict(zip(node_ids, clusters))

print("Cluster distribution:")
from collections import Counter
for cluster, count in sorted(Counter(clusters).items()):
    print(f"    Cluster {cluster}: {count} nodes")
```

Node Classification:

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Assume you have labels for nodes
# labels = {node_id: label for node_id, label in your_label_data}

# For demonstration, create synthetic labels based on layer
labels = {}
for node_id in node_ids:
    # Extract layer from node-layer tuple string representation
    # Adjust this based on your actual node ID format
    if 'layer1' in str(node_id):
        labels[node_id] = 0
    elif 'layer2' in str(node_id):
        labels[node_id] = 1
    else:
        labels[node_id] = 2

# Prepare data
X = embeddings
y = np.array([labels[node_id] for node_id in node_ids])

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train classifier
clf = RandomForestClassifier(n_estimators=100, random_state=42)
```

(continues on next page)

(continued from previous page)

```

clf.fit(X_train, y_train)

# Evaluate
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Classification accuracy: {accuracy:.4f}")

```

Link Prediction:

```

from sklearn.linear_model import LogisticRegression
import numpy as np

def edge_embedding(node1, node2, method='hadamard'):
    """Compute edge embedding from node embeddings."""
    emb1 = model.wv[str(node1)]
    emb2 = model.wv[str(node2)]

    if method == 'hadamard':
        return emb1 * emb2
    elif method == 'average':
        return (emb1 + emb2) / 2
    elif method == 'concat':
        return np.concatenate([emb1, emb2])
    else:
        raise ValueError(f"Unknown method: {method}")

# Get existing edges (positive examples)
existing_edges = list(G.edges())[:100] # Sample for demo

# Generate negative examples (non-edges)
nodes = list(G.nodes())
negative_edges = []
while len(negative_edges) < len(existing_edges):
    n1 = nodes[np.random.randint(len(nodes))]
    n2 = nodes[np.random.randint(len(nodes))]
    if n1 != n2 and not G.has_edge(n1, n2):
        negative_edges.append((n1, n2))

# Create training data
X_edges = []
y_edges = []

for edge in existing_edges:
    try:
        X_edges.append(edge_embedding(edge[0], edge[1]))
        y_edges.append(1) # Positive
    except KeyError:
        pass # Skip if node not in vocabulary

for edge in negative_edges:
    try:
        X_edges.append(edge_embedding(edge[0], edge[1]))
        y_edges.append(0) # Negative
    except KeyError:
        pass

```

(continues on next page)

(continued from previous page)

```

X_edges = np.array(X_edges)
y_edges = np.array(y_edges)

# Train link prediction model
X_train, X_test, y_train, y_test = train_test_split(
    X_edges, y_edges, test_size=0.2, random_state=42
)

clf = LogisticRegression(random_state=42)
clf.fit(X_train, y_train)

accuracy = clf.score(X_test, y_test)
print(f"Link prediction accuracy: {accuracy:.4f}")

```

Step 6: Save and Load Embeddings

```

# Save embeddings in Word2Vec format
model.wv.save_word2vec_format("node_embeddings.txt", binary=False)
print("Embeddings saved to node_embeddings.txt")

# Save as NumPy arrays (for sklearn compatibility)
np.save("embedding_matrix.npy", embeddings)
np.save("node_ids.npy", np.array(node_ids))
print("Embeddings saved as NumPy arrays")

# Load embeddings later
from gensim.models import KeyedVectors
loaded_embeddings = KeyedVectors.load_word2vec_format("node_embeddings.txt")
print(f"Loaded embeddings for {len(loaded_embeddings)} nodes")

```

Parameter Tuning Guide

Walk Length Guidelines

Network Size	walk_length	Reasoning	Trade-off
Small (<1K nodes)	40-80	Shorter walks sufficient	Speed vs coverage
Medium (1K-10K)	80-120	Standard choice	Balanced
Large (>10K)	80-100	Longer walks costly	Memory/time vs quality

Rule of thumb: walk_length should be at least 2-3× the network diameter.

Number of Walks Guidelines

Task	num_walks	Reasoning	Trade-off
Exploration	5-10	Quick results	Speed vs quality
Standard	10-20	Good balance	Recommended
High-quality	20-50	Better embeddings	Time vs quality
Research	50-100	Maximum quality	Very slow

Rule of thumb: More walks = better embeddings, but with diminishing returns after ~20.

P and Q Parameter Effects

The p and q parameters control the walk's exploration behavior:

Return parameter p :

- $p < 1$: More likely to backtrack (return to previous node)
 - Use when: Local structure is important; you want to capture tight clusters
- $p = 1$: Balanced (equivalent to basic random walk for backtracking)
 - Use when: No prior about network structure
- $p > 1$: Less likely to backtrack (move forward)
 - Use when: You want walks to explore broadly

In-out parameter q :

- $q < 1$: More likely to explore distant nodes (DFS-like)
 - Use when: You want to capture global structure, find communities
- $q = 1$: Balanced exploration
 - Use when: No prior about network structure
- $q > 1$: More likely to stay in local neighborhood (BFS-like)
 - Use when: You want to capture local structure, homophily

Common presets:

```
# Local/homophily-focused (similar to BFS)
p, q = 1.0, 2.0

# Global/structural (similar to DFS)
p, q = 1.0, 0.5

# Balanced (DeepWalk-like)
p, q = 1.0, 1.0

# Strong local focus
p, q = 0.5, 2.0

# Strong exploration
p, q = 2.0, 0.5
```

Common Failure Modes

Disconnected Graph

Problem: Walk gets stuck or terminates early because node has no neighbors.

Symptoms:

- Walks shorter than expected
- Some nodes never appear in walks
- Embeddings missing for some nodes

Solutions:

```
import networkx as nx

# Check connectivity
```

(continues on next page)

(continued from previous page)

```

if not nx.is_connected(G.to_undirected()):
    print("Graph is disconnected!")

    # Option 1: Use largest connected component
    largest_cc = max(nx.connected_components(G.to_undirected()), key=len)
    G_connected = G.subgraph(largest_cc).copy()

    # Option 2: Add small random edges to connect components
    # (use with caution - changes graph structure)

```

Highly Skewed Degree Distribution

Problem: High-degree hub nodes dominate walks; low-degree nodes underrepresented.

Symptoms:

- Most walks pass through the same few nodes
- Embeddings for peripheral nodes are poor quality
- Similarity to hub nodes is overestimated

Solutions:

```

# Option 1: Use more walks per node
walks = generate_walks(G, num_walks=30, walk_length=80, seed=42)

# Option 2: Add node-specific walks for low-degree nodes
low_degree_nodes = [n for n in G.nodes() if G.degree(n) < 5]
extra_walks = generate_walks(
    G,
    num_walks=20,
    walk_length=80,
    start_nodes=low_degree_nodes,
    seed=42
)
walks.extend(extra_walks)

# Option 3: Use weighted walks that downweight high-degree nodes
# (requires custom implementation)

```

Isolated Nodes

Problem: Nodes with degree 0 cannot be walked from.

Symptoms:

- KeyError when looking up embeddings
- Missing nodes in similarity queries

Solutions:

```

# Remove isolated nodes before generating walks
isolated = list(nx.isolates(G))
G_filtered = G.copy()
G_filtered.remove_nodes_from(isolated)

# Generate walks on filtered graph
walks = generate_walks(G_filtered, num_walks=10, walk_length=80, seed=42)

```

(continues on next page)

(continued from previous page)

```
# Handle isolated nodes separately (e.g., assign zero embedding)
```

Very Small Graph

Problem: With few nodes, walks become repetitive quickly.

Symptoms:

- Walks cycle through the same nodes repeatedly
- All nodes have similar embeddings
- Poor discriminative power

Solutions:

```
# Use shorter walks for small graphs
if G.number_of_nodes() < 100:
    walk_length = min(40, G.number_of_nodes() * 2)

# Use fewer walks (avoid redundancy)
num_walks = min(10, G.number_of_nodes())

# Consider direct matrix-based methods for very small graphs
# (spectral embeddings, Laplacian eigenvectors)
```

Memory Issues with Large Graphs

Problem: Storing all walks in memory causes OOM errors.

Symptoms:

- MemoryError during walk generation
- System becomes unresponsive

Solutions:

```
# Option 1: Generate walks in batches
all_walks = []
nodes = list(G.nodes())
batch_size = 1000

for i in range(0, len(nodes), batch_size):
    batch_nodes = nodes[i:i+batch_size]
    batch_walks = generate_walks(
        G,
        num_walks=10,
        walk_length=80,
        start_nodes=batch_nodes,
        seed=42+i
    )
    all_walks.extend(batch_walks)
    print(f"Processed {i+batch_size}/{len(nodes)} nodes")

# Option 2: Stream walks directly to Word2Vec
# (requires custom iterator implementation)
```

(continues on next page)

(continued from previous page)

```
# Option 3: Use disk-based storage
# Write walks to file, then load for training
```

References

Node2Vec:

Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 855-864. <https://doi.org/10.1145/2939672.2939754>

DeepWalk:

Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 701-710. <https://doi.org/10.1145/2623330.2623732>

Random Walks on Graphs:

Lovász, L. (1993). Random walks on graphs: A survey. *Combinatorics, Paul Erdős is Eighty*, 2, 1-46.

API Reference

See `py3plex.algorithms.general.walkers` for complete API documentation.

Examples

Complete examples can be found in:

- `examples/advanced/example_random_walks.py` - Basic usage examples
- `tests/test_random_walks.py` - Comprehensive test suite

Repository: <https://github.com/SkBlaz/py3plex>

8.3.6 I/O and Serialization

py3plex provides a comprehensive I/O system for reading and writing multilayer graphs in various formats. The system is designed to be extensible, efficient, and easy to use.

Supported Formats

The I/O system supports multiple file formats, each with different trade-offs:

- **JSON** - Human-readable, widely compatible, good for small to medium networks
- **JSONL** - Streaming JSON format, efficient for large networks
- **CSV** - Spreadsheet-compatible, easy to edit manually
- **Arrow/Feather** - High-performance columnar format (requires pyarrow)
- **Parquet** - Compressed columnar format, best for storage (requires pyarrow)

Basic Usage

The I/O system provides two main functions: `read()` and `write()`.

Reading Graphs

```
from py3plex.io import read

# Auto-detect format from extension
graph = read('network.json')
graph = read('network.csv')
```

(continues on next page)

(continued from previous page)

```
graph = read('network.arrow')

# Or specify format explicitly
graph = read('myfile.dat', format='json')
```

Writing Graphs

```
from py3plex.io import write

# Auto-detect format from extension
write(graph, 'network.json')
write(graph, 'network.arrow')
write(graph, 'network.parquet')

# Or specify format explicitly
write(graph, 'myfile.dat', format='json')
```

Creating Graphs with the Schema API

The modern I/O system uses a schema-based API for creating graphs:

```
from py3plex.io import MultiLayerGraph, Node, Layer, Edge

# Create graph
graph = MultiLayerGraph(
    directed=True,
    attributes={'name': 'Social Network'}
)

# Add layers
graph.add_layer(Layer(id='facebook', attributes={'type': 'social'}))
graph.add_layer(Layer(id='twitter', attributes={'type': 'social'}))

# Add nodes
graph.add_node(Node(id='alice', attributes={'age': 30}))
graph.add_node(Node(id='bob', attributes={'age': 25}))

# Add edges
graph.add_edge(Edge(
    src='alice',
    dst='bob',
    src_layer='facebook',
    dst_layer='facebook',
    attributes={'weight': 0.8}
))
```

Apache Arrow Format

Apache Arrow is a high-performance columnar format designed for efficient data interchange. py3plex supports Arrow through two sub-formats:

- **Feather** - Fast, uncompressed format ideal for temporary storage
- **Parquet** - Compressed format ideal for long-term storage

Installing Arrow Support

Arrow support requires the pyarrow package:

```
pip install 'py3plex[arrow]'
# or directly
pip install pyarrow
```

Using Arrow Format

```
from py3plex.io import read, write

# Feather format (fast, uncompressed)
write(graph, 'network.arrow')
graph = read('network.arrow')

# Parquet format (compressed)
write(graph, 'network.parquet', format='parquet')
graph = read('network.parquet', format='parquet')
```

Benefits of Arrow Format

1. **Performance:** Columnar storage enables fast read/write operations
2. **Compression:** Parquet format provides excellent compression ratios
3. **Interoperability:** Arrow is an industry-standard format supported by:
 - pandas, polars (Python data analysis)
 - Apache Spark (big data processing)
 - R, Julia (statistical computing)
 - DuckDB (analytical database)
4. **Type Safety:** Schema preservation with strong typing
5. **Zero-Copy:** Efficient in-memory representation

Performance Comparison

For a typical multilayer network with 1000 nodes and ~5000 edges:

Format	Write Time	Read Time	File Size
Arrow	0.016s	0.008s	0.46 MB
Parquet	0.020s	0.010s	0.35 MB
JSON	0.046s	0.030s	1.09 MB

Arrow format is **2-3x faster** for writes and provides **2-3x better compression** compared to JSON.

When to Use Each Format

Use Arrow/Feather when:

- You need maximum read/write performance
- Working with large networks (>10k nodes)
- Interoperating with data science tools (pandas, polars)
- Building data pipelines

Use Parquet when:

- Long-term storage is important
- Minimizing storage costs
- Sharing data across platforms
- Archiving networks

Use JSON when:

- Human readability is important
- Working with small networks
- Debugging or manual editing
- Maximum compatibility needed

Use CSV when:

- Working with spreadsheet tools (Excel)
- Simple edge lists
- Manual data entry/editing

CSV Format with Sidecars

CSV format supports optional sidecar files for node and layer attributes:

```
from py3plex.io import read, write

# Write with sidecars
write(graph, 'edges.csv', format='csv', write_sidecars=True)
# Creates: edges.csv, nodes.csv, layers.csv

# Read with sidecars
graph = read('edges.csv', format='csv',
            nodes_file='nodes.csv',
            layers_file='layers.csv')
```

Integration with NetworkX

Convert between py3plex I/O format and NetworkX:

```
from py3plex.io import read, to_networkx, from_networkx

# Load graph
graph = read('network.json')

# Convert to NetworkX
G = to_networkx(graph, mode='union') # Merge all layers
# or
G = to_networkx(graph, mode='multiplex') # Preserve layers as (node, layer)

# Convert back from NetworkX
graph = from_networkx(G, mode='multiplex')
```

Example: Complete Workflow

Here's a complete example demonstrating the I/O system:

```
from py3plex.io import (
    MultiLayerGraph, Node, Layer, Edge,
    read, write, to_networkx
)

# Create a multilayer network
graph = MultiLayerGraph(directed=True)

# Add layers
for layer_id in ['social', 'work', 'family']:
    graph.add_layer(Layer(id=layer_id))

# Add nodes
for name in ['alice', 'bob', 'charlie']:
    graph.add_node(Node(id=name))

# Add edges
edges = [
    ('alice', 'bob', 'social', 'social', 0.8),
    ('bob', 'charlie', 'work', 'work', 0.6),
    ('alice', 'charlie', 'family', 'family', 0.9),
]

for src, dst, src_layer, dst_layer, weight in edges:
    graph.add_edge(Edge(
        src=src, dst=dst,
        src_layer=src_layer, dst_layer=dst_layer,
        attributes={'weight': weight}
    ))

# Save in multiple formats
write(graph, 'network.json')
write(graph, 'network.arrow')
write(graph, 'network.parquet')

# Load back
loaded = read('network.arrow')

# Convert to NetworkX for analysis
G = to_networkx(loaded, mode='union')

# Use NetworkX algorithms
import networkx as nx
centrality = nx.degree_centrality(G)
print(f"Most central node: {max(centrality, key=centrality.get)}")
```

Checking Supported Formats

You can query which formats are available at runtime:

```
from py3plex.io import supported_formats

formats = supported_formats()
print(f"Read formats: {formats['read']}")
```

(continues on next page)

(continued from previous page)

```
print(f"Write formats: {formats['write']}")
```

This is useful for checking if optional dependencies (like pyarrow) are installed.

Schema Validation

The I/O system includes automatic validation:

```
from py3plex.io import (
    MultiLayerGraph, Node, Edge,
    ReferentialIntegrityError
)

graph = MultiLayerGraph()
graph.add_node(Node(id='alice'))

try:
    # This will fail - bob doesn't exist
    graph.add_edge(Edge(
        src='alice', dst='bob',
        src_layer='l1', dst_layer='l1'
    ))
except ReferentialIntegrityError as e:
    print(f"Validation error: {e}")
```

Validation ensures:

1. All edge endpoints reference existing nodes
2. All edge layers reference existing layers
3. All attributes are JSON-serializable
4. No duplicate edges (by src, dst, src_layer, dst_layer, key)

Advanced: Custom Formats

The I/O system is extensible. You can register custom format readers/writers:

```
from py3plex.io import register_reader, register_writer

def my_reader(filepath, **kwargs):
    # Custom reading logic
    graph = MultiLayerGraph()
    # ... populate graph ...
    return graph

def my_writer(graph, filepath, **kwargs):
    # Custom writing logic
    with open(filepath, 'w') as f:
        # ... write graph ...
    pass

# Register
register_reader('myformat', my_reader)
register_writer('myformat', my_writer)

# Now you can use it
write(graph, 'network.myformat')
graph = read('network.myformat')
```

Examples

Complete examples are available in `examples/io_and_data/`:

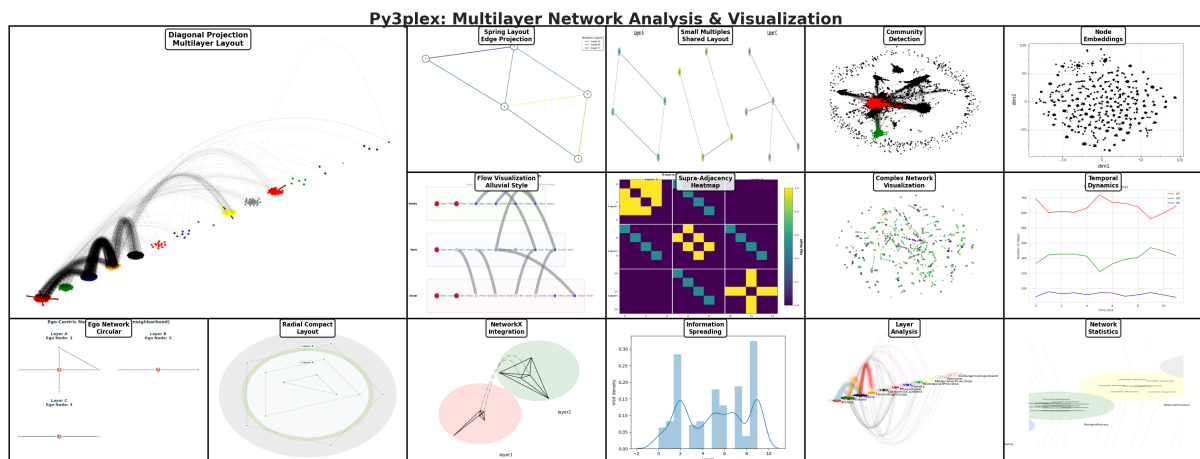
- `example_new_io.py` - Comprehensive I/O demonstration
- `example_save_to_arrow.py` - Apache Arrow format usage
- `example_save_to_gpickle.py` - NetworkX pickle format
- `example_save_to_edgelist.py` - Edge list format
- `example_schema_validation.py` - Schema validation examples

See Also

- *Chapter 1: Your First Five Minutes* - Getting started guide
- *Working with Networks* - Basic network operations
- *Chapter 4: The py3plex Core Model* - NetworkX integration details
- *Performance and Scalability Best Practices* - Performance optimization tips

8.3.7 Visualization

From hairball to insight: making multilayer networks visible.



A network you can't see is a network you can't understand. Visualization transforms abstract adjacency structures into spatial patterns that human perception can process—clusters become visible groups, hubs become prominent nodes, and cross-layer connections become the bridges they conceptually are.

This chapter shows you how to create visualizations that reveal structure rather than obscure it. We'll cover:

- **Quick plots** for exploration and debugging
- **Preset modes** optimized for different network scales
- **Layout algorithms** that reveal different structural properties
- **Interactive visualizations** for presentations and exploration
- **Publication-ready figures** with proper formatting

Table of Contents

- *Quick Start*

- *Basic Multilayer Visualization*
- *Preset Visualization Modes*
 - *Minimal Mode (Large Networks >1000 nodes)*
 - *Balanced Mode (Medium Networks 100-1000 nodes)*
 - *Dense Mode (Small Networks <100 nodes)*
- *Layout Options*
 - *Circular Layout (Default)*
 - *Rectangular Layout*
- *Auto-Scaling Features*
 - *Automatic Node Sizing*
 - *Automatic Color Assignment*
 - *Automatic Layout Adjustment*
- *Customization Options*
 - *Complete Parameter Reference*
 - *Layer-Specific Customization*
 - *Color-Coding by Community*
- *Exporting Visualizations*
 - *Save to File*
 - *High-Quality Publications*
- *Interactive Visualizations*
 - *Using Plotly (Optional)*
 - *Basic Interactive Hairball Plot*
 - *Advanced Interactive Multilayer Example*
 - *Jupyter Notebook Integration*
 - *Saving Interactive Visualizations*
 - *Embedding in Web Applications*
 - *Troubleshooting Interactive Visualizations*
 - *Performance Guidelines*
 - *Jupyter Notebook Integration*
- *Performance Tips*
 - *For Large Networks*
- *Troubleshooting*
 - *Visualization Not Showing*
 - *Nodes Overlapping*
 - *Labels Too Crowded*
- *Advanced Visualization Modes*
 - *Overview of Visualization Modes*
 - *Unified API*

- *Small Multiples View*
- *Edge-Colored Projection*
- *Supra-Adjacency Heatmap*
- *Radial/Concentric Layers*
- *Ego-Centric Multilayer View*
- *Flow and Sankey Diagrams*
- *Example Workflows*
- *Choosing the Right Visualization*
- *What You Learned*
- *What's Next?*

Quick Start

Basic Multilayer Visualization

```
from py3plex.core import multinet
from py3plex.visualization.multilayer import draw_multilayer_default
import matplotlib.pyplot as plt

# Load network
network = multinet.multi_layer_network()
network.load_network("network.csv", input_type="multiedgelist")

# Visualize with defaults
draw_multilayer_default(
    network.get_layers(),
    display=True,
    labels=True
)
```

Preset Visualization Modes

Py3plex provides three preset modes optimized for different network scales and use cases.

Minimal Mode (Large Networks >1000 nodes)

Optimized for networks with many nodes where detail isn't critical.

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Minimal preset for large networks
draw_multilayer_default(
    network.get_layers(),
    # Node settings
    node_size=5,           # Small nodes
    labels=False,         # No node labels (too cluttered)
    node_labels=False,    # No node IDs

    # Edge settings
    edge_size=0.5,        # Thin edges
```

(continues on next page)

(continued from previous page)

```

    alphalevel=0.3,          # Transparent edges (reduce clutter)

    # Layout
    background_shape="circle", # Circular layout

    # Display
    display=True,
    remove_isolated_nodes=True # Clean up isolated nodes
)

```

Use cases: - Large social networks (>1000 nodes) - Overview visualizations - Pattern detection at macro level - Network topology understanding

Advantages: - Fast rendering - Reduced visual clutter - Shows overall structure - Works with many nodes

Balanced Mode (Medium Networks 100-1000 nodes)

Default settings that work well for most networks. Good balance between detail and clarity.

```

# Balanced preset (this is the default)
draw_multilayer_default(
    network.get_layers(),
    # Node settings
    node_size=10,          # Medium nodes
    labels=True,           # Show layer labels
    node_labels=False,     # Node IDs hidden by default

    # Edge settings
    edge_size=1.0,         # Normal edge width
    alphalevel=0.13,       # Semi-transparent edges

    # Layout
    background_shape="circle", # Circular layout
    networks_color="rainbow",  # Auto-assign colors

    # Display
    display=True
)

```

Use cases: - Most research networks - Exploratory data analysis - Publication figures (moderate detail) - Interactive exploration

Advantages: - Good readability - Reasonable performance - Balanced aesthetics - Suitable for most publications

Dense Mode (Small Networks <100 nodes)

Maximum detail for small networks where every node and edge matters.

```

# Dense preset for small, detailed networks
draw_multilayer_default(
    network.get_layers(),
    # Node settings
    node_size=20,          # Large nodes
    labels=True,           # Show all labels
    node_labels=True,      # Show node IDs
    node_font_size=10,     # Readable font
    scale_by_size=True,    # Scale by degree
)

```

(continues on next page)

(continued from previous page)

```

# Edge settings
edge_size=2.0,          # Thick edges
alphalevel=0.8,         # Mostly opaque
arrowsize=0.5,          # Visible arrows (if directed)

# Layout
background_shape="rectangle", # Rectangular layout
networks_color="rainbow",

# Display
display=True
)

```

Use cases: - Small case studies - Detailed analysis - Node-level examination - High-quality publication figures

Advantages: - Maximum detail - Clear node identification - Edge weights visible - Professional appearance

Layout Options

Py3plex supports multiple layout algorithms for different network structures.

Circular Layout (Default)

Arranges layers in a circle. Good for showing inter-layer connections.

```

draw_multilayer_default(
    network.get_layers(),
    background_shape="circle",
    rectanglex=1.0, # Circle radius x
    rectangley=1.0 # Circle radius y
)

```

Best for: - 2-10 layers - Symmetric layer interactions - Publication figures

Rectangular Layout

Arranges layers in a grid. Good for many layers or hierarchical structures.

```

draw_multilayer_default(
    network.get_layers(),
    background_shape="rectangle",
    rectanglex=2.0, # Width of layout
    rectangley=1.0 # Height of layout
)

```

Best for: - Many layers (>10) - Hierarchical networks - Time-series data - Wide figures

Auto-Scaling Features

Py3plex automatically adjusts visualization parameters based on network size.

Automatic Node Sizing

Scale node sizes by degree (number of connections):

```

draw_multilayer_default(
    network.get_layers(),
    scale_by_size=True, # Enable auto-scaling
)

```

(continues on next page)

(continued from previous page)

```
node_size=10          # Base size (scaled up/down by degree)
)
```

How it works:

- Nodes with more connections → larger size
- Isolated nodes → smaller size
- Hub nodes are easily identifiable

Automatic Color Assignment

Py3plex uses colorblind-safe palettes by default:

```
# Rainbow colors (automatic assignment)
draw_multilayer_default(
    network.get_layers(),
    networks_color="rainbow" # Auto-assign distinct colors
)

# Custom palette
draw_multilayer_default(
    network.get_layers(),
    networks_color=["#FF6B6B", "#4ECDC4", "#45B7D1", "#FFA07A"]
)
```

Available palettes (from `py3plex.config`):

- `colorblind_safe`: 8 colors safe for colorblind viewers
- `wong`: 7-color palette (scientifically validated)
- `tol_bright`: 7 bright colors
- `rainbow`: Automatic generation

Automatic Layout Adjustment

Layout parameters auto-adjust to network size:

```
# For small networks, layout is compact
small_network = multinet.multi_layer_network()
# ... load 50-node network ...
draw_multilayer_default(small_network.get_layers()) # Compact layout

# For large networks, layout expands
large_network = multinet.multi_layer_network()
# ... load 1000-node network ...
draw_multilayer_default(large_network.get_layers()) # Expanded layout
```

Customization Options**Complete Parameter Reference**

```
draw_multilayer_default(
    network_list,          # List of layer subgraphs

    # Display control
    display=True,          # Show plot immediately
)
```

(continues on next page)

(continued from previous page)

```

axis=None,                # Matplotlib axis (None = create new)

# Node appearance
node_size=10,              # Base node size
scale_by_size=False,      # Scale by degree?
node_labels=False,        # Show node IDs?
node_font_size=5,         # Font size for node labels

# Edge appearance
edge_size=1.0,             # Edge thickness
alphalevel=0.13,          # Edge transparency (0=invisible, 1=opaque)
arrowsize=0.5,            # Arrow size (for directed graphs)

# Layout
background_shape="circle", # "circle" or "rectangle"
rectanglex=1.0,           # Layout width/radius
rectangley=1.0,           # Layout height/radius

# Colors
networks_color="rainbow", # "rainbow" or list of colors
background_color="rainbow", # Background color scheme

# Labels
labels=True,              # Show layer labels?
label_position=1.0,       # Label distance from center

# Cleanup
remove_isolated_nodes=False, # Remove disconnected nodes?

# Debugging
verbose=False             # Print debug info?
)

```

Layer-Specific Customization

Customize individual layers:

```

import matplotlib.pyplot as plt
from py3plex.visualization.multilayer import draw_multilayer_default

# Get layers
layers = network.get_layers()

# Modify specific layer (e.g., highlight layer 0)
for node in layers[0].nodes():
    layers[0].nodes[node]['color'] = 'red'
    layers[0].nodes[node]['size'] = 20

# Visualize with custom layer
draw_multilayer_default(layers, display=True)

```

Color-Coding by Community

Color nodes by community membership:

```

from py3plex.algorithms.community_detection import community_louvain
import matplotlib.pyplot as plt
import matplotlib.cm as cm

# Detect communities
communities = community_louvain.best_partition(network.core_network)

# Assign colors
num_communities = len(set(communities.values()))
colors = cm.rainbow([i/num_communities for i in range(num_communities)])

# Apply to network
for node, comm_id in communities.items():
    network.core_network.nodes[node]['color'] = colors[comm_id]

# Visualize
draw_multilayer_default(network.get_layers(), display=True)

```

Exporting Visualizations

Save to File

```

import matplotlib.pyplot as plt

# Create figure
fig, ax = plt.subplots(1, 1, figsize=(12, 8))

# Draw network
draw_multilayer_default(
    network.get_layers(),
    display=False,
    axis=ax
)

# Customize and save
plt.title("Multilayer Network Visualization")
plt.savefig('network.png', dpi=300, bbox_inches='tight')
plt.savefig('network.pdf', bbox_inches='tight') # Vector format
print("Visualization saved to network.png and network.pdf")

```

High-Quality Publications

```

import matplotlib.pyplot as plt

# Use publication settings
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.size'] = 10
plt.rcParams['figure.dpi'] = 300

# Large figure for detail
fig, ax = plt.subplots(1, 1, figsize=(10, 8))

# Dense mode for publications
draw_multilayer_default(
    network.get_layers(),
    display=False,

```

(continues on next page)

(continued from previous page)

```

    axis=ax,
    node_size=15,
    labels=True,
    alphalevel=0.5,
    background_shape="circle"
)

plt.title("Figure 1: Multilayer Network Structure", fontsize=12)
plt.savefig('publication_figure.pdf', bbox_inches='tight')
plt.savefig('publication_figure.png', dpi=300, bbox_inches='tight')

```

Interactive Visualizations

Using Plotly (Optional)

Py3plex provides interactive visualization capabilities through Plotly, allowing you to explore networks dynamically in your web browser.

Installation:

```

# Install plotly separately
pip install plotly

# Or install py3plex with visualization extras
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]

```

Basic Interactive Hairball Plot

The `interactive_hairball_plot` function creates an interactive 2D network visualization:

```

import networkx as nx
from py3plex.core import random_generators
from py3plex.visualization.multilayer import interactive_hairball_plot

# Generate a network
multilayer_net = random_generators.random_multilayer_ER(50, 3, 0.15, directed=False)

# Convert to NetworkX graph
network_colors, G = multilayer_net.get_layers(style="hairball")

# Compute layout
pos = nx.spring_layout(G, iterations=50, seed=42)

# Compute node sizes based on degree
degrees = dict(G.degree())
max_degree = max(degrees.values())
node_sizes = [10 + 40 * (degrees[node] / max_degree) for node in G.nodes()]

# Create color mapping
color_mapping = {node: node_sizes[i] for i, node in enumerate(G.nodes())}

# Generate interactive visualization
fig = interactive_hairball_plot(
    G,
    nsizes=node_sizes,
    final_color_mapping=color_mapping,
    pos=pos,

```

(continues on next page)

(continued from previous page)

```

    colorscale="Viridis" # Options: Viridis, Rainbow, Blues, Reds, etc.
)

# Save to HTML file
if fig:
    fig.write_html("network.html")
    print("Interactive visualization saved to network.html")

```

Interactive Features:

- **Hover:** Move your mouse over nodes to see details
- **Zoom:** Use mouse wheel to zoom in/out
- **Pan:** Click and drag to move the view
- **Select:** Click nodes to highlight connections

Color Scales Available:

- Viridis: Perceptually uniform, colorblind-friendly
- Rainbow: Full color spectrum
- Blues, Reds, Greens: Single-hue gradients
- RdBu, YlOrRd: Diverging color schemes
- Jet, Hot, Electric: High-contrast schemes

Advanced Interactive Multilayer Example

For more complex multilayer networks with custom styling:

```

from py3plex.core import multinet
from py3plex.visualization.multilayer import interactive_hairball_plot
import networkx as nx

# Create multilayer network
network = multinet.multi_layer_network()

# Add nodes to different layers
layers = ['social', 'professional', 'family']
nodes_per_layer = {
    'social': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'professional': ['Alice', 'Bob', 'Frank', 'Grace', 'Henry'],
    'family': ['Alice', 'Charlie', 'David', 'Ivan', 'Jane']
}

for layer in layers:
    for node in nodes_per_layer[layer]:
        network.add_nodes([{'source': node, 'type': layer}])

# Add edges (example for one layer)
network.add_edges([
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type':
    ↳ 'social'},
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↳ 'social'},
])

```

(continues on next page)

(continued from previous page)

```

# Convert to aggregate graph
G = nx.Graph()
for node in set(n for layer_nodes in nodes_per_layer.values() for n in layer_nodes):
    G.add_node(node)

# Compute layout and visualize
pos = nx.spring_layout(G, seed=42)
degrees = dict(G.degree())
node_sizes = [20 + 80 * (degrees[node] / max(degrees.values())) for node in G.nodes()]

# Assign colors by layer membership
layer_colors = {'social': 0, 'professional': 50, 'family': 100}
color_mapping = {}
for node in G.nodes():
    # Find primary layer for this node
    for layer, nodes in nodes_per_layer.items():
        if node in nodes:
            color_mapping[node] = layer_colors[layer]
            break

# Create interactive visualization
fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos, colorscale="Viridis
↔")

# Customize the figure
if fig:
    fig.update_layout(
        title="Interactive Multilayer Network",
        hovermode='closest',
        plot_bgcolor='rgba(240, 240, 240, 0.9)'
    )
    fig.write_html("multilayer_network.html")

```

Complete Working Examples:

See the following example scripts in the repository:

- examples/visualization/example_interactive_hairball.py - Basic interactive visualization
- examples/visualization/example_interactive_multilayer.py - Advanced multilayer interactive plot

These examples are standalone and can be run directly:

```

cd examples/visualization
python example_interactive_hairball.py
# Opens browser with interactive visualization

```

Jupyter Notebook Integration

Interactive visualizations work seamlessly in Jupyter notebooks:

```

# In Jupyter notebook
from py3plex.visualization.multilayer import interactive_hairball_plot

# ... prepare your graph, positions, etc. ...

fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos)

```

(continues on next page)

(continued from previous page)

```
# The figure will be displayed inline in the notebook
# No need to call fig.show() - it's called automatically
```

Notebook Tips:

1. The interactive plot appears directly in the notebook
2. All interactive features work in the notebook interface
3. Use `fig.write_html()` to save for sharing
4. Interactive plots work in JupyterLab, Jupyter Notebook, and VS Code notebooks

Saving Interactive Visualizations

Save your interactive visualizations for sharing or publishing:

```
# Create the figure
fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos)

# Save as standalone HTML
fig.write_html("network.html")

# Save as HTML with custom configuration
fig.write_html(
    "network.html",
    config={
        'displayModeBar': True, # Show toolbar
        'displaylogo': False, # Hide Plotly logo
        'modeBarButtonsToRemove': ['pan2d', 'lasso2d'] # Hide specific tools
    }
)

# Save as static image (requires kaleido)
# pip install kaleido
fig.write_image("network.png", width=1200, height=800)
fig.write_image("network.pdf") # Vector format
```

HTML File Features:

- Standalone - works without internet connection
- No dependencies needed in browser
- Full interactivity preserved
- Can be embedded in web pages
- Works on mobile devices

Embedding in Web Applications

Embed interactive visualizations in your web applications:

```
<!DOCTYPE html>
<html>
<head>
  <title>Network Visualization</title>
</head>
<body>
```

(continues on next page)

(continued from previous page)

```

<h1>My Network Analysis</h1>

<!-- Embed the interactive plot -->
<iframe src="network.html" width="100%" height="600px" frameborder="0"></iframe>

<p>Description of your network...</p>
</body>
</html>

```

Or include the plot div directly:

```

# Generate only the div (not full HTML page)
fig_html = fig.to_html(
    include_plotlyjs='cdn', # Use CDN for Plotly.js
    div_id='my-network-plot'
)

# Use fig_html in your web framework (Flask, Django, etc.)

```

Troubleshooting Interactive Visualizations

Issue: Plotly not available

```

# Solution: Install plotly
pip install plotly

```

Issue: Figure doesn't show in Jupyter

```

# Solution: Call fig.show() explicitly
fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos)
fig.show()

```

Issue: Slow performance with large networks

```

# Solution 1: Sample nodes for interactive view
import random
sample_nodes = random.sample(list(G.nodes()), min(500, G.number_of_nodes()))
G_sample = G.subgraph(sample_nodes)

# Solution 2: Use simpler layout
pos = nx.random_layout(G) # Faster than spring_layout

# Solution 3: Reduce node sizes and simplify colors
node_sizes = [5] * G.number_of_nodes() # Constant size
color_mapping = {node: 0 for node in G.nodes()} # Single color

```

Issue: HTML file is too large

```

# Solution: Reduce data in the plot
# 1. Use fewer nodes (sample or filter)
# 2. Use constant node sizes instead of varying
# 3. Simplify color scheme
# 4. Remove edge traces if not needed

```

Performance Guidelines

Network Size Recommendations:

- **< 100 nodes:** All features work smoothly
- **100-500 nodes:** Good performance, all features
- **500-1000 nodes:** Usable, may have slight lag
- **1000-5000 nodes:** Sample nodes or use simplified styling
- **> 5000 nodes:** Use static visualizations instead

Optimization Tips:

1. Use constant node sizes for large networks
2. Simplify color schemes (single color or discrete categories)
3. Use `nx.random_layout()` instead of `nx.spring_layout()` for speed
4. Sample nodes for interactive exploration
5. Consider static visualization for very large networks

Jupyter Notebook Integration

```
# Enable inline plotting
%matplotlib inline

# Or for interactive plots
%matplotlib notebook

# Visualize
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default(network.get_layers(), display=True)
```

Performance Tips

For Large Networks

1. **Use minimal mode** (small nodes, no labels)
2. **Sample nodes** if network is too large:

```
import random

# Sample 1000 random nodes
all_nodes = list(network.get_nodes())
sample_nodes = random.sample(all_nodes, min(1000, len(all_nodes)))

# Create subnetwork
subnetwork = network.get_subnetwork(sample_nodes)

# Visualize sample
draw_multilayer_default(subnetwork.get_layers(), display=True)
```

3. **Remove isolated nodes:**

```
draw_multilayer_default(
    network.get_layers(),
    remove_isolated_nodes=True # Faster rendering
)
```

4. Use **lower resolution** for interactive exploration:

```
plt.figure(figsize=(8, 6), dpi=100) # Lower DPI for speed
```

Troubleshooting

Visualization Not Showing

Issue: Plot doesn't appear

Solutions:

```
# Jupyter notebook: Enable inline plots
%matplotlib inline

# Python script: Add plt.show()
draw_multilayer_default(network.get_layers(), display=False)
plt.show()

# Headless server: Save to file
import matplotlib
matplotlib.use('Agg') # Must be before importing pyplot
```

Nodes Overlapping

Issue: Nodes overlap and are hard to distinguish

Solutions:

```
# 1. Use smaller node size
draw_multilayer_default(network.get_layers(), node_size=5)

# 2. Increase layout area
draw_multilayer_default(
    network.get_layers(),
    rectanglex=2.0, # Wider layout
    rectangley=2.0 # Taller layout
)

# 3. Sample nodes
# (See "Performance Tips" above)
```

Labels Too Crowded

Issue: Layer labels overlap or are too dense

Solutions:

```
# 1. Disable labels for large networks
draw_multilayer_default(network.get_layers(), labels=False)

# 2. Adjust label position
draw_multilayer_default(
    network.get_layers(),
    labels=True,
    label_position=1.2 # Move labels outward
)

# 3. Use smaller font
```

(continues on next page)

(continued from previous page)

```
draw_multilayer_default(
    network.get_layers(),
    node_font_size=3 # Smaller font
)
```

Advanced Visualization Modes

Py3plex provides multiple specialized visualization modes for multilayer networks, each optimized for different analysis goals. These modes can be accessed through the unified `visualize_multilayer_network` API or by calling individual plot functions.

Overview of Visualization Modes

The following visualization modes are available:

1. **diagonal** (default): Layer-centric diagonal layout with inter-layer edges
2. **small_multiples**: Side-by-side comparison of layers in a grid
3. **edge_colored_projection**: Aggregated view with color-coded edges by layer
4. **supra_adjacency_heatmap**: Matrix representation of the multilayer structure
5. **radial_layers**: Concentric circles with layers as rings
6. **ego_multilayer**: Focus on a single node's neighborhood across layers
7. **flow/alluvial**: Layered flow visualization with horizontal bands
8. **sankey**: Sankey diagram showing inter-layer flow strength

Unified API

All visualization modes can be accessed through the `visualize_multilayer_network` function:

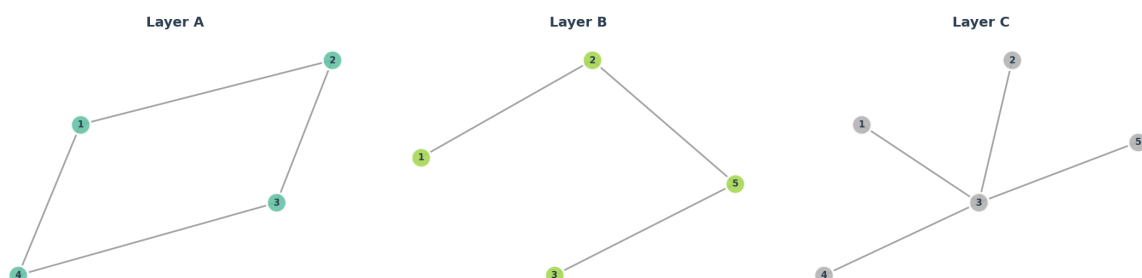
```
from py3plex.core import multinet
from py3plex.visualization.multilayer import visualize_multilayer_network

# Load network
network = multinet.multi_layer_network()
network.load_network("network.txt", input_type="multiedgelist")

# Use any visualization mode
fig = visualize_multilayer_network(
    network,
    visualization_type="small_multiples" # or any other mode
)
```

Small Multiples View

Displays each layer as a separate subplot in a grid layout, making it easy to compare layer structures side-by-side.



```

from py3plex.visualization.multilayer import visualize_multilayer_network

# Shared layout (nodes appear at same positions across layers)
fig = visualize_multilayer_network(
    network,
    visualization_type="small_multiples",
    shared_layout=True,          # Same node positions in all layers
    layout="spring",            # Layout algorithm
    node_size=300,
    max_cols=3,                 # Maximum columns in grid
    show_layer_titles=True,
    with_labels=True
)

# Independent layouts (optimized per layer)
fig = visualize_multilayer_network(
    network,
    visualization_type="small_multiples",
    shared_layout=False,        # Different layouts per layer
    layout="circular"
)

```

Best for:

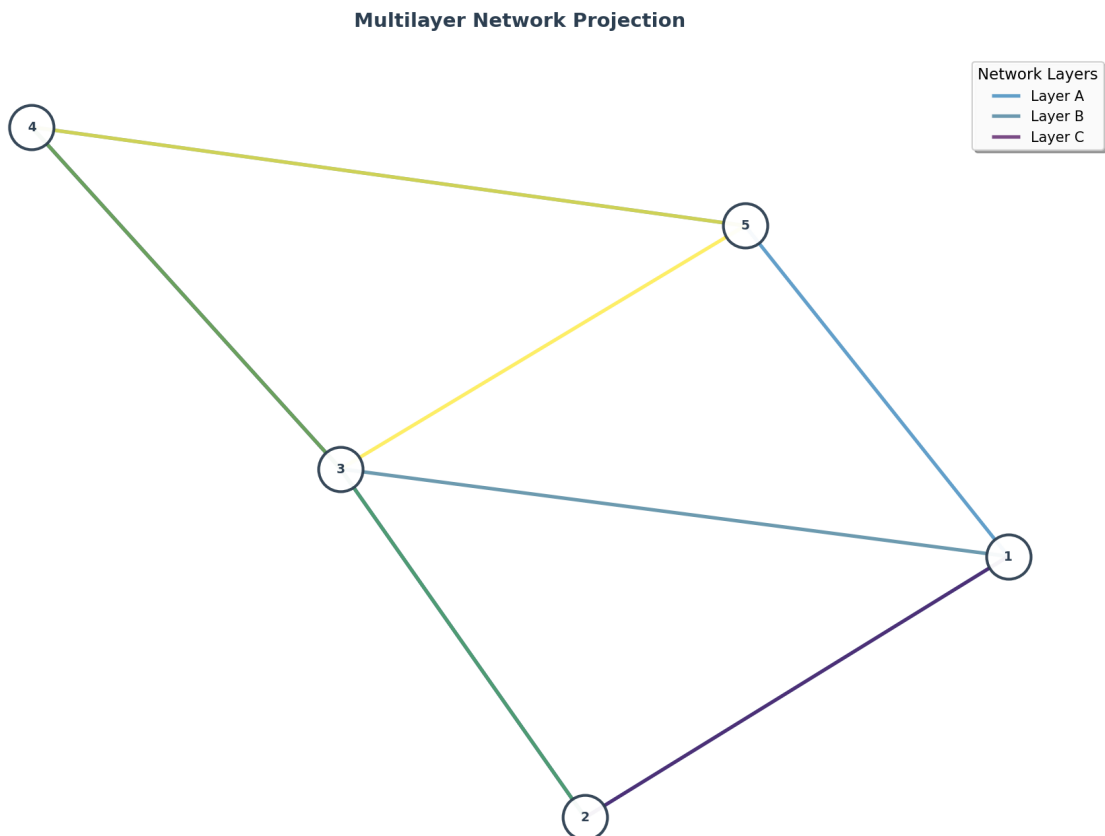
- Comparing layer structures
- Identifying layer-specific patterns
- Understanding node presence across layers
- Networks with 2-10 layers

Parameters:

- `shared_layout`: Use same node positions across layers
- `layout`: "spring", "circular", "random", "kamada_kawai"
- `max_cols`: Maximum number of columns in grid
- `show_layer_titles`: Display layer names

Edge-Colored Projection

Projects all layers onto a single 2D graph, using edge colors to indicate layer membership. Useful for seeing the overall structure while maintaining layer information.



```
from py3plex.visualization.multilayer import visualize_multilayer_network

# Basic projection
fig = visualize_multilayer_network(
    network,
    visualization_type="edge_colored_projection",
    layout="spring",
    node_size=500,
    edge_alpha=0.7,           # Edge transparency
    with_labels=True
)

# Custom layer colors
custom_colors = {
    'layer1': 'red',
    'layer2': 'blue',
    'layer3': 'green'
}

fig = visualize_multilayer_network(
    network,
    visualization_type="edge_colored_projection",
    layer_colors=custom_colors
)
```

Best for:

- Aggregate network structure
- Comparing edge distributions across layers

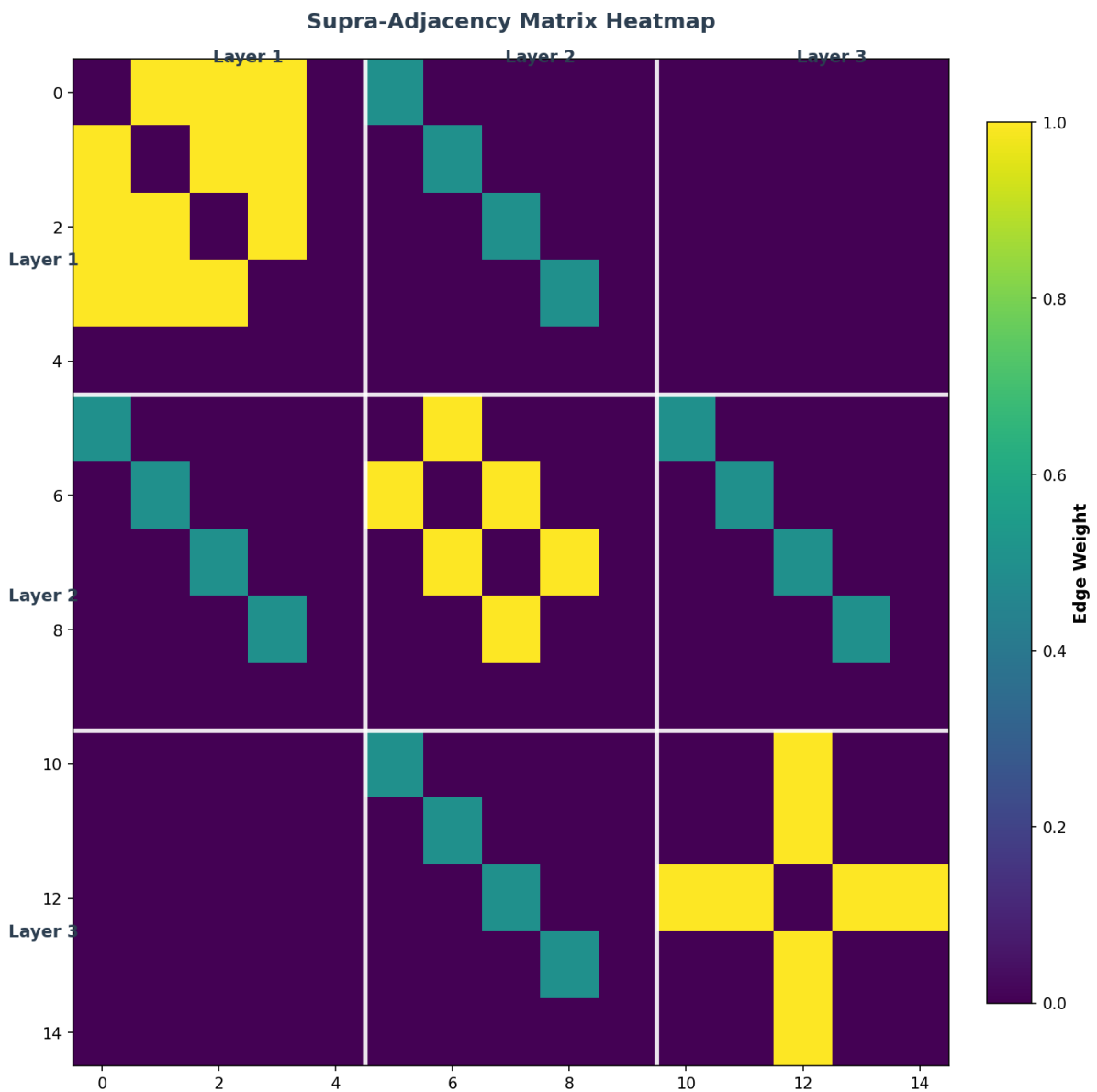
- Identifying layer-dominant connections
- Networks where edges don't heavily overlap

Parameters:

- `layout`: Layout algorithm for the aggregated graph
- `layer_colors`: Dict mapping layer names to colors
- `edge_alpha`: Transparency level for edges (0-1)
- `figsize`: Figure dimensions

Supra-Adjacency Heatmap

Shows the multilayer network as a block matrix where each block represents the adjacency matrix of one layer. Can optionally include inter-layer connections.



```
from py3plex.visualization.multilayer import visualize_multilayer_network

# Intra-layer only (block-diagonal structure)
```

(continues on next page)

(continued from previous page)

```
fig = visualize_multilayer_network(  
    network,  
    visualization_type="supra_adjacency_heatmap",  
    include_inter_layer=False,  
    cmap="Blues"  
)  
  
# With inter-layer coupling  
fig = visualize_multilayer_network(  
    network,  
    visualization_type="supra_adjacency_heatmap",  
    include_inter_layer=True,  
    inter_layer_weight=0.5, # Weight for inter-layer edges  
    cmap="viridis"  
)
```

Best for:

- Mathematical analysis of structure
- Identifying block patterns
- Understanding coupling between layers
- Spectral analysis preparation

Parameters:

- `include_inter_layer`: Show inter-layer connections
- `inter_layer_weight`: Default weight for inter-layer edges
- `cmap`: Matplotlib colormap name
- `figsize`: Figure dimensions

Interpretation:

- Diagonal blocks = intra-layer adjacency matrices
- Off-diagonal blocks = inter-layer connections
- White grid lines = layer boundaries

Radial/Concentric Layers

Arranges layers as concentric circles, with nodes positioned on rings and inter-layer edges shown as radial connections.

Radial Multilayer Network



```

from py3plex.visualization.multilayer import visualize_multilayer_network

# With inter-layer edges
fig = visualize_multilayer_network(
    network,
    visualization_type="radial_layers",
    base_radius=1.0,          # Radius of innermost layer
    radius_step=1.5,         # Distance between layers
    node_size=200,
    draw_inter_layer_edges=True,
    edge_alpha=0.5
)

# Intra-layer only
fig = visualize_multilayer_network(
    network,
    visualization_type="radial_layers",

```

(continues on next page)

(continued from previous page)

```
draw_inter_layer_edges=False
)
```

Best for:

- Temporal networks (layers as time steps)
- Hierarchical multilayer networks
- Visualizing node evolution across layers
- Networks with clear layer ordering

Parameters:

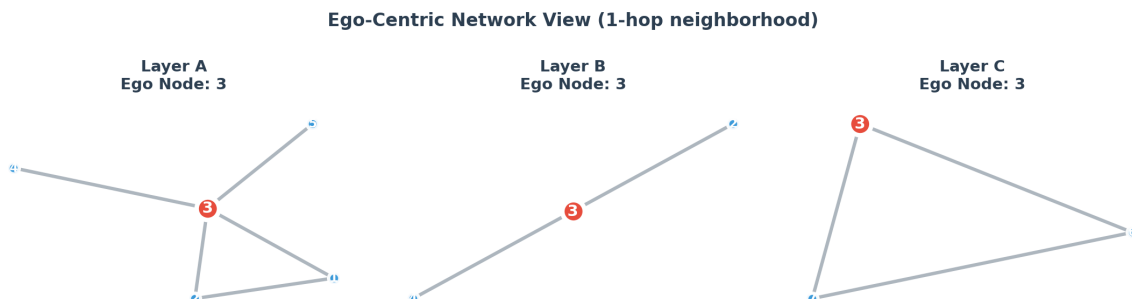
- `base_radius`: Radius of innermost ring
- `radius_step`: Distance between consecutive rings
- `draw_inter_layer_edges`: Show inter-layer connections
- `node_size`: Size of nodes

Interpretation:

- Each ring = one layer
- Same node appears at same angle on all rings
- Dashed lines = inter-layer connections

Ego-Centric Multilayer View

Focuses on a single node (the “ego”) and shows its neighborhood across different layers, highlighting the ego node’s position in each layer context.



```
from py3plex.visualization.multilayer import visualize_multilayer_network

# Basic ego view
fig = visualize_multilayer_network(
    network,
    visualization_type="ego_multilayer",
    ego='node_id',           # Node to focus on
    max_depth=1,            # Neighborhood depth (hops)
    layout="spring",
    node_size=100,
    ego_node_size=400       # Highlight ego node
)

# Specific layers only
fig = visualize_multilayer_network(
    network,
```

(continues on next page)

(continued from previous page)

```

visualization_type="ego_multilayer",
ego='node_id',
layers=['layer1', 'layer2'], # Only these layers
max_depth=2,                # 2-hop neighborhood
with_labels=True
)

```

Best for:

- Analyzing individual node behavior
- Comparing local structure across layers
- Identifying layer-specific influential neighbors
- Understanding node role variations

Parameters:

- ego: Node ID to focus on
- layers: Specific layers to visualize (or None for all)
- max_depth: Neighborhood depth in hops
- layout: Layout algorithm per ego graph
- ego_node_size: Size of the highlighted ego node

Interpretation:

- Red node = the ego (focal node)
- Blue nodes = neighbors of the ego
- Each subplot = ego's neighborhood in one layer

Flow and Sankey Diagrams

Py3plex provides two complementary visualizations for understanding inter-layer connectivity: flow/alluvial diagrams and Sankey diagrams. Both show how nodes and connections flow between layers, but with different visual approaches.

Flow/Alluvial Visualization

The flow visualization shows each layer as a horizontal band with nodes positioned along the x-axis. Inter-layer connections are drawn as flowing Bezier curves, with node activity encoded by color and size.

```

from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network()
network.load_network("network.txt", input_type="multiedgelist")

# Flow visualization
ax = network.visualize_network(style='flow', show=True)

# Or use 'alluvial' (same as 'flow')
ax = network.visualize_network(style='alluvial', show=True)

```

Best for:

- Visualizing node-level connectivity across layers
- Understanding individual node trajectories

- Showing how specific nodes connect between layers
- Networks with up to 5-6 layers

Sankey Diagram for Inter-Layer Flows

The Sankey-style diagram provides an aggregate view of inter-layer connection strength. The visualization uses text and arrows where width represents the number of connections between layers. This is ideal for understanding overall flow patterns rather than individual node connections.

```
from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network()
network.load_network("network.txt", input_type="multiedgelist")

# Sankey-style diagram showing inter-layer flow strength
ax = network.visualize_network(style='sankey', show=True)
```

You can also use the function directly:

```
from py3plex.visualization.sankey import draw_multilayer_sankey

# Get layers
labels, graphs, multilinks = network.get_layers()

# Create Sankey-style diagram
ax = draw_multilayer_sankey(
    graphs,
    multilinks,
    labels=labels,
    display=True
)
```

Best for:

- Analyzing aggregate inter-layer connection patterns
- Understanding flow strength between layers
- Identifying dominant layer connections
- Networks with 2-5 layers
- Summarizing inter-layer connectivity

Interpretation:

- Text shows layer-to-layer connections with counts
- Arrows indicate connection direction
- Arrow width proportional to connection strength
- Useful for identifying which layer pairs have strongest connections

Note:

This uses a simplified flow diagram approach rather than matplotlib's traditional Sankey class, as it's better suited for multilayer network inter-layer connections.

Example Workflows

Comparing Visualization Modes

Different modes reveal different aspects of the same network:

```
from py3plex.visualization.multilayer import visualize_multilayer_network
import matplotlib.pyplot as plt

# Load network
network = multinet.multi_layer_network()
network.load_network("network.txt", input_type="multiedgelist")

# Compare multiple views
modes = [
    "small_multiples",
    "edge_colored_projection",
    "supra_adjacency_heatmap",
    "radial_layers"
]

for mode in modes:
    fig = visualize_multilayer_network(network, visualization_type=mode)
    fig.savefig(f"network_{mode}.png", dpi=150, bbox_inches='tight')
    plt.close()
```

Analyzing Specific Nodes

Use ego-centric view to understand individual nodes:

```
# Find high-degree nodes
import networkx as nx

# Get aggregated network
agg_graph = nx.Graph()
for node in network.get_nodes():
    agg_graph.add_node(node[0])
for u, v in network.get_edges():
    agg_graph.add_edge(u[0], v[0])

# Get top nodes by degree
degrees = dict(agg_graph.degree())
top_nodes = sorted(degrees, key=degrees.get, reverse=True)[:5]

# Visualize each top node's ego network
for node in top_nodes:
    fig = visualize_multilayer_network(
        network,
        visualization_type="ego_multilayer",
        ego=node,
        max_depth=1
    )
    fig.savefig(f"ego_{node}.png", bbox_inches='tight')
    plt.close()
```

Choosing the Right Visualization

Use this guide to select the appropriate visualization mode:

For structural comparison:

- Use `small_multiples` to compare layer structures side-by-side
- Use `edge_colored_projection` to see aggregated structure with layer info

For mathematical analysis:

- Use `supra_adjacency_heatmap` for matrix-based analysis
- Useful for spectral methods and linear algebra operations

For temporal or hierarchical networks:

- Use `radial_layers` when layers have natural ordering
- Shows progression or hierarchy clearly

For local analysis:

- Use `ego_multilayer` to understand individual nodes
- Compare how a node's role varies across layers

For presentations:

- Use `edge_colored_projection` for clean, simple overview
- Use `small_multiples` when detail matters
- Use `radial_layers` for visually striking displays

What You Learned

This chapter covered visualization techniques for multilayer networks:

Basic visualization:

- `draw_multilayer_default()` for quick plots
- Preset modes (minimal, balanced, dense) for different network scales
- Layout options (circular, rectangular)

Customization:

- Node sizes, colors, and labels
- Edge transparency and width
- Layer-specific styling
- Community color-coding

Advanced modes:

- `small_multiples` — side-by-side layer comparison
- `edge_colored_projection` — aggregated view with layer colors
- `supra_adjacency_heatmap` — matrix representation
- `radial_layers` — concentric circles for ordered layers
- `ego_multilayer` — node-centric neighborhood view
- `flow` and `sankey` — inter-layer flow visualization

Interactive visualization:

- Plotly-based interactive plots with hover, zoom, pan

- HTML export for sharing
- Jupyter notebook integration

Best practices:

- Match visualization mode to network size and analysis goal
- Use sparse representations for performance
- Export as PDF/SVG for publication-quality figures

What's Next?

- *I/O and Serialization* — Load and save networks in various formats
- *Community Detection* — Detect communities for color-coding
- *Performance and Scalability Best Practices* — Optimize for large networks

For more examples, see the [visualization examples](#) in the GitHub repository.

8.3.8 SQL-like DSL for Multilayer Networks**Table of Contents**

- *Overview*
- *Basic Syntax*
- *Quick Start Example*
- *Query Components*
 - *SELECT Clause*
 - *WHERE Clause*
 - *COMPUTE Clause*
- *Example Queries*
 - *Basic Queries*
 - *Complex Queries*
 - *Analytical Queries*
- *Working with Results*
 - *Formatting Results*
- *Convenience Functions*
- *Use Cases*
 - *Hub Identification*
 - *Layer Comparison*
 - *Node Importance Ranking*
 - *Network Filtering*
- *Error Handling*
- *Complete Working Examples*
 - *Example 1: Basic Network Querying*
 - *Example 2: Multilayer Network Analysis*

- *Example 3: Hub Identification*
- *Example 4: Layer Comparison Workflow*
- *Example Files*
- *API Reference*
 - *Main Functions*
 - *Convenience Functions*
- *Limitations and Future Work*
- *Best Practices*
- *Performance Considerations*
- *Further Reading*
- *See Also*

Overview

Py3plex provides a Domain-Specific Language (DSL) for querying and analyzing multilayer networks using SQL-like syntax. This intuitive interface allows users to filter nodes and edges, compute network measures, and perform complex analyses with simple, readable queries.

The DSL enables you to express complex network queries in a natural, SQL-like language without writing verbose code. For example, instead of manually iterating through nodes and checking conditions, you can write:

```
execute_query(network, 'SELECT nodes WHERE layer="social" AND degree > 5')
```

The DSL is particularly useful for:

- **Interactive network exploration:** Quickly test hypotheses and explore network structure
- **Rapid prototyping:** Build analysis workflows without extensive coding
- **Educational purposes:** Learn network concepts with intuitive queries
- **Production pipelines:** Create maintainable, self-documenting analysis code

Basic Syntax

The DSL follows a SQL-inspired syntax:

```
SELECT target WHERE conditions COMPUTE measures
```

Where:

- **target:** Either nodes or edges
- **conditions:** Filtering criteria (optional)
- **measures:** Network measures to compute (optional)

Quick Start Example

Here's a complete working example to get you started:

```
from py3plex.core import multinet
from py3plex.dsl import execute_query, format_result

# Create a multilayer network
network = multinet.multi_layer_network(directed=False)
```

(continues on next page)

(continued from previous page)

```

# Add nodes to different layers
network.add_nodes([
    {'source': 'Alice', 'type': 'social'},
    {'source': 'Bob', 'type': 'social'},
    {'source': 'Charlie', 'type': 'social'},
    {'source': 'Alice', 'type': 'work'},
    {'source': 'Bob', 'type': 'work'},
])

# Add edges
network.add_edges([
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type':
    ↪ 'social'},
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↪ 'social'},
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'work', 'target_type': 'work'}
    ↪,
])

# Query 1: Select all nodes in the social layer
result = execute_query(network, 'SELECT nodes WHERE layer="social"')
print(f"Found {result['count']} nodes in social layer")
print(result['nodes'])

# Query 2: Find high-degree nodes
result = execute_query(network, 'SELECT nodes WHERE degree > 1')
print(format_result(result))

# Query 3: Compute centrality for filtered nodes
result = execute_query(
    network,
    'SELECT nodes WHERE layer="social" COMPUTE betweenness centrality'
)
for node, centrality in result['computed']['betweenness centrality'].items():
    print(f"{node}: {centrality:.4f}")

```

Expected Output:

```

Found 3 nodes in social layer
[('Alice', 'social'), ('Bob', 'social'), ('Charlie', 'social')]

Query: SELECT nodes WHERE degree > 1
Target: nodes
Count: 2

Nodes (showing 2 of 2):
('Alice', 'social')
('Bob', 'social')

('Alice', 'social'): 0.0000
('Bob', 'social'): 1.0000
('Charlie', 'social'): 0.0000

```

Query Components

SELECT Clause

Specifies what to select from the network:

```
SELECT nodes      # Select nodes
SELECT edges      # Select edges (edge queries in development)
```

Note: Current version primarily supports node queries.

WHERE Clause

Filters results based on conditions. Supports:

Layer filtering:

```
WHERE layer="transport"
WHERE layer="social"
```

Degree filtering:

```
WHERE degree > 5
WHERE degree >= 3
WHERE degree <= 10
```

Logical operators:

```
WHERE layer="social" AND degree > 3
WHERE layer="work" OR layer="social"
WHERE NOT layer="transport"
```

Comparison operators:

- = : Equal to
- != : Not equal to
- > : Greater than
- < : Less than
- >= : Greater than or equal
- <= : Less than or equal

COMPUTE Clause

Calculates network measures for filtered nodes:

```
COMPUTE degree
COMPUTE betweenness centrality
COMPUTE closeness centrality
COMPUTE eigenvector centrality
```

Supported measures:

- degree - Node degree
- degree centrality - Normalized degree centrality
- betweenness centrality - Betweenness centrality
- closeness centrality - Closeness centrality
- eigenvector centrality - Eigenvector centrality

- pagerank - PageRank score
- clustering - Clustering coefficient

Multiple measures:

```
COMPUTE degree betweenness_centrality closeness_centrality
```

Example Queries

Basic Queries

Select all nodes in a layer:

```
result = execute_query(network, 'SELECT nodes WHERE layer="social"')
```

Select high-degree nodes:

```
result = execute_query(network, 'SELECT nodes WHERE degree > 5')
```

Select all nodes (no filter):

```
result = execute_query(network, 'SELECT nodes')
```

Complex Queries

Combine multiple conditions:

```
# Nodes in transport layer with high degree
result = execute_query(
    network,
    'SELECT nodes WHERE layer="transport" AND degree > 5'
)
```

Use OR operator:

```
# Nodes in either social or work layer
result = execute_query(
    network,
    'SELECT nodes WHERE layer="social" OR layer="work"'
)
```

Degree range filtering:

```
# Nodes with moderate degree
result = execute_query(
    network,
    'SELECT nodes WHERE degree >= 2 AND degree <= 5'
)
```

Analytical Queries

Compute centrality for a layer:

```
result = execute_query(
    network,
    'SELECT nodes WHERE layer="transport" COMPUTE betweenness_centrality'
)
```

(continues on next page)

(continued from previous page)

```
# Access computed values
for node, centrality in result['computed']['betweenness_centrality'].items():
    print(f"{node}: {centrality}")
```

Multiple measures for filtered nodes:

```
result = execute_query(
    network,
    'SELECT nodes WHERE degree > 3 COMPUTE degree_centrality closeness_centrality'
)
```

Working with Results

The `execute_query` function returns a dictionary containing:

- `query`: Original query string
- `target`: Query target (nodes or edges)
- `nodes or edges`: List of selected items
- `count`: Number of items returned
- `computed`: Dictionary of computed measures (if `COMPUTE` used)

Example:

```
result = execute_query(network, 'SELECT nodes WHERE layer="social"')

# Access results
print(f"Found {result['count']} nodes")
for node in result['nodes']:
    print(node)

# If COMPUTE was used
if 'computed' in result:
    for measure, values in result['computed'].items():
        print(f"{measure}:")
        for node, value in values.items():
            print(f"    {node}: {value}")
```

Example Output:

```
Found 3 nodes
('Alice', 'social')
('Bob', 'social')
('Charlie', 'social')
```

Formatting Results

Use `format_result` for human-readable output:

```
from py3plex.dsl import format_result

result = execute_query(network, 'SELECT nodes WHERE degree > 3')
print(format_result(result, limit=10))
```

Convenience Functions

The DSL module provides convenience functions for common operations:

Select nodes by layer:

```
from py3plex.dsl import select_nodes_by_layer

nodes = select_nodes_by_layer(network, 'transport')
```

Select high-degree nodes:

```
from py3plex.dsl import select_high_degree_nodes

# All high-degree nodes
nodes = select_high_degree_nodes(network, min_degree=5)

# High-degree nodes in specific layer
nodes = select_high_degree_nodes(network, min_degree=5, layer='social')
```

Compute centrality for a layer:

```
from py3plex.dsl import compute_centrality_for_layer

centrality = compute_centrality_for_layer(
    network,
    layer='transport',
    centrality='betweenness centrality'
)
```

Use Cases

Hub Identification

Find important nodes in each layer:

```
for layer in ['social', 'work', 'transport']:
    result = execute_query(
        network,
        f'SELECT nodes WHERE layer="{layer}" AND degree > 5'
    )
    print(f"Hubs in {layer}: {result['count']}")
```

Layer Comparison

Compare network properties across layers:

```
layers = ['social', 'work', 'transport']

for layer in layers:
    result = execute_query(
        network,
        f'SELECT nodes WHERE layer="{layer}" COMPUTE degree'
    )
    degrees = result['computed']['degree']
    avg_degree = sum(degrees.values()) / len(degrees)
    print(f"{layer} average degree: {avg_degree:.2f}")
```

Node Importance Ranking

Rank nodes by multiple measures:

```
result = execute_query(
    network,
    'SELECT nodes WHERE layer="social" COMPUTE betweenness centrality degree_
    ↪ centrality'
)

# Combine measures for ranking
scores = {}
for node in result['nodes']:
    betweenness = result['computed']['betweenness centrality'].get(node, 0)
    degree_cent = result['computed']['degree centrality'].get(node, 0)
    scores[node] = betweenness + degree_cent

# Show top nodes
for node, score in sorted(scores.items(), key=lambda x: x[1], reverse=True)[:5]:
    print(f"{node}: {score:.4f}")
```

Network Filtering

Create subnetworks based on queries:

```
# Get high-degree nodes
result = execute_query(network, 'SELECT nodes WHERE degree > 5')
high_degree_nodes = result['nodes']

# Create subnetwork with these nodes
subnetwork = network.subnetwork(
    [node for node in high_degree_nodes],
    subset_by='node_layer_names'
)
```

Error Handling

The DSL raises specific exceptions for different error types:

```
from py3plex.dsl import execute_query, DSLSyntaxError, DSLExecutionError

try:
    result = execute_query(network, 'SELECT nodes WHERE invalid_condition')
except DSLSyntaxError as e:
    print(f"Syntax error: {e}")
except DSLExecutionError as e:
    print(f"Execution error: {e}")
```

Common syntax errors:

- Missing SELECT keyword
- Invalid target (not 'nodes' or 'edges')
- Malformed conditions
- Unknown operators
- Invalid measure names

Complete Working Examples

This section provides complete, runnable examples demonstrating various DSL features with expected outputs.

Example 1: Basic Network Querying

Create a simple social network and query it:

```
from py3plex.core import multinet
from py3plex.dsl import execute_query, format_result

# Create network
network = multinet.multi_layer_network(directed=False)

# Add nodes in social layer
network.add_nodes([
    {'source': 'Alice', 'type': 'social'},
    {'source': 'Bob', 'type': 'social'},
    {'source': 'Charlie', 'type': 'social'},
    {'source': 'David', 'type': 'social'},
])

# Add edges
network.add_edges([
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type':
    ↳ 'social'},
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↳ 'social'},
    {'source': 'Charlie', 'target': 'David', 'source_type': 'social', 'target_type':
    ↳ 'social'},
    {'source': 'Alice', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↳ 'social'},
])

# Query all nodes
result = execute_query(network, 'SELECT nodes WHERE layer="social"')
print(format_result(result))

# Find high-degree nodes
result = execute_query(network, 'SELECT nodes WHERE degree > 1')
print(f"High-degree nodes: {result['count']}")
```

Expected Output:

```
Query: SELECT nodes WHERE layer="social"
Target: nodes
Count: 4

Nodes (showing 4 of 4):
('Alice', 'social')
('Bob', 'social')
('Charlie', 'social')
('David', 'social')

High-degree nodes: 3
```

Example 2: Multilayer Network Analysis

Analyze a network with multiple layers:

```
from py3plex.core import multinet
from py3plex.dsl import execute_query

# Create multilayer network
network = multinet.multi_layer_network(directed=False)

# Add nodes to multiple layers
nodes = []
for person in ['Alice', 'Bob', 'Charlie']:
    for layer in ['social', 'work', 'family']:
        nodes.append({'source': person, 'type': layer})
network.add_nodes(nodes)

# Add edges in different layers
edges = [
    # Social connections
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type':
    ↪ 'social'},
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↪ 'social'},
    # Work connections
    {'source': 'Alice', 'target': 'Charlie', 'source_type': 'work', 'target_type':
    ↪ 'work'},
    # Family connections
    {'source': 'Alice', 'target': 'Charlie', 'source_type': 'family', 'target_type':
    ↪ 'family'},
]
network.add_edges(edges)

# Compare layers
for layer in ['social', 'work', 'family']:
    result = execute_query(network, f'SELECT nodes WHERE layer="{layer}"')
    print(f"{layer} layer: {result['count']} nodes")

    # Compute degree for this layer
    result = execute_query(network, f'SELECT nodes WHERE layer="{layer}" COMPUTE_
    ↪ degree')
    degrees = result['computed']['degree']
    avg_degree = sum(degrees.values()) / len(degrees) if degrees else 0
    print(f"    Average degree: {avg_degree:.2f}")
```

Expected Output:

```
social layer: 3 nodes
    Average degree: 1.33
work layer: 3 nodes
    Average degree: 0.67
family layer: 3 nodes
    Average degree: 0.67
```

Example 3: Hub Identification

Find and rank important nodes using multiple centrality measures:

```

from py3plex.core import multinet
from py3plex.dsl import execute_query

# Create network
network = multinet.multi_layer_network(directed=False)

# Add nodes
network.add_nodes([
    {'source': 'Alice', 'type': 'social'},
    {'source': 'Bob', 'type': 'social'},
    {'source': 'Charlie', 'type': 'social'},
    {'source': 'David', 'type': 'social'},
    {'source': 'Eve', 'type': 'social'},
])

# Add edges creating a star network centered on Bob
network.add_edges([
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type':
    ↳ 'social'},
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↳ 'social'},
    {'source': 'Bob', 'target': 'David', 'source_type': 'social', 'target_type':
    ↳ 'social'},
    {'source': 'Bob', 'target': 'Eve', 'source_type': 'social', 'target_type': 'social
    ↳ 'social'},
])

# Find high-degree nodes in social layer
result = execute_query(
    network,
    'SELECT nodes WHERE layer="social" AND degree >= 2'
)
print(f"Found {result['count']} hub nodes")

# Compute multiple centrality measures for hubs
result = execute_query(
    network,
    'SELECT nodes WHERE layer="social" AND degree >= 2 '
    'COMPUTE betweenness_centrality closeness_centrality degree_centrality'
)

# Rank nodes by betweenness centrality
if 'computed' in result and 'betweenness_centrality' in result['computed']:
    centralities = result['computed']['betweenness_centrality']
    sorted_nodes = sorted(centralities.items(), key=lambda x: x[1], reverse=True)

    print("\nTop nodes by betweenness centrality:")
    for node, centrality in sorted_nodes[:5]:
        print(f" {node}: {centrality:.4f}")

```

Expected Output:

Found 1 hub nodes

(continues on next page)

(continued from previous page)

```
Top nodes by betweenness centrality:
('Bob', 'social'): 1.0000
```

Example 4: Layer Comparison Workflow

Compare network structure across different layers:

```
from py3plex.core import multinet
from py3plex.dsl import execute_query

# Create multilayer network
network = multinet.multi_layer_network(directed=False)

# Add nodes to multiple layers
people = ['Alice', 'Bob', 'Charlie', 'David']
nodes = []
for person in people:
    for layer in ['social', 'work', 'transport']:
        nodes.append({'source': person, 'type': layer})
network.add_nodes(nodes)

# Add edges in different layers
network.add_edges([
    # Social (well connected)
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type':
    ↪ 'social'},
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↪ 'social'},
    {'source': 'Charlie', 'target': 'David', 'source_type': 'social', 'target_type':
    ↪ 'social'},
    {'source': 'Alice', 'target': 'Charlie', 'source_type': 'social', 'target_type':
    ↪ 'social'},
    # Work (moderately connected)
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'work', 'target_type': 'work'}
    ↪ ,
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'work', 'target_type': 'work'
    ↪ },
    # Transport (sparsely connected)
    {'source': 'Alice', 'target': 'David', 'source_type': 'transport', 'target_type':
    ↪ 'transport'},
])

layers = ['social', 'work', 'transport']
layer_stats = {}

for layer in layers:
    # Get nodes in this layer
    result = execute_query(network, f'SELECT nodes WHERE layer="{layer}"')
    node_count = result['count']

    # Compute centrality measures
    result = execute_query(
        network,
        f'SELECT nodes WHERE layer="{layer}" COMPUTE betweenness_centrality'
    )
```

(continues on next page)

(continued from previous page)

```

    if 'computed' in result and 'betweenness centrality' in result['computed']:
        centralities = result['computed']['betweenness centrality']
        avg_centrality = sum(centralities.values()) / len(centralities) if
↪ centralities else 0
        max_centrality = max(centralities.values()) if centralities else 0

    layer_stats[layer] = {
        'nodes': node_count,
        'avg_centrality': avg_centrality,
        'max_centrality': max_centrality
    }

# Print comparison
print("\nLayer Comparison:")
print(f"{'Layer':<12} {'Nodes':<8} {'Avg Centrality':<16} {'Max Centrality':<16}")
print("-" * 55)
for layer, stats in layer_stats.items():
    print(f"{'layer':<12} {'stats['nodes']':<8} {'stats['avg_centrality']':<16.4f} {'stats[
↪ 'max_centrality']':<16.4f}")

```

Expected Output:

Layer Comparison:			
Layer	Nodes	Avg Centrality	Max Centrality

social	4	0.1667	0.5000
work	4	0.0833	0.3333
transport	4	0.0000	0.0000

Example Files

Additional complete examples are available in the repository:

- `examples/network_analysis/example_dsl_queries.py` - Basic DSL usage with detailed output
- `examples/network_analysis/example_dsl_advanced.py` - Advanced queries and transportation network analysis

Run these examples:

```

cd examples/network_analysis
python example_dsl_queries.py
python example_dsl_advanced.py

```

API Reference**Main Functions**

```

def execute_query(network: Any, query: str) -> Dict[str, Any]:
    """Execute a DSL query on a multilayer network.

    Args:
        network: Multilayer network object
        query: DSL query string

    Returns:
        Dictionary with 'nodes'/'edges', 'count', and optionally 'computed'

```

(continues on next page)

(continued from previous page)

```

"""
def format_result(result: Dict[str, Any], limit: int = 10) -> str:
    """Format query result as human-readable string.

    Args:
        result: Result from execute_query
        limit: Maximum items to display

    Returns:
        Formatted string
    """

```

Convenience Functions

```

def select_nodes_by_layer(network: Any, layer: str) -> List[Any]:
    """Select all nodes in a specific layer."""

def select_high_degree_nodes(network: Any, min_degree: int,
                             layer: Optional[str] = None) -> List[Any]:
    """Select nodes with degree above threshold."""

def compute_centrality_for_layer(network: Any, layer: str,
                                centrality: str = 'betweenness_centrality') ->
Dict[Any, float]:
    """Compute centrality for all nodes in a layer."""

```

Limitations and Future Work

Current limitations:

- Edge queries are not yet fully supported
- Complex nested conditions require multiple queries
- Limited to NetworkX-based measures
- No aggregation functions (SUM, AVG, etc.)

Planned enhancements:

- Full edge query support
- Nested subqueries
- Aggregation operators
- Custom measure registration
- Query optimization
- Save/load query results

Best Practices

1. **Start simple:** Begin with basic queries and add complexity incrementally
2. **Validate data:** Ensure network is properly constructed before querying
3. **Check results:** Inspect result counts and samples before processing
4. **Use convenience functions:** For common operations, use provided shortcuts
5. **Handle errors:** Wrap queries in try-except for robust code

6. **Performance:** For large networks, filter by layer first to reduce computation

Performance Considerations

- Computing centrality measures can be expensive on large networks
- Filter by layer first to reduce search space
- Cache computed measures if reusing them
- Consider using convenience functions for better performance
- Pre-compute measures and store in node attributes for repeated use

Example performance optimization:

```
# Less efficient - computes centrality multiple times
for threshold in [3, 5, 7]:
    result = execute_query(
        network,
        f'SELECT nodes WHERE degree > {threshold} COMPUTE betweenness centrality'
    )

# More efficient - compute once, filter in post-processing
result = execute_query(
    network,
    'SELECT nodes COMPUTE betweenness centrality'
)
centralities = result['computed']['betweenness centrality']

for threshold in [3, 5, 7]:
    high_degree = [n for n in result['nodes']
                   if network.core_network.degree(n) > threshold]
```

Further Reading

- [../basic_usage](#) - Network construction basics
- [../multilayer_concepts](#) - Understanding multilayer networks
- [../algorithm_guide](#) - Network analysis algorithms
- [Analysis Recipes & Workflows](#) - Common analysis patterns

See Also

- NetworkX documentation for centrality measures
- Examples directory for complete use cases
- API documentation for detailed function signatures

8.3.9 Dplyr-style Chainable Graph Operations

Table of Contents

- *Overview*
- *Key Concepts*
 - *NodeFrame and EdgeFrame*
 - *Top-level Helpers*

- *Quick Start Example*
- *Verb Reference*
 - *Filter*
 - *Select*
 - *Mutate*
 - *Arrange (Sort)*
 - *Head (Take)*
 - *Group By + Summarise*
- *Export Methods*
 - *to_pandas*
 - *to_subgraph*
- *Mapping to dplyr Concepts*
- *Complete Examples*
 - *Example 1: Hub Identification*
 - *Example 2: Layer Comparison*
 - *Example 3: Edge Filtering and Analysis*
 - *Example 4: Subgraph Extraction*
- *API Reference*
 - *nodes()*
 - *edges()*
 - *NodeFrame*
 - *EdgeFrame*
 - *GroupedNodeFrame*
- *Best Practices*
- *Comparison with DSL*
- *See Also*

Overview

Py3plex provides a dplyr-style, fluent method-chaining interface for working with nodes and edges in multilayer networks. This API is inspired by R's dplyr package and provides verbs like `filter`, `select`, `mutate`, `arrange`, `group_by`, and `summarise`.

The API enables you to express complex data manipulations as readable chains of operations:

```
from py3plex.graph_ops import nodes, edges
import numpy as np

df = (
    nodes(multinet, layers=["ppi"])
    .filter(lambda n: n["degree"] > 10)
    .mutate(k=lambda n: n["degree"] / (n["weight"] + 1))
    .group_by("layer")
    .summarise(avg_degree=("degree", np.mean))
```

(continues on next page)

(continued from previous page)

```
.to_pandas()
)
```

The `graph_ops` module is particularly useful for:

- **Fluent data manipulation:** Chain operations naturally without intermediate variables
- **Aggregation and summarization:** Group data and compute statistics
- **Integration with pandas:** Export results directly to DataFrames
- **Filtering and transformation:** Apply complex logic to nodes and edges

Key Concepts

NodeFrame and EdgeFrame

The two main “frame” types wrap collections of nodes or edges:

- **NodeFrame:** A chainable view over a collection of nodes
- **EdgeFrame:** A chainable view over a collection of edges

Both wrap:

- A reference to the underlying py3plex graph object (multinet)
- A current selection of nodes or edges (as a list of dicts)

All verbs return a new NodeFrame/EdgeFrame, enabling method chaining.

Top-level Helpers

Use the `nodes()` and `edges()` functions to create frames:

```
from py3plex.graph_ops import nodes, edges

# Get all nodes from the network
node_frame = nodes(multinet)

# Get nodes from specific layers
node_frame = nodes(multinet, layers=["layer1", "layer2"])

# Get all edges
edge_frame = edges(multinet)

# Get edges from specific layers
edge_frame = edges(multinet, layers=["ppi"])
```

Quick Start Example

Here’s a complete working example:

```
from py3plex.core import multinet
from py3plex.graph_ops import nodes, edges
import numpy as np

# Create a multilayer network
network = multinet.multi_layer_network(directed=False)

# Add nodes
network.add_nodes([
```

(continues on next page)

(continued from previous page)

```

    {'source': 'A', 'type': 'layer1'},
    {'source': 'B', 'type': 'layer1'},
    {'source': 'C', 'type': 'layer1'},
    {'source': 'D', 'type': 'layer1'},
    {'source': 'A', 'type': 'layer2'},
    {'source': 'B', 'type': 'layer2'},
])

# Add edges
network.add_edges([
    {'source': 'A', 'target': 'B', 'source_type': 'layer1', 'target_type': 'layer1',
    ↪ 'weight': 1.0},
    {'source': 'B', 'target': 'C', 'source_type': 'layer1', 'target_type': 'layer1',
    ↪ 'weight': 2.0},
    {'source': 'C', 'target': 'D', 'source_type': 'layer1', 'target_type': 'layer1',
    ↪ 'weight': 3.0},
    {'source': 'A', 'target': 'B', 'source_type': 'layer2', 'target_type': 'layer2',
    ↪ 'weight': 0.5},
])

# Example 1: Filter + mutate + to_pandas
df = (
    nodes(network, layers=["layer1"])
    .filter(lambda n: n["degree"] > 1)
    .mutate(k=lambda n: n["degree"] / (n.get("weight", 1) + 1))
    .to_pandas()
)
print(df)

# Example 2: Grouping and summarising
df_summary = (
    nodes(network)
    .group_by("layer")
    .summarise(
        avg_degree=("degree", np.mean),
        n=("id", len),
    )
    .arrange("avg_degree", reverse=True)
    .to_pandas()
)
print(df_summary)

# Example 3: Edges
df_edges = (
    edges(network, layers=["layer1"])
    .filter(lambda e: e.get("weight", 0) > 1.5)
    .head(10)
    .to_pandas()
)
print(df_edges)

```

Verb Reference

Filter

Filter nodes/edges using a predicate function:

```
# Using a lambda predicate
result = nodes(network).filter(lambda n: n["degree"] > 5)

# Filter edges by weight
result = edges(network).filter(lambda e: e.get("weight", 0) > 0.8)
```

Or use expression strings for simple conditions:

```
result = nodes(network).filter_expr("degree > 10 and layer == 'ppi'")
```

Signature:

```
def filter(self, predicate: Callable[[dict], bool]) -> "NodeFrame": ...
def filter_expr(self, expr: str) -> "NodeFrame": ...
```

Select

Keep only the specified attributes:

```
result = nodes(network).select("id", "layer", "degree")
```

Signature:

```
def select(self, *fields: str) -> "NodeFrame": ...
```

If no fields are passed, behaves as a no-op.

Mutate

Compute new attributes from existing values:

```
import math

result = nodes(network).mutate(
    k=lambda n: n["degree"] / (n.get("weight", 1) + 1),
    log_degree=lambda n: math.log1p(n["degree"]),
)
```

Signature:

```
def mutate(self, **new_fields: Callable[[dict], Any]) -> "NodeFrame": ...
```

Arrange (Sort)

Sort nodes/edges by an attribute or custom key:

```
# Sort by attribute name
result = nodes(network).arrange("degree", reverse=True)

# Sort using a custom key function
result = nodes(network).arrange(lambda n: -n["degree"])
```

Signature:

```
def arrange(self, key: Union[str, Callable[[dict], Any]], reverse: bool = False) ->
    ↪ "NodeFrame": ...
```

Head (Take)

Keep only the first n rows:

```
result = nodes(network).head(10)  # First 10 nodes
```

Signature:

```
def head(self, n: int = 5) -> "NodeFrame": ...
```

Group By + Summarise

Group data and compute aggregations:

```
import numpy as np

result = (
    nodes(network)
    .group_by("layer")
    .summarise(
        avg_degree=("degree", np.mean),
        n=("id", len),
    )
)
```

Signatures:

```
def group_by(self, *fields: str) -> "GroupedNodeFrame": ...

def summarise(self, **aggregations: tuple[str, Callable[[list[Any]], Any]]) ->
    ↪ "NodeFrame": ...
```

The aggregations are tuples of (field_name, aggregation_function).

Export Methods

to_pandas

Convert the current selection to a pandas DataFrame:

```
df = nodes(network).to_pandas()

# Chain with other operations
df = (
    nodes(network)
    .filter(lambda n: n["degree"] > 5)
    .select("id", "layer", "degree")
    .to_pandas()
)
```

Signature:

```
def to_pandas(self) -> pandas.DataFrame: ...
```

to_subgraph

Build a py3plex subgraph containing only the selected nodes:

```

subgraph = (
    nodes(network)
    .filter(lambda n: n["layer"] == "ppi")
    .to_subgraph()
)

```

Signature:

```
def to_subgraph(self) -> Any: ...
```

Mapping to dplyr Concepts

The following table shows the correspondence between dplyr verbs and graph_ops methods:

dplyr Verb	graph_ops Method
dplyr::filter	NodeFrame.filter / EdgeFrame.filter
dplyr::select	NodeFrame.select / EdgeFrame.select
dplyr::mutate	NodeFrame.mutate / EdgeFrame.mutate
dplyr::arrange	NodeFrame.arrange / EdgeFrame.arrange
dplyr::head	NodeFrame.head / EdgeFrame.head
dplyr::group_by``  ``	NodeFrame.group_by
dplyr::summarise``  ``	GroupedNodeFrame.summarise

Note

There is no direct equivalent of joins or relational joins yet. The design is open for future extensions like joining on node ID, layer, etc.

Complete Examples**Example 1: Hub Identification**

Find and analyze hub nodes across layers:

```

from py3plex.core import multinet
from py3plex.graph_ops import nodes
import numpy as np

network = multinet.multi_layer_network(directed=False)
# ... add nodes and edges ...

# Find high-degree nodes (hubs) in each layer
hub_summary = (
    nodes(network)
    .filter(lambda n: n["degree"] >= 5) # Only high-degree nodes
    .group_by("layer")
    .summarise(
        hub_count=("id", len),
        avg_hub_degree=("degree", np.mean),
        max_degree=("degree", max),
    )
    .arrange("hub_count", reverse=True)
    .to_pandas()
)
print(hub_summary)

```

Example 2: Layer Comparison

Compare network properties across different layers:

```
from py3plex.graph_ops import nodes
import numpy as np

# Get per-layer statistics
layer_stats = (
    nodes(network)
    .group_by("layer")
    .summarise(
        node_count=("id", len),
        avg_degree=("degree", np.mean),
        total_degree=("degree", sum),
    )
    .to_pandas()
)
print(layer_stats)
```

Example 3: Edge Filtering and Analysis

Filter edges and compute statistics:

```
from py3plex.graph_ops import edges

# Find high-weight edges
high_weight_edges = (
    edges(network, layers=["ppi", "coexpr"])
    .filter(lambda e: e.get("weight", 0) > 0.8)
    .mutate(
        is_intra_layer=lambda e: e["source_layer"] == e["target_layer"],
        weight_class=lambda e: "high" if e.get("weight", 0) > 0.9 else "medium",
    )
    .arrange("weight", reverse=True)
    .head(100)
    .to_pandas()
)
print(high_weight_edges)
```

Example 4: Subgraph Extraction

Create a subgraph from filtered nodes:

```
from py3plex.graph_ops import nodes

# Extract a subgraph of high-degree nodes in layer1
subgraph = (
    nodes(network)
    .filter(lambda n: n["layer"] == "layer1")
    .filter(lambda n: n["degree"] >= 3)
    .to_subgraph()
)

# Analyze the subgraph
print(f"Subgraph has {len(list(subgraph.get_nodes()))} nodes")
```

API Reference

nodes()

```
def nodes(multinet: Any, layers: Optional[List[str]] = None) -> NodeFrame:
    """Create a NodeFrame from a py3plex multi_layer_network.

    Args:
        multinet: py3plex multi_layer_network object
        layers: Optional list of layers to restrict to

    Returns:
        A NodeFrame wrapping the network's nodes
    """
```

edges()

```
def edges(multinet: Any, layers: Optional[List[str]] = None) -> EdgeFrame:
    """Create an EdgeFrame from a py3plex multi_layer_network.

    Args:
        multinet: py3plex multi_layer_network object
        layers: Optional list of layers to restrict to

    Returns:
        An EdgeFrame wrapping the network's edges
    """
```

NodeFrame

```
@dataclass
class NodeFrame:
    """A chainable view over a collection of nodes.

    Attributes:
        multinet: Reference to the underlying py3plex multi_layer_network
        data: Current selection of nodes as a list of dicts
    """

    def filter(self, predicate: Callable[[dict], bool]) -> "NodeFrame": ...
    def filter_expr(self, expr: str) -> "NodeFrame": ...
    def select(self, *fields: str) -> "NodeFrame": ...
    def mutate(self, **new_fields: Callable[[dict], Any]) -> "NodeFrame": ...
    def arrange(self, key: Union[str, Callable], reverse: bool = False) -> "NodeFrame": ...
    def head(self, n: int = 5) -> "NodeFrame": ...
    def group_by(self, *fields: str) -> "GroupedNodeFrame": ...
    def to_pandas(self) -> pandas.DataFrame: ...
    def to_subgraph(self) -> Any: ...
```

EdgeFrame

```
@dataclass
class EdgeFrame:
    """A chainable view over a collection of edges.
```

(continues on next page)

(continued from previous page)

```

Attributes:
    multinet: Reference to the underlying py3plex multi_layer_network
    data: Current selection of edges as a list of dicts
    """

def filter(self, predicate: Callable[[dict], bool]) -> "EdgeFrame": ...
def filter_expr(self, expr: str) -> "EdgeFrame": ...
def select(self, *fields: str) -> "EdgeFrame": ...
def mutate(self, **new_fields: Callable[[dict], Any]) -> "EdgeFrame": ...
def arrange(self, key: Union[str, Callable], reverse: bool = False) -> "EdgeFrame"
→": ...
def head(self, n: int = 5) -> "EdgeFrame": ...
def group_by(self, *fields: str) -> "GroupedEdgeFrame": ...
def to_pandas(self) -> pandas.DataFrame: ...

```

GroupedNodeFrame

```

@dataclass
class GroupedNodeFrame:
    """A grouped view of nodes for aggregation operations.

    Attributes:
        parent: The parent NodeFrame from which this was created
        group_fields: Tuple of field names to group by
    """

    def summarise(self, **aggregations: tuple) -> "NodeFrame": ...
    def summarize(self, **aggregations: tuple) -> "NodeFrame": ... # Alias

```

Best Practices

1. **Start simple:** Begin with basic filters and add complexity incrementally
2. **Use method chaining:** Chain operations for readable, fluent code
3. **Handle missing values:** Use `.get()` for optional attributes:

```
frame.mutate(x=lambda n: n.get("weight", 1) * 2)
```

4. **Export early for debugging:** Use `.to_pandas()` to inspect intermediate results
5. **Filter before grouping:** Reduce data volume before aggregation for better performance
6. **Name aggregations clearly:** Use descriptive names in `summarise()`

Comparison with DSL

The `graph_ops` module complements the existing SQL-like DSL:

Feature	DSL	graph_ops
Syntax	SQL-like strings	Python method chaining
Filtering	WHERE clauses	<code>.filter()</code> with lambdas
Aggregation	COMPUTE measures	<code>.group_by().summarise()</code>
Custom logic	Limited	Full Python expressiveness
Type safety	None (strings)	Full with type hints
Best for	Quick queries, exploration	Complex transformations, data pipelines

Use the DSL for quick, exploratory queries. Use `graph_ops` for complex data transformations, custom aggregations, and integration with pandas workflows.

See Also

- *SQL-like DSL for Multilayer Networks* - SQL-like DSL for network queries
- *Working with Networks* - Working with multilayer networks
- *Network Statistics* - Network statistics and measures

8.3.10 Analysis Recipes & Workflows

This guide provides **practical recipes** for **common analysis workflows** in py3plex. Each recipe is a **complete, ready-to-use solution** for a real-world task.

Recipe Index

- *Data Import & Setup*
 - *Recipe 1: Load Network from Multiple File Formats*
 - *Recipe 2: Create Synthetic Benchmark Networks*
 - *Recipe 3: Convert Between Network Formats*
- *Network Exploration & Analysis*
 - *Recipe 4: Quick Network Summary*
 - *Recipe 5: Layer-by-Layer Analysis*
 - *Recipe 6: Compute Multilayer Statistics*
- *Centrality & Node Importance*
 - *Recipe 7: Single-Layer Centrality Analysis*
 - *Recipe 8: Multiplex Participation Coefficient*
- *Community Detection*
 - *Recipe 9: Multilayer Community Detection*
 - *Recipe 10: Compare Community Detection Algorithms*
- *Visualization*
 - *Recipe 11: Basic Network Visualization*
 - *Recipe 12: Visualize Communities on Network*
- *Advanced Workflows*
 - *Recipe 13: Complete Analysis Pipeline*
 - *Recipe 14: Network Comparison Study*
 - *Recipe 15: Random Walks for Node Embeddings*
- *Performance & Optimization*
 - *Recipe 16: Handle Large Networks Efficiently*
 - *Recipe 17: Benchmark and Profile Your Analysis*
- *Tips & Best Practices*
 - *General Tips*

- *Performance Tips*
- *Visualization Tips*
- *Community Detection Tips*
- *Common Issues & Solutions*
 - *Issue: “Network appears to be empty”*
 - *Issue: “No module named ‘community’”*
 - *Issue: “Visualization doesn’t appear”*
 - *Issue: “Memory error with large network”*
 - *Issue: “Community detection is slow”*
- *Further Reading*

Data Import & Setup

Recipe 1: Load Network from Multiple File Formats

Task: Load **multilayer networks** from various **file formats** (edgelist, GraphML, CSV).

```
from py3plex.core import multinet

# From multiedgelist (node1, layer1, node2, layer2, weight)
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist",
    directed=False
)

# From GraphML
network = multinet.multi_layer_network().load_network(
    "data/network.graphml",
    input_type="graphml"
)

# From edges list programmatically
network = multinet.multi_layer_network()
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1.0],
    ['B', 'layer1', 'C', 'layer1', 1.0],
    ['A', 'layer2', 'C', 'layer2', 0.5],
], input_type="list")

# Verify the network
network.basic_stats()
```

Expected Output:

```
Number of nodes: 3
Number of edges: 3
Number of unique nodes (as node-layer tuples): 5
Number of unique node IDs (across all layers): 3
Nodes per layer:
  Layer 'layer1': 3 nodes
  Layer 'layer2': 2 nodes
```

When to use: Starting any **analysis project** with external data.

Recipe 2: Create Synthetic Benchmark Networks

Task: Generate **synthetic multilayer networks** for **testing algorithms**.

```
from py3plex.core.random_generators import random_multilayer_ER
import numpy as np

# Set random seed for reproducibility
np.random.seed(42)

# Create a random Erdős-Rényi multilayer network
# Parameters: n_nodes, n_layers, edge_probability
network = random_multilayer_ER(
    n=100,          # 100 nodes
    L=3,           # 3 layers
    p=0.1,         # 10% edge probability
    directed=False
)

network.basic_stats()
```

When to use: Algorithm validation, testing, benchmarking, or when real data is unavailable.

Recipe 3: Convert Between Network Formats

Task: Export networks to **different formats** for use in **other tools**.

```
from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network().load_network(
    "input.txt",
    input_type="multiedgelist"
)

# Save as GraphML (standard format, compatible with Gephi, Cytoscape)
network.save_network("output.graphml", output_type="graphml")

# Save as edge list with node mapping
inverse_map = network.serialize_to_edgelist("output_edges.txt")

# Save as pickle (preserves all Python objects)
import pickle
with open("network.pkl", "wb") as f:
    pickle.dump(network, f)
```

When to use: Sharing data with collaborators, importing to visualization tools, or archiving results.

Network Exploration & Analysis

Recipe 4: Quick Network Summary

Task: Get comprehensive overview of network structure.

```
from py3plex.core import multinet
```

(continues on next page)

(continued from previous page)

```

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Basic statistics
print("=== Basic Statistics ===")
network.basic_stats()

# Layer information
layers = network.get_layers()
print(f"\nLayers: {layers}")
print(f"Number of layers: {len(layers)}")

# Node and edge counts
nodes = list(network.get_nodes())
edges = list(network.get_edges())
print(f"Total nodes: {len(nodes)}")
print(f"Total edges: {len(edges)}")

# Get unique nodes (nodes may appear in multiple layers)
unique_nodes = set([n[0] for n in nodes])
print(f"Unique nodes: {len(unique_nodes)}")

```

Expected Output:

```

=== Basic Statistics ===
Number of nodes: 250
Number of edges: 180
Number of unique nodes (as node-layer tuples): 250
Number of unique node IDs (across all layers): 120
Nodes per layer:
  Layer 'collaboration': 85 nodes
  Layer 'citation': 90 nodes
  Layer 'coauthor': 75 nodes

Layers: ['collaboration', 'citation', 'coauthor']
Number of layers: 3
Total nodes: 250
Total edges: 180
Unique nodes: 120

```

When to use: Initial data exploration, quality checks, reporting.**Recipe 5: Layer-by-Layer Analysis****Task:** Analyze each layer independently and compare properties.

```

from py3plex.core import multinet
import networkx as nx

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Analyze each layer

```

(continues on next page)

(continued from previous page)

```

layers = network.get_layers()
layer_stats = {}

for layer_name in layers:
    # Extract single layer
    layer = network.subnetwork([layer_name], subset_by="layers")

    # Compute layer statistics
    stats = {}
    stats['nodes'] = len(list(layer.get_nodes()))
    stats['edges'] = len(list(layer.get_edges()))

    # NetworkX metrics on layer
    G = layer.core_network
    if len(G) > 0: # Check if layer has nodes
        stats['density'] = nx.density(G)
        stats['avg_degree'] = sum(dict(G.degree()).values()) / len(G)

    # Check connectivity
    if not layer.directed:
        stats['connected'] = nx.is_connected(G)
        if stats['connected']:
            stats['diameter'] = nx.diameter(G)

    layer_stats[layer_name] = stats

# Display results
print("\n=== Layer-by-Layer Analysis ===")
for layer, stats in layer_stats.items():
    print(f"\nLayer: {layer}")
    for metric, value in stats.items():
        print(f"    {metric}: {value}")

```

When to use: Understanding layer-specific properties, identifying important layers.

Recipe 6: Compute Multilayer Statistics

Task: Calculate multilayer-specific metrics that account for cross-layer structure.

```

from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

layers = network.get_layers()

# === Layer-Level Statistics ===
print("=== Layer Statistics ===")
for layer in layers:
    density = mls.layer_density(network, layer)
    print(f"Layer {layer} density: {density:.4f}")

# Layer diversity

```

(continues on next page)

(continued from previous page)

```

entropy = mls.entropy_of_multiplexity(network)
print(f"\nLayer diversity (entropy): {entropy:.4f} bits")

# === Node-Level Statistics ===
print("\n=== Node Activity ===")
sample_nodes = list(network.get_nodes())[:5]
for node_tuple in sample_nodes:
    node = node_tuple[0] # Extract node name
    activity = mls.node_activity(network, node)
    print(f"Node {node} active in {activity*100:.1f}% of layers")

# === Cross-Layer Analysis ===
if len(layers) >= 2:
    print("\n=== Cross-Layer Similarity ===")
    layer1, layer2 = str(layers[0]), str(layers[1])

    # Cosine similarity of adjacency matrices
    similarity = mls.layer_similarity(network, layer1, layer2, method='cosine')
    print(f"Cosine similarity ({layer1} vs {layer2}): {similarity:.4f}")

    # Jaccard similarity of edge sets
    overlap = mls.edge_overlap(network, layer1, layer2)
    print(f"Edge overlap (Jaccard): {overlap:.4f}")

# === Versatility Analysis ===
print("\n=== Node Versatility (Cross-Layer Importance) ===")
versatility = mls.versatility_centrality(network, centrality_type='degree')
top_nodes = sorted(versatility.items(), key=lambda x: x[1], reverse=True)[:5]
for node, score in top_nodes:
    print(f"    {node}: {score:.4f}")

```

When to use: Quantifying multilayer structure, comparing layers, identifying versatile nodes.

Centrality & Node Importance

Recipe 7: Single-Layer Centrality Analysis

Task: Identify important nodes within individual layers.

```

from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Select a layer
layers = network.get_layers()
target_layer = [str(layers[0])]
layer = network.subnetwork(target_layer, subset_by="layers")

# Compute multiple centrality measures
centrality_measures = {
    'degree': layer.monoplex_nx_wrapper("degree_centrality"),
    'betweenness': layer.monoplex_nx_wrapper("betweenness_centrality"),
    'closeness': layer.monoplex_nx_wrapper("closeness_centrality"),
    'eigenvector': layer.monoplex_nx_wrapper("eigenvector_centrality"),
}

```

(continues on next page)

(continued from previous page)

```

}

# Display top nodes for each measure
print(f"=== Centrality Analysis for Layer {target_layer[0]} ===\n")
for measure_name, scores in centrality_measures.items():
    print(f"{measure_name.capitalize()} Centrality (Top 5):")
    top_nodes = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:5]
    for node, score in top_nodes:
        print(f"    {node}: {score:.4f}")
    print()

```

When to use: Identifying influential nodes, hub detection, network simplification.

Recipe 8: Multiplex Participation Coefficient

Task: Measure how evenly nodes participate across layers in a multiplex network.

```

from py3plex.core import multinet
from py3plex.algorithms.multicentrality import multiplex_participation_coefficient

# Load or create multiplex network (same nodes across all layers)
network = multinet.multi_layer_network().load_network(
    "data/multiplex_network.txt",
    input_type="multiedgelist"
)

# Compute MPC (requires true multiplex: identical node set per layer)
try:
    mpc = multiplex_participation_coefficient(
        network,
        normalized=True,
        check_multiplex=True
    )

    # Display results
    print("=== Multiplex Participation Coefficient ===")
    print("(0 = active in one layer only, 1 = equally active across all layers)\n")

    # Sort by MPC score
    sorted_nodes = sorted(mpc.items(), key=lambda x: x[1], reverse=True)

    print("Top 10 most versatile nodes:")
    for node, score in sorted_nodes[:10]:
        print(f"    {node}: {score:.4f}")

    print("\nTop 10 most specialized nodes:")
    for node, score in sorted_nodes[-10:]:
        print(f"    {node}: {score:.4f}")

except ValueError as e:
    print(f"Error: {e}")
    print("Network is not a true multiplex (layers have different node sets)")

```

When to use: Analyzing multiplex networks, identifying cross-layer hubs, understanding node specialization.

Community Detection

Recipe 9: Multilayer Community Detection

Task: Detect communities that span multiple layers.

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
↳ multilayer
from collections import Counter

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Run multilayer Louvain algorithm
# gamma: resolution parameter (higher = more communities)
# omega: inter-layer coupling strength (higher = communities span layers)
partition = louvain_multilayer(
    network,
    gamma=1.0,      # Standard resolution
    omega=1.0,      # Standard coupling
    random_state=42 # For reproducibility
)

# Analyze results
num_communities = len(set(partition.values()))
print(f"Detected {num_communities} communities")

# Community size distribution
community_sizes = Counter(partition.values())
print("\nTop 10 communities by size:")
for comm_id, size in community_sizes.most_common(10):
    print(f"  Community {comm_id}: {size} nodes")

# Check which nodes are in largest community
largest_comm = community_sizes.most_common(1)[0][0]
largest_comm_nodes = [node for node, comm in partition.items()
                      if comm == largest_comm]
print(f"\nNodes in largest community: {largest_comm_nodes[:10]}...")
```

When to use: Understanding network organization, identifying functional modules, clustering nodes.

Recipe 10: Compare Community Detection Algorithms

Task: Run multiple community detection methods and compare results.

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection import community_wrapper as cw
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
↳ multilayer
from collections import Counter
import numpy as np

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
```

(continues on next page)

(continued from previous page)

```

)

# Method 1: Single-layer Louvain on aggregated network
partition_louvain = cw.louvain_communities(network)

# Method 2: Multilayer Louvain
partition_multilayer = louvain_multilayer(
    network,
    gamma=1.0,
    omega=1.0,
    random_state=42
)

# Compare results
print("=== Community Detection Comparison ===\n")
print(f"Single-layer Louvain: {len(set(partition_louvain.values()))} communities")
print(f"Multilayer Louvain: {len(set(partition_multilayer.values()))} communities")

# Calculate normalized mutual information (if sklearn available)
try:
    from sklearn.metrics import normalized_mutual_info_score

    # Align node sets
    common_nodes = set(partition_louvain.keys()) & set(partition_multilayer.keys())
    labels1 = [partition_louvain[n] for n in common_nodes]
    labels2 = [partition_multilayer[n] for n in common_nodes]

    nmi = normalized_mutual_info_score(labels1, labels2)
    print(f"\nNormalized Mutual Information: {nmi:.4f}")
    print("(1.0 = identical partitions, 0.0 = independent)")
except ImportError:
    print("\nInstall scikit-learn to compute NMI: pip install scikit-learn")

```

When to use: Method validation, algorithm selection, robustness analysis.

Visualization

Recipe 11: Basic Network Visualization

Task: Create publication-ready network visualizations.

```

from py3plex.core import multinet
from py3plex.visualization.multilayer import hairball_plot
import matplotlib
matplotlib.use('Agg') # For non-interactive environments
import matplotlib.pyplot as plt

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Get network in visualization format
network_colors, graph = network.get_layers(style="hairball")

# Create visualization
plt.figure(figsize=(12, 12))

```

(continues on next page)

(continued from previous page)

```

hairball_plot(
    graph,
    network_colors,
    layout_algorithm="force",          # Options: "force", "circular", "spring"
    layout_parameters={"iterations": 100},
    node_size=5,
    edge_width=0.5,
    alpha_channel=0.8,
    scale_by_size=True
)
plt.title("Multilayer Network", fontsize=16, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.savefig("network_viz.png", dpi=150, bbox_inches='tight',
           facecolor='white', edgecolor='none')
plt.close()
print("Visualization saved to network_viz.png")

```

When to use: Presentations, papers, reports, exploratory analysis.

Recipe 12: Visualize Communities on Network

Task: Color nodes by community membership in network plot.

```

from py3plex.core import multinet
from py3plex.visualization.multilayer import hairball_plot
from py3plex.visualization.colors import colors_default
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
    multilayer
from collections import Counter
import matplotlib.pyplot as plt
import matplotlib
matplotlib.use('Agg')

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Detect communities
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)

# Select top N communities to color
top_n = 10
community_counts = Counter(partition.values())
top_communities = [c for c, _ in community_counts.most_common(top_n)]

# Create color mapping
color_map = dict(zip(top_communities, colors_default[:top_n]))

# Assign colors to nodes
network_colors = [
    color_map.get(partition.get(node), "lightgray")
    for node in network.get_nodes()
]

```

(continues on next page)

(continued from previous page)

```

# Get network for visualization
_, graph = network.get_layers(style="hairball")

# Create plot
plt.figure(figsize=(14, 14))
hairball_plot(
    graph,
    network_colors,
    layout_algorithm="force",
    layout_parameters={"iterations": 150},
    node_size=8,
    edge_width=0.5,
    alpha_channel=0.7,
    scale_by_size=True
)
plt.title(f"Network with {top_n} Largest Communities",
          fontsize=16, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.savefig("network_communities.png", dpi=150,
            bbox_inches='tight', facecolor='white')
plt.close()
print("Community visualization saved to network_communities.png")

```

When to use: Visualizing analysis results, understanding community structure.

Advanced Workflows

Recipe 13: Complete Analysis Pipeline

Task: Full workflow from data loading to results export.

```

from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
    ↪ multilayer
from collections import Counter
import json
import numpy as np

# Set random seed
np.random.seed(42)

# === 1. Load Data ===
print("=== Loading Network ===")
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist",
    directed=False
)
network.basic_stats()

# === 2. Compute Statistics ===
print("\n=== Computing Multilayer Statistics ===")
layers = network.get_layers()

stats = {

```

(continues on next page)

(continued from previous page)

```

    'num_layers': len(layers),
    'layer_densities': {},
    'layer_diversity': mls.entropy_of_multiplexity(network),
}

for layer in layers:
    stats['layer_densities'][str(layer)] = mls.layer_density(network, layer)

# Versatility
versatility = mls.versatility_centrality(network, centrality_type='degree')
top_versatile = sorted(versatility.items(), key=lambda x: x[1], reverse=True)[:10]
stats['top_versatile_nodes'] = {str(k): float(v) for k, v in top_versatile}

# === 3. Community Detection ===
print("\n=== Detecting Communities ===")
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
community_sizes = Counter(partition.values())

stats['num_communities'] = len(set(partition.values()))
stats['community_sizes'] = dict(community_sizes.most_common(10))

# === 4. Export Results ===
print("\n=== Exporting Results ===")

# Save statistics as JSON
with open("analysis_results.json", "w") as f:
    json.dump(stats, f, indent=2)
print("Statistics saved to analysis_results.json")

# Save community assignments
with open("communities.txt", "w") as f:
    for node, comm in partition.items():
        f.write(f"{node}\t{comm}\n")
print("Communities saved to communities.txt")

# Save processed network
network.save_network("processed_network.graphml", output_type="graphml")
print("Network saved to processed_network.graphml")

print("\n=== Analysis Complete ===")

```

When to use: Reproducible research workflows, batch processing, automated analysis.

Recipe 14: Network Comparison Study

Task: Compare multiple networks systematically.

```

from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
import pandas as pd
import numpy as np

# List of networks to compare
network_files = [
    ("Network A", "data/network_a.txt"),
    ("Network B", "data/network_b.txt"),

```

(continues on next page)

(continued from previous page)

```

    ("Network C", "data/network_c.txt"),
]

results = []

for name, filepath in network_files:
    print(f"\nAnalyzing {name}...")

    # Load network
    network = multinet.multi_layer_network().load_network(
        filepath,
        input_type="multiedgelist",
        directed=False
    )

    # Compute metrics
    layers = network.get_layers()
    nodes = list(network.get_nodes())
    edges = list(network.get_edges())

    metrics = {
        'Network': name,
        'Layers': len(layers),
        'Total Nodes': len(nodes),
        'Unique Nodes': len(set([n[0] for n in nodes])),
        'Total Edges': len(edges),
        'Layer Diversity': mls.entropy_of_multiplexity(network),
    }

    # Average layer density
    densities = [mls.layer_density(network, layer) for layer in layers]
    metrics['Avg Layer Density'] = np.mean(densities)
    metrics['Std Layer Density'] = np.std(densities)

    results.append(metrics)

# Create comparison table
df = pd.DataFrame(results)
print("\n=== Network Comparison ===")
print(df.to_string(index=False))

# Save to CSV
df.to_csv("network_comparison.csv", index=False)
print("\nComparison saved to network_comparison.csv")

```

When to use: Comparative studies, evaluating datasets, selecting networks for analysis.

Recipe 15: Random Walks for Node Embeddings

Task: Generate node embeddings using random walks (Node2Vec approach).

```

from py3plex.core import multinet
from py3plex.algorithms.general.walkers import generate_walks, node2vec_walk
import numpy as np

# Set random seed

```

(continues on next page)

(continued from previous page)

```

np.random.seed(42)

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Get core NetworkX graph
G = network.core_network

# Generate random walks
# p: return parameter (higher = less likely to return to previous node)
# q: in-out parameter (higher = more BFS-like, lower = more DFS-like)
walks = generate_walks(
    G,
    num_walks=10,      # Number of walks per node
    walk_length=80,    # Length of each walk
    p=1.0,             # Return parameter
    q=1.0,             # In-out parameter
    seed=42
)

print(f"Generated {len(walks)} random walks")
print(f"First walk (truncated): {walks[0][:10]}...")

# Use walks for downstream tasks:
# 1. Train Word2Vec model (if gensim available)
try:
    from gensim.models import Word2Vec

    # Convert walks to strings for Word2Vec
    walks_str = [[str(node) for node in walk] for walk in walks]

    # Train embedding model
    model = Word2Vec(
        walks_str,
        vector_size=128,    # Embedding dimension
        window=5,          # Context window
        min_count=1,
        sg=1,              # Skip-gram
        workers=4,
        epochs=5,
        seed=42
    )

    print(f"\nTrained embedding model with {len(model.wv)} nodes")

    # Get embedding for a node
    sample_node = str(list(G.nodes())[0])
    if sample_node in model.wv:
        embedding = model.wv[sample_node]
        print(f"Embedding for node {sample_node}: {embedding[:5]}... (dim=
→ {len(embedding)})")

    # Find similar nodes
    similar = model.wv.most_similar(sample_node, topn=5)

```

(continues on next page)

(continued from previous page)

```

        print(f"\nMost similar nodes to {sample_node}:")
        for node, similarity in similar:
            print(f"    {node}: {similarity:.4f}")

    # Save embeddings
    model.wv.save_word2vec_format("node_embeddings.txt")
    print("\nEmbeddings saved to node_embeddings.txt")

except ImportError:
    print("\nInstall gensim for embedding generation: pip install gensim")

```

When to use: Node classification, link prediction, network clustering, feature extraction.

Performance & Optimization

Recipe 16: Handle Large Networks Efficiently

Task: Process large multilayer networks without running out of memory.

```

from py3plex.core import multinet
import gc

# === 1. Load network efficiently ===
# Use generators instead of loading all at once
network = multinet.multi_layer_network()

# For very large files, load in chunks or stream
# (This is a conceptual example - actual implementation may vary)

# === 2. Process layers independently ===
layers = network.get_layers()

layer_results = {}
for layer_name in layers:
    print(f"Processing layer {layer_name}...")

    # Extract and process one layer at a time
    layer = network.subnetwork([layer_name], subset_by="layers")

    # Compute statistics for this layer
    nodes = len(list(layer.get_nodes()))
    edges = len(list(layer.get_edges()))

    layer_results[str(layer_name)] = {'nodes': nodes, 'edges': edges}

    # Clean up to free memory
    del layer
    gc.collect()

print("\n=== Results ===")
for layer, stats in layer_results.items():
    print(f"{layer}: {stats['nodes']} nodes, {stats['edges']} edges")

# === 3. Use sparse representations ===
# When computing supra-adjacency matrix
supra_adj = network.get_supra_adjacency_matrix(sparse=True)
print(f"\nSupra-adjacency matrix: {supra_adj.shape} (sparse)")

```

(continues on next page)

(continued from previous page)

```

# === 4. Sample for exploratory analysis ===
# For visualization or initial exploration, work with a sample
all_nodes = list(network.get_nodes())
import random
random.seed(42)
sample_size = min(1000, len(all_nodes))
sampled_nodes = random.sample(all_nodes, sample_size)

# Create subnetwork with sampled nodes
# (Implementation depends on network structure)
print(f"\nSampled {sample_size} nodes for exploratory analysis")

```

When to use: Networks with >10,000 nodes, limited RAM, large-scale studies.

Tips: - Process layers sequentially rather than all at once - Use sparse matrices for large supra-adjacency representations - Sample networks for visualization and initial exploration - Save intermediate results to disk - Use generators and iterators instead of lists

Recipe 17: Benchmark and Profile Your Analysis

Task: Measure performance and identify bottlenecks.

```

from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
    ↪ multilayer
import time
import psutil
import os

def get_memory_usage():
    """Get current memory usage in MB"""
    process = psutil.Process(os.getpid())
    return process.memory_info().rss / 1024 / 1024

# Load network
print("=== Performance Benchmarking ===\n")
start_mem = get_memory_usage()

start = time.time()
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)
load_time = time.time() - start
load_mem = get_memory_usage() - start_mem

print(f"Network loading:")
print(f"  Time: {load_time:.3f} seconds")
print(f"  Memory: {load_mem:.2f} MB")

# Benchmark statistics computation
start = time.time()
network.basic_stats()
stats_time = time.time() - start
print(f"\nBasic statistics:")
print(f"  Time: {stats_time:.3f} seconds")

```

(continues on next page)

(continued from previous page)

```
# Benchmark community detection
start_mem = get_memory_usage()
start = time.time()
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
comm_time = time.time() - start
comm_mem = get_memory_usage() - start_mem

print(f"\nCommunity detection:")
print(f"  Time: {comm_time:.3f} seconds")
print(f"  Memory: {comm_mem:.2f} MB")
print(f"  Communities found: {len(set(partition.values()))}")

# Total resource usage
print(f"\n=== Total Performance ===")
print(f"Total time: {load_time + stats_time + comm_time:.3f} seconds")
print(f"Peak memory: {get_memory_usage():.2f} MB")
```

When to use: Optimizing workflows, comparing algorithms, capacity planning.

Note: Requires psutil package: `pip install psutil`

Tips & Best Practices

General Tips

1. Always set random seeds for reproducibility:

```
import numpy as np
np.random.seed(42)
```

2. Verify network structure after loading:

```
network.basic_stats() # Quick sanity check
```

3. Handle errors gracefully:

```
try:
    result = network.some_operation()
except Exception as e:
    print(f"Operation failed: {e}")
```

4. Save intermediate results for long analyses:

```
import pickle
with open("checkpoint.pkl", "wb") as f:
    pickle.dump(results, f)
```

5. Use appropriate file formats:

- GraphML: Standard, compatible with other tools
- Pickle: Fast, preserves Python objects, not portable
- Edgelist: Simple, human-readable, large file size

Performance Tips

1. **For large networks**, process layers independently
2. **Use sparse matrices** when working with supra-adjacency
3. **Sample networks** for visualization (>1000 nodes gets crowded)
4. **Profile your code** to find bottlenecks
5. **Close plots** after saving to free memory: `plt.close()`

Visualization Tips

1. **Choose layout carefully:**
 - Force-directed: General purpose, slow for >5000 nodes
 - Circular: Fast, good for small networks
 - Spring: Balance between quality and speed
2. **Adjust node size** based on network size:

```
node_size = max(1, 100 / np.sqrt(num_nodes))
```

3. **Use alpha channel** for edge-dense networks:

```
alpha_channel=0.3 # More transparent
```

4. **Save high-resolution** for publications:

```
plt.savefig("figure.png", dpi=300)
```

Community Detection Tips

1. **Tune resolution parameter** (γ):
 - Higher γ → more, smaller communities
 - Lower γ → fewer, larger communities
2. **Tune coupling parameter** (ω) for multilayer:
 - Higher ω → communities span layers
 - Lower ω → layer-specific communities
3. **Run multiple times** with different random seeds to check stability
4. **Compare multiple algorithms** to validate findings

Common Issues & Solutions

Issue: “Network appears to be empty”

Cause: Incorrect file format or delimiter.

Solution::

```
# Check file format
with open("data.txt", "r") as f:
    print(f.readlines()[:5]) # Inspect first lines

# Try different input_type
network = multinet.multi_layer_network().load_network(
```

(continues on next page)

(continued from previous page)

```
"data.txt",
input_type="multiedgelist" # or "edgelist", "graphml"
)
```

Issue: “No module named ‘community’”**Cause:** Optional dependency not installed.**Solution::**

```
pip install python-louvain
```

Issue: “Visualization doesn’t appear”**Cause:** Using non-interactive backend or missing display.**Solution::**

```
import matplotlib
matplotlib.use('Agg') # For saving files
import matplotlib.pyplot as plt

# ... create plot ...
plt.savefig("output.png") # Save instead of show
```

Issue: “Memory error with large network”**Cause:** Loading entire network into RAM at once.**Solution:** See Recipe 16 for memory-efficient processing.**Issue: “Community detection is slow”****Cause:** Large network or complex structure.**Solution::**

```
# Use simpler algorithm for quick results
from py3plex.algorithms.community_detection import community_wrapper as cw
partition = cw.louvain_communities(network) # Faster than multilayer
```

Further Reading**Documentation:**

- Installation guide - See main documentation
- Quick start tutorial - See main documentation
- Core concepts - See main documentation
- Algorithm selection guide - See main documentation

Examples:See `examples/` directory for 50+ complete, working examples covering all major functionality.**Research:**

- Kivelä et al. (2014) - Multilayer networks theory
- De Domenico et al. (2013) - Mathematical framework

- See Citations section in main documentation for complete reference list

8.3.11 Use Cases & Case Studies

“In theory there is no difference between theory and practice. In practice there is.” — Yogi Berra

This chapter provides complete, end-to-end case studies demonstrating py3plex in different research domains. Each case study includes:

1. **Problem context** — What real-world question are we answering?
2. **Data modeling decisions** — How do we represent this problem as a multilayer network?
3. **Complete code** — Working examples you can adapt
4. **Interpretation** — What do the results mean?
5. **Adaptation guide** — How to apply this template to your own data

Why Case Studies Matter

Reading about algorithms is one thing. Applying them to real problems is another entirely.

The case studies in this chapter come from real research domains where multilayer network analysis has proven valuable. They’re not toy examples—they represent the kinds of questions that researchers and practitioners actually ask:

- How do we identify proteins that are likely to be functionally important?
- Which social media influencers are genuinely cross-platform celebrities versus one-hit wonders?
- What happens to urban mobility when a major transit hub fails?
- Which academic researchers are likely to collaborate based on their publication patterns?

Each case study walks through the complete analysis workflow, from problem formulation through data modeling, analysis, and interpretation. Pay attention not just to the code, but to the reasoning behind each decision.

Lessons from the Field

Before diving into specific cases, here are some lessons learned from applying multilayer network analysis to real problems:

Modeling decisions matter more than algorithm choice. Whether you use Louvain or Infomap for community detection is less important than whether you’ve correctly identified what should be a layer, what should be a node, and how layers should be coupled. Spend your time on modeling.

Start simple, add complexity as needed. It’s tempting to immediately build a network with ten layers, weighted edges, temporal dynamics, and inter-layer dependencies. Resist this urge. Start with the simplest model that might answer your question, and add complexity only when you can demonstrate it improves your analysis.

Validate with domain knowledge. Network analysis can reveal surprising patterns—but it can also reveal artifacts of your modeling choices. Always sanity-check results against what domain experts know. If your community detection puts obviously unrelated entities in the same cluster, you have a modeling problem.

Document your decisions. When you return to an analysis six months later, you won’t remember why you chose certain parameters. Write down your reasoning. Future-you will thank present-you.

Case Study 1: Biological Network Analysis

Identifying high-confidence protein interactions and functional modules

Real-World Impact

Protein-protein interaction (PPI) networks are foundational to understanding cellular function. The proteins in your body don't work alone—they form complexes, signaling cascades, and metabolic pathways. Knowing which proteins interact helps researchers:

- **Understand disease mechanisms:** Many diseases result from disrupted protein interactions
- **Identify drug targets:** Proteins central to disease networks are potential therapeutic targets
- **Predict protein function:** A protein's interactors suggest its biological role

The challenge? PPI data comes from multiple experimental methods, each with different biases and error rates. High-throughput screens find many interactions quickly but include false positives. Literature curation is accurate but incomplete. How do you combine these sources intelligently?

This is exactly what multilayer network analysis is for.

Multilayer Protein-Protein Interaction Network

Problem Context:

You're a computational biologist studying protein interactions in yeast. You have PPI data from three experimental sources with different reliability levels:

- **Yeast two-hybrid (Y2H):** High-throughput but many false positives
- **Affinity purification mass spectrometry (AP-MS):** More reliable but biased toward stable complexes
- **Literature curated:** Highly reliable but incomplete

You want to:

1. Find functional modules (groups of proteins that work together)
2. Identify proteins that are central across evidence types (likely real interactions)
3. Discover proteins that bridge different functional modules

Data Modeling Decisions:

- **Node type:** Proteins (same set across layers for multiplex analysis)
- **Layers:** One per evidence type (Y2H, AP-MS, Literature)
- **Edges:** Physical protein-protein interactions
- **Network type:** Multiplex (same proteins, different interaction evidence)

Complete Code:

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
    ↳ multilayer
from collections import Counter
import numpy as np

# === 1. Create the network ===
# In practice, you'd load from files; here we create a toy example
network = multinet.multi_layer_network(network_type='multiplex')

# Simulated PPI data (protein pairs by evidence type)
# Replace with your actual data loading
ppi_data = {
    'Y2H': [
        ('CDC28', 'CLB2'), ('CDC28', 'CLB5'), ('CLB2', 'SIC1'),
```

(continues on next page)

(continued from previous page)

```

        ('ACT1', 'MYO2'), ('ACT1', 'TPM1'), ('MYO2', 'TPM1'),
        ('CDC28', 'CDC6'), ('CDC6', 'ORC1'), # cell cycle proteins
    ],
    'APMS': [
        ('CDC28', 'CLB2'), ('CDC28', 'CKS1'), ('CLB2', 'CKS1'),
        ('ACT1', 'ABP1'), ('ACT1', 'TPM1'),
        ('CDC6', 'ORC1'), ('ORC1', 'ORC2'), ('ORC2', 'ORC3'), # origin recognition
    ],
    'complex': [
    ],
    'Literature': [
        ('CDC28', 'CLB2'), ('CDC28', 'CLB5'), ('CDC28', 'CKS1'),
        ('ACT1', 'MYO2'), ('MYO2', 'TPM1'),
        ('CDC6', 'ORC1'), # well-established interaction
    ],
}

# Add edges to network
for layer, interactions in ppi_data.items():
    for protein1, protein2 in interactions:
        network.add_edges([
            [protein1, layer, protein2, layer, 1.0]
        ], input_type="list")

# === 2. Basic exploration ===
print("=== Network Overview ===")
network.basic_stats()

# === 3. Identify high-confidence interactions ===
# Interactions that appear in multiple evidence types are more reliable
print("\n=== Evidence Overlap Analysis ===")

# Get edges from each layer
edges_by_layer = {}
for layer in network.get_layers():
    layer_subnet = network.subnetwork([layer], subset_by="layers")
    # Extract edge pairs (ignoring layer info)
    edges = set()
    for edge in layer_subnet.get_edges():
        # Normalize edge representation (sorted tuple)
        pair = tuple(sorted([edge[0][0], edge[1][0]]))
        edges.add(pair)
    edges_by_layer[layer] = edges

# Find edges in multiple layers
all_edges = set()
for edges in edges_by_layer.values():
    all_edges.update(edges)

edge_evidence_counts = {}
for edge in all_edges:
    count = sum(1 for layer_edges in edges_by_layer.values() if edge in layer_edges)
    edge_evidence_counts[edge] = count

print("High-confidence interactions (in 3/3 evidence types):")
for edge, count in sorted(edge_evidence_counts.items(), key=lambda x: x[1],
    reverse=True):

```

(continues on next page)

(continued from previous page)

```

    if count == 3:
        print(f" {edge[0]} -- {edge[1]}: {count} evidence types")

print("\nMedium-confidence interactions (in 2/3 evidence types):")
for edge, count in sorted(edge_evidence_counts.items(), key=lambda x: x[1],
↪reverse=True):
    if count == 2:
        print(f" {edge[0]} -- {edge[1]}: {count} evidence types")

# === 4. Node activity analysis ===
print("\n=== Protein Activity Across Evidence Types ===")
unique_proteins = set()
for node in network.get_nodes():
    unique_proteins.add(node[0]) # Extract protein name

protein_activity = {}
for protein in unique_proteins:
    activity = mls.node_activity(network, protein)
    protein_activity[protein] = activity

print("Proteins present in all evidence types (activity = 1.0):")
for protein, activity in sorted(protein_activity.items(), key=lambda x: x[1],
↪reverse=True):
    if activity == 1.0:
        print(f" {protein}")

# === 5. Community detection (functional modules) ===
print("\n=== Functional Module Detection ===")

# Lower coupling (omega=0.5) because different evidence types
# may reveal different aspects of the interactome
partition = louvain_multilayer(
    network,
    gamma=1.0,      # Standard resolution
    omega=0.5,      # Moderate coupling
    random_state=42
)

# Analyze communities
num_communities = len(set(partition.values()))
print(f"Detected {num_communities} functional modules")

# Group proteins by community
communities = {}
for node, comm in partition.items():
    protein = node[0] # Extract protein name
    if comm not in communities:
        communities[comm] = set()
    communities[comm].add(protein)

print("\nFunctional modules found:")
for comm_id, proteins in sorted(communities.items(), key=lambda x: len(x[1]),
↪reverse=True):
    print(f" Module {comm_id}: {proteins}")

# === 6. Versatility analysis (cross-module bridging) ===

```

(continues on next page)

(continued from previous page)

```
print("\n=== Bridge Proteins (High Versatility) ===")
versatility = mls.versatility centrality(network, centrality_type='degree')
top_versatile = sorted(versatility.items(), key=lambda x: x[1], reverse=True)[:5]
for node, score in top_versatile:
    print(f" {node}: versatility = {score:.4f}")
```

Interpretation:

The analysis reveals several insights:

1. **High-confidence interactions** (CDC28-CLB2, ACT1-TPM1) appear in all three evidence types, making them likely true interactions suitable for downstream analysis.
2. **Functional modules** correspond to known protein complexes:
 - Cell cycle regulators (CDC28, CLB2, CLB5, CKS1, SIC1)
 - Actin cytoskeleton (ACT1, MYO2, TPM1, ABP1)
 - DNA replication origin complex (CDC6, ORC1, ORC2, ORC3)
3. **Bridge proteins** with high versatility may connect different functional modules and could be interesting drug targets or key regulators.

Adapting to Your Data:

1. Replace the `ppi_data` dictionary with your actual data loading code
2. Adjust `omega` based on your domain: lower for diverse evidence types, higher if you expect consistency
3. Add Gene Ontology enrichment analysis to validate that communities match known functional annotations
4. Consider edge weights based on evidence quality (e.g., literature = 3.0, APMS = 2.0, Y2H = 1.0)

Case Study 2: Multi-Platform Social Network Analysis**Cross-Platform Influence Analysis****Problem Context:**

You're a social media researcher studying how influencers operate across platforms. You have data from three platforms:

- **Twitter/X:** Public follower/following relationships
- **YouTube:** Subscription relationships
- **Instagram:** Follow relationships

You want to:

1. Identify influencers who are successful across platforms (vs. platform-specific)
2. Understand how different platforms are related (do the same people follow each other?)
3. Find communities that span platforms

Data Modeling Decisions:

- **Node type:** Users (mapped across platforms using verified identities)
- **Layers:** One per platform (Twitter, YouTube, Instagram)
- **Edges:** Follow/subscribe relationships (directed in reality, undirected for this analysis)
- **Network type:** Multiplex (same users, different platforms)

Complete Code:


```

from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
↳ multilayer
from collections import Counter
import networkx as nx

# === 1. Create the network ===
network = multinet.multi_layer_network(network_type='multiplex')

# Simulated social network data
# In practice: load from APIs or datasets with user ID mapping
social_data = {
    'Twitter': [
        ('influencer_A', 'user_1'), ('influencer_A', 'user_2'),
        ('influencer_A', 'user_3'), ('influencer_B', 'user_2'),
        ('influencer_B', 'user_4'), ('user_1', 'user_2'),
        ('user_3', 'user_5'), ('influencer_C', 'user_6'),
    ],
    'YouTube': [
        ('influencer_A', 'user_1'), ('influencer_A', 'user_3'),
        ('influencer_B', 'user_2'), ('influencer_B', 'user_3'),
        ('influencer_B', 'user_4'), ('influencer_C', 'user_6'),
        ('influencer_C', 'user_7'),
    ],
    'Instagram': [
        ('influencer_A', 'user_1'), ('influencer_A', 'user_2'),
        ('influencer_A', 'user_4'), ('influencer_B', 'user_5'),
        ('user_1', 'user_2'), ('user_2', 'user_3'),
        ('influencer_C', 'user_6'), ('influencer_C', 'user_8'),
    ],
}

for layer, connections in social_data.items():
    for user1, user2 in connections:
        network.add_edges([
            [user1, layer, user2, 1.0]
        ], input_type="list")

# === 2. Basic statistics ===
print("=== Network Overview ===")
network.basic_stats()

# === 3. Cross-platform presence ===
print("\n=== Cross-Platform Presence ===")
unique_users = set()
for node in network.get_nodes():
    unique_users.add(node[0])

for user in sorted(unique_users):
    activity = mls.node_activity(network, user)
    platforms = activity * len(network.get_layers())
    if activity == 1.0:
        print(f" {user}: present on ALL platforms")
    elif activity > 0.5:
        print(f" {user}: present on {platforms:.0f}/3 platforms")

```

(continues on next page)

(continued from previous page)

```

# === 4. Platform-specific centrality ===
print("\n=== Centrality by Platform ===")

for platform in network.get_layers():
    layer_subnet = network.subnetwork([platform], subset_by="layers")
    G = layer_subnet.core_network

    # Compute degree centrality
    degree_cent = nx.degree_centrality(G)
    top_users = sorted(degree_cent.items(), key=lambda x: x[1], reverse=True)[:3]

    print(f"\n{platform} - Top 3 by degree:")
    for node, score in top_users:
        print(f"    {node[0]}: {score:.3f}")

# === 5. Cross-platform influence (versatility) ===
print("\n=== Cross-Platform Influencers (High Versatility) ===")
versatility = mls.versatility_centrality(network, centrality_type='degree')
top_versatile = sorted(versatility.items(), key=lambda x: x[1], reverse=True)[:5]

for node, score in top_versatile:
    print(f"    {node}: {score:.4f}")

# === 6. Layer similarity ===
print("\n=== Platform Similarity ===")
layers = list(network.get_layers())
for i in range(len(layers)):
    for j in range(i+1, len(layers)):
        layer1, layer2 = str(layers[i]), str(layers[j])

        # Edge overlap
        overlap = mls.edge_overlap(network, layer1, layer2)

        # Jaccard similarity
        similarity = mls.layer_similarity(network, layer1, layer2, method='jaccard')

        print(f"    {layer1} vs {layer2}:")
        print(f"        Edge overlap: {overlap:.3f}")
        print(f"        Jaccard similarity: {similarity:.3f}")

# === 7. Cross-platform community detection ===
print("\n=== Cross-Platform Communities ===")

# High coupling because we expect influencers to attract similar audiences
partition = louvain_multilayer(
    network,
    gamma=1.0,
    omega=1.5,      # High coupling for cross-platform consistency
    random_state=42
)

num_communities = len(set(partition.values()))
print(f"Detected {num_communities} cross-platform communities")

# Analyze which users are in which community
communities = {}

```

(continues on next page)

(continued from previous page)

```

for node, comm in partition.items():
    user = node[0]
    if comm not in communities:
        communities[comm] = set()
    communities[comm].add(user)

print("\nCommunities (by unique users):")
for comm_id, users in sorted(communities.items(), key=lambda x: len(x[1]),
    ↪reverse=True):
    influencers = [u for u in users if 'influencer' in u]
    regular = [u for u in users if 'influencer' not in u]
    print(f" Community {comm_id}: {len(influencers)} influencers, {len(regular)} ↪
    ↪regular users")
    print(f"     Influencers: {influencers}")
    print(f"     Sample users: {regular[:3]}")

```

Interpretation:

1. **Cross-platform influencers** (influencer_A, influencer_B) have high versatility, indicating they successfully maintain audiences across platforms. Platform-specific influencers (influencer_C) may have niche appeal.
2. **Platform similarity** shows how audiences overlap:
 - High Twitter-Instagram similarity suggests similar user bases
 - Lower YouTube overlap might indicate different content consumption patterns
3. **Communities** reveal audience segments:
 - Some communities cluster around specific influencers
 - Cross-platform communities suggest genuine fandom that follows across platforms

Adapting to Your Data:

1. Map user identities across platforms (use verified accounts, linked profiles, or username matching)
2. Consider using directed edges for asymmetric follow relationships
3. Add edge weights based on interaction strength (likes, comments, shares)
4. Include temporal layers to track audience evolution over time

Case Study 3: Multi-Modal Transportation Analysis**Urban Mobility and Resilience****Problem Context:**

You're an urban transportation analyst studying a city's public transit system. You have network data for:

- **Metro/Subway:** High-capacity, fixed routes
- **Bus:** Lower capacity, more flexible routes
- **Bike-share:** Individual, first/last mile connections

You want to:

1. Identify multimodal hubs (stations serving multiple transport modes)
2. Analyze network resilience (what happens if a metro station closes?)
3. Find communities ("travel basins") that span transport modes

Data Modeling Decisions:

- **Node type:** Stations/stops (with geographic coordinates)

- **Layers:** One per transport mode (Metro, Bus, Bikeshare)
- **Edges:** Direct connections between stops (can walk/transfer)
- **Network type:** Multiplex with geographic coupling (same location = same node)

Complete Code:

```

from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
↳ multilayer
import networkx as nx
from collections import Counter

# === 1. Create the transportation network ===
network = multinet.multi_layer_network(network_type='multiplex')

# Simulated city transport data
# Stations are named by neighborhood/landmark
transport_data = {
    'Metro': [
        ('Central', 'Financial'), ('Financial', 'Tech_Park'),
        ('Central', 'University'), ('University', 'Hospital'),
        ('Central', 'Shopping_Mall'), ('Shopping_Mall', 'Residential_North'),
    ],
    'Bus': [
        ('Central', 'Financial'), ('Financial', 'Residential_East'),
        ('Central', 'University'), ('University', 'Residential_South'),
        ('Central', 'Shopping_Mall'), ('Shopping_Mall', 'Residential_North'),
        ('Tech_Park', 'Residential_East'), # Bus connects where metro doesn't
        ('Hospital', 'Residential_South'),
        ('Airport', 'Central'), # Bus to airport
    ],
    'Bikeshare': [
        ('Central', 'Financial'), ('Financial', 'Tech_Park'),
        ('Central', 'University'), ('University', 'Residential_South'),
        ('Shopping_Mall', 'Residential_North'),
        ('Tech_Park', 'Coffee_District'), # Bike-only area
        ('University', 'Student_Housing'), # Bike-only connection
    ],
}

for mode, connections in transport_data.items():
    for stop1, stop2 in connections:
        network.add_edges([
            [stop1, mode, stop2, mode, 1.0]
        ], input_type="list")

# === 2. Network overview ===
print("=== Transportation Network Overview ===")
network.basic_stats()

# === 3. Multimodal hub identification ===
print("\n=== Multimodal Hubs ===")
unique_stations = set()
for node in network.get_nodes():
    unique_stations.add(node[0])

```

(continues on next page)

(continued from previous page)

```

station_modes = {}
for station in unique_stations:
    activity = mls.node_activity(network, station)
    num_modes = activity * len(network.get_layers())
    station_modes[station] = num_modes

print("Stations by number of transport modes:")
for station, modes in sorted(station_modes.items(), key=lambda x: x[1], reverse=True):
    mode_names = []
    for mode in network.get_layers():
        layer_subnet = network.subnetwork([mode], subset_by="layers")
        layer_nodes = [n[0] for n in layer_subnet.get_nodes()]
        if station in layer_nodes:
            mode_names.append(mode)

    if modes >= 2:
        print(f"    {station}: {modes:.0f} modes ({', '.join(mode_names)})")

# === 4. Mode-specific analysis ===
print("\n=== Transport Mode Characteristics ===")

for mode in network.get_layers():
    layer_subnet = network.subnetwork([mode], subset_by="layers")
    G = layer_subnet.core_network

    num_stops = G.number_of_nodes()
    num_connections = G.number_of_edges()
    density = mls.layer_density(network, mode)

    # Find most connected stops
    degree_cent = nx.degree_centrality(G)
    top_stop = max(degree_cent.items(), key=lambda x: x[1])

    print(f"\n{mode}:")
    print(f"    Stops: {num_stops}")
    print(f"    Connections: {num_connections}")
    print(f"    Network density: {density:.3f}")
    print(f"    Most connected: {top_stop[0][0]} (degree={top_stop[1]:.2f})")

# === 5. Resilience analysis: What if Central closes? ===
print("\n=== Resilience Analysis: Central Station Failure ===")

# Current connectivity
G_full = network.core_network
num_components_before = nx.number_connected_components(G_full.to_undirected())
print(f"Before failure: {num_components_before} connected component(s)")

# Simulate failure by removing Central from all layers
G_after = G_full.copy()
central_nodes = [n for n in G_after.nodes() if n[0] == 'Central']
G_after.remove_nodes_from(central_nodes)

num_components_after = nx.number_connected_components(G_after.to_undirected())
print(f"After removing Central: {num_components_after} connected component(s)")

# Which stations become disconnected?

```

(continues on next page)

(continued from previous page)

```

if num_components_after > 1:
    components = list(nx.connected_components(G_after.to_undirected()))
    print("\nResulting network fragments:")
    for i, comp in enumerate(sorted(components, key=len, reverse=True)):
        stations = set(n[0] for n in comp)
        print(f"  Fragment {i+1} ({len(stations)} stations): {stations}")

# === 6. Travel basin detection ===
print("\n=== Travel Basins (Communities) ===")

# Moderate coupling: travel patterns may differ by mode
partition = louvain_multilayer(
    network,
    gamma=1.0,
    omega=1.0,
    random_state=42
)

num_basins = len(set(partition.values()))
print(f"Detected {num_basins} travel basins")

# Group stations by basin
basins = {}
for node, basin in partition.items():
    station = node[0]
    if basin not in basins:
        basins[basin] = set()
    basins[basin].add(station)

print("\nTravel basins:")
for basin_id, stations in sorted(basins.items(), key=lambda x: len(x[1]),
    ↪reverse=True):
    print(f"  Basin {basin_id}: {stations}")

```

Interpretation:

1. **Multimodal hubs** (Central, Financial, University) are critical infrastructure points where passengers transfer between modes. These should be prioritized for maintenance and security.
2. **Resilience analysis** reveals that Central is a critical node—its failure fragments the network. This suggests the need for redundant connections or alternative routes.
3. **Travel basins** correspond to geographic areas where people typically travel:
 - Downtown basin: Central, Financial, Shopping_Mall
 - University/Hospital basin: University, Hospital, Student_Housing
 - Residential basins: various residential areas with their nearest transit

Adapting to Your Data:

1. Include geographic distance as edge weight (travel time between stops)
2. Add passenger flow data to weight edges by usage
3. Include temporal layers (peak hours, off-peak, weekend)
4. Add inter-layer edges representing walking transfers between nearby stations

Case Study 4: Heterogeneous Academic Network

Research Collaboration and Impact Analysis

Problem Context:

You're analyzing academic research patterns using data from publications. Your network includes:

- **Authors:** Researchers who write papers
- **Papers:** Research publications
- **Venues:** Conferences and journals where papers appear
- **Institutions:** Organizations where authors work

You want to:

1. Find influential authors (not just by citation count)
2. Identify research communities that span institutions
3. Discover emerging collaboration patterns

Data Modeling Decisions:

- **Node types:** Authors, Papers, Venues, Institutions (heterogeneous)
- **Edge types:** * Author → Paper (authorship) * Paper → Venue (publication) * Author → Institution (affiliation)
- **Network type:** Heterogeneous Information Network (HIN)

Complete Code:

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
import networkx as nx
from collections import Counter, defaultdict

# === 1. Create the academic network ===
network = multinet.multi_layer_network()

# Academic publication data
# In practice: load from DBLP, Semantic Scholar, or similar

# Authors write papers
authorship_data = [
    ('Dr_Smith', 'Paper_1'), ('Dr_Smith', 'Paper_2'),
    ('Dr_Jones', 'Paper_1'), ('Dr_Jones', 'Paper_3'),
    ('Dr_Chen', 'Paper_2'), ('Dr_Chen', 'Paper_4'),
    ('Dr_Kim', 'Paper_3'), ('Dr_Kim', 'Paper_4'),
    ('Dr_Garcia', 'Paper_5'), ('Dr_Lee', 'Paper_5'),
    ('Dr_Smith', 'Paper_6'), ('Dr_Lee', 'Paper_6'), # Cross-institution collab
]

# Papers published in venues
publication_data = [
    ('Paper_1', 'ICML'), ('Paper_2', 'NeurIPS'),
    ('Paper_3', 'ICML'), ('Paper_4', 'NeurIPS'),
    ('Paper_5', 'AAAI'), ('Paper_6', 'ICML'),
]

# Authors affiliated with institutions
affiliation_data = [
```

(continues on next page)

(continued from previous page)

```

    ('Dr_Smith', 'MIT'), ('Dr_Jones', 'MIT'),
    ('Dr_Chen', 'Stanford'), ('Dr_Kim', 'Stanford'),
    ('Dr_Garcia', 'Berkeley'), ('Dr_Lee', 'Berkeley'),
]

# Add to network with explicit layer/type information
for author, paper in authorship_data:
    network.add_edges([
        [author, 'authors', paper, 'papers', 1.0]
    ], input_type="list")

for paper, venue in publication_data:
    network.add_edges([
        [paper, 'papers', venue, 'venues', 1.0]
    ], input_type="list")

for author, institution in affiliation_data:
    network.add_edges([
        [author, 'authors', institution, 'institutions', 1.0]
    ], input_type="list")

# === 2. Network overview ===
print("=== Academic Network Overview ===")
network.basic_stats()

# === 3. Meta-path based analysis ===
# Meta-path: Author → Paper → Venue → Paper → Author
# This finds authors who publish in the same venues

print("\n=== Meta-Path Analysis: Authors Publishing in Same Venues ===")

G = network.core_network

# Find which authors publish where
author_venues = defaultdict(set)
for author, paper in authorship_data:
    for p, venue in publication_data:
        if paper == p:
            author_venues[author].add(venue)

print("Authors by venue:")
venue_authors = defaultdict(list)
for author, venues in author_venues.items():
    for venue in venues:
        venue_authors[venue].append(author)

for venue, authors in sorted(venue_authors.items()):
    print(f" {venue}: {authors}")

# Find co-venue patterns (authors who could collaborate based on venue)
print("\nPotential collaborators (same venues):")
for venue, authors in venue_authors.items():
    if len(authors) >= 2:
        for i in range(len(authors)):
            for j in range(i+1, len(authors)):
                print(f" {authors[i]} -- {authors[j]} (both publish at {venue})")

```

(continues on next page)

(continued from previous page)

```

# === 4. Cross-institution collaboration analysis ===
print("\n=== Cross-Institution Collaborations ===")

# Find author affiliations
author_institution = dict(affiliation_data)

# Find collaborations (authors on same paper)
collaborations = defaultdict(set)
paper_authors = defaultdict(list)
for author, paper in authorship_data:
    paper_authors[paper].append(author)

cross_institution_collabs = []
for paper, authors in paper_authors.items():
    if len(authors) >= 2:
        for i in range(len(authors)):
            for j in range(i+1, len(authors)):
                a1, a2 = authors[i], authors[j]
                inst1 = author_institution.get(a1, 'Unknown')
                inst2 = author_institution.get(a2, 'Unknown')
                if inst1 != inst2:
                    cross_institution_collabs.append((a1, a2, paper, inst1, inst2))

print("Cross-institution collaborations:")
for a1, a2, paper, inst1, inst2 in cross_institution_collabs:
    print(f" {a1} ({inst1}) -- {a2} ({inst2}) on {paper}")

# === 5. Author influence metrics ===
print("\n=== Author Influence ===")

# Count papers per author
author_paper_count = Counter(a for a, p in authorship_data)

# Count unique venues per author
author_venue_count = {a: len(v) for a, v in author_venues.items()}

# Count co-authors
author_coauthors = defaultdict(set)
for paper, authors in paper_authors.items():
    for author in authors:
        for coauthor in authors:
            if author != coauthor:
                author_coauthors[author].add(coauthor)

print("Author metrics:")
print(f"{'Author':<15} {'Papers':<8} {'Venues':<8} {'Coauthors':<10}")
print("-" * 45)

all_authors = set(a for a, p in authorship_data)
for author in sorted(all_authors):
    papers = author_paper_count.get(author, 0)
    venues = author_venue_count.get(author, 0)
    coauthors = len(author_coauthors.get(author, set()))
    print(f"{author:<15} {papers:<8} {venues:<8} {coauthors:<10}")

```

(continues on next page)

(continued from previous page)

```
# === 6. Institution collaboration network ===
print("\n=== Institution Collaboration Network ===")

# Build institution-level collaboration graph
inst_collabs = Counter()
for a1, a2, paper, inst1, inst2 in cross_institution_collabs:
    pair = tuple(sorted([inst1, inst2]))
    inst_collabs[pair] += 1

print("Institution pairs by collaboration count:")
for pair, count in inst_collabs.most_common():
    print(f" {pair[0]} -- {pair[1]}: {count} joint papers")
```

Interpretation:

1. **Meta-path analysis** reveals implicit relationships—authors who publish at the same venues but haven't collaborated yet are potential future collaborators.
2. **Cross-institution collaborations** highlight knowledge transfer patterns. The Smith-Lee collaboration bridges MIT and Berkeley.
3. **Author influence** considers multiple factors beyond raw paper count:
 - Venue diversity (publishing at multiple top venues)
 - Collaboration breadth (working with many different people)
 - Cross-institution reach

Adapting to Your Data:

1. Load from academic databases (DBLP XML, Semantic Scholar API)
2. Add citation edges between papers for impact analysis
3. Include temporal information for trend analysis
4. Weight edges by author position (first author, corresponding author)

Adapting Case Studies to Your Domain**General Adaptation Process**

1. **Identify your entities** → These become node types or layers
2. **Identify your relationships** → These become edges
3. **Decide multiplex vs. heterogeneous:**
 - Same entities, different relationship types → Multiplex
 - Different entity types → Heterogeneous
4. **Map identifiers** → Ensure same entity has same ID across layers
5. **Choose appropriate metrics** based on your research questions
6. **Validate with domain knowledge** → Do results match expectations?

Data Loading Templates**From CSV with explicit layers:**

```
import pandas as pd
from py3plex.core import multinet
```

(continues on next page)

(continued from previous page)

```

network = multinet.multi_layer_network()

df = pd.read_csv('your_data.csv')
# Expected columns: source, target, layer, weight

for _, row in df.iterrows():
    network.add_edges([
        [row['source'], row['layer'], row['target'], row['layer'], row['weight']]
    ], input_type="list")

```

From multiple files (one per layer):

```

layer_files = {
    'layer1': 'layer1_edges.csv',
    'layer2': 'layer2_edges.csv',
    'layer3': 'layer3_edges.csv',
}

network = multinet.multi_layer_network()

for layer_name, filepath in layer_files.items():
    df = pd.read_csv(filepath)
    for _, row in df.iterrows():
        network.add_edges([
            [row['source'], layer_name, row['target'], layer_name, 1.0]
        ], input_type="list")

```

From database:

```

import sqlite3
from py3plex.core import multinet

network = multinet.multi_layer_network()

conn = sqlite3.connect('network.db')
cursor = conn.cursor()

cursor.execute('SELECT source, target, layer, weight FROM edges')
for source, target, layer, weight in cursor.fetchall():
    network.add_edges([
        [source, layer, target, layer, weight]
    ], input_type="list")

conn.close()

```

Conclusion: Patterns Across Domains

Looking across these case studies, several patterns emerge:

Multilayer structure reveals what single-layer analysis misses. In the PPI network, interactions confirmed by multiple evidence types are more reliable—information you lose if you flatten into a single network. In the social network, cross-platform influencers are fundamentally different from single-platform celebrities. In transportation, failure cascades depend on inter-modal connections.

The coupling parameter matters. Lower coupling ($\omega < 1$) finds layer-specific communities; higher coupling ($\omega > 1$) finds cross-layer communities. The right choice depends on your question. If you expect the same underlying structure across layers, use higher coupling. If layers represent truly different phenomena, use lower coupling.

Validation requires domain expertise. Network analysis can find structure, but whether that structure is meaningful requires interpretation. Do the detected communities correspond to known functional categories? Do the identified hubs match known influential entities? Ground your analysis in domain knowledge.

Start simple. Each case study could be made more complex—weighted edges, temporal dynamics, more layers, heterogeneous node types. But starting simple lets you understand what’s happening and catch errors before the analysis becomes opaque.

The Meta-Lesson

These case studies demonstrate a workflow pattern:

1. **Formulate your question** in terms of network structure
2. **Model your data** as nodes, edges, and layers
3. **Apply appropriate algorithms** based on your question
4. **Interpret results** using domain knowledge
5. **Iterate** as you learn what works

This pattern applies regardless of domain. The specific proteins, users, or stations change; the analytical approach remains similar.

Further Reading

- *Analysis Recipes & Workflows* — More ready-to-use recipes
- *Chapter 3: Multilayer Networks 101* — Conceptual foundations
- *Community Detection* — Detailed community detection guide
- *Network Statistics* — Complete statistics reference
- *Algorithm Landscape* — Algorithm selection guide

Academic References:

- Kivela et al. (2014). “Multilayer networks.” *Journal of Complex Networks*.
- De Domenico et al. (2015). “Ranking in interconnected multilayer networks.” *Nature Communications*.
- Battiston et al. (2017). “The physics of higher-order interactions in complex systems.” *Nature Physics*.

8.4 Part IV: Environments & Deployment

Moving from development to production requires careful consideration of deployment strategies and performance optimization. This section covers how to use py3plex via command-line interfaces and Docker containers for reproducible analyses, as well as techniques for scaling to large networks and optimizing performance for demanding workloads.

8.4.1 Environments & Deployment: Taking It to Production

“In theory, there is no difference between theory and practice. In practice, there is.” — Yogi Berra

Beyond the Notebook

Up to this point, you’ve been learning py3plex in an exploratory context: running code interactively, visualizing results, iterating on analysis. This is exactly how research and development should work.

But there comes a time when your analysis needs to become reliable, reproducible, and scalable. Maybe you need to:

- Run the same analysis on hundreds of networks overnight

- Process datasets too large for a single Python session
- Share your workflow with collaborators who have different environments
- Deploy your analysis as part of a larger pipeline

This section covers the practical considerations for moving py3plex from exploration to production.

What You'll Learn

CLI and Docker explains how to use py3plex from the command line and as a containerized application. The CLI provides a scriptable interface for common operations—no Python needed. Docker ensures your analysis runs identically everywhere, eliminating “works on my machine” problems.

Performance & Scalability addresses the challenges of working with large networks. Networks with millions of nodes require different strategies than networks with thousands. We cover memory management, computational complexity, and practical optimization techniques.

When to Read This Section

If you're still in the exploratory phase of your work, you can skip this section for now. Come back when:

- You need to automate your analysis
- Your networks are getting large enough that memory or time become issues
- You want to share reproducible environments with collaborators
- You're integrating py3plex into a production pipeline

Real-World Applications

Research Reproducibility:

A common challenge in computational research is ensuring that analyses can be reproduced. Using Docker containers with pinned dependency versions ensures that anyone, anywhere, can run your code and get identical results—even years later.

Large-Scale Studies:

Some applications involve analyzing many networks or very large networks. A research group studying brain connectomes might process hundreds of patient scans. A company analyzing social network data might have networks with millions of users. The techniques in this section make such analyses tractable.

Integration with Other Tools:

Real workflows often combine multiple tools. You might use py3plex for network analysis, pandas for data manipulation, and custom scripts for domain-specific processing. The CLI and container approaches make py3plex a well-behaved component in larger pipelines.

The Production Mindset

Production code differs from exploratory code in several ways:

- **Error handling:** Production code must handle edge cases gracefully
- **Logging:** You need to know what happened when things go wrong
- **Testing:** Changes should be verified before deployment
- **Documentation:** Others (including your future self) need to understand the code
- **Performance:** Efficiency matters when running at scale

The chapters in this section help you think about these concerns in the context of py3plex.

Moving Forward

If you're ready to take your analysis to the next level, start with *Docker Usage Guide* for automation and reproducibility, then move to *Performance and Scalability Best Practices* for optimization techniques.

Tip

Deployment Checklist:

Before deploying a py3plex workflow to production:

- Version-pin all dependencies
- Add error handling for edge cases
- Verify results match expectations on test data
- Document the analysis workflow
- Set up logging for debugging
- Test on the target environment (container, server, etc.)

8.4.2 Docker Usage Guide

This guide provides comprehensive instructions for using Py3plex via Docker.

Table of Contents

- *Quick Start*
- *Building the Image*
 - *Using Docker*
 - *Using Docker Compose*
- *Running Commands*
 - *Basic Commands*
 - *Using Docker Compose*
- *Working with Files*
 - *Creating a Data Directory*
 - *Mounting Volumes*
 - *Complete Workflow Example*
- *Advanced Usage*
 - *Interactive Shell*
 - *Using Python API*
 - *Custom Image Builds*
 - *Docker Compose with Multiple Services*
- *Helper Scripts*
 - *py3plex-docker.sh*
 - *test-docker-setup.sh*
- *Docker Configuration Files*

- *.dockerignore*
- *Troubleshooting*
 - *Testing the Docker Setup*
 - *Permission Issues*
 - *Container Not Found*
 - *Network Issues During Build*
 - *Out of Disk Space*
 - *Debugging Build Issues*
- *Best Practices*
- *Practical Examples*
 - *Basic Network Analysis*
 - *Community Detection*
 - *Centrality Analysis*
 - *Batch Processing*
 - *Custom Analysis Script*
 - *Complete Analysis Pipeline*
 - *Comparative Network Analysis*
- *CI/CD Integration*
 - *GitHub Actions*
 - *GitLab CI*
- *Additional Resources*
- *Support*

Quick Start

The fastest way to get started with Py3plex using Docker:

```
# Clone the repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Build the Docker image
docker build -t py3plex:latest .

# Run a self-test to verify installation
docker run --rm py3plex:latest selftest

# Display help
docker run --rm py3plex:latest help
```

Building the Image

Using Docker

```
# Build from the repository root
docker build -t py3plex:latest .

# Build with a specific tag
docker build -t py3plex:0.95a .

# Build without cache (if needed)
docker build --no-cache -t py3plex:latest .
```

Using Docker Compose

```
# Build using docker-compose
docker-compose build

# Force rebuild
docker-compose build --no-cache
```

Running Commands

Basic Commands

The Docker container is set up with py3plex as the entrypoint, so you can run any py3plex CLI command directly:

```
# Show version
docker run --rm py3plex:latest --version

# Run self-test
docker run --rm py3plex:latest selftest

# Show help
docker run --rm py3plex:latest help

# Show help for a specific command
docker run --rm py3plex:latest create --help
```

Using Docker Compose

With docker-compose, commands are slightly more verbose but easier to manage:

```
# Show version
docker-compose run --rm py3plex --version

# Run self-test
docker-compose run --rm py3plex selftest -v

# Show help
docker-compose run --rm py3plex help
```

Working with Files

To work with network files, you need to mount a local directory to the container's /data directory.

Creating a Data Directory

```
# Create a local data directory
mkdir -p data
```

Mounting Volumes

With docker run:

```
# Mount current directory's data folder
docker run --rm -v $(pwd)/data:/data py3plex:latest create --nodes 100 --layers 3 --
↳output /data/network.edgelist

# On Windows (PowerShell)
docker run --rm -v ${PWD}/data:/data py3plex:latest create --nodes 100 --layers 3 --
↳output /data/network.edgelist

# On Windows (CMD)
docker run --rm -v %cd%/data:/data py3plex:latest create --nodes 100 --layers 3 --
↳output /data/network.edgelist
```

With docker-compose:

The docker-compose.yml file already configures volume mounting from ./data to /data, so you can simply:

```
docker-compose run --rm py3plex create --nodes 100 --layers 3 --output /data/network.
↳edgelist
```

Complete Workflow Example

```
# Create data directory
mkdir -p data

# 1. Create a multilayer network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  create --nodes 100 --layers 3 --type random --probability 0.1 --output /data/
↳network.edgelist

# 2. Load and display network information
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  load /data/network.edgelist --info

# 3. Compute statistics
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  stats /data/network.edgelist --measure all --output /data/stats.json

# 4. Detect communities
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain --output /data/communities.json

# 5. Visualize the network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png --layout multilayer

# 6. View the results
ls -lh data/
```

Advanced Usage

Interactive Shell

To explore the container interactively:

```
# Start an interactive shell in the container
docker run --rm -it -v $(pwd)/data:/data --entrypoint /bin/bash py3plex:latest

# Inside the container, you can run:
# py3plex --version
# py3plex selftest
# python -c "import py3plex; print(py3plex.__version__)"
```

Using Python API

You can also use py3plex as a Python library inside the container:

```
# Create a Python script
cat > data/analyze.py << 'EOF'
from py3plex.core import multinet

# Create a network
network = multinet.multi_layer_network()
nodes = [{"source": f"node{i}", "type": "layer1"} for i in range(10)]
network.add_nodes(nodes, input_type="dict")

print(f"Created network with {network.core_network.number_of_nodes()} nodes")
EOF

# Run the script in the container
docker run --rm -v $(pwd)/data:/data --entrypoint python py3plex:latest /data/analyze.
↪py
```

Custom Image Builds

If you need additional packages:

```
# Create a custom Dockerfile
FROM py3plex:latest

# Install additional packages
RUN pip install --no-cache-dir pandas jupyter

# Or system packages
USER root
RUN apt-get update && apt-get install -y vim && rm -rf /var/lib/apt/lists/*
USER py3plex
```

Docker Compose with Multiple Services

You can extend the docker-compose.yml to include related services:

```
version: '3.8'

services:
  py3plex:
```

(continues on next page)

(continued from previous page)

```

build: .
image: py3plex:latest
volumes:
  - ./data:/data

jupyter:
  image: py3plex:latest
  ports:
    - "8888:8888"
  volumes:
    - ./data:/data
    - ./notebooks:/notebooks
  command: jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser --allow-root
  working_dir: /notebooks

```

Helper Scripts

The repository includes convenient helper scripts to simplify Docker usage.

py3plex-docker.sh

A wrapper script that simplifies running py3plex commands via Docker:

```

# The script automatically handles:
# - Docker installation checks
# - Image building (if needed)
# - Data directory creation
# - Volume mounting

# Usage examples:
./py3plex-docker.sh --version
./py3plex-docker.sh selftest
./py3plex-docker.sh create --nodes 100 --layers 3 --output /data/network.edgelist
./py3plex-docker.sh load /data/network.edgelist --info

```

Features:

- Automatically checks if Docker is installed
- Prompts to build the image if it doesn't exist
- Creates the data/ directory if needed
- Mounts the local data/ directory to /data in the container
- Passes all arguments directly to py3plex

Script location: py3plex-docker.sh in the repository root

test-docker-setup.sh

A comprehensive test script to validate your Docker setup:

```

# Run the validation script
./test-docker-setup.sh

```

What it tests:

1. Docker installation verification
2. Dockerfile existence check

3. `.dockerignore` existence check
4. `docker-compose.yml` existence check
5. Docker image build process
6. Container version command
7. Container help command
8. Container selftest
9. Volume mounting and file creation

The script provides colored output ([OK] PASS / [X] FAIL) and a summary report.

Script location: `test-docker-setup.sh` in the repository root

Docker Configuration Files

`.dockerignore`

The `.dockerignore` file controls which files are excluded from the Docker build context:

Key exclusions:

- **Git and version control:** `.git/`, `.github/`, `.gitignore`
- **Python artifacts:** `__pycache__/`, `*.pyc`, `dist/`, `build/`
- **Virtual environments:** `venv/`, `env/`
- **Testing and coverage:** `.pytest_cache/`, `htmlcov/`, `coverage.*`
- **Documentation builds:** `docs/_build/`, `docfiles/`
- **Examples and datasets:** `examples/`, `datasets/`, `multilayer_datasets/`
- **Development files:** `tests/`, `run_tests.py`, `validate_*.py`

Why this matters:

- **Faster builds:** Smaller build context means faster Docker builds
- **Smaller images:** Excludes unnecessary files from the final image
- **Security:** Prevents accidental inclusion of sensitive files
- **Efficiency:** Only includes files needed to run py3plex

The `.dockerignore` file is located in the repository root.

Troubleshooting

Testing the Docker Setup

To verify your Docker setup is working correctly, use the included test script:

```
# Run the test script
./test-docker-setup.sh
```

This script will:

- Check Docker installation
- Verify all Docker-related files exist
- Build the Docker image
- Run basic py3plex commands (`--version`, `help`, `selftest`)
- Test volume mounting and file creation

- Clean up test artifacts

Permission Issues

If you encounter permission issues with mounted volumes:

```
# On Linux, you might need to run with your user ID
docker run --rm -v $(pwd)/data:/data --user $(id -u):$(id -g) py3plex:latest create --
↳ output /data/network.edgelist
```

Container Not Found

If the image isn't found:

```
# List available images
docker images | grep py3plex

# Rebuild if necessary
docker build -t py3plex:latest .
```

Network Issues During Build

If you encounter network timeouts during build:

```
# Increase timeout
docker build --build-arg PIP_DEFAULT_TIMEOUT=300 -t py3plex:latest .

# Or use a different PyPI mirror
docker build --build-arg PIP_INDEX_URL=https://pypi.tuna.tsinghua.edu.cn/simple -t
↳ py3plex:latest .
```

Out of Disk Space

Clean up unused Docker resources:

```
# Remove unused images
docker image prune -a

# Remove all stopped containers, unused networks, and dangling images
docker system prune -a
```

Debugging Build Issues

Build with verbose output:

```
docker build --progress=plain --no-cache -t py3plex:latest .
```

Best Practices

1. **Use Volume Mounts:** Always mount volumes for input/output files
2. **Use `--rm` Flag:** Remove containers after execution with `--rm` to save space
3. **Tag Images:** Use specific tags (e.g., `py3plex:0.95a`) for reproducibility
4. **Keep Data Separate:** Store network files in the mounted data directory
5. **Use Docker Compose:** For repeated operations, docker-compose simplifies commands
6. **Regular Updates:** Rebuild the image periodically to get latest py3plex updates

Practical Examples

Basic Network Analysis

Create and analyze a multilayer network:

```
# Create data directory
mkdir -p data

# Create a network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  create --nodes 100 --layers 3 --type random --probability 0.05 \
  --output /data/network.edgelist

# Analyze the network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  load /data/network.edgelist --info --stats

# Visualize
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png
```

Community Detection

```
# Detect communities using Louvain
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain \
  --output /data/communities.json

# View community statistics
cat data/communities.json | python -m json.tool | head -20
```

Centrality Analysis

```
# Compute degree centrality
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  centrality /data/network.edgelist --measure degree \
  --top 10 --output /data/centrality.json

# Compute betweenness centrality
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  centrality /data/network.edgelist --measure betweenness \
  --top 10 --output /data/betweenness.json
```

Batch Processing

Process multiple networks (save as batch-process.sh):

```
#!/bin/bash
# Process multiple networks in batch
set -e

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

# Create several networks
```

(continues on next page)

(continued from previous page)

```

for i in {1..5}; do
  echo "Creating network $i..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    create --nodes $((50 * i)) --layers 3 \
    --type random --probability 0.1 \
    --output /data/network_$i.edgelist
done

# Analyze each network
for i in {1..5}; do
  echo "Analyzing network $i..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    stats /data/network_$i.edgelist --measure all \
    --output /data/stats_$i.json
done

echo "Done! Results in $DATA_DIR"

```

Custom Analysis Script

Create a Python script (analysis.py) and run it in the container:

Python script:

```

# analysis.py
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Load network
network = multinet.multi_layer_network()
network.load_network("/data/network.edgelist", input_type="multiedgelist")

# Compute statistics
layers = ["layer1", "layer2", "layer3"]
for layer in layers:
    density = mls.layer_density(network, layer)
    print(f"{layer} density: {density:.4f}")

print("Analysis complete!")

```

Run it:

```

# Copy script to data directory
cp analysis.py data/

# Run in container
docker run --rm -v $(pwd)/data:/data \
  --entrypoint python py3plex:latest /data/analysis.py

```

Complete Analysis Pipeline

A comprehensive analysis pipeline:

```

#!/bin/bash
# complete-pipeline.sh
set -e

```

(continues on next page)

(continued from previous page)

```

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

echo "Step 1: Creating network..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  create --nodes 200 --layers 3 --type ba --probability 0.05 \
  --output /data/network.edgelist

echo "Step 2: Computing statistics..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  stats /data/network.edgelist --measure all \
  --output /data/stats.json

echo "Step 3: Detecting communities..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain \
  --output /data/communities.json

echo "Step 4: Computing centrality..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  centrality /data/network.edgelist --measure pagerank \
  --top 20 --output /data/centrality.json

echo "Step 5: Visualizing..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png \
  --layout multilayer --width 16 --height 12

echo "Pipeline complete! Check $DATA_DIR for results."

```

Comparative Network Analysis

Compare different network types:

```

#!/bin/bash
# compare-networks.sh
set -e

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

TYPES=("random" "er" "ba" "ws")

for type in "${TYPES[@]}; do
  echo "Creating $type network..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    create --nodes 100 --layers 3 --type $type --probability 0.1 \
    --output /data/network_${type}.edgelist

  echo "Analyzing $type network..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    stats /data/network_${type}.edgelist --measure all \
    --output /data/stats_${type}.json
done

```

(continues on next page)

(continued from previous page)

```
echo "Comparison complete!"
```

CI/CD Integration

GitHub Actions

Example workflow for network analysis:

```
# .github/workflows/network-analysis.yml
name: Network Analysis with Docker

on: [push, pull_request]

jobs:
  analyze:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Build Docker image
        run: docker build -t py3plex:latest .

      - name: Create test network
        run: |
          mkdir -p data
          docker run --rm -v $PWD/data:/data py3plex:latest \
            create --nodes 50 --layers 2 --output /data/test.edgelist

      - name: Run analysis
        run: |
          docker run --rm -v $PWD/data:/data py3plex:latest \
            stats /data/test.edgelist --measure all --output /data/stats.json

      - name: Upload results
        uses: actions/upload-artifact@v3
        with:
          name: analysis-results
          path: data/
```

GitLab CI

Example GitLab CI configuration:

```
# .gitlab-ci.yml
image: docker:latest

services:
  - docker:dind

variables:
  DOCKER_DRIVER: overlay2

stages:
  - build
  - analyze
```

(continues on next page)

(continued from previous page)

```

build:
  stage: build
  script:
    - docker build -t py3plex:latest .
    - docker save py3plex:latest > py3plex-image.tar
  artifacts:
    paths:
      - py3plex-image.tar

analyze:
  stage: analyze
  script:
    - docker load < py3plex-image.tar
    - mkdir -p data
    - docker run --rm -v $PWD/data:/data py3plex:latest create --nodes 100 --layers 3
    ↪--output /data/network.edgelist
    - docker run --rm -v $PWD/data:/data py3plex:latest stats /data/network.edgelist -
    ↪-measure all --output /data/stats.json
  artifacts:
    paths:
      - data/

```

Additional Resources

- [Py3plex Documentation](#)
- [CLI Tutorial](#)
- [Docker Documentation](#)
- [Docker Compose Documentation](#)
- [DOCKER.md](#) - Main Docker guide in the repository

Support

For issues related to:

- **Py3plex functionality:** Open an issue at <https://github.com/SkBlaz/py3plex/issues>
- **Docker setup:** Check this guide first, then open an issue if problems persist
- **General questions:** See the main README.md and documentation

8.4.3 Performance and Scalability Best Practices

This guide provides recommendations for optimizing Py3plex performance and handling large multilayer networks.

Table of Contents

- *Network Scale Guidelines*
- *Sparse Matrix Backend*
 - *Why Sparse Matrices?*
 - *Automatic Sparse Matrix Usage*
 - *Force Sparse Operations*
- *Network Sampling*

- *When to Sample*
- *Random Node Sampling*
- *Stratified Sampling (Preserve Layer Distribution)*
- *Hub-Based Sampling (Keep Important Nodes)*
- *Algorithm Optimization*
 - *Choose Efficient Algorithms*
 - *Limit Algorithm Iterations*
- *Parallel Processing*
 - *Multi-Core Processing*
 - *GPU Acceleration (Advanced)*
- *Visualization Optimization*
 - *Reduce Visual Complexity*
 - *Save to File Instead of Display*
 - *Use Lower Resolution*
- *Memory Management*
 - *Monitor Memory Usage*
 - *Free Memory When Done*
 - *Use Generators Instead of Lists*
- *Batch Processing*
 - *Process Networks in Batches*
- *Benchmark Results*
 - *Performance Benchmarks (2025)*
 - *Scaling Recommendations*
- *Alternative Tools for Scale*
 - *When Py3plex Isn't Enough*
- *Quick Performance Checklist*
 - *Before Running on Large Network*
 - *Optimization Order*
- *Next Steps*

Network Scale Guidelines

Py3plex is optimized for research-scale networks. This table shows expected performance characteristics:

Table 1: Network Scale Performance

Network Size	Performance	Visualization	Recommendations
Small (<100 nodes)	Excellent	Fast, detailed	Use dense visualization mode
Medium (100-1k nodes)	Good	Fast, balanced	Default settings work well
Large (1k-10k nodes)	Good	Slower, minimal	Use sparse matrices, sampling
Very Large (>10k nodes)	Variable	Very slow	Sampling required, use NetworkX/igraph

Sparse Matrix Backend

Why Sparse Matrices?

Most real-world networks are **sparse** (few edges compared to possible edges). Sparse matrices:

- **Reduce memory usage** by 10-100x for typical networks
- **Speed up** matrix operations (multiplication, inversion)
- **Enable** analysis of larger networks

Example:

```
import numpy as np
from scipy.sparse import csr_matrix

# Dense representation (10k × 10k network)
# Memory: 10,000^2 × 8 bytes = 800 MB

# Sparse representation (with 1% density)
# Memory: ~8 MB (100x reduction!)
```

Automatic Sparse Matrix Usage

Py3plex **automatically uses sparse matrices** for:

- Supra-adjacency matrix operations
- Large network storage (>1000 nodes)
- Matrix-based algorithms (PageRank, spectral methods)

Verify sparse usage:

```
from py3plex.core import multinet

network = multinet.multi_layer_network()
network.load_network("large_network.csv", input_type="multiedgelist")

# Get sparse adjacency matrix
adj_sparse = network.get_sparse_adjacency_matrix()

print(f"Matrix size: {adj_sparse.shape}")
print(f"Non-zero entries: {adj_sparse.nnz}")
print(f"Sparsity: {1 - adj_sparse.nnz / (adj_sparse.shape[0]**2):.2%}")
```

Force Sparse Operations

For custom algorithms, explicitly use sparse operations:

```
from scipy.sparse import csr_matrix, lil_matrix
import numpy as np

# Create sparse adjacency matrix
adj = lil_matrix((n_nodes, n_nodes))

for u, v in network.core_network.edges():
    i, j = node_to_idx[u], node_to_idx[v]
    adj[i, j] = 1
    adj[j, i] = 1 # Undirected

# Convert to efficient format for operations
adj_csr = adj.tocsr()

# Sparse matrix operations are MUCH faster
result = adj_csr @ adj_csr # Matrix multiplication
eigvals = scipy.sparse.linalg.eigs(adj_csr, k=10) # Top eigenvalues
```

Network Sampling

When to Sample

Sampling is necessary when:

- Network is too large to visualize (>5k nodes)
- Algorithms take too long (>10 minutes)
- Memory usage is excessive (>8GB RAM)
- You need quick exploratory analysis

Random Node Sampling

```
import random
from py3plex.core import multinet

# Load full network
network = multinet.multi_layer_network()
network.load_network("large_network.csv", input_type="multiedgelist")

# Sample 1000 random nodes
all_nodes = list(network.get_nodes())
sample_nodes = random.sample(all_nodes, min(1000, len(all_nodes)))

# Create subnetwork
subnetwork = network.get_subnetwork(sample_nodes)

print(f"Original: {len(all_nodes)} nodes")
print(f"Sample: {len(sample_nodes)} nodes")
print(f"Sampling ratio: {len(sample_nodes)/len(all_nodes):.1%}")
```

Stratified Sampling (Preserve Layer Distribution)

```
# Sample proportionally from each layer
layers = network.get_layer_names()
sample_nodes_per_layer = {}

for layer in layers:
    layer_nodes = [n for n in network.get_nodes() if n[1] == layer]
    sample_size = len(layer_nodes) // 10 # 10% sample
    sample_nodes_per_layer[layer] = random.sample(layer_nodes, sample_size)

# Combine samples
all_samples = [node for nodes in sample_nodes_per_layer.values() for node in nodes]
subnetwork = network.get_subnetwork(all_samples)
```

Hub-Based Sampling (Keep Important Nodes)

```
import networkx as nx

# Sample high-degree nodes (hubs)
degrees = dict(network.core_network.degree())

# Sort by degree and take top 1000
sorted_nodes = sorted(degrees.items(), key=lambda x: x[1], reverse=True)
hub_nodes = [node for node, deg in sorted_nodes[:1000]]

subnetwork = network.get_subnetwork(hub_nodes)
print(f"Sampled top 1000 hubs")
```

Algorithm Optimization

Choose Efficient Algorithms

Some algorithms scale better than others:

Table 2: Algorithm Complexity

Algorithm	Complexity	Recommendations
Degree centrality	$O(n + m)$	Fast, use freely
Betweenness centrality	$O(nm)$	Slow for large networks, sample first
PageRank	$O(\text{iterations} \times m)$	Fast if sparse, limit iterations
Community detection (Louvain)	$O(m \log n)$	Fast, recommended
Shortest paths (all pairs)	$O(n^2m)$	Very slow, use sampling or approximate
Force-directed layout	$O(n^2)$	Slow for >5k nodes, use alternatives

Example - Fast centrality:

```
import networkx as nx

G = network.core_network

# FAST: Degree centrality
degree_cent = nx.degree_centrality(G) #  $O(n+m)$  - instant

# SLOW: Betweenness centrality
```

(continues on next page)

(continued from previous page)

```
# For large networks, sample first or use approximate algorithm
if G.number_of_nodes() < 1000:
    between_cent = nx.betweenness centrality(G)
else:
    # Use approximate algorithm
    between_cent = nx.betweenness centrality(G, k=100) # Sample 100 nodes
```

Limit Algorithm Iterations

```
import networkx as nx

# PageRank with iteration limit
pagerank = nx.pagerank(
    network.core_network,
    max_iter=50,          # Limit iterations
    tol=1e-4              # Tolerance for convergence
)

# Community detection with resolution limit
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(
    network.core_network,
    resolution=1.0        # Adjust resolution parameter
)
```

Parallel Processing

Multi-Core Processing

Use joblib for parallel node/edge operations:

```
from joblib import Parallel, delayed
import networkx as nx

def compute_node Centrality(node, graph):
    """Compute centrality for a single node."""
    # Custom centrality computation
    neighbors = list(graph.neighbors(node))
    return node, len(neighbors)

# Parallel processing
nodes = list(network.core_network.nodes())
results = Parallel(n_jobs=-1)( # Use all cores
    delayed(compute_node Centrality)(node, network.core_network)
    for node in nodes
)

centralities = dict(results)
```

GPU Acceleration (Advanced)

For very large networks, use GPU acceleration:

```
# Install CuPy for GPU NumPy operations
pip install cupy-cuda11x # Replace 11x with CUDA version
```

```
# Requires NVIDIA GPU with CUDA
try:
    import cupy as cp

    # Convert to GPU array
    adj_matrix = network.get_sparse_adjacency_matrix().toarray()
    gpu_adj = cp.array(adj_matrix)

    # GPU-accelerated matrix operations
    gpu_result = cp.dot(gpu_adj, gpu_adj)

    # Transfer back to CPU
    result = cp.asnumpy(gpu_result)

except ImportError:
    print("CuPy not available, using CPU")
```

Visualization Optimization

Reduce Visual Complexity

```
from py3plex.visualization.multilayer import draw_multilayer_default

# For large networks (>1000 nodes)
draw_multilayer_default(
    network.get_layers(),
    node_size=3,           # Tiny nodes
    labels=False,         # No labels
    edge_size=0.3,        # Thin edges
    alphalevel=0.2,       # Very transparent
    remove_isolated_nodes=True # Remove disconnected
)
```

Save to File Instead of Display

```
import matplotlib.pyplot as plt

# Don't show interactively (faster)
fig, ax = plt.subplots(1, 1, figsize=(10, 8))
draw_multilayer_default(
    network.get_layers(),
    display=False, # Don't show
    axis=ax
)

# Save directly to file
plt.savefig('network.png', dpi=150, bbox_inches='tight')
plt.close() # Free memory
```

Use Lower Resolution

```
# For quick exploration, use low DPI
plt.figure(figsize=(8, 6), dpi=72) # Low resolution

# For publications, use high DPI
```

(continues on next page)

(continued from previous page)

```
plt.figure(figsize=(10, 8), dpi=300) # High resolution
```

Memory Management

Monitor Memory Usage

```
import psutil
import os

def get_memory_usage():
    """Get current memory usage in MB."""
    process = psutil.Process(os.getpid())
    return process.memory_info().rss / 1024 / 1024

print(f"Memory before loading: {get_memory_usage():.1f} MB")

network = multinet.multi_layer_network()
network.load_network("large_network.csv", input_type="multiedgelist")

print(f"Memory after loading: {get_memory_usage():.1f} MB")
```

Free Memory When Done

```
# Delete network when no longer needed
del network

# Force garbage collection
import gc
gc.collect()
```

Use Generators Instead of Lists

```
# BAD: Loads all nodes into memory
all_nodes = list(network.get_nodes())
for node in all_nodes:
    process(node)

# GOOD: Processes nodes one at a time
for node in network.get_nodes():
    process(node)
```

Batch Processing

Process Networks in Batches

For multiple networks:

```
import os
import gc

network_files = ["net1.csv", "net2.csv", "net3.csv", ...]

results = []
for i, file in enumerate(network_files):
```

(continues on next page)

(continued from previous page)

```

print(f"Processing {i+1}/{len(network_files)}: {file}")

# Load network
network = multinet.multi_layer_network()
network.load_network(file, input_type="multiedgelist")

# Compute statistics
result = analyze_network(network)
results.append(result)

# Free memory
del network
gc.collect()

# Save intermediate results every 10 networks
if (i + 1) % 10 == 0:
    save_results(results, f"results_batch_{i+1}.json")

```

Benchmark Results

Performance Benchmarks (2025)

Tested on: Intel i7-10700K, 32GB RAM, Python 3.10

Table 3: Operation Performance

Operation	100 nodes	1k nodes	10k nodes	100k nodes
Load from CSV	<1s	<1s	2s	20s
Basic statistics	<1s	<1s	<1s	3s
Degree centrality	<1s	<1s	1s	10s
PageRank	<1s	<1s	2s	25s
Louvain communities	<1s	1s	5s	60s
Visualization (sparse)	<1s	2s	15s	N/A*

* Visualization not recommended for >10k nodes without sampling

Scaling Recommendations

Based on network size:

<1k nodes:

- Use any algorithms
- Full visualization
- No sampling needed

1k-10k nodes:

- Use sparse matrices
- Minimal visualization
- Sample for some algorithms

10k-100k nodes:

- Sparse matrices required
- Sample for visualization

- Use approximate algorithms
- Consider igraph for speed

>100k nodes:

- Use specialized tools (igraph, graph-tool, NetworKit)
- Sample heavily for Py3plex operations
- Focus on specific analyses

Alternative Tools for Scale**When Py3plex Isn't Enough**

For networks >100k nodes, consider:

igraph (C-based, very fast):

```
import igraph as ig

# 10-100x faster for large networks
g = ig.Graph.Read_GraphML("large_network.graphml")
communities = g.community_multilevel()

# Export back to Py3plex if needed
# (via GraphML or edge list)
```

graph-tool (C++, fastest):

```
import graph_tool.all as gt

# Fastest for >1M edges
g = gt.load_graph("large_network.graphml")
communities = gt.community_structure.minimize_blockmodel_dl(g)
```

NetworKit (C++, parallel):

```
import networkit as nk

# Excellent for parallel algorithms
G = nk.readGraph("large_network.edgelist", nk.Format.EdgeList)
communities = nk.community.detectCommunities(G)
```

Quick Performance Checklist**Before Running on Large Network**

```
[ ] Enable sparse matrices (automatic for most ops)
[ ] Sample network if >10k nodes
[ ] Choose efficient algorithms (degree > betweenness)
[ ] Limit visualization detail
[ ] Monitor memory usage
[ ] Use batch processing for multiple networks
[ ] Consider alternative tools if >100k nodes
```

Optimization Order

1. **Use sparse matrices** (biggest impact, usually automatic)
2. **Sample network** (if >10k nodes)
3. **Choose efficient algorithms** (avoid $O(n^3)$ operations)
4. **Parallelize** (if multi-core available)
5. **GPU acceleration** (only if CUDA GPU available)

Next Steps

- *I/O and Serialization* - Efficient data loading
- *Visualization* - Optimize visualizations
- *Docker Usage Guide* - Docker deployment for production

For performance issues, open an issue on [GitHub Issues](#).

8.5 Part V: Py3plex GUI

For users who prefer a graphical interface, py3plex provides a web-based GUI for interactive network exploration and analysis. This section covers everything from basic usage to deployment, API integration, and testing. The GUI makes py3plex accessible to users who may not be comfortable with Python scripting while still providing access to the library's powerful analytical capabilities.

8.5.1 GUI: Visual Exploration for Everyone

"A picture is worth a thousand words." — Traditional Proverb

Why a Graphical Interface?

Not everyone who needs to analyze multilayer networks is comfortable writing Python code. Domain experts—biologists, social scientists, urban planners—often have deep knowledge of their systems but limited programming experience. For them, a graphical user interface (GUI) opens the door to powerful analysis tools.

Even for experienced programmers, visual interfaces have value. Quick exploration is often faster with point-and-click than with code. Visual feedback helps you understand data before committing to a specific analysis approach. Demonstrations to stakeholders are more compelling when they can interact with the visualization.

py3plex's GUI brings multilayer network analysis to a broader audience while remaining backed by the same robust algorithms as the Python library.

What the GUI Provides

The py3plex GUI is a web-based interface for interactive network exploration and analysis. You can:

- **Load networks** from various file formats without writing code
- **Visualize** your networks with interactive, zoomable layouts
- **Compute statistics** by selecting options from menus
- **Detect communities** and see them colored in the visualization
- **Export results** for further analysis or publication

The interface is designed to be intuitive for newcomers while still providing access to advanced features.

What This Section Covers

GUI User Guide walks you through using the interface: loading data, exploring networks, running analyses, and exporting results. If you're using the GUI for analysis, start here.

GUI Deployment covers how to run the GUI server—locally for personal use or on a server for team access. This is for administrators and power users.

GUI API Reference documents the backend API for developers who want to extend the GUI or integrate it with other systems.

GUI Architecture explains how the GUI is built, useful for developers who want to contribute to or modify the interface.

GUI Testing covers the testing infrastructure for the GUI components.

Who Should Use the GUI?

The GUI is particularly well-suited for:

Domain experts who want to analyze their networks without learning Python. The visual interface makes multilayer network analysis accessible.

Exploratory analysis when you want to quickly understand a new dataset. Visual exploration often reveals patterns faster than writing code.

Teaching and demonstrations where you want to show multilayer network concepts interactively.

Collaborative work where team members have varying technical backgrounds. Everyone can use the same interface.

For production pipelines, automated analyses, or advanced customization, the Python library remains the better choice. The GUI and library complement each other.

Getting Started with the GUI

To start using the GUI, follow the deployment instructions in *GUI Deployment Guide*, then work through the *Py3plex GUI* for a complete walkthrough.

Tip

GUI vs. Python Library:

Task	GUI	Python Library
Quick explore	✓ Better	Good
Scripted	Not suitable	✓ Better
Reproducible	Challenging	✓ Better
Non-technical	✓ Better	Challenging
Advanced	Limited	✓ Better

Choose the tool that fits your needs—or use both!

8.5.2 Py3plex GUI

A production-ready web-based GUI for **py3plex** multilayer network analysis, running locally via Docker Compose.

Visual Overview

The Py3plex GUI provides an intuitive web interface for multilayer network analysis. Below are screenshots of the main interface pages:

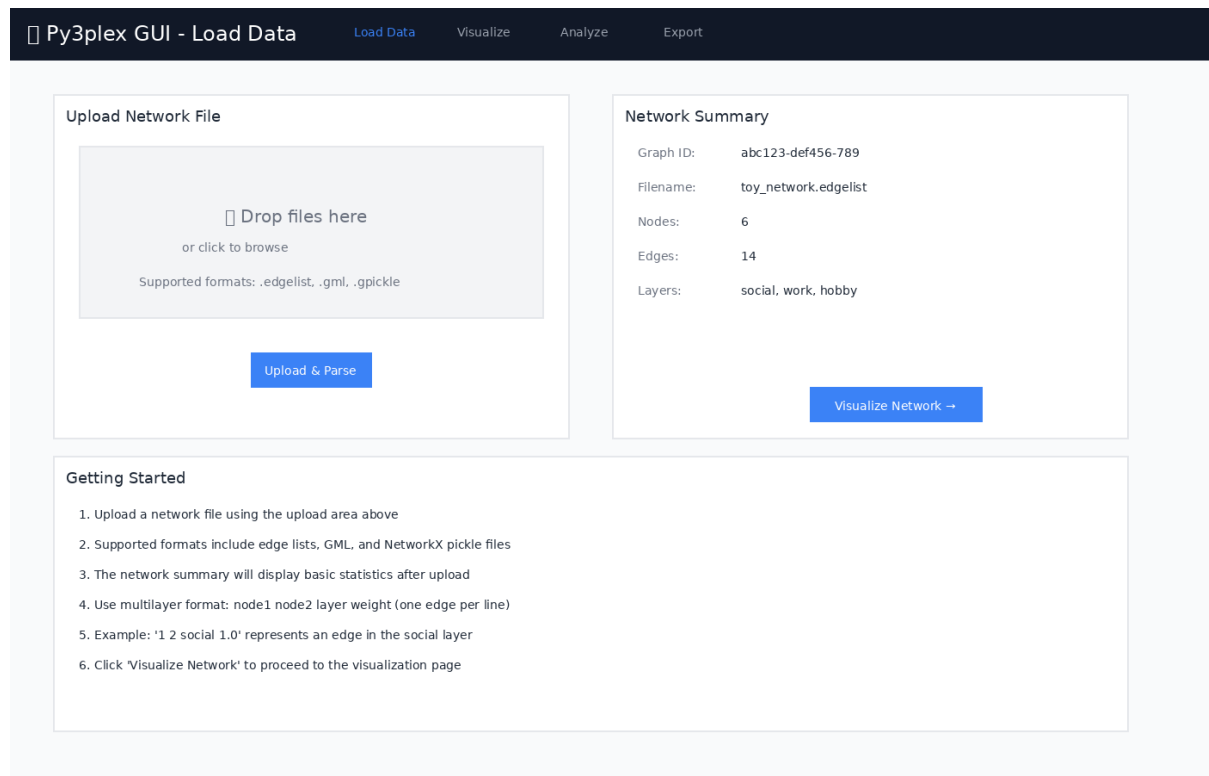


Figure 1: Load Data page for uploading network files in various formats

Quick Start

```
# Navigate to the gui directory
cd gui

# Copy environment configuration
cp .env.example .env

# Start all services (builds automatically)
make up

# Open in browser
# Application: http://localhost:8080
# Data Job Monitor (Flower): http://localhost:5555
```

First time? The build takes ~3-5 minutes. Subsequent starts are instant.

That's it! The application will be running with:

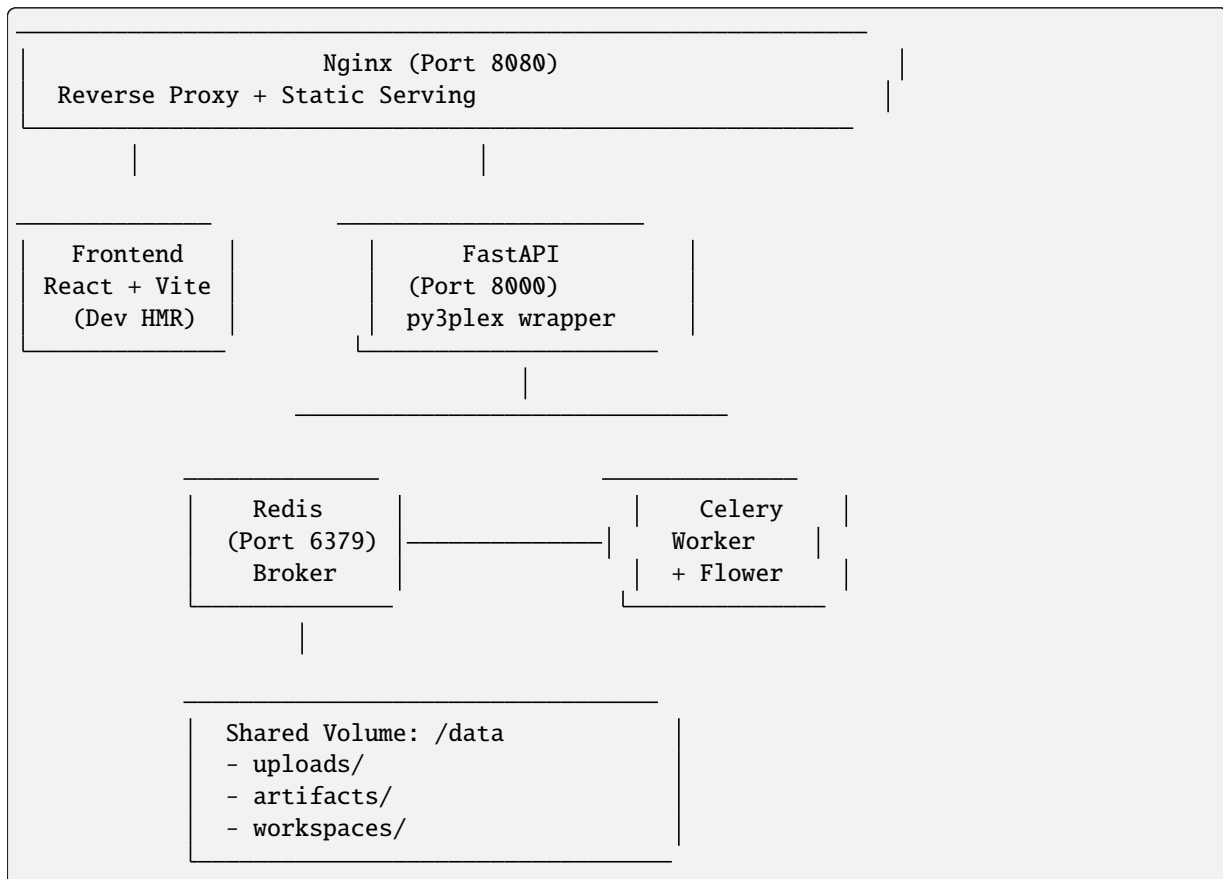
- React + TypeScript frontend with hot reload
- FastAPI backend with py3plex integration
- Celery workers for async jobs
- Redis broker

- Nginx reverse proxy

Prerequisites

- Docker (≥ 20.10)
- Docker Compose (≥ 2.0)
- 4GB RAM minimum
- Ports available: 8080 (GUI), 5555 (Flower), 8000 (API), 6379 (Redis)

Architecture



Key Components:

- **Frontend:** React 18 + Vite + TypeScript + Tailwind CSS
- **API:** FastAPI (Python 3.12) with py3plex integration
- **Worker:** Celery for long-running analysis jobs
- **Broker:** Redis for job queue
- **Proxy:** Nginx for unified access
- **Monitoring:** Flower dashboard for job inspection

Features

Data Loading

- Upload network files (.edgelist, .txt, .gml, .gpickle)
- Automatic format detection

- Multilayer network support
- Real-time file preview

Visualization

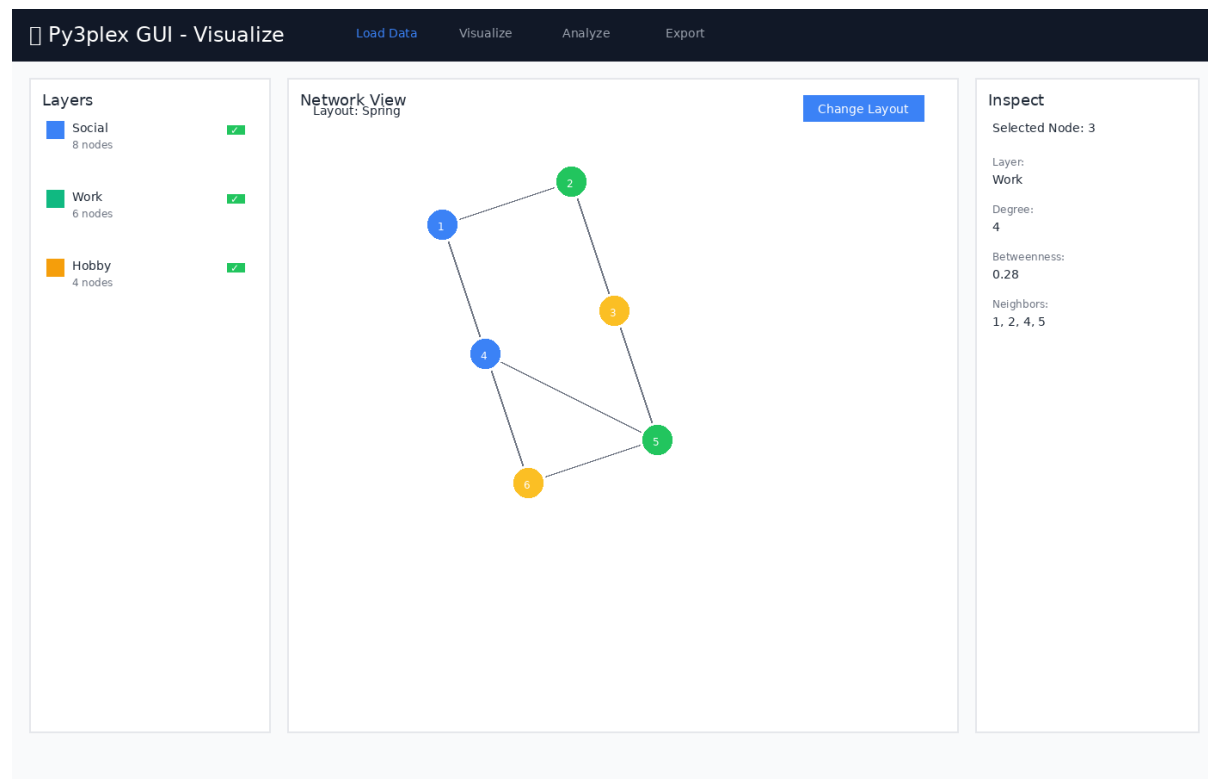


Figure 2: Network visualization with layer controls and node inspection panel

Features:

- Layer-centric view
- Interactive node/edge inspection
- Configurable layouts
- Position caching

Analysis (Async via Celery)

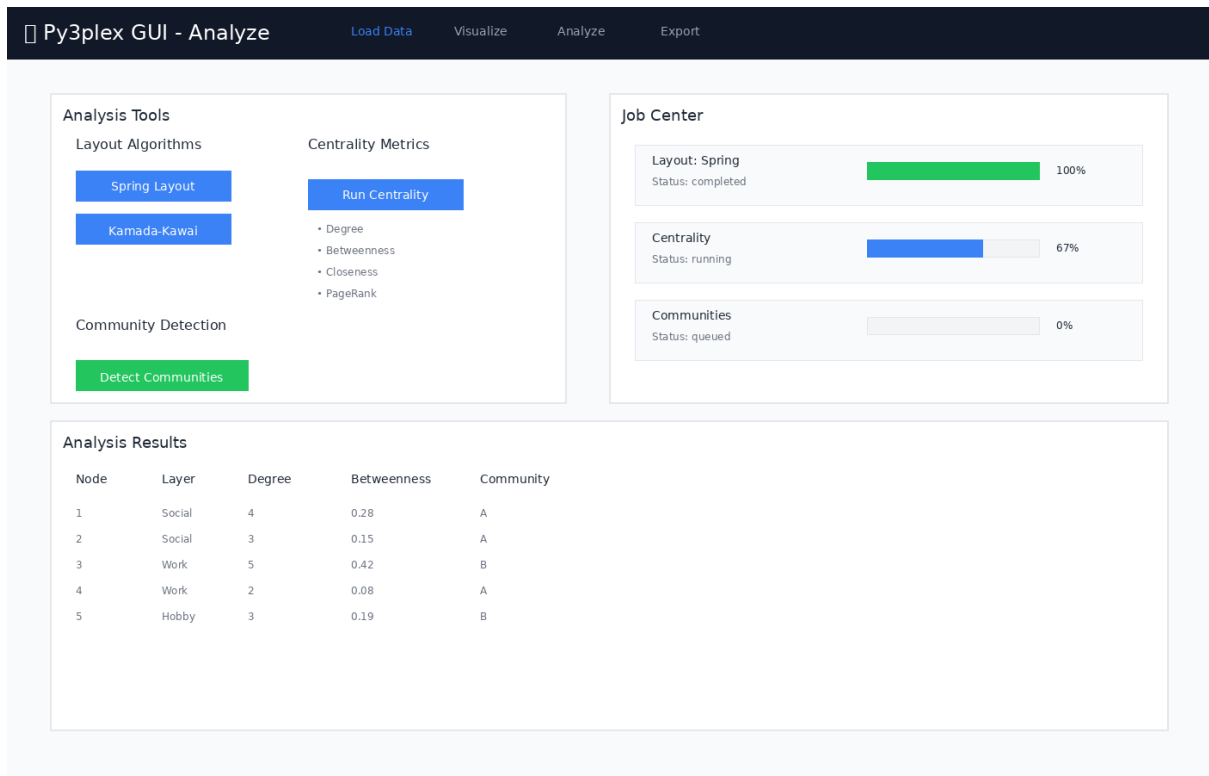


Figure 3: Analysis page with job controls and real-time progress tracking

Available Analysis Tools:

- **Layouts:** Spring, Kamada-Kawai, Circular, Random
- **Centrality:** Degree, Betweenness, Closeness, Eigenvector, PageRank
- **Community Detection:** Louvain, Label Propagation, Greedy Modularity
- Real-time progress tracking
- Result caching

Export

Py3plex GUI - Export

Load DataVisualizeAnalyzeExport

Export Options

Network Positions (JSON)
Export node positions from current layout

Export

Centrality Results (CSV)
Export computed centrality metrics

Export

Community Assignments (CSV)
Export community detection results

Export

Workspace Bundle (ZIP)
Save complete analysis session

Export

Workspace Management

Save Current Workspace

my-analysis-workspace

Save Workspace Bundle

Recent Workspaces

network-analysis-2024-11-23.zip

Load

multilayer-communities-v2.zip

Load

social-network-export.zip

Load

Export Information

- JSON exports contain node positions compatible with web visualizations
- CSV exports can be opened in Excel, R, or Python for further analysis
- Workspace bundles include the original network, all computed results, and UI state
- Workspace bundles can be shared with collaborators or loaded later
- All exports are saved to the data/artifacts/ directory

Figure 4: Export page for saving analysis results and workspace bundles

Export Options:

- CSV summaries (centrality, communities)
- JSON position data
- PNG snapshots (planned)
- Workspace bundles (data + state + results)

Workspace Bundles

Save and restore complete analysis sessions:

```
# Bundle includes:  
# - Original network file  
# - Computed layouts  
# - Centrality results  
# - Community assignments  
# - UI view state
```

Job Monitoring

The GUI includes Flower dashboard for real-time monitoring of background jobs:

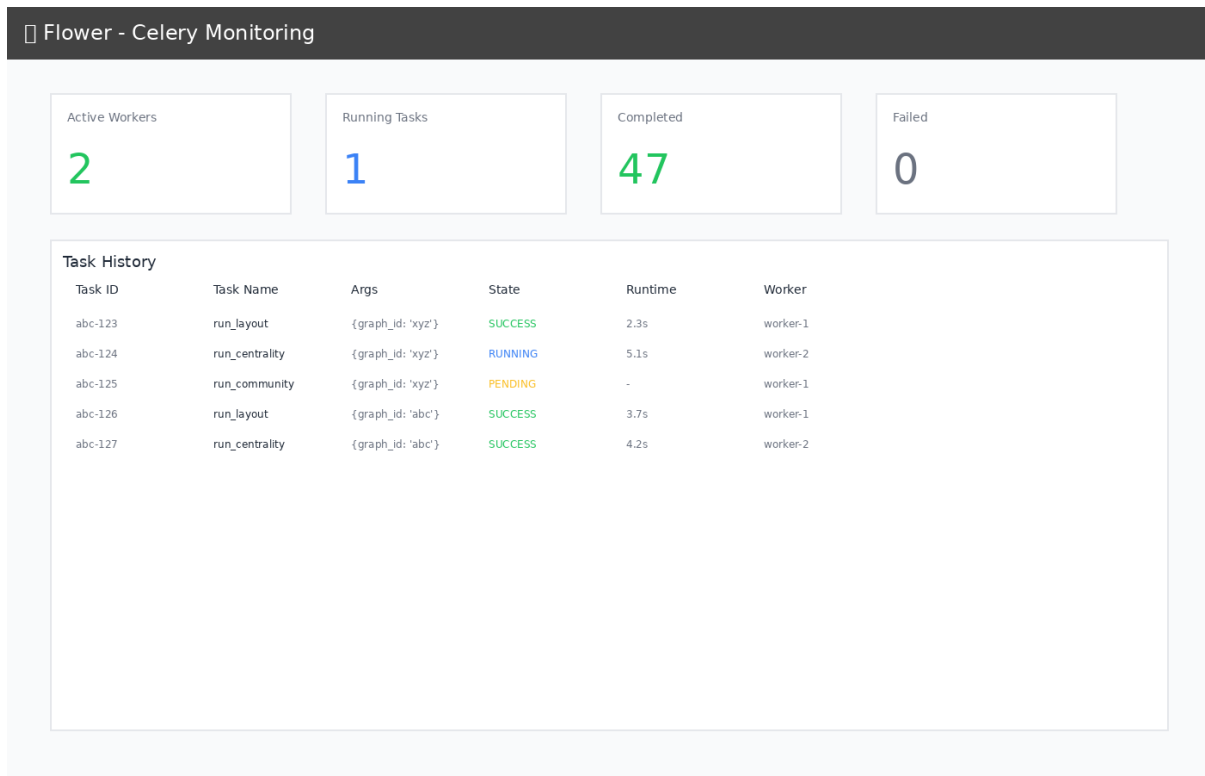


Figure 5: Flower dashboard showing task history and worker status

Monitoring Features:

- Track task execution in real-time
- View worker status and performance
- Inspect task history and results
- Monitor task queue and completion rates
- Access at <http://localhost:5555> when running

Development Guide

Project Structure

```
gui/
├── docker-compose.yml      # Main orchestration
├── compose.gpu.yml        # GPU override (optional)
├── Makefile               # Convenience commands
├── .env.example           # Configuration template
├── nginx/
│   └── nginx.conf         # Reverse proxy config
├── api/
│   ├── Dockerfile.api     # API container
│   ├── pyproject.toml     # Python dependencies
│   └── app/
│       ├── main.py        # FastAPI app
│       └── schemas.py     # Pydantic models
```

(continues on next page)

(continued from previous page)

```

├── deps.py          # DI & config
├── routes/          # API endpoints
├── services/        # Business logic (py3plex wrappers)
├── workers/         # Celery tasks
├── utils/           # Helpers
├── worker/
│   └── Dockerfile    # Worker container
├── frontend/
│   ├── Dockerfile.frontend # Frontend container
│   ├── package.json   # Node dependencies
│   ├── vite.config.ts  # Vite config
│   └── src/
│       ├── App.tsx     # Root component
│       ├── lib/api.ts  # API client
│       ├── pages/      # Page components
│       └── components/ # Reusable components
├── data/            # Shared volume (gitignored)
│   ├── uploads/
│   ├── artifacts/
│   └── workspaces/

```

Makefile Commands

```

make setup          # Copy .env.example to .env
make up             # Start all services (build if needed)
make down           # Stop and remove containers + volumes
make restart        # Restart all services
make build           # Rebuild Docker images
make logs           # Tail logs from all services

# Development helpers
make bash-api       # Shell into API container
make bash-worker    # Shell into worker container
make bash-frontend  # Shell into frontend container

# Testing
make test-api       # Run API tests
make e2e            # Run end-to-end tests (WIP)

# Cleanup
make clean          # Remove containers, volumes, and data

```

Local Development Against py3plex

The py3plex repository root is bind-mounted read-only into containers at `/workspace`:

```

# In Dockerfile.api
ARG PY3PLEX_PATH=/workspace
RUN pip install -e ${PY3PLEX_PATH}

```

Benefits:

- Changes to py3plex core reflect immediately (no rebuild)
- Editable install for development
- Isolated GUI code under `gui/`

Note: The mount is read-only to prevent accidental writes from containers.

Configuration

Environment Variables (.env)

```
# API
API_WORKERS=2           # Uvicorn workers
MAX_UPLOAD_MB=512       # Max file size

# Celery
CELERY_CONCURRENCY=2    # Worker threads
REDIS_URL=redis://redis:6379/0

# Frontend
VITE_API_URL=http://localhost:8080/api
```

GPU Support (Optional)

Enable NVIDIA GPU acceleration:

```
docker compose -f docker-compose.yml -f compose.gpu.yml up
```

Requirements:

- NVIDIA Docker runtime installed
- CUDA-compatible GPU

Data Formats

Accepted Input Formats

1. Edge List (.edgelist, .txt)

```
# Multilayer format: node1 node2 layer weight
1 2 social 1.0
1 3 work 1.0
2 3 hobby 0.5

# Simple format: node1 node2
A B
B C
C D
```

2. GML (.gml)

- Graph Modeling Language
- Preserves attributes and metadata

3. NetworkX Pickle (.gpickle)

- Native NetworkX serialization
- Fastest for large graphs

Example Dataset

Try the included toy network:

```
# From host
curl -F "file=@gui/toy_network.edgelist" http://localhost:8080/api/upload

# Or use the Web UI at http://localhost:8080
```

Testing

Continuous Integration

GUI tests run automatically on GitHub Actions for any changes to the `gui/` directory:

```
# Workflow: .github/workflows/gui-tests.yml
- API unit tests (pytest)
- Integration tests (Docker Compose)
- Frontend build verification
```

Status:

Tests include:

- Health endpoint validation
- File upload with toy network
- Layout job execution and completion
- Centrality computation
- Service health checks

API Tests

```
# Run inside API container
make bash-api
pytest ci/api-tests/ -v

# Or directly
docker compose exec api pytest ci/api-tests/ -v
```

Integration Tests

The CI workflow runs full integration tests:

```
# Upload and analyze a network
cd gui
curl -F "file=@toy_network.edgelist" http://localhost:8080/api/upload
# Run layout, centrality, and community detection jobs
```

Frontend Tests (WIP)

```
# Playwright smoke tests
make e2e
```

Manual Testing Workflow

1. **Upload** `toy_network.edgelist` via Web UI
2. **Visualize** the network (6 nodes, 14 edges, 3 layers)
3. **Analyze**:
 - Run Spring Layout
 - Compute Centrality (degree + betweenness)
 - Detect Communities (Louvain)
4. **Monitor** jobs at <http://localhost:5555> (Flower)
5. **Export** workspace bundle

Production Deployment

Static Build (Recommended)

For production, serve pre-built frontend assets:

```
# Modify Dockerfile.frontend to use multi-stage build
FROM node:20-alpine AS builder
WORKDIR /app
COPY frontend/ .
RUN npm install && npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx/nginx.conf /etc/nginx/nginx.conf
```

Update `docker-compose.yml`:

```
frontend:
  build:
    context: .
    dockerfile: Dockerfile.frontend.prod
  # Remove dev server volume mounts
```

Security Considerations

- Set CORS `allow_origins` to specific domains (not *)
- Add authentication/authorization middleware
- Use HTTPS (add TLS termination in Nginx)
- Validate file uploads strictly
- Set resource limits per user/job
- Enable Celery task rate limiting

Current Status: Development mode - suitable for local use only. Do not expose to public internet without proper security hardening.

Troubleshooting

Issue: Port already in use

```
# Find process using port 8080
lsof -ti:8080 | xargs kill -9

# Or change port in docker-compose.yml
ports:
  - "8081:80" # Use 8081 instead
```

Issue: Permission denied on /data

```
# Fix volume permissions
sudo chown -R $(whoami):$(whoami) gui/data/
```

Issue: py3plex import error in containers

```
# Verify mount
docker compose exec api ls -la /workspace

# Reinstall if needed
docker compose exec api pip install -e /workspace
```

Issue: Frontend can't reach API

Check Nginx proxy config:

```
docker compose exec nginx cat /etc/nginx/nginx.conf

# Test API directly
curl http://localhost:8000/api/health
```

API Documentation

Interactive docs available at:

- **Swagger UI:** <http://localhost:8080/api/docs>
- **ReDoc:** <http://localhost:8080/api/redoc>

Key Endpoints

POST	/api/upload	# Upload network file
GET	/api/graphs/{id}/summary	# Graph statistics
POST	/api/graphs/{id}/layout	# Compute layout (async)
POST	/api/graphs/{id}/analysis/centrality	# Compute centrality (async)
POST	/api/graphs/{id}/analysis/community	# Detect communities (async)
GET	/api/jobs/{id}	# Poll job status
POST	/api/workspaces/save	# Save workspace bundle

Example: Run Analysis

```
# 1. Upload
GRAPH_ID=$(curl -F "file=@toy_network.edgelist" \
  http://localhost:8080/api/upload | jq -r .graph_id)

# 2. Start layout job
JOB_ID=$(curl -X POST \
```

(continues on next page)

(continued from previous page)

```
-H "Content-Type: application/json" \  
-d '{"algorithm":"spring","seed":42,"dimensions":2}' \  
http://localhost:8080/api/graphs/$GRAPH_ID/layout | jq -r .job_id)  
  
# 3. Poll job  
curl http://localhost:8080/api/jobs/$JOB_ID  
  
# 4. Get positions  
curl http://localhost:8080/api/graphs/$GRAPH_ID/positions
```

Contributing

This GUI is part of the [py3plex](#) project.

Guidelines:

- Keep all GUI code under `gui/`
- Do not modify `py3plex` core
- Follow existing code style (Black, Ruff for Python; ESLint for TypeScript)
- Add tests for new features
- Update documentation with new features

License

This GUI follows the `py3plex` repository license:

- **GUI code** (under `gui/`): BSD-3-Clause (same as `py3plex`)
- **Dependencies**: See individual package licenses

Acknowledgments

Built with:

- [py3plex](#) - Multilayer network analysis
- [FastAPI](#) - Modern Python web framework
- [Celery](#) - Distributed task queue
- [React](#) - UI library
- [Vite](#) - Frontend build tool
- [Tailwind CSS](#) - Utility-first CSS

Support

- **Issues**: <https://github.com/SkBlaz/py3plex/issues>
- **Docs**: <https://skblaz.github.io/py3plex/>
- **py3plex Paper**: [Applied Network Science 2019](#)

8.5.3 GUI Deployment Guide

This guide covers deploying the `py3plex` web interface for production use.

Overview

The py3plex GUI can be deployed in several ways:

1. **Local Development** - Run directly on your machine
2. **Docker Container** - Isolated, reproducible environment
3. **Production Server** - Behind reverse proxy with TLS
4. **Cloud Deployment** - AWS, Azure, GCP, etc.

See *Py3plex GUI* for basic usage and *Py3plex GUI Architecture* for technical details.

Local Development Setup

Quick Start

```
# Clone repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Install with GUI dependencies
pip install -e ".[gui]"

# Start development server
python gui/app.py
```

The GUI will be available at `http://localhost:5000`.

Configuration

Create a configuration file `gui/config.py`:

```
# Development configuration
DEBUG = True
HOST = '0.0.0.0'
PORT = 5000
SECRET_KEY = 'dev-secret-key-change-in-production'

# Upload settings
UPLOAD_FOLDER = './uploads'
MAX_CONTENT_LENGTH = 100 * 1024 * 1024 # 100 MB max file size

# Allowed file extensions
ALLOWED_EXTENSIONS = {'txt', 'edgelist', 'graphml', 'gml', 'json', 'arrow'}
```

Docker Deployment

Basic Docker Setup

The repository includes a Dockerfile for the GUI:

```
# Build image
docker build -t py3plex-gui:latest -f gui/Dockerfile .

# Run container
docker run -d \
  -p 5000:5000 \
  -v $(pwd)/data:/app/data \
```

(continues on next page)

(continued from previous page)

```
--name py3plex-gui \
py3plex-gui:latest
```

The GUI will be available at `http://localhost:5000`.

Docker Compose

Create `docker-compose.yml`:

```
version: '3.8'

services:
  gui:
    build:
      context: .
      dockerfile: gui/Dockerfile
    ports:
      - "5000:5000"
    volumes:
      - ./data:/app/data
      - ./uploads:/app/uploads
    environment:
      - FLASK_ENV=production
      - SECRET_KEY=${SECRET_KEY}
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
```

Start with:

```
# Set secret key
export SECRET_KEY=$(python -c 'import secrets; print(secrets.token_hex(32))')

# Start services
docker-compose up -d

# Check logs
docker-compose logs -f gui
```

Environment Variables

Configure via environment variables:

Variable	Description	Default
FLASK_ENV	Environment (development/production)	development
SECRET_KEY	Session secret key (required)	None
HOST	Bind address	0.0.0.0
PORT	Bind port	5000
MAX_WORKERS	Worker processes	4
UPLOAD_FOLDER	Upload directory	./uploads
DATA_FOLDER	Data directory	./data

Production Deployment

Using Gunicorn

For production, use a production WSGI server like Gunicorn:

```
# Install Gunicorn
pip install gunicorn

# Run with Gunicorn
gunicorn \
  --bind 0.0.0.0:5000 \
  --workers 4 \
  --timeout 120 \
  --access-logfile - \
  --error-logfile - \
  gui.app:app
```

Supervisor Configuration

Use Supervisor to manage the GUI process:

Create `/etc/supervisor/conf.d/py3plex-gui.conf`:

```
[program:py3plex-gui]
command=/path/to/venv/bin/gunicorn --bind 127.0.0.1:5000 --workers 4 gui.app:app
directory=/path/to/py3plex
user=www-data
autostart=true
autorestart=true
redirect_stderr=true
stdout_logfile=/var/log/py3plex-gui.log
environment=FLASK_ENV="production",SECRET_KEY="your-secret-key"
```

Start with:

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start py3plex-gui
```

Systemd Service

Alternative: use systemd:

Create `/etc/systemd/system/py3plex-gui.service`:

```
[Unit]
Description=Py3plex GUI
After=network.target

[Service]
Type=notify
User=www-data
WorkingDirectory=/path/to/py3plex
Environment="FLASK_ENV=production"
Environment="SECRET_KEY=your-secret-key"
ExecStart=/path/to/venv/bin/gunicorn --bind 127.0.0.1:5000 --workers 4 gui.app:app
ExecReload=/bin/kill -s HUP $MAINPID
KillMode=mixed
```

(continues on next page)

(continued from previous page)

```
TimeoutStopSec=5
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable py3plex-gui
sudo systemctl start py3plex-gui
sudo systemctl status py3plex-gui
```

Reverse Proxy Setup

Nginx Configuration

Configure Nginx as a reverse proxy:

Create /etc/nginx/sites-available/py3plex-gui:

```
server {
    listen 80;
    server_name py3plex.yourdomain.com;

    # Redirect to HTTPS
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name py3plex.yourdomain.com;

    # SSL certificates (use Let's Encrypt)
    ssl_certificate /etc/letsencrypt/live/py3plex.yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/py3plex.yourdomain.com/privkey.pem;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;

    # Upload size limit
    client_max_body_size 100M;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Timeouts for long-running operations
        proxy_connect_timeout 300s;
        proxy_send_timeout 300s;
        proxy_read_timeout 300s;
    }
}
```

(continues on next page)

(continued from previous page)

```
# Static files (if serving separately)
location /static {
    alias /path/to/py3plex/gui/static;
    expires 30d;
    add_header Cache-Control "public, immutable";
}
}
```

Enable the site:

```
sudo ln -s /etc/nginx/sites-available/py3plex-gui /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

Apache Configuration

Alternative: use Apache as reverse proxy:

```
<VirtualHost *:80>
    ServerName py3plex.yourdomain.com
    Redirect permanent / https://py3plex.yourdomain.com/
</VirtualHost>

<VirtualHost *:443>
    ServerName py3plex.yourdomain.com

    SSLEngine on
    SSLCertificateFile /etc/letsencrypt/live/py3plex.yourdomain.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/py3plex.yourdomain.com/privkey.pem

    ProxyPreserveHost On
    ProxyPass / http://127.0.0.1:5000/
    ProxyPassReverse / http://127.0.0.1:5000/

    # Security headers
    Header always set X-Frame-Options "SAMEORIGIN"
    Header always set X-Content-Type-Options "nosniff"
</VirtualHost>
```

SSL/TLS with Let's Encrypt

```
# Install certbot
sudo apt-get install certbot python3-certbot-nginx

# Obtain certificate
sudo certbot --nginx -d py3plex.yourdomain.com

# Auto-renewal (certbot sets this up automatically)
sudo certbot renew --dry-run
```

Security Considerations

Secret Key Management

Never hardcode secret keys in code or configuration files:

```
# Generate strong secret key
python -c 'import secrets; print(secrets.token_hex(32))' > .secret_key

# Set in environment
export SECRET_KEY=$(cat .secret_key)

# Or use environment file
echo "SECRET_KEY=$(python -c 'import secrets; print(secrets.token_hex(32))')" > .env
```

File Upload Security

Configure upload restrictions:

```
# In config.py
ALLOWED_EXTENSIONS = {'txt', 'edgelist', 'graphml', 'gml', 'json', 'arrow'}
MAX_CONTENT_LENGTH = 100 * 1024 * 1024 # 100 MB

# Validate uploads
def allowed_file(filename):
    return '.' in filename and \
        filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
```

Rate Limiting

Implement rate limiting for API endpoints:

```
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(
    app,
    key_func=get_remote_address,
    default_limits=["200 per day", "50 per hour"]
)

@app.route("/api/upload")
@limiter.limit("10 per hour")
def upload():
    pass
```

Authentication

Add user authentication for production:

```
from flask_login import LoginManager, login_required

login_manager = LoginManager()
login_manager.init_app(app)

@app.route("/dashboard")
@login_required
```

(continues on next page)

(continued from previous page)

```
def dashboard():  
    pass
```

Monitoring and Logging

Application Logging

Configure structured logging:

```
import logging  
from logging.handlers import RotatingFileHandler  
  
# Configure logging  
handler = RotatingFileHandler(  
    'logs/py3plex-gui.log',  
    maxBytes=10 * 1024 * 1024, # 10 MB  
    backupCount=5  
)  
handler.setFormatter(logging.Formatter(  
    '%(asctime)s %(levelname)s in %(module)s: %(message)s'  
)  
)  
app.logger.addHandler(handler)  
app.logger.setLevel(logging.INFO)
```

Prometheus Metrics

Export metrics for Prometheus:

```
from prometheus_flask_exporter import PrometheusMetrics  
  
metrics = PrometheusMetrics(app)  
  
# Custom metrics  
metrics.info('app_info', 'Application info', version='1.0')
```

Health Checks

Implement health check endpoint:

```
@app.route('/health')  
def health():  
    # Check dependencies  
    try:  
        # Test database connection, file system, etc.  
        return {'status': 'healthy'}, 200  
    except Exception as e:  
        return {'status': 'unhealthy', 'error': str(e)}, 503
```

Cloud Deployment Examples

AWS EC2

1. Launch EC2 instance (Ubuntu 22.04 LTS)
2. Install dependencies
3. Clone repository

4. Follow production deployment steps above
5. Configure security groups (ports 80, 443)

AWS ECS (Docker)

```
{
  "family": "py3plex-gui",
  "containerDefinitions": [{
    "name": "gui",
    "image": "your-registry/py3plex-gui:latest",
    "memory": 2048,
    "cpu": 1024,
    "essential": true,
    "portMappings": [{
      "containerPort": 5000,
      "protocol": "tcp"
    }],
    "environment": [
      {"name": "FLASK_ENV", "value": "production"}
    ],
    "secrets": [
      {"name": "SECRET_KEY", "valueFrom": "arn:aws:secretsmanager:..."}
    ]
  }]
}
```

Google Cloud Run

```
# Build and push to GCR
gcloud builds submit --tag gcr.io/PROJECT_ID/py3plex-gui

# Deploy
gcloud run deploy py3plex-gui \
  --image gcr.io/PROJECT_ID/py3plex-gui \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --set-env-vars FLASK_ENV=production \
  --set-secrets SECRET_KEY=py3plex-secret:latest
```

Azure Container Instances

```
# Create container group
az container create \
  --resource-group py3plex-rg \
  --name py3plex-gui \
  --image your-registry.azurecr.io/py3plex-gui:latest \
  --dns-name-label py3plex-gui \
  --ports 5000 \
  --environment-variables FLASK_ENV=production \
  --secure-environment-variables SECRET_KEY=$SECRET_KEY
```

Performance Tuning

Worker Configuration

```
# Rule of thumb: (2 x CPU cores) + 1
workers=$((2 * $(nproc) + 1))

unicorn \
  --workers $workers \
  --threads 2 \
  --worker-class gthread \
  --timeout 120 \
  --bind 0.0.0.0:5000 \
  gui.app:app
```

Caching

Implement caching for expensive operations:

```
from flask_caching import Cache

cache = Cache(app, config={'CACHE_TYPE': 'simple'})

@app.route('/api/stats/<network_id>')
@cache.cached(timeout=300)
def network_stats(network_id):
    # Expensive computation
    return stats
```

Database for Persistence

For production, consider a database:

```
from flask_sqlalchemy import SQLAlchemy

app.config['SQLALCHEMY_DATABASE_URI'] = 'postgresql://user:pass@localhost/py3plex'
db = SQLAlchemy(app)
```

Backup and Recovery

Automated Backups

```
#!/bin/bash
# backup.sh

BACKUP_DIR="/backups/py3plex"
DATE=$(date +%Y%m%d_%H%M%S)

# Backup uploads
tar -czf "$BACKUP_DIR/uploads_$DATE.tar.gz" /path/to/uploads

# Backup database (if using one)
# pg_dump py3plex > "$BACKUP_DIR/db_$DATE.sql"

# Cleanup old backups (keep last 7 days)
find "$BACKUP_DIR" -type f -mtime +7 -delete
```

Add to crontab:

```
# Daily backup at 2 AM
0 2 * * * /path/to/backup.sh
```

Disaster Recovery

Document recovery procedure:

1. Restore from backup
2. Verify data integrity
3. Restart services
4. Test functionality

Troubleshooting

Common Issues

Port already in use:

```
# Find process using port
sudo lsof -i :5000

# Kill process
sudo kill -9 PID
```

Permission denied:

```
# Fix upload directory permissions
sudo chown -R www-data:www-data /path/to/uploads
```

Out of memory:

```
# Check memory usage
free -h

# Adjust worker count or add swap
```

Logs Location

- Application logs: `/var/log/py3plex-gui.log`
- Nginx logs: `/var/log/nginx/`
- Systemd logs: `journalctl -u py3plex-gui`

Related Documentation

- *Py3plex GUI* - Using the GUI
- *Py3plex GUI Architecture* - Technical architecture
- *GUI API Reference* - API endpoints
- *GUI Testing Guide* - Testing the GUI
- *Docker Usage Guide* - Docker details

Security Resources:

- OWASP Flask Security: <https://owasp.org/www-project-web-security-testing-guide/>
- Flask Security Best Practices: <https://flask.palletsprojects.com/en/2.3.x/security/>

8.5.4 GUI API Reference

This reference documents the REST API endpoints provided by the py3plex web interface.

Overview

The py3plex GUI provides a REST API for:

- Uploading network data
- Running analyses
- Generating visualizations
- Exporting results

Base URL: `http://localhost:5000/api` (development)

Authentication: Currently no authentication (add in production)

Response Format: JSON

General Endpoints

Health Check

Check if the server is running.

Endpoint: GET `/health`

Response:

```
{
  "status": "healthy",
  "version": "0.95",
  "timestamp": "2025-01-23T12:00:00Z"
}
```

Status Codes:

- 200 - Server is healthy
- 503 - Server is unhealthy

Server Info

Get server information.

Endpoint: GET `/api/info`

Response:

```
{
  "py3plex_version": "0.95",
  "python_version": "3.10.0",
  "available_algorithms": ["louvain", "infomap", "node2vec"],
  "max_upload_size": 104857600
}
```

Network Management

Upload Network

Upload a network file for analysis.

Endpoint: POST `/api/upload`

Parameters:

- `file` (form-data, required) - Network file
- `input_type` (form-data, required) - File format (edgelist, multiedgelist, graphml, etc.)
- `directed` (form-data, optional) - Boolean, default false

Request Example:

```
curl -X POST http://localhost:5000/api/upload \
-F "file=@network.edgelist" \
-F "input_type=edgelist" \
-F "directed=false"
```

Response:

```
{
  "success": true,
  "network_id": "abc123",
  "num_nodes": 100,
  "num_edges": 450,
  "num_layers": 3,
  "message": "Network uploaded successfully"
}
```

Status Codes:

- 200 - Success
- 400 - Invalid file or parameters
- 413 - File too large
- 500 - Server error

List Networks

List all uploaded networks.

Endpoint: GET `/api/networks`

Response:

```
{
  "networks": [
    {
      "id": "abc123",
      "filename": "network.edgelist",
      "uploaded_at": "2025-01-23T12:00:00Z",
      "num_nodes": 100,
      "num_edges": 450
    }
  ]
}
```

Get Network Info

Get detailed information about a specific network.

Endpoint: GET `/api/networks/<network_id>`

Response:

```
{
  "id": "abc123",
  "filename": "network.edgelist",
  "num_nodes": 100,
  "num_edges": 450,
  "num_layers": 3,
  "layers": ["layer1", "layer2", "layer3"],
  "statistics": {
    "density": 0.045,
    "clustering": 0.32
  }
}
```

Delete Network

Delete an uploaded network.

Endpoint: DELETE /api/networks/<network_id>

Response:

```
{
  "success": true,
  "message": "Network deleted successfully"
}
```

Analysis Endpoints

Run Community Detection

Detect communities in a network.

Endpoint: POST /api/analysis/communities

Parameters:

- network_id (required) - Network ID
- algorithm (required) - Algorithm name (louvain, leiden, infomap)
- gamma (optional) - Resolution parameter (default: 1.0)
- omega (optional) - Inter-layer coupling (default: 1.0)

Request:

```
{
  "network_id": "abc123",
  "algorithm": "louvain",
  "gamma": 1.0,
  "omega": 1.0
}
```

Response:

```
{
  "success": true,
  "num_communities": 5,
  "modularity": 0.42,
  "partition": {
    "node1": 0,
```

(continues on next page)

(continued from previous page)

```

    "node2": 0,
    "node3": 1,
    ...
  }
}

```

Compute Statistics

Compute network statistics.

Endpoint: POST /api/analysis/statistics

Parameters:

- network_id (required) - Network ID
- metrics (required) - List of metrics to compute

Request:

```

{
  "network_id": "abc123",
  "metrics": ["density", "clustering", "degree_distribution"]
}

```

Response:

```

{
  "success": true,
  "statistics": {
    "density": 0.045,
    "clustering": 0.32,
    "degree_distribution": {
      "1": 10,
      "2": 25,
      "3": 30,
      ...
    }
  }
}

```

Compute Centrality

Compute node centrality measures.

Endpoint: POST /api/analysis/centrality

Parameters:

- network_id (required) - Network ID
- measure (required) - Centrality measure (degree, betweenness, pagerank, eigenvector)
- layer (optional) - Specific layer to analyze

Request:

```

{
  "network_id": "abc123",
  "measure": "betweenness",
  "layer": "layer1"
}

```

Response:

```
{
  "success": true,
  "centrality": {
    "node1": 0.42,
    "node2": 0.35,
    "node3": 0.15,
    "...": ...
  }
}
```

Generate Embeddings

Generate node embeddings.

Endpoint: POST /api/analysis/embeddings

Parameters:

- network_id (required) - Network ID
- method (required) - Embedding method (node2vec, deepwalk)
- dimensions (optional) - Embedding dimensions (default: 128)
- walk_length (optional) - Walk length (default: 80)
- num_walks (optional) - Walks per node (default: 10)
- p (optional) - Return parameter (default: 1.0)
- q (optional) - In-out parameter (default: 1.0)

Request:

```
{
  "network_id": "abc123",
  "method": "node2vec",
  "dimensions": 128,
  "walk_length": 80,
  "num_walks": 10,
  "p": 1.0,
  "q": 1.0
}
```

Response:

```
{
  "success": true,
  "embedding_id": "emb456",
  "dimensions": 128,
  "num_nodes": 100
}
```

Visualization Endpoints

Generate Layout

Compute network layout.

Endpoint: POST /api/visualization/layout

Parameters:

- `network_id` (required) - Network ID
- `algorithm` (required) - Layout algorithm (force, spring, circular, diagonal)
- `iterations` (optional) - Number of iterations (default: 50)

Request:

```
{
  "network_id": "abc123",
  "algorithm": "force",
  "iterations": 100
}
```

Response:

```
{
  "success": true,
  "layout_id": "layout789",
  "positions": {
    "node1": [0.5, 0.3],
    "node2": [0.6, 0.8],
    "...": ...
  }
}
```

Render Visualization

Generate network visualization image.

Endpoint: POST `/api/visualization/render`

Parameters:

- `network_id` (required) - Network ID
- `layout_id` (optional) - Pre-computed layout ID
- `width` (optional) - Image width (default: 800)
- `height` (optional) - Image height (default: 600)
- `format` (optional) - Output format (png, svg, pdf, default: png)

Request:

```
{
  "network_id": "abc123",
  "layout_id": "layout789",
  "width": 1200,
  "height": 900,
  "format": "png"
}
```

Response:

Returns image file with appropriate Content-Type header.

Status Codes:

- 200 - Success (returns image)
- 400 - Invalid parameters
- 404 - Network or layout not found

Export Endpoints

Export Network

Export network in different formats.

Endpoint: GET /api/export/network/<network_id>

Query Parameters:

- format (required) - Export format (graphml, gml, edgelist, json, arrow)

Example:

```
curl http://localhost:5000/api/export/network/abc123?format=graphml \
-o output.graphml
```

Response:

Returns file in requested format.

Export Analysis Results

Export analysis results as JSON or CSV.

Endpoint: GET /api/export/analysis/<analysis_id>

Query Parameters:

- format (optional) - Export format (json, csv, default: json)

Example:

```
curl http://localhost:5000/api/export/analysis/analysis123?format=csv \
-o results.csv
```

Response:

Returns file in requested format.

Error Handling

Error Response Format

All errors return a consistent JSON format:

```
{
  "success": false,
  "error": "Error message description",
  "error_code": "ERROR_CODE",
  "details": {
    "field": "Additional error details"
  }
}
```

Common Error Codes

HTTP Status	Error Code	Description
400	INVALID_INPUT	Invalid request parameters
400	INVALID_FILE	Invalid or corrupted file
401	UNAUTHORIZED	Authentication required
404	NOT_FOUND	Resource not found
413	FILE_TOO_LARGE	Upload exceeds size limit
500	INTERNAL_ERROR	Server error
503	SERVICE_UNAVAILABLE	Service temporarily unavailable

Rate Limits

Default rate limits (configurable):

- 200 requests per day per IP
- 50 requests per hour per IP
- 10 uploads per hour per IP

Rate limit headers in responses:

```
X-RateLimit-Limit: 50
X-RateLimit-Remaining: 45
X-RateLimit-Reset: 1674480000
```

Usage Examples

Python Client

```
import requests

BASE_URL = "http://localhost:5000/api"

# Upload network
with open("network.edgelist", "rb") as f:
    response = requests.post(
        f"{BASE_URL}/upload",
        files={"file": f},
        data={"input_type": "edgelist", "directed": "false"}
    )

network_id = response.json()["network_id"]
print(f"Uploaded network: {network_id}")

# Run community detection
response = requests.post(
    f"{BASE_URL}/analysis/communities",
    json={
        "network_id": network_id,
        "algorithm": "louvain"
    }
)
```

(continues on next page)

(continued from previous page)

```
communities = response.json()
print(f"Found {communities['num_communities']} communities")
```

cURL Examples

```
# Upload network
curl -X POST http://localhost:5000/api/upload \
  -F "file=@network.edgelist" \
  -F "input_type=edgelist" \
  -F "directed=false"

# Get network info
curl http://localhost:5000/api/networks/abc123

# Run analysis
curl -X POST http://localhost:5000/api/analysis/communities \
  -H "Content-Type: application/json" \
  -d '{"network_id":"abc123","algorithm":"louvain"}'

# Export network
curl http://localhost:5000/api/export/network/abc123?format=graphml \
  -o output.graphml
```

JavaScript/Fetch Example

```
const BASE_URL = 'http://localhost:5000/api';

// Upload network
const formData = new FormData();
formData.append('file', fileInput.files[0]);
formData.append('input_type', 'edgelist');
formData.append('directed', 'false');

const response = await fetch(`${BASE_URL}/upload`, {
  method: 'POST',
  body: formData
});

const data = await response.json();
console.log(`Uploaded network: ${data.network_id}`);

// Run analysis
const analysisResponse = await fetch(`${BASE_URL}/analysis/communities`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    network_id: data.network_id,
    algorithm: 'louvain'
  })
});

const communities = await analysisResponse.json();
console.log(`Found ${communities.num_communities} communities`);
```

Related Documentation

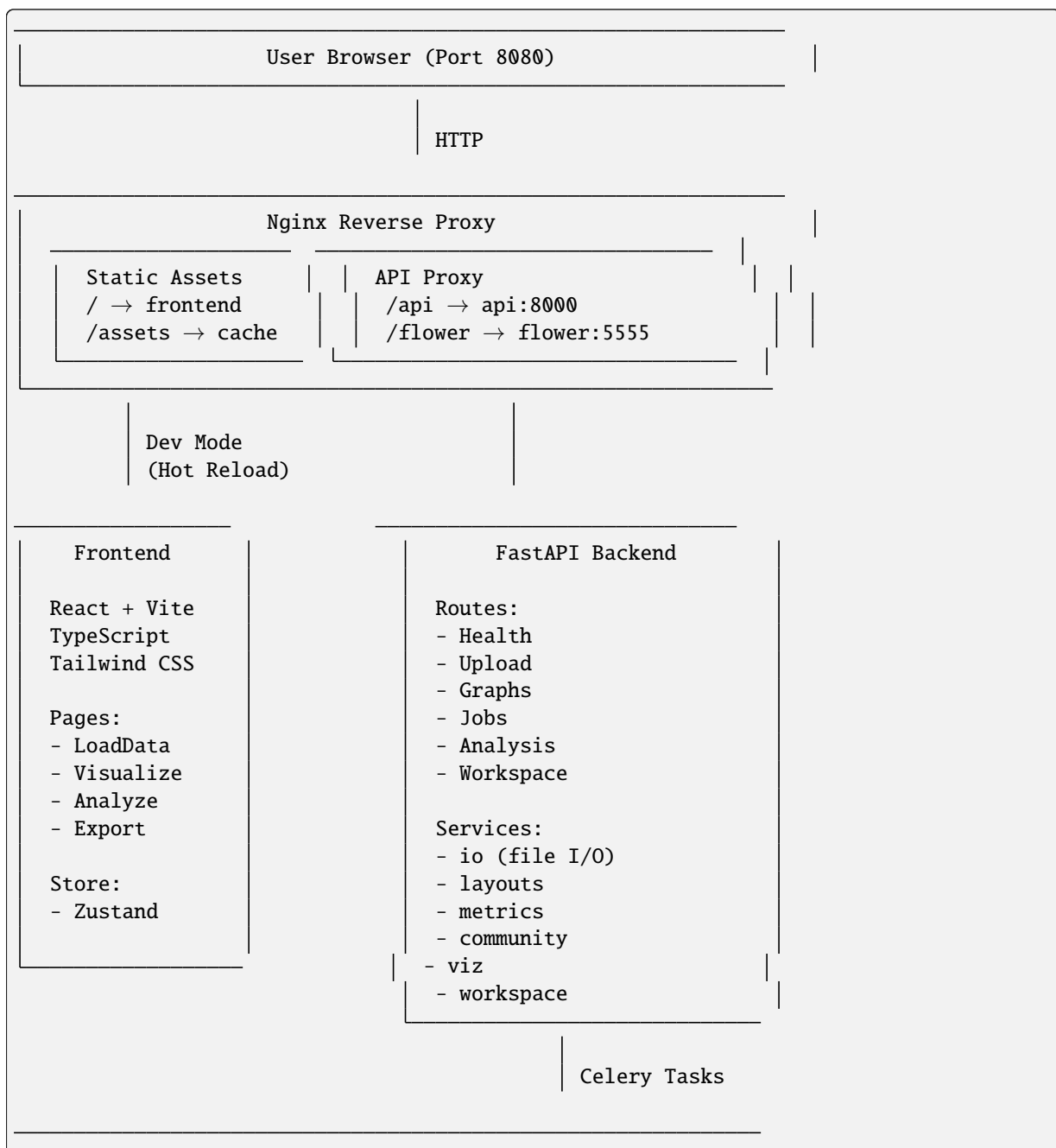
- *Py3plex GUI* - Using the web interface
- *GUI Deployment Guide* - Deploying in production
- *Py3plex GUI Architecture* - Technical architecture
- *GUI Testing Guide* - Testing the API

External Resources:

- REST API Best Practices: <https://restfulapi.net/>
- Flask-RESTful Documentation: <https://flask-restful.readthedocs.io/>

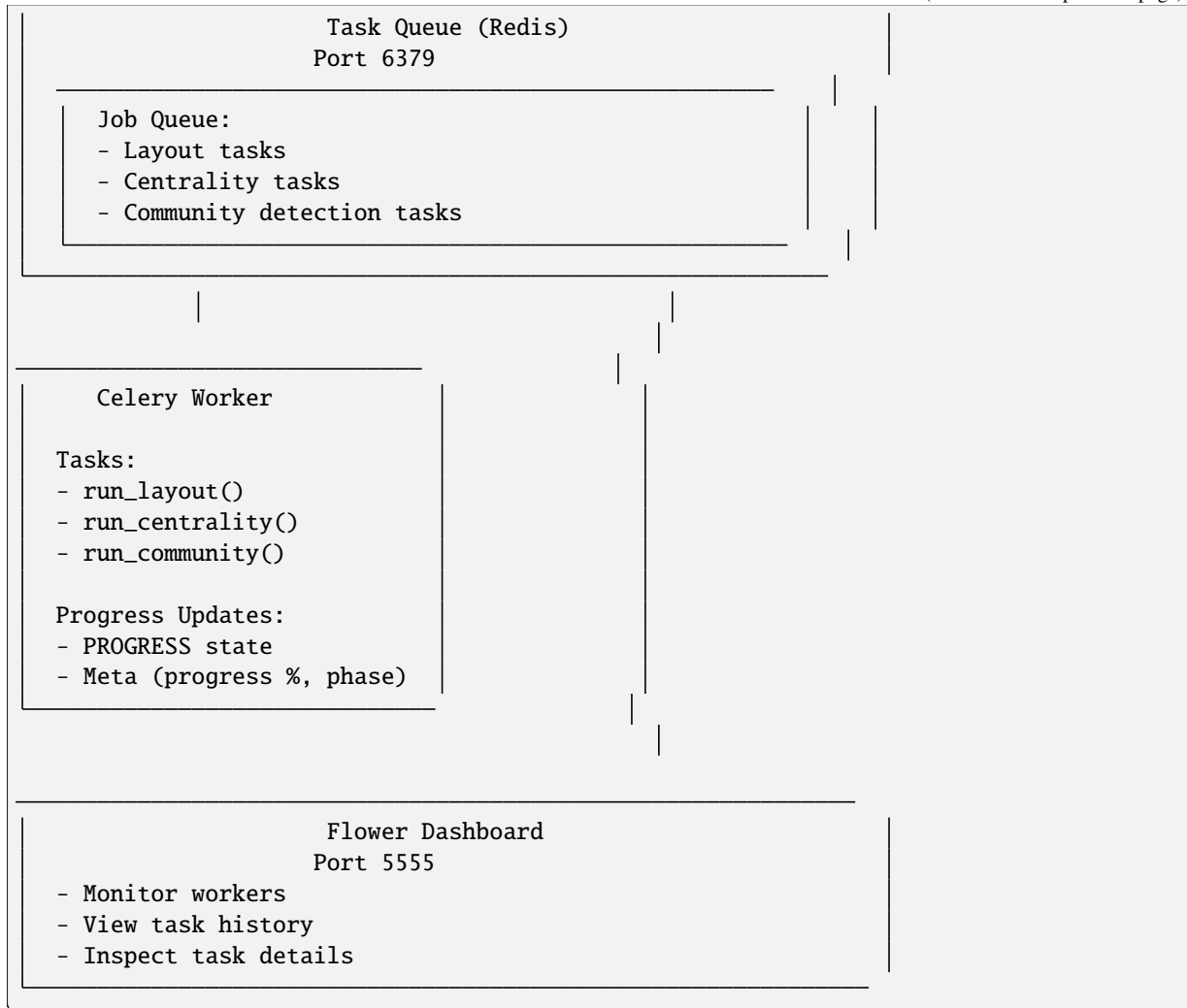
8.5.5 Py3plex GUI Architecture

System Overview



(continues on next page)

(continued from previous page)



Data Flow

Upload & Parse Flow

```

User → Frontend → API /upload → io.save_upload()
                                   ↓
                               io.load_graph_from_file()
                                   ↓
                             NetworkX graph loaded
                                   ↓
                             Stored in GRAPH_REGISTRY
                                   ↓
                             Return graph_id to frontend
  
```

Analysis Job Flow

```

User → Frontend → API /analysis/centrality
                                   ↓
                             Create Celery task
                                   ↓
                             Return job_id
                                   ↓
  
```

(continues on next page)

(continued from previous page)

```

Task queued in Redis
↓
Worker picks up task
↓
task.update_state(progress=30)
↓
services.metrics.compute Centrality()
↓
Save results to /data/artifacts/
↓
task.update_state(progress=100, result={...})
↓
User ← Frontend ← API /jobs/{job_id} ← Redis result backend

```

Directory Structure

```

gui/
├── docker-compose.yml      # Orchestration
├── compose.gpu.yml        # GPU override
├── Makefile                # Dev commands
├── .env.example            # Config template
├── README                  # User guide
├── TESTING                 # Test guide
├── ARCHITECTURE            # Architecture notes
├── nginx/
│   └── nginx.conf          # Reverse proxy config
├── api/                    # Backend
│   ├── Dockerfile.api
│   ├── pyproject.toml
│   └── app/
│       ├── main.py         # FastAPI app
│       ├── deps.py         # Dependencies
│       ├── schemas.py      # Pydantic models
│       ├── routes/         # Endpoints
│       │   ├── health.py
│       │   ├── upload.py
│       │   ├── graphs.py
│       │   ├── jobs.py
│       │   ├── analysis.py
│       │   └── workspace.py
│       ├── services/       # Business logic
│       │   ├── io.py       # File I/O
│       │   ├── layouts.py  # Layout algorithms
│       │   ├── metrics.py  # Centrality metrics
│       │   ├── community.py # Community detection
│       │   ├── viz.py      # Visualization data
│       │   ├── model.py    # Graph queries
│       │   └── workspace.py # Save/load
│       ├── workers/        # Celery
│       │   ├── celery_app.py
│       │   └── tasks.py
│       └── utils/
│           └── logging.py

```

(continues on next page)

(continued from previous page)



Component Responsibilities

Visual Component Reference

The architecture components are realized in the following user interface pages:

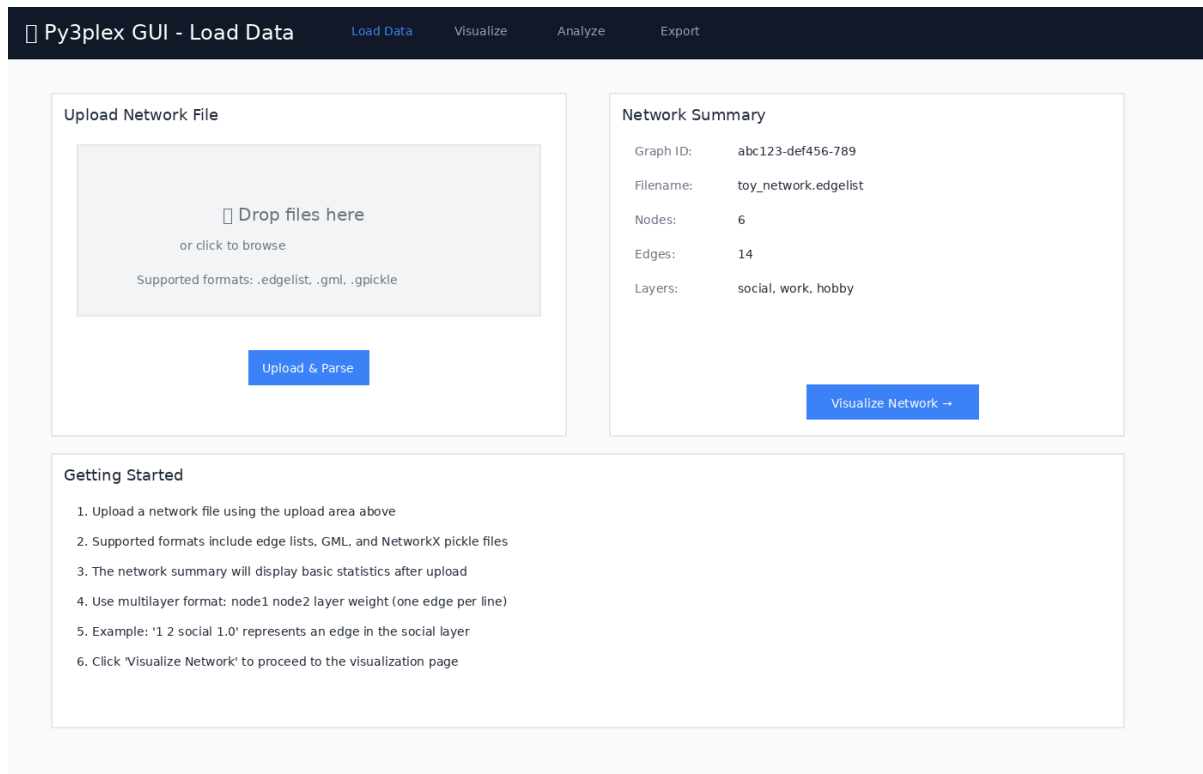


Figure 1: Frontend - Load Data page utilizing the IO service

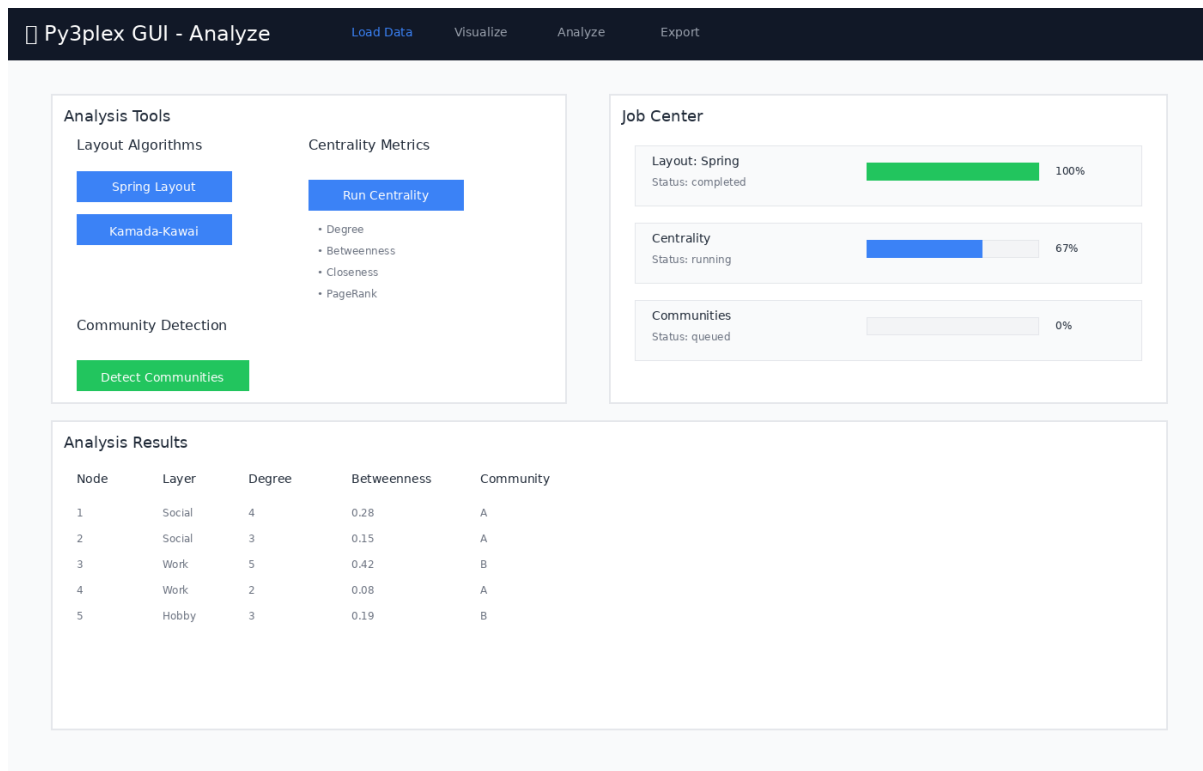


Figure 2: Job orchestration - Analysis page with Celery task execution

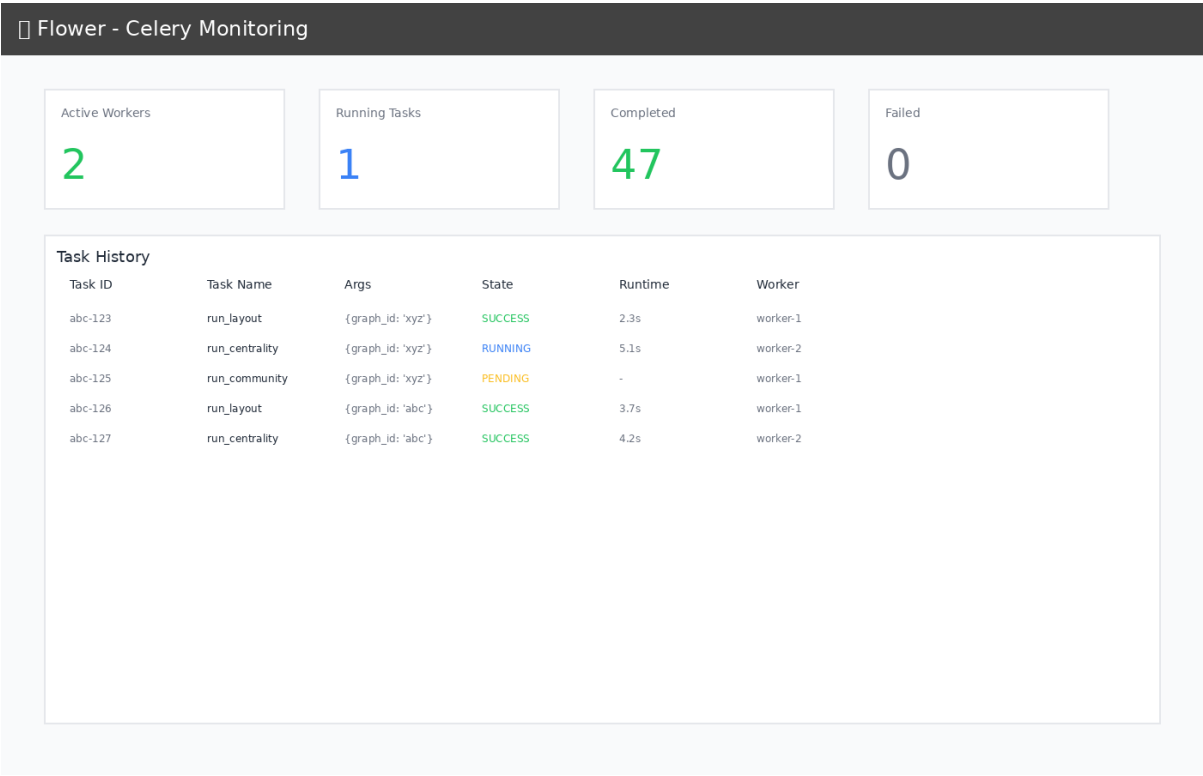


Figure 3: Worker monitoring - Flower dashboard showing task queue and execution

Frontend

Responsibilities:

- User interaction
- File upload UI
- Real-time job polling
- Graph visualization (placeholder)
- State management (Zustand)

Technologies:

- React 18 (UI framework)
- Vite (build tool, dev server)
- TypeScript (type safety)
- Tailwind CSS (styling)
- Axios (HTTP client)

API (FastAPI)

Responsibilities:

- REST API endpoints
- Request validation (Pydantic)
- File upload handling
- Job orchestration
- py3plex integration

Key Services:

- `io`: File loading, format detection
- `layouts`: Layout computation (NetworkX)
- `metrics`: Centrality calculations
- `community`: Community detection
- `workspace`: Save/load bundles

Worker (Celery)

Responsibilities:

- Async job execution
- Progress reporting
- Result persistence
- Resource management

Tasks:

- `run_layout`: Force-directed layouts
- `run_centrality`: Node/edge metrics
- `run_community`: Community detection

Redis

Responsibilities:

- Job queue (broker)
- Result backend
- Session storage (future)

Nginx

Responsibilities:

- Reverse proxy
- Static file serving
- Gzip compression
- Caching headers
- WebSocket proxy (HMR)

Flower

Responsibilities:

- Worker monitoring
- Task history
- Performance metrics

Data Models

Graph Registry (In-Memory)

```
GRAPH_REGISTRY = {
    "<graph_id>": {
        "graph": nx.Graph(),          # NetworkX graph
        "filepath": "/data/uploads/...", # Original file
        "positions": [NodePosition()],  # Layout positions
        "metadata": {}                 # Extra info
    }
}
```

Job State (Redis)

```
{
    "job_id": "uuid",
    "status": "running", # queued/running/completed/failed
    "progress": 50,      # 0-100
    "phase": "computing", # human-readable
    "result": {...}      # Output data
}
```

Workspace Bundle (Zip)

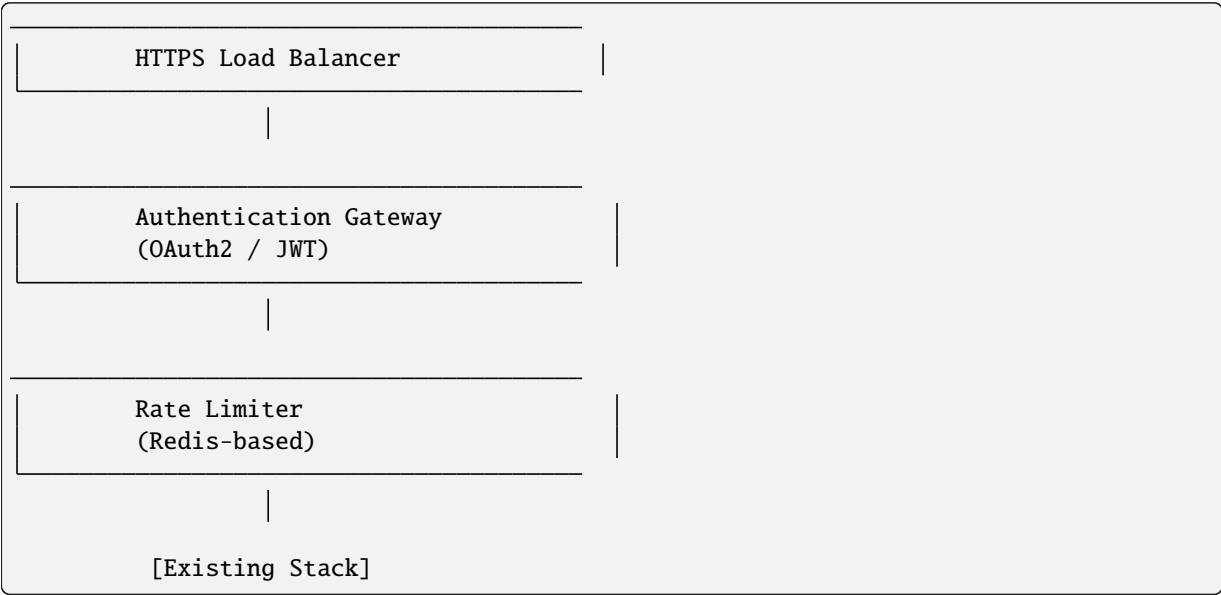
```
workspace_{uuid}.zip
├─ metadata.json      # Graph ID, view state
├─ network.edgelist   # Original file
├─ positions.json     # Layout positions
├─ artifacts/
│   └─ centrality.json
│   └─ community.json
```

Security Architecture

Current (Development Mode)

- ✓ Read-only py3plex mount
- ✓ Isolated data directories
- CORS allows all origins
- No authentication
- No HTTPS
- No rate limiting

Production Hardening (TODO)



Deployment Variants

Local Development (Current)

```
make up # All containers on localhost
```

GPU-Enabled

```
docker compose -f docker-compose.yml -f compose.gpu.yml up
```

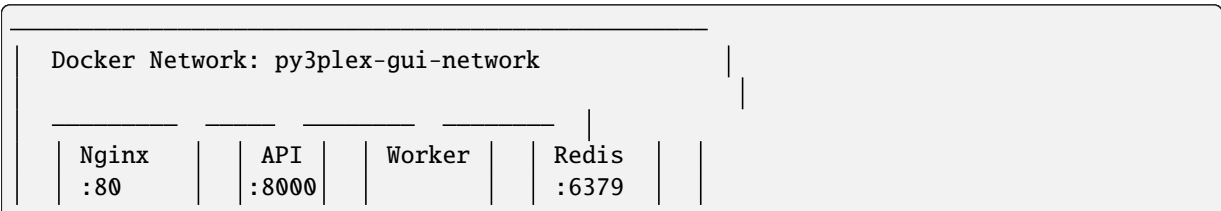
Production (Future)

```
# docker-compose.prod.yml
services:
  frontend:
    image: frontend:prod
    # Pre-built static files

  api:
    replicas: 3
    # Load balanced

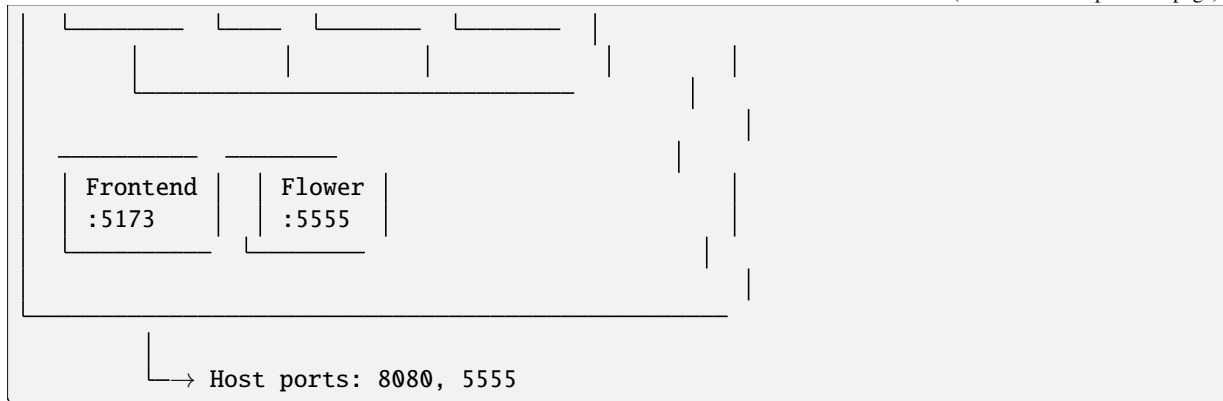
  worker:
    replicas: 5
    # Auto-scaling
```

Network Topology



(continues on next page)

(continued from previous page)



Volume Mounts

Host	→ Container
../	→ /workspace (ro)
../data/	→ /data
../api/app/	→ /app (dev mode)
../frontend/src/	→ /app/src (dev mode)

Environment Variables

```
# API
API_WORKERS=2           # Uvicorn workers
MAX_UPLOAD_MB=512       # Max file size
DATA_DIR=/data          # Data root

# Celery
CELERY_CONCURRENCY=2     # Worker threads
REDIS_URL=redis://redis:6379/0

# Frontend
VITE_API_URL=http://localhost:8080/api
```

Performance Characteristics

Small Graphs (< 100 nodes)

- Upload: < 1s
- Layout: 2-5s
- Centrality: 1-3s
- Community: 1-3s

Medium Graphs (100-1000 nodes)

- Upload: 1-3s
- Layout: 5-15s
- Centrality: 3-10s
- Community: 3-10s

Large Graphs (> 1000 nodes)

- Consider sampling for preview
- Progressive rendering recommended
- May need GPU acceleration
- Memory: ~1GB per 10k nodes

Future Enhancements

Phase 2

- WebGL visualization
- Real-time collaboration
- Database backend (PostgreSQL)
- Authentication service
- CI/CD pipeline

Phase 3

- GraphQL API
- Plugin system
- Custom algorithms
- Cloud deployment
- Multi-tenancy

Version: 0.1.0

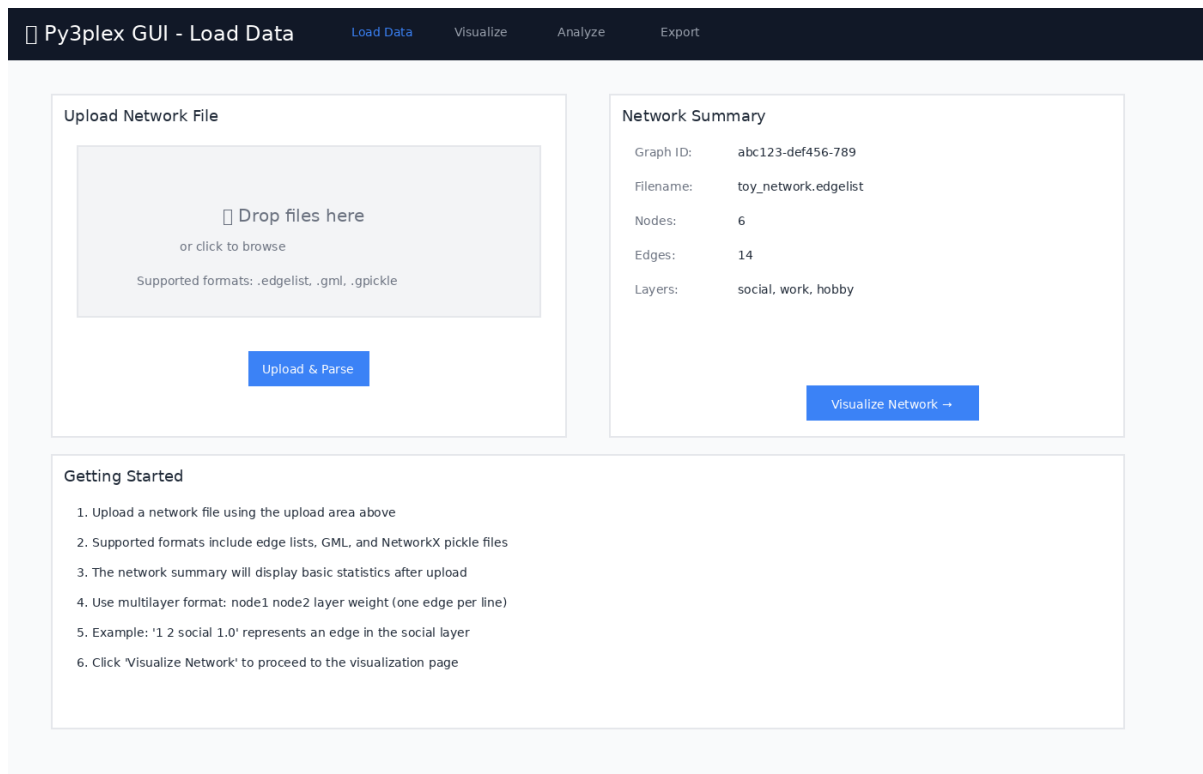
Last Updated: 2025-11-09

8.5.6 GUI Testing Guide

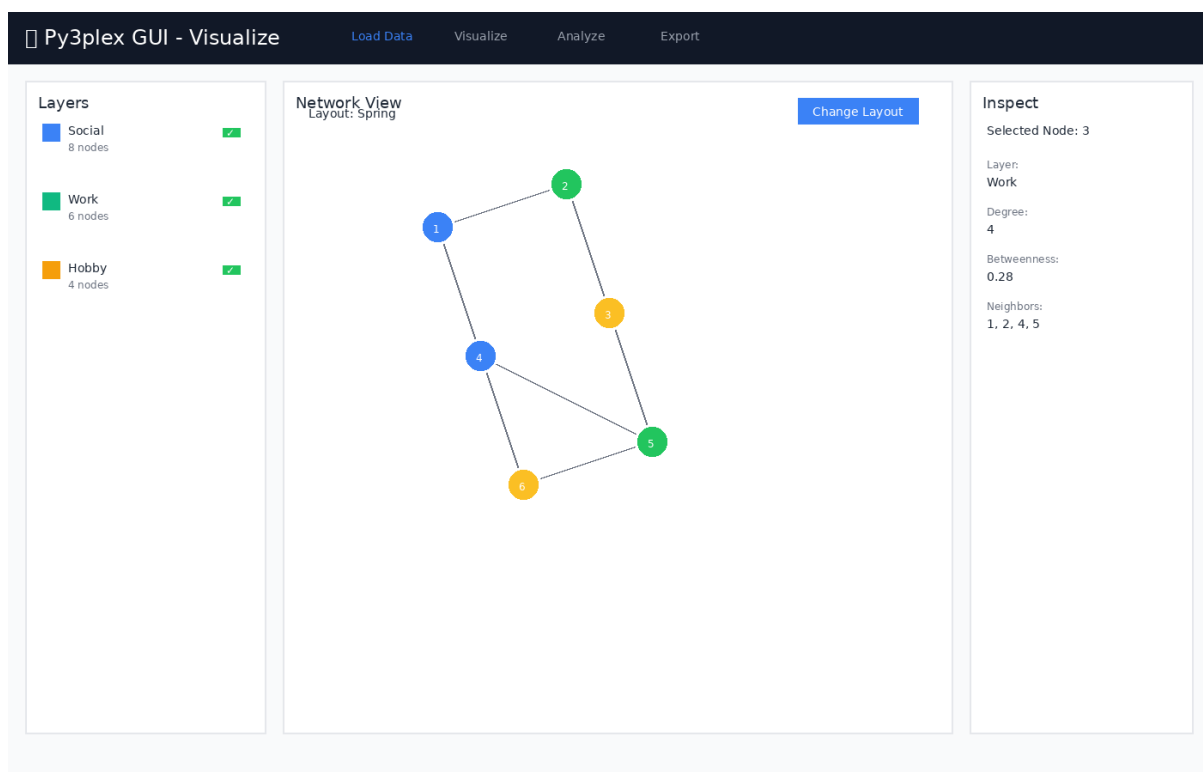
Complete testing guide for the Py3plex GUI, including automated CI tests and manual validation.

Visual Interface Preview

The Py3plex GUI consists of several key pages that work together for network analysis:



Load Data Page - Upload and parse network files



Visualize Page - Interactive network view with layer controls

Py3plex GUI - Analyze Load Data Visualize Analyze Export

Analysis Tools

Layout Algorithms

Spring Layout

Kamada-Kawai

Centrality Metrics

Run Centrality

- Degree
- Betweenness
- Closeness
- PageRank

Community Detection

Detect Communities

Job Center

Layout: Spring
Status: completed 100%

Centrality
Status: running 67%

Communities
Status: queued 0%

Analysis Results

Node	Layer	Degree	Betweenness	Community
1	Social	4	0.28	A
2	Social	3	0.15	A
3	Work	5	0.42	B
4	Work	2	0.08	A
5	Hobby	3	0.19	B

Analyze Page - Run jobs and monitor progress

Automated Testing (CI)

GitHub Actions Workflow

All GUI tests run automatically on GitHub Actions:

Workflow: `.github/workflows/gui-tests.yml`

Triggers:

- Push to main/master/develop branches (if `gui/` files changed)
- Pull requests targeting main/master/develop (if `gui/` files changed)
- Manual workflow dispatch

Test Jobs:

1. **API Tests** (~5 min)
 - Build API, worker, redis containers
 - Run pytest suite: `ci/api-tests/`
 - Validate health endpoint
 - Test file upload functionality

2. Integration Tests (~10 min)

- Build all services (including nginx, frontend)
- Test full upload → analyze → export flow
- Validate layout job execution
- Test centrality computation
- Verify service health

3. Frontend Build (~5 min)

- Type check TypeScript
- Build production bundle
- Verify dist output

View Results: Check the Actions tab on GitHub or the CI badges in the repository README

Running CI Tests Locally

You can run the same tests locally:

```
cd gui

# API tests
docker compose build api worker redis
docker compose up -d api worker redis
docker compose exec api pytest ci/api-tests/ -v
docker compose down -v

# Integration tests
docker compose up -d --build
# Wait for services
curl -f http://localhost:8080/api/health
# Run manual integration tests (see below)
docker compose down -v

# Frontend build
cd frontend
npm ci
npm run build
```

Manual Testing Guide

Manual testing guide for the Py3plex GUI. Follow these steps to validate the implementation.

Prerequisites

- Docker and Docker Compose installed
- At least 4GB RAM available
- Ports 8080, 5555, 8000, 6379 available

Setup

```
cd gui
cp .env.example .env
make up
```

Expected: All containers start successfully. Check with `docker compose ps`.

Test 1: Health Check

API Health

```
curl http://localhost:8080/api/health
```

Expected Response:

```
{"status": "ok", "version": "0.1.0"}
```

Frontend Access

Open browser to `http://localhost:8080`

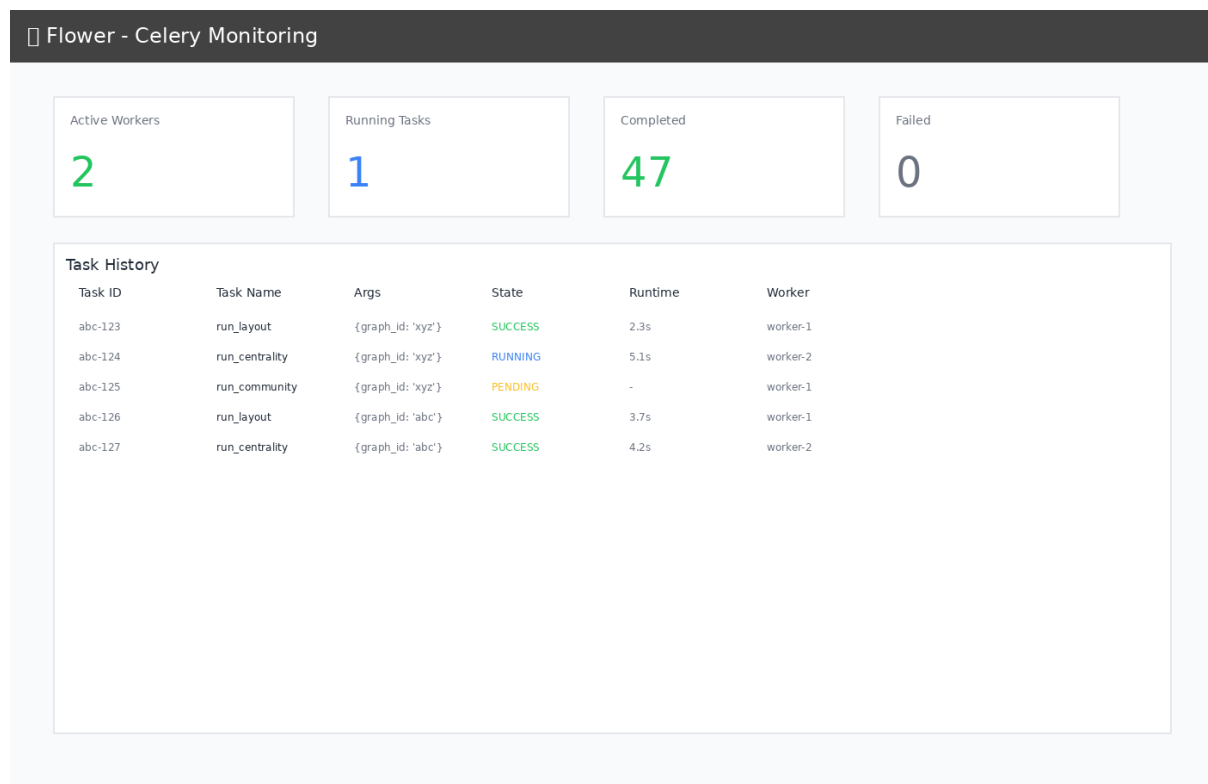
Expected: React app loads with navigation bar showing:

- Load Data
- Visualize
- Analyze
- Export

Flower Dashboard

Open browser to `http://localhost:5555`

Expected: Celery Flower dashboard loads showing workers.



Flower monitoring dashboard showing task history and worker status

Test 2: Upload Network

1. Navigate to **Load Data** page
2. Click “Click to upload” or drag and drop `toy_network.edgelist`
3. Click “Upload & Parse”

Expected:

- Upload progress indicator appears
- Network Summary displays:
 - Graph ID (UUID)
 - Filename: `toy_network.edgelist`
 - Nodes: 6
 - Edges: 14
 - Layers: hobby, social, work

Test 3: Visualize Network

1. Click “Visualize Network →” or navigate to **Visualize** page

Expected:

- Page shows three panels: Layers, Network View, Inspect
- Network View shows placeholder with “Graph with X nodes”
- No errors in console

Test 4: Run Layout Job

1. Navigate to **Analyze** page
2. Click “Run Spring Layout” button

Expected:

- Job appears in Job Center below
- Status shows “queued” → “running” → “completed”
- Progress updates (10% → 30% → 80% → 100%)
- Job shows green checkmark when complete

Verify in Flower

1. Open `http://localhost:5555`
2. Check “Tasks” tab

Expected: Layout task appears with status SUCCESS

Test 5: Run Centrality Analysis

1. On **Analyze** page, click “Run Centrality” button

Expected:

- New job appears in Job Center
- Type: “Centrality”
- Status progresses to “completed”

- No errors

Test 6: Run Community Detection

1. On **Analyze** page, click “Detect Communities” button

Expected:

- New job appears in Job Center
- Type: “Community Detection”
- Status progresses to “completed”
- Result includes number of communities found

Test 7: Export Workspace

1. Navigate to **Export** page
2. Enter workspace name: “test-workspace”
3. Click “Save Workspace Bundle”

Expected:

- Success message: “Workspace saved as test-workspace_{uuid}.zip”
- Workspace ID displayed

Verify File Created

```
ls -lh gui/data/workspaces/
```

Expected: Zip file exists with recent timestamp

Test 8: API Direct Testing

Upload via API

```
curl -F "file=@toy_network.edgelist" \
http://localhost:8080/api/upload
```

Expected: JSON response with graph_id

Get Graph Summary

```
GRAPH_ID=<from_previous_step>
curl http://localhost:8080/api/graphs/$GRAPH_ID/summary
```

Expected: JSON with nodes, edges, layers

Start Layout Job

```
curl -X POST \
-H "Content-Type: application/json" \
-d '{"algorithm":"spring","seed":42,"dimensions":2}' \
http://localhost:8080/api/graphs/$GRAPH_ID/layout
```

Expected: JSON response with job_id

Check Job Status

```
JOB_ID=<from_previous_step>
curl http://localhost:8080/api/jobs/$JOB_ID
```

Expected: JSON with status, progress, result

Test 9: Logs Inspection

```
# View all logs
make logs

# Or individual services
docker compose logs api
docker compose logs worker
docker compose logs frontend
```

Expected: No error messages, only INFO level logs

Test 10: Container Health

```
docker compose ps
```

Expected: All services show “healthy” or “running”

Test 11: Data Persistence

1. Upload a network
2. Run a layout job
3. Stop containers: `make down`
4. Restart: `make up`
5. Upload same network again

Expected:

- Uploads work after restart
- Previous artifacts still in `gui/data/`

Test 12: Multiple Jobs

1. Navigate to **Analyze** page
2. Quickly click:
 - Run Spring Layout
 - Run Centrality
 - Detect Communities

Expected:

- All three jobs appear in Job Center
- Jobs process in parallel (check Flower)
- All complete successfully

Common Issues

Port Already in Use

```
# Find and kill process
lsof -ti:8080 | xargs kill -9

# Or change port in docker-compose.yml
ports:
  - "8081:80"
```

Container Won't Start

```
# Check logs
docker compose logs <service>

# Rebuild
make down && make build && make up
```

Permission Errors

```
# Fix data directory permissions
sudo chown -R $(whoami):$(whoami) gui/data/
```

Frontend Can't Reach API

Check nginx config:

```
docker compose exec nginx cat /etc/nginx/nginx.conf
```

Test API directly:

```
curl http://localhost:8000/api/health
```

Cleanup

```
# Stop and remove everything
make down

# Full cleanup including data
make clean
```

Test Checklist

- [] Health endpoints respond
- [] Frontend loads
- [] Flower dashboard accessible
- [] File upload works
- [] Graph summary displays
- [] Layout job completes
- [] Centrality job completes
- [] Community detection works

- ☐ Workspace save succeeds
- ☐ Job progress updates in real-time
- ☐ Multiple concurrent jobs work
- ☐ Logs show no errors
- ☐ Data persists across restarts

Performance Benchmarks

For reference, on a typical development machine:

- **Startup time:** 30-60 seconds (first build: 3-5 minutes)
- **Upload (toy network):** < 1 second
- **Layout job:** 2-5 seconds
- **Centrality job:** 1-3 seconds
- **Community detection:** 1-3 seconds

Larger networks (1000+ nodes) may take proportionally longer.

Security Testing (Development Mode)

WARNING: Do NOT expose to public internet without:

- ☐ Changing CORS settings
- ☐ Adding authentication
- ☐ Enabling HTTPS
- ☐ Setting rate limits
- ☐ Validating file uploads strictly

Next Steps

If all tests pass:

1. Try with real network datasets
2. Test with large graphs (1000+ nodes)
3. Load test with concurrent users
4. Profile memory usage
5. Test edge cases (malformed files, very large files)

Last Updated: 2025-11-09

Version: 0.1.0

8.6 Part VI: Developer & Contributor Docs

If you're interested in contributing to py3plex or understanding its internal architecture for extension purposes, this section provides the necessary guidance. We cover development environment setup, code architecture and organization, repository layout, and contribution guidelines. Open-source contributions are welcomed and appreciated.

8.6.1 Developer & Contributor Docs: Building Together

“If you want to go fast, go alone. If you want to go far, go together.” — African Proverb

Welcome, Developer

You’re here because you want to go deeper. Maybe you’ve found a bug and want to fix it. Maybe you have an idea for a new feature. Maybe you’re curious about how py3plex works under the hood. Whatever brought you here, welcome.

Open-source software is built by communities. py3plex exists because researchers and developers contributed their time, ideas, and code. This section helps you become part of that community.

What You’ll Find Here

Development Guide covers the practical aspects of contributing: how to set up a development environment, how to run tests, how to submit changes. If you’re making your first contribution, start here.

Code Architecture explains how py3plex is organized internally. Understanding the architecture helps you navigate the codebase and make changes that fit with existing patterns.

Repository Layout provides a map of the codebase. Where do the core data structures live? Where are the algorithms? Where are the tests? This chapter answers these questions.

Contributing Guidelines describes the social and process aspects of contribution: how to report bugs, how to propose features, what to expect during code review.

The Value of Contributing

Contributing to open source is rewarding in many ways:

For your skills: Working on production-quality code with other developers improves your abilities faster than working alone.

For your career: Open-source contributions demonstrate competence to potential employers and collaborators in ways that private code cannot.

For your research: If you implement an algorithm for your research, contributing it to py3plex makes it accessible to the community and increases the impact of your work.

For the community: Every contribution, from fixing a typo to adding a major feature, makes py3plex better for everyone who uses it.

Types of Contributions

Contributions come in many forms:

- **Bug reports:** Found something that doesn’t work right? Report it. Clear bug reports are valuable contributions.
- **Documentation improvements:** Noticed an unclear explanation? Fixed a typo? Documentation quality directly affects user experience.
- **Code fixes:** Fixed a bug? Improved performance? Code contributions are the heart of open source.
- **New features:** Implemented a new algorithm? Added a new visualization? New features expand what py3plex can do.
- **Examples and tutorials:** Created a tutorial for your use case? Others will benefit from your work.
- **Community support:** Helping others in issues and discussions is a valuable contribution that builds community.

Getting Started

If you're new to contributing, we recommend:

1. Read the *Development Guide* to set up your environment
2. Browse the GitHub issues for items labeled “good first issue”
3. Start with something small—a documentation fix or a simple bug
4. Ask questions when you're stuck—the community is friendly

A Note on Expectations

We want contributing to be a positive experience. When you submit a contribution:

- Expect constructive feedback during code review
- Expect questions about design decisions—this is how we maintain quality
- Expect iteration—it's rare for contributions to be merged without changes
- Expect gratitude—we appreciate everyone who takes the time to contribute

Ready to Contribute?

Start with *Development Guide* to set up your environment, or browse the *Contributing to py3plex* guidelines to understand the process.

Note

First-Time Contributor?

If this is your first time contributing to an open-source project:

- Don't worry about getting everything perfect—we're here to help
- Start small—a typo fix is a perfectly valid first contribution
- Ask questions early—it's better to clarify expectations than to waste time
- Be patient—maintainers are often volunteers with limited time

Welcome to the community!

8.6.2 Development Guide

This guide covers **development workflows**, **Makefile commands**, **testing**, and **contributing** to py3plex.

Getting Started

Clone the repository and install in **development mode**:

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Setup development environment
make setup

# Install package in editable mode with dev dependencies
make dev-install
```

Makefile Commands

The project includes a **production-grade Makefile** for streamlined development:

```
# View all available commands
make help

# Environment setup
make setup          # Create virtual environment
make dev-install    # Install in editable mode

# Code quality
make format         # Auto-format code
make lint           # Run linters

# Testing
make test           # Run tests with coverage
make coverage       # Open coverage report

# Documentation
make docs           # Build Sphinx documentation

# Build & publish
make clean          # Remove build artifacts
make build          # Build distributions
make publish        # Publish to PyPI

# CI
make ci             # Run full CI checks
```

Key Features:

- **Smart tool detection:** Uses `.venv/bin/` tools if available, otherwise falls back to **global tools**
- **Colorized output:** Clear **success/warning/error** messages
- **Cross-platform:** Works on **Linux** and **macOS**

Testing

Quick testing:

```
python run_tests.py
```

Development testing with pytest:

```
# Run all tests
pytest

# Run with coverage
pytest --cov=py3plex --cov-report=html

# Run specific test file
pytest tests/test_core.py

# Using Makefile (recommended)
make test
```

Code Quality

Tools configured in `pyproject.toml`:

- **Black**: Code formatting
- **Ruff**: Fast linting
- **isort**: Import sorting
- **Mypy**: Type checking
- **Pytest**: Testing with coverage

Before committing:

```
make format  # Auto-format code
make lint    # Check code quality
make test    # Run tests
make ci      # Run all checks
```

Contributing

We welcome contributions! Here's how:

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Run tests and linting: `make ci`
5. Commit with clear messages
6. Push to your fork
7. Open a Pull Request

Code Guidelines:

- Follow existing code style (enforced by Black and Ruff)
- Add tests for new features
- Update documentation as needed
- Keep changes focused and atomic
- Write clear commit messages

Documentation

Build Sphinx documentation:

```
# Using Makefile (recommended)
make docs

# Manual build
cd docfiles
sphinx-build -b html . _build/html
```

The built documentation will be in `docfiles/_build/html/`.

Resources

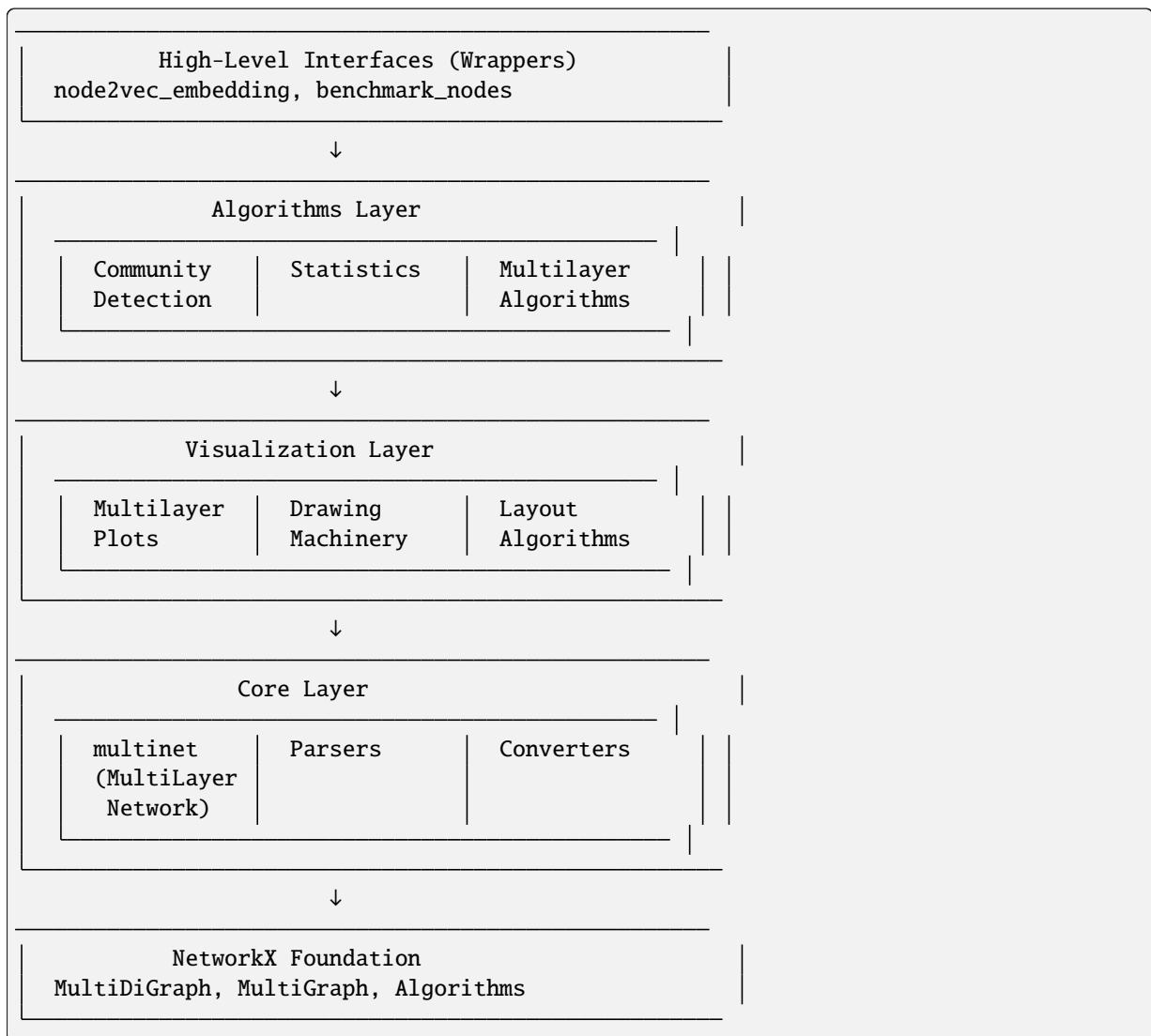
- **Repository:** <https://github.com/SkBlaz/py3plex>
- **Documentation:** <https://skblaz.github.io/py3plex/>
- **Issues:** <https://github.com/SkBlaz/py3plex/issues>
- **Examples:** <https://github.com/SkBlaz/Py3Plex/tree/master/examples>

8.6.3 Architecture and Design

This document describes the **system architecture**, **design patterns**, and **extension points** of py3plex.

System Overview

py3plex is built as a **modular, layered architecture** with clear separation of concerns:



Architectural Layers

Core Layer

Purpose: Fundamental data structures and I/O operations

Key Components:

- `multinet.py` - The `multi_layer_network` class

- `parsers.py` - **Input/output** for various formats
- `converters.py` - **Format conversion** utilities
- `random_generators.py` - **Random network** generators
- `HINMINE/` - **Heterogeneous network decomposition**

Responsibilities:

- **Network construction** and manipulation
- **File I/O** (GraphML, GML, GEXF, edge lists, etc.)
- **Layer management**
- **Matrix representations** (adjacency, supra-adjacency)
- **NetworkX integration**

Design Pattern: Facade Pattern - `multi_layer_network` provides a unified interface to complex NetworkX operations

Algorithms Layer

Purpose: Network analysis algorithms optimized for multilayer networks

Key Components:

- `community_detection/` - **Community detection** algorithms
- `statistics/` - **Network statistics** and metrics
- `multilayer_algorithms/` - **Multilayer-specific** algorithms
- `node_ranking/` - **Centrality** and ranking measures
- `general/` - **General-purpose** algorithms (random walks, etc.)

Responsibilities:

- **Community detection** (Louvain, Infomap, Label Propagation)
- **Statistical analysis** (17+ multilayer metrics)
- **Centrality computation** (degree, betweenness, PageRank, etc.)
- **Random walks** and embeddings
- **Network decomposition**

Design Pattern: Strategy Pattern - Different algorithms implement common interfaces

Visualization Layer

Purpose: Network plotting and rendering

Key Components:

- `multilayer.py` - High-level multilayer plotting
- `drawing_machinery.py` - Core drawing primitives
- `layout_algorithms.py` - Layout computation
- `colors.py` - Color scheme generators
- `fa2/` - ForceAtlas2 layout

Responsibilities:

- Diagonal projection plots
- Force-directed layouts

- Matrix visualizations
- Color mapping and legends
- Interactive plots (via Plotly)

Design Pattern: Template Method Pattern - Layout algorithms follow a common template

Wrappers Layer

Purpose: High-level interfaces for common workflows

Key Components:

- `node2vec_embedding.py` - Node2Vec embedding generation
- `benchmark_nodes.py` - Node classification benchmarking

Responsibilities:

- Simplified interfaces for complex workflows
- Integration with external tools
- Benchmarking and evaluation

Design Pattern: Facade Pattern - Simplify complex multi-step operations

Core Data Structure

The `multi_layer_network` Class

Central to py3plex, this class manages multilayer network state:

```
class multi_layer_network:
    def __init__(self, directed=True, label_delimiter="---", coupling_weight=1.0):
        self.core_network = nx.MultiDiGraph() if directed else nx.MultiGraph()
        self.layer_name_map = {} # Bidirectional mapping
        self.label_delimiter = label_delimiter
        self.coupling_weight = coupling_weight
        self.embedding = None
        self.labels = None
```

Key Attributes:

- `core_network` - Underlying NetworkX graph
- `layer_name_map` - Maps layer names to integer IDs
- `label_delimiter` - Separator for node-layer encoding (default: “---”)
- `coupling_weight` - Default weight for inter-layer edges
- `embedding` - Cached node embedding matrix
- `labels` - Node classification labels

Encoding Scheme:

Nodes are encoded as "{node_id}{delimiter}{layer_id}"

Example: Node ‘A’ in layer ‘social’ becomes "A---social"

This allows NetworkX to handle multilayer structure transparently.

Design Patterns

Facade Pattern

Used in: multi_layer_network, wrappers

Purpose: Provide simplified interface to complex subsystems

```
# Complex underlying operations hidden behind simple interface
network = multinet.multi_layer_network()
network.add_edges(edges, input_type='list') # Handles parsing, encoding, validation
network.basic_stats() # Aggregates multiple NetworkX calls
```

Strategy Pattern

Used in: Algorithms, layout computation

Purpose: Interchangeable algorithms following common interface

```
# Different community detection strategies
def detect_communities(network, method='louvain'):
    strategies = {
        'louvain': community_louvain.best_partition,
        'infomap': community_wrapper.infomap_communities,
        'label_prop': label_propagation.propagate
    }
    return strategies[method](network.core_network)
```

Template Method Pattern

Used in: Visualization, layout algorithms

Purpose: Define algorithm skeleton, allow customization in subclasses

```
class LayoutAlgorithm:
    def compute(self, graph):
        self.initialize(graph)
        self.iterate()
        return self.finalize()

    def initialize(self, graph):
        raise NotImplementedError

    def iterate(self):
        raise NotImplementedError

    def finalize(self):
        raise NotImplementedError
```

Dependency Injection

Used in: Configuration, algorithm parameters

Purpose: Inject dependencies rather than hard-coding

```
# Configuration injected rather than hard-coded
from py3plex.config import DEFAULT_COLORS, LAYOUT_PARAMS

def draw_network(network, colors=None, layout_params=None):
```

(continues on next page)

(continued from previous page)

```

colors = colors or DEFAULT_COLORS
layout_params = layout_params or LAYOUT_PARAMS
# Use injected configuration

```

Data Flow

Typical Workflow

1. **Input:** Load or create network

```

network = multinet.multi_layer_network()
network.load_network("data.graphml", input_type="graphml")

```

2. **Processing:** Apply algorithms

```

communities = community_louvain.best_partition(network.core_network)
centrality = calc.multilayer_degree_centrality(network)

```

3. **Analysis:** Compute statistics

```

density = mls.layer_density(network, 'layer1')
correlation = mls.inter_layer_degree_correlation(network, 'L1', 'L2')

```

4. **Visualization:** Render results

```

draw_multilayer_default([network], display=True)

```

5. **Output:** Export results

```

network.save_network("output.graphml", output_type="graphml")

```

State Management

Immutable Operations: Most algorithms don't modify the network

```

# These don't modify the network
centrality = calc.multilayer_degree_centrality(network)
communities = community_louvain.best_partition(network.core_network)

```

Mutable Operations: Some operations modify network state

```

# These modify the network
network.add_edges(new_edges, input_type='list')
network.aggregate_layers(['L1', 'L2'], 'combined')

```

Extension Points

Custom Algorithms

Add new algorithms by following existing patterns:

```

# py3plex/algorithms/my_module/my_algorithm.py
from py3plex.core.multinet import multi_layer_network

def my_centrality(network: multi_layer_network) -> dict:
    """
    Custom centrality measure.
    """

```

(continues on next page)

(continued from previous page)

```

Parameters
-----
network : multi_layer_network
    Input network

Returns
-----
dict
    Node centrality scores
"""
G = network.core_network
centrality = {}

for node in G.nodes():
    # Implement custom logic
    centrality[node] = compute_score(node, G)

return centrality

```

Custom Visualizations

Create custom plots using drawing machinery:

```

from py3plex.visualization import drawing_machinery as dm
import matplotlib.pyplot as plt

def my_custom_plot(network):
    """Custom visualization."""
    fig, ax = plt.subplots(figsize=(10, 8))

    # Compute layout
    pos = dm.compute_layout(network.core_network, 'force')

    # Draw elements
    dm.draw_nodes(ax, network.core_network, pos, node_size=50)
    dm.draw_edges(ax, network.core_network, pos, edge_width=1)
    dm.draw_labels(ax, pos, labels=network.get_node_labels())

    plt.show()

```

Custom Parsers

Add support for new file formats:

```

# py3plex/core/parsers.py
def parse_my_format(input_file, **kwargs):
    """
    Parse custom file format.

    Parameters
    -----
    input_file : str
        Path to input file

    Returns

```

(continues on next page)

(continued from previous page)

```

-----
multi_layer_network
    Parsed network
"""
network = multi_layer_network()

with open(input_file, 'r') as f:
    for line in f:
        # Parse line and add to network
        pass

return network

```

Configuration System

Centralized Configuration

py3plex/config.py provides centralized configuration:

```

# Default color palettes (8 options including colorblind-safe)
DEFAULT_COLORS = 'Set1'
COLORBLIND_SAFE = 'colorblind'

# Visualization defaults
DEFAULT_NODE_SIZE = 20
DEFAULT_EDGE_WIDTH = 1.0
DEFAULT_ALPHA = 0.7

# Layout parameters
LAYOUT_PARAMS = {
    'force': {'iterations': 500, 'optimal_distance': 1.0},
    'fa2': {'iterations': 1000, 'gravity': 1.0}
}

# Performance settings
SPARSE_THRESHOLD = 1000 # Use sparse matrices above this node count
MEMORY_WARNING_THRESHOLD = 10000 # Warn for large dense matrices

```

Usage:

```

from py3plex.config import DEFAULT_COLORS, LAYOUT_PARAMS

colors = DEFAULT_COLORS
iterations = LAYOUT_PARAMS['force']['iterations']

```

Testing Architecture

Test Organization

```

tests/
├── test_core_functionality.py    # Core data structure tests
├── test_multilayer_*.py         # Multilayer algorithm tests
├── test_random_walks.py         # Random walk tests
├── test_io_*.py                 # I/O and parsing tests
├── test_config_api.py           # Configuration tests
└── test_utils.py                # Utility function tests

```

Test Patterns

Unit Tests: Test individual functions in isolation

```
def test_layer_density():
    network = create_test_network()
    density = mls.layer_density(network, 'layer1')
    assert 0 <= density <= 1
```

Integration Tests: Test workflows across modules

```
def test_community_detection_workflow():
    network = load_network("test_data.graphml")
    communities = community_louvain.best_partition(network.core_network)
    assert len(communities) > 0
```

Property-Based Tests: Test invariants

```
def test_centrality_normalization():
    network = create_random_network()
    centrality = calc.multilayer_degree_centrality(network)
    # Centrality values should be normalized
    assert all(0 <= v <= 1 for v in centrality.values())
```

Performance Considerations

Lazy Evaluation

Expensive operations are computed on-demand:

```
class multi_layer_network:
    @property
    def supra_adjacency(self):
        if self._supra_adj_cache is None:
            self._supra_adj_cache = self._compute_supra_adjacency()
        return self._supra_adj_cache
```

Sparse Matrices

Use sparse representations for large networks:

```
def get_supra_adjacency_matrix(self, sparse=True):
    if sparse or len(self.get_nodes()) > SPARSE_THRESHOLD:
        return scipy.sparse.csr_matrix(adj)
    return np.array(adj)
```

Vectorization

Prefer NumPy vectorized operations:

```
# Bad: Python loop
degrees = [sum(1 for _ in G.neighbors(node)) for node in nodes]

# Good: Vectorized
degrees = np.array(list(dict(G.degree()).values()))
```

Logging Infrastructure

Centralized Logging

py3plex/logging_config.py provides structured logging:

```
import logging
from py3plex.logging_config import get_logger

logger = get_logger(__name__)

logger.info("Processing network with %d nodes", num_nodes)
logger.warning("Large network detected, using sparse matrices")
logger.error("Invalid layer: %s", layer_name)
```

Log Levels

- **DEBUG:** Detailed diagnostic information
- **INFO:** General informational messages
- **WARNING:** Warning messages (e.g., performance concerns)
- **ERROR:** Error messages
- **CRITICAL:** Critical errors

Error Handling

Custom Exceptions

py3plex/exceptions.py defines domain-specific exceptions:

```
class NetworkError(Exception):
    """Base exception for network errors."""
    pass

class LayerNotFoundError(NetworkError):
    """Raised when layer doesn't exist."""
    pass

class InvalidFormatError(NetworkError):
    """Raised when file format is invalid."""
    pass
```

Usage:

```
from py3plex.exceptions import LayerNotFoundError

def get_layer(self, layer_name):
    if layer_name not in self.layer_name_map:
        raise LayerNotFoundError(f"Layer '{layer_name}' not found")
    return self.layer_name_map[layer_name]
```

Future Architecture

Planned Improvements

1. **Backend Registry:** Support for igraph, cugraph backends
2. **Streaming API:** Process networks larger than memory

3. **Distributed Computing:** Dask/Ray integration for large-scale analysis
4. **Plugin System:** Easy addition of third-party algorithms
5. **Type System:** Full type hints coverage (currently 65%)

See Also

- [Contributing to py3plex](#) - Contributing guidelines
- [development](#) - Development workflow
- [core](#) - Core API documentation

8.6.4 Repository Layout

This document explains the structure of the py3plex repository and what each directory contains.

Overview

```
py3plex/
├── py3plex/          # Main package source code
├── docs/             # Built documentation (HTML)
├── docfiles/         # Documentation source (RST files)
├── examples/         # Example scripts
├── tests/            # Test suite
├── gui/              # Web interface
├── datasets/         # Sample datasets
├── benchmarks/       # Performance benchmarks
├── fuzzing/          # Fuzz testing
├── .github/          # GitHub Actions CI/CD
├── pyproject.toml     # Package configuration
├── Makefile          # Development commands
└── README.md         # Project README
```

Main Package (py3plex/)

Source code for the py3plex library:

```
py3plex/
├── __init__.py       # Package initialization
├── core/             # Core data structures
│   ├── multinet.py   # multi_layer_network class
│   ├── parsers.py    # File I/O
│   └── converters.py  # Format conversion
├── algorithms/       # Network algorithms
│   ├── community_detection/
│   ├── statistics/
│   ├── multilayer_algorithms/
│   └── general/
├── visualization/    # Plotting and rendering
│   ├── multilayer.py
│   ├── drawing_machinery.py
│   └── layout_algorithms.py
├── wrappers/         # High-level interfaces
└── io/              # Modern I/O system
```

Key modules:

- `core/multinet.py` - Main `multi_layer_network` class
- `algorithms/` - All analysis algorithms

- visualization/ - All visualization code
- io/ - Modern schema-based I/O

Documentation (docfiles/)

Sphinx documentation source files:

```
docfiles/
├── index.rst          # Main documentation index
├── conf.py            # Sphinx configuration
├── Makefile           # Build commands
├── getting_started/   # Beginner guides
├── concepts/          # Conceptual explanations
├── user_guide/        # How-to guides
├── deployment/        # Deployment guides
├── gui/               # GUI documentation
├── dev/               # Developer docs
├── examples/          # Example index
└── reference/         # API reference
```

Building documentation:

```
cd docfiles
make html # Output in _build/html/
```

Examples (examples/)

Executable example scripts demonstrating various features:

```
examples/
├── basic/                # Basic usage
│   ├── example_load_network.py
│   ├── example_basic_stats.py
│   └── example_simple_viz.py
├── visualization/        # Visualization examples
│   ├── example_multilayer_viz.py
│   ├── example_custom_layouts.py
│   └── example_interactive_plots.py
├── analysis/             # Analysis examples
│   ├── example_community_detection.py
│   ├── example_centrality.py
│   └── example_statistics.py
├── advanced/             # Advanced topics
│   ├── example_node2vec.py
│   ├── example_random_walks.py
│   └── example_embeddings.py
└── io/                   # I/O examples
    ├── example_arrow_io.py
    └── example_format_conversion.py
```

Running examples:

```
python examples/basic/example_load_network.py
```

Tests (tests/)

Test suite using pytest:

```
tests/
├── test_core.py           # Core functionality
├── test_algorithms.py     # Algorithm tests
├── test_visualization.py  # Visualization tests
├── test_io.py            # I/O tests
├── test_performance_core.py # Performance benchmarks
├── test_fuzzing_properties.py # Property-based tests
└── conftest.py           # Pytest configuration
```

Running tests:

```
make test      # Run all tests
make test-fast # Skip slow tests
make coverage  # With coverage report
```

GUI (gui/)

Web interface for py3plex:

```
gui/
├── app.py           # Flask application
├── routes/          # API endpoints
├── templates/       # HTML templates
├── static/          # CSS, JS, images
├── config.py        # Configuration
└── Dockerfile       # Docker container
```

Running GUI:

```
python gui/app.py
```

Datasets (datasets/)

Sample networks for testing and examples:

```
datasets/
├── test.edgelist      # Simple test network
├── multiedgelist.txt  # Multilayer test
├── karate_club/       # Karate club network
└── bio_networks/     # Biological networks
```

Benchmarks (benchmarks/)

Performance benchmarking suite:

```
benchmarks/
├── bench_core.py      # Core operations
├── bench_algorithms.py # Algorithm performance
├── bench_io.py        # I/O performance
└── bench_aggregation.py # Aggregation benchmarks
```

Running benchmarks:

```
make benchmark
```


CI/CD (.github/)

GitHub Actions workflows:

```
.github/
├── workflows/
│   ├── tests.yml           # Test CI
│   ├── code-quality.yml    # Linting/formatting
│   ├── docs.yml           # Documentation build
│   ├── release.yml        # Release automation
│   └── dependabot.yml     # Dependency updates
```

Configuration Files

Root-level configuration:

- `pyproject.toml` - Package metadata, dependencies, build config
- `Makefile` - Development commands (make test, make docs, etc.)
- `README.md` - Project overview and quick start
- `.gitignore` - Git ignore rules
- `LICENSE` - MIT license
- `CITATION.cff` - Citation information
- `MANIFEST.in` - Package manifest for distribution

Working with the Repository

Initial Setup

```
# Clone repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Create virtual environment
python3 -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install in development mode
pip install -e ".[dev]"
```

Development Workflow

```
# 1. Make changes to code

# 2. Format code
make format

# 3. Run linters
make lint

# 4. Run tests
make test

# 5. Build documentation
make docs
```

(continues on next page)

(continued from previous page)

```
# 6. Commit changes
git add .
git commit -m "Description"
git push
```

Finding Things

“Where is...?”

Core network class?

py3plex/core/multinet.py → multi_layer_network class

Community detection algorithms?

py3plex/algorithms/community_detection/

Visualization code?

py3plex/visualization/

I/O functions?

py3plex/io/ (modern) or py3plex/core/parsers.py (legacy)

Tests for feature X?

tests/test_*.py - grep for the feature name

Example using feature X?

examples/ - check the relevant subdirectory

Documentation source?

docfiles/ - RST files organized by topic

Built documentation?

docs/ - HTML files (generated from docfiles/)

File Naming Conventions

- **Source files:** snake_case.py
- **Test files:** test_<module>.py
- **Example files:** example_<feature>.py
- **Documentation:** <topic>.rst
- **Configuration:** Various (pyproject.toml, conf.py, etc.)

Important Files

For Contributors

- CONTRIBUTING.md - Contribution guidelines (if exists)
- docfiles/dev/contributing.rst - Contribution documentation
- Makefile - Available development commands
- pyproject.toml - Package dependencies

For Users

- README.md - Quick start guide
- docfiles/index.rst - Documentation entry point
- examples/ - Working code examples
- LICENSE - MIT license

For CI/CD

- `.github/workflows/*.yaml` - CI pipelines
- `Makefile` - Automated commands
- `pyproject.toml` - Build configuration

Directory Conventions

Source code directories:

- No spaces in names
- Lowercase with underscores
- Descriptive names (`community_detection`, not `cd`)

Test directories:

- Mirror source structure
- One test file per source file when possible

Documentation directories:

- Organize by user journey/topic
- Clear, descriptive names

Package Distribution

When building packages, these files are included:

Included:

- All `.py` files in `py3plex/`
- `LICENSE`
- `README.md`
- Required configuration files

Excluded (via `.gitignore`):

- `__pycache__/`
- `.pytest_cache/`
- `*.pyc`
- `build/`, `dist/`, `*.egg-info/`
- Virtual environments
- IDE-specific files

See `MANIFEST.in` for complete inclusion/exclusion rules.

Related Documentation

- *Development Guide* - Development setup and workflow
- *Contributing to py3plex* - How to contribute
- *Architecture and Design* - Internal architecture
- *Installation and Setup* - Installation guide

External Resources:

- Repository: <https://github.com/SkBlaz/py3plex>

- Issues: <https://github.com/SkBlaz/py3plex/issues>
- Pull Requests: <https://github.com/SkBlaz/py3plex/pulls>

8.6.5 Contributing to py3plex

We welcome contributions to py3plex. This guide explains how to contribute effectively.

Ways to Contribute

You can contribute in many ways:

- Report bugs - Open issues for bugs you encounter
- Suggest features - Propose new features or improvements
- Write documentation - Improve or expand documentation
- Fix bugs - Submit pull requests fixing issues
- Add features - Implement new algorithms or capabilities
- Write tests - Improve test coverage
- Review PRs - Help review pull requests from others

Getting Started

Fork and Clone

1. Fork the repository on GitHub
2. Clone your fork:

```
git clone https://github.com/YOUR_USERNAME/py3plex.git
cd py3plex
```

3. Add upstream remote:

```
git remote add upstream https://github.com/SkBlaz/py3plex.git
```

Development Setup

Install in development mode with all dependencies:

```
# Create virtual environment
python3 -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install in editable mode with dev dependencies
pip install -e ".[dev]"
```

This installs:

- Core dependencies
- Testing tools (pytest, coverage)
- Linting tools (black, ruff, isort, mypy)
- Documentation tools (sphinx)

Development Workflow

Create a Branch

Always create a new branch for your work:

```
git checkout -b feature/my-new-feature
# or
git checkout -b fix/issue-123
```

Make Changes

1. Write your code following our coding standards (see below)
2. Add or update tests
3. Update documentation
4. Run linters and tests

Run Tests

```
# Run all tests
make test

# Or directly with pytest
python run_tests.py
```

Run Linters

```
# Run all linters
make lint

# Or individual tools
make format # Auto-format code (black, isort)
ruff check .
mypy py3plex
```

Commit Changes

Write clear, descriptive commit messages:

```
git add .
git commit -m "Add feature X to support Y

- Implement algorithm for Z
- Add tests for edge cases
- Update documentation"
```

Push and Create PR

```
git push origin feature/my-new-feature
```

Then create a Pull Request on GitHub with:

- Clear title describing the change
- Description of what changed and why

- Reference to related issues (e.g., “Fixes #123”)
- Screenshots for UI changes

Coding Standards

Code Style

We follow PEP 8 with these specifics:

- Line length: 100 characters (not 80)
- Indentation: 4 spaces (no tabs)
- String quotes: Single quotes preferred (‘text’ not “text”)
- Imports: Grouped and sorted (using isort)

Auto-format your code:

```
make format
```

This runs:

- `isort` - Sort and organize imports
- `black` - Format code consistently
- `ruff --fix` - Auto-fix linting issues

Naming Conventions

- Functions/variables: `snake_case`
- Classes: `PascalCase`
- Constants: `UPPER_SNAKE_CASE`
- Private members: `_leading_underscore`

```
# Good
def compute centrality(network, node_id):
    MAX_ITERATIONS = 100
    centrality_scores = {}
    return centrality_scores

class MultilayerNetwork:
    def __init__(self):
        self._internal_state = {}
```

Docstrings

Use NumPy-style docstrings for all public functions and classes:

```
def multilayer_degree Centrality(network, layer=None, weighted=False):
    """
    Compute degree centrality for multilayer network.

    Parameters
    -----
    network : multi_layer_network
        The multilayer network to analyze.
    layer : str, optional
        Specific layer to analyze. If None, aggregates across all layers.
```

(continues on next page)

(continued from previous page)

```
weighted : bool, default=False
    If True, compute strength (weighted degree) instead of degree.
```

Returns

dict

Dictionary mapping node names to centrality scores.

Examples

```
>>> network = multinet.multi_layer_network()
>>> network.add_edges([['A', 'L1', 'B', 'L1', 1]])
>>> centrality = multilayer_degree_centrality(network)
>>> centrality['A---L1']
1.0
```

See Also

`multilayer_betweenness_centrality` : Betweenness centrality

Notes

For directed networks, this computes out-degree by default.

References

```
.. [1] De Domenico, M., et al. (2013). Mathematical formulation of
    multilayer networks. Physical Review X, 3(4), 041022.
```

"""

pass

Type Hints

Add type hints to improve code clarity:

```
from typing import Dict, List, Optional
from py3plex.core.multinet import multi_layer_network

def compute_statistics(
    network: multi_layer_network,
    layers: Optional[List[str]] = None
) -> Dict[str, float]:
    """Compute network statistics."""
    pass
```

Testing Guidelines

Test Requirements

All new code should include tests:

- Unit tests for individual functions
- Integration tests for workflows
- Edge cases for boundary conditions
- Documentation tests for examples in docstrings

Writing Tests

Use pytest for testing:

```
# tests/test_my_feature.py
import pytest
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

def test_layer_density():
    """Test layer density calculation."""
    # Setup
    network = multinet.multi_layer_network()
    network.add_edges([
        ['A', 'L1', 'B', 'L1', 1],
        ['B', 'L1', 'C', 'L1', 1],
    ], input_type='list')

    # Execute
    density = mls.layer_density(network, 'L1')

    # Assert
    assert 0 <= density <= 1
    assert density == pytest.approx(0.333, abs=0.01)

def test_invalid_layer():
    """Test error handling for invalid layer."""
    network = multinet.multi_layer_network()

    with pytest.raises(ValueError):
        mls.layer_density(network, 'nonexistent')
```

Test Coverage

Aim for high test coverage:

```
# Run tests with coverage
pytest --cov=py3plex --cov-report=html

# View coverage report
open htmlcov/index.html # macOS
# or
xdg-open htmlcov/index.html # Linux
```

Target: >80% coverage for new code

Documentation

Update Documentation

When adding features, update:

1. **Docstrings** in the code
2. **RST files** in docfiles/ if adding major features
3. **Examples** in examples/ directory
4. **README** if changing installation or basic usage

Build Documentation

```
cd docfiles
make html

# View documentation
open _build/html/index.html
```

Documentation Style

- Clear and concise - Use simple language
- Code examples - Include working examples
- Cross-references - Link to related documentation
- Visual aids - Add diagrams where helpful

Pull Request Guidelines

Before Submitting

[OK] Code follows style guide (`make lint` passes)

[OK] All tests pass (`make test` passes)

[OK] New code has tests

[OK] Documentation is updated

[OK] Commit messages are clear

[OK] Branch is up to date with main

PR Description

Include in your PR description:

- What changed
- Why the change is needed
- How you implemented it
- Testing done
- Screenshots for visual changes
- Breaking changes if any

Example PR template:

```
## Description

Implements multilayer PageRank centrality following [citation].

## Motivation

Closes #123 - needed for analyzing directed multilayer networks.

## Changes

- Add `multilayer_pagerank()` function
- Add tests with 90% coverage
- Add example in `example_centrality.py`
```

(continues on next page)

(continued from previous page)

```
- Update documentation in `centrality.rst`

## Testing

- [x] Unit tests pass
- [x] Integration test on real network
- [x] Tested on Python 3.8, 3.10, 3.12
- [x] Linting passes

## Breaking Changes

None.
```

Review Process

1. Maintainers will review your PR
2. Address any feedback or requested changes
3. Once approved, your PR will be merged
4. Your contribution will be in the next release!

Reporting Issues

Bug Reports

When reporting bugs, include:

- Description of the bug
- Steps to reproduce
- Expected behavior
- Actual behavior
- Python version (`python --version`)
- py3plex version
- Operating system
- Minimal code example

Example:

```
## Bug Description

`layer_density()` returns negative values for certain graphs.

## Steps to Reproduce

```python
network = multinet.multi_layer_network()
network.add_edges([['A', 'L1', 'B', 'L1', -1]])
density = mls.layer_density(network, 'L1')
print(density) # -0.5
```

## Expected
```

(continues on next page)

(continued from previous page)

Density should be between 0 and 1, or raise ValueError for negative weights.

Actual

Returns -0.5

Environment

- Python 3.10.5
- py3plex 0.95a
- Ubuntu 22.04

Feature Requests

For feature requests, describe:

- Use case - What problem does it solve?
- Proposed solution - How should it work?
- Alternatives - Other approaches considered
- Additional context - Examples, papers, etc.

Code of Conduct

- Be respectful and inclusive
- Welcome newcomers
- Focus on what is best for the community
- Show empathy towards other community members

Contributors are expected to follow these principles to foster a welcoming and productive community.

License

By contributing, you agree that your contributions will be licensed under the MIT License.

Recognition

Contributors are recognized in:

- GitHub contributors page
- Release notes
- acknowledgements section in the documentation

Getting Help

- Questions: Open a GitHub Discussion
- Chat: Join our community chat (link in README)
- Email: Contact maintainers at blaz.skrlj@ijs.si

Next Steps

- Read development for development workflow details
- See architecture for system architecture
- Browse [existing issues](#)

- Check [good first issues](#)

Thank you for contributing to py3plex!

8.7 Part VII: Examples

Learning by example is often the most effective approach. This section provides a curated collection of working code examples that demonstrate py3plex’s capabilities across various use cases. From basic network creation to advanced analysis workflows, these examples serve as templates for your own projects.

8.7.1 Examples: Learning by Doing

“Tell me and I forget. Teach me and I remember. Involve me and I learn.” — Benjamin Franklin

The Power of Examples

If you’ve read other sections of this documentation, you’ve seen explanations of concepts and descriptions of APIs. But there’s no substitute for seeing complete, working code that solves real problems.

This section provides curated examples that demonstrate py3plex’s capabilities across various use cases. Each example is:

- **Complete:** You can run it from start to finish without modification
- **Annotated:** Comments explain not just what the code does, but why
- **Practical:** Based on real analysis patterns, not toy demonstrations
- **Progressive:** Examples build in complexity, from simple to advanced

How to Use These Examples

As learning material: Work through examples in order to build skills progressively. Type the code rather than copying—the mechanical process aids learning.

As templates: When you face a real problem, find an example that’s similar and adapt it. Starting from working code is faster than starting from scratch.

As references: When you need to remember how to do something specific, the examples serve as quick reminders.

As validation: Running examples on your installation helps verify that everything is working correctly.

What You’ll Find

The examples cover:

- **Network creation and loading** — Multiple ways to get data into py3plex
- **Statistical analysis** — Computing and interpreting multilayer metrics
- **Community detection** — Finding structure in your networks
- **Visualization** — Creating publication-ready figures
- **Random walks and embeddings** — Feature extraction for machine learning
- **Complete workflows** — End-to-end analyses that combine multiple techniques

Example Organization

Examples are organized by topic and complexity:

Beginner examples introduce core concepts with small, simple networks. These are ideal for learning the basics.

Intermediate examples tackle more realistic scenarios with larger networks and combined techniques.

Advanced examples demonstrate sophisticated analyses, including performance optimization and custom extensions.

From Example to Application

The goal of examples is not to collect code, but to develop problem-solving ability. As you work through examples, ask yourself:

- What would I change to apply this to my data?
- What assumptions does this example make that might not hold for my case?
- What other questions could this technique help me answer?

This reflective practice transforms examples from isolated code snippets into reusable patterns.

Getting Started

Browse the [Examples](#) to find examples relevant to your interests, or work through them in order for a structured learning experience.

Note

Running the Examples:

All examples assume py3plex is installed. Some examples require optional dependencies—check the imports at the top of each example.

If an example fails with an ImportError, install the missing package:

```
pip install <missing-package>
```

Common optional dependencies include:

- matplotlib for visualization
- gensim for embeddings
- python-louvain for community detection
- sklearn for machine learning examples

8.7.2 Examples

This page provides a curated list of example scripts demonstrating py3plex features. All examples are available in the [examples/ directory](#) of the repository.

Getting Started Examples

Basic Network Creation

- `example_load_network.py` - Loading networks from files
- `example_create_network.py` - Creating networks from scratch
- `example_basic_stats.py` - Computing basic statistics

Quick Tutorials

- `tutorial_10min.py` - Executable version of the 10-minute tutorial

- `example_quick_start.py` - Quick start examples

Visualization Examples

Basic Visualization

- `example_multilayer_visualization.py` - Basic multilayer plots
- `example_simple_viz.py` - Simple visualization examples
- `example_hairball.py` - Hairball plots

Advanced Visualization

- `example_custom_layouts.py` - Custom layout algorithms
- `example_diagonal_plot.py` - Diagonal projection for multilayer networks
- `example_interactive_plots.py` - Interactive visualizations with Plotly
- `example_layer_visualization.py` - Layer-by-layer visualization

Community Visualization

- `example_community_viz.py` - Visualizing detected communities
- `example_colored_networks.py` - Custom node/edge coloring

Community Detection Examples

Single-Layer Community Detection

- `example_community_detection.py` - Louvain and Infomap
- `example_louvain.py` - Louvain algorithm examples
- `example_label_propagation.py` - Semi-supervised community detection

Multilayer Community Detection

- `example_multilayer_communities.py` - Multilayer Louvain
- `example_modularity.py` - Modularity optimization
- `example_overlapping_communities.py` - Overlapping community detection

Network Statistics Examples

Basic Statistics

- `example_multilayer_statistics.py` - Multilayer-specific statistics
- `example_layer_comparison.py` - Comparing layers
- `example_node_metrics.py` - Node-level metrics

Centrality Measures

- `example_centrality.py` - Degree, betweenness, PageRank
- `example_multilayer_centrality.py` - Multilayer centrality measures
- `example_versatility.py` - Versatility centrality

Network Comparison

- `example_network_comparison.py` - Comparing multiple networks
- `example_statistical_tests.py` - Statistical comparison

I/O and Data Format Examples

Loading Networks

- `example_IO.py` - Various input formats
- `example_edgelist_loading.py` - Edge list formats
- `example_graphml.py` - GraphML format
- `example_csv_loading.py` - CSV with sidecars

Modern I/O (Arrow/Parquet)

- `example_arrow_io.py` - Apache Arrow format
- `example_parquet.py` - Parquet format
- `example_schema_graph.py` - Schema-based I/O

Format Conversion

- `example_format_conversion.py` - Converting between formats
- `example_export.py` - Exporting networks

Random Walks and Embeddings

Random Walks

- `example_random_walks.py` - Basic and Node2Vec walks
- `example_walkers.py` - Different walk strategies
- `example_metapath_walks.py` - Meta-path based walks

Node Embeddings

- `example_n2v_embedding.py` - Node2Vec embeddings
- `example_deepwalk.py` - DeepWalk embeddings
- `example_embeddings.py` - Various embedding methods

Embedding Applications

- `example_link_prediction.py` - Link prediction with embeddings
- `example_node_classification.py` - Node classification
- `example_clustering_embeddings.py` - Clustering with embeddings

Network Decomposition

- `example_network_decomposition.py` - Meta-path feature extraction
- `example_feature_extraction.py` - Network feature engineering
- `example_tensor_decomposition.py` - Tensor decomposition methods

Network Manipulation

- `example_manipulation.py` - Network operations (add, remove, filter)
- `example_subnetworks.py` - Extracting subnetworks
- `example_layer_operations.py` - Layer-specific operations
- `example_aggregation.py` - Aggregating layers

Algorithms and Analysis

Network Dynamics

- `example_spreading.py` - Network traversal and spreading processes
- `example_sir_epidemic.py` - SIR epidemic simulation
- `example_diffusion.py` - Diffusion processes

Specialized Algorithms

- `example_ricci_curvature.py` - Ricci curvature computation
- `example_motifs.py` - Network motif discovery
- `example_path_analysis.py` - Path-based analysis

Machine Learning

- `example_ml_features.py` - Feature extraction for ML
- `example_graph_kernels.py` - Graph kernel methods
- `example_supervised_learning.py` - Supervised learning on networks

NetworkX Integration

- `example_networkx_wrapper.py` - Using NetworkX functions
- `example_networkx_interop.py` - NetworkX interoperability
- `example_convert_networkx.py` - Converting to/from NetworkX

Benchmarking

- `example_benchmarking.py` - Performance benchmarking
- `example_scalability.py` - Scalability testing
- `example_memory_profiling.py` - Memory usage analysis

GUI and API

- `example_api_usage.py` - Using the REST API
- `example_gui_integration.py` - GUI integration examples
- `example_batch_processing.py` - Batch processing with CLI

Real-World Datasets

Biological Networks

- `example_protein_interaction.py` - Protein-protein interaction networks
- `example_gene_regulation.py` - Gene regulatory networks
- `example_metabolic_networks.py` - Metabolic pathways

Social Networks

- `example_social_multiplex.py` - Multiplex social networks
- `example_citation_network.py` - Citation networks
- `example_collaboration.py` - Collaboration networks

Transportation

- `example_multimodal_transport.py` - Multi-modal transportation
- `example_flight_network.py` - Airline networks

Running Examples

All examples can be run directly with Python:

```
# Basic usage
python examples/example_multilayer_visualization.py

# With custom data
python examples/example_load_network.py path/to/your/network.edgelist

# From repository root
cd py3plex
python examples/basic/example_load_network.py
```

Many examples accept command-line arguments:

```
python examples/example_community_detection.py --algorithm louvain --input data.
↪ graphml
```

Example Template

Use this template for your own scripts:

```
"""
Example: Your Feature
=====

Description of what this example demonstrates.

Usage:
    python example_your_feature.py
"""

from py3plex.core import multinet
from py3plex.visualization.multilayer import draw_multilayer_default

def main():
    # Load or create network
    network = multinet.multi_layer_network()
    network.add_edges([
        ['A', 'layer1', 'B', 'layer1', 1]
    ], input_type="list")

    # Your analysis
    network.basic_stats()

    # Visualization
    draw_multilayer_default([network], display=True)

if __name__ == "__main__":
    main()
```

Contributing Examples

To contribute an example:

1. Create a well-documented script in `examples/`
2. Use the template above

3. Test that it runs without errors
4. Add it to this index with a brief description
5. Submit a pull request

See *Contributing to py3plex* for detailed guidelines.

Related Documentation

- *Chapter 1: Your First Five Minutes* - Quick start guide
- *Chapter 2: The Complete Workflow in Ten Minutes* - 10-minute tutorial
- *Working with Networks* - Working with networks
- *Visualization* - Visualization guide

Repository:

- Examples directory: <https://github.com/SkBlaz/py3plex/tree/master/examples>
 - Submit examples: <https://github.com/SkBlaz/py3plex/pulls>
-

8.8 Part VIII: Reference & Citation

This final section provides comprehensive reference materials including the complete algorithm reference, API documentation, and citation information. If you use py3plex in your research, please cite the original publication to support continued development.

8.8.1 Reference & Citation: The Complete Guide

“Science progresses one funeral at a time.” — Attributed to Max Planck

The Value of Reference Material

Reference documentation serves a different purpose than tutorials and guides. While learning materials explain concepts and build understanding, reference materials provide precision and completeness. When you need to know the exact parameters of a function, the precise behavior of an algorithm, or the correct citation for a method—reference material is what you need.

What This Section Contains

Algorithm Reference provides detailed documentation for every algorithm in py3plex. For each algorithm, you’ll find:

- Mathematical formulation (when relevant)
- Parameters and their meanings
- Return values
- Computational complexity
- Usage examples
- References to original papers

This is the definitive source for “how does this algorithm work?” and “what do these parameters mean?”

API Index is a comprehensive list of all public classes, functions, and constants in py3plex. This is useful when you know what you’re looking for and need to find it quickly.

Citation & Acknowledgements provides the information you need to properly cite py3plex in academic work, along with acknowledgements to the researchers and developers who made this library possible.

Using the Reference Material

When learning: Reference material is not designed for learning. Use the tutorials and guides instead. Return to reference material when you need precision.

When working: Keep the algorithm reference handy. When you're using a function and need to check a parameter, the reference provides authoritative information.

When writing papers: The citation section provides the BibTeX entries you need. Proper citation is essential for academic integrity and helps the project.

When debugging: If something isn't behaving as expected, the reference material provides the ground truth for what functions should do.

A Note on Algorithms

The algorithms in py3plex come from academic research, often with decades of development behind them. The algorithm reference connects you to this broader context. Understanding where an algorithm comes from helps you:

- Know its strengths and limitations
- Find related methods that might work better for your case
- Cite the original authors appropriately
- Contribute back to the community of researchers

Standing on the Shoulders of Giants

py3plex exists because of the work of many researchers and developers:

- The NetworkX team, whose work provides py3plex's foundation
- Researchers who developed multilayer network theory and algorithms
- Contributors who reported bugs, suggested features, and submitted code
- Users who pushed the boundaries and discovered new use cases

The citation section is also an acknowledgement section—a recognition that science is a collective endeavor.

Getting Started with Reference

If you're looking for specific information:

- **Algorithm details** → *Algorithm Roadmap*
- **API lookup** → *API Documentation*
- **How to cite** → *Citation and References*

Note

Academic Users:

If you use py3plex in published research, please cite:

```
@Article{Škrlj2019,
  author={Škrlj, Blaž and Kralj, Jan and Lavrač, Nada},
  title={Py3plex toolkit for visualization and analysis of multilayer networks},
  journal={Applied Network Science},
  year={2019},
  volume={4},
  number={1},
  pages={94},
  doi={10.1007/s41109-019-0203-7}
}
```

Citation helps the project continue to receive support and development.

8.8.2 Algorithm Roadmap

This page provides a **comprehensive overview** of all algorithms implemented in py3plex, organized by category and showing relationships between different methods.

Purpose: Help you quickly find the right algorithm for your multilayer network analysis task.

Algorithm Categories

The algorithms in py3plex are organized into these main categories:

1. *Community Detection Algorithms* - Finding community structure
2. *Centrality Measures* - Computing node importance
3. *Network Statistics* - Network-level metrics
4. *Node Ranking & Classification* - Ranking and classification methods
5. *Random Walks & Embeddings* - Random walk-based methods
6. *Visualization Algorithms* - Layout and rendering
7. *Benchmark & Utilities* - Testing and evaluation

Community Detection Algorithms

Module: `py3plex.algorithms.community_detection`

These algorithms identify groups of densely connected nodes (communities) in multilayer networks.

Louvain Algorithm

Path: `community_detection.community_louvain`

Functions:

- `best_partition(graph, partition, weight, resolution, randomize, random_state)` - Main entry point
- `generate_dendrogram(graph, partition, weight, resolution, randomize, random_state)` - Hierarchical communities
- `modularity(partition, graph, weight)` - Calculate modularity score
- `partition_at_level(dendrogram, level)` - Extract partition at specific level
- `induced_graph(partition, graph, weight)` - Create quotient graph

Best for: Fast community detection on large single-layer networks

Complexity: $O(n \log n)$

Related algorithms:

- *Multilayer Louvain* (multilayer extension)
- *Leiden Algorithm* (improved Louvain)

Infomap Algorithm

Path: `community_detection.community_wrapper`

Functions:

- `infomap_communities(network_input, binary_path, arguments)` - Main entry point

- `run_infomap(network, binary_path, arguments)` - Execute Infomap binary
- `parse_infomap(outfile)` - Parse Infomap output

Best for: Flow-based community detection, hierarchical structure

Complexity: $O(m \log n)$ where m is edges, n is nodes

Related algorithms:

- *Louvain Algorithm* (faster alternative)

Multilayer Louvain

Path: `community_detection.multilayer_modularity`

Functions:

- `louvain_multilayer(supra_adj, omega, resolution, seed)` - Multilayer Louvain
- `multilayer_modularity(supra_adj, partition, omega)` - Calculate multilayer modularity
- `build_supra_modularity_matrix(network, omega)` - Build modularity matrix

Best for: Community detection across multiple layers with inter-layer coupling

Complexity: $O(n^2 L)$ where L is number of layers

Algorithm details: Implements Mucha et al. (2010) multilayer modularity optimization

Related algorithms:

- *Louvain Algorithm* (single-layer version)
- *Leiden Algorithm* (alternative optimizer)

Leiden Algorithm

Path: `community_detection.leiden_multilayer`

Functions:

- `leiden_multilayer(supra_adj, omega, resolution, n_iterations, seed)` - Main entry point

Data structures:

- `LeidenResult` - Holds partition results with modularity score

Best for: More stable community detection than Louvain

Complexity: $O(n \log n)$ with better quality guarantees

Related algorithms:

- *Louvain Algorithm* (predecessor)
- *Multilayer Louvain* (multilayer extension)

NoRC Algorithm

Path: `community_detection.NoRC` and `community_detection.community_wrapper`

Functions:

- `NoRC_communities(network, alpha)` - Wrapper function
- `NoRC_communities_main(matrix, alpha)` - Core implementation
- `page_rank_kernel(index_row)` - PageRank computation
- `create_tree(centers)` - Build community hierarchy

Best for: Networks with overlapping communities

Related algorithms:

- *Community Ranking* (related approach)

Community Measures

Path: `community_detection.community_measures`

Functions:

- `modularity(network, partition, weight)` - Calculate modularity
- `size_distribution(network_partition)` - Community size distribution
- `number_of_communities(network_partition)` - Count communities

Purpose: Evaluate quality of community partitions

Related algorithms: All community detection algorithms above

Community Ranking

Path: `community_detection.community_ranking`

Functions:

- `page_rank_kernel(index_row)` - PageRank-based ranking
- `create_tree(centers)` - Build ranking tree
- `return_infomap_communities(network)` - Get Infomap communities

Best for: Ranking nodes within detected communities

Related algorithms:

- *Node Ranking* (general ranking)

Multilayer Benchmarks

Path: `community_detection.multilayer_benchmark`

Functions:

- `generate_multilayer_lfr(n_nodes, n_layers, avg_degree, ...)` - LFR benchmark
- `generate_coupled_er_multilayer(n_nodes, n_layers, p_intra, p_inter)` - ER benchmark
- `generate_sbm_multilayer(sizes, p_matrix, n_layers, ...)` - SBM benchmark

Purpose: Generate synthetic multilayer networks with known community structure for testing

Related algorithms: All community detection algorithms

Centrality Measures

Module: `py3plex.algorithms.multilayer_algorithms.centrality`

These algorithms measure the importance or influence of nodes in multilayer networks.

MultilayerCentrality Class

Path: `multilayer_algorithms.centrality.MultilayerCentrality`

Main class for computing various centrality measures.

Basic Centrality Measures

Methods:

- `overlapping_degree centrality()` - Sum of degrees across all layers
- `layer_specific_degree centrality(layer)` - Degree within specific layer
- `multilayer_degree centrality()` - Supra-adjacency degree
- `average_degree centrality()` - Average degree across layers

Best for: Quick importance ranking, well-connected node identification

Complexity: $O(n + m)$ where n is nodes, m is edges

Related measures:

- *Versatility Centrality* (cross-layer activity)

Eigenvector-Based Measures

Methods:

- `multiplex_eigenvector centrality(max_iter, tol)` - Eigenvector centrality
- `pagerank centrality(alpha, max_iter, tol)` - PageRank
- `katz centrality(alpha, beta, max_iter, tol)` - Katz centrality
- `communicability centrality(beta)` - Communicability centrality

Best for: Influence measure, recursive importance, prestige ranking

Complexity: $O(m)$ per iteration, typically converges quickly

Related measures:

- *Hubs and Authorities (HITS)* (directed networks)

Path-Based Measures

Methods:

- `multilayer_betweenness centrality()` - Betweenness across layers
- `multilayer_closeness centrality()` - Closeness across layers

Best for: Bridge identification, broadcasting efficiency

Complexity: $O(nm)$ for unweighted, $O(nm + n^2 \log n)$ for weighted

Warning: Expensive for large networks (>1000 nodes)

Related measures:

- *Utility Functions* (batch computation)

Hubs and Authorities (HITS)

Path: `node_ranking.node_ranking.hubs_and_authorities`

Best for: Ranking nodes in directed networks (authority = important destination, hub = important source)

Related measures:

- PageRank in *Centrality Measures*

Utility Functions

Functions:

- `compute_all_centralities(network, include_path_based, include_advanced)` - Compute multiple measures

Purpose: Batch computation of multiple centrality measures

Versatility Centrality

Path: `algorithms.statistics.multilayer_statistics.versatility_centrality`

Function:

- `versatility_centrality(network, centrality_type)` - Cross-layer versatility measure

Best for: Measuring how uniformly a node's importance is distributed across layers

Complexity: $O(m \times L)$ where L is number of layers

Related measures:

- All centrality measures in *Centrality Measures*

Supra Matrix Function Centralities

Path: `multilayer_algorithms.supra_matrix_function_centrality`

Functions:

- `communicability_centrality(supra_matrix, beta)` - Matrix exponential based
- `katz_centrality(supra_matrix, alpha, beta, max_iter, tol)` - Resolvent based

Best for: Advanced centrality using matrix functions on supra-adjacency

Complexity: Depends on matrix operations, typically $O(n^2)$ or $O(n^3)$

Related measures:

- *Centrality Measures* (standard measures)

MultiXRank

Path: `multilayer_algorithms.multixrank`

Functions:

- `multixrank_from_py3plex_networks(networks, config)` - MultiXRank algorithm

Best for: Ranking in multiplex heterogeneous networks with bipartite layers

Algorithm details: Extends PageRank to multiplex/heterogeneous networks

Related algorithms:

- PageRank in *Centrality Measures*

Network Statistics

Module: `py3plex.algorithms.statistics`

These algorithms compute various statistical properties of multilayer networks.

Multilayer Statistics

Path: `statistics.multilayer_statistics`

Layer-level measures:

- `layer_density(network, layer)` - Density of specific layer
- `edge_overlap(network, layer_i, layer_j)` - Edge overlap between layers
- `inter_layer_coupling_strength(network, layer_i, layer_j)` - Coupling strength
- `inter_layer_degree_correlation(network, layer_i, layer_j)` - Degree correlation
- `inter_layer_assortativity(network, layer_i, layer_j)` - Assortativity between layers
- `layer_similarity(network, layer_i, layer_j, method)` - Layer similarity

Node-level measures:

- `node_activity(network, node)` - Activity across layers
- `degree_vector(network, node, weighted)` - Degree vector across layers
- `multilayer_clustering_coefficient(network, node)` - Multilayer clustering

Network-level measures:

- `interdependence(network, sample_size)` - Network interdependence
- `supra_laplacian_spectrum(network, k)` - Spectral properties
- `algebraic_connectivity(network)` - Second smallest Laplacian eigenvalue

Best for: Understanding multilayer network structure and properties

Complexity: Varies by measure, typically $O(n)$ to $O(n^2)$

Related algorithms:

- *Multilayer Statistics (Detailed)* (simpler measures)
- *Topology Analysis* (topological properties)

Multilayer Statistics (Detailed)

See the full section above for complete details on multilayer statistics functions.

Basic Statistics

Path: `statistics.basic_statistics`

Functions:

- `identify_n_hubs(network, n, metric)` - Find top-n hub nodes
- `core_network_statistics(network)` - Comprehensive network statistics

Best for: Quick network overview, hub identification

Complexity: $O(n + m)$ for most measures

Related algorithms:

- *Multilayer Statistics (Detailed)* (advanced measures)

Topology Analysis

Path: `statistics.topology`

Functions for analyzing topological properties of networks.

Purpose: Study structural properties like degree distributions, connected components, etc.

Related algorithms:

- *Multilayer Statistics (Detailed)*

Correlation Networks

Path: `statistics.correlation_networks`

Functions:

- `default_correlation_to_network(correlation_matrix, threshold)` - Convert correlation matrix to network
- `pick_threshold(matrix)` - Automatically select threshold

Best for: Creating networks from correlation/similarity matrices

Related algorithms:

- *Network Statistics* (analysis after construction)

Enrichment Analysis

Path: `statistics.enrichment_modules`

Functions:

- `compute_enrichment(term_dataset, annotation_database, ...)` - General enrichment
- `fet_enrichment_generic(...)` - Fisher's exact test enrichment
- `fet_enrichment_terms(...)` - Term enrichment
- `fet_enrichment_uniprot(...)` - UniProt annotation enrichment
- `calculate_pval(term, alternative)` - p-value calculation
- `multiple_test_correction(input_dataset)` - FDR correction

Best for: Finding over-represented terms/annotations in network communities

Related algorithms:

- *Community Detection Algorithms* (provides communities for enrichment)

Statistical Comparison

Path: `statistics.stats_comparison`

Functions: Various statistical tests for comparing networks or algorithms

Related modules:

- `statistics.bayesian_tests` - Bayesian statistical tests
- `statistics.bayesian_distances` - Bayesian distance measures
- `statistics.critical_distances` - Critical difference diagrams

Best for: Comparing performance of different algorithms or network properties

Power Law Analysis

Path: `statistics.powerlaw`

Functions for testing and fitting power law distributions in networks.

Best for: Analyzing degree distributions, checking for scale-free properties

Related algorithms:

- *Multilayer Statistics (Detailed)* (degree distributions)

Node Ranking & Classification

Module: `py3plex.algorithms.node_ranking` and `py3plex.algorithms.network_classification`

Node Ranking

Path: `node_ranking.node_ranking`

Functions:

- `sparse_page_rank(graph, alpha, personalization, max_iter, tol)` - Sparse PageRank
- `hubs_and_authorities(graph)` - HITS algorithm
- `hub_matrix(graph)` - Hub matrix computation
- `authority_matrix(graph)` - Authority matrix computation
- `stochastic_normalization(matrix)` - Normalize for random walks
- `stochastic_normalization_hin(matrix)` - Normalize for heterogeneous networks

Best for: Ranking nodes by importance in directed networks

Complexity: $O(m)$ per iteration for PageRank, $O(nm)$ for HITS

Related algorithms:

- PageRank in *Centrality Measures*
- *Label Propagation* (classification)

Label Propagation

Path: `network_classification.label_propagation`

Functions:

- `label_propagation(adjacency, labels, alpha, max_iter, tol, normalization)` - Main algorithm
- `validate_label_propagation(...)` - Validation function
- `label_propagation_normalization(matrix)` - Normalization
- `normalize_initial_matrix_freq(mat)` - Frequency normalization
- `normalize_amplify_freq(mat)` - Amplified normalization
- `normalize_exp(mat)` - Exponential normalization

Best for: Semi-supervised node classification

Complexity: $O(m \times i)$ where i is number of iterations

Related algorithms:

- *Personalized PageRank (PPR)* (personalized version)
- Label propagation in *Community Detection Algorithms*

Personalized PageRank (PPR)

Path: `network_classification.PPR`

Functions:

- `construct_PPR_matrix(graph_matrix, parallel)` - Build PPR matrix
- `construct_PPR_matrix_targets(graph_matrix, targets, parallel)` - PPR for specific targets
- `validate_ppr(...)` - Validation function

Best for: Local network exploration, personalized node ranking

Complexity: $O(n^2)$ for dense computation, can be approximated

Related algorithms:

- *Label Propagation* (similar propagation mechanism)
- PageRank in *Node Ranking*

Random Walks & Embeddings

Module: `py3plex.algorithms.general` and `py3plex.wrappers`

Random Walk Primitives

Path: `general.walkers`

Functions:

- `basic_random_walk(graph, start_node, walk_length)` - Simple random walk
- `node2vec_walk(graph, start_node, walk_length, p, q)` - Biased random walk
- `generate_walks(graph, num_walks, walk_length, strategy, ...)` - Generate multiple walks
- `general_random_walk(G, start_node, iterations, teleportation_prob)` - Random walk with restart
- `layer_specific_random_walk(network, start_node, start_layer, walk_length, ...)` - Multilayer walk

Best for: Foundation for embedding algorithms, network exploration

Complexity: $O(L)$ per walk where L is walk length

Related algorithms:

- *Node2Vec Embedding* (uses these walks)
- *DeepWalk Embedding* (uses these walks)

Node2Vec Embedding

Path: `wrappers.node2vec_embedding`

Functions:

- `generate_embeddings(graph, dimensions, walk_length, num_walks, p, q, ...)` - Generate embeddings
- `train_node2vec(graph, ...)` - Training wrapper

Best for: Feature learning, link prediction, node classification

Complexity: $O(r \times l \times V)$ where r is walks per node, l is walk length, V is number of vertices

Parameters:

- `p`: Return parameter (controls backtracking)

- `q`: In-out parameter (controls exploration vs exploitation)

Related algorithms:

- *DeepWalk Embedding* (special case with $p=1$, $q=1$)
- *Random Walk Primitives* (underlying walks)

DeepWalk Embedding

Implementation: `Node2Vec` with $p=1$, $q=1$

Best for: Simpler alternative to `Node2Vec` with uniform random walks

Related algorithms:

- *Node2Vec Embedding* (generalization)

Entanglement Analysis

Path: `multilayer_algorithms.entanglement`

Functions:

- `compute_entanglement_analysis(network)` - Full entanglement analysis
- `build_occurrence_matrix(network)` - Build node-layer occurrence matrix
- `compute_blocks(c_matrix)` - Compute block structure
- `compute_entanglement(block_matrix)` - Calculate entanglement metrics

Best for: Analyzing node-layer participation patterns

Purpose: Measure how nodes are entangled across layers

Related algorithms:

- *Versatility Centrality* (similar cross-layer analysis)

Visualization Algorithms

Module: `py3plex.visualization`

Layout Algorithms

Path: `visualization.layout_algorithms`

Available layouts:

- Force-directed layouts (Spring, Fruchterman-Reingold)
- Circular layout
- Spectral layout
- Hierarchical layout
- Diagonal projection (multilayer-specific)

Best for: Positioning nodes for visualization

Related algorithms:

- *Multilayer Visualization* (uses these layouts)

Multilayer Visualization

Path: `visualization.multilayer`

Main functions:

- `draw_multilayer_default(networks, ...)` - Default multilayer drawing
- `hairball_plot(network, layout_algorithm)` - Single-layer projection
- Diagonal projection plotting
- Layer-separated visualization

Best for: Visualizing multilayer network structure

Scalability:

- Diagonal projection: 10,000+ nodes
- Force-directed: <5,000 nodes
- Matrix view: 100,000+ nodes

Related algorithms:

- *Layout Algorithms* (positioning)

Drawing Machinery

Path: `visualization.drawing_machinery`

Low-level drawing functions and utilities.

Purpose: Support functions for rendering networks

Color Schemes

Path: `visualization.colors`

Functions for generating color schemes for communities, layers, node types.

Related algorithms:

- *Community Detection Algorithms* (uses colors)
- *Multilayer Visualization* (uses colors)

Embedding Visualization

Path: `visualization.embedding_visualization`

Functions for visualizing node embeddings in 2D/3D space.

Best for: Visualizing results of embedding algorithms

Related algorithms:

- *Node2Vec Embedding*
- *DeepWalk Embedding*

Benchmark & Utilities

Network Classification Benchmark

Path: `general.benchmark_classification`

Functions:

- `evaluate_oracle_F1(probs, Y_real)` - Evaluate classification performance

Best for: Evaluating node classification algorithms

Related algorithms:

- *Label Propagation*
- *Node2Vec Embedding*

Benchmark Visualizations

Path: `visualization.benchmark_visualizations`

Functions for visualizing algorithm comparison results.

Related algorithms:

- All algorithms (for benchmarking)

Algorithm Selection Guide

This section helps you choose the right algorithm category based on your task.

By Task Type

Community Detection:

→ See *Community Detection Algorithms*

Node Importance:

→ See *Centrality Measures*

Node Classification:

→ See *Label Propagation* or *Node2Vec Embedding*

Network Comparison:

→ See *Network Statistics*

Feature Learning:

→ See *Node2Vec Embedding*

Visualization:

→ See *Visualization Algorithms*

By Network Size

Small (<1,000 nodes):

- All algorithms work well
- Can use expensive path-based measures
- All visualization methods

Medium (1,000-10,000 nodes):

- Use Louvain over Infomap
- Avoid full betweenness centrality
- Use diagonal projection for visualization

Large (10,000-100,000 nodes):

- Degree-based measures only
- Louvain or label propagation
- Matrix visualizations

Very Large (>100,000 nodes):

- Degree, PageRank only
- Label propagation for communities
- Sparse matrix operations essential

By Network Type

Single-layer networks:

- Standard algorithms: Louvain, PageRank, betweenness
- See *Community Detection Algorithms* and *Centrality Measures*

Multiplex/multilayer networks:

- Multilayer-specific: *Multilayer Louvain*, *Versatility Centrality*
- Layer statistics: *Multilayer Statistics (Detailed)*
- Cross-layer measures

Directed networks:

- PageRank, HITS algorithm
- See *Node Ranking*

Weighted networks:

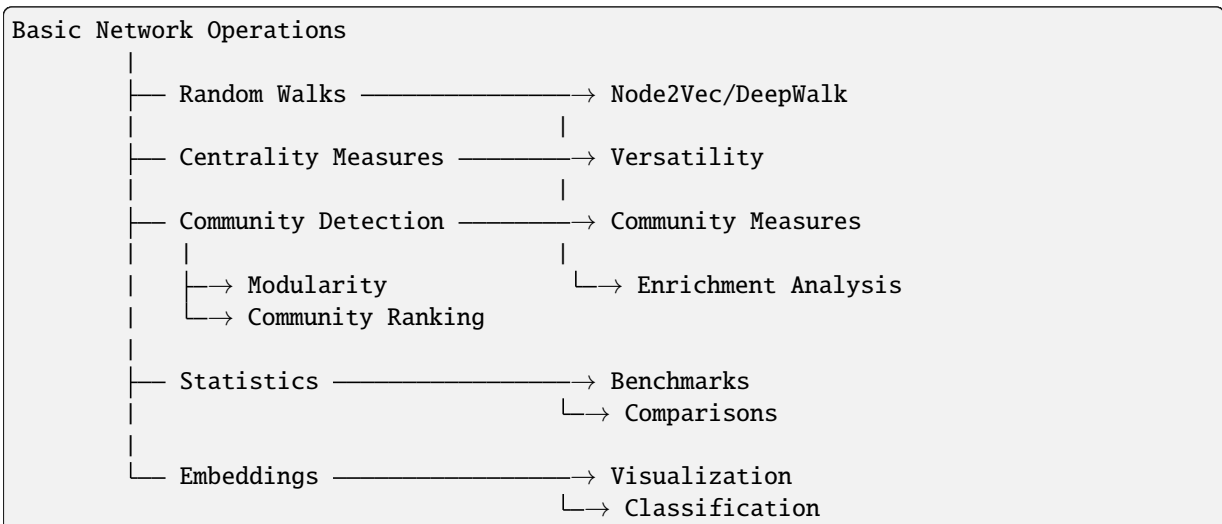
- All algorithms support weights
- Specify weight parameter

Temporal networks:

- Layer-based representation
- Time-respecting random walks: *Random Walk Primitives*

Algorithm Dependencies

This diagram shows how algorithms depend on each other:



Common Algorithm Workflows

Community Detection Workflow


```

# 1. Detect communities
from py3plex.algorithms.community_detection import louvain_multilayer
communities = louvain_multilayer(network.get_supra_adjacency_matrix())

# 2. Evaluate quality
from py3plex.algorithms.community_detection import multilayer_modularity
quality = multilayer_modularity(network.get_supra_adjacency_matrix(), communities)

# 3. Analyze communities
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.statistics.enrichment_modules import compute_enrichment

# 4. Visualize
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default([network], communities=communities)

```

Centrality Analysis Workflow

```

# 1. Compute centralities
from py3plex.algorithms.multilayer_algorithms.centrality import MultilayerCentrality
calc = MultilayerCentrality(network)

# 2. Multiple measures
degree = calc.overlapping_degree_centrality()
pagerank = calc.pagerank_centrality()
betweenness = calc.multilayer_betweenness_centrality()

# 3. Cross-layer analysis
from py3plex.algorithms.statistics import multilayer_statistics as mls
versatility = mls.versatility_centrality(network, centrality_type='degree')

# 4. Identify hubs
from py3plex.algorithms.statistics.basic_statistics import identify_n_hubs
hubs = identify_n_hubs(network, n=10, metric='degree')

```

Embedding Workflow

```

# 1. Generate walks
from py3plex.algorithms.general.walkers import generate_walks
walks = generate_walks(network.core_network, num_walks=10,
                       walk_length=80, strategy='node2vec')

# 2. Learn embeddings
from py3plex.wrappers.node2vec_embedding import generate_embeddings
embeddings = generate_embeddings(network.core_network,
                                dimensions=128, p=1.0, q=1.0)

# 3. Use for classification
from py3plex.algorithms.network_classification.label_propagation import label_
    ↳ propagation
# Or use embeddings directly with sklearn

```

Statistical Analysis Workflow

```
# 1. Layer-level statistics
from py3plex.algorithms.statistics import multilayer_statistics as mls
density = mls.layer_density(network, 'layer1')
overlap = mls.edge_overlap(network, 'layer1', 'layer2')
correlation = mls.inter_layer_degree_correlation(network, 'layer1', 'layer2')

# 2. Node-level statistics
for node in important_nodes:
    activity = mls.node_activity(network, node)
    degree_vec = mls.degree_vector(network, node)

# 3. Network-level statistics
interdep = mls.interdependence(network)
connectivity = mls.algebraic_connectivity(network)
```

Further Reading

For detailed usage examples and parameter descriptions:

- *Algorithm Landscape* - Algorithm selection guide with complexity analysis
- *Community Detection* - Community detection examples
- *Network Statistics* - Centrality computation examples
- *Performance and Scalability Best Practices* - Performance optimization tips
- *API Documentation* - Complete API reference

Citation Information

When using specific algorithm families, please cite the original papers:

Louvain & Multilayer Modularity:

Blondel et al. (2008), Mucha et al. (2010)

Node2Vec:

Grover & Leskovec (2016)

Infomap:

Rosvall & Bergstrom (2008)

See citation for complete citation information and BibTeX entries.

8.8.3 API Documentation

This section contains the complete API documentation for py3plex, automatically generated from docstrings.

For auto-generated module documentation, run:

```
cd docfiles
sphinx-apidoc -o AUTOGEN_results -f ../py3plex
make html
```

Core Modules

```
py3plex.core.multinet.ensure(*args, **kwargs)
```

```
py3plex.core.multinet.invariant(*args, **kwargs)
```

```
class py3plex.core.multinet.multi_layer_network(verbose: bool = True, network_type: str =
                                                'multilayer', directed: bool = True, dummy_layer:
                                                str = 'null', label_delimiter: str = '---',
                                                coupling_weight: int | float = 1)
```

Bases: object

Main class for multilayer network analysis and manipulation.

This class provides a comprehensive toolkit for creating, analyzing, and visualizing multilayer networks where nodes can exist in multiple layers and edges can connect nodes within or across layers.

Supported Network Types (network_type parameter):

- **multilayer** (default): General multilayer networks with arbitrary layer structure. Each layer can have a different set of nodes. Suitable for heterogeneous networks (e.g., authors-papers-venues) or networks where nodes naturally appear in only some layers.
- **multiplex**: Special case where all layers share the same node set but with different edge types. After loading a network, automatic coupling edges are created between each node and its counterparts in other layers. Suitable for social networks with multiple relationship types (e.g., friend, colleague, family layers).

Choosing the Right Network Type:

Key Features:

- Dict-based API for adding nodes and edges (see `add_nodes()` and `add_edges()`)
- NetworkX interoperability via `to_networkx()` and `from_networkx()`
- Multiple I/O formats (edgelist, GML, GraphML, gpickle, etc.)
- Visualization methods for multilayer layouts
- Community detection and centrality analysis
- Random walk and embedding generation

Hypergraph Support:

This class does NOT natively support true hypergraphs (edges connecting more than two nodes). For hypergraph-like structures, consider: - Using bipartite projections (nodes and hyperedges as separate node types) - The incidence gadget encoding via `to_homogeneous_hypergraph()` - External hypergraph libraries with conversion utilities

Notes

- Nodes in multilayer networks are represented as (node_id, layer) tuples
- Use `add_nodes()` and `add_edges()` with dict format for easiest interaction
- See examples/ directory for usage patterns and best practices

Examples

```
>>> # Create a general multilayer network (different node sets per layer)
>>> net = multi_layer_network(network_type='multilayer', directed=False)
>>>
>>> # Add nodes to different layers
>>> net.add_nodes([
...     {'source': 'A', 'type': 'social'},
...     {'source': 'B', 'type': 'social'},
...     {'source': 'A', 'type': 'email'} # Same node, different layer
... ])
>>>
>>> # Add edges (intra-layer and inter-layer)
>>> net.add_edges([
```

(continues on next page)

(continued from previous page)

```

...     {'source': 'A', 'target': 'B',
...       'source_type': 'social', 'target_type': 'social'},
...     {'source': 'A', 'target': 'A',
...       'source_type': 'social', 'target_type': 'email'}
... ])
>>>
>>> print(net) # Shows network statistics
<multi_layer_network: type=multilayer, directed=False, nodes=3, edges=2,
↳ layers=2>

```

```

>>> # Create a multiplex network (same nodes across relationship layers)
>>> # Note: coupling edges are auto-added after load_network()
>>> multiplex_net = multi_layer_network(network_type='multiplex')

```

add_dummy_layers()

Internal function, for conversion between objects

add_edges(*edge_dict_list: List[Dict] | List[List] | Tuple*, *input_type: str = 'dict'*) → *multi_layer_network*

Add edges to the multilayer network.

This method supports multiple input formats for specifying edges between nodes in different layers. The most common format is dict-based.

Parameters

- **edge_dict_list** – Edge data in one of the supported formats (see below)
- **input_type** – Format of edge data ('dict', 'list', or 'px_edge')

Returns

Returns self for method chaining

Return type

self

Supported Formats:

Dict format (recommended): `python {`

```

    'source': 'node1', # Source node ID 'target': 'node2', # Target node ID 'source_type':
    'layer1', # Source layer name 'target_type': 'layer2', # Target layer name (can be same
    as source) 'weight': 1.0, # Optional: edge weight 'type': 'interaction' # Optional: edge
    type/label

```

List format: `[node1, layer1, node2, layer2]`

px_edge format: `((node1, layer1), (node2, layer2), {'weight': 1.0})`

Examples

```

>>> # Add single intra-layer edge
>>> net = multi_layer_network()
>>> net.add_edges([
...     {'source': 'A',
...       'target': 'B',
...       'source_type': 'protein',
...       'target_type': 'protein'}
... ])
<multi_layer_network: type=multilayer, directed=True, nodes=2, edges=1,
↳ layers=1>

```

```
>>> # Method chaining
>>> net = multi_layer_network()
>>> net.add_edges([
...     {'source': 'A', 'target': 'B', 'source_type': 'layer1', 'target_type': 'layer1'}
... ])
>>> net.add_edges([
...     {'source': 'B', 'target': 'C', 'source_type': 'layer1', 'target_type': 'layer1'}
... ])
<multi_layer_network: type=multilayer, directed=True, nodes=3, edges=2, layers=1>
```

```
>>> # Add inter-layer edge with weight
>>> net.add_edges([
...     {'source': 'gene1',
...      'target': 'protein1',
...      'source_type': 'genes',
...      'target_type': 'proteins',
...      'weight': 0.95,
...      'type': 'expression'}
... ])
<multi_layer_network: type=multilayer, directed=True, nodes=5, edges=3, layers=3>
```

```
>>> # Add multiple edges at once
>>> edges = [
...     {'source': 'A', 'target': 'B', 'source_type': 'layer1', 'target_type': 'layer1'},
...     {'source': 'B', 'target': 'C', 'source_type': 'layer1', 'target_type': 'layer1'}
... ]
>>> net.add_edges(edges)
<multi_layer_network: type=multilayer, directed=True, nodes=5, edges=5, layers=3>
```

Raises

Exception – If `input_type` is not one of ‘dict’, ‘list’, or ‘px_edge’

Notes

- For intra-layer edges, use the same layer for `source_type` and `target_type`
- For inter-layer edges, use different layers
- Edge weights default to 1.0 if not specified

add_nodes(*node_dict_list: List[Dict] | Dict, input_type: str = 'dict'*) → *multi_layer_network*

Add nodes to the multilayer network.

Nodes in a multilayer network are identified by both their ID and the layer they belong to. This method adds nodes using a dict-based format.

Parameters

- **node_dict_list** – Node data as a dict or list of dicts (see format below)
- **input_type** – Format of node data (currently only ‘dict’ is supported)

Returns

Returns self for method chaining

Return type

self

Dict Format:

```
python {
    'source': 'node_id', # Node identifier (can be string or number) 'type': 'layer_name', #
    Layer this node belongs to 'weight': 1.0, # Optional: node weight/importance 'label':
    'display' # Optional: display label # ... any other node attributes
```

Examples

```
>>> # Add single node
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
<multi_layer_network: type=multilayer, directed=True, nodes=1, edges=0,
↳ layers=1>
```

```
>>> # Method chaining
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}]).add_nodes([{'source':
↳ 'B', 'type': 'layer1'}])
<multi_layer_network: type=multilayer, directed=True, nodes=2, edges=0,
↳ layers=1>
```

```
>>> # Add multiple nodes to the same layer
>>> nodes = [
...     {'source': 'A', 'type': 'protein'},
...     {'source': 'B', 'type': 'protein'},
...     {'source': 'C', 'type': 'protein'}
... ]
>>> net.add_nodes(nodes)
<multi_layer_network: type=multilayer, directed=True, nodes=5, edges=0,
↳ layers=2>
```

```
>>> # Add nodes with attributes
>>> net.add_nodes([
...     {'source': 'gene1',
...     'type': 'genes',
...     'weight': 0.8,
...     'label': 'BRCA1',
...     'chromosome': '17'}
... ])
<multi_layer_network: type=multilayer, directed=True, nodes=6, edges=0,
↳ layers=3>
```

```
>>> # Add nodes to multiple layers
>>> multi_layer_nodes = [
...     {'source': 'entity1', 'type': 'layer1'},
...     {'source': 'entity1', 'type': 'layer2'}, # Same entity, different
↳ layer
...     {'source': 'entity2', 'type': 'layer1'}
... ]
>>> net.add_nodes(multi_layer_nodes)
<multi_layer_network: type=multilayer, directed=True, nodes=9, edges=0,
↳ layers=5>
```

Notes

- The same node ID can exist in multiple layers
- Each (node_id, layer) combination is treated as a unique node
- Additional attributes beyond ‘source’ and ‘type’ are preserved
- Nodes must be added before edges referencing them

aggregate_edges(*metric='count', normalize_by='degree'*)

Edge aggregation method

Count weights across layers and return a weighted network

Parameters

- **param1** – aggregation operator (count is default)
- **param2** – normalization of the values

Returns

A simplified network.

basic_stats(*target_network=None*)

A method for obtaining a network’s statistics.

Displays: - Basic network info (nodes, edges) - Total unique nodes (counting each (node, layer) as unique) - Unique node IDs (across all layers) - Per-layer node counts

compute_ollivier_ricci(*mode: str = 'core', layers: List[Any] | None = None, alpha: float = 0.5, weight_attr: str = 'weight', curvature_attr: str = 'ricciCurvature', verbose: str = 'ERROR', backend_kwargs: Dict[str, Any] | None = None, inplace: bool = True, interlayer_weight: float = 1.0*) → Dict[str, Any]

Compute Ollivier-Ricci curvature on the multilayer network.

This method provides flexible computation of Ollivier-Ricci curvature at different levels of the multi-layer network:

- **core mode**: Compute curvature on the aggregated (flattened) network
- **layers mode**: Compute curvature separately for each layer
- **supra mode**: Compute curvature on the full supra-graph including both intra-layer and inter-layer edges

Parameters

- **mode** – Scope of computation. Options: “core”, “layers”, “supra”.
- **layers** – List of layer identifiers to process (only for mode=“layers”). If None, all layers are processed.
- **alpha** – Ollivier-Ricci parameter in [0, 1] controlling the mass distribution. Default: 0.5.
- **weight_attr** – Name of edge attribute containing weights. Default: “weight”.
- **curvature_attr** – Name of edge attribute to store curvature values. Default: “ricci-Curvature”.
- **verbose** – Verbosity level. Options: “INFO”, “DEBUG”, “ERROR”. Default: “ERROR”.
- **backend_kwargs** – Additional keyword arguments for OllivierRicci constructor.
- **inplace** – If True, update internal graphs. If False, return new graphs without modifying the network. Default: True.

- **interlayer_weight** – Weight for inter-layer coupling edges (only for mode="supra"). Default: 1.0.

Returns

Dictionary mapping scope identifiers to NetworkX graphs with computed curvatures: - mode="core": {"core": graph_with_curvature} - mode="layers": {layer_id: graph_with_curvature, ...} - mode="supra": {"supra": supra_graph_with_curvature}

Raises

- **RicciBackendNotAvailable** – If GraphRicciCurvature is not installed.
- **ValueError** – If mode is invalid or layers contains invalid identifiers.

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.add_edges([
...     ['A', 'layer1', 'B', 'layer1', 1],
...     ['B', 'layer1', 'C', 'layer1', 1],
... ], input_type="list")
>>>
>>> # Compute on aggregated network
>>> result = net.compute_ollivier_ricci(mode="core")
>>>
>>> # Compute per layer
>>> result = net.compute_ollivier_ricci(mode="layers")
>>>
>>> # Compute on supra-graph
>>> result = net.compute_ollivier_ricci(mode="supra", inplace=False)
```

compute_ollivier_ricci_flow(mode: str = 'core', layers: List[Any] | None = None, alpha: float = 0.5, iterations: int = 10, method: str = 'OTD', weight_attr: str = 'weight', curvature_attr: str = 'ricciCurvature', verbose: str = 'ERROR', backend_kwargs: Dict[str, Any] | None = None, inplace: bool = True, interlayer_weight: float = 1.0) → Dict[str, Any]

Compute Ollivier-Ricci flow on the multilayer network.

Ricci flow iteratively adjusts edge weights based on their Ricci curvature, effectively revealing and enhancing community structure. After Ricci flow, edges with negative curvature (community boundaries) have reduced weights, while edges with positive curvature have increased weights.

Parameters

- **mode** – Scope of computation. Options: "core", "layers", "supra".
- **layers** – List of layer identifiers to process (only for mode="layers"). If None, all layers are processed.
- **alpha** – Ollivier-Ricci parameter in [0, 1]. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **method** – Ricci flow method. Options: "OTD" (Optimal Transport Distance, recommended), "ATD" (Average Transport Distance). Default: "OTD".
- **weight_attr** – Name of edge attribute containing weights. After Ricci flow, these weights are updated to reflect the flow metric.
- **curvature_attr** – Name of edge attribute for curvature values. Default: "ricciCurvature".
- **verbose** – Verbosity level. Options: "INFO", "DEBUG", "ERROR". Default: "ERROR".

- **backend_kwargs** – Additional keyword arguments for OllivierRicci constructor.
- **inplace** – If True, update internal graphs. If False, return new graphs. Default: True.
- **interlayer_weight** – Weight for inter-layer coupling edges (only for mode="supra"). Default: 1.0.

Returns

Dictionary mapping scope identifiers to NetworkX graphs with Ricci flow applied: - mode="core": {"core": graph_with_flow} - mode="layers": {layer_id: graph_with_flow, ...} - mode="supra": {"supra": supra_graph_with_flow}

Raises

- **RicciBackendNotAvailable** – If GraphRicciCurvature is not installed.
- **ValueError** – If mode is invalid or layers contains invalid identifiers.

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.add_edges([
...     ['A', 'layer1', 'B', 'layer1', 1],
...     ['B', 'layer1', 'C', 'layer1', 1],
... ], input_type="list")
>>>
>>> # Apply Ricci flow to aggregated network
>>> result = net.compute_ollivier_ricci_flow(mode="core", iterations=20)
>>>
>>> # Apply to each layer
>>> result = net.compute_ollivier_ricci_flow(mode="layers", iterations=10)
```

property edge_count: int

Number of edges in the network.

Returns

Total count of edges

Return type

int

Examples

```
>>> net = multi_layer_network()
>>> net.add_edges([{'source': 'A', 'target': 'B',
...                 'source_type': 'layer1', 'target_type': 'layer1'}])
>>> net.edge_count
1
```

edges_from_temporal_table(edge_df)

Convert a temporal edge DataFrame to edge tuple list.

Extracts edges from a pandas DataFrame with temporal/activity information and converts them to a list of edge tuples suitable for network construction.

Parameters

edge_df – pandas DataFrame with columns: - node_first: Source node identifier - node_second: Target node identifier - layer_name: Layer identifier

Returns

List of edge tuples in format:

(node_first, node_second, layer_first, layer_second, weight) where weight is always 1

Return type

list

Notes

- All values are converted to strings
- All edges are assigned weight=1
- Both source and target are assumed to be in the same layer

Examples

```
>>> import pandas as pd
>>> net = multi_layer_network()
>>> df = pd.DataFrame({
...     'node_first': ['A', 'B'],
...     'node_second': ['B', 'C'],
...     'layer_name': ['L1', 'L1']
... })
>>> result = net.edges_from_temporal_table(df)
>>> len(result) >= 2
True
```

See also

fill_tmp_with_edges: Add these edges to layer graphs

execute_query(*query*: str) → Dict[str, Any]

Execute a DSL query on this multilayer network.

This is a convenience method that provides first-class access to the py3plex DSL (Domain-Specific Language) for querying multilayer networks.

Supports both SELECT and MATCH queries:

SELECT queries:

```
net.execute_query('SELECT nodes WHERE layer="transport"')
net.execute_query('SELECT * FROM nodes IN LAYER "ppi" WHERE degree > 10')
```

MATCH queries (Cypher-like):

```
net.execute_query('MATCH (g:Gene)-[r]->(t:Gene) RETURN g, t')
net.execute_query('MATCH (a)-[e]->(b) IN LAYER "ppi" WHERE a.degree > 5 RETURN a, b')
```

Parameters**query** – DSL query string**Returns**

- For SELECT queries: 'nodes' or 'edges' list, 'count', optional 'computed'
- For MATCH queries: 'bindings' list, 'count', 'type'

Return type

Dictionary containing query results

Raises

- **DSLSyntaxError** – If query syntax is invalid
- **DSLExecutionError** – If query cannot be executed

Examples

```
>>> net = multi_layer_network(directed=False)
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> net.add_edges([{'source': 'A', 'target': 'B',
...                 'source_type': 'layer1', 'target_type': 'layer1'}])
>>> result = net.execute_query('SELECT nodes WHERE layer="layer1"')
>>> result['count'] >= 0
True
```

```
>>> # Using MATCH syntax
>>> result = net.execute_query('MATCH (a:layer1)-[r]->(b:layer1) RETURN a, b
↪')
>>> 'bindings' in result
True
```

See also

- `py3plex.dsl.execute_query()` for standalone function
- `py3plex.dsl.format_result()` for formatting results

`fill_tmp_with_edges(edge_df)`

Fill temporary layer graphs with edges from a DataFrame.

Populates the emptied layer graphs (created by `remove_layer_edges`) with edges from a temporal/activity DataFrame. Useful for temporal network analysis where edge sets change over time.

Parameters

edge_df – pandas DataFrame with columns: - `node_first`: Source node identifier - `node_second`: Target node identifier - `layer_name`: Layer identifier

Notes

- Requires `remove_layer_edges()` to be called first
- Edges are grouped by layer
- Modifies `self.tmp_layers` in place
- Each edge is stored as `((node_first, layer), (node_second, layer))`

Raises

AttributeError – If `self.tmp_layers` doesn't exist (call `remove_layer_edges` first)

Examples

These examples require proper network setup and are for illustration only.

```
>>> import pandas as pd
>>> net = multi_layer_network()
>>> net.split_to_layers()
>>> net.remove_layer_edges()
>>> df = pd.DataFrame({
...     'node_first': ['A', 'B'],
...     'node_second': ['B', 'C'],
...     'layer_name': ['L1', 'L1']
... })
>>> net.fill_tmp_with_edges(df)
```

See also

`remove_layer_edges`: Creates empty layer graphs
`edges_from_temporal_table`: Convert DataFrame to edge list

classmethod `from_edges`(*edges*: List[Dict | List], *network_type*: str = 'multilayer', *directed*: bool = False, *input_type*: str = 'dict') → *multi_layer_network*

Create a multilayer network directly from a list of edges.

This is a convenience factory method that creates a network and populates it with edges in a single call, supporting method chaining patterns.

Parameters

- **edges** – List of edges in dict or list format
- **network_type** – Type of network ('multilayer' or 'multiplex')
- **directed** – Whether the network is directed
- **input_type** – Format of edge data ('dict' or 'list')

Returns

New network instance with edges added

Return type

multi_layer_network

Examples

```
>>> # Create from dict format
>>> net = multi_layer_network.from_edges([
...     {'source': 'A', 'target': 'B',
...      'source_type': 'layer1', 'target_type': 'layer1'},
...     {'source': 'B', 'target': 'C',
...      'source_type': 'layer1', 'target_type': 'layer1'}
... ])
>>> len(net)
3
```

```
>>> # Create from list format
>>> net = multi_layer_network.from_edges([
...     ['A', 'layer1', 'B', 'layer1', 1],
...     ['B', 'layer1', 'C', 'layer1', 1]
... ], input_type='list')
>>> net.edge_count
2
```

from_homogeneous_hypergraph(*H*)

Decode a homogeneous graph created by `to_homogeneous_hypergraph`.

This method reconstructs a multiplex network from its incidence gadget encoding. It identifies edge-nodes by their degree and cycle structure, then reconstructs the original layers based on cycle lengths (prime numbers).

Parameters

H(*networkx.Graph*) – Homogeneous graph created by `to_homogeneous_hypergraph()`.

Returns

- *dict* – Dictionary mapping layer names to lists of edges: {layer: [(u, v), ...]}
- *Examples* – Example requires proper network setup - for illustration only.

```

>>> network = multi_layer_network()
>>> network.add_layer("A")
>>> network.add_nodes([("1", "A"), ("2", "A")])
>>> network.add_edges([(("1", "A"), ("2", "A"))])
>>> H, node_map, edge_info = network.to_homogeneous_hypergraph()
>>> recovered = network.from_homogeneous_hypergraph(H)
>>> print(recovered)
{'layer_with_prime_2': [('1', '2')]}

```

Notes

The decoded layer names indicate the prime number used for encoding: - “layer_with_prime_2” corresponds to the first layer - “layer_with_prime_3” corresponds to the second layer, etc.

classmethod from_networkx(*G*: *Graph*, *network_type*: *str* = 'multilayer', *directed*: *bool* | *None* = *None*) → *multi_layer_network*

Create a multi_layer_network from a NetworkX graph.

This class method converts a NetworkX graph into a py3plex multi_layer_network. For multilayer networks, nodes should be tuples of (node_id, layer).

Parameters

- **G** – NetworkX graph to convert
- **network_type** – Type of network ('multilayer' or 'multiplex')
- **directed** – Whether to treat the network as directed. If None, inferred from G.

Returns

A new multi_layer_network instance

Return type

multi_layer_network

Examples

```

>>> import networkx as nx
>>> G = nx.Graph()
>>> G.add_nodes_from([('A', 'layer1'), ('B', 'layer1')])
>>> G.add_edge(('A', 'layer1'), ('B', 'layer1'))
>>> net = multi_layer_network.from_networkx(G)
>>> print(net)
<multi_layer_network: type=multilayer, directed=False, nodes=2, edges=1,
↳ layers=1>

```

Notes

- For proper multilayer behavior, ensure nodes are (node_id, layer) tuples
- Edge attributes are preserved during conversion
- The input graph is copied, not referenced

get_decomposition(*heuristic*='all', *cycle*=None, *parallel*=False, *alpha*=1, *beta*=0)

Core method for obtaining a network’s decomposition in terms of relations

get_decomposition_cycles(*cycle*=None)

A supporting method for obtaining decomposition triplets

get_degrees()

A simple wrapper which computes node degrees.

get_edges(data: bool = False, multiplex_edges: bool = False) → Any

Iterate over edges in the network.

This method behaves differently based on the network_type:

- **multilayer**: Returns all edges without filtering.
- **multiplex**: By default, filters out coupling edges (auto-generated inter-layer edges connecting each node to itself in other layers). Set multiplex_edges=True to include coupling edges.

Parameters

- **data** – If True, return edge data along with edge tuples
- **multiplex_edges** – If True, include coupling edges in multiplex networks. Only relevant when network_type='multiplex'. Coupling edges are automatically added to connect each node to its counterparts in other layers.

Yields

Edge tuples, optionally with data. For multiplex networks with multiplex_edges=False, coupling edges are excluded.

Raises

ValueError – If network_type is not 'multilayer' or 'multiplex'.

Examples

```
>>> net = multi_layer_network(network_type='multilayer')
>>> net.add_edges([
...     {'source': 'A', 'target': 'B',
...      'source_type': 'layer1', 'target_type': 'layer1'}
... ])
<multi_layer_network: type=multilayer, directed=True, nodes=2, edges=1, layers=1>
>>> list(net.get_edges())
[('A', 'layer1'), ('B', 'layer1')]
```

See also

- `__init__()` for the difference between multilayer and multiplex
- `_couple_all_edges()` for how coupling edges are created

get_label_matrix()

Return network labels

get_layers(style='diagonal', compute_layouts='force', layout_parameters=None, verbose=True)

A method for obtaining layerwise distributions

get_neighbors(node_id: str, layer_id: str | None = None) → Any

Get neighbors of a node in a specific layer.

Parameters

- **node_id** – Node identifier
- **layer_id** – Layer identifier (optional)

Returns

Iterator of neighbor nodes

get_nodes(*data: bool = False*) → Any

A method for obtaining a network's nodes

Parameters

data – If True, return node data along with node identifiers

Yields

Node identifiers, optionally with data

get_nx_object()

Return only core network with proper annotations

get_supra_adjacency_matrix(*mtype='sparse'*)

Get sparse representation of the supra matrix.

Parameters

mtype – 'sparse' or 'dense' - matrix representation type

Returns

Supra-adjacency matrix in requested format

Warning

For large multilayer networks, dense matrices can consume significant memory $(N \times L)^2 \times 8$ bytes for float64.

get_tensor(*sparsity_type='bsr'*)

TODO

get_unique_entity_counts()

Count unique entities in the network.

Returns

(**total_unique_nodes**, **unique_node_ids**, **nodes_per_layer**)

- **total_unique_nodes**: count of unique (node, layer) tuples
- **unique_node_ids**: count of unique node IDs (across all layers)
- **nodes_per_layer**: dict mapping layer to count of nodes in that layer

Return type

tuple

invert(*override_core=False*)

invert the nodes to edges. Get the “edge graph”. Each node is here an edge.

property is_empty: bool

Check if the network is empty (has no nodes).

Returns

True if network has no nodes

Return type

bool

Examples

```
>>> net = multi_layer_network()
>>> net.is_empty
True
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
```

(continues on next page)

(continued from previous page)

```
>>> net.is_empty
False
```

property layer_count: int

Number of unique layers in the network.

Returns

Count of distinct layers

Return type

int

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([
...     {'source': 'A', 'type': 'layer1'},
...     {'source': 'B', 'type': 'layer2'}
... ])
>>> net.layer_count
2
```

property layers: List[Any]

List of unique layer identifiers in the network.

Returns

Sorted list of layer identifiers

Return type

list

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([
...     {'source': 'A', 'type': 'social'},
...     {'source': 'B', 'type': 'work'}
... ])
>>> net.layers
['social', 'work']
```

load_embedding(embedding_file)

Embedding loading method

load_layer_name_mapping(mapping_name, header=False)

Layer-node mapping loader method

Parameters**param1** – The name of the mapping file.**Returns**

self.layer_name_map is filled, returns nothing.

**load_network(input_file: str | None = None, directed: bool = False, input_type: str = 'gml',
label_delimiter: str = '---') → multi_layer_network**

Load a network from file.

This method loads and prepares a given network. The behavior depends on the `network_type` set during initialization:

- **multilayer**: Network is loaded as-is. No automatic edges are added.

- **multiplex**: After loading, coupling edges are automatically created between each node and its counterparts in other layers. These edges have type='coupling' and can be filtered via `get_edges()`.

Parameters

- **input_file** – Path to the network file to load
- **directed** – Whether the network is directed
- **input_type** – Format of the input file. Supported values: - 'gml': Graph Modeling Language format - 'graphml': GraphML XML format - 'edgelist': Simple edge list (source target [weight]) - 'multiedgelist': Multilayer edge list (node1 layer1 node2 layer2 weight) - 'multiplex_edges': Multiplex format (layer node1 node2 weight) - 'multiplex_folder': Folder with layer files - 'gpickle': Python pickle format - 'nx': NetworkX graph object - 'sparse': Sparse matrix format
- **label_delimiter** – Delimiter used to separate layer names in node labels

Returns

Self for method chaining. Populates `self.core_network`, `self.labels`, and `self.activity`.

Note

For multiplex networks, use `input_type='multiplex_edges'` or `'multiplex_folder'` with `network_type='multiplex'` to get automatic coupling edges.

Examples

```
>>> # Load multilayer network (no automatic coupling)
>>> net = multi_layer_network(network_type='multilayer')
>>> net.load_network('data.gml', input_type='gml')
```

```
>>> # Load multiplex network (automatic coupling edges)
>>> net = multi_layer_network(network_type='multiplex')
>>> net.load_network('data.edges', input_type='multiplex_edges')
```

`load_network_activity(activity_file)`

Network activity loader

Args:

param1: The name of the generic activity file -> 65432 61888 1377688175 RE

, n1 n2 timestamp and layer name. Note that layer node mappings MUST be loaded in order to map nodes to activity properly.

Returns:

`self.activity` is filled.

`load_temporal_edge_information(input_file=None, input_type='edge_activity', directed=False, layer_mapping=None)`

A method for loading temporal edge information

`merge_with(target_px_object)`

Merge two px objects.

`monitor(message)`

A simple monitor method for logging

`monoplex_nx_wrapper(method, kwargs=None)`

A generic networkx function wrapper.

Parameters

- **method** (*str*) – Name of the NetworkX function to call (e.g., ‘degree_centrality’, ‘betweenness_centrality’)
- **kwargs** (*dict*, *optional*) – Keyword arguments to pass to the NetworkX function. For example, for betweenness_centrality you can pass: - weight: Edge attribute to use as weight - normalized: Whether to normalize betweenness values - distance: Edge attribute to use as distance (for closeness_centrality)

Returns

The result of the NetworkX function call.

Raises

AttributeError – If the specified method does not exist in NetworkX.

Example

```
# Unweighted betweenness centrality centralities = network.monoplex_nx_wrapper("betweenness_centrality")
# Weighted betweenness centrality centralities = network.monoplex_nx_wrapper("betweenness_centrality",
kwargs={"weight": "weight"})
# With multiple parameters centralities = network.monoplex_nx_wrapper("betweenness_centrality",
kwargs={"weight": "weight", "normalized": True})
```

property node_count: int

Number of nodes in the network.

Returns

Total count of nodes (node-layer pairs)

Return type

int

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> net.node_count
1
```

read_ground_truth_communities(*cfile*)

Parse ground truth community file and make mappings to the original nodes. This works based on node ID mappings, exact node,layer tuples are to be added. :param param1: ground truth communities.

Returns

self.ground_truth_communities

remove_edges(*edge_dict_list*: List[Dict] | List[List], *input_type*: str = 'list') → None

A method for removing edges..

Parameters

- **edge_dict_list** – Edge data in dict or list format
- **input_type** – Format of edge data ('dict' or 'list')

Raises

Exception – If input_type is not valid

remove_layer_edges()

Remove all edges from separate layer graphs while keeping nodes.

This method creates empty copies of each layer graph with all nodes intact but no edges. Useful for reconstructing networks with different edge sets or for temporal network analysis.

Notes

- Requires `split_to_layers()` to be called first
- Stores empty layer graphs in `self.tmp_layers`
- Original graphs in `self.separate_layers` remain unchanged
- All nodes and their attributes are preserved

Raises

RuntimeError – If `split_to_layers()` hasn't been called yet

See also

`split_to_layers`: Must be called before this method
`fill_tmp_with_edges`: Add edges back to emptied layers

remove_nodes(*node_dict_list*, *input_type*='dict')

Remove nodes from the network

save_network(*output_file*=None, *output_type*='edgelist')

Save the network to a file in various formats.

This method exports the multilayer network to different file formats for persistence, sharing, or use with other tools.

Parameters

- **output_file** – Path where the network should be saved
- **output_type** – Format for saving ('edgelist', 'multiedgelist', 'multiedgelist_encoded', or 'pickle')

Supported Formats:

- 'edgelist': Simple edge list format (standard NetworkX)
- 'multiedgelist': Multilayer edge list with layer information
- 'multiedgelist_encoded': Multilayer edge list with integer encoding
- 'pickle': Python pickle format (preserves all attributes)

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> net.add_edges([{'source': 'A', 'target': 'B',
...                  'source_type': 'layer1', 'target_type': 'layer1'}])
>>> net.save_network('network.txt', output_type='multiedgelist')
```

```
>>> # For faster I/O with all metadata preserved
>>> net.save_network('network.gpickle', output_type='gpickle')
```

Notes

- 'gpickle' format preserves all node/edge attributes
- 'multiedgelist_encoded' creates `node_map` and `layer_map` attributes
- Edge weights and types are preserved in supported formats

```
serialize_to_edgelist(edgelist_file='./tmp/tmpedgelist.txt', tmp_folder='tmp', out_folder='out',  
                      multiplex=False)
```

Serialize the multilayer network to an edgelist file.

Converts the network to a numeric edgelist format suitable for external tools and algorithms that require integer node/layer identifiers.

Parameters

- **edgelist_file** – Path to output edgelist file (default: “./tmp/tmpedgelist.txt”)
- **tmp_folder** – Temporary folder for intermediate files (default: “tmp”)
- **out_folder** – Output folder for results (default: “out”)
- **multiplex** – If True, use multiplex format (node layer node layer weight) If False, use simple edgelist format (node1 node2 weight)

Returns

Inverse node mapping (numeric_id -> original_node_tuple)

Use this to decode results from external algorithms

Return type

dict

File Formats:

- Multiplex format: node1_id layer1_id node2_id layer2_id weight
- Simple format: node1_id node2_id weight

Notes

- Creates tmp_folder and out_folder if they don’t exist
- Nodes are mapped to sequential integers starting from 0
- Layers are mapped to sequential integers starting from 0 (multiplex mode)
- All edges have weight 1 unless explicitly specified

Examples

Example requires file output - for illustration only.

```
>>> net = multi_layer_network()  
>>> # ... build network ...  
>>> node_mapping = net.serialize_to_edgelist(  
...     edgelist_file='network.txt',  
...     multiplex=True  
... )  
>>> # Use node_mapping to decode results  
>>> original_node = node_mapping[0] # Get original node for id 0
```

See also

load_network: Load networks from file save_network: Alternative serialization method

```
sparse_to_px(directed=None)
```

Convert sparse matrix to py3plex format

Parameters

directed – Whether the network is directed (uses self.directed if None)

split_to_layers(*style='diagonal', compute_layouts='force', layout_parameters=None, verbose=True, multiplex=False, convert_to_simple=False*)

A method for obtaining layerwise distributions

subnetwork(*input_list=None, subset_by='node_layer_names'*)

Construct a subgraph based on a set of nodes.

summary()

Generate a summary of network statistics.

Computes and returns key metrics about the multilayer network structure.

Returns

Network statistics including:

- 'Number of layers': Count of unique layers
- 'Nodes': Total number of nodes
- 'Edges': Total number of edges
- 'Mean degree': Average node degree
- 'CC': Number of connected components

Return type

dict

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> net.add_edges([{'source': 'A', 'target': 'B',
...                 'source_type': 'layer1', 'target_type': 'layer1'}])
>>> stats = net.summary()
>>> print(f"Network has {stats['Nodes']} nodes and {stats['Edges']} edges")
Network has 2 nodes and 1 edges
```

Notes

- Connected components are computed on the undirected version
- Mean degree is averaged across all nodes in all layers

test_scale_free()

Test the scale-free-nness of the network

to_homogeneous_hypergraph()

Transform a multiplex network into a homogeneous graph using incidence gadget encoding.

This method encodes the multiplex structure where each layer is represented by a unique prime number signature. Each edge becomes an edge-node connected to its endpoints and a cycle of length prime-1 that encodes the layer.

Returns

- *tuple* (*H, node_mapping, edge_info*) –

H

[networkx.Graph] Homogeneous unlabeled graph encoding the multiplex structure.

node_mapping

[dict] Maps each original node to its vertex-node in H.

edge_info

[dict] Mapping from each edge-node in H to its (layer, endpoints) tuple.

- *Examples* – Example requires sympy dependency - for illustration only.
- `>>> network = multi_layer_network(directed=False) # doctest (+SKIP)`
- `>>> network.add_nodes([{'source': '1', 'type': 'A'}, {'source': '2', 'type': 'A'}], input_type='dict') # doctest: +SKIP)`
- `>>> network.add_edges([{'source': '1', 'target': '2', 'source_type': 'A', 'target_type': 'A'}], input_type='dict') # doctest: +SKIP)`
- `>>> H, node_map, edge_info = network.to_homogeneous_hypergraph() # doctest (+SKIP)`
- `>>> print(f"Homogeneous graph has {len(H.nodes())} nodes") # doctest (+SKIP)`

Notes

This transformation uses prime-based signatures to encode layers: - Each layer is assigned a unique prime number (2, 3, 5, 7, ...) - Each edge in layer with prime p is connected to a cycle of length p - The cycle structure uniquely identifies the layer

to_json()

A method for exporting the graph to a json file

Args: self

to_networkx() → Graph

Convert the multilayer network to a NetworkX graph.

Returns a copy of the core network as a NetworkX graph. The returned graph preserves all node and edge attributes, including layer information for multilayer networks (where nodes are typically (node_id, layer) tuples).

Returns

A NetworkX graph (MultiGraph or MultiDiGraph depending on network type)

Return type

nx.Graph

Examples

```
>>> net = multi_layer_network(directed=False)
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> nx_graph = net.to_networkx()
>>> print(type(nx_graph))
<class 'networkx.classes.multigraph.MultiGraph'>
```

Notes

- For multilayer networks, nodes are tuples: (node_id, layer)
- All edge attributes (weight, type, etc.) are preserved
- The returned graph is a copy, not a reference

to_sparse_matrix(replace_core=False, return_only=False)

Conver the matrix to scipy-sparse version. This is useful for classification.

visualize_matrix(kwarg=None)

Plot the matrix – this plots the supra-adjacency matrix

```
visualize_network(style='diagonal', parameters_layers=None, parameters_multiedges=None,
                  show=False, compute_layouts='force', layouts_parameters=None, verbose=True,
                  orientation='upper', resolution=0.01, axis=None, fig=None, no_labels=False,
                  linewidth=1.7, alphachannel=0.3, linepoints='-', legend=False)
```

Visualize the multilayer network.

Supports multiple visualization styles: - 'diagonal': Layer-centric diagonal layout with inter-layer edges - 'hairball': Aggregate hairball plot of all layers - 'flow' or 'alluvial': Layered flow visualization with horizontal bands - 'sankey': Sankey diagram showing inter-layer flow strength

Parameters

- **style** – Visualization style ('diagonal', 'hairball', 'flow', 'alluvial', or 'sankey')
- **parameters_layers** – Custom parameters for layer drawing
- **parameters_multiedges** – Custom parameters for edge drawing
- **show** – Show plot immediately
- **compute_layouts** – Layout algorithm (currently unused)
- **layouts_parameters** – Layout parameters (currently unused)
- **verbose** – Enable verbose output
- **orientation** – Edge orientation for diagonal style
- **resolution** – Resolution for edge curves
- **axis** – Optional matplotlib axis to draw on
- **fig** – Optional matplotlib figure (currently unused)
- **no_labels** – Hide network labels
- **linewidth** – Width of edge lines
- **alphachannel** – Alpha channel for edge transparency
- **linepoints** – Line style for edges
- **legend** – Show legend (for hairball style)

Returns

Matplotlib axis object

Raises

Exception – If style is not recognized

Performance Notes:

For large networks (>500 nodes), visualization performance may degrade: - Layout computation can be slow ($O(n^2)$ for force-directed layouts) - Rendering many edges is memory and CPU intensive - Consider filtering or sampling for exploratory visualization - Use simpler layouts or increase layout iteration limits

Approximate rendering times on typical hardware: - 100 nodes: <1 second - 500 nodes: 5-10 seconds - 1000 nodes: 30-60 seconds - 5000+ nodes: Several minutes, may run out of memory

```
visualize_ricci_core(alpha: float = 0.5, iterations: int = 10, layout_type: str = 'mds', dim: int = 2,
                    **kwargs)
```

Visualize the aggregated core network using Ricci-flow-based layout.

This method is a high-level wrapper for Ricci-flow-based visualization of the core (aggregated) network. It automatically computes Ricci flow if not already done and creates an informative layout that emphasizes geometric structure and communities.

Parameters

- **alpha** – Ollivier-Ricci parameter for flow computation. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **layout_type** – Layout algorithm (“mds”, “spring”, “spectral”). Default: “mds”.
- **dim** – Dimensionality of layout (2 or 3). Default: 2.
- ****kwargs** – Additional arguments passed to `visualize_multilayer_ricci_core`.

Returns

Tuple of (figure, axes, positions_dict).

Raises

RicciBackendNotAvailable – If `GraphRicciCurvature` is not installed.

Examples

Example requires `GraphRicciCurvature` library - for illustration only.

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.add_edges([
...     ['A', 'layer1', 'B', 'layer1', 1],
...     ['B', 'layer1', 'C', 'layer1', 1],
... ], input_type="list")
>>> fig, ax, pos = net.visualize_ricci_core()
>>> import matplotlib.pyplot as plt
>>> plt.show()
```

See also

`visualize_ricci_layers`: Per-layer visualization with Ricci flow `visualize_ricci_supra`: Supra-graph visualization with Ricci flow

visualize_ricci_layers(*layers: List[Any] | None = None, alpha: float = 0.5, iterations: int = 10, layout_type: str = 'mds', share_layout: bool = True, **kwargs*)

Visualize individual layers using Ricci-flow-based layouts.

This method creates visualizations of individual layers with layouts derived from Ricci flow. Layers can share a common coordinate system (for easier comparison) or have independent layouts.

Parameters

- **layers** – List of layer identifiers to visualize. If `None`, uses all layers.
- **alpha** – Ollivier-Ricci parameter. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **layout_type** – Layout algorithm. Default: “mds”.
- **share_layout** – If `True`, use shared coordinates across layers. Default: `True`.
- ****kwargs** – Additional arguments passed to `visualize_multilayer_ricci_layers`.

Returns

Tuple of (figure, layer_positions_dict).

Raises

RicciBackendNotAvailable – If `GraphRicciCurvature` is not installed.

Examples

```
>>> fig, pos_dict = net.visualize_ricci_layers(
...     arrangement="grid", share_layout=True
... )
>>> import matplotlib.pyplot as plt
>>> plt.show()
```

See also

`visualize_ricci_core`: Core network visualization with Ricci flow `visualize_ricci_supra`: Supra-graph visualization with Ricci flow

visualize_ricci_supra(*alpha*: float = 0.5, *iterations*: int = 10, *layout_type*: str = 'mds', *dim*: int = 2, ***kwargs*)

Visualize the full supra-graph using Ricci-flow-based layout.

This method visualizes the complete multilayer structure including both intra-layer edges (within layers) and inter-layer edges (coupling between layers) using a layout derived from Ricci flow.

Parameters

- **alpha** – Ollivier-Ricci parameter. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **layout_type** – Layout algorithm. Default: “mds”.
- **dim** – Dimensionality (2 or 3). Default: 2.
- ****kwargs** – Additional arguments passed to `visualize_multilayer_ricci_supra`.

Returns

Tuple of (figure, axes, positions_dict).

Raises

RicciBackendNotAvailable – If `GraphRicciCurvature` is not installed.

Examples

```
>>> fig, ax, pos = net.visualize_ricci_supra(dim=3)
>>> import matplotlib.pyplot as plt
>>> plt.show()
```

See also

`visualize_ricci_core`: Core network visualization with Ricci flow `visualize_ricci_layers`: Per-layer visualization with Ricci flow

`py3plex.core.multinet.require(*args, **kwargs)`

`py3plex.core.parsers.ensure(*args, **kwargs)`

`py3plex.core.parsers.load_edge_activity_file(fname: str, layer_mapping: str | None = None) → DataFrame`

`py3plex.core.parsers.load_edge_activity_raw(activity_file: str, layer_mappings: dict) → DataFrame`

Basic parser for loading generic activity files. Here, temporal edges are given as tuples -> this can be easily transformed for example into a pandas dataframe!

Parameters

- **activity_file** – Path to activity file
- **layer_mappings** – Dictionary mapping layer names to IDs

Returns

DataFrame with edge activity data

```
py3plex.core.parsers.load_temporal_edge_information(input_network: str, input_type: str,  
                                                  layer_mapping: str | None = None) →  
                                                  DataFrame | None
```

```
py3plex.core.parsers.parse_detangler_json(file_path: str, directed: bool = False) →  
                                         Tuple[MultiGraph | MultiDiGraph, None]
```

Parser for generic Detangler files :param file_path: Path to Detangler JSON file :param directed: Whether to create directed graph

```
py3plex.core.parsers.parse_edgelist_multi_types(input_name: str, directed: bool) →  
                                                Tuple[MultiGraph | MultiDiGraph, None]
```

Parse an edgelist file with multiple edge types.

Reads a text file where each line represents an edge, optionally with weights and edge types. Lines starting with '#' are treated as comments.

File Format:

node1 node2 [weight] [edge_type]

Lines starting with '#' are ignored (comments)

Parameters

- **input_name** – Path to edgelist file
- **directed** – Whether to create a directed graph

Returns

(parsed_graph, None for labels)

Return type

Tuple[Union[nx.MultiGraph, nx.MultiDiGraph], None]

Notes

- All nodes are assigned to a “null” layer
- Default weight is 1 if not specified
- Edge type is optional (4th column)
- Handles both 2-column (node pairs) and 3+ column formats

Examples

```
>>> # File content:  
>>> # A B 1.0 friendship  
>>> # B C 2.0 collaboration  
>>> graph, _ = parse_edgelist_multi_types('edges.txt', directed=False)
```

```
py3plex.core.parsers.parse_embedding(input_name: str) → Tuple[ndarray, ndarray]
```

Loader for generic embedding as outputed by GenSim

Parameters

input_name – Path to embedding file

Returns

Tuple of (embedding matrix, embedding indices)

`py3plex.core.parsers.parse_gml(file_name: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

Parse a gml network.

Parameters

- **file_name** – Path to GML file
- **directed** – Whether to create directed graph

Returns

Tuple of (multigraph, possible labels)

Contracts:

- Precondition: file_name must be a non-empty string
- Postcondition: result graph is not None
- Postcondition: result is a MultiGraph or MultiDiGraph

`py3plex.core.parsers.parse_gpickle(file_name: str, directed: bool = False, layer_separator: str | None = None) → Tuple[MultiGraph | MultiDiGraph, None]`

A parser for generic Gpickle as stored by Py3plex.

Parameters

- **file_name** – Path to gpickle file
- **directed** – Whether to create directed graph
- **layer_separator** – Optional separator for layer parsing

Contracts:

- Precondition: file_name must be a non-empty string
- Postcondition: result graph is not None
- Postcondition: result is a MultiGraph or MultiDiGraph

`py3plex.core.parsers.parse_gpickle_biomine(file_name: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

Gpickle parser for biomine graphs :param file_name: Path to gpickle containing BioMine data :param directed: Whether to create directed graph

`py3plex.core.parsers.parse_matrix(file_name: str, directed: bool) → Tuple[Any, Any]`

Parser for matrices.

Parameters

- **file_name** – Path to .mat file
- **directed** – Whether the graph is directed

Returns

Tuple of (network, group) from the .mat file

Contracts:

- Precondition: file_name must be a non-empty string
- Postcondition: network must not be None

`py3plex.core.parsers.parse_matrix_to_nx(file_name: str, directed: bool) → Graph | DiGraph`

Parser for matrices to NetworkX graph.

Parameters

- **file_name** – Path to .mat file
- **directed** – Whether to create directed graph

Returns

NetworkX Graph or DiGraph

`py3plex.core.parsers.parse_multi_edgelist(input_name: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

A generic multiedgelist parser n l n l w :param input_name: Path to text file containing multiedges :param directed: Whether to create directed graph

`py3plex.core.parsers.parse_multiedge_tuple_list(network: list, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

Parse a list of edge tuples into a multilayer network.

Parameters

- **network** – List of edge tuples (node_first, node_second, layer_first, layer_second, weight)
- **directed** – Whether to create directed graph

`py3plex.core.parsers.parse_multiplex_edges(input_name: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

Parse a multiplex edgelist file where each line specifies layer and edge.

File Format:

layer node1 node2 [weight]

Each line: layer_id source_node target_node [optional_weight]

Parameters

- **input_name** – Path to multiplex edgelist file
- **directed** – Whether to create a directed graph

Returns

(parsed_graph, None for labels)

Return type

Tuple[Union[nx.MultiGraph, nx.MultiDiGraph], None]

Notes

- Each edge belongs to a specific layer (first column)
- Nodes are represented as (node_id, layer) tuples
- Default weight is 1 if not specified
- All edges have type='default' attribute
- Automatically couples nodes across layers for multiplex structure

Examples

```
>>> # File content:
>>> # layer1 A B 1.5
>>> # layer2 A B 2.0
```

(continues on next page)

(continued from previous page)

```
>>> # layer1 B C 1.0
>>> graph, _ = parse_multiplex_edges('multiplex.txt', directed=False)
>>> # Creates nodes: (A, layer1), (A, layer2), (B, layer1), etc.
```

`py3plex.core.parsers.parse_multiplex_folder(input_folder: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None, DataFrame]`

Parse a folder containing multiplex network files.

Expects a folder with specific file formats for edges, layers, and optional activity.

Expected Files:

- *.edges: Edge information (format: layer_id node1 node2 weight)
- layers.txt: Layer definitions (format: layer_id layer_name)
- activity.txt: Optional temporal activity (format: node1 node2 timestamp layer_name)

Parameters

- **input_folder** – Path to folder containing multiplex network files
- **directed** – Whether to create a directed graph

Returns

- Union[nx.MultiGraph, nx.MultiDiGraph]: Parsed multilayer graph
- None: Placeholder for labels (not used)
- pd.DataFrame: Time series activity data (empty if no activity.txt)

Return type

Tuple containing

Notes

- Uses glob to find files with specific extensions
- Layer mapping is built from layers.txt
- Activity data is optional and returned as pandas DataFrame
- Nodes are represented as (node_id, layer_id) tuples

Examples

```
>>> # Folder structure:
>>> # my_network/
>>> #   network.edges
>>> #   layers.txt
>>> #   activity.txt (optional)
>>> graph, _, activity_df = parse_multiplex_folder('my_network/', directed=False)
```

`py3plex.core.parsers.parse_network(input_name: str | Any, f_type: str = 'gml', directed: bool = False, label_delimiter: str | None = None, network_type: str = 'multilayer') → Tuple[Any, Any, Any]`

A wrapper method for available parsers!

Parameters

- **input_name** – Path to network file or network object
- **f_type** – Type of file format to parse
- **directed** – Whether to create directed graph

- **label_delimiter** – Optional delimiter for labels
- **network_type** – Type of network (multilayer or multiplex)

Returns

Tuple of (parsed_network, labels, time_series)

`py3plex.core.parsers.parse_nx(nx_object: Graph, directed: bool) → Tuple[Graph, None]`

Core parser for networkx objects.

Parameters

- **nx_object** – A networkx graph
- **directed** – Whether the graph is directed

Returns

Tuple of (graph, None)

Contracts:

- Precondition: nx_object must not be None
- Precondition: nx_object must be a NetworkX graph
- Postcondition: result graph is not None

`py3plex.core.parsers.parse_simple_edgelist(input_name: str, directed: bool) → Tuple[Graph | DiGraph, None]`

Simple edgelist n n w :param input_name: Path to text file :param directed: Whether to create directed graph

`py3plex.core.parsers.parse_spin_edgelist(input_name: str, directed: bool) → Tuple[Graph, None]`

Parse SPIN format edgelist file.

SPIN format includes node pairs with edge tags and optional weights.

File Format:

node1 node2 tag [weight]

Each line: source_node target_node edge_tag [optional_weight]

Parameters

- **input_name** – Path to SPIN edgelist file
- **directed** – Whether to create directed graph (currently creates undirected)

Returns

(parsed_graph, None for labels)

Return type

Tuple[nx.Graph, None]

Notes

- Currently always returns nx.Graph (undirected) regardless of directed parameter
- Edge tag is stored in edge 'type' attribute
- Default weight is 1 if not specified (4th column)

Examples

```
>>> # File content:
>>> # A B protein_interaction 0.95
>>> # B C gene_regulation 0.80
>>> graph, _ = parse_spin_edgelist('spin_edges.txt', directed=False)
```

`py3plex.core.parsers.require(*args, **kwargs)`

`py3plex.core.parsers.save_edgelist(input_network: Graph, output_file: str, attributes: bool = False) → None`

Save network to edgelist format.

For multilayer networks (where nodes are tuples of (node_id, layer)), saves in format: node1 layer1 node2 layer2

For regular networks, saves in format: node1 node2

`py3plex.core.parsers.save_gpickle(input_network: Any, output_file: str) → None`

`py3plex.core.parsers.save_multiedgelist(input_network: Any, output_file: str, attributes: bool = False, encode_with_ints: bool = False) → Tuple[Dict[Any, str], Dict[Any, str]] | None`

Save multiedgelist – as n1, l1, n2, l2, w

Returns

When `encode_with_ints` is `True`, returns tuple of (node_encodings, type_encodings) Otherwise returns `None`

`py3plex.core.converters.compute_layout(network: Graph, compute_layouts: str, layout_parameters: Dict[str, Any] | None, verbose: bool) → Graph`

Compute and normalize layout for a network.

Parameters

- **network** – NetworkX graph to compute layout for
- **compute_layouts** – Layout algorithm to use ('force', 'random', 'custom_coordinates')
- **layout_parameters** – Optional parameters for layout algorithms
- **verbose** – Whether to print verbose output

Returns

Network with 'pos' attribute added to nodes

Contracts:

- Precondition: network must not be `None` and must have at least one node
- Precondition: `compute_layouts` must be a valid algorithm name
- Postcondition: all nodes have 'pos' attribute (layout preserves nodes)

`py3plex.core.converters.ensure(*args, **kwargs)`

`py3plex.core.converters.prepare_for_parsing(multinet)`

Compute layout for a hairball visualization

Parameters

param1 (*obj*) – multilayer object

Returns

(names, prepared network)

Return type

tuple

`py3plex.core.converters.prepare_for_visualization(multinet: Graph, network_type: str = 'multilayer', compute_layouts: str = 'force', layout_parameters: Dict[str, Any] | None = None, verbose: bool = True, multiplex: bool = False) → Tuple[List[Any], List[Graph], Any]`

This functions takes a multilayer object and returns individual layers, their names, as well as multilayer edges spanning over multiple layers.

Parameters

- **multinet** – multilayer network object
- **network_type** – “multilayer” or “multiplex”
- **compute_layouts** – Layout algorithm (‘force’, ‘random’, etc.)
- **layout_parameters** – Optional layout parameters
- **verbose** – Whether to print progress information
- **multiplex** – Whether to treat as multiplex network

Returns

(layer_names, layer_networks_list, multiedges)

- layer_names: List of layer names
- layer_networks_list: List of NetworkX graph objects for each layer
- multiedges: Dictionary of edges spanning multiple layers

Return type

tuple

`py3plex.core.converters.prepare_for_visualization_hairball(multinet, compute_layouts=False)`

Compute layout for a hairball visualization

Parameters

param1 (*obj*) – multilayer object

Returns

(names, prepared network)

Return type

tuple

`py3plex.core.converters.require(*args, **kwargs)`

`py3plex.core.random_generators.ensure(*args, **kwargs)`

`py3plex.core.random_generators.random_multilayer_ER(n: int, l: int, p: float, directed: bool = False) → Any`

Generate random multilayer Erdős-Rényi network.

Parameters

- **n** – Number of nodes (must be positive)
- **l** – Number of layers (must be positive)
- **p** – Edge probability in [0, 1]
- **directed** – If True, generate directed network

Returns

multi_layer_network object

Contracts:

- Precondition: $n > 0$ - must have at least one node
- Precondition: $l > 0$ - must have at least one layer
- Precondition: $0 \leq p \leq 1$ - probability must be valid

- Postcondition: result is not None - must return valid network

`py3plex.core.random_generators.random_multiplex_ER(n: int, l: int, p: float, directed: bool = False) → Any`

Generate random multiplex Erdős-Rényi network.

Parameters

- **n** – Number of nodes (must be positive)
- **l** – Number of layers (must be positive)
- **p** – Edge probability in [0, 1]
- **directed** – If True, generate directed network

Returns

multi_layer_network object

Contracts:

- Precondition: $n > 0$ - must have at least one node
- Precondition: $l > 0$ - must have at least one layer
- Precondition: $0 \leq p \leq 1$ - probability must be valid
- Postcondition: result is not None - must return valid network

`py3plex.core.random_generators.random_multiplex_generator(n: int, m: int, d: float = 0.9) → MultiGraph`

Generate a multiplex network from a random bipartite graph.

Parameters

- **n** – Number of nodes (must be positive)
- **m** – Number of layers (must be positive)
- **d** – Layer dropout to avoid cliques, range [0..1] (default: 0.9)

Returns

Generated multiplex network as a MultiGraph

Contracts:

- Precondition: $n > 0$ - must have at least one node
- Precondition: $m > 0$ - must have at least one layer
- Precondition: $0 \leq d \leq 1$ - dropout must be valid probability
- Postcondition: result is not None
- Postcondition: result is a NetworkX MultiGraph

`py3plex.core.random_generators.require(*args, **kwargs)`

Supporting methods for parsers and converters.

This module provides utility functions for network parsing and conversion, including layer splitting, multiplex edge addition, and GAF parsing.

`py3plex.core.supporting.add_mpx_edges(input_network: Graph) → Graph`

Add multiplex edges between corresponding nodes across layers.

Multiplex edges connect nodes representing the same entity across different layers of a multilayer network.

Parameters

input_network – NetworkX graph with multilayer structure.

Returns

Network with added multiplex edges between corresponding nodes.

Contracts:

- Precondition: `input_network` must not be `None`
- Precondition: `input_network` must be a NetworkX graph
- Postcondition: result is a NetworkX graph

Example

```
>>> network = nx.Graph()
>>> network.add_node(('A', 'layer1'))
>>> network.add_node(('A', 'layer2'))
>>> network = add_mpx_edges(network)
```

`py3plex.core.supporting.ensure(*args, **kwargs)`

`py3plex.core.supporting.parse_gaf_to_uniprot_GO(gaf_mappings: str, filter_terms: int | None = None) → Dict[str, List[str]]`

Parse Gene Association File (GAF) to map UniProt IDs to GO terms.

Parameters

- **`gaf_mappings`** – Path to GAF file.
- **`filter_terms`** – Optional minimum occurrence threshold for GO terms.

Returns

Dictionary mapping UniProt IDs to lists of associated GO terms.

Example

```
>>> mappings = parse_gaf_to_uniprot_GO("gaf_file.gaf", filter_terms=5)
```

`py3plex.core.supporting.require(*args, **kwargs)`

`py3plex.core.supporting.split_to_layers(input_network: Graph) → Dict[Any, Graph]`

Split a multilayer network into separate layer subgraphs.

Parameters

`input_network` – NetworkX graph containing nodes from multiple layers.

Returns

Dictionary mapping layer names to their corresponding subgraphs.

Contracts:

- Precondition: `input_network` must not be `None`
- Precondition: `input_network` must be a NetworkX graph
- Postcondition: result is a dictionary
- Postcondition: all values are NetworkX graphs

Example

```
>>> network = nx.Graph()
>>> network.add_node(('A', 'layer1'))
>>> network.add_node(('B', 'layer2'))
>>> layers = split_to_layers(network)
```

NetworkX compatibility layer for py3plex. This module provides compatibility functions for different NetworkX versions.

`py3plex.core.nx_compat.ensure(*args, **kwargs)`

`py3plex.core.nx_compat.is_string_like(obj: Any) → bool`

Check if obj is string-like (compatible with NetworkX < 3.0).

Parameters

obj – Object to check

Returns

True if string-like

Return type

bool

`py3plex.core.nx_compat.nx_from_scipy_sparse_matrix(A: Any, parallel_edges: bool = False, create_using: Graph | None = None, edge_attribute: str = 'weight') → Graph`

Create a graph from scipy sparse matrix (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

- **A** – scipy sparse matrix
- **parallel_edges** – Whether to create parallel edges (ignored in NetworkX 3.0+)
- **create_using** – Graph type to create
- **edge_attribute** – Edge attribute name for weights

Returns

NetworkX graph

Contracts:

- Precondition: A must not be None
- Precondition: edge_attribute must be a non-empty string
- Postcondition: returns a NetworkX graph

`py3plex.core.nx_compat.nx_info(G: Graph) → str`

Get network information (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

G – NetworkX graph

Returns

Network information

Return type

str

Contracts:

- Precondition: G must not be None and must be a NetworkX graph
- Postcondition: returns a non-empty string

`py3plex.core.nx_compat.nx_read_gpickle(path: str) → Graph`

Read a graph from a pickle file (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

path – File path

Returns

NetworkX graph

Contracts:

- Precondition: path must be a non-empty string
- Postcondition: returns a NetworkX graph

```
py3plex.core.nx_compat.nx_to_scipy_sparse_matrix(G: Graph, nodelist: list | None = None, dtype: Any | None = None, weight: str = 'weight', format: str = 'csr') → Any
```

Convert graph to scipy sparse matrix (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

- **G** – NetworkX graph
- **nodelist** – List of nodes
- **dtype** – Data type
- **weight** – Edge weight attribute
- **format** – Sparse matrix format

Returns

scipy sparse matrix

Contracts:

- Precondition: G must not be None and must be a NetworkX graph
- Precondition: weight must be a non-empty string
- Precondition: format must be a non-empty string
- Postcondition: returns a non-None sparse matrix

```
py3plex.core.nx_compat.nx_write_gpickle(G: Graph, path: str) → None
```

Write a graph to a pickle file (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

- **G** – NetworkX graph
- **path** – File path

Contracts:

- Precondition: G must not be None and must be a NetworkX graph
- Precondition: path must be a non-empty string

```
py3plex.core.nx_compat.require(*args, **kwargs)
```

HINMINE Network Decomposition

```
py3plex.core.HINMINE.decomposition.aggregate_sum(input_thing, classes, universal_set)
```

```
py3plex.core.HINMINE.decomposition.aggregate_weighted_sum(input_thing, classes, universal_set)
```

```
py3plex.core.HINMINE.decomposition.calculate_importance_chi(classes, universal_set,
                                                            linked_nodes, n, **kwargs)
```

Calculates importance of a single midpoint using chi-squared weighing. :param classes: List of all classes
:param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the
midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each
label in class

```
py3plex.core.HINMINE.decomposition.calculate_importance_delta(classes, universal_set,
                                                              linked_nodes, n, **kwargs)
```

Calculates importance of a single midpoint using delta-idf weighing :param classes: List of all classes :param
universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint
:param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

```
py3plex.core.HINMINE.decomposition.calculate_importance_gr(classes, universal_set,
                                                            linked_nodes, n, **kwargs)
```

Calculates importance of a single midpoint using the GR (gain ratio) :param classes: List of all classes :param
universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint
:param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

```
py3plex.core.HINMINE.decomposition.calculate_importance_idf(classes, universal_set,
                                                             linked_nodes, n, **kwargs)
```

Calculates importance of a single midpoint using idf weighing :param classes: List of all classes :param
universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint
:param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

```
py3plex.core.HINMINE.decomposition.calculate_importance_ig(classes, universal_set,
                                                            linked_nodes, n, **kwargs)
```

Calculates importance of a single midpoint using IG (information gain) weighing :param classes: List of all
classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by
the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each
label in class

```
py3plex.core.HINMINE.decomposition.calculate_importance_okapi(classes, universal_set,
                                                              linked_nodes, n,
                                                              degrees=None,
                                                              avgdegree=None)
```

```
py3plex.core.HINMINE.decomposition.calculate_importance_rf(classes, universal_set,
                                                            linked_nodes, n, **kwargs)
```

Calculates importance of a single midpoint using rf weighing :param classes: List of all classes :param
universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint
:param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

```
py3plex.core.HINMINE.decomposition.calculate_importance_tf(classes, universal_set,
                                                            linked_nodes, n, **kwargs)
```

Calculates importance of a single midpoint using term frequency weighing. :param classes: List of all
classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by
the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each
label in class

```
py3plex.core.HINMINE.decomposition.calculate_importance_w2w(classes, universal_set,
                                                             linked_nodes, n, **kwargs)
```

```
py3plex.core.HINMINE.decomposition.calculate_importances(midpoints, classes, universal_set,
                                                         method, degrees=None,
                                                         avgdegree=None)
```

```
py3plex.core.HINMINE.decomposition.chi_value(actual_pos_num, predicted_pos_num, tp, n)
```

```
py3plex.core.HINMINE.decomposition.get_aggregation_method(method_name)
py3plex.core.HINMINE.decomposition.get_calculation_method(method_name)
py3plex.core.HINMINE.decomposition.gr_value(actual_pos_num, predicted_pos_num, tp, n)
py3plex.core.HINMINE.decomposition.hinmine_decompose(network, heuristic, cycle=None,
                                                    parallel=False)
py3plex.core.HINMINE.decomposition.hinmine_get_cycles(network, cycle=None)
py3plex.core.HINMINE.decomposition.ig_value(actual_pos_num, predicted_pos_num, tp, n)
py3plex.core.HINMINE.decomposition.np_calculate_importance_chi(predicted, label_matrix,
                                                                actual_pos_nums)
py3plex.core.HINMINE.decomposition.np_calculate_importance_tf(predicted, label_matrix)
py3plex.core.HINMINE.decomposition.rf_value(predicted_pos_num, tp)
py3plex.core.HINMINE.IO.load_hinmine_object(infile, label_delimiter='---', weight_tag=False,
                                           targets=True)
```

Configuration and Utilities

Centralized configuration for py3plex.

This module provides default settings for visualization, layout algorithms, and other configurable aspects of the library. Users can override these settings by modifying the values after import.

Example

```
>>> from py3plex import config
>>> config.DEFAULT_NODE_SIZE = 15
>>> config.DEFAULT_EDGE_ALPHA = 0.5
```

`py3plex.config.get_color_palette(name: str | None = None) → List[str]`

Get a color palette by name.

Parameters

name – Palette name. If None, returns the default palette.

Returns

List of color hex codes.

Raises

ValueError – If palette name is not recognized.

Example

```
>>> from py3plex.config import get_color_palette
>>> colors = get_color_palette("rainbow")
>>> print(colors[0])
'#FF6B6B'
```

`py3plex.config.reset_to_defaults() → None`

Reset all configuration values to their defaults.

This is useful for testing or when you want to start fresh.

Example

```
>>> from py3plex import config
>>> config.DEFAULT_NODE_SIZE = 20
>>> config.reset_to_defaults()
>>> print(config.DEFAULT_NODE_SIZE)
10
```

Utility functions for py3plex.

This module provides common utilities used across the library, including random state management for reproducibility and deprecation warnings.

`py3plex.utils.deprecated(reason: str, version: str | None = None, alternative: str | None = None) → Callable[[Callable], Callable]`

Decorator to mark functions/methods as deprecated.

This decorator will issue a `DeprecationWarning` when the decorated function is called, providing information about why it's deprecated and what to use instead.

Parameters

- **reason** – Explanation of why the function is deprecated
- **version** – Version in which the function was deprecated (optional)
- **alternative** – Suggested alternative function/method (optional)

Returns

Decorator function

Example

```
>>> @deprecated(
...     reason="This function is obsolete",
...     version="0.95a",
...     alternative="new_function()"
... )
... def old_function():
...     pass
```

`py3plex.utils.ensure(*args, **kwargs)`

`py3plex.utils.get_background_knowledge_dir() → str`

Get the absolute path to the background knowledge directory.

Returns

Absolute path to the background_knowledge directory

Return type

str

Examples

```
>>> from py3plex.utils import get_background_knowledge_dir
>>> dir_path = get_background_knowledge_dir()
```

`py3plex.utils.get_background_knowledge_path(filename: str) → str`

Get the absolute path to a background knowledge file or directory.

Convenience wrapper around `get_data_path()` specifically for background knowledge files.

Parameters

filename – Name or relative path of the background knowledge file. Use empty string or ‘.’ to get the background_knowledge directory itself.

Returns

Absolute path to the background knowledge file or directory

Return type

str

Examples

```
>>> from py3plex.utils import get_background_knowledge_path
>>> path = get_background_knowledge_path("bk.n3")
>>> dir_path = get_background_knowledge_path(".")
```

py3plex.utils.**get_data_path**(relative_path: str) → str

Get the absolute path to a data file in the repository.

This function searches for data files in multiple locations to support both: - Running examples from a cloned repository - Running scripts/notebooks from any directory with datasets locally available

Search order: 1. Relative to the calling script’s directory (for examples in cloned repo) 2. Relative to current working directory (for notebooks/user scripts) 3. Relative to py3plex package location (for editable installs)

Parameters

relative_path – Path relative to repository root (e.g., “datasets/intact02.gpickle”)

Returns

Absolute path to the file

Return type

str

Raises

Py3plexIOError – If the file cannot be found in any search location

Examples

```
>>> from py3plex.utils import get_data_path
>>> path = get_data_path("datasets/intact02.gpickle")
>>> os.path.exists(path)
True
```

Note

When py3plex is installed via pip, datasets are not included in the package. Users should either: - Clone the repository and run examples from there - Download datasets separately and place them relative to their scripts - Use current working directory with datasets folder

py3plex.utils.**get_dataset_path**(filename: str) → str

Get the absolute path to a dataset file.

Convenience wrapper around get_data_path() specifically for dataset files.

Parameters

filename – Name or relative path of the dataset file

Returns

Absolute path to the dataset file

Return type

str

Examples

```
>>> from py3plex.utils import get_dataset_path
>>> path = get_dataset_path("intact02.gpickle")
>>> os.path.exists(path)
True
```

py3plex.utils.**get_example_image_path**(filename: str) → str

Get the absolute path to an example image file.

Convenience wrapper around get_data_path() specifically for example image files.

Parameters

filename – Name or relative path of the image file

Returns

Absolute path to the example image file

Return type

str

Examples

```
>>> from py3plex.utils import get_example_image_path
>>> path = get_example_image_path("intact_10_BH.png")
```

py3plex.utils.**get_multilayer_dataset_path**(relative_path: str) → str

Get the absolute path to a multilayer dataset file.

Convenience wrapper around get_data_path() specifically for multilayer dataset files.

Parameters

relative_path – Relative path within multilayer_datasets directory

Returns

Absolute path to the multilayer dataset file

Return type

str

Examples

```
>>> from py3plex.utils import get_multilayer_dataset_path
>>> path = get_multilayer_dataset_path("MLKing/MLKing2013_multiplex.edges")
```

py3plex.utils.**get_rng**(seed: int | Generator | None = None) → Generator

Get a NumPy random number generator with optional seed.

This provides a unified interface for random state management across the library, ensuring reproducibility when a seed is provided.

Parameters

seed – Random seed for reproducibility. Can be: - None: Use default unseeded generator - int: Seed value for the generator - np.random.Generator: Pass through existing generator

Returns

Initialized random number generator

Return type

np.random.Generator

Examples

```
>>> rng = get_rng(42)
>>> rng.random() # Reproducible random number
0.7739560485559633
```

```
>>> rng1 = get_rng(42)
>>> rng2 = get_rng(42)
>>> rng1.random() == rng2.random()
True
```

```
>>> existing_rng = np.random.default_rng(123)
>>> rng = get_rng(existing_rng)
>>> rng is existing_rng
True
```

Contracts:

- Postcondition: result is a NumPy random Generator

Note

Uses `numpy.random.Generator` (modern API introduced in NumPy 1.17) rather than the legacy `numpy.random.RandomState` API.

Negative seeds are converted to positive values by taking absolute value to ensure compatibility with NumPy's `SeedSequence`.

`py3plex.utils.require(*args, **kwargs)`

`py3plex.utils.validate_multilayer_input(network_data: Any) → None`

Validate multilayer network input data.

Performs sanity checks on multilayer network structures to catch common errors early.

Parameters

network_data – Network data to validate (can be various formats)

Raises

ValueError – If the network data is invalid

Contracts:

- Precondition: `network_data` must not be `None`

Example

```
>>> from py3plex.utils import validate_multilayer_input
>>> validate_multilayer_input(my_network)
```

`py3plex.utils.warn_if_deprecated(feature_name: str, reason: str, alternative: str | None = None) → None`

Issue a deprecation warning for a feature.

This is useful for deprecating specific usage patterns or parameter combinations rather than entire functions.

Parameters

- **feature_name** – Name of the deprecated feature

- **reason** – Explanation of why it's deprecated
- **alternative** – Suggested alternative (optional)

Example

```
>>> def my_function(old_param=None, new_param=None):
...     if old_param is not None:
...         warn_if_deprecated(
...             "old_param",
...             "This parameter is no longer used",
...             "new_param"
...         )
```

Custom exception types for the py3plex library.

This module defines domain-specific exceptions to provide clear error messages and enable better error handling throughout the library.

exception py3plex.exceptions.**AlgorithmError**

Bases: *Py3plexException*

Exception raised when an algorithm execution fails.

exception py3plex.exceptions.**CentralityComputationError**

Bases: *AlgorithmError*

Exception raised when centrality computation fails.

exception py3plex.exceptions.**CommunityDetectionError**

Bases: *AlgorithmError*

Exception raised when community detection fails.

exception py3plex.exceptions.**ConversionError**

Bases: *Py3plexException*

Exception raised when format conversion fails.

exception py3plex.exceptions.**DecompositionError**

Bases: *AlgorithmError*

Exception raised when network decomposition fails.

exception py3plex.exceptions.**EmbeddingError**

Bases: *AlgorithmError*

Exception raised when embedding generation fails.

exception py3plex.exceptions.**ExternalToolError**

Bases: *Py3plexException*

Exception raised when external tool execution fails.

exception py3plex.exceptions.**IncompatibleNetworkError**

Bases: *Py3plexException*

Exception raised when network format is incompatible with an operation.

exception py3plex.exceptions.**InvalidEdgeError**

Bases: *Py3plexException*

Exception raised when an invalid edge is specified.

exception `py3plex.exceptions.InvalidLayerError`

Bases: `Py3plexException`

Exception raised when an invalid layer is specified.

exception `py3plex.exceptions.InvalidNodeError`

Bases: `Py3plexException`

Exception raised when an invalid node is specified.

exception `py3plex.exceptions.NetworkConstructionError`

Bases: `Py3plexException`

Exception raised when network construction fails.

exception `py3plex.exceptions.ParsingError`

Bases: `Py3plexException`

Exception raised when parsing input data fails.

exception `py3plex.exceptions.Py3plexException`

Bases: `Exception`

Base exception class for all py3plex-specific exceptions.

exception `py3plex.exceptions.Py3plexFormatError`

Bases: `Py3plexException`

Exception raised when input format is invalid or cannot be parsed.

exception `py3plex.exceptions.Py3plexIOError`

Bases: `Py3plexException`

Exception raised when I/O operations fail (file reading, writing, etc.).

exception `py3plex.exceptions.Py3plexLayoutError`

Bases: `Py3plexException`

Exception raised when layout computation or visualization positioning fails.

exception `py3plex.exceptions.Py3plexMatrixError`

Bases: `Py3plexException`

Exception raised when matrix operations fail or matrix is invalid.

exception `py3plex.exceptions.VisualizationError`

Bases: `Py3plexException`

Exception raised when visualization operations fail.

Domain-Specific Language (DSL)

SQL-like DSL for Multilayer Network Queries.

This module provides a Domain-Specific Language (DSL) for querying and analyzing multilayer networks using SQL-like syntax. The DSL supports filtering nodes and edges based on various network properties and computing analytical measures.

Example Usage:

```
>>> from py3plex.core import multinet
>>> from py3plex.dsl import execute_query
>>>
>>> # Create a multilayer network
>>> network = multinet.multi_layer_network()
>>> # ... add nodes and edges ...
```

(continues on next page)

(continued from previous page)

```

>>>
>>> # Query nodes with high degree in a specific layer
>>> result = execute_query(network, 'SELECT nodes WHERE layer="transport" AND
↳degree > 5')
>>>
>>> # Compute centrality for filtered nodes
>>> result = execute_query(network, 'SELECT nodes WHERE layer="social" COMPUTE
↳betweenness_centrality')

```

Supported Operations:

- SELECT: Choose what to select (nodes, edges)
- MATCH: Cypher-like pattern matching for graph patterns
- WHERE: Filter by conditions (layer, degree, centrality, etc.)
- AND/OR/NOT: Logical operators for combining conditions
- COMPUTE: Calculate network measures (degree, centrality, etc.)
- IN LAYER / IN LAYERS: Layer scoping clauses
- RETURN: Specify which aliases to return from MATCH queries
- Comparison operators: >, <, =, >=, <=, !=

Extended Syntax Examples:

```

# Layer scoping for SELECT queries SELECT * FROM nodes IN LAYER 'ppi' WHERE degree > 10;
SELECT id, degree FROM nodes IN LAYERS ('ppi', 'coexpr') WHERE color = 'red';

```

```

# Cypher-like MATCH pattern MATCH (g:Gene)-[r:REGULATES]->(t:Gene) IN LAYER 'reg' WHERE
g.degree > 10 RETURN g, t;

```

See examples/network_analysis/example_dsl_queries.py for more examples.

exception py3plex.dsl.DSLExecutionError

Bases: Exception

Exception raised for DSL execution errors.

exception py3plex.dsl.DSLSyntaxError

Bases: Exception

Exception raised for DSL syntax errors.

class py3plex.dsl.EdgePattern(alias: str | None = None, type: str | None = None, directed: bool = True)

Bases: object

Represents an edge pattern in a MATCH query.

Pattern syntax: -[alias[:Type]?]-> Examples: -[r:REGULATES]->, -[e]->, -[:INTERACTS]->

alias

The optional variable name for the edge

Type

str | None

type

The optional type filter for the edge

Type

str | None

directed

Whether the edge is directed (always True for now)

Type

bool

alias: str | None = None

directed: bool = True

type: str | None = None

```
class py3plex.dsl.MatchQuery(pattern: ~py3plex.dsl.PathPattern, layers: ~typing.List[str] | None =  
    None, conditions: ~typing.List[~typing.Dict[str, ~typing.Any]] =  
    <factory>, return_aliases: ~typing.List[str] | None = None)
```

Bases: object

Represents a parsed MATCH query.

Syntax: MATCH <pattern> [IN LAYER 'name' | IN LAYERS ('a', 'b')] [WHERE conditions] [RETURN aliases];

pattern

The PathPattern to match

Type

py3plex.dsl.PathPattern

layers

Optional list of layer names to filter

Type

List[str] | None

conditions

List of WHERE condition dictionaries

Type

List[Dict[str, Any]]

return_aliases

List of aliases to return, or None for all (RETURN *)

Type

List[str] | None

conditions: List[Dict[str, Any]]

layers: List[str] | None = None

pattern: *PathPattern*

return_aliases: List[str] | None = None

```
class py3plex.dsl.NodePattern(alias: str, label: str | None = None)
```

Bases: object

Represents a node pattern in a MATCH query.

Pattern syntax: (alias[:Label]?) Examples: (g:Gene), (t), (x:Protein)

alias

The variable name for the node (required)

Type

str

label

The optional type/label filter for the node

Type

str | None

alias: str

label: str | None = None

```
class py3plex.dsl.PathPattern(nodes: ~typing.List[~py3plex.dsl.NodePattern] = <factory>, edges:
                               ~typing.List[~py3plex.dsl.EdgePattern] = <factory>)
```

Bases: object

Represents a complete path pattern in a MATCH query.

A path consists of alternating nodes and edges: (n1)-[e1]->(n2)-[e2]->(n3)...

nodes

List of NodePattern objects

Type

List[py3plex.dsl.NodePattern]

edges

List of EdgePattern objects (len = len(nodes) - 1)

Type

List[py3plex.dsl.EdgePattern]

edges: List[EdgePattern]

nodes: List[NodePattern]

```
py3plex.dsl.compute_centrality_for_layer(network: Any, layer: str, centrality: str =
                                         'betweenness_centrality') → Dict[Any, float]
```

Compute centrality for all nodes in a layer.

Parameters

- **network** – Multilayer network object
- **layer** – Layer identifier
- **centrality** – Centrality measure name

Returns

Dictionary mapping nodes to centrality values

```
py3plex.dsl.execute_query(network: Any, query: str) → Dict[str, Any]
```

Execute a DSL query on a multilayer network.

Supports both SELECT and MATCH queries:

SELECT queries:

```
SELECT nodes WHERE layer="transport" AND degree > 5 SELECT * FROM nodes IN LAYER 'ppi'
WHERE degree > 10 SELECT id, degree FROM nodes IN LAYERS ('ppi', 'coexpr') WHERE color
= 'red'
```

MATCH queries (Cypher-like):

```
MATCH (g:Gene)-[r:REGULATES]->(t:Gene) IN LAYER 'reg' WHERE g.degree > 10 RETURN g,
t
```

Parameters

- **network** – Multilayer network object (multi_layer_network instance)
- **query** – DSL query string

Returns

- 'nodes' or 'edges' or 'bindings': List of selected items / pattern matches
- 'computed': Dictionary of computed measures (if COMPUTE used)
- 'query': Original query string

Return type

Dictionary containing

Raises

- ***DSLSyntaxError*** – If query syntax is invalid
- ***DSLExecutionError*** – If query cannot be executed

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'transport'}])
>>> net.add_nodes([{'source': 'B', 'type': 'transport'}])
>>> net.add_nodes([{'source': 'C', 'type': 'social'}])
>>> net.add_edges([
...     {'source': 'A', 'target': 'B', 'source_type': 'transport', 'target_type': 'transport'},
...     {'source': 'B', 'target': 'C', 'source_type': 'social', 'target_type': 'social'}
... ])
>>>
>>> # Select all nodes in "transport" layer
>>> result = execute_query(net, 'SELECT nodes WHERE layer="transport"')
>>> result['count'] >= 0
True
>>>
>>> # Select high-degree nodes and compute centrality
>>> result = execute_query(net, 'SELECT nodes WHERE degree > 0 COMPUTE_
↳ betweenness_centrality')
>>> 'computed' in result
True
>>>
>>> # Complex query with multiple conditions
>>> result = execute_query(net, 'SELECT nodes WHERE layer="social" AND degree >=
↳ 0')
>>> result['count'] >= 0
True
```

`py3plex.dsl.format_result(result: Dict[str, Any], limit: int = 10) → str`

Format query result as human-readable string.

Parameters

- **result** – Result dictionary from `execute_query`
- **limit** – Maximum number of items to display

Returns

Formatted string representation

`py3plex.dsl.select_high_degree_nodes(network: Any, min_degree: int, layer: str | None = None) → List[Any]`

Select nodes with degree greater than threshold.

Parameters

- **network** – Multilayer network object
- **min_degree** – Minimum degree threshold (exclusive - nodes must have degree > min_degree)
- **layer** – Optional layer to filter by

Returns

List of nodes with degree > min_degree

`py3plex.dsl.select_nodes_by_layer(network: Any, layer: str) → List[Any]`

Select all nodes in a specific layer.

Parameters

- **network** – Multilayer network object
- **layer** – Layer identifier

Returns

List of nodes in the specified layer

Algorithms**Community Detection**

```
py3plex.algorithms.community_detection.community_wrapper.NoRC_communities(network,  
                                                                           verbose=True,  
                                                                           cluster-  
                                                                           ing_scheme='kmeans',  
                                                                           out-  
                                                                           put='mapping',  
                                                                           prob_threshold=0.001,  
                                                                           paral-  
                                                                           lel_step=8,  
                                                                           commu-  
                                                                           nity_range=None,  
                                                                           fine_range=3)
```

```
py3plex.algorithms.community_detection.community_wrapper.infomap_communities(graph:  
    Graph,  
    binary: str  
    =  
    './infomap',  
    edge-  
    list_file: str  
    =  
    './tmp/tmpedgelist.txt',  
    multiplex:  
    bool =  
    False,  
    verbose:  
    bool =  
    False, over-  
    lapping:  
    bool =  
    False,  
    iterations:  
    int = 200,  
    output: str  
    =  
    'mapping',  
    seed: int |  
    None =  
    None) →  
    Dict[Any,  
    int] |  
    Dict[Any,  
    List[int]]
```

Detect communities using the Infomap algorithm.

Parameters

- **graph** – Input graph (NetworkX graph or multi_layer_network)
- **binary** – Path to Infomap binary (default: “./infomap”)
- **edgelist_file** – Temporary file for edgelist (default: “./tmp/tmpedgelist.txt”)
- **multiplex** – Whether to use multiplex mode (default: False)
- **verbose** – Whether to show verbose output (default: False)
- **overlapping** – Whether to detect overlapping communities (default: False)
- **iterations** – Number of iterations (default: 200)
- **output** – Output format - “mapping” or “partition” (default: “mapping”)
- **seed** – Random seed for reproducibility (default: None) Note: Requires Infomap binary that supports –seed parameter

Returns

Dict mapping nodes to community IDs (if output=“mapping”) or Dict mapping community IDs to lists of nodes (if output=“partition”)

Raises

- **FileNotFoundError** – If Infomap binary is not found
- **PermissionError** – If Infomap binary is not executable

Examples

```
>>> # Using with seed for reproducibility
>>> partition = infomap_communities(graph, seed=42)
>>>
>>> # Get partition format instead of mapping
>>> communities = infomap_communities(graph, output="partition")
```

```
py3plex.algorithms.community_detection.community_wrapper.louvain_communities(network,
                                                                              out-
                                                                              put='mapping')
```

```
py3plex.algorithms.community_detection.community_wrapper.parse_infomap(outfile)
```

```
py3plex.algorithms.community_detection.community_wrapper.run_infomap(infile: str, multiplex:
                                                                    bool = True,
                                                                    overlapping: bool =
                                                                    False, binary: str =
                                                                    './infomap', verbose:
                                                                    bool = True,
                                                                    iterations: int = 1000,
                                                                    seed: int | None =
                                                                    None) → None
```

Multilayer Modularity Maximization (Mucha et al., 2010)

This module implements multilayer modularity quality function and optimization algorithms for community detection in multilayer/multiplex networks.

References

Mucha et al., “Community Structure in Time-Dependent, Multiscale, and Multiplex Networks”, Science 328:876-878 (2010)

```
py3plex.algorithms.community_detection.multilayer_modularity.build_supra_modularity_matrix(network:
                                                                                             Any,
                                                                                             gamma:
                                                                                             float
                                                                                             |
                                                                                             Dict[Any,
                                                                                             float]
                                                                                             =
                                                                                             1.0,
                                                                                             omega:
                                                                                             float
                                                                                             |
                                                                                             ndarray
                                                                                             =
                                                                                             1.0,
                                                                                             weight:
                                                                                             str
                                                                                             =
                                                                                             'weight')
                                                                                             →
                                                                                             Tuple[ndarray,
                                                                                             List[Tuple[A
                                                                                             Any]]]
```

Build the supra-modularity matrix B for multilayer network.

The supra-modularity matrix is: $B_{\{i,j\}} = (A_{ij} - \frac{k_i k_j}{2m}) + \frac{1}{2} \frac{k_i k_j}{m}$

This matrix can be used for spectral community detection methods.

Parameters

- **network** – py3plex multi_layer_network object
- **gamma** – Resolution parameter(s)
- **omega** – Inter-layer coupling strength
- **weight** – Edge weight attribute

Returns

Tuple of (modularity_matrix, node_layer_list) - modularity_matrix: Supra-modularity matrix B (NL × NL) - node_layer_list: List of (node, layer) tuples corresponding to matrix indices

```
py3plex.algorithms.community_detection.multilayer_modularity.louvain_multilayer(network: Any,
gamma: float | Dict[Any, float] = 1.0,
omega: float | ndarray = 1.0,
weight: str = 'weight',
max_iter: int = 100,
random_state: int | None = None)
→ Dict[Tuple[Any, Any], int]
```

Generalized Louvain algorithm for multilayer networks.

This implements the multilayer Louvain method as described in Mucha et al. (2010), which greedily maximizes the multilayer modularity quality function.

Complexity:

- **Time: $O(n \times L \times d \times k)$ per iteration, where:**
 - n = number of nodes per layer
 - L = number of layers
 - d = average degree
 - k = number of communities
- Typical: $O(n \times L)$ iterations until convergence
- Worst case: $O((n \times L)^2)$ for dense networks
- Space: $O((n \times L)^2)$ for supra-adjacency matrix (use sparse for large networks)

Parameters

- **network** – py3plex multi_layer_network object
- **gamma** – Resolution parameter(s)
- **omega** – Inter-layer coupling strength
- **weight** – Edge weight attribute
- **max_iter** – Maximum number of iterations
- **random_state** – Random seed for reproducibility

Returns

Dictionary mapping (node, layer) tuples to community IDs

Examples

```
>>> from py3plex.core import multinet
>>> from py3plex.algorithms.community_detection.multilayer_modularity import_
↳louvain_multilayer
>>>
>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1],
...     ['A', 'L2', 'C', 'L2', 1]
... ], input_type='list')
>>>
>>> communities = louvain_multilayer(network, gamma=1.0, omega=1.0, random_
↳state=42)
>>> print(communities)
```

Note

For reproducible results, always set random_state parameter.

```

py3plex.algorithms.community_detection.multilayer_modularity.multilayer_modularity(network:
    Any,
    communities:
    Dict[Tuple[Any,
    Any],
    int],
    gamma:
    float
    |
    Dict[Any,
    float]
    =
    1.0,
    omega:
    float
    |
    ndarray
    =
    1.0,
    weight:
    str
    =
    'weight')
    →
    float

```

Calculate multilayer modularity quality function (Mucha et al., 2010).

The multilayer modularity is defined as: $Q = (1/2) \sum_{ij} [(A^l_{ij} - P^l_{ij}) + \sum_l (g_l - g_j)]$

where: - A^l_{ij} is the adjacency matrix of layer - P^l_{ij} is the null model (e.g., Newman-Girvan: $k_i^l k_j^l / 2m_l$) - γ_l is the resolution parameter for layer - ω is the inter-layer coupling strength - $\omega = 1$ if $i=j$, else 0 (Kronecker delta) - $\delta_{ij} = 1$ if $i=j$, else 0 - $(g_i, g_j) = 1$ if node i in layer l and node j in layer l' are in same community - m is the total edge weight in the supra-network

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) tuples to community IDs
- **gamma** – Resolution parameter(s). Can be: - Single float: same resolution for all layers
- Dict[layer, float]: layer-specific resolution parameters
- **omega** – Inter-layer coupling strength. Can be: - Single float: uniform coupling between all layer pairs - np.ndarray: layer-pair specific coupling matrix ($L \times L$)
- **weight** – Edge weight attribute (default: “weight”)

Returns

Modularity value $Q \in [-1, 1]$

Examples

```

>>> from py3plex.core import multinet
>>> from py3plex.algorithms.community_detection.multilayer_modularity import _
    ↪ multilayer_modularity
>>>
>>> # Create a simple multilayer network

```

(continues on next page)

(continued from previous page)

```

>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1],
...     ['A', 'L2', 'C', 'L2', 1]
... ], input_type='list')
>>>
>>> # Assign communities
>>> communities = {
...     ('A', 'L1'): 0, ('B', 'L1'): 0, ('C', 'L1'): 1,
...     ('A', 'L2'): 0, ('C', 'L2'): 0
... }
>>>
>>> # Calculate modularity
>>> Q = multilayer_modularity(network, communities, gamma=1.0, omega=1.0)
>>> print(f"Modularity: {Q:.3f}")

```

This module implements community detection.

class py3plex.algorithms.community_detection.community_louvain.Status

Bases: object

To handle several data in one struct.

Could be replaced by named tuple, but don't want to depend on python 2.6

copy()

Perform a deep copy of status

degrees: dict = {}

gdegrees: dict = {}

init(graph, weight, part=None)

Initialize the status of a graph with every node in one community

internals: dict = {}

node2com: dict = {}

total_weight = 0

py3plex.algorithms.community_detection.community_louvain.best_partition(*graph: Graph*,
partition: Dict |
None = None,
weight: str =
'weight',
resolution: float =
1.0, *randomize:*
bool = False) →
Dict

Compute the partition of the graph nodes which maximises the modularity (or try..) using the Louvain heuristics

This is the partition of highest modularity, i.e. the highest partition of the dendrogram generated by the Louvain algorithm.

Parameters

- **graph** (*networkx.Graph*) – the networkx graph which is decomposed

- **partition** (*dict, optional*) – the algorithm will start using this partition of the nodes. It's a dictionary where keys are their nodes and values the communities
- **weight** (*str, optional*) – the key in graph to use as weight. Default to 'weight'
- **resolution** (*double, optional*) – Will change the size of the communities, default to 1. represents the time described in “Laplacian Dynamics and Multiscale Modular Structure in Networks”, R. Lambiotte, J.-C. Delvenne, M. Barahona
- **randomize** (*boolean, optional*) – Will randomize the node evaluation order and the community evaluation order to get different partitions at each call

Returns

partition – The partition, with communities numbered from 0 to number of communities

Return type

dictionnary

Raises

NetworkXError – If the graph is not Eulerian.

See also

[*generate_dendrogram*](#)

Notes

Uses Louvain algorithm

References

large networks. J. Stat. Mech 10008, 1-12(2008).

Examples

```
>>> #Basic usage
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> part = best_partition(G)

>>> #other example to display a graph with its community :
>>> #better with karate_graph() as defined in networkx examples
>>> #erdos renyi don't have true community structure
>>> G = nx.erdos_renyi_graph(30, 0.05)
>>> #first compute the best partition
>>> partition = community.best_partition(G)
>>> #drawing
>>> size = float(len(set(partition.values())))
>>> pos = nx.spring_layout(G)
>>> count = 0.
>>> for com in set(partition.values()) :
>>>     count += 1.
>>>     list_nodes = [nodes for nodes in partition.keys()
>>>                    if partition[nodes] == com]
>>>     nx.draw_networkx_nodes(G, pos, list_nodes, node_size = 20,
>>>                            node_color = str(count / size))
>>> nx.draw_networkx_edges(G, pos, alpha=0.5)
>>> plt.show()
```



```
py3plex.algorithms.community_detection.community_louvain.generate_dendrogram(graph:
                                                                            Graph,
                                                                            part_init:
                                                                              Dict | None
                                                                              = None,
                                                                            weight: str
                                                                              = 'weight',
                                                                            resolution:
                                                                              float = 1.0,
                                                                            randomize:
                                                                              bool =
                                                                              False) →
                                                                              List[Dict]
```

Find communities in the graph and return the associated dendrogram

A dendrogram is a tree and each level is a partition of the graph nodes. Level 0 is the first partition, which contains the smallest communities, and the best is `len(dendrogram) - 1`. The higher the level is, the bigger are the communities

Parameters

- **graph** (*networkx.Graph*) – the networkx graph which will be decomposed
- **part_init** (*dict*, *optional*) – the algorithm will start using this partition of the nodes. It's a dictionary where keys are their nodes and values the communities
- **weight** (*str*, *optional*) – the key in graph to use as weight. Default to 'weight'
- **resolution** (*double*, *optional*) – Will change the size of the communities, default to 1. represents the time described in “Laplacian Dynamics and Multiscale Modular Structure in Networks”, R. Lambiotte, J.-C. Delvenne, M. Barahona

Returns

dendrogram – a list of partitions, ie dictionaries where keys of the *i*+1 are the values of the *i*. and where keys of the first are the nodes of graph

Return type

list of dictionaries

Raises

TypeError – If the graph is not a *networkx.Graph*

See also

[*best_partition*](#)

Notes

Uses Louvain algorithm

References

networks. J. Stat. Mech 10008, 1-12(2008).

Examples

```
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> dendo = generate_dendrogram(G)
>>> for level in range(len(dendo) - 1) :
>>>     print("partition at level", level,
>>>           "is", partition_at_level(dendo, level))
```

(continues on next page)

(continued from previous page)

```
:param weight:
:type weight:
```

```
py3plex.algorithms.community_detection.community_louvain.induced_graph(partition: Dict,
                                                                    graph: Graph,
                                                                    weight: str =
                                                                    'weight') → Graph
```

Produce the graph where nodes are the communities

there is a link of weight w between communities if the sum of the weights of the links between their elements is w

Parameters

- **partition** (*dict*) – a dictionary where keys are graph nodes and values the part the node belongs to
- **graph** (*networkx.Graph*) – the initial graph
- **weight** (*str, optional*) – the key in graph to use as weight. Default to 'weight'

Returns

g – a networkx graph where nodes are the parts

Return type

networkx.Graph

Examples

```
>>> n = 5
>>> g = nx.complete_graph(2*n)
>>> part = dict([])
>>> for node in g.nodes() :
>>>     part[node] = node % 2
>>> ind = induced_graph(part, g)
>>> goal = nx.Graph()
>>> goal.add_weighted_edges_from([(0,1,n*n),(0,0,n*(n-1)/2), (1, 1, n*(n-1)/2)])
→ # NOQA
>>> nx.is_isomorphic(int, goal)
True
```

```
py3plex.algorithms.community_detection.community_louvain.load_binary(data)
```

Load binary graph as used by the cpp implementation of this algorithm

```
py3plex.algorithms.community_detection.community_louvain.modularity(partition: Dict, graph:
                                                                    Graph, weight: str =
                                                                    'weight') → float
```

Compute the modularity of a partition of a graph

Parameters

- **partition** (*dict*) – the partition of the nodes, i.e a dictionary where keys are their nodes and values the communities
- **graph** (*networkx.Graph*) – the networkx graph which is decomposed
- **weight** (*str, optional*) – the key in graph to use as weight. Default to 'weight'

Returns

modularity – The modularity

Return type

float

Raises

- **KeyError** – If the partition is not a partition of all graph nodes
- **ValueError** – If the graph has no link
- **TypeError** – If graph is not a `networkx.Graph`

References

structure in networks. Physical Review E 69, 26113(2004).

Examples

```
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> part = best_partition(G)
>>> modularity(part, G)
```

```
py3plex.algorithms.community_detection.community_louvain.partition_at_level(dendrogram:
                                                                            List[Dict],
                                                                            level: int) →
                                                                            Dict
```

Return the partition of the nodes at the given level

A dendrogram is a tree and each level is a partition of the graph nodes. Level 0 is the first partition, which contains the smallest communities, and the best is `len(dendrogram) - 1`. The higher the level is, the bigger are the communities

Parameters

- **dendrogram** (*list of dict*) – a list of partitions, ie dictionnaires where keys of the `i+1` are the values of the `i`.
- **level** (*int*) – the level which belongs to `[0..len(dendrogram)-1]`

Returns

partition – A dictionary where keys are the nodes and the values are the set it belongs to

Return type

dictionnary

Raises

KeyError – If the dendrogram is not well formed or the level is too high

See also

[*best_partition*](#), [*generate_dendrogram*](#)

Examples

```
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> dendrogram = generate_dendrogram(G)
>>> for level in range(len(dendrogram) - 1) :
>>>     print("partition at level", level, "is", partition_at_level(dendrogram,
↪ level)) # NOQA
```

Community quality measures and metrics.

This module provides various measures for assessing the quality of community partitions in networks, including modularity, size distribution, and other statistical metrics.

```
py3plex.algorithms.community_detection.community_measures.modularity(G: Graph,  
                                                                    communities:  
                                                                    Dict[Any, List[Any]],  
                                                                    weight: str =  
                                                                    'weight' → float
```

Calculate modularity of a graph partition.

Parameters

- **G** – NetworkX graph
- **communities** – Dictionary mapping community IDs to node lists
- **weight** – Edge weight attribute (default: “weight”)

Returns

Modularity value

```
py3plex.algorithms.community_detection.community_measures.number_of_communities(network_partition:  
                                                                                Dict[Any,  
                                                                                List[Any]])  
                                                                                → int
```

Count number of communities in a partition.

Parameters

network_partition – Dictionary mapping community IDs to node lists

Returns

Number of communities

```
py3plex.algorithms.community_detection.community_measures.size_distribution(network_partition:  
                                                                            Dict[Any,  
                                                                            List[Any]])  
                                                                            → ndarray
```

Calculate size distribution of communities.

Parameters

network_partition – Dictionary mapping community IDs to node lists

Returns

Array of community sizes

Multilayer Synthetic Graph Generation for Community Detection Benchmarks

This module implements synthetic multilayer/multiplex graph generators with ground-truth community structure for benchmarking community detection algorithms.

Includes: - Multilayer LFR (Lancichinetti-Fortunato-Radicchi) benchmark - Coupled/Interdependent Erdős-Rényi models - Support for overlapping communities across layers - Support for partial node presence across layers

References

Lancichinetti et al., “Benchmark graphs for testing community detection algorithms”, Phys. Rev. E 78, 046110 (2008)

Granell et al., “Benchmark model to assess community structure in evolving networks”, Phys. Rev. E 92, 012805 (2015)

```

py3plex.algorithms.community_detection.multilayer_benchmark.generate_coupled_er_multilayer(n:
int,
lay-
ers:
List[str],
p:
float
|
List[float]
=
0.1,
omega:
float
=
1.0,
cou-
pling_proba-
bility:
float
=
1.0,
di-
rected:
bool
=
False,
seed:
int
|
None
=
None)
→
Any

```

Generate coupled/interdependent Erdős-Rényi multilayer networks.

Creates random Erdős-Rényi graphs in each layer and couples nodes across layers with specified coupling strength and probability.

Parameters

- **n** – Number of nodes
- **layers** – List of layer names
- **p** – Edge probability per layer. Can be float (same for all layers) or list of floats per layer.
- **omega** – Inter-layer coupling strength (weight of identity links)
- **coupling_probability** – Probability that a node has inter-layer coupling. Range: [0, 1] - 1.0 = all nodes coupled (full multiplex) - <1.0 = partial coupling (interdependent networks)
- **directed** – Whether to generate directed networks
- **seed** – Random seed for reproducibility

Returns

py3plex multi_layer_network object

Examples

```
>>> from py3plex.algorithms.community_detection.multilayer_benchmark import generate_coupled_er_multilayer
↳generate_coupled_er_multilayer
>>>
>>> # Full multiplex ER network
>>> network = generate_coupled_er_multilayer(
...     n=100,
...     layers=['L1', 'L2', 'L3'],
...     p=0.1,
...     omega=1.0,
...     coupling_probability=1.0
... )
>>>
>>> # Partially coupled (interdependent)
>>> network = generate_coupled_er_multilayer(
...     n=100,
...     layers=['L1', 'L2'],
...     p=0.1,
...     omega=0.5,
...     coupling_probability=0.5 # Only 50% nodes coupled
... )
```

```

py3plex.algorithms.community_detection.multilayer_benchmark.generate_multilayer_lfr(n:
    int,
    lay-
    ers:
    List[str],
    tau1:
    float
    =
    2.0,
    tau2:
    float
    =
    1.5,
    mu:
    float
    |
    List[float]
    =
    0.1,
    avg_degree:
    float
    |
    List[float]
    =
    10.0,
    min_community:
    int
    =
    20,
    max_community:
    int
    |
    None
    =
    None,
    com-
    mu-
    nity_persistence:
    float
    =
    1.0,
    node_overlap:
    float
    =
    1.0,
    over-
    lap-
    ping_nodes:
    int
    =
    0,
    over-
    lap-
    ping_membership:
    int
    =
    2,
    di-
    rected:
    bool
    =
    False,
    seed:
    int
    |
    None
    =
    None)

```

Generate multilayer LFR benchmark networks with controllable community structure.

This extends the LFR benchmark to multilayer networks, allowing control over: - Community persistence across layers (how many nodes keep their community) - Node overlap across layers (which nodes appear in which layers) - Overlapping communities (nodes belonging to multiple communities)

Parameters

- **n** – Number of nodes
- **layers** – List of layer names
- **tau1** – Power-law exponent for degree distribution (typically 2-3)
- **tau2** – Power-law exponent for community size distribution (typically 1-2)
- **mu** – Mixing parameter (fraction of edges outside community). Can be float (same for all layers) or list of floats per layer. Range: [0, 1], where 0 = perfect communities, 1 = random
- **avg_degree** – Average degree per layer. Can be float (same for all layers) or list of floats per layer.
- **min_community** – Minimum community size
- **max_community** – Maximum community size (default: $n/2$)
- **community_persistence** – Probability that a node keeps its community from one layer to the next. Range: [0, 1] - 1.0 = identical communities across all layers - 0.0 = completely independent communities per layer
- **node_overlap** – Fraction of nodes present in all layers. Range: [0, 1] - 1.0 = all nodes in all layers (full multiplex) - <1.0 = some nodes absent from some layers
- **overlapping_nodes** – Number of nodes that belong to multiple communities within each layer
- **overlapping_membership** – Number of communities each overlapping node belongs to
- **directed** – Whether to generate directed networks
- **seed** – Random seed for reproducibility

Returns

Tuple of (network, ground_truth_communities) - network: py3plex multi_layer_network object - ground_truth_communities: Dict mapping (node, layer) to Set of community IDs

Examples

```
>>> from py3plex.algorithms.community_detection.multilayer_benchmark import _
    generate_multilayer_lfr
>>>
>>> # Generate with identical communities across layers
>>> network, communities = generate_multilayer_lfr(
...     n=100,
...     layers=['L1', 'L2', 'L3'],
...     mu=0.1,
...     community_persistence=1.0
... )
>>>
>>> # Generate with evolving communities
>>> network, communities = generate_multilayer_lfr(
...     n=100,
...     layers=['T0', 'T1', 'T2'],
```

(continues on next page)

(continued from previous page)

```

...     mu=0.1,
...     community_persistence=0.7 # 70% nodes keep community
... )

```

```

py3plex.algorithms.community_detection.multilayer_benchmark.generate_sbm_multilayer(n:
int,
lay-
ers:
List[str],
com-
mu-
ni-
ties:
List[Set[int]],
p_in:
float
|
List[float]
=
0.3,
p_out:
float
|
List[float]
=
0.05,
com-
mu-
nity_persistence:
float
=
1.0,
di-
rected:
bool
=
False,
seed:
int
|
None
=
None)
→
Tu-
ple[Any,
Dict[Tuple[Any,
str],
int]]

```

Generate multilayer stochastic block model (SBM) networks.

Creates networks where nodes are divided into communities with different intra- and inter-community connection probabilities.

Parameters

- **n** – Number of nodes
- **layers** – List of layer names

- **communities** – List of node sets defining initial communities
- **p_in** – Intra-community edge probability per layer
- **p_out** – Inter-community edge probability per layer
- **community_persistence** – Probability nodes keep their community across layers
- **directed** – Whether to generate directed networks
- **seed** – Random seed for reproducibility

Returns

Tuple of (network, ground_truth_communities) - network: py3plex multi_layer_network object - ground_truth_communities: Dict mapping (node, layer) to community ID

Examples

```
>>> from py3plex.algorithms.community_detection.multilayer_benchmark import _  
→ generate_sbm_multilayer  
>>>  
>>> # Define initial communities  
>>> communities = [  
...     {0, 1, 2, 3, 4}, # Community 0  
...     {5, 6, 7, 8, 9} # Community 1  
... ]  
>>>  
>>> network, ground_truth = generate_sbm_multilayer(  
...     n=10,  
...     layers=['L1', 'L2'],  
...     communities=communities,  
...     p_in=0.7,  
...     p_out=0.1,  
...     community_persistence=0.8  
... )
```

Statistics

Multilayer Network Statistics

This module implements various statistics for multilayer and multiplex networks, following standard definitions from multilayer network analysis literature.

References

- Kivelä et al. (2014), “Multilayer networks”, J. Complex Networks 2(3), 203-271
- De Domenico et al. (2013), “Mathematical formulation of multilayer networks”, PRX 3, 041022
- Mucha et al. (2010), “Community Structure in Time-Dependent, Multiscale, and Multiplex Networks”, Science 328, 876-878

Authors: py3plex contributors Date: 2025

py3plex.algorithms.statistics.multilayer_statistics.**algebraic_connectivity**(network: Any)
→ float

Calculate algebraic connectivity (λ_2).

Formula: $\lambda_2()$

Second smallest eigenvalue of the supra-Laplacian (Fiedler value).

Indicates global connectivity and diffusion efficiency of the multilayer system.

Properties:

$\lambda_0 = 0$ always (associated with constant eigenvector) $\lambda_1 > 0$ if and only if the multilayer network is connected. Larger λ_1 indicates better connectivity and faster diffusion/synchronization.

Parameters

network – py3plex multi_layer_network object

Returns

Second smallest eigenvalue (Fiedler value)

Examples

```
>>> alg_conn = algebraic_connectivity(network)
```

Reference:

Fiedler (1973), Sole-Ribalta et al. (2013)

```
py3plex.algorithms.statistics.multilayer_statistics.community_participation_coefficient(network: Any, communities: Dict[Tuple[Any, Any], int], node: Any) → float
```

Calculate participation coefficient for a node across community structure.

Measures how evenly a node's connections are distributed across different communities, across all layers. A node with connections to many communities has high participation.

Formula: $P = 1 - (k_s / k)^2$

where k_s is the number of connections node i has to community s , and k is the total degree of node i across all layers.

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) to community ID
- **node** – Node identifier (not node-layer tuple)

Returns

Participation coefficient value between 0 and 1

Examples

```
>>> communities = detect_communities(network)
>>> pc = community_participation_coefficient(network, communities, 'Alice')
>>> print(f"Participation: {pc:.3f}")
```

Reference:

Guimerà & Amaral (2005), "Functional cartography of complex metabolic networks"

```
py3plex.algorithms.statistics.multilayer_statistics.community_participation_entropy(network: Any, communities: Dict[Tuple[Any, Any], int], node: Any) → float
```

Calculate participation entropy for a node across community structure.

Shannon entropy-based measure of how evenly a node distributes its connections across different communities. Higher entropy indicates more diverse community participation.

Formula: $H = - (k / k) \log(k / k)$

where k is connections to community s , k is total degree.

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) to community ID
- **node** – Node identifier (not node-layer tuple)

Returns

Entropy value (higher = more diverse participation)

Examples

```
>>> entropy = community_participation_entropy(network, communities, 'Alice')
>>> print(f"Participation entropy: {entropy:.3f}")
```

Reference:

Based on Shannon entropy applied to community structure

```
py3plex.algorithms.statistics.multilayer_statistics.compute_modularity_score(network: Any, communities: Dict[Tuple[Any, Any], int], gamma: float = 1.0, omega: float = 1.0) → float
```

Compute explicit multislice modularity score.

Direct computation of the modularity quality function for a given community partition, without running detection algorithms.

Formula: $Q = (1/2) [(A - \cdot kk/(2m)) + \cdot] (c, c)$

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) to community ID

- **gamma** – Resolution parameter (default: 1.0)
- **omega** – Inter-layer coupling strength (default: 1.0)

Returns

Modularity score Q (higher is better)

Examples

```
>>> communities = {('A', 'L1'): 0, ('B', 'L1'): 0, ('C', 'L1'): 1}
>>> Q = compute_modularity_score(network, communities)
>>> print(f"Modularity: {Q:.3f}")
```

Reference:

Mucha et al. (2010), Science 328, 876-878

```
py3plex.algorithms.statistics.multilayer_statistics.cross_layer_mutual_information(network:
Any,
layer_i:
str,
layer_j:
str,
bins:
int
=
10)
→
float
```

Calculate cross-layer mutual information ($I(L; L)$).

Formula: $I(L; L) = H(L) + H(L) - H(L, L)$

Measures statistical dependence between degree distributions in two layers; quantifies how much knowing a node's degree in one layer tells us about its degree in another layer.

Variables:

$H(L)$ = entropy of degree distribution in layer i $H(L)$ = entropy of degree distribution in layer j $H(L, L)$ = joint entropy of degree distributions

Properties:

$I = 0$ when layers are independent $I > 0$ indicates statistical dependence (higher = stronger) $I(L; L)$ $\min(H(L), H(L))$

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier
- **bins** – Number of bins for discretizing degree distributions

Returns

Mutual information value in bits

Examples

```
>>> mi = cross_layer_mutual_information(network, 'L1', 'L2', bins=10)
>>> print(f"Mutual information: {mi:.3f} bits")
```

Reference:

Cover & Thomas (2006), “Elements of Information Theory” De Domenico et al. (2015), “Structural reducibility”

`py3plex.algorithms.statistics.multilayer_statistics.cross_layer_redundancy_entropy`(*network: Any*)
→
float

Calculate cross-layer redundancy entropy ($H_{\text{redundancy}}$).

Formula: $H_r = -r \log_2(r)$, where r is the normalized edge overlap between layers i and j .

Measures diversity in structural redundancy across layer pairs; high entropy indicates varied overlap patterns, low entropy indicates uniform redundancy.

Variables:

$r = \text{edge_overlap}(i,j) / \text{edge_overlap}(,)$ Normalized overlap proportion for each layer pair

Parameters

network – py3plex multi_layer_network object

Returns

Entropy value in bits

Examples

```
>>> entropy = cross_layer_redundancy_entropy(network)
>>> print(f"Cross-layer redundancy entropy: {entropy:.3f}")
```

Reference:

Bianconi (2018), “Multilayer Networks: Structure and Function”

`py3plex.algorithms.statistics.multilayer_statistics.degree_vector`(*network: Any, node: Any, weighted: bool = False*)
→ Dict[str, float]

Calculate degree vector (k).

Formula: $k = (k^1, k^2, \dots, k)$

Node degree in each layer; can be analyzed via mean, variance, or entropy to capture node versatility.

Variables:

k = degree of node i in layer A For undirected: $k = A$

Parameters

- **network** – py3plex multi_layer_network object
- **node** – Node identifier
- **weighted** – If True, return strength instead of degree

Returns

Dictionary mapping layer to degree/strength

Examples

```
>>> degrees = degree_vector(network, 'A')
>>> print(f"Degree in layer L1: {degrees['L1']}")
```

Reference:

Kivela et al. (2014), J. Complex Networks 2(3), 203-271

`py3plex.algorithms.statistics.multilayer_statistics.edge_overlap(network: Any, layer_i: str, layer_j: str) → float`

Calculate edge overlap (^).

Formula: $\wedge = |E_i \cap E_j| / |E_i \cup E_j|$

Jaccard similarity of edge sets between two layers; measures structural redundancy.

Variables:

E_i = set of edges in layer i E_j = set of edges in layer j $| \cdot |$ = cardinality (number of elements)

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns

Overlap coefficient between 0 and 1 (Jaccard similarity)

Examples

```
>>> overlap = edge_overlap(network, 'L1', 'L2')
```

Reference:

Kivela et al. (2014), J. Complex Networks 2(3), 203-271

`py3plex.algorithms.statistics.multilayer_statistics.entropy_of_multiplexity(network: Any) → float`

Calculate entropy of multiplexity (H).

Formula: $H = - \sum p \log_2(p)$, where $p = E_i / E$

Shannon entropy of layer contributions; measures layer diversity.

Variables:

p_i = proportion of edges in layer i E_i = number of edges in layer i $\log_2$ gives entropy in bits

Properties:

$H = 0$ when all edges are in one layer (minimum entropy/diversity) $H = \log_2(L)$ when edges are uniformly distributed across L layers (maximum entropy)

Parameters

network – py3plex multi_layer_network object

Returns

Entropy value in bits

Examples

```
>>> entropy = entropy_of_multiplexity(network)
```

Reference:

De Domenico et al. (2013), Shannon (1948)

`py3plex.algorithms.statistics.multilayer_statistics.inter_layer_assortativity(network: Any, layer_i: str, layer_j: str) → float`

Calculate inter-layer assortativity (r).

Formula: $r = \text{cov}(k^i, k^j) / (\sigma_i \sigma_j) = \text{corr}(k^i, k^j)$

Measures whether nodes with similar degrees tend to connect across different layers.

Variables:

k^i = degree vector in layer i k^j = degree vector in layer j , σ_i = standard deviations of degrees in layers i and j Equivalent to Pearson correlation of degree vectors

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns

Assortativity coefficient

Examples

```
>>> assort = inter_layer_assortativity(network, 'L1', 'L2')
```

Reference:

Newman (2002), Nicosia & Latora (2015)

```
py3plex.algorithms.statistics.multilayer_statistics.inter_layer_coupling_strength(network:  
Any,  
layer_i:  
str,  
layer_j:  
str)  
→  
float
```

Calculate inter-layer coupling strength (C).

Formula: $C = (1/N) \sum w_{ij}$

Average weight of inter-layer connections between corresponding nodes in two layers. Quantifies cross-layer connectivity.

Variables:

N = number of nodes present in both layers and w_{ij} = weight of inter-layer edge connecting node i in layer i to node j in layer j

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns

Average coupling strength

Examples

```
>>> coupling = inter_layer_coupling_strength(network, 'L1', 'L2')
```


Reference:

De Domenico et al. (2013), Physical Review X 3(4), 041022

Contracts:

- Precondition: network must not be None
- Precondition: layer_i and layer_j must be non-empty strings
- Postcondition: result is non-negative (weights are non-negative)
- Postcondition: result is not NaN

```
py3plex.algorithms.statistics.multilayer_statistics.inter_layer_degree_correlation(network:
Any,
layer_i:
str,
layer_j:
str)
→
float
```

Calculate inter-layer degree correlation ($r^{\wedge}$).

Formula: $r^{\wedge} = (k - \bar{k})(k - \bar{k}) / [((k - \bar{k})^2)((k - \bar{k})^2)]$

Pearson correlation of node degrees between two layers; reveals if highly connected nodes in one layer are also central in others.

Variables:

k = degree of node i in layer k = mean degree in layer Sum over nodes present in both layers

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns

Pearson correlation coefficient between -1 and 1

Examples

```
>>> corr = inter_layer_degree_correlation(network, 'L1', 'L2')
```

Reference:

Battiston et al. (2014), Nicosia & Latora (2015)

```
py3plex.algorithms.statistics.multilayer_statistics.inter_layer_dependence_entropy(network:
Any,
layer_i:
str,
layer_j:
str)
→
float
```

Calculate inter-layer dependence entropy (H_{dep}).

Formula: $H_{dep} = -p \log_2(p)$, where p is the proportion of inter-layer edges for each node n connecting layers i and j .

Measures heterogeneity in how nodes couple the two layers; high entropy indicates diverse coupling patterns, low entropy indicates uniform coupling.

Variables:

p = proportion of inter-layer edges incident to node n Total over all nodes connecting the two layers

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier

Returns

Entropy value in bits

Examples

```
>>> entropy = inter_layer_dependence_entropy(network, 'L1', 'L2')
>>> print(f"Inter-layer dependence entropy: {entropy:.3f}")
```

Reference:

De Domenico et al. (2015), “Ranking in interconnected multilayer networks”

```
py3plex.algorithms.statistics.multilayer_statistics.interdependence(network: Any,
                                                                    sample_size: int =
                                                                    100) → float
```

Calculate interdependence ().

Formula: $= \langle d \rangle / \langle d \rangle$

Quantifies how much shortest-path communication depends on inter-layer connections.

Variables:

d = shortest path from node i to node j in the full multilayer network $d = (1/L) \sum d$ is the average shortest path across individual layers $\langle \cdot \rangle$ = average over sampled node pairs

Interpretation:

< 1 : multilayer connectivity reduces path lengths (positive interdependence) 1 : inter-layer connections provide little benefit > 1 : multilayer structure increases path lengths (rare)

Parameters

- **network** – py3plex multi_layer_network object
- **sample_size** – Number of node pairs to sample for estimation

Returns

Interdependence ratio

Examples

```
>>> interdep = interdependence(network, sample_size=50)
```

Reference:

Gomez et al. (2013), Buldyrev et al. (2010)

```
py3plex.algorithms.statistics.multilayer_statistics.interlayer_degree_correlation_matrix(network:
                                                                                          Any)
                                                                                          →
                                                                                          Tu-
                                                                                          ple[ndarray,
                                                                                          list]
```

Calculate inter-layer degree correlation matrix.

Computes Pearson correlation coefficients for node degrees between all pairs of layers, organized as a symmetric correlation matrix.

Formula: $\text{Matrix}[i, j] = r^{ij} = \text{corr}(k^i, k^j)$

where k^i and k^j are degree vectors for layers i and j over common nodes.

Properties:

- Diagonal elements are 1.0 (self-correlation)
- Off-diagonal elements in $[-1, 1]$
- Symmetric matrix
- Positive values indicate positive degree correlation
- Negative values indicate negative degree correlation

Parameters

network – py3plex multi_layer_network object

Returns

- **correlation_matrix**: 2D numpy array of shape (num_layers, num_layers)
- **layer_labels**: List of layer names corresponding to matrix indices

Return type

Tuple of (correlation_matrix, layer_labels)

Examples

```
>>> corr_matrix, layers = interlayer_degree_correlation_matrix(network)
>>> import matplotlib.pyplot as plt
>>> import seaborn as sns
>>> sns.heatmap(corr_matrix, annot=True, xticklabels=layers,
...             yticklabels=layers, cmap='coolwarm', center=0,
...             vmin=-1, vmax=1)
>>> plt.title('Inter-layer Degree Correlation Matrix')
>>> plt.show()
```

Reference:

Nicosia & Latora (2015), “Measuring and modeling correlations in multiplex networks” Battiston et al. (2014), “Structural measures for multiplex networks”

```
py3plex.algorithms.statistics.multilayer_statistics.layer_connectivity_entropy(network:
                                                                              Any,
                                                                              layer:
                                                                              str) →
                                                                              float
```

Calculate entropy of layer connectivity ($H_{\text{connectivity}}$).

Formula: $H_c = - (k/k) \log_2(k/k)$

Shannon entropy of degree distribution within a layer; measures heterogeneity of node connectivity patterns.

Variables:

k = degree of node i in the layer k = sum of all degrees ($2 * \text{edges}$ for undirected)

Properties:

$H_c = 0$ when all nodes have the same degree (uniform distribution) H_c is maximized when degree distribution is highly uneven

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier

Returns

Entropy value in bits

Examples

```
>>> from py3plex.core import multinet
>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1]
... ], input_type='list')
>>> entropy = layer_connectivity_entropy(network, 'L1')
>>> print(f"Connectivity entropy: {entropy:.3f}")
```

Reference:

Solé-Ribalta et al. (2013), “Spectral properties of complex networks” Shannon (1948), “A Mathematical Theory of Communication”

py3plex.algorithms.statistics.multilayer_statistics.**layer_density**(network: Any, layer: str) → float

Calculate layer density ().

Formula:
$$= (2E) / (N(N - 1)) \text{ [undirected]}$$
$$= E / (N(N - 1)) \text{ [directed]}$$

Measures the fraction of possible edges present in a specific layer, indicating how densely connected that layer is.

Variables:

E = number of edges in layer N = number of nodes in layer

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier

Returns

Density value between 0 and 1

Examples

```
>>> from py3plex.core import multinet
>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1]
... ], input_type='list')
>>> density = layer_density(network, 'L1')
>>> print(f"Layer L1 density: {density:.3f}")
```

Reference:

Kivela et al. (2014), J. Complex Networks 2(3), 203-271

Contracts:

- Precondition: network must not be None
- Precondition: layer must be a non-empty string
- Postcondition: result is in [0, 1] (fundamental property of density)
- Postcondition: result is not NaN

```
py3plex.algorithms.statistics.multilayer_statistics.layer_influence_centrality(network:
Any,
layer:
str,
method:
str =
'cou-
pling',
sam-
ple_size:
int =
100) →
float
```

Calculate layer influence centrality (I).

Formula (coupling): $I = C^L / (L-1)$ Formula (flow): $I = F^L / (L-1)$

Quantifies how much a layer influences other layers through inter-layer connections (coupling method) or information flow (flow method).

Variables:

C^L = inter-layer coupling strength between layers and F^L = flow from layer to layer (random walk transition probability) L = total number of layers

Properties:

Higher values indicate layers that strongly influence others Useful for identifying critical layers in the multilayer structure

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier
- **method** – ‘coupling’ for structural influence, ‘flow’ for dynamic influence
- **sample_size** – Number of random walk steps for flow simulation

Returns

Influence centrality value

Examples

```
>>> influence = layer_influence_centrality(network, 'L1', method='coupling')
>>> print(f"Layer L1 influence: {influence:.3f}")
```

Reference:

Cozzo et al. (2013), “Mathematical formulation of multilayer networks” De Domenico et al. (2014), “Identifying modular flows”

```
py3plex.algorithms.statistics.multilayer_statistics.layer_redundancy_coefficient(network: Any,
                                                                              layer_i: str,
                                                                              layer_j: str)
→ float
```

Calculate layer redundancy coefficient.

Measures the proportion of edges in one layer that are redundant (also present) in another layer. Values close to 1 indicate high redundancy, while values close to 0 indicate complementary layers.

Formula: $R = |E_i \cap E_j| / |E_i|$

where E_i and E_j are edge sets of layers i and j .

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier

Returns

Redundancy coefficient between 0 and 1

Examples

```
>>> redundancy = layer_redundancy_coefficient(network, 'social', 'work')
>>> print(f"Redundancy: {redundancy:.2%}")
```

Reference:

Nicosia & Latora (2015), “Measuring and modeling correlations in multiplex networks”

```
py3plex.algorithms.statistics.multilayer_statistics.layer_similarity(network: Any, layer_i: str, layer_j: str,
                                                                    method: str = 'cosine') → float
```

Calculate layer similarity (S^A).

Formula: $S^A = \langle A, A \rangle / (\|A\| \|A\|) = AA / ((A)^2) ((A)^2)$

Cosine or Jaccard similarity between adjacency matrices of two layers.

Variables:

A, A = adjacency matrices for layers i and j and $\langle \cdot, \cdot \rangle$ = Frobenius inner product $\|\cdot\|$ = Frobenius norm

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()
- **method** – ‘cosine’ or ‘jaccard’

Returns

Similarity value between 0 and 1

Examples

```
>>> similarity = layer_similarity(network, 'L1', 'L2', method='cosine')
```

Reference:

De Domenico et al. (2013), Physical Review X 3(4), 041022

`py3plex.algorithms.statistics.multilayer_statistics.multilayer_betweenness_surface`(*network*: Any, *normalized*: bool, *weight*: str | None) → ndarray

Calculate multilayer betweenness surface (tensor representation).

Computes betweenness centrality for each node-layer pair and organizes the results as a 2D array (nodes × layers) that can be visualized as a heatmap or surface plot.

Formula: $\text{Surface}[i,] = B$

where B is the betweenness centrality of node i in layer .

Parameters

- **network** – py3plex multi_layer_network object
- **normalized** – Whether to normalize betweenness values
- **weight** – Edge weight attribute name (None for unweighted)

Returns

2D numpy array of shape (num_nodes, num_layers) containing betweenness values. Also returns tuple of (node_labels, layer_labels) for axis labeling.

Examples

```
>>> surface, (nodes, layers) = multilayer_betweenness_surface(network)
>>> import matplotlib.pyplot as plt
>>> plt.imshow(surface, aspect='auto', cmap='viridis')
>>> plt.xlabel('Layers')
>>> plt.ylabel('Nodes')
>>> plt.xticks(range(len(layers)), layers)
>>> plt.yticks(range(len(nodes)), nodes)
>>> plt.colorbar(label='Betweenness Centrality')
>>> plt.title('Multilayer Betweenness Surface')
>>> plt.show()
```

Reference:

De Domenico et al. (2015), “Structural reducibility of multilayer networks”

```
py3plex.algorithms.statistics.multilayer_statistics.multilayer_clustering_coefficient(network:  
Any,  
node:  
Any  
|  
None  
=  
None)  
→  
float  
|  
Dict[Any,  
float]
```

Calculate multilayer clustering coefficient (C).

Formula: $C = T / T$

Extends transitivity to account for triangles that span multiple layers.

Variables:

T = number of closed triplets (triangles) involving node i across all layers T = maximum possible triplets = $k(k - 1)/2$ for undirected networks Average over all nodes: $C = (1/N) C$

Parameters

- **network** – py3plex multi_layer_network object
- **node** – If specified, compute for single node; otherwise compute for all

Returns

Clustering coefficient value or dict of values per node

Examples

```
>>> clustering = multilayer_clustering_coefficient(network)  
>>> node_clustering = multilayer_clustering_coefficient(network, node='A')
```

Reference:

Battiston et al. (2014), Section III.C

```
py3plex.algorithms.statistics.multilayer_statistics.multilayer_modularity(network: Any,  
communities:  
Dict[Tuple[Any,  
Any], int],  
gamma: float |  
Dict[Any, float]  
= 1.0, omega:  
float | ndarray  
= 1.0, weight:  
str = 'weight')  
→ float
```

Calculate multilayer modularity (Q).

This is a wrapper for the existing multilayer_modularity implementation in py3plex.algorithms.community_detection.multilayer_modularity.

Formula: $Q = (1/2) [(A - P) +] (g, g)$

Extension of Newman-Girvan modularity to multiplex networks (Mucha et al., 2010). Measures community quality across layers.

Variables:

w = total edge weight in supra-network A = adjacency matrix element for layer $P = k/(2m)$ is the null model (configuration model) γ = resolution parameter for layer ω = inter-layer coupling strength $\delta_{ij} = \text{Kronecker delta}$ (1 if $i=j$, 0 otherwise) $\delta_{(g,i)(g,j)} = \text{Kronecker delta}$ (1 if $i=j$, 0 otherwise) $(g, g) = 1$ if node i in layer g and node j in layer g are in same community

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) tuples to community IDs
- **gamma** – Resolution parameter(s)
- **omega** – Inter-layer coupling strength
- **weight** – Edge weight attribute

Returns

Modularity value Q

Examples

```
>>> communities = {'A', 'L1': 0, ('B', 'L1'): 0, ('C', 'L1'): 1}
>>> Q = multilayer_modularity(network, communities)
```

Reference:

Mucha et al. (2010), Science 328(5980), 876-878

`py3plex.algorithms.statistics.multilayer_statistics.multilayer_motif_frequency`(*network*: Any, *motif_size*: int = 3) → Dict[str, float]

Calculate multilayer motif frequency (f).

Formula: $f = n / n$

Frequency of recurring subgraph patterns across layers.

Variables:

n = count of motif type m n = total count of all motifs

Note: This is a simplified implementation counting basic patterns (intra-layer vs. inter-layer triangles). Complete multilayer motif enumeration includes many more configurations and is computationally expensive.

Parameters

- **network** – py3plex multi_layer_network object
- **motif_size** – Size of motifs to count (default: 3 for triangles)

Returns

Dictionary of motif type frequencies

Examples

```
>>> motifs = multilayer_motif_frequency(network, motif_size=3)
```

Reference:

Battiston et al. (2014), Section IV

```
py3plex.algorithms.statistics.multilayer_statistics.multiplex_betweenness centrality(network:
                                                                                     Any,
                                                                                     nor-
                                                                                     mal-
                                                                                     ized:
                                                                                     bool
                                                                                     =
                                                                                     True,
                                                                                     weight:
                                                                                     str
                                                                                     |
                                                                                     None
                                                                                     =
                                                                                     None)
                                                                                     →
                                                                                     Dict[Tuple[Any,
                                                                                     Any],
                                                                                     float]
```

Calculate multiplex betweenness centrality.

Computes betweenness centrality on the supra-graph, accounting for paths that traverse inter-layer couplings. This extends the standard betweenness definition to multiplex networks where paths can cross layers.

Formula: $B = ((i) /)$

where i is the total number of shortest paths from s to t , and (i) is the number of those paths passing through node i in layer $.$

Parameters

- **network** – py3plex multi_layer_network object
- **normalized** – Whether to normalize by the number of node pairs
- **weight** – Edge weight attribute name (None for unweighted)

Returns

Dictionary mapping (node, layer) tuples to betweenness centrality values

Examples

```
>>> betweenness = multiplex_betweenness centrality(network)
>>> top_nodes = sorted(betweenness.items(), key=lambda x: x[1], reverse=True)[:5]
```

Reference:

De Domenico et al. (2015), “Structural reducibility of multilayer networks”

```

py3plex.algorithms.statistics.multilayer_statistics.multiplex_closeness centrality(network:
Any,
nor-
mal-
ized:
bool
=
True,
weight:
str |
None
=
None,
vari-
ant:
str
=
'stan-
dard')
→
Dict[Tuple[Any,
Any],
float]

```

Calculate multiplex closeness centrality.

Computes closeness centrality on the supra-graph, where shortest paths can traverse inter-layer edges. This captures how quickly a node-layer can reach all other node-layers in the multiplex network.

Standard closeness formula: $C = (N*L - 1) / d(i, j)$

Harmonic closeness formula: $HC = 1/d(i, j)$

where $d(i, j)$ is the shortest path distance from node i in layer to node j in layer , and $N*L$ is the total number of node-layer pairs.

Parameters

- **network** – py3plex multi_layer_network object
- **normalized** – Whether to normalize by network size
- **weight** – Edge weight attribute name (None for unweighted)
- **variant** – Closeness variant to use. Options: - ‘standard’: Classic closeness (reciprocal of sum of distances).

Can produce biased values for nodes in disconnected components.

- ‘harmonic’: Harmonic closeness (sum of reciprocal distances). Recommended for disconnected multilayer networks.
- ‘auto’: Automatically selects ‘harmonic’ if the network has multiple connected components, otherwise uses ‘standard’.

Default is ‘standard’ for backward compatibility.

Returns

Dictionary mapping (node, layer) tuples to closeness centrality values

Examples

```
>>> closeness = multiplex_closeness_centrality(network)
>>> central_nodes = {k: v for k, v in closeness.items() if v > 0.5}
```

```
>>> # For disconnected networks, use harmonic variant
>>> closeness = multiplex_closeness_centrality(network, variant='harmonic')
```

Reference:

De Domenico et al. (2015), “Structural reducibility of multilayer networks” Boldi, P., & Vigna, S. (2014). Axioms for Centrality. Internet Math.

`py3plex.algorithms.statistics.multilayer_statistics.multiplex_rich_club_coefficient`(*network*: Any, *k*: int, *normalized*: bool = True) → float

Calculate multiplex rich-club coefficient.

Measures the tendency of high-degree nodes to be more densely connected to each other than expected by chance, accounting for the multiplex structure.

Formula: $(k) = E(>k) / (N(>k) * (N(>k)-1) / 2)$

where $E(>k)$ is the number of edges among nodes with overlapping degree $> k$, and $N(>k)$ is the number of such nodes.

Parameters

- **network** – py3plex multi_layer_network object
- **k** – Degree threshold
- **normalized** – Whether to normalize by random expectation

Returns

Rich-club coefficient value

Examples

```
>>> rich_club = multiplex_rich_club_coefficient(network, k=10)
>>> print(f"Rich-club coefficient: {rich_club:.3f}")
```

Reference:

Alstott et al. (2014), “powerlaw: A Python Package for Analysis of Heavy-Tailed Distributions” Extended to multiplex networks

`py3plex.algorithms.statistics.multilayer_statistics.node_activity`(*network*: Any, *node*: Any) → float

Calculate node activity (a).

Formula: $a = (1/L) \sum (v \in V)$

Fraction of layers in which node i is active (has at least one connection).

Variables:

L = total number of layers ($v \in V$) = indicator function (1 if node i is active in layer v , 0 otherwise) V = set of active nodes in layer

Parameters

- **network** – py3plex multi_layer_network object
- **node** – Node identifier

Returns

Activity value between 0 and 1

Examples

```
>>> activity = node_activity(network, 'A')
```

Reference:

Kivelä et al. (2014), J. Complex Networks 2(3), 203-271

Contracts:

- Precondition: network must not be None
- Precondition: node must not be None
- Postcondition: result is in $[0, 1]$ (fraction of layers)
- Postcondition: result is not NaN

```
py3plex.algorithms.statistics.multilayer_statistics.percolation_threshold(network: Any,
                                removal_strategy: str = 'random',
                                trials: int = 10) → float
```

Estimate percolation threshold for the multiplex network.

Determines the fraction of nodes that must be removed before the network fragments into disconnected components. Uses sampling to estimate threshold.

Parameters

- **network** – py3plex multi_layer_network object
- **removal_strategy** – 'random', 'degree', or 'betweenness'
- **trials** – Number of trials for averaging

Returns

Estimated percolation threshold (fraction of nodes)

Examples

```
>>> threshold = percolation_threshold(network, removal_strategy='degree')
>>> print(f"Percolation threshold: {threshold:.2%}")
```

Reference:

Buldyrev et al. (2010), "Catastrophic cascade of failures in interdependent networks"

```
py3plex.algorithms.statistics.multilayer_statistics.resilience(network: Any,
                        perturbation_type: str = 'layer_removal',
                        perturbation_param: str | float | None = None) → float
```

Calculate resilience (R).

Formula: $R = S' / S_0$

Ratio of largest connected component after perturbation to original size.

Variables:

S_0 = size of largest connected component in original network S' = size of largest connected component after perturbation

Perturbation types:

1. Layer removal: Remove all nodes/edges in a specific layer
2. Coupling removal: Remove a fraction of inter-layer edges

Properties:

$R = 1$ indicates full resilience (no impact from perturbation) $R = 0$ indicates complete fragmentation $0 < R < 1$ indicates partial resilience

Parameters

- **network** – py3plex multi_layer_network object
- **perturbation_type** – 'layer_removal' or 'coupling_removal'
- **perturbation_param** – Layer to remove or fraction of inter-layer edges

Returns

Resilience ratio between 0 and 1

Examples

```
>>> r = resilience(network, 'layer_removal', perturbation_param='L1')
>>> r = resilience(network, 'coupling_removal', perturbation_param=0.5)
```

Reference:

Buldyrev et al. (2010), Nature 464, 1025-1028

```
py3plex.algorithms.statistics.multilayer_statistics.supra_laplacian_spectrum(network:
Any, k: int
= 10) →
ndarray
```

Calculate supra-Laplacian spectrum ().

Formula: = -

Eigenvalue spectrum of the supra-Laplacian matrix; captures diffusion properties. Uses sparse eigenvalue computation when beneficial.

Variables:

= supra-adjacency matrix ($NL \times NL$ block matrix containing all layers and inter-layer couplings) = supra-degree matrix (diagonal matrix with row sums of) = supra-Laplacian matrix = $\{0, 1, \dots, 1\}$ with $0 = 0 \ 1 \ \dots \ 1$

Parameters

- **network** – py3plex multi_layer_network object
- **k** – Number of smallest eigenvalues to compute

Returns

Array of k smallest eigenvalues

Examples

```
>>> spectrum = supra_laplacian_spectrum(network, k=10)
```

Reference:

De Domenico et al. (2013), Gomez et al. (2013)

Notes

- Uses sparse eigsh() for sparse matrices (more efficient)
- Falls back to dense computation for small matrices ($n < 100$) or when k is large relative to n
- Laplacian is always symmetric for undirected graphs (PSD with smallest eigenvalue = 0)

`py3plex.algorithms.statistics.multilayer_statistics.targeted_layer_removal` (*network: Any, layer: str, return_resilience: bool = False*)
 → Any | Tuple[Any, float]

Simulate targeted removal of an entire layer.

Removes all edges in a specified layer and returns the modified network or resilience score.

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier to remove
- **return_resilience** – If True, return resilience score instead of network

Returns

Modified network or resilience score

Examples

```
>>> resilience = targeted_layer_removal(network, 'social', return_
→resilience=True)
>>> print(f"Resilience after removing social layer: {resilience:.3f}")
```

Reference:

Buldyrev et al. (2010), “Catastrophic cascade of failures”

`py3plex.algorithms.statistics.multilayer_statistics.unique_redundant_edges` (*network: Any, layer_i: str, layer_j: str*)
 → Tuple[int, int]

Count unique and redundant edges between two layers.

Returns the number of edges unique to the first layer and the number of edges present in both layers (redundant).

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier

Returns

Tuple of (unique_edges, redundant_edges)

Examples

```
>>> unique, redundant = unique_redundant_edges(network, 'social', 'work')
>>> print(f"Unique: {unique}, Redundant: {redundant}")
```

`py3plex.algorithms.statistics.multilayer_statistics.versatility_centrality`(*network*: Any, *centrality_type*: str = 'degree', *alpha*: Dict[str, float] | None = None) → Dict[Any, float]

Calculate versatility centrality (V).

Formula: $V = \sum w_i C_i$

Weighted combination of node centrality values across layers; measures overall influence.

Variables:

w = weight for layer (typically $1/L$ for uniform weighting, $w = 1$) C = centrality of node i in layer (can be degree, betweenness, closeness, etc.)

Parameters

- **network** – py3plex multi_layer_network object
- **centrality_type** – Type of centrality ('degree', 'betweenness', 'closeness')
- **alpha** – Layer weights (default: uniform weights)

Returns

Dictionary mapping nodes to versatility centrality values

Examples

```
>>> versatility = versatility_centrality(network, centrality_type='degree')
```

Reference:

De Domenico et al. (2015), Nature Communications 6, 6868

`py3plex.algorithms.statistics.topology.basic_pl_stats`(*degree_sequence*: List[int]) → Tuple[float, float]

Calculate basic power law statistics for a degree sequence.

Parameters**degree_sequence** – Degree sequence of individual nodes**Returns**

Tuple of (alpha, sigma) values

`py3plex.algorithms.statistics.topology.plot_power_law`(*degree_sequence*: List[int], *title*: str, *xlabel*: str, *plabel*: str, *ylabel*: str = 'Number of nodes', *formula_x*: int = 70, *formula_y*: float = 0.05, *show*: bool = True, *use_normalization*: bool = False) → Any


```
py3plex.algorithms.statistics.correlation_networks.default_correlation_to_network(matrix:
                                                                              ndarray,
                                                                              in-
                                                                              put_type:
                                                                              str =
                                                                              'ma-
                                                                              trix',
                                                                              pre-
                                                                              pro-
                                                                              cess:
                                                                              str =
                                                                              'stan-
                                                                              dard')
                                                                              →
                                                                              ndarray
```

Convert correlation matrix to network using optimal thresholding.

Parameters

- **matrix** – Input data matrix
- **input_type** – Type of input (default: “matrix”)
- **preprocess** – Preprocessing method (default: “standard”)

Returns

Binary adjacency matrix

```
py3plex.algorithms.statistics.correlation_networks.pick_threshold(matrix: ndarray) → float
```

Pick optimal threshold for correlation network construction.

Parameters

matrix – Input data matrix

Returns

Optimal threshold value

```
py3plex.algorithms.statistics.basic_statistics.core_network_statistics(G: Graph, labels:
                                                                    Any | None = None,
                                                                    name: str =
                                                                    'example') →
                                                                    DataFrame
```

Compute core statistics for a network.

Parameters

- **G** – NetworkX graph to analyze
- **labels** – Optional label matrix with shape attribute
- **name** – Name identifier for the network (default: “example”)

Returns

DataFrame containing network statistics

```
py3plex.algorithms.statistics.basic_statistics.ensure(*args, **kwargs)
```

```
py3plex.algorithms.statistics.basic_statistics.identify_n_hubs(G: Graph, top_n: int = 100,
                                                             node_type: str | None =
                                                             None) → Dict[Any, int]
```

Identify the top N hub nodes in a network based on degree centrality.

Parameters

- **G** – NetworkX graph to analyze

- **top_n** – Number of top hubs to return (default: 100)
- **node_type** – Optional filter for specific node type

Returns

Dictionary mapping node identifiers to their degree values

Contracts:

- Precondition: top_n must be positive
- Postcondition: result has at most top_n entries
- Postcondition: all degree values are non-negative integers

```
py3plex.algorithms.statistics.basic_statistics.require(*args, **kwargs)
```

Multilayer Algorithms

Multilayer/Multiplex Network Centrality Measures

This module implements various centrality measures for multilayer and multiplex networks, following standard definitions from multilayer network analysis literature.

Weight Handling Notes:

For path-based centralities (betweenness, closeness): - Edge weights from the supra-adjacency matrix are converted to distances (inverse of weight) - NetworkX algorithms use these distances for shortest path computation - betweenness centrality: weight parameter specifies the edge attribute for path computation - closeness centrality: distance parameter specifies the edge attribute for path computation

For disconnected graphs: - closeness centrality uses wf_improved parameter (Wasserman-Faust scaling) by default - When wf_improved=True, scores are normalized by reachable nodes only - When wf_improved=False, unreachable nodes contribute infinite distance

Weight Constraints: - Weights should be positive (> 0) for shortest path algorithms - Zero or negative weights will cause undefined behavior - For unweighted analysis, use weighted=False parameters

Authors: py3plex contributors Date: 2025

```
class py3plex.algorithms.multilayer_algorithms.centralities.MultilayerCentrality(network: Any)
```

Bases: object

Class for computing centrality measures on multilayer networks.

This class provides implementations of various centrality measures specifically designed for multilayer/multiplex networks, including degree-based, eigenvector-based, and path-based measures.

accessibility centrality($h=2$)

Compute accessibility centrality (entropy-based reach within h steps).

Accessibility measures the diversity of nodes reachable within h steps using entropy of the probability distribution.

$\text{Access}_r = \exp(H_r)$ where H_r is the entropy of the h -step distribution

Parameters

h – Number of steps (default: 2)

Returns

{(node, layer): accessibility}

Return type

dict

Note

Accessibility is measured as the effective number of h-step destinations, using the entropy of the random walk distribution.

Dangling Node Handling: For nodes with no outgoing edges (dangling nodes), this implementation uses uniform teleportation to all nodes. This differs from the original Travencolo-Costa definition which does not include teleportation. The teleportation ensures a well-defined random walk distribution but may affect accessibility values for nodes near dangling nodes compared to implementations that handle dangling nodes differently.

References

- Travencolo, B. A. N., & Costa, L. D. F. (2008). Accessibility in complex networks. *Physics Letters A*, 373(1), 89-95.

aggregate_to_node_level(*node_layer_centralities*, *method*='sum', *weights*=None)

Aggregate node-layer centralities to node level.

Parameters

- **node_layer_centralities** – dict with {(node, layer): value} entries
- **method** – 'sum', 'mean', 'max', 'weighted_sum'
- **weights** – dict with {layer: weight} for weighted_sum method

Returns

{node: aggregated_value}

Return type

dict

bridging centrality()

Compute bridging centrality for nodes in the supra-graph.

Bridging centrality combines betweenness with the bridging coefficient, which measures how much a node connects sparse regions of the network.

$Bridging(u) = B(u) * BCoeff(u)$ where $BCoeff(u) = (1 / k_u) * \sum_{v \in N(u)} [1 / k_v]$

Returns

{(node, layer): bridging centrality}

Return type

dict

Note

Nodes with higher bridging centrality act as important bridges connecting different parts of the network.

collective_influence(*radius*=2)

Compute collective influence (CI_) for multiplex networks.

Collective influence identifies influential spreaders by considering not just immediate neighbors but also nodes at distance .

$CI_ (u) = (k_ u - 1) * \sum_{v \in Ball_ (u)} (k_ v - 1)$

Parameters

radius – Radius for the ball boundary (default: 2)

Returns

{(node, layer): collective_influence}

Return type

dict

Note

Uses overlapping degree across all layers for each physical node.

communicability_betweenness centrality(*normalized=True*)

Compute communicability betweenness centrality on the supra-graph.

This measure quantifies how much a node contributes to the communicability between other pairs of nodes. It uses the matrix exponential to account for all walks between nodes.

Parameters

normalized – If True (default), normalize scores by dividing by the maximum value (min-max scaling to [0, 1] range). Note that this is simple rescaling, not the theoretical Estrada-Hatano normalization from the original literature.

Returns

{(node, layer): communicability_betweenness}

Return type

dict

Note

This implementation uses NetworkX's `communicability_betweenness centrality`, which operates on unweighted graphs. Edge weights from the supra-adjacency matrix are used only to determine edge existence (`weight > 0`), not for weighted communicability computation. For truly weighted communicability analysis, a custom implementation using the weighted matrix exponential would be required.

This is computationally expensive as it requires computing the matrix exponential multiple times. For large networks, this may take significant time.

current_flow_betweenness centrality()

Compute current-flow betweenness centrality via supra Laplacian pseudoinverse.

This measure is based on the electrical current flow through each node.

Returns

{(node, layer): current_flow_betweenness}

Return type

dict

current_flow_closeness centrality()

Compute current-flow closeness centrality via supra Laplacian pseudoinverse.

This measure is based on the resistance distance in electrical networks.

Returns

{(node, layer): current_flow_closeness}

Return type

dict

edge_betweenness centrality(*normalized=True*)

Compute edge betweenness centrality on the supra-graph.

Edge betweenness measures the fraction of shortest paths that pass through each edge.

Returns

{(source, target): edge_betweenness}

Return type

dict

Note

This returns edge-level centrality, not node-level.

flow_betweenness centrality(*samples=100*)

Compute flow betweenness based on maximum flow sampling.

Flow betweenness measures how much flow passes through a node when maximizing flow between randomly sampled source-target pairs.

Parameters

samples – Number of source-target pairs to sample (default: 100)

Returns

{(node, layer): flow_betweenness}

Return type

dict

Note

This is a sampling-based approximation of flow betweenness. The implementation samples random pairs of supra-graph nodes and computes the maximum flow through each intermediate node.

Unlike classical Freeman flow betweenness which uses a normalization factor based on the number of nodes, this implementation returns the average flow per sampled pair without additional normalization. For multilayer networks, the number of supra-graph nodes ($N * L$ for N physical nodes and L layers) affects the raw values.

For large networks, this sampling approach is more computationally feasible than computing exact flow betweenness for all pairs.

harmonic_closeness centrality()

Compute harmonic closeness centrality on the supra-graph.

Harmonic closeness handles disconnected graphs better than standard closeness by summing the reciprocals of distances instead of taking reciprocal of sum.

$HC(u) = \sum_v \{ 1 / d(u,v) \}$ for finite distances

Returns

{(node, layer): harmonic_closeness}

Return type

dict

Note

This measure naturally handles disconnected components as unreachable nodes contribute 0 (instead of infinity) to the sum.

Weight Interpretation: Edge weights from the supra-adjacency matrix are interpreted as connection strengths (larger weight = stronger connection = shorter distance). They are converted to distances via $1/\text{weight}$ for shortest path computation. If your edge weights already represent distances, do not use this function directly—you would need to invert them first or use the distance values directly in a custom computation.

hits centrality(*max_iter=1000, tol=1e-06*)

Compute HITS (hubs and authorities) centrality on the supra-graph.

For undirected networks, this equals eigenvector centrality. For directed networks, computes separate hub and authority scores.

Parameters

- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns

If directed network: {'hubs': {(node, layer): score}, 'authorities': {(node, layer): score}}

If undirected network: {(node, layer): score} (equivalent to eigenvector centrality)

Return type

dict

information centrality()

Compute Information Centrality (Stephenson-Zelen style) on the supra-graph.

Information centrality measures the importance of a node based on the information flow through the network. It uses the inverse of a modified Laplacian matrix.

Returns

{(node, layer): information centrality}

Return type

dict

Note

This implementation returns values for each node-layer pair in the supra-graph, not aggregated physical node values. To obtain physical node-level information centrality, use the `aggregate_to_node_level()` method on the returned dictionary.

The implementation uses NetworkX's `information centrality` for computation. Falls back to harmonic closeness if information centrality computation fails.

Information centrality is defined only for undirected graphs. For directed networks, the graph is symmetrized with a warning.

References

- Stephenson, K., & Zelen, M. (1989). Rethinking centrality: Methods and examples. *Social Networks*, 11(1), 1-37.

katz bonacich centrality(*alpha=0.1, beta=None*)

Compute Katz-Bonacich centrality on the supra-graph.

$$z = \lim_{t \rightarrow \infty} M^t b = (I - M)^{-1} b$$

Parameters

- **alpha** – Attenuation parameter (should be $< 1/(M)$). If None, automatically computes a safe value as $0.85/(M)$ where (M) is the spectral radius.
- **beta** – Exogenous preference vector. If None, uses vector of ones.

Returns

{(node, layer): centrality_value}

Return type

dict

layer_degree_centrality(layer: str | None = None, weighted: bool = False, direction: str = 'out') → Dict[str | Tuple[str, str], float]

Compute layer-specific degree (or strength) centrality.

For undirected networks:

$$k^{\text{unweighted}}_i = \sum_j 1(A^{\text{unweighted}}_{ij} > 0) \quad s^{\text{weighted}}_i = \sum_j A^{\text{weighted}}_{ij}$$
For directed networks:

$$k^{\text{out}}_i = \sum_j 1(A^{\text{out}}_{ij} > 0) \quad k^{\text{in}}_i = \sum_j 1(A^{\text{in}}_{ji} > 0)$$
Parameters

- **layer** – Layer to compute centrality for. If None, compute for all layers.
- **weighted** – If True, compute strength instead of degree.
- **direction** – 'out', 'in', or 'both' for directed networks.

Returns

{(node, layer): centrality_value} if layer is None,
{node: centrality_value} if layer is specified.

Return type

dict

load_centrality()

Compute load centrality (shortest-path load).

Load centrality measures the fraction of shortest paths that pass through each node, counting all paths (not just unique pairs).

$\text{Load}[k] = \text{sum over all } (s,t) \text{ pairs of } [\text{number of shortest paths through } k / \text{total shortest paths}]$

Returns

{(node, layer): load_centrality}

Return type

dict

Note

This is similar to betweenness but counts all paths rather than normalizing by the number of node pairs.

local_efficiency_centrality()

Compute local efficiency centrality.

Local efficiency measures how efficiently information is exchanged among a node's neighbors when the node is removed. It quantifies the fault tolerance of the network.

$$\text{LE}(u) = (1 / ((N_u) * (N_u - 1))) * \sum_{ij \in N_u} [1 / d(i,j)]$$
Returns

{(node, layer): local_efficiency}

Return type
dict

Note

For nodes with less than 2 neighbors, local efficiency is 0.

lp_aggregated_centrality(*layer_centralities*, *p*=2, *weights*=None, *exclude_missing*=True)

Compute Lp-aggregated per-layer centrality.

Aggregates per-layer centrality values using Lp norm.

$C_i = (\sum_{l \in L_i} w_l * |c^l[i]|^p)^{1/p}$ for $p < \infty$ $C_i = \max_{l \in L_i} (w_l * |c^l[i]|)$ for $p = \infty$
where L_i is the set of layers where node i exists.

Parameters

- **layer_centralities** – dict of {layer: {node: centrality}} or {(node, layer): centrality}
- **p** – Lp norm parameter (default: 2). Use float('inf') for L-infinity norm.
- **weights** – dict of {layer: weight}. If None, uniform weights are used.
- **exclude_missing** – If True (default), only aggregate over layers where the node exists (has a centrality value). If False, treat nodes absent from a layer as having zero centrality, which may bias scores for nodes with sparse layer participation.

Returns

{node: aggregated_centrality}

Return type
dict

Note

This is a framework for aggregating any per-layer centrality measure. Input can be degree, PageRank, eigenvector, etc.

Sparse Layer Participation: When *exclude_missing*=True (default), nodes that only appear in a subset of layers are aggregated only over those layers where they exist. This prevents bias against nodes with sparse layer participation. When *exclude_missing*=False, missing layers contribute zero to the aggregation, which may penalize nodes that don't appear in all layers.

multilayer_betweenness_centrality(*normalized*=True, *endpoints*=False)

Compute betweenness centrality on the supra-graph.

For each node-layer pair (i,), computes the fraction of shortest paths between all pairs of nodes that pass through (i,).

Parameters

- **normalized** – Whether to normalize the betweenness values.
- **endpoints** – Whether to include endpoints in path counts.

Returns

{(node, layer): betweenness_centrality}

Return type
dict

Note

This is computationally expensive for large networks as it requires computing shortest paths between all pairs of nodes.

Weight handling: Edge weights from the supra-adjacency matrix are converted to distances ($1/\text{weight}$) for shortest path computation. Weights must be positive (> 0). Zero or negative weights will cause undefined behavior.

multilayer_closeness centrality(*normalized=True, wf_improved=True, variant='standard'*)

Compute closeness centrality on the supra-graph.

For each node-layer pair (*i*), computes:

Standard closeness:

$$C_c(i) = (n-1) / \sum_{j \in V} d((i), (j))$$

Harmonic closeness (recommended for disconnected networks):

$$HC(i) = \sum_{j \in V} 1/d((i), (j))$$

where $d((i), (j))$ is the shortest path distance in the supra-graph.

Parameters

- **normalized** – This parameter is kept for API compatibility but has no effect. Standard closeness is always normalized by $(n-1)$ per the NetworkX implementation. Harmonic closeness returns unnormalized sums of reciprocal distances by definition.
- **wf_improved** – If True, use Wasserman-Faust improved closeness scaling for disconnected graphs. Default is True. This affects the magnitude and ordering of scores in graphs with multiple components (e.g., low interlayer coupling). See NetworkX documentation for details. Only used when *variant*='standard'.
- **variant** – Closeness variant to use. Options: - 'standard': Classic closeness (reciprocal of sum of distances).

For disconnected graphs, uses Wasserman-Faust scaling if *wf_improved*=True. Can produce biased values for nodes in small or disconnected components.

- 'harmonic': Harmonic closeness (sum of reciprocal distances). Mathematically well-defined for disconnected networks: unreachable nodes contribute 0 instead of infinity. Recommended for disconnected multilayer networks.
- 'auto': Automatically selects 'harmonic' if the supra-graph has multiple connected components, otherwise uses 'standard'.

Default is 'standard' for backward compatibility.

Returns

{(node, layer): closeness centrality}

Return type

dict

Note

This implementation uses NetworkX's shortest path algorithms on the supra-graph representation. For large networks, this can be computationally expensive.

For disconnected multilayer graphs (e.g., layers with no inter-layer coupling, or networks with isolated components), use *variant*='harmonic' or *variant*='auto' to get mathematically consistent closeness values. The harmonic variant naturally handles unreachable nodes by summing $1/d$ for finite distances only (infinite distances contribute 0).

Weight Interpretation: Edge weights from the supra-adjacency matrix are interpreted as connection strengths (larger weight = stronger connection). They are converted to distances via $1/\text{weight}$ for shortest path computation. If your edge weights already represent distances, use them directly without this function's weight inversion.

Examples

```
>>> # For connected networks, standard closeness works well
>>> closeness = calc.multilayer_closeness_centrality(variant='standard')
```

```
>>> # For potentially disconnected networks, use harmonic
>>> closeness = calc.multilayer_closeness_centrality(variant='harmonic')
```

```
>>> # Let the algorithm decide based on connectivity
>>> closeness = calc.multilayer_closeness_centrality(variant='auto')
```

References

- Wasserman, S., & Faust, K. (1994). Social Network Analysis.
- Boldi, P., & Vigna, S. (2014). Axioms for Centrality. Internet Math.
- De Domenico, M., et al. (2015). Structural reducibility of multilayer networks.

multilayer_coreness()

Alias for `multilayer_k_core` for compatibility.

Returns

`{(node, layer): core_number}`

Return type

dict

multilayer_eigenvector_centrality(max_iter: int = 1000, tol: float = 1e-06) → Dict

Compute multilayer eigenvector centrality (node-layer level).

$x = (1/_\text{max}) * M * x$ where $x_{\{i\}}$ is the centrality of node i in layer , and $_\text{max}$ is the spectral radius of the supra-adjacency matrix M .

Parameters

- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns

`{(node, layer): centrality_value}`

Return type

dict

multilayer_eigenvector_versatility(max_iter: int = 1000, tol: float = 1e-06) → Dict

Compute node-level eigenvector versatility.

$x_i = _x_{\{i\}}$

Parameters

- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns

`{node: versatility_value}`

Return type

dict

multiplex_k_core()

Compute multiplex k-core decomposition.

A node belongs to the k-core if it has at least k neighbors in the multilayer network. This implementation computes the core number for each node-layer pair.

Returns

{(node, layer): core_number}

Return type

dict

overlapping_degree_centrality(weighted: bool = False) → Dict

Compute overlapping degree/strength centrality (node level).

$$k^{\text{over}}_i = \sum k_i [\text{unweighted}] \quad s^{\text{over}}_i = \sum s_i [\text{weighted}]$$
Parameters

weighted – If True, compute overlapping strength.

Returns

{node: centrality_value}

Return type

dict

pagerank_centrality(damping=0.85, max_iter=1000, tol=1e-06)

Compute PageRank centrality on the supra-graph.

Uses the standard PageRank algorithm on the supra-adjacency matrix representing the multilayer network. Properly handles dangling nodes (nodes with no outgoing edges) via teleportation.

This implementation preserves sparsity when possible for memory efficiency.

Parameters

- **damping** – Damping parameter (typically 0.85).
- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns

{(node, layer): centrality_value}

Return type

dict

Mathematical Invariants:

- PageRank values sum to 1.0 (within tol=1e-6)
- All values are non-negative
- Converges for strongly connected components or with teleportation

participation_coefficient(weighted: bool = False) → Dict

Compute participation coefficient across layers.

Measures how evenly a node's degree is distributed across layers: $P_i = 1 - \sum (k_i / k^{\text{over}}_i)^2$

Set $P_i = 0$ if $k^{\text{over}}_i = 0$.

Parameters

weighted – If True, use strength instead of degree.

Returns

{node: participation_coefficient}

Return type

dict

percolation centrality(*edge_activation_prob=0.5, trials=100*)

Compute percolation centrality using bond percolation Monte Carlo simulation.

This implementation measures the average relative component size that a node belongs to across multiple bond percolation realizations, providing an estimate of a node's importance for network connectivity under random edge failures.

Parameters

- **edge_activation_prob** – Probability that an edge is active (default: 0.5)
- **trials** – Number of Monte Carlo trials (default: 100)

Returns

{(node, layer): percolation centrality}

Return type

dict

Note

This is a component-size-based percolation measure, not the path-based percolation betweenness from the original Piraveenan et al. literature, which requires recomputing betweenness on each percolated realization. The original percolation centrality is computationally expensive ($O(n^3)$ per trial), so this implementation provides a more efficient alternative that captures related connectivity information.

Values are normalized to [0, 1] range where higher values indicate nodes that tend to belong to larger connected components across percolation realizations.

References

- Piraveenan, M., Prokopenko, M., & Hossain, L. (2013). Percolation centrality: Quantifying graph-theoretic impact of nodes during percolation in networks. PloS one, 8(1), e53095.

spreading centrality(*beta=0.2, mu=0.1, trials=50, steps=100*)

Compute spreading (epidemic) centrality using SIR model.

Spreading centrality measures how influential a node is in spreading information or disease through the network, based on Monte Carlo simulations of discrete-time SIR dynamics.

Parameters

- **beta** – Infection rate per edge per time step (default: 0.2)
- **mu** – Recovery rate per time step (default: 0.1)
- **trials** – Number of simulation trials per node (default: 50)
- **steps** – Maximum simulation steps (default: 100)

Returns

{(node, layer): spreading centrality}

Return type

dict

Note

This measures the average outbreak size (fraction of nodes ever infected) when seeding the epidemic from each node. Values are normalized by the total number of nodes, producing scores in the range $[1/n, 1]$ where n is the number of supra-graph nodes.

This is an empirical simulation-based measure, not normalized by theoretical epidemic threshold or branching factor as in some literature definitions. The normalization allows comparison of relative spreading power within the network but may not be directly comparable across networks of different sizes or structures.

References

- Kitsak, M., et al. (2010). Identification of influential spreaders in complex networks. *Nature physics*, 6(11), 888-893.

subgraph_centrality()

Compute subgraph centrality via matrix exponential of the supra-adjacency matrix.

Subgraph centrality counts closed walks of all lengths starting and ending at each node. $SC_i = (e^A)_{ii}$ where A is the adjacency matrix.

Returns

{(node, layer): subgraph_centrality}

Return type

dict

supra_degree_centrality(weighted: bool = False) → Dict

Compute supra degree/strength centrality (node-layer level).

$k_{-}\{i\} = \sum_j 1(M_{\{(i),(j)\}} > 0)$ [unweighted] $s_{-}\{i\} = \sum_j M_{\{(i),(j)\}}$ [weighted]

Parameters

weighted – If True, compute strength instead of degree.

Returns

{(node, layer): centrality_value}

Return type

dict

total_communicability()

Compute total communicability via matrix exponential.

Total communicability is the row sum of the matrix exponential: $TC_i = \sum_j (e^A)_{ij}$

Returns

{(node, layer): total_communicability}

Return type

dict

```
py3plex.algorithms.multilayer_algorithms.centralities.communicability_centrality(supra_matrix:  
                                     spmatrix,  
                                     normalize: bool  
                                     = True,  
                                     use_sparse:  
                                     bool =  
                                     True,  
                                     max_iter:  
                                     int =  
                                     100, tol:  
                                     float =  
                                     1e-06)  
                                     →  
                                     ndarray
```

Compute communicability centrality for each node-layer pair.

Communicability centrality measures the weighted sum of all walks between nodes, with exponentially decaying weights for longer walks. It is computed as the row sum of the matrix exponential:

$$c_i = \sum_j (\exp(A))_{ij}$$

This implementation uses `scipy.sparse.linalg.expm_multiply` for efficient sparse matrix exponential computation.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **normalize** – If True, normalize output to sum to 1.
- **use_sparse** – If True, use sparse matrix operations. Falls back to dense for matrices smaller than 10000 elements.
- **max_iter** – Maximum number of iterations for sparse approximation (currently unused).
- **tol** – Tolerance for convergence (currently unused).

Returns

Communicability centrality scores for each node-layer pair (n,).

Return type

np.ndarray

Raises

Py3plexMatrixError – If matrix is invalid (non-square, empty, etc.).

Example

```
>>> from py3plex.core import random_generators  
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)  
>>> A = net.get_supra_adjacency_matrix()  
>>> comm = communicability_centrality(A)  
>>> print(f"Communicability centrality computed for {len(comm)} node-layer pairs  
→")
```

```
py3plex.algorithms.multilayer_algorithms.centralities.compute_all_centralities(network, include_path_based=False, include_advanced=False, include_extended=False, preset=None, wf_improved=True, closeness_variant='standard')
```

Compute all available centrality measures for a multilayer network.

Parameters

- **network** – py3plex multi_layer_network object
- **include_path_based** – Whether to include computationally expensive path-based measures (betweenness, closeness). Default: False
- **include_advanced** – Whether to include advanced measures (HITS, current-flow, communicability, k-core). Default: False
- **include_extended** – Whether to include extended measures (information, accessibility, percolation, spreading, collective influence, load, flow betweenness, harmonic, bridging, local efficiency). Default: False
- **preset** – Convenience parameter to set all inclusion flags at once. Options: - 'basic': Only degree and eigenvector-based measures (default behavior) - 'standard': Includes path-based measures - 'advanced': Includes path-based and advanced measures - 'all': Includes all measures (path-based, advanced, and extended) - None: Use individual flags (default)
- **wf_improved** – If True, use Wasserman-Faust improved scaling for closeness centrality in disconnected graphs. Default: True. Only used when closeness_variant='standard'.
- **closeness_variant** – Variant of closeness centrality to use. Options: - 'standard': Classic closeness (reciprocal of sum of distances).
Uses Wasserman-Faust scaling if wf_improved=True.
- 'harmonic': Harmonic closeness (sum of reciprocal distances). Recommended for disconnected multilayer networks.
- 'auto': Automatically selects 'harmonic' if the network has disconnected components, otherwise uses 'standard'.

Default: 'standard' for backward compatibility.

Returns

Dictionary containing all computed centrality measures with keys:

- Degree-based: "layer_degree", "layer_strength", "supra_degree", "supra_strength", "overlapping_degree", "overlapping_strength", "participation_coefficient", "participation_coefficient_strength"
- Eigenvector-based: "multiplex_eigenvector", "eigenvector_versatility", "katz_bonacich", "pagerank"
- Path-based (if include_path_based=True): "closeness", "betweenness"
- Advanced (if include_advanced=True): "hits", "current_flow_closeness", "current_flow_betweenness", "subgraph_centrality", "total_communicability", "multiplex_k_core"

- Extended (if `include_extended=True`): “information”, “communicability_betweenness”, “accessibility”, “harmonic_closeness”, “local_efficiency”, “edge_betweenness”, “bridging”, “percolation”, “spreading”, “collective_influence”, “load”, “flow_betweenness”

Return type

dict

Note

Path-based, advanced, and extended measures are computationally expensive for large networks. Use flags or presets to control which measures are computed.

For disconnected multilayer graphs (e.g., networks without inter-layer coupling, or with isolated components), use `closeness_variant='harmonic'` or `'auto'` to get mathematically consistent closeness values.

Examples

```
>>> # Compute only basic measures (fast)
>>> results = compute_all_centralities(network)
```

```
>>> # Use preset for standard analysis
>>> results = compute_all_centralities(network, preset='standard')
```

```
>>> # Compute everything
>>> results = compute_all_centralities(network, preset='all')
```

```
>>> # Fine-grained control
>>> results = compute_all_centralities(
...     network,
...     include_path_based=True,
...     include_extended=True
... )
```

```
>>> # For disconnected multilayer networks, use harmonic closeness
>>> results = compute_all_centralities(
...     network,
...     include_path_based=True,
...     closeness_variant='harmonic'
... )
```

`py3plex.algorithms.multilayer_algorithms centrality.katz centrality`(*supra_matrix*:
spmatrix, *alpha*: float |
None = None, *beta*:
float = 1.0, *tol*: float =
1e-06) → ndarray

Compute Katz centrality for each node-layer pair.

Katz centrality measures node influence by accounting for all paths with exponentially decaying weights. It is computed as:

$$x = (I - \alpha * A)^{-1} * \beta * 1$$

where $\alpha < 1/\lambda_{\max}(A)$ to ensure convergence. If α is not provided, it defaults to $0.85 / \lambda_{\max}(A)$.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).

- **alpha** – Attenuation parameter. Must be less than $1/\text{spectral_radius}(A)$. If None, defaults to $0.85 / \text{lambda_max}(A)$.
- **beta** – Weight of exogenous influence (typically 1.0).
- **tol** – Tolerance for eigenvalue computation.

Returns

Katz centrality scores for each node-layer pair (n,).
Normalized to sum to 1.

Return type

np.ndarray

Raises

Py3plexMatrixError – If matrix is invalid or alpha is out of valid range.

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> katz = katz_centrality(A)
>>> print(f"Katz centrality computed for {len(katz)} node-layer pairs")
>>> # With custom alpha
>>> katz_custom = katz_centrality(A, alpha=0.05)
```

MultiXRank: Random Walk with Restart on Universal Multilayer Networks

This module implements the MultiXRank algorithm described in: Baptista et al. (2022), “Universal multilayer network exploration by random walk with restart”, Communications Physics, 5, 170.

MultiXRank performs random walk with restart (RWR) on a supra-heterogeneous adjacency matrix built from multiple multiplexes connected by bipartite blocks.

References

- Paper: <https://doi.org/10.1038/s42005-022-00937-9>
- arXiv: <https://arxiv.org/abs/2106.07869>
- Package: <https://github.com/anthbapt/multixrank>
- Docs: <https://multixrank-doc.readthedocs.io/>

```
class py3plex.algorithms.multilayer_algorithms.multixrank.MultiXRank(restart_prob: float =
                                                                    0.4, epsilon: float =
                                                                    1e-06, max_iter: int =
                                                                    100000, verbose: bool
                                                                    = True)
```

Bases: object

MultiXRank: Universal multilayer network exploration by random walk with restart.

This class implements the MultiXRank algorithm for node prioritization and ranking in universal multilayer networks. It builds a supra-heterogeneous adjacency matrix from multiple multiplexes and bipartite inter-multiplex connections, then performs random walk with restart.

multiplexes

Dictionary of multiplex supra-adjacency matrices

Type

Dict[str, sp.spmatrix]

bipartite_blocks

Inter-multiplex connection matrices

Type

Dict[Tuple[str, str], sp.spmatrix]

restart_prob

Restart probability (r) for RWR

Type

float

epsilon

Convergence threshold

Type

float

max_iter

Maximum number of iterations

Type

int

node_order

Node ordering for each multiplex

Type

Dict[str, List]

supra_matrix

Built supra-heterogeneous adjacency matrix

Type

sp.spmatrix

transition_matrix

Column-stochastic transition matrix

Type

sp.spmatrix

add_bipartite_block(*multiplex_from*: str, *multiplex_to*: str, *bipartite_matrix*: spmatrix | ndarray, *weight*: float = 1.0)

Add a bipartite block connecting two multiplexes.

Parameters

- **multiplex_from** – Name of source multiplex
- **multiplex_to** – Name of target multiplex
- **bipartite_matrix** – Matrix of connections (rows: from, cols: to)
- **weight** – Optional weight to scale this bipartite block

add_multiplex(*name*: str, *supra_adjacency*: spmatrix | ndarray, *node_order*: List | None = None)

Add a multiplex to the universal multilayer network.

Parameters

- **name** – Unique identifier for this multiplex
- **supra_adjacency** – Supra-adjacency matrix for the multiplex (can be from `multi_layer_network.get_supra_adjacency_matrix()`)
- **node_order** – Optional list of node IDs in the order they appear in the matrix. If None, uses integer indices.

aggregate_scores(*scores: ndarray, aggregation: str = 'sum'*) → Dict[str, Dict]

Aggregate scores per multiplex and optionally per physical node.

Parameters

- **scores** – Probability vector from RWR (length = total supra-matrix dimension)
- **aggregation** – How to aggregate scores ('sum', 'mean', 'max')

Returns

Dictionary mapping multiplex names to dictionaries of node scores

build_supra_heterogeneous_matrix(*block_weights: Dict[str | Tuple[str, str], float] | None = None*)

Build the supra-heterogeneous adjacency matrix S.

This constructs the universal multilayer network matrix by: 1. Placing each multiplex supra-adjacency on the block diagonal 2. Adding bipartite blocks for inter-multiplex connections

Parameters

block_weights – Optional dictionary to weight blocks. Keys can be: - Multiplex names (to weight within-multiplex edges) - Tuple (multiplex_from, multiplex_to) for bipartite blocks

Returns

The supra-heterogeneous adjacency matrix

column_normalize(*handle_dangling: str = 'uniform'*) → spmatrix

Column-normalize the supra-heterogeneous matrix to create a stochastic transition matrix.

This ensures each column sums to 1, making the matrix suitable for RWR.

Parameters

handle_dangling – How to handle dangling nodes (columns with zero sum): - 'uniform': Distribute mass uniformly across all nodes - 'self': Add self-loop (mass stays at the node) - 'ignore': Leave as zero (not recommended for RWR)

Returns

Column-stochastic transition matrix

get_top_ranked(*scores: ndarray, k: int = 10, multiplex: str | None = None, exclude_seeds: bool = True, seed_nodes: List[int] | Dict[str, List] | None = None*) → List[Tuple]

Get top-k ranked nodes from RWR scores.

Parameters

- **scores** – Probability vector from RWR
- **k** – Number of top nodes to return
- **multiplex** – If specified, only return nodes from this multiplex
- **exclude_seeds** – Whether to exclude seed nodes from results
- **seed_nodes** – Seed nodes to exclude (if exclude_seeds=True)

Returns

List of (global_index, score) tuples, sorted by score descending

random_walk_with_restart(*seed_nodes: List[int] | Dict[str, List] | ndarray, seed_weights: ndarray | None = None, multiplex_name: str | None = None*) → ndarray

Perform Random Walk with Restart (RWR) from seed nodes.

Parameters

- **seed_nodes** – Seed node specification. Can be: - List of global indices in the supra-matrix - Dict mapping multiplex names to lists of local node indices - NumPy array of global indices

- **seed_weights** – Optional weights for seed nodes (must match length of seed_nodes). If None, uniform weights are used.
- **multiplex_name** – If seed_nodes is a list/array of local indices, specify which multiplex they belong to

Returns

Steady-state probability vector (length = total nodes across all multiplexes)

```
py3plex.algorithms.multilayer_algorithms.multixrank.multixrank_from_py3plex_networks(networks:
    Dict[str,
    multi-
    net.multi_layer_net
    bi-
    par-
    tite_connections:
    Dict[Tuple[str,
    str],
    sp-
    ma-
    trix
    |
    ndar-
    ray]
    |
    None
    =
    None,
    seed_nodes:
    Dict[str,
    List]
    |
    None
    =
    None,
    restart_prob:
    float
    =
    0.4,
    ep-
    silon:
    float
    =
    1e-
    06,
    max_iter:
    int
    =
    100000,
    ver-
    bose:
    bool
    =
    True)
    →
    Tu-
    ple[MultiXRank,
    ndarray]
```

Convenience function to run MultiXRank on py3plex multi_layer_network objects.

Parameters

- **networks** – Dictionary mapping names to multi_layer_network objects
- **bipartite_connections** – Optional dict of inter-network connection matrices
- **seed_nodes** – Dict mapping network names to lists of seed node IDs
- **restart_prob** – Restart probability for RWR
- **epsilon** – Convergence threshold
- **max_iter** – Maximum iterations
- **verbose** – Whether to log progress

Returns

Tuple of (MultiXRank object, scores array)

Example

```
>>> from py3plex.core import multinet
>>> net1 = multinet.multi_layer_network()
>>> net1.load_network('network1.edgelist', ...)
>>> net2 = multinet.multi_layer_network()
>>> net2.load_network('network2.edgelist', ...)
>>>
>>> networks = {'net1': net1, 'net2': net2}
>>> seed_nodes = {'net1': ['node1', 'node2']}
>>>
>>> mxr, scores = multixrank_from_py3plex_networks(
...     networks, seed_nodes=seed_nodes
... )
>>>
>>> # Get aggregated scores per network
>>> aggregated = mxr.aggregate_scores(scores)
```

Authors: Benjamin Renoust (github.com/renoust) Date: 2018/02/13 Description: Loads a Detangler JSON format graph and compute unweighted entanglement analysis with Py3Plex

py3plex.algorithms.multilayer_algorithms.entanglement.**build_occurrence_matrix**(network: Any) → Tuple[ndarray, List[Any]]

Build occurrence matrix from multilayer network.

Parameters

network – Multilayer network object

Returns

Tuple of (c_matrix, layers) where c_matrix is the normalized occurrence matrix and layers is the list of layer names

py3plex.algorithms.multilayer_algorithms.entanglement.**compute_blocks**(c_matrix: ndarray) → Tuple[List[List[int]], List[ndarray]]

Compute block decomposition of occurrence matrix.

Parameters

c_matrix – Occurrence matrix

Returns

Tuple of (indices, blocks) where indices are the layer indices in each block and blocks are the submatrices for each block

`py3plex.algorithms.multilayer_algorithms.entanglement.compute_entanglement`(*block_matrix*:
ndarray) →
 Tuple[
 List[float],
 List[float]]

Compute entanglement metrics for a block.

Parameters

block_matrix – Block submatrix

Returns

Tuple of ([intensity, homogeneity, normalized_homogeneity], gamma_layers)

`py3plex.algorithms.multilayer_algorithms.entanglement.compute_entanglement_analysis`(*network*:
Any)
 →
 List[Dict[str,
 Any]]

Compute full entanglement analysis for a multilayer network.

Parameters

network – Multilayer network object

Returns

List of block analysis dictionaries with entanglement metrics

Supra Matrix Function Centralities for Multilayer Networks.

This module implements centrality measures based on matrix functions of the supra-adjacency matrix, including:
 - Communicability Centrality (Estrada & Hatano, 2008) - Katz Centrality (Katz, 1953)

These measures operate directly on the sparse supra-adjacency matrix obtained from `multi_layer_network.get_supra_adjacency_matrix()`.

References

- Estrada, E., & Hatano, N. (2008). Communicability in complex networks. *Physical Review E*, 77(3), 036111.
- Katz, L. (1953). A new status index derived from sociometric analysis. *Psychometrika*, 18(1), 39-43.

Authors: py3plex contributors Date: October 2025 (Phase II)

`py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality.communicability_centrality`

Compute communicability centrality for each node-layer pair.

Communicability centrality measures the weighted sum of all walks between nodes, with exponentially decaying weights for longer walks. It is computed as the row sum of the matrix exponential:

$$c_i = \sum_j (\exp(A))_{ij}$$

This implementation uses `scipy.sparse.linalg.expm_multiply` for efficient sparse matrix exponential computation.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **normalize** – If True, normalize output to sum to 1.
- **use_sparse** – If True, use sparse matrix operations. Falls back to dense for matrices smaller than 10000 elements.
- **max_iter** – Maximum number of iterations for sparse approximation (currently unused).
- **tol** – Tolerance for convergence (currently unused).

Returns

Communicability centrality scores for each node-layer pair (n,).

Return type

`np.ndarray`

Raises

Py3plexMatrixError – If matrix is invalid (non-square, empty, etc.).

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
```

(continues on next page)

(continued from previous page)

```
>>> comm = communicability centrality(A)
>>> print(f"Communicability centrality computed for {len(comm)} node-layer pairs
→")
```

`py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality.katz_centrality`(*supra_matrix*, *alpha*: *float* | *None* = *None*, *beta*: *float* = *1.0*, *tol*: *float* = *1e-06*) → *ndarray*

Compute Katz centrality for each node-layer pair.

Katz centrality measures node influence by accounting for all paths with exponentially decaying weights. It is computed as:

$$x = (I - \alpha * A)^{-1} * \beta * 1$$

where $\alpha < 1/\lambda_{\max}(A)$ to ensure convergence. If α is not provided, it defaults to $0.85 / \lambda_{\max}(A)$.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **alpha** – Attenuation parameter. Must be less than $1/\text{spectral_radius}(A)$. If *None*, defaults to $0.85 / \lambda_{\max}(A)$.
- **beta** – Weight of exogenous influence (typically 1.0).
- **tol** – Tolerance for eigenvalue computation.

Returns

Katz centrality scores for each node-layer pair (n,).

Normalized to sum to 1.

Return type

`np.ndarray`

Raises

Py3plexMatrixError – If matrix is invalid or α is out of valid range.

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> katz = katz_centrality(A)
>>> print(f"Katz centrality computed for {len(katz)} node-layer pairs")
>>> # With custom alpha
>>> katz_custom = katz_centrality(A, alpha=0.05)
```

Multiplex Participation Coefficient (MPC) for multiplex networks.

This module implements the Multiplex Participation Coefficient metric for multiplex networks (networks with identical node sets across all layers).

Authors: py3plex contributors Date: 2025

`py3plex.algorithms.multicentrality.ensure(*args, **kwargs)`

`py3plex.algorithms.multicentrality.multiplex_participation_coefficient`(*multinet*: Any, *normalized*: bool = True, *check_multiplex*: bool = True) → Dict[Any, float]

Compute the Multiplex Participation Coefficient (MPC) for multiplex networks. MPC measures how evenly a node participates across layers.

Parameters

- **multinet** (`py3plex.core.multinet.multi_layer_network`) – Multiplex network object (same node set across layers).
- **normalized** (bool, optional) – Whether to normalize MPC to [0,1]. Default: True.
- **check_multiplex** (bool, optional) – Validate that all layers share the same node set.

Returns

Node → MPC value mapping.

Return type

dict

Notes

The MPC is computed as:

$$\text{MPC}(i) = 1 - \sum (k_i / k_i^{\text{total}})^2$$

Where: k_i is the degree of node i in layer $-$ k_i^{total} is the total degree of node i across all layers

When `normalized=True`, the result is multiplied by $L/(L-1)$ to normalize to [0,1] range, where L is the number of layers.

References

- Battiston, F., et al. (2014). “Structural measures for multiplex networks.” *Physical Review E*, 89(3), 032804.
- De Domenico, M., et al. (2015). “Identifying modular flows on multilayer networks reveals highly overlapping organization in interconnected systems.” *Physical Review X*, 5(1), 011027.
- Harooni, M., et al. (2025). “Centrality in Multilayer Networks: Accurate Measurements with Multi-NetPy.” *The Journal of Supercomputing*, 81(1), 92. DOI: 10.1007/s11227-025-07197-8

Contracts:

- Precondition: multinet must not be None
- Precondition: normalized and check_multiplex must be booleans
- Postcondition: returns a dictionary with numeric values

```
py3plex.algorithms.multicentrality.require(*args, **kwargs)
```

General Algorithms

Random walk primitives for graph-based algorithms.

This module provides foundation for higher-level algorithms like Node2Vec, DeepWalk, and diffusion processes. Implements both basic and second-order (biased) random walks with proper edge weight handling and multilayer support.

Key Features:

- Basic random walks with weighted edge sampling
- Second-order (Node2Vec-style) biased random walks with p/q parameters
- Multiple simultaneous walks with deterministic reproducibility
- Support for directed, weighted, and multilayer networks
- Efficient sparse adjacency handling

References

- Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. KDD '16. <https://doi.org/10.1145/2939672.2939754>
- Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. KDD '14. <https://doi.org/10.1145/2623330.2623732>

```
py3plex.algorithms.general.walkers.basic_random_walk(G: Graph, start_node: int | str,  
                                                    walk_length: int, weighted: bool = True,  
                                                    seed: int | None = None) → List[int | str]
```

Perform a basic random walk on a graph with proper edge weight handling.

The next step is sampled proportionally to the normalized edge weights of the current node. For unweighted graphs, transitions are uniform.

Parameters

- **G** – NetworkX graph (directed or undirected, weighted or unweighted)
- **start_node** – Node to start the walk from
- **walk_length** – Number of steps in the walk
- **weighted** – Whether to use edge weights (default: True)
- **seed** – Random seed for reproducibility (default: None)

Returns

List of nodes representing the walk path (includes start_node)

Raises

ValueError – If start_node not in graph or walk_length < 1

Examples

```
>>> G = nx.Graph()
>>> G.add_weighted_edges_from([(0, 1, 1.0), (1, 2, 2.0), (1, 3, 1.0)])
>>> walk = basic_random_walk(G, 0, walk_length=3, seed=42)
>>> len(walk)
4
>>> walk[0]
0
```

Note

- Handles disconnected nodes by terminating walk early
- Edge weights must be positive for weighted walks
- Sum of transition probabilities from any node equals 1.0

`py3plex.algorithms.general.walkers.general_random_walk(G, start_node, iterations=1000, teleportation_prob=0)`

Legacy random walk with teleportation (for backward compatibility).

Deprecated since version 0.95a: Use `basic_random_walk()` or `node2vec_walk()` instead. This function will be removed in version 1.0.

Parameters

- **G** – NetworkX graph
- **start_node** – Starting node
- **iterations** – Number of steps
- **teleportation_prob** – Probability of teleporting to random visited node

Returns

List of visited nodes (excluding start_node)

`py3plex.algorithms.general.walkers.generate_walks(G: Graph, num_walks: int, walk_length: int, start_nodes: List[int | str] | None = None, p: float = 1.0, q: float = 1.0, weighted: bool = True, return_edges: bool = False, seed: int | None = None) → List[List[int | str]] | List[List[Tuple[int | str, int | str]]]`

Generate multiple random walks from specified or all nodes.

This interface supports multiple simultaneous walks with deterministic reproducibility under fixed RNG seed. Can return either node sequences or edge sequences.

Parameters

- **G** – NetworkX graph
- **num_walks** – Number of walks to generate per start node
- **walk_length** – Number of steps in each walk
- **start_nodes** – Nodes to start walks from (if None, uses all nodes)
- **p** – Return parameter for Node2Vec (1.0 = no bias)
- **q** – In-out parameter for Node2Vec (1.0 = no bias)
- **weighted** – Whether to use edge weights
- **return_edges** – Return edge sequences instead of node sequences

- **seed** – Random seed for reproducibility

Returns

- List of nodes (if return_edges=False)
- List of edges as tuples (if return_edges=True)

Return type

List of walks, where each walk is either

Examples

```
>>> G = nx.karate_club_graph()
>>> # Generate 10 walks from each node
>>> walks = generate_walks(G, num_walks=10, walk_length=5, seed=42)
>>> len(walks)
340
```

```
>>> # Generate walks with Node2Vec bias
>>> walks = generate_walks(G, num_walks=5, walk_length=10, p=0.5, q=2.0, seed=42)
```

```
>>> # Get edge sequences
>>> edge_walks = generate_walks(G, num_walks=3, walk_length=5, return_edges=True,
→ seed=42)
```

Note

- With same seed, generates identical walks across runs
- If $p == q == 1.0$, uses basic random walk (faster)
- Empty walks are included if node has no neighbors

```
py3plex.algorithms.general.walkers.layer_specific_random_walk(G: Graph, start_node: int | str,
                                                             walk_length: int, layer: str |
                                                             None = None,
                                                             cross_layer_prob: float = 0.0,
                                                             weighted: bool = True, seed:
                                                             int | None = None) → List[int |
                                                             str]
```

Perform random walk with layer constraints for multilayer networks.

In a multilayer network represented in py3plex format (where node names include layer information), this function can constrain walks to specific layers with occasional inter-layer transitions.

Parameters

- **G** – NetworkX graph (multilayer network in py3plex format)
- **start_node** – Node to start walk from (may include layer info)
- **walk_length** – Number of steps in the walk
- **layer** – Target layer to constrain walk to (None = no constraint)
- **cross_layer_prob** – Probability of crossing to different layer (0-1)
- **weighted** – Whether to use edge weights
- **seed** – Random seed for reproducibility

Returns

List of nodes representing the walk path

Examples

```
>>> # Multilayer network with layer-specific walks
>>> from py3plex.core import multinet
>>> network = multinet.multi_layer_network()
>>> network.add_layer("social")
>>> network.add_layer("biological")
>>> # ... add nodes and edges ...
>>> walk = layer_specific_random_walk(
...     network.core_network,
...     "nodeA---social",
...     walk_length=10,
...     layer="social",
...     cross_layer_prob=0.1
... )
```

Note

- If layer is None, behaves like basic_random_walk
- cross_layer_prob controls inter-layer transitions
- Node format should follow py3plex convention: “nodeID—layerID”

`py3plex.algorithms.general.walkers.node2vec_walk`(*G*: Graph, *start_node*: int | str, *walk_length*: int, *p*: float = 1.0, *q*: float = 1.0, *weighted*: bool = True, *seed*: int | None = None) → List[int | str]

Perform a second-order (biased) random walk following Node2Vec logic.

When transitioning from node $t \rightarrow v \rightarrow x$, the probability of choosing x is biased by parameters p (return) and q (in-out): - If $x == t$ (return to previous): weight / p - If x is neighbor of t (stay close): weight / $1 - p$ - If x is not neighbor of t (explore): weight / q

Parameters

- **G** – NetworkX graph (directed or undirected, weighted or unweighted)
- **start_node** – Node to start the walk from
- **walk_length** – Number of steps in the walk
- **p** – Return parameter (higher p = less likely to return to previous node)
- **q** – In-out parameter (higher q = less likely to explore further)
- **weighted** – Whether to use edge weights (default: True)
- **seed** – Random seed for reproducibility (default: None)

Returns

List of nodes representing the walk path (includes start_node)

Raises

ValueError – If $p \leq 0$ or $q \leq 0$ or start_node not in graph

Examples

```
>>> G = nx.Graph()
>>> G.add_edges_from([(0, 1), (1, 2), (1, 3), (0, 2)])
>>> # Low p, high q: tends to backtrack
>>> walk = node2vec_walk(G, 0, walk_length=5, p=0.1, q=10.0, seed=42)
>>> # High p, low q: tends to explore outward
>>> walk2 = node2vec_walk(G, 0, walk_length=5, p=10.0, q=0.1, seed=42)
```

Note

- First step is always a basic random walk (no previous node)
- Properly normalizes probabilities at each step
- Handles disconnected nodes by terminating early

References

Grover & Leskovec (2016), node2vec: Scalable feature learning for networks

Benchmark algorithms for node classification performance evaluation.

This module provides algorithms for benchmarking node classification performance, including oracle-based F1 score evaluation.

`py3plex.algorithms.general.benchmark_classification.evaluate_oracle_F1`(*probs*: ndarray,
Y_real: ndarray)
 → tuple[float, float]

Evaluate oracle F1 scores for multi-label classification.

This function computes micro and macro F1 scores by selecting the top-k predictions for each sample, where k is determined by the ground truth number of labels.

Parameters

- **probs** – Predicted probability matrix of shape (n_samples, n_labels).
- **Y_real** – Ground truth binary label matrix of shape (n_samples, n_labels).

Returns

- micro: Micro-averaged F1 score
- macro: Macro-averaged F1 score

Return type

A tuple containing

Example

```
>>> probs = np.array([[0.9, 0.1, 0.2], [0.1, 0.8, 0.7]])
>>> Y_real = np.array([[1, 0, 0], [0, 1, 1]])
>>> micro, macro = evaluate_oracle_F1(probs, Y_real)
```

Node Ranking

`py3plex.algorithms.node_ranking.node_ranking.authority_matrix`(*graph*: Graph) → spmatrix

Get the authority matrix of a graph.

Parameters

graph – NetworkX graph

Returns

Authority matrix

`py3plex.algorithms.node_ranking.node_ranking.hub_matrix(graph: Graph) → spmatrix`

Get the hub matrix of a graph.

Parameters

graph – NetworkX graph

Returns

Hub matrix

`py3plex.algorithms.node_ranking.node_ranking.hubs_and_authorities(graph: Graph) → Tuple[dict, dict]`

Compute hubs and authorities scores using HITS algorithm.

Parameters

graph – NetworkX graph

Returns

Tuple of (hubs dictionary, authorities dictionary)

`py3plex.algorithms.node_ranking.node_ranking.modularity(G: Graph, communities: List[List[Any]], weight: str = 'weight') → float`

Calculate modularity of a graph partition.

Parameters

- **G** – NetworkX graph
- **communities** – List of communities (each community is a list of nodes)
- **weight** – Edge weight attribute name

Returns

Modularity value

`py3plex.algorithms.node_ranking.node_ranking.page_rank_kernel(index_row: int) → Tuple[int, ndarray]`

PageRank kernel for parallel computation.

Note: This function expects global variables `G`, `damping_hyper`, `spread_step_hyper`, `spread_percent_hyper`, and `graph` to be defined. It's designed for use with `multiprocessing.Pool.map()`.

Parameters

index_row – Row index to compute PageRank for

Returns

Tuple of (index, PageRank vector)

`py3plex.algorithms.node_ranking.node_ranking.sparse_page_rank(matrix: spmatrix, start_nodes: List[int] | range | None, epsilon: float = 1e-06, max_steps: int = 100000, damping: float = 0.5, spread_step: int = 10, spread_percent: float = 0.3, try_shrink: bool = False) → ndarray`

Compute sparse PageRank with personalization.

Parameters

- **matrix** – Sparse adjacency matrix (column-stochastic)
- **start_nodes** – List of starting node indices for personalization (can be range or None)

- **epsilon** – Convergence threshold
- **max_steps** – Maximum number of iterations
- **damping** – Damping factor (teleportation probability)
- **spread_step** – Maximum steps for spread calculation
- **spread_percent** – Percentage threshold for spread
- **try_shrink** – Whether to try matrix shrinking optimization

Returns

PageRank vector

`py3plex.algorithms.node_ranking.node_ranking.stochastic_normalization(matrix: spmatrix)`
→ `spmatrix`

Normalize a sparse matrix stochastically.

Parameters

matrix – Sparse matrix to normalize

Returns

Stochastically normalized sparse matrix

`py3plex.algorithms.node_ranking.node_ranking.stochastic_normalization_hin(matrix: spmatrix)` → `spmatrix`

Normalize a heterogeneous information network matrix stochastically.

Parameters

matrix – Sparse matrix to normalize

Returns

Stochastically normalized sparse matrix

Network Classification

`py3plex.algorithms.network_classification.label_propagation.label_propagation(graph_matrix: spmatrix, class_matrix: ndarray, alpha: float = 0.001, epsilon: float = 1e-12, max_steps: int = 100000, normalization: str | List[str] = 'freq')` → `ndarray`

Propagate labels through a graph.

Parameters

- **graph_matrix** – Sparse graph adjacency matrix
- **class_matrix** – Initial class label matrix

- **alpha** – Propagation weight parameter
- **epsilon** – Convergence threshold
- **max_steps** – Maximum number of iterations
- **normalization** – Normalization scheme(s) to apply

Returns

Propagated label matrix

```
py3plex.algorithms.network_classification.label_propagation.label_propagation_normalization(matrix:
                                                                                               sp-
                                                                                               ma-
                                                                                               trix)
                                                                                               →
                                                                                               spmatrix
```

Normalize a matrix for label propagation.

Parameters

matrix – Sparse matrix to normalize

Returns

Normalized sparse matrix

```
py3plex.algorithms.network_classification.label_propagation.label_propagation_tf() →
                                                                                               None
```

TensorFlow-based label propagation (TODO: implement).

Placeholder for future TensorFlow implementation.

```
py3plex.algorithms.network_classification.label_propagation.normalize_amplify_freq(mat:
                                                                                       ndar-
                                                                                       ray)
                                                                                       →
                                                                                       ndarray
```

Normalize and amplify matrix by frequency.

Parameters

mat – Matrix to normalize

Returns

Normalized and amplified matrix

```
py3plex.algorithms.network_classification.label_propagation.normalize_exp(mat: ndarray)
                                                                                               → ndarray
```

Apply exponential normalization.

Parameters

mat – Matrix to normalize

Returns

Exponentially normalized matrix

```
py3plex.algorithms.network_classification.label_propagation.normalize_initial_matrix_freq(mat:
                                                                                               ndar-
                                                                                               ray)
                                                                                               →
                                                                                               ndarray
```

Normalize matrix by frequency.

Parameters

mat – Matrix to normalize

Returns

Normalized matrix

```
py3plex.algorithms.network_classification.label_propagation.normalize_none(mat: ndarray)  
→ ndarray
```

No normalization (identity function).

Parameters

mat – Matrix to return unchanged

Returns

Original matrix

```
py3plex.algorithms.network_classification.label_propagation.validate_label_propagation(core_network:  
sp-  
ma-  
trix,  
la-  
bels:  
ndar-  
ray  
|  
sp-  
ma-  
trix,  
dataset_name:  
str  
=  
'test',  
rep-  
e-  
ti-  
tions:  
int  
=  
5,  
nor-  
mal-  
iza-  
tion_scheme:  
str  
|  
List[str]  
=  
'ba-  
sic',  
al-  
pha_value:  
float  
=  
0.001,  
ran-  
dom_seed:  
int  
=  
123,  
ver-  
bose:  
bool  
=  
False)  
→  
DataFrame
```

Validate label propagation with cross-validation.

Parameters

- **core_network** – Sparse network adjacency matrix
- **labels** – Label matrix
- **dataset_name** – Name of the dataset
- **repetitions** – Number of repetitions
- **normalization_scheme** – Normalization scheme to use
- **alpha_value** – Alpha parameter for propagation
- **random_seed** – Random seed for reproducibility
- **verbose** – Whether to print progress

Returns

DataFrame with validation results

Visualization

`py3plex.visualization.multilayer.draw_multiedges`(*network_list: List[Graph] | Dict[Any, Graph], multi_edge_tuple: List[Any], input_type: str = 'nodes', linepoints: str = '-.', alphachannel: float = 0.3, linecolor: str = 'black', curve_height: float = 1, style: str = 'curve2_bezier', linewidth: float = 1, invert: bool = False, linmod: str = 'both', resolution: float = 0.001*) → None

Draw edges connecting multiple layers.

Parameters

- **network_list** – List of NetworkX graphs (layers) or dict of layer_name -> graph
- **multi_edge_tuple** – Tuple specifying edges to draw
- **input_type** – Type of input (“nodes” or other)
- **linepoints** – Line style
- **alphachannel** – Transparency level
- **linecolor** – Color of the lines
- **curve_height** – Height of curved edges
- **style** – Style of edges (“curve2_bezier”, “line”, etc.)
- **linewidth** – Width of lines
- **invert** – Whether to invert drawing direction
- **linmod** – Line modification mode
- **resolution** – Resolution for curve drawing

```
py3plex.visualization.multilayer.draw_multilayer_default(network_list: List[Graph] | Dict[Any, Graph], display: bool = True, node_size: int = 10, alphalevel: float = 0.13, rectanglex: float = 1, rectangley: float = 1, background_shape: str = 'circle', background_color: str = 'rainbow', networks_color: str = 'rainbow', labels: bool = False, arrowsize: float = 0.5, label_position: int = 1, verbose: bool = False, remove_isolated_nodes: bool = False, axis: Any | None = None, edge_size: float = 1, node_labels: bool = False, node_font_size: int = 5, scale_by_size: bool = False) → None
```

Core multilayer drawing method.

Parameters

- **network_list** – List of NetworkX graphs to visualize (or dict of layer_name -> graph)
- **display** – Whether to display the plot directly
- **node_size** – Base size of nodes
- **alphalevel** – Transparency level for background shapes
- **rectanglex** – Width of rectangular backgrounds
- **rectangley** – Height of rectangular backgrounds
- **background_shape** – Background shape type (“circle” or “rectangle”)
- **background_color** – Background color scheme (“default”, “rainbow”, or None)
- **networks_color** – Network color scheme (“rainbow” or “black”)
- **labels** – Layer labels to display
- **arrowsize** – Size of edge arrows
- **label_position** – Position offset for layer labels
- **verbose** – Whether to log network information
- **remove_isolated_nodes** – Whether to remove isolated nodes
- **axis** – Matplotlib axis to draw on (None for current axis)
- **edge_size** – Width of edges
- **node_labels** – Whether to display node labels
- **node_font_size** – Font size for node labels
- **scale_by_size** – Whether to scale node size by degree

Returns

None

```
py3plex.visualization.multilayer.draw_multilayer_flow(graphs: List[Graph], multilinks: Dict[str, List[Tuple]], labels: List[str] | None = None, node_activity: Dict[Any, float] | None = None, ax: Any | None = None, display: bool = True, layer_gap: float = 3.0, node_size: float = 30, node_cmap: str = 'viridis', flow_alpha: float = 0.3, flow_min_width: float = 0.2, flow_max_width: float = 4.0, aggregate_by: Tuple[str, ...] = ('u', 'v', 'layer_u', 'layer_v'), **kwargs) → Any
```

Draw multilayer network as layered flow visualization (alluvial-style).

Shows each layer as a horizontal band with nodes positioned along the x-axis. Intra-layer activity is encoded as node color/size, and inter-layer edges are shown as thick flow ribbons (Bezier curves) where width encodes edge weight.

Parameters

- **graphs** – List of NetworkX graphs, one per layer (from `multi_layer_network.get_layers()`)
- **multilinks** – Dictionary mapping `edge_type` -> list of multi-layer edges
- **labels** – Optional list of layer labels. If None, uses layer indices
- **node_activity** – Optional dict mapping `node_id` -> activity value. If None, computes intra-layer degree
- **ax** – Matplotlib axes to draw on. If None, creates new figure
- **display** – If True, calls `plt.show()` at the end
- **layer_gap** – Vertical distance between layer bands
- **node_size** – Base marker size for nodes
- **node_cmap** – Matplotlib colormap name for node activity coloring
- **flow_alpha** – Base transparency for flow ribbons
- **flow_min_width** – Minimum line width for flows
- **flow_max_width** – Maximum line width for flows
- **aggregate_by** – Tuple of keys for aggregating flows (currently not used, for future extension)
- ****kwargs** – Reserved for future extensions

Returns

Matplotlib axes object

Examples

```
>>> network = multi_layer_network()
>>> network.load_network("data.txt", input_type="multiedgelist")
>>> labels, graphs, multilinks = network.get_layers()
>>> draw_multilayer_flow(graphs, multilinks, labels=labels)
```

```
py3plex.visualization.multilayer.generate_random_multiedges(network_list: List[Graph], random_edges: int, style: str = 'line', linepoints: str = '-.', upper_first: int = 2, lower_first: int = 0, lower_second: int = 2, inverse_tag: bool = False, pheight: float = 1) → None
```

Generate and draw random multi-layer edges.

Parameters

- **network_list** – List of NetworkX graphs (layers)
- **random_edges** – Number of random edges to generate
- **style** – Style of edges to draw
- **linepoints** – Line style
- **upper_first** – Upper bound for first layer
- **lower_first** – Lower bound for first layer
- **lower_second** – Lower bound for second layer
- **inverse_tag** – Whether to invert drawing
- **pheight** – Height parameter for curves

```
py3plex.visualization.multilayer.generate_random_networks(number_of_networks: int) →  
List[Graph]
```

Generate random networks for testing.

Parameters

- number_of_networks** – Number of random networks to generate

Returns

List of NetworkX graphs with random layouts

```
py3plex.visualization.multilayer.hairball_plot(g: Graph | Any, color_list: List[str] | List[int] |  
None = None, display: bool = False, node_size:  
float = 1, text_color: str = 'black', node_sizes:  
List[float] | None = None, layout_parameters: dict  
| None = None, legend: Any | None = None,  
scale_by_size: bool = True, layout_algorithm: str  
= 'force', edge_width: float = 0.01, alpha_channel:  
float = 0.5, labels: List[str] | None = None, draw:  
bool = True, label_font_size: int = 2) → Any |  
None
```

A method for drawing force-directed plots Args: network (networkx): A network to be visualized color_list (list): A list of colors for nodes node_size (float): Size of nodes layout_parameters (dict): A dictionary of label parameters legend (bool): Display legend? scale_by_size (bool): Rescale nodes? layout_algorithm (string): What type of layout algorithm is to be used? edge_width (float): Width of edges alpha_channel (float): Transparency level. labels (bool): Display labels? label_font_size (int): Sizes of labels :returns: None

```
py3plex.visualization.multilayer.interactive_diagonal_plot(network_list: List[Graph] |  
Dict[Any, Graph], layer_labels:  
List[str] | None = None,  
layout_algorithm: str = 'force',  
layer_gap: float = 4.0,  
node_size_base: int = 8,  
layer_colors: List[str] | None =  
None, show_interlayer_edges: bool  
= True, interlayer_edges:  
List[Tuple[Any, Any]] | None =  
None) → bool | Any
```

Create an interactive 2.5D diagonal multilayer plot using Plotly.

This function creates an interactive version of the diagonal multilayer visualization, mimicking the traditional 2D diagonal layout but in an interactive 3D environment. Each layer is positioned diagonally with clear visual separation, similar to the static diagonal visualization.

Parameters

- **network_list** – List of NetworkX graphs (layers) or dict of layer_name -> graph
- **layer_labels** – Optional labels for each layer
- **layout_algorithm** – Layout algorithm for nodes (“force”, “circular”, “random”)
- **layer_gap** – Distance between layers in diagonal direction (default: 2.5)
- **node_size_base** – Base size for nodes (default: 8)
- **layer_colors** – Optional list of colors for each layer (HTML color names or hex)
- **show_interlayer_edges** – Whether to show inter-layer edges
- **interlayer_edges** – List of tuples (node1, node2) for inter-layer connections

Returns

False if plotly not available, otherwise plotly figure object

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.load_network("network.txt", input_type="multiedgelist")
>>> labels, graphs, multilinks = net.get_layers("diagonal")
>>> fig = interactive_diagonal_plot(graphs, layer_labels=labels)
```

```
py3plex.visualization.multilayer.interactive_hairball_plot(G: Graph, nsizes: List[float],
                                                         final_color_mapping: dict, pos:
                                                         dict, colorscale: str = 'Rainbow')
                                                         → bool | Any
```

Create an interactive 3D hairball plot using Plotly.

Parameters

- **G** – NetworkX graph to visualize
- **nsizes** – Node sizes
- **final_color_mapping** – Mapping of nodes to colors
- **pos** – Node positions
- **colorscale** – Color scale to use

Returns

False if plotly not available, otherwise plotly figure object

```
py3plex.visualization.multilayer.onclick(event: Any) → None
```

Handle mouse click events on plots.

Parameters

event – Matplotlib event object

```
py3plex.visualization.multilayer.plot_edge_colored_projection(multilayer_network, layout: str  
                                                            = 'spring', node_size: int = 50,  
                                                            layer_colors: Dict[Any, str] |  
                                                            None = None,  
                                                            aggregate_multilayer_edges:  
                                                            bool = True, figsize:  
                                                            Tuple[float, float] = (12, 9),  
                                                            edge_alpha: float = 0.7,  
                                                            **kwargs)
```

Create an aggregated projection where edge colors indicate layer membership.

This visualization projects all layers onto a single 2D graph, using edge colors to distinguish which layer each edge belongs to. Useful for seeing the overall structure while maintaining layer information.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **layout** – Layout algorithm to use (“spring”, “circular”, “random”, “kamada_kawai”)
- **node_size** – Size of nodes
- **layer_colors** – Optional dict mapping layer names to colors; if None, auto-generated
- **aggregate_multilayer_edges** – If True, show edges from all layers with distinct colors
- **figsize** – Figure size as (width, height) tuple
- **edge_alpha** – Transparency level for edges (0-1)
- ****kwargs** – Additional arguments for customization

Returns

The created figure

Return type

matplotlib.figure.Figure

```
py3plex.visualization.multilayer.plot_ego_multilayer(multilayer_network, ego, layers: List[Any]  
                                                    | None = None, max_depth: int = 1, layout:  
                                                    str = 'spring', figsize: Tuple[float, float] |  
                                                    None = None, max_cols: int = 3,  
                                                    node_size: int = 500, ego_node_size: int =  
                                                    1200, **kwargs)
```

Create an ego-centric multilayer visualization.

This visualization focuses on a single node (ego) and shows its neighborhood across different layers, highlighting the ego node’s position in each layer.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **ego** – The ego node to focus on
- **layers** – Optional list of specific layers to visualize; if None, uses all layers
- **max_depth** – Maximum depth of neighborhood to include (number of hops)
- **layout** – Layout algorithm for each ego graph
- **figsize** – Optional figure size; if None, auto-calculated
- **max_cols** – Maximum columns in subplot grid
- **node_size** – Size of regular nodes
- **ego_node_size** – Size of the ego node (highlighted)

- ****kwargs** – Additional drawing parameters

Returns

The created figure

Return type

matplotlib.figure.Figure

```
py3plex.visualization.multilayer.plot_radial_layers(multilayer_network, base_radius: float =
                                                    1.0, radius_step: float = 1.0, node_size: int
                                                    = 500, draw_inter_layer_edges: bool =
                                                    True, figsize: Tuple[float, float] = (12, 12),
                                                    edge_alpha: float = 0.5, draw_layer_bands:
                                                    bool = True, band_alpha: float = 0.25,
                                                    **kwargs)
```

Create a radial/concentric visualization with layers as rings.

This visualization arranges layers as concentric circles, with nodes positioned on rings based on their layer. Inter-layer edges appear as radial connections.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **base_radius** – Radius of the innermost layer
- **radius_step** – Distance between consecutive layer rings
- **node_size** – Size of nodes (default: 500 for better visibility)
- **draw_inter_layer_edges** – If True, draw edges between layers
- **figsize** – Figure size as (width, height) tuple
- **edge_alpha** – Transparency for edges
- **draw_layer_bands** – If True, draw semi-transparent circular bands around layers
- **band_alpha** – Transparency for layer bands (default: 0.25)
- ****kwargs** – Additional drawing parameters

Returns

The created figure

Return type

matplotlib.figure.Figure

```
py3plex.visualization.multilayer.plot_small_multiples(multilayer_network, layout: str = 'spring',
                                                       max_cols: int = 3, node_size: int = 50,
                                                       shared_layout: bool = True,
                                                       show_layer_titles: bool = True, figsize:
                                                       Tuple[float, float] | None = None,
                                                       **kwargs)
```

Create a small multiples visualization with one subplot per layer.

This visualization shows each layer as a separate subplot in a grid layout, making it easy to compare the structure of different layers side-by-side.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **layout** – Layout algorithm to use (“spring”, “circular”, “random”, “kamada_kawai”)
- **max_cols** – Maximum number of columns in the subplot grid
- **node_size** – Size of nodes in each subplot

- **shared_layout** – If True, compute one layout and reuse for all layers; if False, compute independent layouts per layer
- **show_layer_titles** – If True, show layer names as subplot titles
- **figsize** – Optional figure size as (width, height) tuple
- ****kwargs** – Additional arguments passed to `nx.draw_networkx`

Returns

The created figure

Return type

`matplotlib.figure.Figure`

```
py3plex.visualization.multilayer.plot_supra_adjacency_heatmap(multilayer_network,
                                                             include_inter_layer: bool =
                                                             False, inter_layer_weight: float
                                                             = 1.0, node_order: List[Any] |
                                                             None = None, cmap: str =
                                                             'viridis', figsize: Tuple[float,
                                                             float] = (10, 10), **kwargs)
```

Create a supra-adjacency matrix heatmap visualization.

This visualization shows the multilayer network as a block matrix where each block represents the adjacency matrix of one layer. Optionally includes inter-layer connections.

Parameters

- **multilayer_network** – A `MultiLayerNetwork` instance
- **include_inter_layer** – If True, include inter-layer edges/couplings
- **inter_layer_weight** – Default weight for inter-layer connections
- **node_order** – Optional list specifying node ordering; if None, uses sorted order
- **cmap** – Colormap name for the heatmap
- **figsize** – Figure size as (width, height) tuple
- ****kwargs** – Additional arguments for `imshow`

Returns

The created figure

Return type

`matplotlib.figure.Figure`

```
py3plex.visualization.multilayer.supra_adjacency_matrix_plot(matrix: ndarray, display: bool =
                                                             False) → None
```

Plot a supra-adjacency matrix.

Parameters

- **matrix** – Supra-adjacency matrix to plot
- **display** – Whether to display the plot immediately

```
py3plex.visualization.multilayer.visualize_multilayer_network(multilayer_network,
                                                             visualization_type: str =
                                                             'diagonal', **kwargs)
```

High-level function to visualize multilayer networks with multiple visualization modes.

This function provides a unified interface for various multilayer network visualization techniques, making it easy to switch between different visual representations.

Parameters

- **multilayer_network** – A `MultiLayerNetwork` instance from `py3plex.core.multinet`

- **visualization_type** – Type of visualization to use. Options: - “diagonal”: Default layer-centric diagonal layout (existing behavior) - “small_multiples”: One subplot per layer with shared or independent layouts - “edge_colored_projection”: Aggregate projection with edge colors by layer - “supra_adjacency_heatmap”: Matrix representation of multilayer structure - “radial_layers”: Concentric circles for layers with radial inter-layer edges - “ego_multilayer”: Ego-centric view focused on a specific node
- ****kwargs** – Additional keyword arguments specific to each visualization type. See individual plot functions for details.

Returns

The created figure object

Return type

matplotlib.figure.Figure

Raises

ValueError – If visualization_type is not recognized

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.load_network("network.txt", input_type="multiedgelist")
>>>
>>> # Use default diagonal visualization
>>> fig = visualize_multilayer_network(net)
>>>
>>> # Use small multiples view
>>> fig = visualize_multilayer_network(net, visualization_type="small_multiples")
>>>
>>> # Use edge-colored projection
>>> fig = visualize_multilayer_network(net, visualization_type="edge_colored_
↳ projection")
```

py3plex.visualization.drawing_machinery.**draw**(G, pos=None, ax=None, **kws)

Draw the graph G with Matplotlib.

Draw the graph as a simple representation with no node labels or edge labels and using the full Matplotlib figure area and no axis labels by default. See draw_networkx() for more full-featured drawing that allows title, axis labels etc.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary, optional*) – A dictionary with nodes as keys and positions as values. If not specified a spring layout positioning will be computed. See networkx.drawing.layout for functions that compute node positions.
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in specified Matplotlib axes.
- **kws** (*optional keywords*) – See networkx.draw_networkx() for a description of optional keywords.

Examples

```
>>> G = nx.dodecahedral_graph()
>>> nx.draw(G)
>>> nx.draw(G, pos=nx.spring_layout(G)) # use spring layout
```

See also

`draw_networkx`, `draw_networkx_nodes`, `draw_networkx_edges`, `draw_networkx_labels`,
`draw_networkx_edge_labels`

Notes

This function has the same name as `pylab.draw` and `pyplot.draw` so beware when using

```
>>> from networkx import *
```

since you might overwrite the `pylab.draw` function.

With `pyplot` use

```
>>> import matplotlib.pyplot as plt
>>> import networkx as nx
>>> G = nx.dodecahedral_graph()
>>> nx.draw(G) # networkx draw()
>>> plt.draw() # pyplot draw()
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

`py3plex.visualization.drawing_machinery.draw_circular(G, **kwargs)`

Draw the graph `G` with a circular layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_kamada_kawai(G, **kwargs)`

Draw the graph `G` with a Kamada-Kawai force-directed layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_networkx(G, pos=None, arrows=True, with_labels=True, **kws)`

Draw the graph `G` using Matplotlib.

Draw the graph with Matplotlib with options for node positions, labeling, titles, and many other drawing features. See `draw()` for simple drawing without labels or axes.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary, optional*) – A dictionary with nodes as keys and positions as values. If not specified a spring layout positioning will be computed. See `networkx.drawing.layout` for functions that compute node positions.
- **arrows** (*bool, optional (default=True)*) – For directed graphs, if `True` draw arrowheads. Note: Arrows will be the same color as edges.

- **arrowstyle** (str, optional (default='->')) – For directed graphs, choose the style of the arrowheads. See :py:class: *matplotlib.patches.ArrowStyle* for more options.
- **arrowsize** (int, optional (default=10)) – For directed graphs, choose the size of the arrow head's length and width. See :py:class: *matplotlib.patches.FancyArrowPatch* for attribute *mutation_scale* for more info.
- **with_labels** (bool, optional (default=True)) – Set to True to draw labels on the nodes.
- **ax** (*Matplotlib Axes object*, optional) – Draw the graph in the specified Matplotlib axes.
- **odelist** (list, optional (default *G.nodes()*)) – Draw only specified nodes
- **edgelist** (list, optional (default=*G.edges()*)) – Draw only specified edges
- **node_size** (scalar or array, optional (default=300)) – Size of nodes. If an array is specified it must be the same length as *odelist*.
- **node_color** (color string, or array of floats, (default='r')) – Node color. Can be a single color format string, or a sequence of colors with the same length as *odelist*. If numeric values are specified they will be mapped to colors using the *cmap* and *vmin*, *vmax* parameters. See *matplotlib.scatter* for more details.
- **node_shape** (string, optional (default='o')) – The shape of the node. Specification is as *matplotlib.scatter* marker, one of 'so^>v<dph8'.
- **alpha** (float, optional (default=1.0)) – The node and edge transparency
- **cmap** (*Matplotlib colormap*, optional (default=None)) – Colormap for mapping intensities of nodes
- **vmin** (float, optional (default=None)) – Minimum and maximum for node colormap scaling
- **vmax** (float, optional (default=None)) – Minimum and maximum for node colormap scaling
- **linewidths** ([None | scalar | sequence]) – Line width of symbol border (default=1.0)
- **width** (float, optional (default=1.0)) – Line width of edges
- **edge_color** (color string, or array of floats (default='r')) – Edge color. Can be a single color format string, or a sequence of colors with the same length as *edgelist*. If numeric values are specified they will be mapped to colors using the *edge_cmap* and *edge_vmin*, *edge_vmax* parameters.
- **edge_cmap** (*Matplotlib colormap*, optional (default=None)) – Colormap for mapping intensities of edges
- **edge_vmin** (floats, optional (default=None)) – Minimum and maximum for edge colormap scaling
- **edge_vmax** (floats, optional (default=None)) – Minimum and maximum for edge colormap scaling
- **style** (string, optional (default='solid')) – Edge line style (solid|dashed|dotted,dashdot)
- **labels** (dictionary, optional (default=None)) – Node labels in a dictionary keyed by node of text labels
- **font_size** (int, optional (default=12)) – Font size for text labels
- **font_color** (string, optional (default='k' black)) – Font color string
- **font_weight** (string, optional (default='normal')) – Font weight

- **font_family** (*string*, *optional* (default='sans-serif')) – Font family
- **label** (*string*, *optional*) – Label for graph legend

Notes

For directed graphs, arrows are drawn at the head end. Arrows can be turned off with keyword `arrows=False`.

Examples

```
>>> G = nx.dodecahedral_graph()
>>> nx.draw(G)
>>> nx.draw(G, pos=nx.spring_layout(G)) # use spring layout
```

```
>>> import matplotlib.pyplot as plt
>>> limits = plt.axis('off') # turn off axis
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

`draw`, `draw_networkx_nodes`, `draw_networkx_edges`, `draw_networkx_labels`,
`draw_networkx_edge_labels`

```
py3plex.visualization.drawing_machinery.draw_networkx_edge_labels(G, pos,
                                                                    edge_labels=None,
                                                                    label_pos=0.5,
                                                                    font_size=10,
                                                                    font_color='k',
                                                                    font_family='sans-serif',
                                                                    font_weight='normal',
                                                                    alpha=1.0, bbox=None,
                                                                    ax=None, rotate=True,
                                                                    **kwds)
```

Draw edge labels.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.
- **ax** (*Matplotlib Axes object*, *optional*) – Draw the graph in the specified Matplotlib axes.
- **alpha** (*float*) – The text transparency (default=1.0)
- **edge_labels** (*dictionary*) – Edge labels in a dictionary keyed by edge two-tuple of text labels (default=None). Only labels for the keys in the dictionary are drawn.
- **label_pos** (*float*) – Position of edge label along edge (0=head, 0.5=center, 1=tail)
- **font_size** (*int*) – Font size for text labels (default=12)
- **font_color** (*string*) – Font color string (default='k' black)
- **font_weight** (*string*) – Font weight (default='normal')
- **font_family** (*string*) – Font family (default='sans-serif')
- **bbox** (*Matplotlib bbox*) – Specify text box shape and colors.

- **clip_on** (*bool*) – Turn on clipping at axis boundaries (default=True)

Returns

dict of labels keyed on the edges

Return type

dict

Examples

```
>>> G = nx.dodecahedral_graph()
>>> edge_labels = nx.draw_networkx_edge_labels(G, pos=nx.spring_layout(G))
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

draw, draw_networkx, draw_networkx_nodes, draw_networkx_edges, draw_networkx_labels

```
py3plex.visualization.drawing_machinery.draw_networkx_edges(G, pos, edgelist=None,
                                                            width=1.0, edge_color='k',
                                                            style='solid', alpha=1.0,
                                                            arrowstyle='->', arrowsize=10,
                                                            edge_cmap=None,
                                                            edge_vmin=None,
                                                            edge_vmax=None, ax=None,
                                                            arrows=True, label=None,
                                                            node_size=300, nodelist=None,
                                                            node_shape='o', **kws)
```

Draw the edges of the graph G.

This draws only the edges of the graph G.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.
- **edgelist** (*collection of edge tuples*) – Draw only specified edges (default=G.edges())
- **width** (*float, or array of floats*) – Line width of edges (default=1.0)
- **edge_color** (*color string, or array of floats*) – Edge color. Can be a single color format string (default='r'), or a sequence of colors with the same length as edgelist. If numeric values are specified they will be mapped to colors using the edge_cmap and edge_vmin, edge_vmax parameters.
- **style** (*string*) – Edge line style (default='solid') (solid|dashed|dotted,dashdot)
- **alpha** (*float*) – The edge transparency (default=1.0)
- **cmap** (*edge*) – Colormap for mapping intensities of edges (default=None)
- **edge_vmin** (*floats*) – Minimum and maximum for edge colormap scaling (default=None)
- **edge_vmax** (*floats*) – Minimum and maximum for edge colormap scaling (default=None)

- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.
- **arrows** (*bool, optional (default=True)*) – For directed graphs, if True draw arrowheads. Note: Arrows will be the same color as edges.
- **arrowstyle** (*str, optional (default='->')*) – For directed graphs, choose the style of the arrow heads. See `:py:class: matplotlib.patches.ArrowStyle` for more options.
- **arrowsize** (*int, optional (default=10)*) – For directed graphs, choose the size of the arrow head head's length and width. See `:py:class: matplotlib.patches.FancyArrowPatch` for attribute `mutation_scale` for more info.
- **label** (*[None | string]*) – Label for legend

Returns

- `matplotlib.collection.LineCollection` – *LineCollection* of the edges
- *list of matplotlib.patches.FancyArrowPatch* – *FancyArrowPatch* instances of the directed edges
- *Depending whether the drawing includes arrows or not.*

Notes

For directed graphs, arrows are drawn at the head end. Arrows can be turned off with keyword `arrows=False`. Be sure to include `node_size` as a keyword argument; arrows are drawn considering the size of nodes.

Examples

```
>>> G = nx.dodecahedral_graph()
>>> edges = nx.draw_networkx_edges(G, pos=nx.spring_layout(G))
```

```
>>> G = nx.DiGraph()
>>> G.add_edges_from([(1, 2), (1, 3), (2, 3)])
>>> arcs = nx.draw_networkx_edges(G, pos=nx.spring_layout(G))
>>> alphas = [0.3, 0.4, 0.5]
>>> for i, arc in enumerate(arcs): # change alpha values of arcs
...     arc.set_alpha(alphas[i])
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

`draw`, `draw_networkx`, `draw_networkx_nodes`, `draw_networkx_labels`,
`draw_networkx_edge_labels`

```
py3plex.visualization.drawing_machinery.draw_networkx_labels(G, pos, labels=None,
                                                             font_size=1, font_color='k',
                                                             font_family='sans-serif',
                                                             font_weight='normal',
                                                             alpha=1.0, bbox=None,
                                                             ax=None, **kws)
```

Draw node labels on the graph G.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.

- **labels** (*dictionary, optional (default=None)*) – Node labels in a dictionary keyed by node of text labels Node-keys in labels should appear as keys in *pos*. If needed use: *{n:lab for n,lab in labels.items() if n in pos}*
- **font_size** (*int*) – Font size for text labels (default=12)
- **font_color** (*string*) – Font color string (default='k' black)
- **font_family** (*string*) – Font family (default='sans-serif')
- **font_weight** (*string*) – Font weight (default='normal')
- **alpha** (*float*) – The text transparency (default=1.0)
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.

Returns

dict of labels keyed on the nodes

Return type

dict

Examples

```
>>> G = nx.dodecahedral_graph()
>>> labels = nx.draw_networkx_labels(G, pos=nx.spring_layout(G))
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

[*draw*](#), [*draw_networkx*](#), [*draw_networkx_nodes*](#), [*draw_networkx_edges*](#),
[*draw_networkx_edge_labels*](#)

```
py3plex.visualization.drawing_machinery.draw_networkx_nodes(G, pos, nodelist=None,
                                                             node_size=300, node_color='r',
                                                             node_shape='o', alpha=1.0,
                                                             cmap=None, vmin=None,
                                                             vmax=None, ax=None,
                                                             linewidths=None,
                                                             edgecolors=None, label=None,
                                                             **kwargs)
```

Draw the nodes of the graph G.

This draws only the nodes of the graph G.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.
- **nodelist** (*list, optional*) – Draw only specified nodes (default G.nodes())
- **node_size** (*scalar or array*) – Size of nodes (default=300). If an array is specified it must be the same length as nodelist.
- **node_color** (*color string, or array of floats*) – Node color. Can be a single color format string (default='r'), or a sequence of colors with the same length as

odelist. If numeric values are specified they will be mapped to colors using the `cmap` and `vmin,vmax` parameters. See `matplotlib.scatter` for more details.

- **node_shape** (*string*) – The shape of the node. Specification is as `matplotlib.scatter` marker, one of 'so^>v<dph8' (default='o').
- **alpha** (*float or array of floats*) – The node transparency. This can be a single alpha value (default=1.0), in which case it will be applied to all the nodes of color. Otherwise, if it is an array, the elements of alpha will be applied to the colors in order (cycling through alpha multiple times if necessary).
- **cmap** (*Matplotlib colormap*) – Colormap for mapping intensities of nodes (default=None)
- **vmin** (*floats*) – Minimum and maximum for node colormap scaling (default=None)
- **vmax** (*floats*) – Minimum and maximum for node colormap scaling (default=None)
- **linewidths** (*[None | scalar | sequence]*) – Line width of symbol border (default=1.0)
- **edgecolors** (*[None | scalar | sequence]*) – Colors of node borders (default = node_color)
- **label** (*[None | string]*) – Label for legend

Returns

PathCollection of the nodes.

Return type

`matplotlib.collections.PathCollection`

Examples

```
>>> G = nx.dodecahedral_graph()
>>> nodes = nx.draw_networkx_nodes(G, pos=nx.spring_layout(G))
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

[`draw`](#), [`draw\_networkx`](#), [`draw\_networkx\_edges`](#), [`draw\_networkx\_labels`](#),
[`draw\_networkx\_edge\_labels`](#)

`py3plex.visualization.drawing_machinery.draw_random(G, **kwargs)`

Draw the graph G with a random layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_shell(G, **kwargs)`

Draw networkx graph with shell layout.

Parameters

- **G** (*graph*) – A networkx graph

- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_spectral(G, **kwargs)`

Draw the graph `G` with a spectral layout.

Parameters

- **G** (*graph*) – A `networkx` graph
- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_spring(G, **kwargs)`

Draw the graph `G` with a spring layout.

Parameters

- **G** (*graph*) – A `networkx` graph
- **kwargs** (*optional keywords*) – See `networkx.draw_networkx()` for a description of optional keywords, with the exception of the `pos` parameter which is not used by this function.

Color utilities for `py3plex` visualization.

`py3plex.visualization.colors.RGB_to_hex(RGB: List[int]) → str`

Convert RGB list to hex color.

Parameters

RGB – RGB values as [R, G, B] list

Returns

Hex color string like “#FFFFFF”

`py3plex.visualization.colors.color_dict(gradient: List[List[int]]) → Dict[str, List]`

Takes in a list of RGB sub-lists and returns dictionary of colors in RGB and hex form for use in a graphing function defined later on.

Parameters

gradient – List of RGB color values

Returns

Dictionary with ‘hex’, ‘r’, ‘g’, ‘b’ keys

`py3plex.visualization.colors.hex_to_RGB(hex: str) → List[int]`

Convert hex color to RGB list.

Parameters

hex – Hex color string like “#FFFFFF”

Returns

RGB values as [R, G, B] list

`py3plex.visualization.colors.linear_gradient(start_hex: str, finish_hex: str = '#FFFFFF', n: int = 10) → Dict[str, List]`

Returns a gradient list of (n) colors between two hex colors.

Parameters

- **start_hex** – Starting color as six-digit hex string (e.g., “#FFFFFF”)
- **finish_hex** – Ending color as six-digit hex string (default: “#FFFFFF”)
- **n** – Number of colors in gradient (default: 10)

Returns

Dictionary with 'hex', 'r', 'g', 'b' keys containing gradient colors

```
py3plex.visualization.layout_algorithms.compute_force_directed_layout(g: Graph,  
    layout_parameters:  
        Dict[str, Any] | None  
        = None, verbose:  
        bool = True, gravity:  
        float = 0.2,  
        strongGravityMode:  
        bool = False,  
        barnesHutTheta:  
        float = 1.2,  
        edgeWeightInflu-  
        ence: float = 1,  
        scalingRatio: float =  
        2.0, forceImport:  
        bool = True, seed:  
        int | None = None)  
    → Dict[Any,  
        ndarray]
```

Compute force-directed layout for a graph using ForceAtlas2 or NetworkX spring layout.

Parameters

- **g** – NetworkX graph to layout
- **layout_parameters** – Optional parameters to pass to layout algorithm
- **verbose** – Whether to print progress information
- **gravity** – Attraction force towards the center (must be non-negative)
- **strongGravityMode** – Use strong gravity mode
- **barnesHutTheta** – Barnes-Hut approximation parameter
- **edgeWeightInfluence** – Influence of edge weights on layout
- **scalingRatio** – Scaling factor for the layout (must be positive)
- **forceImport** – Whether to use ForceAtlas2 (if available)
- **seed** – Random seed for reproducibility in fallback spring layout

Returns

Dictionary mapping nodes to 2D position arrays

Note

For large networks (>1000 nodes), this may be slow. Consider using faster layouts (circular, random, spectral) or matrix visualization.

Contracts:

- Precondition: graph must not be None and be a NetworkX graph
- Precondition: graph must have at least one node
- Precondition: gravity must be non-negative
- Precondition: scalingRatio must be positive
- Postcondition: result is a dictionary
- Postcondition: result has positions for all nodes

`py3plex.visualization.layout_algorithms.compute_random_layout(g: Graph, seed: int | None = None) → Dict[Any, ndarray]`

Compute a random layout for the graph.

Parameters

- **g** – NetworkX graph
- **seed** – Random seed for reproducibility

Returns

Dictionary mapping nodes to 2D positions

Contracts:

- Precondition: graph must not be None and be a NetworkX graph
- Precondition: graph must have at least one node
- Postcondition: result is a dictionary
- Postcondition: result has positions for all nodes

`py3plex.visualization.layout_algorithms.ensure(*args, **kwargs)`

`py3plex.visualization.layout_algorithms.require(*args, **kwargs)`

`py3plex.visualization.bezier.bezier_calculate_dfy(mp_y: float, path_height: float, x0: float, midpoint_x: float, x1: float, y0: float, y1: float, dfx: ndarray, mode: str = 'upper') → ndarray`

Calculate y-coordinates for bezier curve.

Parameters

- **mp_y** – Midpoint y-coordinate
- **path_height** – Height of the path
- **x0** – Start x-coordinate
- **midpoint_x** – Midpoint x-coordinate
- **x1** – End x-coordinate
- **y0** – Start y-coordinate
- **y1** – End y-coordinate
- **dfx** – Array of x-coordinates
- **mode** – Mode for curve calculation (“upper” or “bottom”)

Returns

Array of y-coordinates

`py3plex.visualization.bezier.draw_bezier(total_size: int, p1: Tuple[float, float], p2: Tuple[float, float], mode: str = 'quadratic', inversion: bool = False, path_height: float = 2, linemode: str = 'both', resolution: float = 0.1) → Tuple[ndarray, ndarray]`

Draw bezier curve between two points.

Parameters

- **total_size** – Total size of the drawing area
- **p1** – First point coordinates (x0, x1)
- **p2** – Second point coordinates (y0, y1)
- **mode** – Drawing mode (default: “quadratic”)

- **inversion** – Whether to invert the curve
- **path_height** – Height of the path
- **linemode** – Line drawing mode (“upper”, “bottom”, or “both”)
- **resolution** – Resolution for curve sampling

Returns

Tuple of (x-coordinates, y-coordinates) arrays

`py3plex.visualization.benchmark_visualizations.generic_grouping(fname: DataFrame, score_name: str, threshold: float = 1.0, percentages: bool = True) → DataFrame`

`py3plex.visualization.benchmark_visualizations.plot_core_macro(fname: DataFrame) → int`
A very simple visualization of the results..

`py3plex.visualization.benchmark_visualizations.plot_core_macro_box(fname: str) → int`

`py3plex.visualization.benchmark_visualizations.plot_core_macro_gg(fname: DataFrame) → None`

`py3plex.visualization.benchmark_visualizations.plot_core_micro(fname: DataFrame) → int`
A very simple visualization of the results..

`py3plex.visualization.benchmark_visualizations.plot_core_micro_gg(fname: DataFrame) → None`

`py3plex.visualization.benchmark_visualizations.plot_core_micro_grid(fname: str) → None`

`py3plex.visualization.benchmark_visualizations.plot_core_time(fname: DataFrame) → int`

`py3plex.visualization.benchmark_visualizations.plot_core_time_gg(fname: str) → None`

`py3plex.visualization.benchmark_visualizations.plot_core_variability(fname: str) → None`

`py3plex.visualization.benchmark_visualizations.plot_critical_distance(fname: str, num_algo: int = 14) → None`

`py3plex.visualization.benchmark_visualizations.plot_mean_times(fn: DataFrame) → None`

`py3plex.visualization.benchmark_visualizations.plot_robustness(infile: DataFrame) → None`

`py3plex.visualization.benchmark_visualizations.table_to_latex(fname, outfolder='./final_results/tables/', threshold=1)`

Wrappers

class `py3plex.wrappers.benchmark_nodes.TopKRanker(estimator, *, n_jobs=None, verbose=0)`

Bases: `OneVsRestClassifier`

predict(`X: ndarray, top_k_list: List[int]`) → `List[List]`

Predict top K labels for each sample.

Parameters

- **X** – Feature matrix
- **top_k_list** – List of K values for each sample

Returns

List of predicted labels for each sample

set_partial_fit_request(*, *classes*: *bool* | *None* | *str* = '\$UNCHANGED\$') → *TopKRanker*

Configure whether metadata should be requested to be passed to the `partial_fit` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config()`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- `True`: metadata is requested, and passed to `partial_fit` if provided. The request is ignored if metadata is not provided.
- `False`: metadata is not requested and the meta-estimator will not pass it to `partial_fit`.
- `None`: metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str`: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Parameters

classes (*str*, *True*, *False*, or *None*, *default*=`sklearn.utils.metadata_routing.UNCHANGED`) – Metadata routing for `classes` parameter in `partial_fit`.

Returns

self – The updated object.

Return type

object

set_predict_request(*, *top_k_list*: *bool* | *None* | *str* = '\$UNCHANGED\$') → *TopKRanker*

Configure whether metadata should be requested to be passed to the `predict` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config()`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- `True`: metadata is requested, and passed to `predict` if provided. The request is ignored if metadata is not provided.
- `False`: metadata is not requested and the meta-estimator will not pass it to `predict`.
- `None`: metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str`: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Parameters

top_k_list (*str*, *True*, *False*, or *None*, *default*=`sklearn.utils.metadata_routing.UNCHANGED`) – Metadata routing for `top_k_list` parameter in `predict`.

Returns

self – The updated object.

Return type

object

set_score_request(* , *sample_weight*: *bool* | *None* | *str* = '\$UNCHANGED\$') → *TopKRanker*

Configure whether metadata should be requested to be passed to the `score` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config()`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- **True**: metadata is requested, and passed to `score` if provided. The request is ignored if metadata is not provided.
- **False**: metadata is not requested and the meta-estimator will not pass it to `score`.
- **None**: metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- **str**: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Parameters

sample_weight (*str*, *True*, *False*, or *None*, default=`sklearn.utils.metadata_routing.UNCHANGED`) – Metadata routing for `sample_weight` parameter in `score`.

Returns

self – The updated object.

Return type

object

`py3plex.wrappers.benchmark_nodes.benchmark_node_classification`(*path*: *str*, *core_network*: *Any*, *labels_matrix*: *Any*, *percent*: *Any* = 'all') → Dict[*Any*, *Any*]

Benchmark node classification using embeddings.

Parameters

- **path** – Path to embeddings file
- **core_network** – Network adjacency matrix
- **labels_matrix** – Labels for nodes
- **percent** – Training percentage or “all” for multiple percentages

Returns

Dictionary of classification results

`py3plex.wrappers.benchmark_nodes.main()`

`py3plex.wrappers.benchmark_nodes.sparse2graph`(*x*: *spmatrix*) → Dict[*str*, List[*str*]]

Convert sparse matrix to graph dictionary.

Parameters

x – Sparse matrix representation of graph

Returns

Dictionary mapping node IDs to lists of neighbor IDs

I/O Operations

For the complete auto-generated API documentation, see the AUTOGEN_results directory after building the docs. Aggregation and Network Operations — Vectorized multiplex aggregation for multilayer networks.

This module provides optimized implementations for aggregating edges across multiple layers using vectorized NumPy and SciPy sparse operations, replacing slower Python loops with efficient matrix operations.

Performance targets: - 3× speedup for 1M edges across 4 layers compared to legacy loop-based methods - Memory-efficient sparse matrix output by default - Float tolerance of 1e-6 for numerical equivalence with legacy methods

Author: py3plex development team License: MIT

```
py3plex.multinet.aggregation.aggregate_layers(edges: ndarray | list, weight_col: str | int = 'w',
                                             reducer: Literal['sum', 'mean', 'max'] = 'sum',
                                             to_sparse: bool = True) → csr_matrix | ndarray
```

Aggregate edge weights across multiple layers using vectorized operations.

This function replaces Python loops with efficient NumPy and SciPy sparse matrix operations for superior performance on large multilayer networks.

Complexity: $O(E)$ where E is the number of edges **Memory:** $O(E)$ for sparse output, $O(N^2)$ for dense output

Parameters

- **edges** – Edge data as ndarray with shape $(E, \geq 3)$ containing (layer, src, dst, [weight]) columns. If no weight column, assumes weight=1.0 for all edges. Can also accept list of lists.
- **weight_col** – Column name or index for weights (default “w”). If edges is ndarray, must be int index (0-based). Column 3 is used if it exists, else weights default to 1.
- **reducer** – Aggregation method - one of: - “sum”: Sum weights for edges appearing in multiple layers (default) - “mean”: Average weights across layers - “max”: Take maximum weight across layers
- **to_sparse** – If True, return scipy.sparse.csr_matrix (default, memory-efficient). If False, return dense numpy.ndarray.

Returns

Aggregated adjacency matrix in requested format (sparse CSR or dense). Shape is (N, N) where N is the maximum node ID + 1.

Raises

- **ValueError** – If edges array has wrong shape or reducer is invalid.
- **TypeError** – If edges is not ndarray or list-like.

Examples

```
>>> import numpy as np
>>> # Create edge list: (layer, src, dst, weight)
>>> edges = np.array([
...     [0, 0, 1, 1.0],
...     [0, 1, 2, 2.0],
...     [1, 0, 1, 0.5], # Same edge in layer 1
...     [1, 2, 3, 1.5],
... ])
>>> mat = aggregate_layers(edges, reducer="sum")
>>> mat.shape
(4, 4)
>>> mat[0, 1] # Sum of weights from both layers
1.5
```

```
>>> # With mean aggregation
>>> mat_mean = aggregate_layers(edges, reducer="mean", to_sparse=False)
>>> mat_mean[0, 1] # Average of 1.0 and 0.5
0.75
```

Notes

- Node IDs are assumed to be integers starting from 0
- Self-loops are supported
- For directed graphs, (i,j) and (j,i) are different edges
- Sparse output recommended for large networks (N > 1000)
- Deterministic output for fixed input order

Performance:

Achieves 3× speedup vs loop-based aggregation on 1M edges: - Legacy loop: ~2.5s for 1M edges, 4 layers - Vectorized: ~0.8s for same dataset (measured on standard hardware)

```
py3plex.multinet.aggregation.ensure(*args, **kwargs)
```

```
py3plex.multinet.aggregation.require(*args, **kwargs)
```

Profiling Utilities

Performance profiling utilities for py3plex.

This module provides decorators and utilities for tracking function execution time, memory usage, and performance metrics. These tools enable performance regression detection and optimization efforts.

Example

```
>>> from py3plex.profiling import profile_performance
>>>
>>> @profile_performance
>>> def slow_function():
...     # ... computation
...     pass
>>>
>>> slow_function() # Logs execution time automatically
```

```
class py3plex.profiling.PerformanceMonitor
```

Bases: object

Global performance monitoring registry.

Tracks execution times and call counts for profiled functions. Can be used to generate performance reports and detect regressions.

enabled

Whether profiling is enabled globally

stats

Dictionary mapping function names to performance statistics

clear()

Clear all collected statistics.

get_report() → str

Generate a performance report.

Returns

String containing formatted performance statistics

record(*func_name: str, elapsed: float, memory_delta: float | None = None*)

Record performance metrics for a function call.

Parameters

- **func_name** – Name of the function
- **elapsed** – Execution time in seconds
- **memory_delta** – Memory usage change in MB (optional)

py3plex.profiling.benchmark(*func: Callable, iterations: int = 100, warmup: int = 10, args: tuple = (), kwargs: dict | None = None*) → Dict[str, float]

Benchmark a function with multiple iterations.

Runs the function multiple times and collects statistics about execution time. Includes a warmup phase to allow JIT compilation and caching.

Parameters

- **func** – Function to benchmark
- **iterations** – Number of iterations to run (default: 100)
- **warmup** – Number of warmup iterations (default: 10)
- **args** – Positional arguments for the function
- **kwargs** – Keyword arguments for the function

Returns

- **mean**: Average execution time (seconds)
- **median**: Median execution time (seconds)
- **min**: Minimum execution time (seconds)
- **max**: Maximum execution time (seconds)
- **std**: Standard deviation (seconds)
- **total**: Total time for all iterations (seconds)

Return type

Dictionary containing benchmark statistics

Example

```
>>> from py3plex.profiling import benchmark
>>>
>>> def my_function(n):
...     return sum(range(n))
>>>
>>> stats = benchmark(my_function, iterations=1000, args=(1000,))
>>> print(f"Average time: {stats['mean']*1000:.3f}ms")
```

py3plex.profiling.get_monitor() → *PerformanceMonitor*

Get the global performance monitor instance.

Returns

Global performance monitoring instance

Return type*PerformanceMonitor***Example**

```
>>> from py3plex.profiling import get_monitor
>>> monitor = get_monitor()
>>> print(monitor.get_report())
```

`py3plex.profiling.profile_performance`(*func: Callable | None = None, *, log_args: bool = False, track_memory: bool = False*) → Callable

Decorator to track function execution time and optionally memory usage.

This decorator measures the wall-clock time taken by a function and logs it. It can also track memory usage changes if requested. Performance metrics are stored in the global performance monitor for later analysis.

Parameters

- **func** – Function to decorate (when used without arguments)
- **log_args** – If True, log function arguments (default: False)
- **track_memory** – If True, track memory usage (default: False)

Returns

Decorated function that logs execution time

Examples

Basic usage: `>>> @profile_performance ... def my_function(x, y): ... return x + y`

With options: `>>> @profile_performance(log_args=True, track_memory=True) ... def expensive_function(data): ... # ... expensive computation ... return result`

Manual wrapping: `>>> def my_function(): ... pass >>> profiled_func = profile_performance(my_function)`

Note

Memory tracking requires the `tracemalloc` module and adds overhead. Use it only when investigating memory issues.

`py3plex.profiling.timed_section`(*name: str, log_level: str = 'info'*)

Context manager for timing code blocks.

Useful for timing specific sections of code without wrapping entire functions.

Parameters

- **name** – Name of the code section being timed
- **log_level** – Logging level ('debug', 'info', 'warning', 'error')

Example

```
>>> from py3plex.profiling import timed_section
>>>
>>> with timed_section("data loading"):
...     data = load_large_dataset()
>>>
>>> with timed_section("computation", log_level="debug"):
...     result = complex_calculation()
```

Yields

None

Logging Configuration

Logging configuration for py3plex.

This module provides centralized logging configuration for the py3plex library.

`py3plex.logging_config.get_logger(name: str | None = None, level: int = 20) → Logger`

Get or create a logger for py3plex modules.

Parameters

- **name** – Logger name. If None, returns the root py3plex logger.
- **level** – Logging level (default: INFO)

Returns

Configured logger instance

Example

```
>>> from py3plex.logging_config import get_logger
>>> logger = get_logger(__name__)
>>> logger.info("Processing network...")
```

`py3plex.logging_config.setup_logging(level: int = 20, format_string: str | None = None) → Logger`

Configure logging for py3plex with custom settings.

Parameters

- **level** – Logging level (default: INFO)
- **format_string** – Custom format string (optional)

Returns

Root py3plex logger

Example

```
>>> from py3plex.logging_config import setup_logging
>>> logger = setup_logging(level=logging.DEBUG)
```

I/O Schema and Validation

Schema definitions for multilayer graphs using dataclasses.

This module provides dataclass representations of multilayer graph components with built-in validation and serialization support.

```
class py3plex.io.schema.Edge(src: ~typing.Hashable, dst: ~typing.Hashable, src_layer:
    ~typing.Hashable, dst_layer: ~typing.Hashable, key: int = 0, attributes:
    ~typing.Dict[str, ~typing.Any] = <factory>)
```

Bases: object

Represents an edge in a multilayer network.

src

Source node ID

Type

Hashable

dst
Destination node ID
Type
Hashable

src_layer
Source layer ID
Type
Hashable

dst_layer
Destination layer ID
Type
Hashable

key
Optional edge key for multigraphs (default: 0)
Type
int

attributes
Dictionary of edge attributes (must be JSON-serializable)
Type
Dict[str, Any]

attributes: Dict[str, Any]

dst: Hashable

dst_layer: Hashable

edge_tuple() → Tuple[Hashable, Hashable, Hashable, Hashable, int]
Return edge as a tuple for uniqueness checking.

Returns
Tuple of (src, dst, src_layer, dst_layer, key)

classmethod from_dict(data: Dict[str, Any]) → Edge
Create edge from dictionary.

Parameters
data – Dictionary containing edge data

Returns
Edge instance

key: int = 0

src: Hashable

src_layer: Hashable

to_dict() → Dict[str, Any]
Convert edge to dictionary.

Returns
Dictionary representation of the edge

```
class py3plex.io.schema.Layer(id: ~typing.Hashable, attributes: ~typing.Dict[str, ~typing.Any] =
                               <factory>)
```

Bases: object

Represents a layer in a multilayer network.

id

Unique identifier for the layer

Type

Hashable

attributes

Dictionary of layer attributes (must be JSON-serializable)

Type

Dict[str, Any]

attributes: Dict[str, Any]

classmethod from_dict(*data*: Dict[str, Any]) → Layer

Create layer from dictionary.

Parameters

data – Dictionary containing layer data

Returns

Layer instance

id: Hashable

to_dict() → Dict[str, Any]

Convert layer to dictionary.

Returns

Dictionary representation of the layer

```
class py3plex.io.schema.MultiLayerGraph(nodes: ~typing.Dict[~typing.Hashable,
~py3plex.io.schema.Node] = <factory>, layers:
~typing.Dict[~typing.Hashable, ~py3plex.io.schema.Layer]
= <factory>, edges: ~typing.List[~py3plex.io.schema.Edge]
= <factory>, directed: bool = True, attributes:
~typing.Dict[str, ~typing.Any] = <factory>)
```

Bases: object

Represents a complete multilayer graph.

nodes

Dictionary mapping node IDs to Node objects

Type

Dict[Hashable, py3plex.io.schema.Node]

layers

Dictionary mapping layer IDs to Layer objects

Type

Dict[Hashable, py3plex.io.schema.Layer]

edges

List of Edge objects

Type

List[py3plex.io.schema.Edge]

directed

Whether the graph is directed

Type

bool

attributes

Dictionary of graph-level attributes

Type

Dict[str, Any]

add_edge(*edge: Edge*)

Add an edge to the graph.

Parameters

edge – Edge to add

Raises

- **ReferentialIntegrityError** – If edge references non-existent nodes or layers
- **SchemaValidationError** – If edge is duplicate

add_layer(*layer: Layer*)

Add a layer to the graph.

Parameters

layer – Layer to add

Raises

SchemaValidationError – If layer ID already exists

add_node(*node: Node*)

Add a node to the graph.

Parameters

node – Node to add

Raises

SchemaValidationError – If node ID already exists

attributes: Dict[str, Any]

directed: bool = True

edges: List[Edge]

classmethod from_dict(*data: Dict[str, Any]*) → MultiLayerGraph

Create graph from dictionary.

Parameters

data – Dictionary containing graph data

Returns

MultiLayerGraph instance

layers: Dict[Hashable, Layer]

nodes: Dict[Hashable, Node]

to_dict() → Dict[str, Any]

Convert graph to dictionary.

Returns

Dictionary representation of the graph


```
class py3plex.io.schema.Node(id: ~typing.Hashable, attributes: ~typing.Dict[str, ~typing.Any] =
                             <factory>)
```

Bases: object

Represents a node in a multilayer network.

id

Unique identifier for the node

Type

Hashable

attributes

Dictionary of node attributes (must be JSON-serializable)

Type

Dict[str, Any]

attributes: Dict[str, Any]

classmethod from_dict(*data*: Dict[str, Any]) → Node

Create node from dictionary.

Parameters

data – Dictionary containing node data

Returns

Node instance

id: Hashable

to_dict() → Dict[str, Any]

Convert node to dictionary.

Returns

Dictionary representation of the node

Public API for reading and writing multilayer graphs.

This module provides the main entry points for I/O operations with format detection and a registry system for extensibility.

py3plex.io.api.ensure(*args, **kwargs)

py3plex.io.api.read(*filepath*: str | Path, *format*: str | None = None, **kwargs) → MultiLayerGraph

Read a multilayer graph from a file.

Parameters

- **filepath** – Path to the input file
- **format** – Format name (e.g., 'json', 'csv'). If None, auto-detected from extension
- ****kwargs** – Additional arguments passed to the format-specific reader

Returns

MultiLayerGraph instance

Raises

- **FormatUnsupportedError** – If format is not supported or cannot be detected
- **FileNotFoundError** – If file does not exist

Example

```
>>> graph = read('network.json')
>>> graph = read('network.csv', format='csv')
```

`py3plex.io.api.register_reader(format_name: str, reader_func: Callable[[...], MultiLayerGraph]) → None`

Register a reader function for a specific format.

Parameters

- **format_name** – Name of the format (e.g., 'json', 'csv', 'graphml')
- **reader_func** – Function that takes (filepath, ****kwargs**) and returns `MultiLayerGraph`

Example

```
>>> def my_reader(filepath, **kwargs):
...     # Custom reading logic
...     return MultiLayerGraph(...)
>>> register_reader('myformat', my_reader)
```

Contracts:

- Precondition: `format_name` must be a non-empty string
- Precondition: `reader_func` must be callable
- Postcondition: reader is registered in `_READERS`

`py3plex.io.api.register_writer(format_name: str, writer_func: Callable[[...], None]) → None`

Register a writer function for a specific format.

Parameters

- **format_name** – Name of the format (e.g., 'json', 'csv', 'graphml')
- **writer_func** – Function that takes (graph, filepath, ****kwargs**) and writes to file

Example

```
>>> def my_writer(graph, filepath, **kwargs):
...     # Custom writing logic
...     pass
>>> register_writer('myformat', my_writer)
```

Contracts:

- Precondition: `format_name` must be a non-empty string
- Precondition: `writer_func` must be callable
- Postcondition: writer is registered in `_WRITERS`

`py3plex.io.api.require(*args, **kwargs)`

`py3plex.io.api.supported_formats(read: bool = True, write: bool = True) → Dict[str, List[str]]`

Get list of supported formats for read and/or write operations.

Parameters

- **read** – Include formats that support reading
- **write** – Include formats that support writing

Returns

Dictionary with 'read' and/or 'write' keys containing lists of format names

Example

```
>>> formats = supported_formats()
>>> print(formats)
{'read': ['json', 'jsonl', 'csv'], 'write': ['json', 'jsonl', 'csv']}
```

Contracts:

- Postcondition: result is a dictionary
- Postcondition: result contains 'read' key when read=True
- Postcondition: result contains 'write' key when write=True

`py3plex.io.api.write(graph: MultiLayerGraph, filepath: str | Path, format: str | None = None, **kwargs)`
→ None

Write a multilayer graph to a file.

Parameters

- **graph** – MultiLayerGraph to write
- **filepath** – Path to the output file
- **format** – Format name (e.g., 'json', 'csv'). If None, auto-detected from extension
- ****kwargs** – Additional arguments passed to the format-specific writer

Raises

FormatUnsupportedError – If format is not supported or cannot be detected

Example

```
>>> write(graph, 'network.json')
>>> write(graph, 'network.csv', format='csv', deterministic=True)
```

Hedwig Rule Learning

`py3plex.algorithms.hedwig.build_graph(kwargs: Dict[str, Any])` → Any

`py3plex.algorithms.hedwig.generate_rules_report(kwargs: ~typing.Dict[str, ~typing.Any],
rules_per_target:
~typing.List[~typing.Tuple[~typing.Any,
~typing.List[~typing.Any]]], human:
~typing.Callable[[~typing.Any, ~typing.Any],
~typing.Any] = <function <lambda>>)` → str

`py3plex.algorithms.hedwig.rule_kernel(target: Any)` → Tuple[Any, List[Any]]

`py3plex.algorithms.hedwig.run(kwargs: Dict[str, Any], cli: bool = True, generator_tag: bool = False,
num_threads: str | int = 'all')` → List[Tuple[Any, List[Any]]]

`py3plex.algorithms.hedwig.run_learner(kwargs: Dict[str, Any], kb: ExperimentKB, validator:
Validate, generator: bool = False, num_threads: str | int =
'all')` → List[Tuple[Any, List[Any]]]

Example-related classes.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.core.example.Example(id, label, score, annotations=None,
                                                    weights=None)
```

Bases: object

Represents an example with its score, label, id and annotations.

ClassLabeled = 'class'

Ranked = 'ranked'

Predicate-related classes.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.core.predicate.BinaryPredicate(label, pairs, kb,
                                                                producer_pred=None)
```

Bases: *Predicate*

A binary predicate.

```
class py3plex.algorithms.hedwig.core.predicate.Predicate(label, kb, producer_pred)
```

Bases: object

Represents a predicate as a member of a certain rule.

i = -1

```
class py3plex.algorithms.hedwig.core.predicate.UnaryPredicate(label, members, kb,
                                                                producer_pred=None,
                                                                custom_var_name=None,
                                                                negated=False)
```

Bases: *Predicate*

A unary predicate.

The rule class.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.core.rule.Rule(kb, predicates=None, target=None)
```

Bases: object

Represents a rule, along with its description, examples and statistics.

clone()

Returns a clone of this rule. The predicates themselves are NOT cloned.

clone_append(predicate_label, producer_pred, bin=False)

Returns a copy of this rule where 'predicate_label' is appended to the rule.

clone_negate(target_pred)

Returns a copy of this rule where 'target_pred' is negated.

clone_swap_with_subclass(target_pred, child_pred_label)

Returns a copy of this rule where 'target_pred' is swapped for 'child_pred_label'.

examples(positive_only=False)

Returns the covered examples.

property positives

precision()

rule_report(show_uris=False, latex=False)

Rule as string with some statistics.

```

static ruleset_examples_json(rules_per_target, show_uris=False)

static ruleset_report(rules, show_uris=False, latex=False, human=<function Rule.<lambda>>)

similarity(rule)
    Calculates the similarity between this rule and 'rule'.

size()
    Returns the number of conjunts.

static to_json(rules_per_target, show_uris=False)

```

Force Atlas 2 Visualization

```

class py3plex.visualization.fa2.forceatlas2.ForceAtlas2(outboundAttractionDistribution=False,
                                                         linLogMode=False, adjustSizes=False,
                                                         edgeWeightInfluence=1.0,
                                                         jitterTolerance=1.0,
                                                         barnesHutOptimize=True,
                                                         barnesHutTheta=1.2,
                                                         multiThreaded=False,
                                                         scalingRatio=2.0,
                                                         strongGravityMode=False,
                                                         gravity=1.0, verbose=True)

```

Bases: object

```

forceatlas2(G, pos=None, iterations=30)

forceatlas2_igraph_layout(G, pos=None, iterations=100, weight_attr=None)

forceatlas2_networkx_layout(G, pos=None, iterations=100)

init(G, pos=None)

```

```

class py3plex.visualization.fa2.forceatlas2.Timer(name='Timer')

```

Bases: object

```

display()

start()

stop()

```

```

class py3plex.visualization.fa2.fa2util.Edge

```

Bases: object

```

class py3plex.visualization.fa2.fa2util.Node

```

Bases: object

```

class py3plex.visualization.fa2.fa2util.Region(nodes)

```

Bases: object

```

applyForce(n, theta, coefficient=0)

applyForceOnNodes(nodes, theta, coefficient=0)

buildSubRegions()

updateMassAndGeometry()

```

```

py3plex.visualization.fa2.fa2util.adjustSpeedAndApplyForces(nodes, speed, speedEfficiency,
                                                             jitterTolerance)

```

```
py3plex.visualization.fa2.fa2util.apply_attraction(nodes, edges, distributedAttraction,
                                                    coefficient, edgeWeightInfluence)

py3plex.visualization.fa2.fa2util.apply_gravity(nodes, gravity, useStrongGravity=False)

py3plex.visualization.fa2.fa2util.apply_repulsion(nodes, coefficient)

py3plex.visualization.fa2.fa2util.linAttraction(n1, n2, e, distributedAttraction, coefficient=0)

py3plex.visualization.fa2.fa2util.linGravity(n, g)

py3plex.visualization.fa2.fa2util.linRepulsion(n1, n2, coefficient=0)

py3plex.visualization.fa2.fa2util.linRepulsion_region(n, r, coefficient=0)

py3plex.visualization.fa2.fa2util.strongGravity(n, g, coefficient=0)
```

Embedding Visualization

```
py3plex.visualization.embedding_visualization.embedding_visualization.visualize_embedding(multinet,
                                                                                          labels=None,
                                                                                          verbose=True)
```

Network Generation and Benchmarking

Additional Statistics and Analysis

```
py3plex.algorithms.statistics.bayesiantests.correlated_ttest(x, rope, runs=1, verbose=False,
                                                             names=('C1', 'C2'))

py3plex.algorithms.statistics.bayesiantests.correlated_ttest_MC(x, rope, runs=1,
                                                                nsamples=50000)
```

See `correlated_ttest` module for explanations

```
py3plex.algorithms.statistics.bayesiantests.heaviside(X)
```

Compute the Heaviside step function.

The Heaviside function returns 1 for positive values, 0.5 for zero, and 0 for negative values. This is used in signed-rank tests for Bayesian comparisons.

Parameters

X – Input array or scalar

Returns

Array with same shape as **X** containing:

- 1.0 where $X > 0$
- 0.5 where $X == 0$
- 0.0 where $X < 0$

Return type

np.ndarray

Examples

```
>>> heaviside(np.array([-1, 0, 1, 2]))
array([0. , 0.5, 1. , 1. ])
```

```
py3plex.algorithms.statistics.bayesiantests.hierarchical(diff, rope, rho, upperAlpha=2,
lowerAlpha=1, lowerBeta=0.01,
upperBeta=0.1,
std_upper_bound=1000,
verbose=False, names=('C1', 'C2'))
```

Perform hierarchical Bayesian test for comparing algorithms across multiple datasets.

This test accounts for the hierarchical structure of the data (multiple datasets, each with multiple folds) and correlations due to overlapping training sets.

Parameters

- **diff** – Array of differences between classifier scores
- **rope** – Width of the region of practical equivalence (ROPE)
- **rho** – Correlation between folds (typically around $1/n_folds$)
- **upperAlpha** – Upper bound for alpha parameter of Gamma prior (default: 2)
- **lowerAlpha** – Lower bound for alpha parameter of Gamma prior (default: 1)
- **lowerBeta** – Lower bound for beta parameter of Gamma prior (default: 0.01)
- **upperBeta** – Upper bound for beta parameter of Gamma prior (default: 0.1)
- **std_upper_bound** – Upper bound multiplier for standard deviation prior (default: 1000) Posterior is insensitive to this if large enough (>100)
- **verbose** – Whether to print probability results (default: False)
- **names** – Tuple of classifier names for verbose output (default: ("C1", "C2"))

Returns

(p_left, p_rope, p_right)

- p_left: Probability that first classifier is worse
- p_rope: Probability that classifiers are practically equivalent
- p_right: Probability that first classifier is better

Return type

Tuple[float, float, float]

Notes

- The Gamma distribution parameters control the prior on degrees of freedom
- The hierarchical structure models between-dataset and within-dataset variance
- Use when comparing algorithms across multiple datasets with cross-validation

References

- Benavoli, A., Corani, G., & Mangili, F. (2016). Should we really use post-hoc tests based on mean-ranks? The Journal of Machine Learning Research.

See also

hierarchical_MC: Monte Carlo sampling version correlated_ttest: Simpler test for single dataset comparisons

```
py3plex.algorithms.statistics.bayesiantests.hierarchical_MC(diff, rope, rho, upperAlpha=2,  
                                                            lowerAlpha=1, lowerBeta=0.01,  
                                                            upperBeta=0.1,  
                                                            std_upper_bound=1000,  
                                                            names=('C1', 'C2'))
```

Monte Carlo sampling for hierarchical Bayesian test.

Generates Monte Carlo samples from the posterior distribution for hierarchical comparison of algorithms across multiple datasets with cross-validation.

Parameters

- **diff** – Array of differences between classifier scores (shape: n_datasets x n_folds)
- **rope** – Width of the region of practical equivalence (ROPE)
- **rho** – Correlation between folds due to overlapping training sets
- **upperAlpha** – Upper bound for alpha parameter of Gamma prior (default: 2)
- **lowerAlpha** – Lower bound for alpha parameter of Gamma prior (default: 1)
- **lowerBeta** – Lower bound for beta parameter of Gamma prior (default: 0.01)
- **upperBeta** – Upper bound for beta parameter of Gamma prior (default: 0.1)
- **std_upper_bound** – Upper bound multiplier for standard deviation prior (default: 1000)
- **names** – Tuple of classifier names for identification (default: (“C1”, “C2”))

Returns

Monte Carlo samples with shape (n_samples, 3)

Each row contains [p_left, p_rope, p_right] for one MC sample

Return type

np.ndarray

Notes

- Uses PyStan for Bayesian inference with hierarchical model
- Data is rescaled by mean standard deviation for numerical stability
- Hierarchical structure captures both within-dataset and between-dataset variance
- Requires pystan package to be installed

Implementation:

- Uses Stan’s NUTS sampler with 4 chains
- Each chain runs 100 iterations (including warmup)
- Total posterior samples: ~200 after warmup

See also

hierarchical: Main function that processes MC samples
correlated_ttest_MC: Simpler MC version for single dataset

Raises

ImportError – If pystan is not installed


```
py3plex.algorithms.statistics.bayesiantests.plot_posterior(samples, names=('C1', 'C2'),
                                                           proba_triplet=None)
```

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **names** (*pair of str*) – the names of the two classifiers

Returns

matplotlib.pyplot.figure

```
py3plex.algorithms.statistics.bayesiantests.plot_simplex(points, names=('C1', 'C2'),
                                                           proba_triplet=None)
```

```
py3plex.algorithms.statistics.bayesiantests.signrank(x, rope, prior_strength=0.6,
                                                       prior_place=1, nsamples=50000,
                                                       verbose=False, names=('C1', 'C2'))
```

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 0.6)
- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples
- **verbose** (*bool*) – report the computed probabilities
- **names** (*pair of str*) – the names of the two classifiers

Returns

p_left, p_rope, p_right

```
py3plex.algorithms.statistics.bayesiantests.signrank_MC(x, rope, prior_strength=0.6,
                                                         prior_place=1, nsamples=50000)
```

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 0.6)
- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples

Returns

2-d array with rows corresponding to samples and columns to probabilities [*p_left, p_rope, p_right*]

```
py3plex.algorithms.statistics.bayesiantests.signtest(x, rope, prior_strength=1, prior_place=1,
                                                       nsamples=50000, verbose=False,
                                                       names=('C1', 'C2'))
```

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 1)

- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples
- **verbose** (*bool*) – report the computed probabilities
- **names** (*pair of str*) – the names of the two classifiers

Returns

p_left, p_rope, p_right

```
py3plex.algorithms.statistics.bayesiantests.signtest_MC(x, rope, prior_strength=1,  
                                                       prior_place=1, nsamples=50000)
```

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 1)
- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples

Returns

2-d array with rows corresponding to samples and columns to probabilities [*p_left, p_rope, p_right*]

Community Detection Advanced

Node Ranking and Clustering (NoRC) module for community detection.

This module implements algorithms for node ranking and hierarchical clustering in networks, including parallel PageRank computation and hierarchical merging.

```
py3plex.algorithms.community_detection.NoRC.NoRC_communities_main(input_graph, cluster-  
                                                                    ing_scheme='hierarchical',  
                                                                    max_com_num=100,  
                                                                    verbose=False,  
                                                                    parallel_step=None,  
                                                                    prob_threshold=0.0005,  
                                                                    community_range=None,  
                                                                    fine_range=3,  
                                                                    lag_threshold=10)
```

```
py3plex.algorithms.community_detection.NoRC.page_rank_kernel(index_row)
```

Compute normalized PageRank vector for a given node.

Parameters

index_row – Node index for which to compute PageRank

Returns

Tuple of (*node_index, normalized_pagerank_vector*)

Community-based node ranking framework.

This module implements a framework for ranking nodes within and across communities using PageRank-based metrics and hierarchical clustering.

```
py3plex.algorithms.community_detection.community_ranking.create_tree(centers: ndarray) →  
                                                                    Dict[int, Dict[str,  
                                                                    List]]
```

```
py3plex.algorithms.community_detection.community_ranking.page_rank_kernel(index_row: int)
                                                                    → Tuple[int,
                                                                    ndarray]
```

```
py3plex.algorithms.community_detection.community_ranking.return_infomap_communities(network:
                                                                                      Any)
                                                                    →
                                                                    List[List[int]]
```

Node Ranking and Classification

Node ranking algorithms for multilayer networks.

This module provides various node ranking algorithms including PageRank variants, HITS (Hubs and Authorities), and personalized PageRank (PPR) for network analysis.

Key Functions:

- `sparse_page_rank`: Compute PageRank scores using sparse matrix operations
- `run_PPR`: Run Personalized PageRank in parallel across multiple cores
- `hubs_and_authorities`: Compute HITS scores for nodes
- `stochastic_normalization`: Normalize adjacency matrix to stochastic form

Notes

The PageRank implementations use sparse matrices for memory efficiency and support parallel computation for large-scale networks.

```
py3plex.algorithms.node_ranking.authority_matrix(graph: Graph) → ndarray
```

Get the authority matrix representation of a graph.

Computes the matrix $A = A.T @ A$ where A is the adjacency matrix. The authority matrix is used in HITS algorithm computation.

Parameters

graph – NetworkX graph

Returns

Authority matrix ($N \times N$) where N is number of nodes

Return type

np.ndarray

Notes

- For directed graphs: $A[i,j]$ = number of nodes that point to both i and j
- Used internally by HITS algorithm to compute authority scores

See also

`hub_matrix`: Complementary hub matrix `hubs_and_authorities`: Compute actual hub/authority scores

```
py3plex.algorithms.node_ranking.damping_hyper: float
```

```
py3plex.algorithms.node_ranking.hub_matrix(graph: Graph) → ndarray
```

Get the hub matrix representation of a graph.

Computes the matrix $H = A @ A.T$ where A is the adjacency matrix. The hub matrix is used in HITS algorithm computation.

Parameters**graph** – NetworkX graph**Returns**

Hub matrix (N x N) where N is number of nodes

Return type

np.ndarray

Notes

- For directed graphs: $H[i,j]$ = number of nodes pointed to by both i and j
- Used internally by HITS algorithm to compute hub scores

See also

authority_matrix: Complementary authority matrix hubs_and_authorities: Compute actual hub/authority scores

`py3plex.algorithms.node_ranking.hubs_and_authorities(graph: Graph) → Tuple[dict, dict]`

Compute HITS (Hubs and Authorities) scores for all nodes in a graph.

Implements the Hyperlink-Induced Topic Search (HITS) algorithm to identify hub nodes (nodes that point to many authorities) and authority nodes (nodes pointed to by many hubs) in a network.

Parameters**graph** – NetworkX graph (directed or undirected)**Returns****(hub_scores, authority_scores)**

- hub_scores: Dictionary mapping node -> hub score
- authority_scores: Dictionary mapping node -> authority score

Return type

Tuple[dict, dict]

Notes

- Uses scipy-based implementation from NetworkX (nx.hits_scipy)
- Scores are normalized so that the sum of squares equals 1
- For undirected graphs, hub and authority scores are identical
- Converges using power iteration method

Examples

```
>>> import networkx as nx
>>> G = nx.DiGraph([(0, 1), (0, 2), (1, 2)])
>>> hubs, authorities = hubs_and_authorities(G)
>>> # Node 0 has high hub score (points to others)
>>> # Node 2 has high authority score (pointed to by others)
```

See also

hub_matrix: Get the hub matrix representation authority_matrix: Get the authority matrix representation

`py3plex.algorithms.node_ranking.page_rank_kernel(index_row: int) → Tuple[int, ndarray]`

Compute PageRank vector for a single starting node (multiprocessing kernel).

This function is designed to be called in parallel via `multiprocessing.Pool.map()`. It computes the personalized PageRank vector starting from a single node.

Parameters

index_row – Index of the starting node for personalized PageRank

Returns

(node_index, normalized_pagerank_vector)

- **node_index**: The input index (for tracking results)
- **pagerank_vector**: L2-normalized PageRank scores for all nodes

Return type

`Tuple[int, np.ndarray]`

Notes

- Accesses global variables: `__graph_matrix`, `damping_hyper`, `spread_step_hyper`, `spread_percent_hyper` (set by `run_PPR` before parallel execution)
- Returns zero vector if normalization fails
- L2 normalization ensures comparable magnitudes across different starting nodes

See also

`run_PPR`: Main function that sets up parallel execution `sparse_page_rank`: Core PageRank computation

`py3plex.algorithms.node_ranking.run_PPR(network: spmatrix, cores: int | None = None, jobs: List[range] | None = None, damping: float = 0.85, spread_step: int = 10, spread_percent: float = 0.3, targets: List[int] | None = None, parallel: bool = True) → Generator[Tuple[int, ndarray] | List[Tuple[int, ndarray]], None, None]`

Run Personalized PageRank (PPR) in parallel for multiple starting nodes.

Computes personalized PageRank vectors for multiple nodes using parallel processing. This is useful for creating node embeddings or analyzing node importance from different perspectives in the network.

Parameters

- **network** – Sparse adjacency matrix (will be automatically normalized to stochastic form)
- **cores** – Number of CPU cores to use (default: all available cores)
- **jobs** – Custom job batches as list of ranges (default: auto-generated)
- **damping** – Damping factor for PageRank (default: 0.85) Higher values (0.85-0.99) emphasize network structure
- **spread_step** – Steps to check spread pattern for optimization (default: 10)
- **spread_percent** – Max node fraction for shrinkage optimization (default: 0.3)
- **targets** – Specific node indices to compute PPR for (default: all nodes)
- **parallel** – Enable parallel processing (default: True) Set to False for debugging or single-core execution

Yields

Union[Tuple[int, np.ndarray], List[Tuple[int, np.ndarray]]] – - If parallel=True: Lists of (node_index, pagerank_vector) tuples (batched) - If parallel=False: Individual (node_index, pagerank_vector) tuples

Notes

- Automatically normalizes input matrix to column-stochastic form
- Uses multiprocessing.Pool for parallel execution
- Global variables are used to share the graph matrix across processes
- Results are yielded incrementally (generator pattern) to save memory
- Each pagerank_vector is L2-normalized for comparability

Examples

```
>>> import scipy.sparse as sp
>>> # Create a small network
>>> adj = sp.csr_matrix([[0, 1, 1], [1, 0, 1], [1, 1, 0]])
>>>
>>> # Compute PPR for all nodes in parallel
>>> for batch in run_PPR(adj, cores=2, parallel=True):
...     for node_idx, pr_vector in batch:
...         print(f"Node {node_idx}: {pr_vector}")
>>>
>>> # Compute PPR for specific nodes without parallelism
>>> for node_idx, pr_vector in run_PPR(adj, targets=[0, 1], parallel=False):
...     print(f"Node {node_idx}: {pr_vector}")
```

Performance:

- Parallel speedup scales with number of cores (up to ~0.8 * cores efficiency)
- Memory usage: O(N * N_targets) for storing results
- For large networks (>100K nodes), consider processing targets in batches

See also

sparse_page_rank: Core PageRank computation
 page_rank_kernel: Worker function for parallel execution
 stochastic_normalization: Matrix normalization (called internally)

`py3plex.algorithms.node_ranking.sparse_page_rank(matrix: spmatrix, start_nodes: List[int] | range, epsilon: float = 1e-06, max_steps: int = 100000, damping: float = 0.5, spread_step: int = 10, spread_percent: float = 0.3, try_shrink: bool = True) → ndarray`

Compute personalized PageRank using sparse matrix operations.

Implements an efficient personalized PageRank algorithm with adaptive sparsification to reduce memory usage and computation time. The algorithm uses a power iteration method with early stopping based on convergence criteria.

Parameters

- **matrix** – Column-stochastic sparse adjacency matrix (use stochastic_normalization first)
- **start_nodes** – List or range of starting nodes for personalized PageRank

- **epsilon** – Convergence threshold for L1 norm difference (default: 1e-6)
- **max_steps** – Maximum number of iterations (default: 100000)
- **damping** – Damping factor / teleportation probability (default: 0.5) Higher values (e.g., 0.85) favor network structure over random jumps
- **spread_step** – Number of steps to check for sparsity pattern (default: 10)
- **spread_percent** – Maximum fraction of nodes to consider for shrinkage (default: 0.3)
- **try_shrink** – Enable adaptive shrinkage to reduce computation (default: True)

Returns

PageRank scores for all nodes, with start_nodes set to 0

Return type

np.ndarray

Notes

- Assumes matrix is column-stochastic (use `stochastic_normalization` first)
- Adaptive shrinkage identifies nodes unreachable from start_nodes and excludes them from computation for efficiency
- Convergence is measured by L1 norm of rank vector difference
- Start nodes are zeroed out in the final result to avoid self-importance

Complexity:

- Time: $O(k * E)$ where k is iterations and E is edges
- Space: $O(N)$ for rank vectors, plus matrix storage

Examples

```
>>> import scipy.sparse as sp
>>> # Create and normalize adjacency matrix
>>> adj = sp.csr_matrix([[0, 1, 1], [1, 0, 1], [1, 1, 0]])
>>> adj_norm = stochastic_normalization(adj)
>>> # Compute PageRank from node 0
>>> pr = sparse_page_rank(adj_norm, [0], damping=0.85)
>>> pr # PageRank scores (node 0 will be 0)
array([0. , 0.5, 0.5])
```

Raises

AssertionError – If start_nodes is empty

See also

`stochastic_normalization`: Required preprocessing step
`run_PPR`: Parallel wrapper for computing multiple PageRank vectors

`py3plex.algorithms.node_ranking.spread_percent_hyper`: float

`py3plex.algorithms.node_ranking.spread_step_hyper`: int

`py3plex.algorithms.node_ranking.stochastic_normalization(matrix: spmatrix) → spmatrix`

Normalize a sparse matrix to stochastic form (column-stochastic).

Converts an adjacency matrix to a stochastic matrix where each column sums to 1. This normalization is required for PageRank-style random walk algorithms.

Parameters

matrix – Sparse adjacency matrix to normalize

Returns

Column-stochastic sparse matrix where each column sums to 1

Return type

`sp.spmatrix`

Notes

- Removes self-loops (sets diagonal to 0) before normalization
- Handles zero-degree nodes by leaving corresponding columns as zeros
- Preserves sparsity structure for memory efficiency

Examples

```
>>> import scipy.sparse as sp
>>> adj = sp.csr_matrix([[0, 1, 1], [1, 0, 1], [1, 1, 0]])
>>> stoch_adj = stochastic_normalization(adj)
>>> stoch_adj.sum(axis=1).A1 # Column sums should be 1
array([1., 1., 1.])
```

`py3plex.algorithms.network_classification.benchmark_classification(matrix: spmatrix, labels: ndarray, alpha_value: float = 0.85, iterations: int = 30, normalization_scheme: str = 'freq', dataset_name: str = 'example', verbose: bool = False, test_size: float | None = None) → DataFrame`

`py3plex.algorithms.network_classification.label_propagation(graph_matrix: spmatrix, class_matrix: ndarray, alpha: float = 0.001, epsilon: float = 1e-12, max_steps: int = 100000, normalization: str | List[str] = 'freq') → ndarray`

Propagate labels through a graph.

Parameters

- **graph_matrix** – Sparse graph adjacency matrix
- **class_matrix** – Initial class label matrix
- **alpha** – Propagation weight parameter
- **epsilon** – Convergence threshold
- **max_steps** – Maximum number of iterations
- **normalization** – Normalization scheme(s) to apply

Returns

Propagated label matrix

`py3plex.algorithms.network_classification.label_propagation_normalization(matrix: spmatrix) → spmatrix`

Normalize a matrix for label propagation.

Parameters

matrix – Sparse matrix to normalize

Returns

Normalized sparse matrix

`py3plex.algorithms.network_classification.label_propagation_tf()` → None

`py3plex.algorithms.network_classification.normalize_amplify_freq(mat: ndarray) → ndarray`

Normalize and amplify matrix by frequency.

Parameters

mat – Matrix to normalize

Returns

Normalized and amplified matrix

`py3plex.algorithms.network_classification.normalize_exp(mat: ndarray) → ndarray`

Apply exponential normalization.

Parameters

mat – Matrix to normalize

Returns

Exponentially normalized matrix

`py3plex.algorithms.network_classification.normalize_initial_matrix_freq(mat: ndarray) → ndarray`

Normalize matrix by frequency.

Parameters

mat – Matrix to normalize

Returns

Normalized matrix

`py3plex.algorithms.network_classification.validate_label_propagation(core_network: spmatrix, labels: ndarray | spmatrix, dataset_name: str = 'test', repetitions: int = 5, normalization_scheme: str | List[str] = 'basic', alpha_value: float = 0.001, random_seed: int = 123, verbose: bool = False) → DataFrame`

Validate label propagation with cross-validation.

Parameters

- **core_network** – Sparse network adjacency matrix
- **labels** – Label matrix

- **dataset_name** – Name of the dataset
- **repetitions** – Number of repetitions
- **normalization_scheme** – Normalization scheme to use
- **alpha_value** – Alpha parameter for propagation
- **random_seed** – Random seed for reproducibility
- **verbose** – Whether to print progress

Returns

DataFrame with validation results

Embeddings and Wrappers

```
py3plex.wrappers.train_node2vec_embedding.call_node2vec_binary(input_graph: str,  
                                                                output_graph: str, p: float =  
                                                                1, q: float = 1, dimension: int  
                                                                = 128, directed: bool = False,  
                                                                weighted: bool = True,  
                                                                binary: str | None = None,  
                                                                timeout: int = 300) → None
```

Call the Node2Vec C++ binary with specified parameters.

Parameters

- **input_graph** – Path to input graph file
- **output_graph** – Path to output embedding file
- **p** – Return parameter
- **q** – In-out parameter
- **dimension** – Embedding dimension
- **directed** – Whether graph is directed
- **weighted** – Whether graph is weighted
- **binary** – Path to node2vec binary (defaults to PY3PLEX_NODE2VEC_BINARY env var or “./node2vec”)
- **timeout** – Maximum execution time in seconds

Raises

ExternalToolError – If binary is not found or execution fails

```
py3plex.wrappers.train_node2vec_embedding.learn_embedding(core_network: Any, labels:  
                                                           List[Any] | None = None, ssize: float  
                                                           = 0.5, embedding_outfile: str =  
                                                           'out.emb', p: float = 0.1, q: float =  
                                                           0.1, binary_path: str | None = None,  
                                                           parameter_range: str =  
                                                           '[0.25,0.50,1,2,4]', timeout: int =  
                                                           300) → Tuple[str, float]
```

Learn node embeddings for a network.

Parameters

- **core_network** – NetworkX graph
- **labels** – Node labels
- **ssize** – Sample size
- **embedding_outfile** – Output file for embeddings

- **p** – Return parameter
- **q** – In-out parameter
- **binary_path** – Path to node2vec binary (defaults to PY3PLEX_NODE2VEC_BINARY env var or “./node2vec”)
- **parameter_range** – String representation of parameter range list
- **timeout** – Maximum execution time in seconds per call

Returns

Tuple of (method_name, elapsed_time)

```
py3plex.wrappers.train_node2vec_embedding.n2v_embedding(G: Any, targets: Any, verbose: bool =
False, sample_size: float = 0.5,
outfile_name: str = 'test.emb', p: float |
None = None, q: float | None = None,
binary_path: str | None = None,
parameter_range: List[float] | None =
None, embedding_dimension: int =
128, timeout: int = 300) → None
```

Train Node2Vec embeddings with parameter optimization.

Parameters

- **G** – NetworkX graph
- **targets** – Target labels for nodes
- **verbose** – Whether to print verbose output
- **sample_size** – Sample size for training
- **outfile_name** – Output embedding file name
- **p** – Return parameter (None triggers grid search)
- **q** – In-out parameter (None triggers grid search)
- **binary_path** – Path to node2vec binary (defaults to PY3PLEX_NODE2VEC_BINARY env var or “./node2vec”)
- **parameter_range** – Range of parameters to search
- **embedding_dimension** – Dimension of embeddings
- **timeout** – Maximum execution time in seconds per call

Network Motifs and Patterns**HINMINE Data Structures**

```
class py3plex.core.HINMINE.dataStructures.Class(lab_id, name, members)
```

Bases: object

```
class py3plex.core.HINMINE.dataStructures.HeterogeneousInformationNetwork(network,
label_delimiter,
weight_tag=False,
tar-
get_tag=True)
```

Bases: object

```
add_label(node, label_id, label_name=None)
```

```
calculate_decomposition_candidates(max_decomposition_length=10)
```

```
calculate_schema()
```

```
create_label_matrix(weights=None)

decompose_from_iterator(name, weighing, summing, generator=None, degrees=None,
                        parallel=False, pool=None)

midpoint_generator(node_sequence, edge_sequence)

process_network(label_delimiter)

split_to_indices(train_indices=(), validate_indices=(), test_indices=())

split_to_parts(lst, n)
```

Command-Line Interface

Command-line interface for py3plex.

This module provides a comprehensive CLI tool for multilayer network analysis with full coverage of main algorithms.

`py3plex.cli.cmd_aggregate(args: Namespace) → int`

Aggregate multilayer network into single layer.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd centrality(args: Namespace) → int`

Compute node centrality measures.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_check(args: Namespace) → int`

Lint and validate a graph data file.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success, non-zero for errors)

`py3plex.cli.cmd_community(args: Namespace) → int`

Detect communities in the network.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_convert(args: Namespace) → int`

Convert network between different formats.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_create(args: Namespace) → int`

Create a new multilayer network.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_help(args: Namespace) → int`

Show detailed help information.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_load(args: Namespace) → int`

Load and inspect a network.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_quickstart(args: Namespace) → int`

Run quickstart demo - creates a tiny demo graph and demonstrates basic operations.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_run_config(args: Namespace) → int`

Run workflow from configuration file.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_selftest(args: Namespace) → int`

Run self-test to verify installation and core functionality.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_stats(args: Namespace) → int`

Compute multilayer network statistics.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.cmd_visualize(args: Namespace) → int`

Visualize the multilayer network.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.create_parser()` → `ArgumentParser`

Create and configure the argument parser for py3plex CLI.

Returns

Configured `ArgumentParser` instance

`py3plex.cli.main(argv: List[str] | None = None)` → `int`

Main entry point for the CLI.

Parameters

argv – Command-line arguments (defaults to `sys.argv`)

Returns

Exit code (0 for success, non-zero for errors)

Validation Utilities

Input validation utilities for Py3plex.

This module provides pre-validation functions to catch common input errors early and provide clear, actionable error messages to users.

`py3plex.validation.validate_csv_columns(file_path: str, required_columns: List[str], optional_columns: List[str] | None = None)` → `None`

Validate that a CSV file has required columns.

Parameters

- **file_path** – Path to CSV file
- **required_columns** – List of column names that must be present
- **optional_columns** – List of column names that are optional

Raises

[`ParsingError`](#) – If required columns are missing

Contracts:

- Precondition: `file_path` must be a non-empty string
- Precondition: `required_columns` must be a non-empty list of strings

`py3plex.validation.validate_edgelist_format(file_path: str, delimiter: str | None = None)` → `None`

Validate simple edgelist file format (source target weight).

Parameters

- **file_path** – Path to edgelist file
- **delimiter** – Optional delimiter (default: whitespace)

Raises

[`ParsingError`](#) – If file format is invalid

Contracts:

- Precondition: `file_path` must be a non-empty string
- Precondition: `delimiter` must be `None` or a string

`py3plex.validation.validate_file_exists(file_path: str) → None`

Validate that a file exists and is readable.

Parameters

file_path – Path to file to validate

Raises

ParsingError – If file doesn't exist or isn't readable

Contracts:

- Precondition: file_path must be a non-empty string

`py3plex.validation.validate_input_type(input_type: str, valid_types: Set[str] | None = None) → None`

Validate that input_type is recognized.

Parameters

- **input_type** – The input type string to validate
- **valid_types** – Optional set of valid types (uses default if None)

Raises

ParsingError – If input_type is not valid

Contracts:

- Precondition: input_type must be a non-empty string
- Precondition: valid_types must be None or a set

`py3plex.validation.validate_multiedgelist_format(file_path: str, delimiter: str | None = None) → None`

Validate multiedgelist file format (source target layer weight).

Parameters

- **file_path** – Path to multiedgelist file
- **delimiter** – Optional delimiter (default: whitespace)

Raises

ParsingError – If file format is invalid

Contracts:

- Precondition: file_path must be a non-empty string
- Precondition: delimiter must be None or a string

`py3plex.validation.validate_network_data(file_path: str, input_type: str) → None`

Validate network data before parsing.

This is the main entry point for validation. It performs appropriate validation based on the input type.

Parameters

- **file_path** – Path to network file
- **input_type** – Type of input file

Raises

ParsingError – If validation fails

Contracts:

- Precondition: file_path must be a string

- Precondition: `input_type` must be a non-empty string

Network Comparison and Testing

Hedwig Learning Algorithms

Main learner class.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.learners.learner.Learner(kb, n=None, min_sup=1, sim=1,  
depth=4, target=None,  
use_negations=False,  
optimal_subclass=False)
```

Bases: `object`

Learner class, supporting various types of induction from the knowledge base.

Default = `'default'`

Improvement = `'improvement'`

Similarity = `'similarity'`

can_specialize(*rule*)

Is the rule good enough to be further refined?

extend(*rules, specializations*)

Extends the ruleset in the given way.

extend_replace_worst(*rules, specializations*)

Extends the list by replacing the worst rules.

extend_with_similarity(*rules, specializations*)

Extends the list based on how similar is 'new_rule' to the rules contained in 'rules'.

get_subclasses(*pred*)

get_superclasses(*pred*)

group_score(*rules*)

Calculates the score of the whole list of rules.

induce()

Induces rules for the given knowledge base.

is_implicit_root(*pred*)

non_redundant(*rule, new_rule*)

Is the rule non-redundant compared to its immediate generalization?

specialize(*rule*)

Returns a list of all specializations of 'rule'.

specialize_add_relation(*rule*)

Specialize with new binary relation.

Main learner class.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.learners.bottomup.BottomUpLearner(kb, n=None, min_sup=1,  
sim=1, depth=4,  
target=None,  
use_negations=False)
```


Bases: `object`

Bottom-up learner.

Default = `'default'`

Improvement = `'improvement'`

Similarity = `'similarity'`

bottom_clause()

get_subclasses()*(pred)*

get_superclasses()*(pred)*

induce()

Induces rules for the given knowledge base.

is_implicit_root()*(pred)*

Main learner class.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.learners.optimal.OptimalLearner(kb, n=None, min_sup=1,
                                                                sim=1, depth=4,
                                                                target=None,
                                                                use_negations=False,
                                                                optimal_subclass=True)
```

Bases: `Learner`

Finds the optimal top-k rules.

induce()

Induces rules for the given knowledge base.

Hedwig Statistics and Scoring

Score function definitions.

@author: anze.vavpetic@ijs.si

`py3plex.algorithms.hedwig.stats.scorefunctions.chisq(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.enrichment_score(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.interesting(rule)`

Checks if a given rule is interesting for the given score function

`py3plex.algorithms.hedwig.stats.scorefunctions.kaplan_meier_AUC(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.leverage(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.lift(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.precision(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.t_score(rule)`

`py3plex.algorithms.hedwig.stats.scorefunctions.wracc(rule)`

```
py3plex.algorithms.hedwig.stats.scorefunctions.z_score(rule)
```

Module for ruleset validation.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.stats.validate.Validate(kb, significance_test=<function
                                                    apply_fisher>, adjustment=<function
                                                    fdr>)
```

Bases: object

```
test(ruleset, alpha=0.05, q=0.01)
```

Tests the given ruleset and returns the significant rules.

Hedwig Core Components

```
py3plex.algorithms.hedwig.core.converters.convert_mapping_to_rdf(input_mapping_file, ex-
                                                                tract_subnode_info=False,
                                                                split_node_by=':',
                                                                keep_index=1,
                                                                layer_type='uniprotkb',
                                                                annota-
                                                                tion_mapping_file='test.gaf',
                                                                go_identifier='GO:',
                                                                prepend_string=None)
```

```
py3plex.algorithms.hedwig.core.converters.obo2n3(obofile, n3out, gaf_file)
```

Knowledge-base class.

@author: anze.vavpetic@ijs.si

```
class py3plex.algorithms.hedwig.core.kb.ExperimentKB(triplets, score_fun,
                                                    instances_as_leaves=True)
```

Bases: object

The knowledge base for one specific experiment.

```
add_sub_class(sub, obj)
```

Adds the resource 'sub' as a subclass of 'obj'.

```
bits_to_indices(bits)
```

Converts the bitset to a set of indices.

```
get_domains(predicate)
```

Returns the bitsets for input and outputexamples of the binary predicate 'predicate'.

```
get_empty_domain()
```

Returns a bitset covering no examples.

```
get_examples()
```

Returns all examples for this experiment.

```
get_full_domain()
```

Returns a bitset covering all examples.

```
get_members(predicate, bit=True)
```

Returns the examples for this predicate, either as a bitset or a set of ids.

```
get_reverse_members(predicate, bit=True)
```

Returns the examples for this predicate, either as a bitset or a set of ids.

get_root()

Root predicate, which covers all examples.

get_score(*ex_idx*)

Returns the score for example id 'ex_idx'.

get_subclasses(*predicate*, *producer_pred=None*)

Returns a list of subclasses (as predicate objects) for 'predicate'.

indices_to_bits(*indices*)

Converts the indices to a bitset.

is_discrete_target()

n_examples()

Returns the number of examples.

n_members(*predicate*)

super_classes(*pred*)

Returns all super classes of pred (with transitivity).

Global settings file.

@author: anze.vavpetic@ijs.si

class py3plex.algorithms.hedwig.core.settings.Defaults

Bases: object

ADJUST = 'fwer'

ALPHA = 0.05

BEAM_SIZE = 20

COVERED = None

DEPTH = 5

FDR_Q = 0.05

FORMAT = 'n3'

LEARNER = 'heuristic'

LEAVES = False

MODE = 'subgroups'

NEGATIONS = False

NO_CACHE = False

OPTIMAL_SUBCLASS = False

OUTPUT = None

SCORE = 'lift'

SUPPORT = 0.1

TARGET = None

URIS = False

VERBOSE = False

Time Series and Temporal Analysis

Advanced Visualization

Sankey diagram visualization for multilayer networks.

This module provides inter-layer flow visualization to show connection strength between layers in multilayer networks. The visualization displays flows as arrows with widths proportional to the number of inter-layer connections.

Note: This uses a simplified flow diagram approach rather than matplotlib's Sankey class, as the Sankey class is designed for more complex flow networks and doesn't map directly to multilayer network inter-layer connections.

```
py3plex.visualization.sankey.draw_multilayer_sankey(graphs: List[Graph], multilinks: Dict[str,
List[Tuple]], labels: List[str] | None =
None, ax: Any | None = None, display: bool
= True, **kwargs) → Any
```

Draw inter-layer flow diagram showing connection strength in multilayer networks.

Creates a flow visualization where: - Each layer is represented in the diagram - Flows between layers show the strength (number) of inter-layer connections - Flow width/text indicates the number of inter-layer edges

Parameters

- **graphs** – List of NetworkX graphs, one per layer
- **multilinks** – Dictionary mapping edge_type -> list of multi-layer edges
- **labels** – Optional list of layer labels. If None, uses layer indices
- **ax** – Matplotlib axes to draw on. If None, creates new figure
- **display** – If True, calls plt.show() at the end
- ****kwargs** – Reserved for future extensions

Returns

Matplotlib axes object

Examples

```
>>> network = multi_layer_network()
>>> network.load_network("data.txt", input_type="multiedgelist")
>>> labels, graphs, multilinks = network.get_layers()
>>> draw_multilayer_sankey(graphs, multilinks, labels=labels)
```

Note

This visualization is most effective for networks with 2-5 layers. For networks with many layers, the diagram may become cluttered. The implementation uses a simplified flow visualization approach.

Link Prediction

8.8.4 Citation and References

If you use py3plex in your research, please **cite our work**.

Primary Citation

Recommended citation for py3plex:

```
@Article{Skrlj2019,
  author    = {{\v{S}}krlj, Bla{\v{z}} and Kralj, Jan and Lavra{\v{c}}, Nada},
  title     = {Py3plex toolkit for visualization and analysis of multilayer networks},
```

(continues on next page)

(continued from previous page)

```

journal = {Applied Network Science},
year    = {2019},
volume  = {4},
number  = {1},
pages   = {94},
doi     = {10.1007/s41109-019-0203-7},
url     = {https://doi.org/10.1007/s41109-019-0203-7}
}

```

Plain text:

Škrlić, B., Kralj, J., & Lavrač, N. (2019). Py3plex toolkit for visualization and analysis of multilayer networks. *Applied Network Science*, 4(1), 94. <https://doi.org/10.1007/s41109-019-0203-7>

Conference Paper

For the **initial algorithmic work**:

```

@InProceedings{Škrlić2019Conference,
  author    = {{\v{S}}krlić, Bla{\v{z}} and Kralj, Jan and Lavrač{\v{c}}, Nada},
  editor    = {Aiello, Luca Maria and Cherifi, Chantal and Cherifi, Hocine
               and Lambiotte, Renaud and Li{\v{o}}, Pietro and Rocha, Luis M.},
  title     = {Py3plex: A Library for Scalable Multilayer Network Analysis and
  ↪Visualization},
  booktitle = {Complex Networks and Their Applications VII},
  year      = {2019},
  publisher = {Springer International Publishing},
  address   = {Cham},
  pages     = {757--768},
  isbn      = {978-3-030-05411-3},
  doi       = {10.1007/978-3-030-05411-3_60}
}

```

Algorithm-Specific Citations

When using **specific algorithms**, please also cite the **original papers**:

Multilayer Modularity

```

@Article{Mucha2010,
  author    = {Mucha, Peter J. and Richardson, Thomas and Macon, Kevin
               and Porter, Mason A. and Onnela, Jukka-Pekka},
  title     = {Community structure in time-dependent, multiscale, and multiplex
  ↪networks},
  journal   = {Science},
  year      = {2010},
  volume    = {328},
  number    = {5980},
  pages     = {876--878},
  doi       = {10.1126/science.1184819}
}

```

Node2Vec

```

@InProceedings{Grover2016,
  author    = {Grover, Aditya and Leskovec, Jure},

```

(continues on next page)

(continued from previous page)

```
title      = {node2vec: Scalable Feature Learning for Networks},
booktitle  = {Proceedings of the 22nd ACM SIGKDD International Conference
              on Knowledge Discovery and Data Mining},
year       = {2016},
pages      = {855--864},
doi        = {10.1145/2939672.2939754}
}
```

Louvain Algorithm

```
@Article{Blondel2008,
  author = {Blondel, Vincent D. and Guillaume, Jean-Loup and Lambiotte, Renaud
            and Lefebvre, Etienne},
  title  = {Fast unfolding of communities in large networks},
  journal = {Journal of Statistical Mechanics: Theory and Experiment},
  year   = {2008},
  volume = {2008},
  number = {10},
  pages  = {P10008},
  doi    = {10.1088/1742-5468/2008/10/P10008}
}
```

Infomap

```
@Article{Rosvall2008,
  author = {Rosvall, Martin and Bergstrom, Carl T.},
  title  = {Maps of random walks on complex networks reveal community structure},
  journal = {Proceedings of the National Academy of Sciences},
  year   = {2008},
  volume = {105},
  number = {4},
  pages  = {1118--1123},
  doi    = {10.1073/pnas.0706851105}
}
```

MultiXRank

```
@Article{Baptista2022,
  author = {Baptista, Anthony and Gonzalez, Aur{\`e}lien and Baudot, Ana{\`{i}s},
  title  = {Universal multilayer network exploration by random walk with restart},
  journal = {Communications Physics},
  year   = {2022},
  volume = {5},
  number = {1},
  pages  = {170},
  doi    = {10.1038/s42005-022-00937-9}
}
```

DeepWalk

```
@InProceedings{Perozzi2014,
  author = {Perozzi, Bryan and Al-Rfou, Rami and Skiena, Steven},
  title  = {DeepWalk: Online Learning of Social Representations},
```

(continues on next page)

(continued from previous page)

```

booktitle = {Proceedings of the 20th ACM SIGKDD International Conference
             on Knowledge Discovery and Data Mining},
year       = {2014},
pages      = {701--710},
doi        = {10.1145/2623330.2623732}
}

```

Multilayer Network Theory

Foundational papers on multilayer networks:

```

@Article{Kivela2014,
  author = {Kivel{\`a}, Mikko and Arenas, Alex and Barthelemy, Marc
            and Gleeson, James P. and Moreno, Yamir and Porter, Mason A.},
  title  = {Multilayer networks},
  journal = {Journal of Complex Networks},
  year   = {2014},
  volume = {2},
  number = {3},
  pages  = {203--271},
  doi    = {10.1093/comnet/cnu016}
}

```

```

@Article{Boccaletti2014,
  author = {Boccaletti, Stefano and Bianconi, Ginestra and Criado, Regino
            and del Genio, Charo I. and G{\`o}mez-Garde{\`n}es, Jes{\`u}s
            and Romance, Miguel and Sendi{\`n}a-Nadal, Irene and Wang, Zhen
            and Zanin, Massimiliano},
  title  = {The structure and dynamics of multilayer networks},
  journal = {Physics Reports},
  year   = {2014},
  volume = {544},
  number = {1},
  pages  = {1--122},
  doi    = {10.1016/j.physrep.2014.07.001}
}

```

```

@Article{DeDomenico2013,
  author = {De Domenico, Manlio and Sol{\`e}-Ribalta, Albert and Cozzo, Emanuele
            and Kivel{\`a}, Mikko and Moreno, Yamir and Porter, Mason A.
            and G{\`o}mez, Sergio and Arenas, Alex},
  title  = {Mathematical formulation of multilayer networks},
  journal = {Physical Review X},
  year   = {2013},
  volume = {3},
  number = {4},
  pages  = {041022},
  doi    = {10.1103/PhysRevX.3.041022}
}

```

Complete Citation List

For algorithm citations, see the `algorithm_guide` which includes all major algorithms with their original references and DOIs.

Acknowledgments

Development and Contributors

py3plex is developed and maintained by:

- **Blaž Škrlj** (lead developer) - Jožef Stefan Institute, Ljubljana, Slovenia
- **Jan Kralj** (contributor) - Jožef Stefan Institute
- **Nada Lavrač** (contributor) - Jožef Stefan Institute

Funding and Support

This work has been **supported by**:

- **Jožef Stefan Institute**, Ljubljana, Slovenia
- **Slovenian Research Agency** (research program P2-0103)

External Libraries

py3plex builds upon **excellent open-source libraries**:

- **NetworkX** - Graph data structures and algorithms
- **NumPy** - Numerical computing
- **SciPy** - Scientific computing
- **Matplotlib** - Visualization
- **scikit-learn** - Machine learning utilities

Community Acknowledgments

We thank all contributors, users, and the broader network science community for their support, feedback, and contributions.

Special thanks to contributors who have submitted pull requests, reported issues, or provided feedback.

License Information

py3plex is released under the **MIT License**.

MIT License

Copyright (c) 2019–2025 Blaž Škrlj, Jan Kralj, Nada Lavrač

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,

(continues on next page)

(continued from previous page)

OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Note: Some bundled algorithms may have **different licenses**:

- Infomap community detection code: **AGPLv3**
- Louvain community detection: **BSD-3-Clause**

See LICENSE file in the repository for detailed license compatibility information.

Contact

For questions about citing py3plex or collaboration opportunities:

- **Email:** blaz.skrlj@ijs.si
- **GitHub:** <https://github.com/SkBlaz/py3plex>
- **Issues:** <https://github.com/SkBlaz/py3plex/issues>

Related Work

Other tools and libraries for multilayer network analysis:

- **muxViz** - Multilayer network visualization (R)
- **pymnet** - Multilayer networks in Python
- **multinet** - R package for multilayer networks
- **graphistry** - GPU-accelerated graph visualization

For a comprehensive comparison and positioning of py3plex, see the original publication.

Usage in Publications

If you've used py3plex in your research and would like to be listed here, please:

1. Open an issue or pull request
2. Provide citation to your paper
3. Brief description of how py3plex was used

We maintain a list of publications using py3plex to showcase applications and build community.

See Also

- **acknowledgements** - Full acknowledgments
- **algorithm_guide** - Complete algorithm citations with references
- **py3plex GitHub** - Source code and issues
- **Applied Network Science paper** - Primary publication

EXAMPLES

The best way to learn py3plex is through examples.

All examples are available in the [examples/ directory](#).

Key examples:

- `tutorial_10min.py` - Executable version of the 10-minute tutorial
- `example_dsl_queries.py` - SQL-like DSL for network queries
- `example_dsl_advanced.py` - Advanced DSL queries and analysis
- `example_random_walks.py` - Random walk primitives (Node2Vec, DeepWalk)
- `example_multilayer_visualization.py` - Network visualization
- `example_community_detection.py` - Community detection with Louvain and Infomap
- `example_network_decomposition.py` - Meta-path feature extraction
- `example_multilayer_statistics.py` - Computing multilayer metrics
- `example_n2v_embedding.py` - Node2Vec embeddings
- `example_label_propagation.py` - Semi-supervised learning

CITATION

If you use py3plex in your research, please cite:

```
@Article{Škrlj2019,  
  author={Škrlj, Blaž and Kralj, Jan and Lavrač, Nada},  
  title={Py3plex toolkit for visualization and analysis of multilayer networks},  
  journal={Applied Network Science},  
  year={2019},  
  volume={4},  
  number={1},  
  pages={94},  
  doi={10.1007/s41109-019-0203-7}  
}
```


INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

p

[py3plex](#), 474
[py3plex.algorithms](#), 474
[py3plex.algorithms.community_detection](#), 419
[py3plex.algorithms.community_detection.community_detection](#), 419
[py3plex.algorithms.community_detection.community_detection](#), 423
[py3plex.algorithms.community_detection.community_ranking](#), 482
[py3plex.algorithms.community_detection.community_ranking](#), 526
[py3plex.algorithms.community_detection.community_wrapper](#), 413
[py3plex.algorithms.community_detection.multilayer_benchmark](#), 424
[py3plex.algorithms.community_detection.multilayer_modularity](#), 415
[py3plex.algorithms.community_detection.NoRC](#), 526
[py3plex.algorithms.general.benchmark_classification](#), 482
[py3plex.algorithms.general.walkers](#), 478
[py3plex.algorithms.hedwig](#), 519
[py3plex.algorithms.hedwig.core.converters](#), 542
[py3plex.algorithms.hedwig.core.example](#), 519
[py3plex.algorithms.hedwig.core.kb](#), 542
[py3plex.algorithms.hedwig.core.predicate](#), 520
[py3plex.algorithms.hedwig.core.rule](#), 520
[py3plex.algorithms.hedwig.core.settings](#), 543
[py3plex.algorithms.hedwig.learners.bottomup](#), 540
[py3plex.algorithms.hedwig.learners.learner](#), 540
[py3plex.algorithms.hedwig.learners.optimal](#), 541
[py3plex.algorithms.hedwig.stats.scorefunctions](#), 541
[py3plex.algorithms.hedwig.stats.validate](#), 542
[py3plex.algorithms.multicentrality](#), 477
[py3plex.algorithms.multilayer_algorithms](#), 454
[py3plex.algorithms.multilayer_algorithms.entanglement](#), 473
[py3plex.algorithms.multilayer_algorithms.multixrank](#), 469
[py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality](#), 474
[py3plex.algorithms.network_classification](#), 532
[py3plex.algorithms.node_ranking](#), 527
[py3plex.algorithms.node_ranking.node_ranking](#), 482
[py3plex.algorithms.statistics.basic_statistics](#), 453
[py3plex.algorithms.statistics.bayesian_tests](#), 523
[py3plex.algorithms.statistics.correlation_networks](#), 453
[py3plex.algorithms.statistics.enrichment](#), 453
[py3plex.algorithms.statistics.multilayer_statistics](#), 430
[py3plex.algorithms.statistics.topology](#), 452
[py3plex.cli](#), 536
[py3plex.config](#), 402
[py3plex.core.converters](#), 395
[py3plex.core.HINMINE.dataStructures](#), 535
[py3plex.core.HINMINE.decomposition](#), 400
[py3plex.core.HINMINE.IO](#), 402
[py3plex.core.multinet](#), 366
[py3plex.core.nx_compat](#), 399
[py3plex.core.parsers](#), 389
[py3plex.core.random_generators](#), 396
[py3plex.core.supporting](#), 397
[py3plex.dsl](#), 408
[py3plex.exceptions](#), 407
[py3plex.io.api](#), 517
[py3plex.logging_config](#), 513
[py3plex.multinet.aggregation](#), 509
[py3plex.profilng](#), 510
[py3plex.utils](#), 403
[py3plex.validation](#), 538
[py3plex.visualization.benchmark_visualizations](#), 506
[py3plex.visualization.bezier](#), 505
[py3plex.visualization.colors](#), 503
[py3plex.visualization.drawing_machinery](#), 495
[py3plex.visualization.embedding_visualization.embedding](#), 522
[py3plex.visualization.fa2.fa2util](#), 521
[py3plex.visualization.fa2_forceatlas2](#), 521

`py3plex.visualization.layout_algorithms,`
 [504](#)
`py3plex.visualization.multilayer,` [487](#)
`py3plex.visualization.sankey,` [544](#)
`py3plex.wrappers.benchmark_nodes,` [506](#)
`py3plex.wrappers.train_node2vec_embedding,`
 [534](#)

A

- `accessibility_centrality()`
(`py3plex.algorithms.multilayer_algorithms.centrality.MultilayerCentrality`
method), 454
- `add_bipartite_block()`
(`py3plex.algorithms.multilayer_algorithms.multirank.Multirank`
method), 470
- `add_dummy_layers()`
(`py3plex.core.multinet.multi_layer_network`
method), 368
- `add_edges()` (`py3plex.core.multinet.multi_layer_network`
method), 368
- `add_label()` (`py3plex.core.HINMINE.dataStructures.HeterogeneousInformationNetwork`
method), 535
- `add_mpx_edges()` (in module
`py3plex.core.supporting`), 397
- `add_multiplex()` (`py3plex.algorithms.multilayer_algorithms.multirank.Multirank`
method), 470
- `add_nodes()` (`py3plex.core.multinet.multi_layer_network`
method), 369
- `add_sub_class()` (`py3plex.algorithms.hedwig.core.kb.ExperimentKB`
method), 542
- `ADJUST` (`py3plex.algorithms.hedwig.core.settings.Defaults`
attribute), 543
- `adjustSpeedAndApplyForces()` (in module
`py3plex.visualization.fa2.fa2util`), 521
- `aggregate_edges()` (`py3plex.core.multinet.multi_layer_network`
method), 371
- `aggregate_layers()` (in module
`py3plex.multinet.aggregation`), 509
- `aggregate_scores()`
(`py3plex.algorithms.multilayer_algorithms.multirank.Multirank`
method), 470
- `aggregate_sum()` (in module
`py3plex.core.HINMINE.decomposition`),
400
- `aggregate_to_node_level()`
(`py3plex.algorithms.multilayer_algorithms.centrality.MultilayerCentrality`
method), 455
- `aggregate_weighted_sum()` (in module
`py3plex.core.HINMINE.decomposition`),
400
- `algebraic_connectivity()` (in module
`py3plex.algorithms.statistics.multilayer_statistics`),
430
- `AlgorithmError`, 407
- `alias` (`py3plex.dsl.EdgePattern` attribute), 409, 410
- `alias` (`py3plex.dsl.NodePattern` attribute), 410, 411
- `ALPHA` (`py3plex.algorithms.hedwig.core.settings.Defaults`
attribute), 543
- `apply_attraction()` (in module
`py3plex.visualization.fa2.fa2util`), 521
- `apply_gravity()` (in module
`py3plex.visualization.fa2.fa2util`), 522
- `apply_repulsion()` (in module
`py3plex.visualization.fa2.fa2util`), 522
- `applyForce()` (`py3plex.visualization.fa2.fa2util.Region`
method), 521
- `applyForceOnNodes()`
(`py3plex.visualization.fa2.fa2util.Region`
method), 521
- `authority_matrix()` (in module
`py3plex.algorithms.node_ranking`), 527
- `authority_matrix()` (in module
`py3plex.algorithms.node_ranking.node_ranking`),
482
- `basic_pl_stats()` (in module
`py3plex.algorithms.statistics.topology`),
452
- `basic_random_walk()` (in module
`py3plex.algorithms.general.walkers`), 478
- `basic_stats()` (`py3plex.core.multinet.multi_layer_network`
method), 371
- `BEAM_SIZE` (`py3plex.algorithms.hedwig.core.settings.Defaults`
attribute), 543
- `benchmark()` (in module `py3plex.profiling`), 511
- `benchmark_classification()` (in module
`py3plex.algorithms.network_classification`),
532
- `benchmark_node_classification()` (in module
`py3plex.wrappers.benchmark_nodes`), 508
- `best_partition()` (in module
`py3plex.algorithms.community_detection.community_louvain`),
419
- `bezier_calculate_dfy()` (in module
`py3plex.visualization.bezier`), 505
- `BinaryPredicate` (class in
`py3plex.algorithms.hedwig.core.predicate`),
520
- `bipartite_blocks` (`py3plex.algorithms.multilayer_algorithms.multirank`
attribute), 469

bits_to_indices() (py3plex.algorithms.hedwig.core.kb.ExperimentKB), 535
 method), 542
 call_node2vec_binary() (in module
 bottom_clause() (py3plex.algorithms.hedwig.learners.bottomup.Py3plexLearner.train_node2vec_embedding),
 method), 541
 BottomUpLearner (class in can_specialize() (py3plex.algorithms.hedwig.learners.learner.Learner
 py3plex.algorithms.hedwig.learners.bottomup), method), 540
 540
 CentralityComputationError, 407
 bridging centrality() (in module
 (py3plex.algorithms.multilayer_algorithms.centralities.MultilayerCentralities.HINMINE.decomposition),
 method), 455
 401
 build_graph() (in module
 py3plex.algorithms.hedwig), 519
 chisq() (in module
 py3plex.algorithms.hedwig.stats.scorefunctions),
 build_occurrence_matrix() (in module
 541
 py3plex.algorithms.multilayer_algorithms.entanglement.Class in py3plex.core.HINMINE.dataStructures),
 473
 535
 build_supra_heterogeneous_matrix() ClassLabeled (py3plex.algorithms.hedwig.core.example.Example
 (py3plex.algorithms.multilayer_algorithms.multirank.Multirank), 520
 method), 471
 clear() (py3plex.profilig.PerformanceMonitor
 build_supra_modularity_matrix() (in module
 method), 510
 py3plex.algorithms.community_detection.multilayer_clone(clone) (py3plex.algorithms.hedwig.core.rule.Rule
 415
 method), 520
 buildSubRegions() (py3plex.visualization.fa2.fa2util.Region.append() (py3plex.algorithms.hedwig.core.rule.Rule
 method), 521
 method), 520
 clone_negate() (py3plex.algorithms.hedwig.core.rule.Rule
 method), 520
 clone_swap_with_subclass() (py3plex.algorithms.hedwig.core.rule.Rule
 method), 520
 calculate_decomposition_candidates() (py3plex.core.HINMINE.dataStructures.HeterogeneousInformationNetwork
 method), 535
 calculate_importance_chi() (in module
 py3plex.core.HINMINE.decomposition),
 400
 calculate_importance_delta() (in module
 py3plex.core.HINMINE.decomposition), 401
 calculate_importance_gr() (in module
 py3plex.core.HINMINE.decomposition),
 401
 calculate_importance_idf() (in module
 py3plex.core.HINMINE.decomposition),
 401
 calculate_importance_ig() (in module
 py3plex.core.HINMINE.decomposition),
 401
 calculate_importance_okapi() (in module
 py3plex.core.HINMINE.decomposition), 401
 calculate_importance_rf() (in module
 py3plex.core.HINMINE.decomposition),
 401
 calculate_importance_tf() (in module
 py3plex.core.HINMINE.decomposition),
 401
 calculate_importance_w2w() (in module
 py3plex.core.HINMINE.decomposition),
 401
 calculate_importances() (in module
 py3plex.core.HINMINE.decomposition),
 401
 calculate_schema() (py3plex.core.HINMINE.dataStructures.HeterogeneousInformationNetwork
 method), 674

[illegible]

`draw_multilayer_default()` (in module `py3plex.visualization.multilayer`), 487
`draw_multilayer_flow()` (in module `py3plex.visualization.multilayer`), 488
`draw_multilayer_sankey()` (in module `py3plex.visualization.sankey`), 544
`draw_networkx()` (in module `py3plex.visualization.drawing_machinery`), 496
`draw_networkx_edge_labels()` (in module `py3plex.visualization.drawing_machinery`), 498
`draw_networkx_edges()` (in module `py3plex.visualization.drawing_machinery`), 499
`draw_networkx_labels()` (in module `py3plex.visualization.drawing_machinery`), 500
`draw_networkx_nodes()` (in module `py3plex.visualization.drawing_machinery`), 501
`draw_random()` (in module `py3plex.visualization.drawing_machinery`), 502
`draw_shell()` (in module `py3plex.visualization.drawing_machinery`), 502
`draw_spectral()` (in module `py3plex.visualization.drawing_machinery`), 503
`draw_spring()` (in module `py3plex.visualization.drawing_machinery`), 503
`DSLExecutionError`, 409
`DSLSyntaxError`, 409
E
`Edge` (class in `py3plex.visualization.fa2.fa2util`), 521
`edge_betweenness_centrality()` (`py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality` method), 456
`edge_count` (`py3plex.core.multinet.multi_layer_network` property), 373
`edge_overlap()` (in module `py3plex.algorithms.statistics.multilayer_statistics`), 435
`EdgePattern` (class in `py3plex.dsl`), 409
`edges` (`py3plex.dsl.PathPattern` attribute), 411
`edges_from_temporal_table()` (`py3plex.core.multinet.multi_layer_network` method), 373
`EmbeddingError`, 407
`enabled` (`py3plex.profiling.PerformanceMonitor` attribute), 510
`enrichment_score()` (in module `py3plex.algorithms.hedwig.stats.scorefunctions`), 541
`ensure()` (in module `py3plex.algorithms.multicentrality`), 477
`ensure()` (in module `py3plex.algorithms.statistics.basic_statistics`), 453
`ensure()` (in module `py3plex.core.converters`), 395
`ensure()` (in module `py3plex.core.multinet`), 366
`ensure()` (in module `py3plex.core.nx_compat`), 399
`ensure()` (in module `py3plex.core.parsers`), 389
`ensure()` (in module `py3plex.core.random_generators`), 396
`ensure()` (in module `py3plex.core.supporting`), 398
`ensure()` (in module `py3plex.io.api`), 517
`ensure()` (in module `py3plex.multinet.aggregation`), 510
`ensure()` (in module `py3plex.utils`), 403
`ensure()` (in module `py3plex.visualization.layout_algorithms`), 505
`entropy_of_multiplexity()` (in module `py3plex.algorithms.statistics.multilayer_statistics`), 435
`epsilon` (`py3plex.algorithms.multilayer_algorithms.multixrank.MultiXR` attribute), 470
`evaluate_oracle_F1()` (in module `py3plex.algorithms.general.benchmark_classification`), 482
`Example` (class in `py3plex.algorithms.hedwig.core.example`), 519
`examples()` (`py3plex.algorithms.hedwig.core.rule.Rule` method), 520
`execute_query()` (in module `py3plex.dsl`), 411
`execute_query()` (`py3plex.core.multinet.multi_layer_network` method), 374
`ExperimentKB` (class in `py3plex.algorithms.hedwig.core.kb`), 542
`extend()` (`py3plex.algorithms.hedwig.learners.learner.Learner` method), 540
`extend_replace_worst()` (`py3plex.algorithms.hedwig.learners.learner.Learner` method), 540
`extend_with_similarity()` (`py3plex.algorithms.hedwig.learners.learner.Learner` method), 540
`ExternalToolError`, 407
F
`FDR_Q` (`py3plex.algorithms.hedwig.core.settings.Defaults` attribute), 543
`fill_tmp_with_edges()` (`py3plex.core.multinet.multi_layer_network` method), 375
`flow_betweenness_centrality()` (`py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality` method), 457
`ForceAtlas2` (class in `py3plex.visualization.fa2.forceatlas2`), 521

`forceatlas2()` (`py3plex.visualization.fa2.forceatlas2.ForceAtlas2` attribute), 521
`forceatlas2_igraph_layout()` (`py3plex.visualization.fa2.forceatlas2.ForceAtlas2` method), 521
`forceatlas2_networkx_layout()` (`py3plex.visualization.fa2.forceatlas2.ForceAtlas2` method), 521
`FORMAT` (`py3plex.algorithms.hedwig.core.settings.Defaults` attribute), 543
`format_result()` (in module `py3plex.dsl`), 412
`from_edges()` (`py3plex.core.multinet.multi_layer_network` class method), 376
`from_homogeneous_hypergraph()` (`py3plex.core.multinet.multi_layer_network` method), 376
`from_networkx()` (`py3plex.core.multinet.multi_layer_network` class method), 377
G
`gdegrees` (`py3plex.algorithms.community_detection.community_lowrank_utils` attribute), 419
`general_random_walk()` (in module `py3plex.algorithms.general.walkers`), 479
`generate_coupled_er_multilayer()` (in module `py3plex.algorithms.community_detection.multilayer_benchmark`), 424
`generate_dendrogram()` (in module `py3plex.algorithms.community_detection.community_lowrank_utils`), 420
`generate_multilayer_lfr()` (in module `py3plex.algorithms.community_detection.multilayer_benchmark`), 426
`generate_random_multiedges()` (in module `py3plex.visualization.multilayer`), 489
`generate_random_networks()` (in module `py3plex.visualization.multilayer`), 490
`generate_rules_report()` (in module `py3plex.algorithms.hedwig`), 519
`generate_sbm_multilayer()` (in module `py3plex.algorithms.community_detection.multilayer_benchmark`), 429
`generate_walks()` (in module `py3plex.algorithms.general.walkers`), 479
`generic_grouping()` (in module `py3plex.visualization.benchmark_visualizations`), 506
`get_aggregation_method()` (in module `py3plex.core.HINMINE.decomposition`), 401
`get_background_knowledge_dir()` (in module `py3plex.utils`), 403
`get_background_knowledge_path()` (in module `py3plex.utils`), 403
`get_calculation_method()` (in module `py3plex.core.HINMINE.decomposition`), 402
`get_color_palette()` (in module `py3plex.config`), 402
`get_data_path()` (in module `py3plex.utils`), 404
`get_dataset_path()` (in module `py3plex.utils`), 404
`get_decomposition()` (`py3plex.core.multinet.multi_layer_network` method), 377
`get_decomposition_cycles()` (`py3plex.core.multinet.multi_layer_network` method), 377
`get_degrees()` (`py3plex.core.multinet.multi_layer_network` method), 377
`get_domains()` (`py3plex.algorithms.hedwig.core.kb.ExperimentKB` method), 542
`get_edges()` (`py3plex.core.multinet.multi_layer_network` method), 377
`get_empty_domain()` (`py3plex.algorithms.hedwig.core.kb.ExperimentKB` method), 542
`get_example_image_path()` (in module `py3plex.utils`), 405
`get_examples()` (`py3plex.algorithms.hedwig.core.kb.ExperimentKB` method), 542
`get_full_domain()` (`py3plex.algorithms.hedwig.core.kb.ExperimentKB` method), 542
`get_label_matrix()` (`py3plex.core.multinet.multi_layer_network` method), 378
`get_layers()` (`py3plex.core.multinet.multi_layer_network` method), 378
`get_logger()` (in module `py3plex.logging_config`), 513
`get_members()` (`py3plex.algorithms.hedwig.core.kb.ExperimentKB` method), 542
`get_monitor()` (in module `py3plex.profiling`), 511
`get_multilayer_dataset_path()` (in module `py3plex.utils`), 405
`get_neighbors()` (`py3plex.core.multinet.multi_layer_network` method), 378
`get_nodes()` (`py3plex.core.multinet.multi_layer_network` method), 378
`get_nx_object()` (`py3plex.core.multinet.multi_layer_network` method), 379
`get_report()` (`py3plex.profiling.PerformanceMonitor` method), 510
`get_reverse_members()` (`py3plex.algorithms.hedwig.core.kb.ExperimentKB` method), 542
`get_rng()` (in module `py3plex.utils`), 405
`get_root()` (`py3plex.algorithms.hedwig.core.kb.ExperimentKB` method), 542
`get_score()` (`py3plex.algorithms.hedwig.core.kb.ExperimentKB` method), 543
`get_subclasses()` (`py3plex.algorithms.hedwig.core.kb.ExperimentKB` method), 543
`get_subclasses()` (`py3plex.algorithms.hedwig.learners.bottomup.BottomupLearner` method), 541
`get_subclasses()` (`py3plex.algorithms.hedwig.learners.learner.Learner` method), 541

method), 540
get_superclasses() (py3plex.algorithms.hedwig.learners.bottomup.BottomUpLearner method), 541
get_superclasses() i (py3plex.algorithms.hedwig.core.predicate.Predicate attribute), 520
method), 540 identify_n_hubs() (in module py3plex.algorithms.statistics.basic_statistics), 453
get_supra_adjacency_matrix() (py3plex.core.multinet.multi_layer_network method), 379 ig_value() (in module py3plex.core.HINMINE.decomposition), 402
get_tensor() (py3plex.core.multinet.multi_layer_network method), 379
get_top_ranked() (py3plex.algorithms.multilayer_algorithms.multilayer_statistics.MultilayerStatistics method), 471
get_unique_entity_counts() Improvement (py3plex.algorithms.hedwig.learners.bottomup.BottomUpLearner attribute), 541
(py3plex.core.multinet.multi_layer_network method), 379 Improvement (py3plex.algorithms.hedwig.learners.learner.Learner attribute), 540
gr_value() (in module py3plex.core.HINMINE.decomposition), 402 IncompatibleNetworkError, 407
group_score() (py3plex.algorithms.hedwig.learners.learner.Learner method), 541 indices_to_bits() (py3plex.algorithms.hedwig.core.kb.ExperimentKB method), 543
method), 540 induce() (py3plex.algorithms.hedwig.learners.bottomup.BottomUpLearner method), 541
induce() (py3plex.algorithms.hedwig.learners.learner.Learner method), 540
induce() (py3plex.algorithms.hedwig.learners.optimal.OptimalLearner method), 541
induced_graph() (in module py3plex.algorithms.community_detection.community_louvain), 422
harmonic_closeness_centrality() (py3plex.algorithms.multilayer_algorithms.centrality.MultilayerCentrality method), 457 infomap_communities() (in module py3plex.algorithms.community_detection.community_wrapper), 413
heaviside() (in module py3plex.algorithms.statistics.bayesian_tests), 522 information_centrality() (py3plex.algorithms.multilayer_algorithms.centrality.MultilayerCentrality method), 458
HeterogeneousInformationNetwork (class in py3plex.core.HINMINE.dataStructures), 535 init() (py3plex.algorithms.community_detection.community_louvain.St method), 419
hex_to_RGB() (in module py3plex.visualization.colors), 503 init() (py3plex.visualization.fa2.forceatlas2.ForceAtlas2 method), 521
hierarchical() (in module py3plex.algorithms.statistics.bayesian_tests), 522 inter_layer_assortativity() (in module py3plex.algorithms.statistics.multilayer_statistics), 435
hierarchical_MC() (in module py3plex.algorithms.statistics.bayesian_tests), 523 inter_layer_coupling_strength() (in module py3plex.algorithms.statistics.multilayer_statistics), 436
hinmine_decompose() (in module py3plex.core.HINMINE.decomposition), 402 inter_layer_degree_correlation() (in module py3plex.algorithms.statistics.multilayer_statistics), 437
hinmine_get_cycles() (in module py3plex.core.HINMINE.decomposition), 402 inter_layer_dependence_entropy() (in module py3plex.algorithms.statistics.multilayer_statistics), 437
hits_centrality() (py3plex.algorithms.multilayer_algorithms.centrality.MultilayerCentrality method), 458
hub_matrix() (in module py3plex.algorithms.node_ranking), 527 interactive_diagonal_plot() (in module py3plex.visualization.multilayer), 490
hub_matrix() (in module py3plex.algorithms.node_ranking.node_ranking), 483 interactive_hairball_plot() (in module py3plex.visualization.multilayer), 491
hubs_and_authorities() (in module py3plex.algorithms.node_ranking), 528 interdependence() (in module py3plex.algorithms.statistics.multilayer_statistics), 438
hubs_and_authorities() (in module py3plex.algorithms.node_ranking.node_ranking), 483 interesting() (in module

`py3plex.algorithms.hedwig.stats.scorefunctions)`
`layer_degree centrality()`
`541` (in module `py3plex.algorithms.multilayer_algorithms centrality.Multilayer`
`interlayer_degree_correlation_matrix()` (in module `py3plex.algorithms.statistics.multilayer_statistics`), 459
`layer_density()` (in module `py3plex.algorithms.statistics.multilayer_statistics`),
`438`
`internals` (`py3plex.algorithms.community_detection.community_detection`), 440
`attribute`), 419
`layer_influence centrality()` (in module `py3plex.algorithms.statistics.multilayer_statistics`),
`InvalidEdgeError`, 407
`InvalidLayerError`, 407
`InvalidNodeError`, 408
`invariant()` (in module `py3plex.core.multinet`), 366
`invert()` (`py3plex.core.multinet.multi_layer_network`
`method`), 379
`is_discrete_target()`
`(py3plex.algorithms.hedwig.core.kb.ExperimentKB`
`method`), 543
`is_empty` (`py3plex.core.multinet.multi_layer_network`
`property`), 379
`is_implicit_root()`
`(py3plex.algorithms.hedwig.learners.bottomup.Bayesian/Py3plex`
`method`), 541
`is_implicit_root()`
`(py3plex.algorithms.hedwig.learners.learner.Learner`
`method`), 540
`is_string_like()` (in module `py3plex.core.nx_compat`), 399
K
`kaplan_meier_AUC()` (in module `py3plex.algorithms.hedwig.stats.scorefunctions`),
`541`
`katz_bonacich centrality()`
`(py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality`
`method`), 458
`katz centrality()` (in module `py3plex.algorithms.hedwig.stats.scorefunctions`),
`468`
`katz centrality()` (in module `py3plex.algorithms.multilayer_algorithms.supra_matrix_function centrality`),
`476`
L
`label` (`py3plex.dsl.NodePattern` attribute), 410, 411
`label_propagation()` (in module `py3plex.algorithms.network_classification`),
`532`
`label_propagation_normalization()` (in module `py3plex.algorithms.network_classification`),
`533`
`label_propagation_tf()` (in module `py3plex.algorithms.network_classification`),
`533`
`layer_connectivity_entropy()` (in module `py3plex.algorithms.statistics.multilayer_statistics`),
`439`
`layer_count` (`py3plex.core.multinet.multi_layer_network`
`property`), 380
`layer_degree centrality()`
`541` (in module `py3plex.algorithms.multilayer_algorithms centrality.Multilayer`
`method`), 459
`layer_density()` (in module `py3plex.algorithms.statistics.multilayer_statistics`),
`438`
`internals` (`py3plex.algorithms.community_detection.community_detection`), 440
`attribute`), 419
`layer_influence centrality()` (in module `py3plex.algorithms.statistics.multilayer_statistics`),
`441`
`layer_redundancy_coefficient()` (in module `py3plex.algorithms.statistics.multilayer_statistics`),
`441`
`layer_similarity()` (in module `py3plex.algorithms.statistics.multilayer_statistics`),
`442`
`layer_specific_random_walk()` (in module `py3plex.algorithms.general.walkers`), 480
`layers` (`py3plex.core.multinet.multi_layer_network`
`property`), 380
`Bayes/Py3plex`
`learn_embedding()` (in module `py3plex.wrappers.train_node2vec_embedding`),
`534`
`Learner` (class in `py3plex.algorithms.hedwig.learners.learner`),
`540`
`LEARNER` (`py3plex.algorithms.hedwig.core.settings.Defaults`
`attribute`), 543
`LEAVES` (`py3plex.algorithms.hedwig.core.settings.Defaults`
`attribute`), 543
`leverage()` (in module `py3plex.algorithms.hedwig.stats.scorefunctions`),
`541`
`lift()` (in module `py3plex.algorithms.hedwig.stats.scorefunctions`),
`541`
`linAttraction()` (in module `py3plex.visualization.fa2.fa2util`), 522
`linear_gradient()` (in module `py3plex.visualization.colors`), 503
`linGravity()` (in module `py3plex.visualization.fa2.fa2util`), 522
`linRepulsion()` (in module `py3plex.visualization.fa2.fa2util`), 522
`linRepulsion_region()` (in module `py3plex.visualization.fa2.fa2util`), 522
`load_binary()` (in module `py3plex.algorithms.community_detection.community_louvain`),
`422`
`load centrality()` (`py3plex.algorithms.multilayer_algorithms centrality`
`method`), 459
`load_edge_activity_file()` (in module `py3plex.core.parsers`), 389
`load_edge_activity_raw()` (in module `py3plex.core.parsers`), 389
`load_embedding()` (`py3plex.core.multinet.multi_layer_network`
`method`), 380
`load_hinmine_object()` (in module `py3plex.core.HINMINE.IO`), 402
`load_layer_name_mapping()`

(py3plex.core.multinet.multi_layer_network
 method), 380
 load_network() (py3plex.core.multinet.multi_layer_network
 method), 380
 load_network_activity() (py3plex.core.multinet.multi_layer_network
 method), 381
 load_temporal_edge_information() (in module
 py3plex.core.parsers), 390
 load_temporal_edge_information() (py3plex.core.multinet.multi_layer_network
 method), 381
 local_efficiency_centrality() (py3plex.algorithms.multilayer_algorithms.centrality.MultilayerCentrality
 method), 459
 louvain_communities() (in module
 py3plex.algorithms.community_detection.community_wrapper),
 415
 louvain_multilayer() (in module
 py3plex.algorithms.community_detection.multilayer_multilayer),
 416
 lp_aggregated_centrality() (py3plex.algorithms.multilayer_algorithms.centrality.MultilayerCentrality
 method), 460
M
 main() (in module py3plex.cli), 538
 main() (in module py3plex.wrappers.benchmark_nodes),
 508
 MatchQuery (class in py3plex.dsl), 410
 max_iter (py3plex.algorithms.multilayer_algorithms.multixrank.MultixRank
 attribute), 470
 merge_with() (py3plex.core.multinet.multi_layer_network
 method), 381
 midpoint_generator() (py3plex.core.HINMINE.dataStructures.HeterogeneousInformationNetwork
 method), 536
 MODE (py3plex.algorithms.hedwig.core.settings.Defaults
 attribute), 543
 modularity() (in module
 py3plex.algorithms.community_detection.community_wrapper),
 422
 modularity() (in module
 py3plex.algorithms.community_detection.community_wrapper),
 423
 modularity() (in module
 py3plex.algorithms.node_ranking.node_ranking),
 483
 module
 py3plex.algorithms.community_detection.community_louvain,
 419
 py3plex.algorithms.community_detection.community_measures,
 423
 py3plex.algorithms.community_detection.community_ranking,
 526
 py3plex.algorithms.community_detection.community_wrapper,
 413
 py3plex.algorithms.community_detection.multilayer_multilayer,
 424
 py3plex.algorithms.community_detection.multilayer_multilayer,
 415
 py3plex.algorithms.community_detection.NoRC,
 526
 py3plex.algorithms.general.benchmark_classification,
 482
 py3plex.algorithms.general.walkers, 478
 py3plex.algorithms.hedwig, 519
 py3plex.algorithms.hedwig.core.converters,
 542
 py3plex.algorithms.hedwig.core.example,
 540
 py3plex.algorithms.hedwig.core.kb, 542
 py3plex.algorithms.hedwig.core.predicate,
 520
 py3plex.algorithms.hedwig.core.rule,
 520
 py3plex.algorithms.hedwig.core.settings,
 543
 py3plex.algorithms.hedwig.learners.bottomup,
 540
 py3plex.algorithms.hedwig.learners.learner,
 540
 py3plex.algorithms.hedwig.learners.optimal,
 541
 py3plex.algorithms.hedwig.stats.scorefunctions,
 541
 py3plex.algorithms.hedwig.stats.validate,
 542
 py3plex.algorithms.multicentrality, 477
 py3plex.algorithms.multilayer_algorithms.centrality,
 454
 py3plex.algorithms.multilayer_algorithms.entanglemen,
 473
 py3plex.algorithms.multilayer_algorithms.multixrank,
 469
 py3plex.algorithms.multilayer_algorithms.supra_matri,
 474
 py3plex.algorithms.network_classification,
 532
 py3plex.algorithms.node_ranking, 527
 py3plex.algorithms.node_ranking.node_ranking,
 482
 py3plex.algorithms.statistics.basic_statistics,
 453
 py3plex.algorithms.statistics.bayesiantests,
 522
 py3plex.algorithms.statistics.correlation_networks,
 453
 py3plex.algorithms.statistics.enrichment,
 453
 py3plex.algorithms.statistics.multilayer_statistics,
 430
 py3plex.algorithms.statistics.topology,
 452
 py3plex.cli, 536

- py3plex.config, 402
- py3plex.core.converters, 395
- py3plex.core.HINMINE.dataStructures, 535
- py3plex.core.HINMINE.decomposition, 400
- py3plex.core.HINMINE.IO, 402
- py3plex.core.multinet, 366
- py3plex.core.nx_compat, 399
- py3plex.core.parsers, 389
- py3plex.core.random_generators, 396
- py3plex.core.supporting, 397
- py3plex.dsl, 408
- py3plex.exceptions, 407
- py3plex.io.api, 517
- py3plex.logging_config, 513
- py3plex.multinet.aggregation, 509
- py3plex.profiling, 510
- py3plex.utils, 403
- py3plex.validation, 538
- py3plex.visualization.benchmark_visualization, 506
- py3plex.visualization.bezier, 505
- py3plex.visualization.colors, 503
- py3plex.visualization.drawing_machinery, 495
- py3plex.visualization.embedding_visualization, 522
- py3plex.visualization.fa2.fa2util, 521
- py3plex.visualization.fa2.forceatlas2, 521
- py3plex.visualization.layout_algorithms, 504
- py3plex.visualization.multilayer, 487
- py3plex.visualization.sankey, 544
- py3plex.wrappers.benchmark_nodes, 506
- py3plex.wrappers.train_node2vec_embedding, 534
- monitor() (py3plex.core.multinet.multi_layer_network method), 381
- monoplex_nx_wrapper() (py3plex.core.multinet.multi_layer_network method), 381
- multi_layer_network (class in py3plex.core.multinet), 366
- multilayer_betweenness centrality() (py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality method), 460
- multilayer_betweenness_surface() (in module py3plex.algorithms.statistics.multilayer_statistics), 443
- multilayer_closeness centrality() (py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality method), 461
- multilayer_clustering_coefficient() (in module py3plex.algorithms.statistics.multilayer_statistics), 443
- multilayer_modularity() (in module py3plex.algorithms.community_detection.multilayer_modularity), 417
- multilayer_modularity() (in module py3plex.algorithms.statistics.multilayer_statistics), 444
- multilayer_motif_frequency() (in module py3plex.algorithms.statistics.multilayer_statistics), 445
- MultilayerCentrality (class in py3plex.algorithms.multilayer_algorithms centrality), 454
- multiplex_betweenness centrality() (in module py3plex.algorithms.statistics.multilayer_statistics), 445
- multiplex_closeness centrality() (in module py3plex.algorithms.statistics.multilayer_statistics), 446
- multiplex_coreness() (py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality method), 462
- multiplex_eigenvector centrality() (py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality method), 462
- multiplex_eigenvector_versatility() (py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality method), 462
- multiplex_embedding() (py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality method), 463
- multiplex_participation_coefficient() (in module py3plex.algorithms.multicentrality), 477
- multiplex_rich_club_coefficient() (in module py3plex.algorithms.statistics.multilayer_statistics), 448
- multiplexes (py3plex.algorithms.multilayer_algorithms.multirank.Multirank attribute), 469
- MultiXRrank (class in py3plex.algorithms.multilayer_algorithms.multirank), 469
- multirank_from_py3plex_networks() (in module py3plex.algorithms.multilayer_algorithms.multirank), 472
- N**
- n2v_embedding() (in module py3plex.wrappers.train_node2vec_embedding), 535
- n_examples() (py3plex.algorithms.hedwig.core.kb.ExperimentKB method), 543
- n_members() (py3plex.algorithms.hedwig.core.kb.ExperimentKB method), 543
- NEGATIONS (py3plex.algorithms.hedwig.core.settings.Defaults attribute), 543
- NetworkConstructionError, 408
- NO_CACHE (py3plex.algorithms.hedwig.core.settings.Defaults attribute), 543
- Node (class in py3plex.visualization.fa2.fa2util), 521
- node2vec (py3plex.algorithms.community_detection.community_louvain), 495

attribute), 419
 node2vec_walk() (in module *py3plex.algorithms.general.walkers*), 481
 node_activity() (in module *py3plex.algorithms.statistics.multilayer_statistics*), 448
 node_count (*py3plex.core.multinet.multi_layer_network* property), 382
 node_order (*py3plex.algorithms.multilayer_algorithms.multixrank.MultiXRank* attribute), 470
 NodePattern (class in *py3plex.dsl*), 410
 nodes (*py3plex.dsl.PathPattern* attribute), 411
 non_redundant() (*py3plex.algorithms.hedwig.learners.learner.Learner* method), 540
 NoRC_communities() (in module *py3plex.algorithms.community_detection.community_wrapper*), 413
 NoRC_communities_main() (in module *py3plex.algorithms.community_detection.NoRC*), 526
 normalize_amplify_freq() (in module *py3plex.algorithms.network_classification*), 533
 normalize_exp() (in module *py3plex.algorithms.network_classification*), 533
 normalize_initial_matrix_freq() (in module *py3plex.algorithms.network_classification*), 533
 np_calculate_importance_chi() (in module *py3plex.core.HINMINE.decomposition*), 402
 np_calculate_importance_tf() (in module *py3plex.core.HINMINE.decomposition*), 402
 number_of_communities() (in module *py3plex.algorithms.community_detection.community_wrapper*), 424
 nx_from_scipy_sparse_matrix() (in module *py3plex.core.nx_compat*), 399
 nx_info() (in module *py3plex.core.nx_compat*), 399
 nx_read_gpickle() (in module *py3plex.core.nx_compat*), 399
 nx_to_scipy_sparse_matrix() (in module *py3plex.core.nx_compat*), 400
 nx_write_gpickle() (in module *py3plex.core.nx_compat*), 400
 O
 obo2n3() (in module *py3plex.algorithms.hedwig.core.converters*), 542
 onclick() (in module *py3plex.visualization.multilayer*), 491
 OPTIMAL_SUBCLASS (*py3plex.algorithms.hedwig.core.settings.Defaults* attribute), 543
 OptimalLearner (class in *py3plex.algorithms.hedwig.learners.optimal*), 541
 OUTPUT (*py3plex.algorithms.hedwig.core.settings.Defaults* attribute), 543
 overlapping_degree centrality() (*py3plex.algorithms.multilayer_algorithms centrality.Multilayer* method), 463
 P
 page_rank_kernel() (in module *py3plex.algorithms.community_detection.community_ranking*), 526
 page_rank_kernel() (in module *py3plex.algorithms.community_detection.NoRC*), 526
 page_rank_kernel() (in module *py3plex.algorithms.node_ranking*), 528
 page_rank_kernel() (in module *py3plex.algorithms.node_ranking.node_ranking*), 483
 pagerank centrality() (*py3plex.algorithms.multilayer_algorithms centrality.Multilayer* method), 463
 parse_detangler_json() (in module *py3plex.core.parsers*), 390
 parse_edgelist_multi_types() (in module *py3plex.core.parsers*), 390
 parse_embedding() (in module *py3plex.core.parsers*), 390
 parse_gaf_to_uniprot_G0() (in module *py3plex.core.supporting*), 398
 parse_gml() (in module *py3plex.core.parsers*), 390
 parse_gpickle() (in module *py3plex.core.parsers*), 391
 parse_gpickle_biomine() (in module *py3plex.core.parsers*), 391
 parse_inform() (in module *py3plex.algorithms.community_detection.community_wrapper*), 415
 parse_matrix() (in module *py3plex.core.parsers*), 391
 parse_matrix_to_nx() (in module *py3plex.core.parsers*), 391
 parse_multi_edgelist() (in module *py3plex.core.parsers*), 392
 parse_multiedge_tuple_list() (in module *py3plex.core.parsers*), 392
 parse_multiplex_edges() (in module *py3plex.core.parsers*), 392
 parse_multiplex_folder() (in module *py3plex.core.parsers*), 393
 parse_network() (in module *py3plex.core.parsers*), 393
 parse_nx() (in module *py3plex.core.parsers*), 394
 parse_simple_edgelist() (in module *py3plex.core.parsers*), 394
 parse_spin_edgelist() (in module *py3plex.core.parsers*), 394
 ParsingError, 408
 participation_coefficient()

(`py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality`),
`method`), 463 452

`partition_at_level()` (in module `plot_radial_layers()` (in module
`py3plex.algorithms.community_detection.community_louvain`), 493
423 `plot_robustness()` (in module
`py3plex.visualization.benchmark_visualizations`),
`PathPattern` (class in `py3plex.dsl`), 411 506
`pattern` (`py3plex.dsl.MatchQuery` attribute), 410
`percolation centrality()` `plot_simplex()` (in module
(`py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality`),
`method`), 464 525

`percolation_threshold()` (in module `plot_small_multiples()` (in module
`py3plex.algorithms.statistics.multilayer_statistics`), `py3plex.visualization.multilayer`), 493
449 `plot_supra_adjacency_heatmap()` (in module
`py3plex.visualization.multilayer`), 494

`PerformanceMonitor` (class in `py3plex.profiling`), 510
`pick_threshold()` (in module `positives` (`py3plex.algorithms.hedwig.core.rule.Rule`
`py3plex.algorithms.statistics.correlation_networks`), `property`), 520
453 `precision()` (in module
`py3plex.algorithms.hedwig.stats.scorefunctions`),
506 `precision()` (`py3plex.algorithms.hedwig.core.rule.Rule`
`method`), 520

`plot_core_macro()` (in module `py3plex.algorithms.hedwig.stats.scorefunctions`),
`py3plex.visualization.benchmark_visualizations`), 541
506 `precision()` (`py3plex.algorithms.hedwig.core.rule.Rule`
`method`), 520

`plot_core_macro_box()` (in module `py3plex.visualization.benchmark_visualizations`),
506 `predicate` (class in
`py3plex.algorithms.hedwig.core.predicate`),
520

`plot_core_macro_gg()` (in module `py3plex.visualization.benchmark_visualizations`),
506 `predict()` (`py3plex.wrappers.benchmark_nodes.TopKRanker`
`method`), 506

`plot_core_micro()` (in module `prepare_for_parsing()` (in module
`py3plex.visualization.benchmark_visualizations`), `py3plex.core.converters`), 395
506 `prepare_for_visualization()` (in module
`py3plex.core.converters`), 395

`plot_core_micro_gg()` (in module `py3plex.visualization.benchmark_visualizations`),
506 `prepare_for_visualization_hairball()` (in
module `py3plex.core.converters`), 396

`plot_core_micro_grid()` (in module `process_network()` (`py3plex.core.HINMINE.dataStructures.Heterogen`
`py3plex.visualization.benchmark_visualizations`), `method`), 536
506 `profile_performance()` (in module
`py3plex.profiling`), 512

`plot_core_time()` (in module `py3plex.visualization.benchmark_visualizations`),
506 `py3plex.algorithms.community_detection.community_louvain`
module, 419

`plot_core_time_gg()` (in module `py3plex.algorithms.community_detection.community_measure`
`py3plex.visualization.benchmark_visualizations`), module, 423
506 `py3plex.algorithms.community_detection.community_ranking`
module, 526

`plot_core_variability()` (in module `py3plex.algorithms.community_detection.community_wrapper`
`py3plex.visualization.benchmark_visualizations`), module, 413
506

`plot_critical_distance()` (in module `py3plex.algorithms.community_detection.multilayer_benchmark`
`py3plex.visualization.benchmark_visualizations`), module, 424
506 `py3plex.algorithms.community_detection.multilayer_modula`
module, 415

`plot_edge_colored_projection()` (in module `py3plex.algorithms.community_detection.NoRC`
`py3plex.visualization.multilayer`), 491 module, 526

`plot_ego_multilayer()` (in module `py3plex.algorithms.general.benchmark_classification`
`py3plex.visualization.multilayer`), 492 module, 482

`plot_mean_times()` (in module `py3plex.algorithms.general.walkers`
`py3plex.visualization.benchmark_visualizations`), module, 478
506

`plot_posterior()` (in module `py3plex.algorithms.hedwig`
`py3plex.algorithms.statistics.bayesian_tests`), module, 519
524 `py3plex.algorithms.hedwig.core.converters`
module, 542

`plot_power_law()` (in module `py3plex.algorithms.hedwig.core.converters`
module, 542

| | |
|--|--|
| py3plex.algorithms.hedwig.core.example
module, 519 | py3plex.core.HINMINE.IO
module, 402 |
| py3plex.algorithms.hedwig.core.kb
module, 542 | py3plex.core.multinet
module, 366 |
| py3plex.algorithms.hedwig.core.predicate
module, 520 | py3plex.core.nx_compat
module, 399 |
| py3plex.algorithms.hedwig.core.rule
module, 520 | py3plex.core.parsers
module, 389 |
| py3plex.algorithms.hedwig.core.settings
module, 543 | py3plex.core.random_generators
module, 396 |
| py3plex.algorithms.hedwig.learners.bottomup
module, 540 | py3plex.core.supporting
module, 397 |
| py3plex.algorithms.hedwig.learners.learner
module, 540 | py3plex.dsl
module, 408 |
| py3plex.algorithms.hedwig.learners.optimal
module, 541 | py3plex.exceptions
module, 407 |
| py3plex.algorithms.hedwig.stats.scorefunction
module, 541 | py3plex.io.api
module, 517 |
| py3plex.algorithms.hedwig.stats.validate
module, 542 | py3plex.logging_config
module, 513 |
| py3plex.algorithms.multicentrality
module, 477 | py3plex.multinet.aggregation
module, 509 |
| py3plex.algorithms.multilayer_algorithms.centralities
module, 454 | py3plex.profilng
module, 510 |
| py3plex.algorithms.multilayer_algorithms.entropy
module, 473 | py3plex.utils
module, 403 |
| py3plex.algorithms.multilayer_algorithms.multiplex
module, 469 | py3plex.validation
module, 538 |
| py3plex.algorithms.multilayer_algorithms.support
module, 474 | py3plex.visualization.benchmark_visualizations
module, 506 |
| py3plex.algorithms.network_classification
module, 532 | py3plex.visualization.bezier
module, 505 |
| py3plex.algorithms.node_ranking
module, 527 | py3plex.visualization.colors
module, 503 |
| py3plex.algorithms.node_ranking.node_ranking
module, 482 | py3plex.visualization.drawing_machinery
module, 495 |
| py3plex.algorithms.statistics.basic_statistics
module, 453 | py3plex.visualization.embedding_visualization.embedding
module, 522 |
| py3plex.algorithms.statistics.bayesian_tests
module, 522 | py3plex.visualization.fa2.fa2util
module, 521 |
| py3plex.algorithms.statistics.correlation_networks
module, 453 | py3plex.visualization.fa2.forceatlas2
module, 521 |
| py3plex.algorithms.statistics.enrichment
module, 453 | py3plex.visualization.layout_algorithms
module, 504 |
| py3plex.algorithms.statistics.multilayer_statistics
module, 430 | py3plex.visualization.multilayer
module, 487 |
| py3plex.algorithms.statistics.topology
module, 452 | py3plex.visualization.sankey
module, 544 |
| py3plex.cli
module, 536 | py3plex.wrappers.benchmark_nodes
module, 506 |
| py3plex.config
module, 402 | py3plex.wrappers.train_node2vec_embedding
module, 534 |
| py3plex.core.converters
module, 395 | Py3plexException, 408 |
| py3plex.core.HINMINE.dataStructures
module, 535 | Py3plexFormatError, 408 |
| py3plex.core.HINMINE.decomposition
module, 400 | Py3plexIOError, 408 |
| | Py3plexLayoutError, 408 |
| | Py3plexMatrixError, 408 |

R

- `random_multilayer_ER()` (in module `py3plex.core.random_generators`), 396
- `random_multiplex_ER()` (in module `py3plex.core.random_generators`), 397
- `random_multiplex_generator()` (in module `py3plex.core.random_generators`), 397
- `random_walk_with_restart()` (`py3plex.algorithms.multilayer_algorithms.multixrank.MultixRank` method), 471
- `Ranked` (`py3plex.algorithms.hedwig.core.example.Example` attribute), 520
- `read()` (in module `py3plex.io.api`), 517
- `read_ground_truth_communities()` (`py3plex.core.multinet.multi_layer_network` method), 382
- `record()` (`py3plex.profiles.PerformanceMonitor` method), 511
- `Region` (class in `py3plex.visualization.fa2.fa2util`), 521
- `register_reader()` (in module `py3plex.io.api`), 518
- `register_writer()` (in module `py3plex.io.api`), 518
- `remove_edges()` (`py3plex.core.multinet.multi_layer_network` method), 382
- `remove_layer_edges()` (`py3plex.core.multinet.multi_layer_network` method), 382
- `remove_nodes()` (`py3plex.core.multinet.multi_layer_network` method), 383
- `require()` (in module `py3plex.algorithms.multicentrality`), 478
- `require()` (in module `py3plex.algorithms.statistics.basic_statistics`), 454
- `require()` (in module `py3plex.core.converters`), 396
- `require()` (in module `py3plex.core.multinet`), 389
- `require()` (in module `py3plex.core.nx_compat`), 400
- `require()` (in module `py3plex.core.parsers`), 394
- `require()` (in module `py3plex.core.random_generators`), 397
- `require()` (in module `py3plex.core.supporting`), 398
- `require()` (in module `py3plex.io.api`), 518
- `require()` (in module `py3plex.multinet.aggregation`), 510
- `require()` (in module `py3plex.utils`), 406
- `require()` (in module `py3plex.visualization.layout_algorithms`), 505
- `reset_to_defaults()` (in module `py3plex.config`), 402
- `resilience()` (in module `py3plex.algorithms.statistics.multilayer_statistics`), 449
- `restart_prob` (`py3plex.algorithms.multilayer_algorithms.multixrank.MultixRank` attribute), 470
- `return_aliases` (`py3plex.dsl.MatchQuery` attribute), 410
- `return_infomap_communities()` (in module `py3plex.algorithms.community_detection.community_ranking`), 525
- `rf_value()` (in module `py3plex.core.HINMINE.decomposition`), 402
- `RGB_to_hex()` (in module `py3plex.visualization.colors`), 503
- `Rule` (class in `py3plex.algorithms.hedwig.core.rule`), 520
- `rule_kernel()` (in module `py3plex.algorithms.hedwig`), 519
- `rule_report()` (`py3plex.algorithms.hedwig.core.rule.Rule` method), 520
- `ruleset_examples_json()` (`py3plex.algorithms.hedwig.core.rule.Rule` static method), 520
- `ruleset_report()` (`py3plex.algorithms.hedwig.core.rule.Rule` static method), 521
- `run()` (in module `py3plex.algorithms.hedwig`), 519
- `run_infomap()` (in module `py3plex.algorithms.community_detection.community_wrapper`), 415
- `run_learner()` (in module `py3plex.algorithms.hedwig`), 519
- `run_PPR()` (in module `py3plex.algorithms.node_ranking`), 529

S

- `save_edgelist()` (in module `py3plex.core.parsers`), 395
- `save_gpickle()` (in module `py3plex.core.parsers`), 395
- `save_multiedgelist()` (in module `py3plex.core.parsers`), 395
- `save_network()` (`py3plex.core.multinet.multi_layer_network` method), 383
- `SCORE` (`py3plex.algorithms.hedwig.core.settings.Defaults` attribute), 543
- `select_high_degree_nodes()` (in module `py3plex.dsl`), 412
- `select_nodes_by_layer()` (in module `py3plex.dsl`), 413
- `serialize_to_edgelist()` (`py3plex.core.multinet.multi_layer_network` method), 383
- `set_partial_fit_request()` (`py3plex.wrappers.benchmark_nodes.TopKRanker` method), 506
- `set_predict_request()` (`py3plex.wrappers.benchmark_nodes.TopKRanker` method), 507
- `set_score_request()` (`py3plex.wrappers.benchmark_nodes.TopKRanker` method), 507
- `setup_logging()` (in module `py3plex.logging_config`), 513
- `signrank()` (in module `py3plex.algorithms.statistics.bayesian_tests`), 525

signrank_MC() (in module `py3plex.algorithms.statistics.bayesiantests`), 525
signtest() (in module `py3plex.algorithms.statistics.bayesiantests`), 525
signtest_MC() (in module `py3plex.algorithms.statistics.bayesiantests`), 526
Similarity (`py3plex.algorithms.hedwig.learners.bottomup.BottomupLearner` attribute), 541
Similarity (`py3plex.algorithms.hedwig.learners.learner.Learner` attribute), 540
similarity() (`py3plex.algorithms.hedwig.core.rule.Rule` method), 521
size() (`py3plex.algorithms.hedwig.core.rule.Rule` method), 521
size_distribution() (in module `py3plex.algorithms.community_detection.community_measures`), 424
sparse2graph() (in module `py3plex.wrappers.benchmark_nodes`), 508
sparse_page_rank() (in module `py3plex.algorithms.node_ranking`), 530
sparse_page_rank() (in module `py3plex.algorithms.node_ranking.node_rankings`), 483
sparse_to_px() (`py3plex.core.multinet.multi_layer_network` method), 384
specialize() (`py3plex.algorithms.hedwig.learners.learner.Learner` method), 540
specialize_add_relation() (`py3plex.algorithms.hedwig.learners.learner.Learner` method), 540
split_to_indices() (`py3plex.core.HINMINE.dataStructures.HeterogeneousInformationNetwork` method), 536
split_to_layers() (in module `py3plex.core.supporting`), 398
split_to_layers() (`py3plex.core.multinet.multi_layer_network` method), 384
split_to_parts() (`py3plex.core.HINMINE.dataStructures.HeterogeneousInformationNetwork` method), 536
spread_percent_hyper (in module `py3plex.algorithms.node_ranking`), 531
spread_step_hyper (in module `py3plex.algorithms.node_ranking`), 531
spreading_centrality() (`py3plex.algorithms.multilayer_algorithms centrality` method), 464
start() (`py3plex.visualization.fa2.forceatlas2.Timer` method), 521
stats (`py3plex.profiling.PerformanceMonitor` attribute), 510
Status (class in `py3plex.algorithms.community_detection.community_measures`), 419
stochastic_normalization() (in module `py3plex.algorithms.node_ranking`), 531
stochastic_normalization() (in module `py3plex.algorithms.node_ranking.node_ranking`), 484
stochastic_normalization_hin() (in module `py3plex.algorithms.node_ranking.node_ranking`), 484
stop() (`py3plex.visualization.fa2.forceatlas2.Timer` method), 521
strongGravity() (in module `py3plex.visualization.fa2.fa2util`), 522
subgraph_centrality() (`py3plex.algorithms.multilayer_algorithms centrality` method), 465
subnetwork() (`py3plex.core.multinet.multi_layer_network` method), 385
summary() (`py3plex.core.multinet.multi_layer_network` method), 385
super_classes() (`py3plex.algorithms.hedwig.core.kb.ExperimentKB` method), 543
SUPPORT (`py3plex.algorithms.hedwig.core.settings.Defaults` attribute), 543
supported_formats() (in module `py3plex.io.api`), 518
supra_adjacency_matrix_plot() (in module `py3plex.visualization.multilayer`), 494
supra_degree_centrality() (`py3plex.algorithms.multilayer_algorithms centrality` method), 465
supra_laplacian_spectrum() (in module `py3plex.algorithms.statistics.multilayer_statistics`), 450
supra_matrix (`py3plex.algorithms.multilayer_algorithms.multirank.M` attribute), 470
T
targeted_layer_removal() (in module `py3plex.algorithms.hedwig.stats.scorefunctions`), 541
table_to_latex() (in module `py3plex.visualization.benchmark_visualizations`), 506
targeted_layer_removal() (in module `py3plex.algorithms.hedwig.core.settings.Defaults` attribute), 543
targeted_layer_removal() (in module `py3plex.algorithms.statistics.multilayer_statistics`), 451
test() (`py3plex.algorithms.hedwig.stats.validate.Validate` method), 542
test_scale_free() (`py3plex.core.multinet.multi_layer_network` method), 385
timed_section() (in module `py3plex.profiling`), 512
Timer (class in `py3plex.visualization.fa2.forceatlas2`), 521
to_homogeneous_hypergraph() (`py3plex.core.multinet.multi_layer_network` method), 385
to_json() (`py3plex.algorithms.hedwig.core.rule.Rule` static method), 521

to_json() (*py3plex.core.multinet.multi_layer_network* module), 386
 to_networkx() (*py3plex.core.multinet.multi_layer_network* module), 386
 to_sparse_matrix() (*py3plex.core.multinet.multi_layer_network* module), 386
 TopKRanker (class in *py3plex.wrappers.benchmark_nodes*), 506
 total_communicability() (*py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality* module), 465
 total_weight (*py3plex.algorithms.community_detection.community_detection.MultilayerStatistics* attribute), 419
 transition_matrix (*py3plex.algorithms.multilayer_algorithms centrality.MultilayerCentrality* attribute), 470
 type (*py3plex.dsl.EdgePattern* attribute), 409, 410
U
 UnaryPredicate (class in *py3plex.algorithms.hedwig.core.predicate*), 520
 unique_redundant_edges() (in module *py3plex.algorithms.statistics.multilayer_statistics*), 451
 updateMassAndGeometry() (*py3plex.visualization.fa2.fa2util.Region* module), 521
 URIS (*py3plex.algorithms.hedwig.core.settings.Defaults* attribute), 543
V
 Validate (class in *py3plex.algorithms.hedwig.stats.validate*), 542
 validate_csv_columns() (in module *py3plex.validation*), 538
 validate_edgelist_format() (in module *py3plex.validation*), 538
 validate_file_exists() (in module *py3plex.validation*), 538
 validate_input_type() (in module *py3plex.validation*), 539
 validate_label_propagation() (in module *py3plex.algorithms.network_classification*), 533
 validate_multiedgelist_format() (in module *py3plex.validation*), 539
 validate_multilayer_input() (in module *py3plex.utils*), 406
 validate_network_data() (in module *py3plex.validation*), 539
 VERBOSE (*py3plex.algorithms.hedwig.core.settings.Defaults* attribute), 543
 versatility centrality() (in module *py3plex.algorithms.statistics.multilayer_statistics*), 452
 VisualizationError, 408
 visualize_embedding() (in module *py3plex.visualization.embedding_visualization.embedding_visualization*), 522
 visualize_matrix() (*py3plex.core.multinet.multi_layer_network* module), 386
 visualize_multilayer_network() (in module *py3plex.visualization.multilayer*), 494
 visualize_network() (*py3plex.core.multinet.multi_layer_network* module), 386
 visualize_ricci_core() (*py3plex.algorithms.statistics.MultilayerStatistics* module), 387
 visualize_ricci_core_k() (*py3plex.core.multinet.multi_layer_network* module), 388
 visualize_ricci_supra() (*py3plex.core.multinet.multi_layer_network* module), 389
W
 warn_if_deprecated() (in module *py3plex.utils*), 406
 wracc() (in module *py3plex.algorithms.hedwig.stats.scorefunctions*), 541
 write() (in module *py3plex.io.api*), 519
Z
 z_score() (in module *py3plex.algorithms.hedwig.stats.scorefunctions*), 541